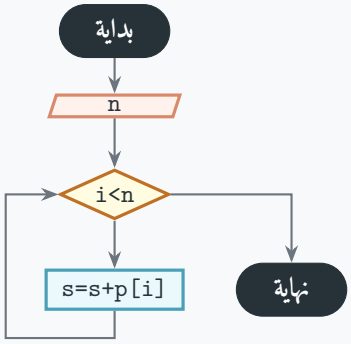
الخوارزميات ولغة C

كتاب التمارين المحلولة

650 تمرينًا من التجارة الإلكترونية وعلم البيانات

- **13 فصلًا**: من المتغيرات إلى الأشجار الثنائية
- لكل فصل **50 تمرينًا**: 10 سهلة، 30 متوسطة، 10 صعبة
- لكل تمرين: نص، خطوات الحل، برنامج C مشروح، نتيجة التنفيذ، ومخطط انسيابي
- دون أي دالة جاهزة: كل خوارزمية مكتوبة يدويًا



الأستاذ: د. بن عبد الرزاق يوسف
الفئة: طلبة السنة الأولى

المحتويات

مقدمة viii

1 مدخل إلى الخوارزميات: المتغيرات والعمليات 1
1.1 السعر مع الرسم على القيمة المضافة . . . 2
1.2 مبلغ سطر في الفاتورة . . . 3
1.3 متوسط تقييمات منتج . . . 4
1.4 تحويل العملة . . . 5
1.5 الوزن الإجمالي للشحنة . . . 6
1.6 التخفيض الموسمي . . . 7
1.7 الربح وهامش الربح . . . 8
1.8 تبديل سعرين . . . 9
1.9 مدة جلسة التصفح . . . 10
1.10 معدل التحويل . . . 11
1.11 عدد علب التغليف . . . 13
1.12 المسافة بين زبونين . . . 14
1.13 المتوسط والتباين . . . 15
1.14 التطبيع بين 0 و1 . . . 16
1.15 الدرجة المعيارية . . . 17
1.16 متوسط النجوم الموزون . . . 18
1.17 نمو رقم الأعمال . . . 19
1.18 تفكيك مبلغ إلى أوراق نقدية . . . 20
1.19 ساعة وصول الطلبية . . . 22
1.20 مدة الاشتراك بالأسابيع . . . 23
1.21 عكس رمز المنتج . . . 24
1.22 مجموع التحقق لرقم الطلبية . . . 25
1.23 تقريب السعر إلى منزلتين . . . 26
1.24 مؤشرات الحملة الإعلانية . . . 28
1.25 القيمة الدائمة للزبون . . . 29
1.26 متوسط قيمة الطلبية . . . 30
1.27 أيام تغطية المخزون . . . 31
1.28 التنبؤ بنموذج خطي . . . 32
1.29 مستقيم يمر بنقطتين . . . 33
1.30 حصص الأصناف من رقم الأعمال . . . 34
1.31 الطابع الزمني إلى ساعة . . . 35

1.32 فاتورة كاملة 36
1.33 مقارنة سعر الكيلوغرام 38
1.34 نقاط الوفاء 39
1.35 متوسط مربع الخطأ 40
1.36 السعر بعد التضخم 41
1.37 تبديل دون متغير مساعد 42
1.38 المبلغ بالسنتيم 43
1.39 رموز الأصناف والحروف 45
1.40 أسبوع ويوم التسليم 46
1.41 الأكبر بين سعرين دون شرط 48
1.42 الانحراف المعياري بطريقة نيوتن 49
1.43 التغاير بين السعر والمبيعات 50
1.44 انحدار خطي بالمربعات الصغرى 52
1.45 مدة المشاهدة الكلية 53
1.46 خصائص المنتج بالبتات 54
1.47 المتوسط المتحرك الأسي 55
1.48 رقم التحقق لرمز EAN-8 57
1.49 مرونة الطلب السعرية 58
1.50 تعبئة حاوية الشحن 59

2 البنيات الشرطية 62
2.1 الشحن المجاني 63
2.2 توفر المنتج في المخزون 64
2.3 رقم طلبية زوجي أم فردي 65
2.4 الأرخص بين عرضين 66
2.5 التحقق من التقييم 67
2.6 ربح أم خسارة 68
2.7 رمز القسيمة 69
2.8 بيع منتج محدد بالسن 70
2.9 طريقة الدفع 71
2.10 نجاح تجربة A/B 73
2.11 تخفيض حسب الشرائح 75
2.12 تكلفة الشحن حسب الوزن 76
2.13 أفضل تقييم بين ثلاثة منتجات 78
2.14 ترتيب ثلاثة أسعار 80
2.15 السنة الكبيسة 82
2.16 عدد أيام الشهر 84
2.17 تصنيف الزبائن حسب الإنفاق 85

2.18 الرسم حسب صنف المنتج 86
2.19 كشف القيم الشاذة 88
2.20 هيمنة منتج على آخر 90
2.21 التقييم إلى نجوم 91
2.22 نقطة التعادل 92
2.23 صحة تاريخ الطلبية 94
2.24 اليوم الموالي 96
2.25 خلية مصفوفة الالتباس 98
2.26 قاعدة توصية 99
2.27 منطقة التوصيل 101
2.28 عرض 3 بثمن 2 102
2.29 تداخل فترتي ترويج 103
2.30 آلة حاسبة للأسعار 105
2.31 تأجيل التوصيل في عطلة نهاية الأسبوع 106
2.32 صلاحية البطاقة البنكية 108
2.33 هدية حسب حجم السلة 109
2.34 تطور مؤشر الأداء 111
2.35 السعر الأمثل 113
2.36 الوسيط بين ثلاث قيم 114
2.37 تسعير حسب ساعة الطلب 115
2.38 تصنيف المشاعر 117
2.39 كشف الاحتيال بالقواعد 118
2.40 الفائز في تجربة A/B 120
2.41 يوم الأسبوع لتاريخ ما 123
2.42 شجرة قرار للتنبؤ بمغادرة الزبون 124
2.43 ترتيب أربعة أسعار بشبكة مقارنات 125
2.44 الضريبة التصاعدية على دخل البائع 127
2.45 عدد الأيام بين تاريخين 129
2.46 اختيار شركة الشحن الأرخص 132
2.47 هل العنوان داخل منطقة التوصيل؟ 133
2.48 أقرب جار لتصنيف زبون جديد 135
2.49 محرك التسعير الكامل 137
2.50 توازن العرض والطلب 139

3 الحلقات التكرارية 142

3.1 مجموع أسعار السلة 144
3.2 عدد الطلبيات الكبيرة 145
3.3 متوسط التقييمات 147

3.4 جدول الأسعار حسب الكمية 149
3.5 العد التنازلي للتخفيضات 150
3.6 الإدخال حتى الصفر 152
3.7 أغلى منتج 153
3.8 أرقام الطلبيات الزوجية 155
3.9 الادخار لشراء حاسوب 157
3.10 إعادة طلب تقييم صالح 159
3.11 الأصغر والأكبر والمدى 161
3.12 المتوسط والتباين في مرور واحد 162
3.13 الجذر التربيعي بطريقة نيوتن 164
3.14 ترتيبات عرض المنتجات 166
3.15 عدد الحزم الممكنة 168
3.16 أكبر قاسم مشترك لأحجام العبوات 170
3.17 تزامن دورات التموين 171
3.18 هل رقم المنتج أولي؟ 173
3.19 أرقام رقم الطلبية 175
3.20 رمز تخفيض متناظر 177
3.21 نمو المستخدمين بمتتالية فيبوناتشي 179
3.22 الفائدة المركبة على عدة سنوات 181
3.23 جدول تسديد الشراء بالتقسيط 183
3.24 أطول فترة نمو متواصل 184
3.25 المتوسط المتحرك لثلاثة أيام 186
3.26 مدرج تكراري للتقييمات 188
3.27 رقم المنتج بالنظام الثنائي 190
3.28 شبكة التخفيضات 192
3.29 عد الزبائن في كل شريحة 194
3.30 محاكاة نفاد المخزون 196
3.31 التنعيم الأسي لسلسلة زمنية 198
3.32 الانحدار الخطي على n نقطة 200
3.33 معامل الارتباط 202
3.34 ثاني أغلى منتج 204
3.35 هرم مستويات الوفاء 206
3.36 سعر التوازن بالتنصيف 208
3.37 الفائدة المستمرة بسلسلة تايلور 210
3.38 القفزات المفاجئة في الزيارات 212
3.39 تخمين السعر بالبحث الثنائي 214
3.40 يوم بلوغ هدف رقم الأعمال 216

3.41 الأرقام الأولية حتى N 219
3.42 التجميع بـ K-Means في بعد واحد 220
3.43 محاكاة مونت كارلو بمولد عشوائي يدوي 222
3.44 الانحدار بالنزول التدريجي 224
3.45 التحقق من رقم البطاقة بخوارزمية Luhn 226
3.46 مثلث باسكال لعدد الحزم 228
3.47 التحويل إلى النظام الست عشري 230
3.48 الجذر الصحيح بالبحث الثنائي 232
3.49 محاكاة سياسة التموين (s, S) 234
3.50 أفضل يوم للشراء والبيع 236

4 الجداول أحادية البعد 239
4.1 عرض كتالوج الأسعار 240
4.2 مبيعات الأسبوع 241
4.3 أكثر منتج مبيعًا 243
4.4 المنتجات النافدة 245
4.5 رفع كل الأسعار 247
4.6 آخر الطلبيات أولًا 249
4.7 البحث عن منتج 251
4.8 نسخ المنتجات الفاخرة 253
4.9 نسبة التقييمات الإيجابية 255
4.10 الفاتورة من جدولين 257
4.11 عكس الجدول في مكانه 260
4.12 تدوير جدول المناوبات 261
4.13 إدراج منتج جديد في موضع 263
4.14 حذف منتج متوقف 265
4.15 الترتيب بالفقاعات 267
4.16 الترتيب بالانتقاء 269
4.17 الترتيب بالإدراج 271
4.18 البحث الثنائي في كتالوج مرتب 273
4.19 وسيط أسعار السوق 275
4.20 المنوال بجدول عدادات 277
4.21 توزيع الطلبيات حسب الساعة 279
4.22 حذف المكرر من جدول مرتب 281
4.23 دمج قائمتي أسعار مرتبتين 283
4.24 المجاميع التراكمية واستعلام الفترات 285
4.25 المتوسط المتحرك بنافذة k 287
4.26 تطبيع جدول الأسعار 289

4.27 التوحيد المعياري للبيانات 291
4.28 المئين التسعون لزمن التوصيل 293
4.29 منتجان بقيمة بطاقة الهدية 294
4.30 التشابه بين زبونين 296
4.31 أكبر هبوط يومي 298
4.32 القيم الشاذة بطريقة المدى الربيعي 300
4.33 نقل المنتجات النافدة إلى الآخر 302
4.34 غربال إراتوستينس 304
4.35 عدد المنتجات المختلفة في سلة 306
4.36 مطابقة جردين 308
4.37 البتات في جدول 310
4.38 جمع أعداد ضخمة 312
4.39 أطول فترة صعود في جدول 314
4.40 الدرجة الموزونة للمنتج 316
4.41 أفضل فترة ربح متتالية (خوارزمية كادان) 319
4.42 الترتيب السريع دون عودية 320
4.43 الترتيب بالدمج من الأسفل إلى الأعلى 322
4.44 الترتيب بالعد لمليون تقييم 324
4.45 تصنيف زبون بـ k أقرب جيران 326
4.46 تجميع الزبائن إلى ثلاث فئات 328
4.47 اختيار الحملات الإعلانية بميزانية محدودة 330
4.48 المنتجات التي تُشترى معًا 332
4.49 معامل جيني لتركز الإنفاق 334
4.50 أطول فترة ضمن الميزانية 336

5 الجداول ثنائية البعد 339

5.1 جدول مبيعات المتاجر 340
5.2 المجموع الكلي للمبيعات 341
5.3 مجموع كل متجر 343
5.4 مجموع كل يوم 345
5.5 أفضل خانة في المصفوفة 347
5.6 الرفوف الفارغة في المستودع 349
5.7 تعديل كل الأسعار في المصفوفة 351
5.8 قراءة مصفوفة تقييمات 353
5.9 القطر الرئيسي لمصفوفة التحويلات 355
5.10 البحث عن منتج في الرفوف 357
5.11 منقول المصفوفة 360
5.12 جمع مبيعات شهرين 361

5.13 جداء المصفوفات: كلفة الطلبيات عند الموردين 363
5.14 فاتورة كل زبون بجداء مصفوفة في شعاع 365
5.15 أفضل متجر في كل يوم 367
5.16 متوسط تقييم كل منتج دون الخانات الفارغة 369
5.17 تطبيع أعمدة جدول الخصائص 371
5.18 مصفوفة الارتباط 373
5.19 مصفوفة المسافات بين الزبائن 375
5.20 مصفوفة الالتباس والدقة 377

6 السلاسل النصية 380

7 الدوال 381

8 العودية 382

9 الهياكل 383

10 المؤشرات والحجز الديناميكي 384

11 الملفات 385

12 القوائم المتصلة 386

13 الأشجار الثنائية 387

مقدمة

إعداد: د. بن عبد الرزاق يوسف — الفئة: طلبة السنة الأولى

يجمع هذا الكتاب **650 تمرينًا محلولًا** في أساسيات الخوارزميات ولغة C، موزعة على الفصول الثلاثة عشر للمقياس. كل التمارين مستوحاة من مجالين قريبين من الواقع المهني: **التجارة الإلكترونية** (سلات، طلبيات، مخزون، فواتير، توصيل) و**علم البيانات** (متوسطات، تباين، تطبيع، انحدار، تصنيف، تجميع).

بنية كل تمرين



- **نص التمرين**: المسألة كما يطرحها متجر أو محلل بيانات.
- **خطوات الحل**: الفكرة الخوارزمية مرتبة خطوة بخطوة قبل أي كود.
- **البرنامج**: برنامج C كامل، تعليقاته **باللون الأحمر** تشرح كل سطر مهم.
- **نتيجة التنفيذ**: المخرجات **الحقيقية** للبرنامج (كل البرامج تُرجمت ونُفّذت).
- **المخطط الانسيابي** (Organigramme): التمثيل البياني للخوارزمية.

رموز المخطط الإنسيابي

بداية / نهاية	بداية الخوارزمية أو نهايتها.
اقرأ x	عملية إدخال (قراءة) أو إخراج (كتابة).
s = s + x	معالجة: إسناد أو حساب.
f(x)	استدعاء دالة (إجراء معرَّف في مكان آخر).
x > 0	اختبار شرط: مخرجان نعم ولا.
←	اتجاه سير التنفيذ.

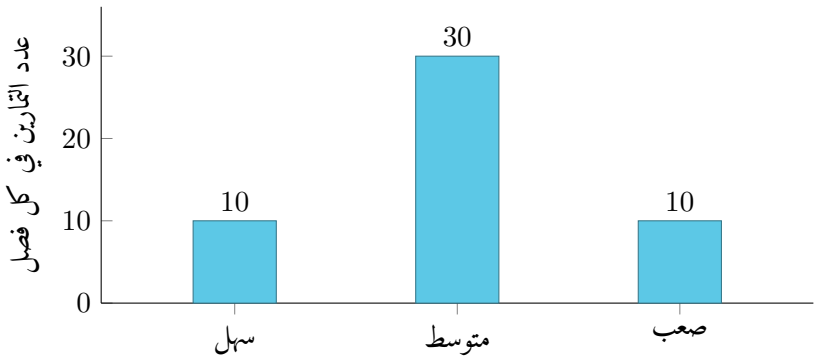
قاعدة الكتاب: لا دوال جاهزة

ممنوع استعمال الدوال المعرفة مسبقًا

الهدف هو تعلم الخوارزميات، لذلك **نكتب بأنفسنا** كل ما توفره المكتبات عادة: طول سلسلة، مقارنة سلسلتين، الجذر التربيعي، القيمة المطلقة، الترتيب، البحث، التحويل بين الأعداد والنصوص...

الصنف	الدوال
✔ مسموح	الإدخال والإخراج: `printf`، `scanf`، `getchar`، `putchar`
✔ مسموح (الفصول 10–13)	الذاكرة والملفات: `malloc`، `free`، `fopen`، `fclose`، `fprintf`، `fscanf`، `fgetc`، `fputc`
✘ ممنوع	`strlen`، `strcpy`، `strcmp`، `sqrt`، `pow`، `abs`، `qsort`، `atoi`، `toupper`، `rand`، ...

مستويات الصعوبة



- سهل ★ — الفصل. من واحد لمفهوم مباشر تطبيق
- متوسط ★★ — خوارزميًا. تفكيرًا وتتطلب مفاهيم عدة تجمع حقيقية مسألة
- صعب ★★★ — يدويًا. بالكامل تُبنى تحسين...) تجميع، بحث، (ترتيب، متقدمة خوارزمية

ترجمة البرامج وتنفيذها

كل برنامج موجود في المجلد `code/chNN/exMM.c` مع نتيجة تنفيذه `exMM.out`:

```
gcc -std=c99 -Wall -Wextra code/ch04/ex12.c -o ex12
./ex12 < code/ch04/ex12.in
```

الفصل 1

مدخل إلى الخوارزميات: المتغيرات والعمليات

تعريفات

- **الخوارزمية:** سلسلة منتهية ومرتبة من التعليمات تحل مسألة: مدخلات ← معالجة ← مخرجات.
- **المتغير:** خانة في الذاكرة لها اسم ونوع وقيمة يمكن أن تتغير.
- **الثابت:** قيمة لا تتغير أثناء التنفيذ (`#define TVA 0.19` أو `const`).
- **التعبير:** تركيب من متغيرات وثوابت وعوامل يُحسب ليعطي قيمة.

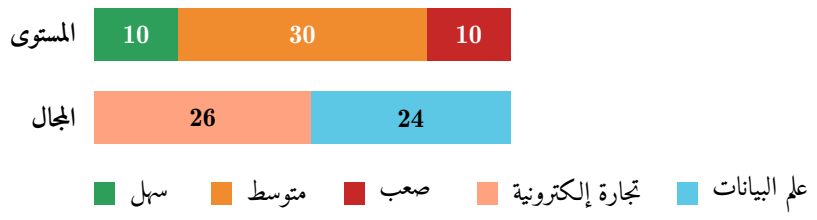


النوع	الصيغة	الاستعمال في التمارين
`int`	`%d`	الكميات، عدد الطلبيات، الثواني
`long`	`%ld`	الأعداد الكبيرة (رموز المنتجات، الأحجام)
`double`	`%f` / `%lf`	الأسعار، النسب، المتوسطات
`char`	`%c`	رموز الأصناف

القسمة في C

`2 / 7` تعطي `3` (قسمة صحيحة) و`2 % 7` تعطي `1`، أما `2.0 / 7` فتعطي `3.5`.

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 1.1 السعر مع الرسم على القيمة المضافة

تجارة إلكترونية سهل

يبيع متجر إلكتروني منتجًا بسعر دون رسوم HT. اكتب برنامجًا يقرأ هذا السعر ثم يحسب قيمة الرسم على القيمة المضافة بنسبة 19% والسعر النهائي TTC.

خطوات الحل

1) نعرّف ثابتًا TVA = 0.19.
2) نقرأ السعر ht.
3) نحسب الرسم tax = ht * TVA.
4) نحسب السعر النهائي ttc = ht + tax ثم نعرض النتيجتين.

البرنامج بلغة C

```c
#include <stdio.h>

#define TVA 0.19                    // نسبة الرسم 19%

int main(void) {
    double ht, tax, ttc;            // السعر دون رسوم، الرسم، السعر النهائي
    printf("Price HT: ");
    scanf("%lf", &ht);              // قراءة السعر دون رسوم
    tax = ht * TVA;                 // قيمة الرسم
    ttc = ht + tax;                 // السعر النهائي
    printf("\nTax = %.2f DA\n", tax);
    printf("TTC = %.2f DA\n", ttc);
    return 0;
}
```

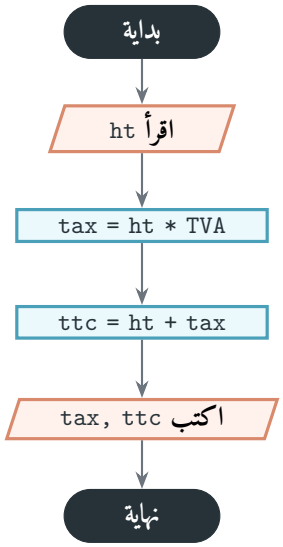
المدخلات

```
2500
```

نتيجة التنفيذ

```
Price HT:
Tax = 475.00 DA
TTC = 2975.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.2 مبلغ سطر في الفاتورة

تجارة إلكترونية سهل

اشترى زبون عدة وحدات من نفس المنتج. اكتب برنامجًا يقرأ **سعر الوحدة** و**الكمية** ويعرض مبلغ هذا السطر من الفاتورة.

خطوات الحل

1) نقرأ سعر الوحدة `price` (عدد حقيقي) والكمية `qty` (عدد صحيح).
2) نضرب السعر في الكمية: `amount = price * qty`.
3) نعرض المبلغ بمنزلتين عشريتين.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, amount;          // سعر الوحدة ومبلغ السطر
    int qty;                       // الكمية عدد صحيح
    scanf("%lf %d", &price, &qty);
    amount = price * qty;          // المبلغ = السعر × الكمية
    printf("%d x %.2f = %.2f DA\n", qty, price, amount);
    return 0;
}
```

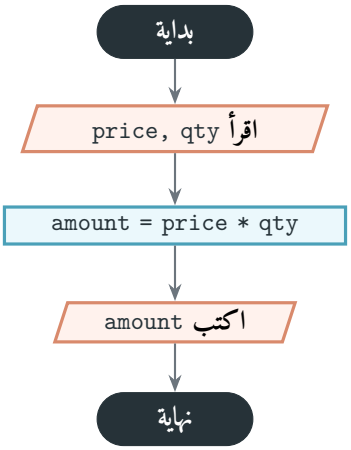
المدخلات

```
349.90 3
```

نتيجة التنفيذ

```
3 x 349.90 = 1049.70 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.3 متوسط تقييمات منتج

علم البيانات سهل

ترك ثلاثة زبائن تقييمات لمنتج (من 1 إلى 5). اكتب برنامجًا يقرأ التقييمات الثلاثة ويحسب متوسطها كعدد حقيقي.

خطوات الحل

1) نقرأ التقييمات r1 وr2 وr3 كأعداد صحيحة.
2) نحسب المجموع ثم نقسمه على 3.0 (وليس 3) لنحصل على قسمة حقيقية.
3) نعرض المتوسط.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int r1, r2, r3;              // التقييمات الثلاثة
    double avg;
    scanf("%d %d %d", &r1, &r2, &r3);
    // القسمة على 3.0 تجعل النتيجة حقيقية
    avg = (r1 + r2 + r3) / 3.0;
    printf("Average rating = %.2f / 5\n", avg);
    return 0;
}
```

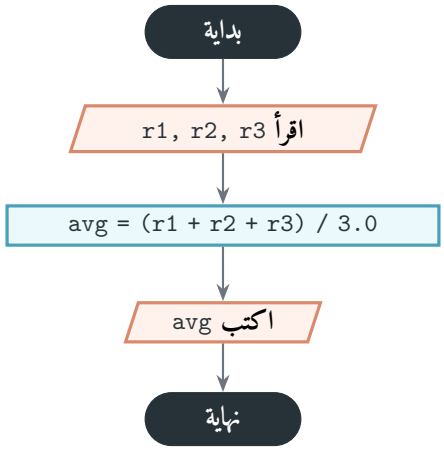
المدخلات

```
4 5 3
```

نتيجة التنفيذ

```
Average rating = 4.00 / 5
```

المخطط الإنسيابي (Organigramme)



تمرين 1.4 تحويل العملة

تجارة إلكترونية سهل

يعرض متجر جزائري أسعاره بالدينار، ويريد عرضها أيضًا باليورو. سعر الصرف ثابت: 1 يورو = 146.5 دج. اكتب برنامجًا يحول مبلغًا مقروءًا من الدينار إلى اليورو.

خطوات الحل

1) نعرّف الثابت RATE = 146.5.
2) نقرأ المبلغ بالدينار da.
3) نقسم على سعر الصرف: eur = da / RATE.

البرنامج بلغة C

```c
#include <stdio.h>

#define RATE 146.5              // سعر صرف اليورو بالدينار

int main(void) {
    double da, eur;
    scanf("%lf", &da);          // المبلغ بالدينار
    eur = da / RATE;            // التحويل إلى اليورو
    printf("%.2f DA = %.2f EUR\n", da, eur);
    return 0;
}
```

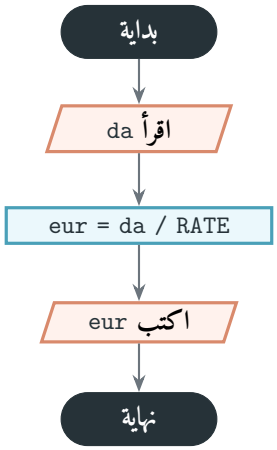
المدخلات

```
12000
```

نتيجة التنفيذ

```
12000.00 DA = 81.91 EUR
```

المخطط الإنسيابي (Organigramme)



تمرين 1.5 الوزن الإجمالي للشحنة

سهل تجارة إلكترونية

تتكون شحنة من ثلاثة طرود. اكتب برنامجًا يقرأ أوزانها بالكيلوغرام ويعرض **الوزن الإجمالي** و**متوسط** وزن الطرد.

خطوات الحل

1) نقرأ الأوزان الثلاثة `w1` و`w2` و`w3`.
2) نحسب المجموع `total`.
3) نحسب المتوسط `total / 3`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double w1, w2, w3, total, avg;
    scanf("%lf %lf %lf", &w1, &w2, &w3);
    total = w1 + w2 + w3;          // الوزن الإجمالي
    avg = total / 3;               // total حقيقي، فالقسمة حقيقية
    printf("Total weight = %.2f kg\n", total);
    printf("Average weight = %.2f kg\n", avg);
    return 0;
}
```

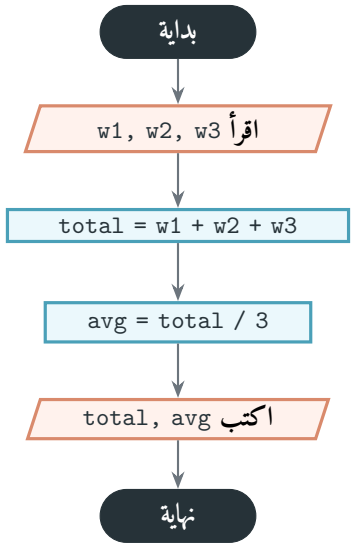
المدخلات

```
2.4 0.8 5.1
```

نتيجة التنفيذ

```
Total weight = 8.30 kg
Average weight = 2.77 kg
```

المخطط الإنسيابي (Organigramme)



تمرين 1.6 التخفيض الموسمي

تجارة إلكترونية سهل

في فترة التخفيضات، يُطبَّق على كل منتج نسبة تخفيض. اكتب برنامجًا يقرأ السعر الأصلي ونسبة التخفيض (بالمئة) ويعرض قيمة التخفيض والسعر الجديد.

خطوات الحل

1) نقرأ السعر `price` والنسبة `rate`.
2) قيمة التخفيض: `disc = price * rate / 100`.
3) السعر الجديد: `newPrice = price - disc`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, rate, disc, newPrice;
    scanf("%lf %lf", &price, &rate);
    disc = price * rate / 100;     // قيمة التخفيض
    newPrice = price - disc;       // السعر بعد التخفيض
    printf("Discount = %.2f DA\n", disc);
    printf("New price = %.2f DA\n", newPrice);
    return 0;
}
```

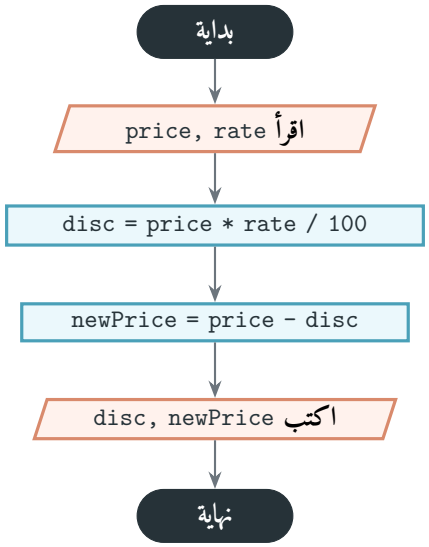
المدخلات

```
8900 30
```

نتيجة التنفيذ

```
Discount = 2670.00 DA
New price = 6230.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.7 الربح وهامش الربح

تجارة إلكترونية سهل

يشتري بائع منتجًا بسعر buy ويبيعه بسعر sell. اكتب برنامجًا يحسب **الربح وهامش الربح** كنسبة مئوية من سعر البيع.

خطوات الحل

1) نقرأ سعر الشراء وسعر البيع.
2) الربح: profit = sell - buy.
3) الهامش: margin = profit / sell * 100.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double buy, sell, profit, margin;
    scanf("%lf %lf", &buy, &sell);
    profit = sell - buy;              // الربح لكل وحدة
    margin = profit / sell * 100;     // الهامش بالمئة
    printf("Profit = %.2f DA\n", profit);
    printf("Margin = %.2f %%\n", margin);   // % تطبع الرمز %%
    return 0;
}
```

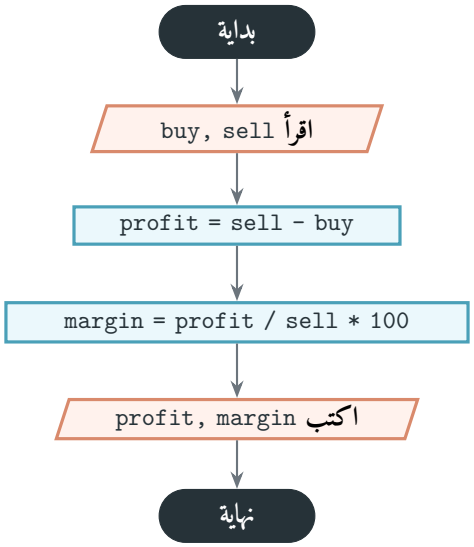
المدخلات

```
1500 2100
```

نتيجة التنفيذ

```
Profit = 600.00 DA
Margin = 28.57 %
```

المخطط الإنسيابي (Organigramme)



تمرين 1.8 تبديل سعرين

تجارة إلكترونية سهل

أدخل موظف سعري منتجين بالعكس. اكتب برنامجًا يقرأ السعرين priceA وpriceB ثم **يبدل** قيمتيهما باستعمال متغير مساعد، ويعرضهما قبل التبديل وبعده.

خطوات الحل

1) نقرأ priceA وpriceB.
2) نحفظ priceA في متغير مساعد tmp.
3) نضع priceB في priceA ثم tmp في priceB.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int priceA, priceB, tmp;
    scanf("%d %d", &priceA, &priceB);
    printf("Before: a = %d, b = %d\n", priceA, priceB);
    tmp = priceA;          // نحفظ priceA قبل أن نفقدها
    priceA = priceB;            // priceA تأخذ قيمة priceB
    priceB = tmp;          // priceB تأخذ القيمة القديمة لـ priceA
    printf("After:  a = %d, b = %d\n", priceA, priceB);
    return 0;
}
```

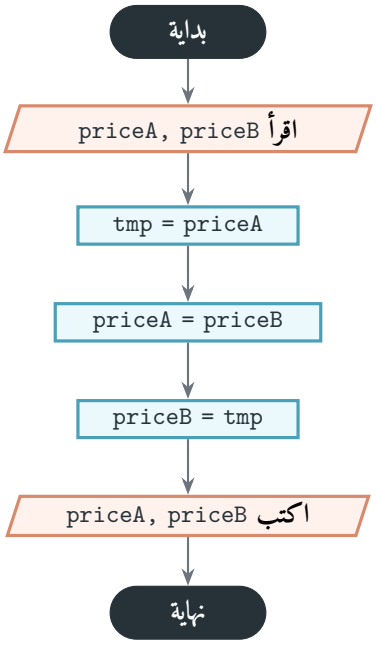
المدخلات

```
450 1200
```

نتيجة التنفيذ

```
Before: a = 450, b = 1200
After:  a = 1200, b = 450
```

المخطط الإنسيابي (Organigramme)



تمرين 1.9 مدة جلسة التصفح

علم البيانات سهل

يسجل موقع التجارة الإلكترونية مدة كل جلسة بالثواني. اكتب برنامجًا يحول المدة إلى **دقائق وثوانٍ** باستعمال القسمة الصحيحة / وباقي القسمة %.

خطوات الحل

1) نقرأ المدة `seconds` بالثواني.
2) عدد الدقائق: `minutes = seconds / 60` (قسمة صحيحة).
3) الثواني المتبقية: `rest = seconds % 60`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int seconds, minutes, rest;
    scanf("%d", &seconds);              // المدة بالثواني
    minutes = seconds / 60;                 // قسمة صحيحة: عدد الدقائق الكاملة
    rest = seconds % 60;                    // الباقي: الثواني المتبقية
    printf("%d s = %d min %d s\n", seconds, minutes, rest);
    return 0;
}
```

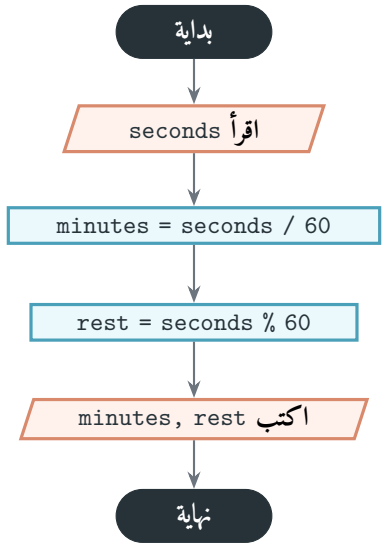
المدخلات

```
437
```

نتيجة التنفيذ

```
437 s = 7 min 17 s
```

المخطط الإنسيابي (Organigramme)



تمرين 1.10 معدل التحويل

علم البيانات سهل

معدل التحويل هو نسبة الزوار الذين قاموا بالشراء. اكتب برنامجًا يقرأ عدد الزوار وعدد الطلبيات ويعرض معدل التحويل بالمئة.

خطوات الحل

1) نقرأ `visitors` و`orders` كأعداد صحيحة.
2) نحول أحدهما إلى `double` قبل القسمة لتجنب القسمة الصحيحة.
3) `rate = orders / visitors * 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int visitors, orders;
    double rate;
    scanf("%d %d", &visitors, &orders);
    // التحويل (double) يمنع القسمة الصحيحة التي تعطي 0
    rate = (double)orders / visitors * 100;
    printf("Conversion rate = %.2f %%\n", rate);
    return 0;
}
```

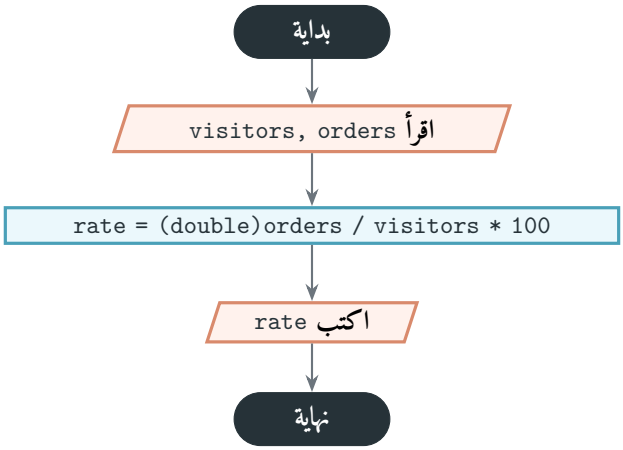
المدخلات

```
5400 162
```

نتيجة التنفيذ

```
Conversion rate = 3.00 %
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 1.11 عدد علب التغليف

تجارة إلكترونية ⭐⭐ متوسط

يجب تغليف `items` منتجًا في علب تتسع كل واحدة لـ `capacity` منتجات. اكتب برنامجًا يحسب **عدد العلب اللازمة** وعدد المنتجات في **العلبة الأخيرة**، دون استعمال أي شرط.

خطوات الحل

1) عدد العلب هو القسمة الصحيحة مقربة إلى الأعلى: `boxes = (items + capacity - 1) / capacity`.
2) المنتجات الموجودة في العلب الممتلئة: `(boxes - 1) * capacity`.
3) العلبة الأخيرة: `last = items - (boxes - 1) * capacity`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int items, capacity, boxes, last;
    scanf("%d %d", &items, &capacity);
    // إضافة capacity-1 قبل القسمة تقرب النتيجة إلى الأعلى
    boxes = (items + capacity - 1) / capacity;
    // ما تبقى بعد ملء كل العلب ما عدا الأخيرة
    last = items - (boxes - 1) * capacity;
    printf("Boxes needed = %d\n", boxes);
    printf("Items in last box = %d\n", last);
    return 0;
}
```

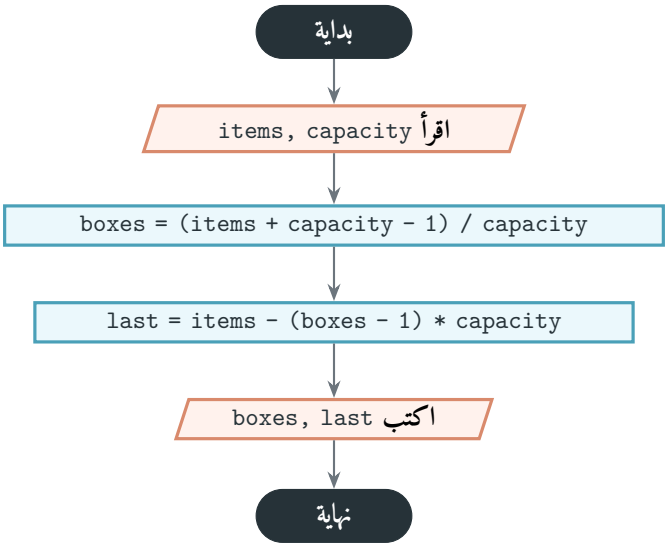
المدخلات

```
47 12
```

نتيجة التنفيذ

```
Boxes needed = 4
Items in last box = 11
```

المخطط الإنسيابي (Organigramme)



تمرين 1.12 المسافة بين زبونين

علم البيانات متوسط

نمثل كل زبون بخاصيتين: عدد الطلبيات x ومتوسط السلة بالآلاف y. اكتب برنامجًا يحسب **مربع المسافة الإقليدية** بين زبونين (دون جذر)، ويعرض أيضًا الفرق في عدد الطلبيات.

خطوات الحل

1) نقرأ إحداثيات الزبونين (x1, y1) و(x2, y2).
2) نحسب الفرقين dx = x1 - x2 و dy = y1 - y2.
3) مربع المسافة: d2 = dx*dx + dy*dy.
4) مسافة مانهاتن للخاصية الأولى هي dx مباشرة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, dx, dy, d2;
    scanf("%lf %lf", &x1, &y1);   // الزبون الأول
    scanf("%lf %lf", &x2, &y2);   // الزبون الثاني
    dx = x1 - x2;                 // الفرق في عدد الطلبيات
    dy = y1 - y2;                 // الفرق في متوسط السلة
    d2 = dx * dx + dy * dy;       // مربع المسافة دون جذر
    printf("Squared distance = %.2f\n", d2);
    printf("Orders gap = %.0f\n", dx);
    return 0;
}
```

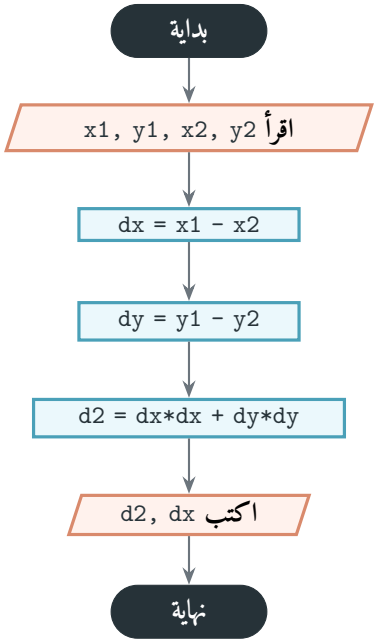
المدخلات

```
12 4.5
7 2.5
```

نتيجة التنفيذ

```
Squared distance = 29.00
Orders gap = 5
```

المخطط الإنسيابي (Organigramme)



تمرين 1.13 المتوسط والتباين

علم البيانات ★★ متوسط

لدينا مبيعات أربعة أيام. اكتب برنامجًا يحسب **المتوسط** و**التباين** (متوسط مربعات الانحرافات عن المتوسط).

خطوات الحل

1) نقرأ القيم الأربع ونحسب المتوسط `mean`.
2) نحسب انحراف كل قيمة: `ai - mean`.
3) التباين: مجموع مربعات الانحرافات مقسوم على 4.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double day1, day2, day3, day4, mean, variance;
    scanf("%lf %lf %lf %lf", &day1, &day2, &day3, &day4);
    mean = (day1 + day2 + day3 + day4) / 4;          // المتوسط
    // التباين: متوسط مربعات الانحرافات
    variance = ((day1 - mean) * (day1 - mean) + (day2 - mean) * (day2 - mean)
       + (day3 - mean) * (day3 - mean) + (day4 - mean) * (day4 - mean)) / 4;
    printf("Mean = %.2f\n", mean);
    printf("Variance = %.2f\n", variance);
    return 0;
}
```

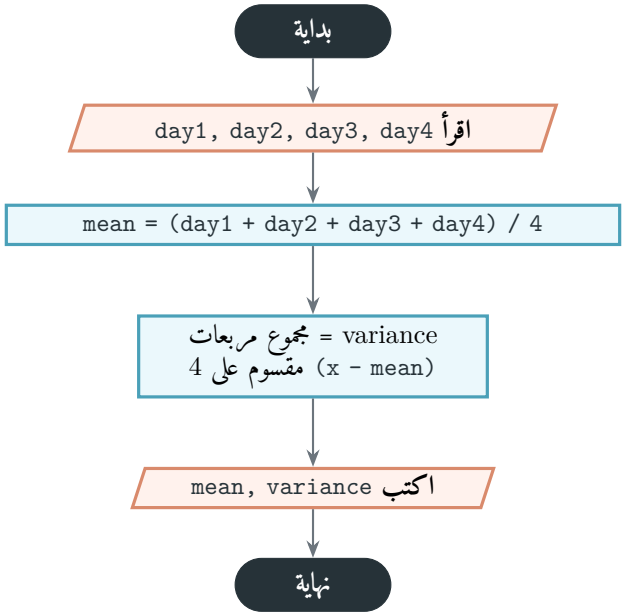
المدخلات

```
120 95 140 105
```

نتيجة التنفيذ

```
Mean = 115.00
Variance = 287.50
```

المخطط الإنسيابي (Organigramme)



تمرين 1.14 التطبيع بين 0 و1

علم البيانات ★★ متوسط

قبل تدريب نموذج، نحول الأسعار إلى المجال [0, 1] بطريقة **Min-Max**. اكتب برنامجًا يقرأ أصغر سعر وأكبر سعر وسعرًا price ويعرض قيمته المطبّعة.

خطوات الحل

1) نقرأ min وmax وprice.
2) الصيغة: xn = (price - min) / (max - min).
3) نعرض xn بثلاث منازل عشرية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double min, max, price, xn;
    scanf("%lf %lf %lf", &min, &max, &price);
    // نطرح الأصغر ثم نقسم على المدى
    xn = (price - min) / (max - min);
    printf("Normalized value = %.3f\n", xn);
    return 0;
}
```

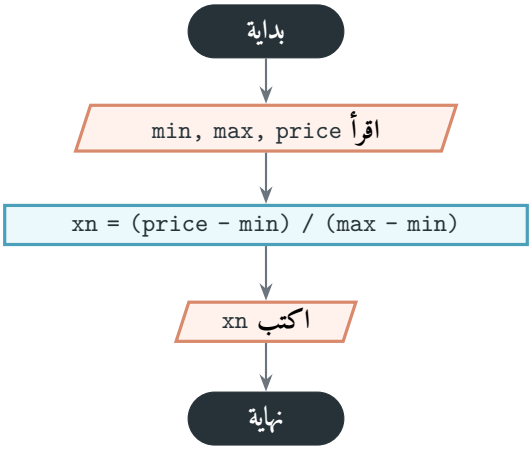
المدخلات

```
200 5200 1450
```

نتيجة التنفيذ

```
Normalized value = 0.250
```

المخطط الإنسيابي (Organigramme)



تمرين 1.15 الدرجة المعيارية

علم البيانات ★★ متوسط

لاكتشاف الطلبيات غير العادية نحسب **الدرجة المعيارية** zscore = (amount - mean) / std. اكتب برنامجًا يقرأ مبلغ طلبية ومتوسط المبالغ وانحرافها المعياري، ويعرض zscore ثم المبلغ المقابل لـ zscore = 2.

خطوات الحل

1) نقرأ amount وmean وstd.
2) zscore = (amount - mean) / std.
3) المبلغ الموافق لدرجة معيارية 2: x2 = mean + 2 * std.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, mean, std, zscore, x2;
    scanf("%lf %lf %lf", &amount, &mean, &std);
    zscore = (amount - mean) / std;          // كم انحرافًا معياريًا يبعد amount عن المتوسط
    x2 = mean + 2 * std;           // العتبة الموافقة لـ zscore = 2
    printf("z-score = %.2f\n", zscore);
    printf("Threshold (z = 2) = %.2f DA\n", x2);
    return 0;
}
```

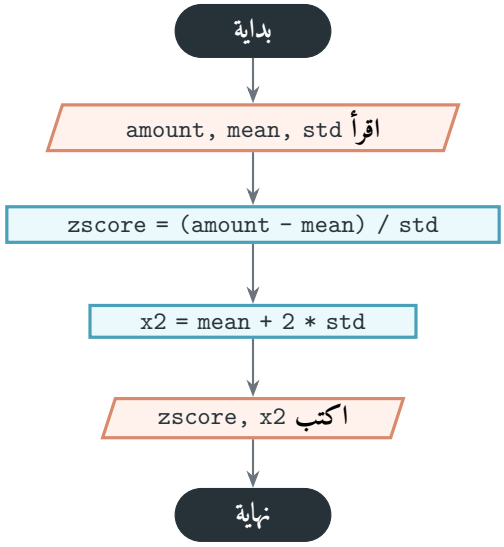
المدخلات

```
9800 4200 1900
```

نتيجة التنفيذ

```
z-score = 2.95
Threshold (z = 2) = 8000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.16 متوسط النجوم الموزون

علم البيانات ★★ متوسط

تعرض صفحة المنتج عدد التقييمات لكل نجمة (من نجمة إلى خمس نجوم). اكتب برنامجًا يقرأ الأعداد الخمسة ويحسب **المتوسط الموزون** وعدد التقييمات الكلي.

خطوات الحل

1) نقرأ `c1` إلى `c5`.
2) العدد الكلي: `nbReviews = c1 + c2 + c3 + c4 + c5`.
3) المجموع الموزون: `weighted = 1*c1 + 2*c2 + 3*c3 + 4*c4 + 5*c5`.
4) المتوسط: `weighted / nbReviews` بقسمة حقيقية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int c1, c2, c3, c4, c5, nbReviews, weighted;
    double avg;
    scanf("%d %d %d %d %d", &c1, &c2, &c3, &c4, &c5);
    nbReviews = c1 + c2 + c3 + c4 + c5;                  // عدد التقييمات
    weighted = 1 * c1 + 2 * c2 + 3 * c3 + 4 * c4 + 5 * c5; // كل نجمة × عددها
    avg = (double)weighted / nbReviews;                  // متوسط موزون
    printf("Reviews = %d\n", nbReviews);
    printf("Weighted average = %.2f stars\n", avg);
    return 0;
}
```

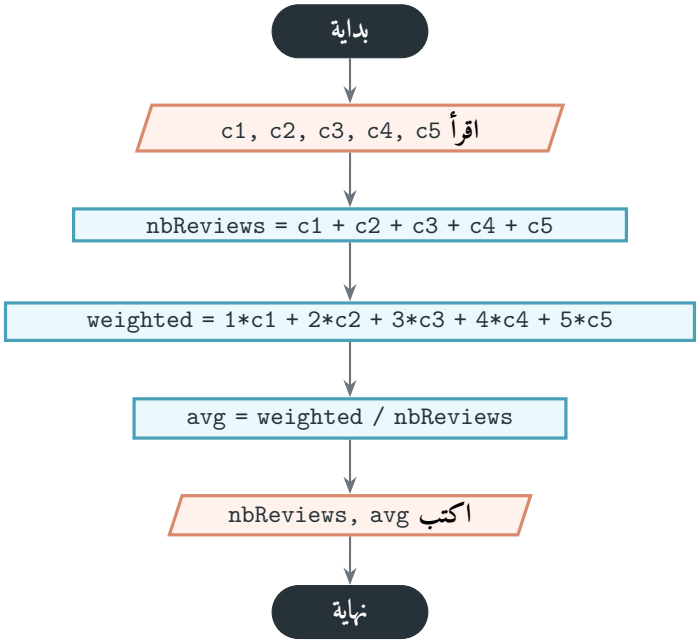
المدخلات

```
3 5 12 40 60
```

نتيجة التنفيذ

```
Reviews = 120
Weighted average = 4.24 stars
```

المخطط الإنسيابي (Organigramme)



تمرين 1.17 نمو رقم الأعمال

علم البيانات متوسط

رقم أعمال متجر هذا الشهر `revenue`، ويتوقع نموًا شهريًا ثابتًا بنسبة `growth` بالمئة. اكتب برنامجًا يحسب رقم الأعمال المتوقع بعد 1 و2 و3 أشهر، دون حلقات ودون دالة `pow`.

خطوات الحل

1) نحسب المعامل `factor = 1 + growth / 100`.
2) بعد شهر: `r1 = revenue * factor`، بعد شهرين: `r2 = r1 * factor`، بعد ثلاثة: `r3 = r2 * factor`.
3) نعرض القيم الثلاث.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double revenue, growth, factor, r1, r2, r3;
    scanf("%lf %lf", &revenue, &growth);
    factor = 1 + growth / 100;              // معامل النمو الشهري
    r1 = revenue * factor;                  // بعد شهر
    r2 = r1 * factor;                       // بعد شهرين = revenue × factor × factor
    r3 = r2 * factor;                       // بعد ثلاثة أشهر
```

```c
    printf("Month 1: %.2f\n", r1);
    printf("Month 2: %.2f\n", r2);
    printf("Month 3: %.2f\n", r3);
    return 0;
}
```

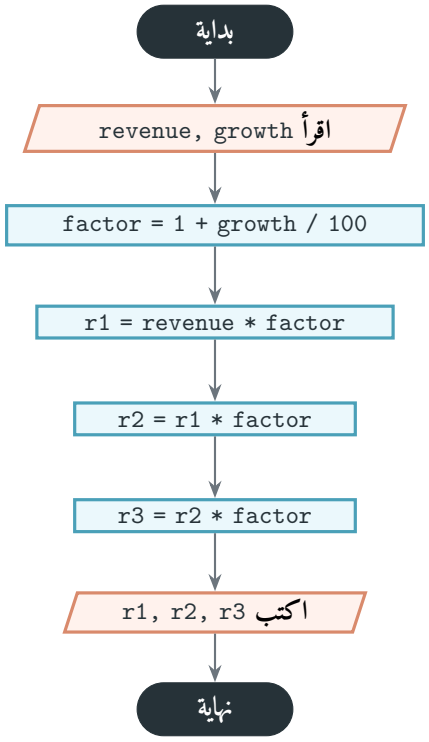
المدخلات

```
250000 8
```

نتيجة التنفيذ

```
Month 1: 270000.00
Month 2: 291600.00
Month 3: 314928.00
```

المخطط الإنسيابي (Organigramme)



تمرين 1.18 تفكيك مبلغ إلى أوراق نقدية

تجارة إلكترونية ★★ متوسط

عند الدفع نقدًا عند الاستلام، يريد عامل التوصيل تحضير المبلغ بأقل عدد من الأوراق: 2000 و1000 و500 و200، والباقي قطع نقدية. اكتب برنامجًا يفكك المبلغ.

خطوات الحل

1) عدد أوراق 2000: `notes2000 = money / 2000` ثم نأخذ الباقي `money = money % 2000`.
2) نكرر نفس الفكرة مع 1000 ثم 500 ثم 200.
3) ما بقي في `money` هو مبلغ القطع النقدية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int money, notes2000, notes1000, notes500, notes200;
    scanf("%d", &money);              // المبلغ بالدينار
    notes2000 = money / 2000; money = money % 2000;   // أوراق 2000 ثم ما تبقى
    notes1000 = money / 1000; money = money % 1000;   // أوراق 1000
    notes500 = money / 500;  money = money % 500;     // أوراق 500
    notes200 = money / 200;  money = money % 200;     // أوراق 200
    printf("2000 DA x %d\n1000 DA x %d\n", notes2000, notes1000);
    printf("500 DA x %d\n200 DA x %d\n", notes500, notes200);
    printf("Coins: %d DA\n", money);  // الباقي قطع نقدية
    return 0;
}
```

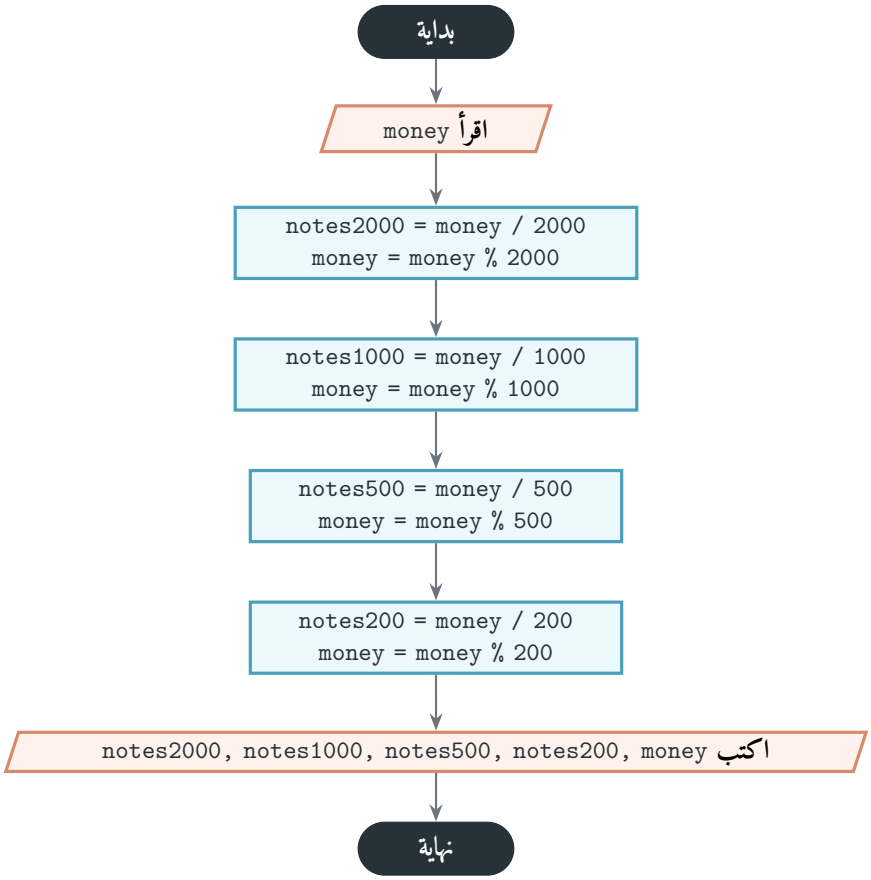
المدخلات

```
7900
```

نتيجة التنفيذ

```
2000 DA x 3
1000 DA x 1
500 DA x 1
200 DA x 2
Coins: 0 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.19 ساعة وصول الطلبية

تجارة إلكترونية ★★ متوسط

تُسجَّل طلبية في الساعة `hour:minute` ويستغرق توصيلها `delay` دقيقة. اكتب برنامجًا يحسب **ساعة الوصول** بصيغة 24 ساعة، حتى لو تجاوزت منتصف الليل.

خطوات الحل

1) نحول وقت الطلب إلى دقائق منذ منتصف الليل: `total = hour * 60 + minute + delay`.
2) ساعة الوصول: `(total / 60) % 24`.
3) الدقائق: `total % 60`.
4) نستعمل `%02d` لعرض صفر على اليسار.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int hour, minute, delay, total, ah, am;
    scanf("%d %d %d", &hour, &minute, &delay);
    total = hour * 60 + minute + delay;        // الدقائق منذ منتصف الليل
    ah = (total / 60) % 24;        // 24 % تعيد الساعة بعد 23 إلى 0
    am = total % 60;               // الدقائق
    printf("Arrival time: %02d:%02d\n", ah, am);
    return 0;
}
```

```
}
```

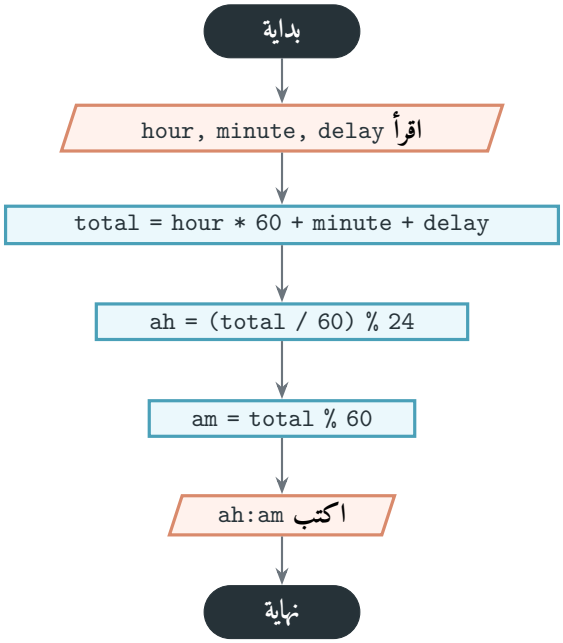
المدخلات

```
22 35 150
```

نتيجة التنفيذ

```
Arrival time: 01:05
```

المخطط الإنسيابي (Organigramme)



تمرين 1.20 مدة الإشتراك بالأسابيع

تجارة إلكترونية ★★ متوسط

يشترك زبون في خدمة توصيل مجاني لمدة days يومًا. اكتب برنامجًا يعرض هذه المدة بالأسابيع والأيام، وعدد الأشهر التقريبية (شهر = 30 يومًا) مع الأيام المتبقية.

خطوات الحل

1) الأسابيع: weeks = days / 7 والأيام: days % 7.
2) الأشهر: mo = days / 30 والأيام: days % 30.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int days, weeks, rw, mo, rm;
    scanf("%d", &days);
    weeks = days / 7;   rw = days % 7;      // أسابيع كاملة + أيام
    mo = days / 30; rm = days % 30;     // أشهر تقريبية + أيام
    printf("%d days = %d weeks + %d days\n", days, weeks, rw);
```

```
    printf("%d days = %d months + %d days\n", days, mo, rm);
    return 0;
}
```

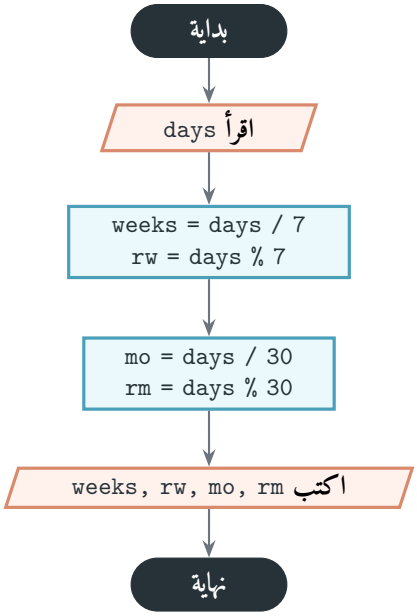
المدخلات

```
200
```

نتيجة التنفيذ

```
200 days = 28 weeks + 4 days
200 days = 6 months + 20 days
```

المخطط الإنسيابي (Organigramme)



تمرين 1.21 عكس رمز المنتج

تجارة إلكترونية متوسط

لكل منتج رمز من **ثلاثة أرقام**. لإنشاء رمز سري يعكس النظام ترتيب الأرقام (مثلًا 472 يصبح 274). اكتب برنامجًا يقوم بذلك باستعمال / و% فقط.

خطوات الحل

1) رقم الآحاد: `units = code % 10`.
2) رقم العشرات: `tens = (code / 10) % 10`.
3) رقم المئات: `hundreds = code / 100`.
4) الرمز المعكوس: `reversed = units * 100 + tens * 10 + hundreds`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int code, units, tens, hundreds, reversed;
```

```c
    scanf("%d", &code);
    units = code % 10;              // رقم الآحاد
    tens = (code / 10) % 10;        // نحذف الآحاد ثم نأخذ الرقم الأخير
    hundreds = code / 100;          // رقم المئات
    reversed = units * 100 + tens * 10 + hundreds; // نعيد التركيب بالترتيب المعاكس
    printf("Code %d -> %d\n", code, reversed);
    return 0;
}
```

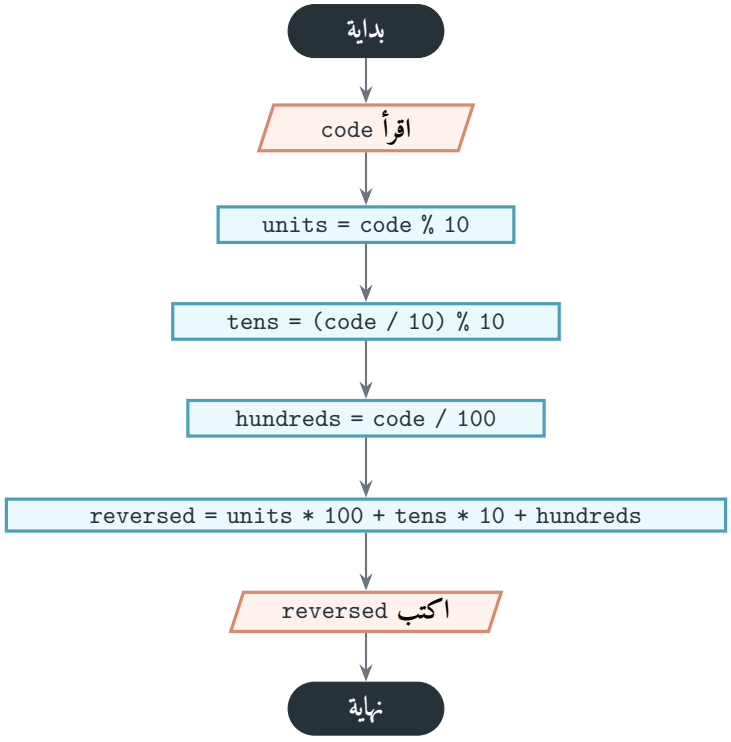
المدخلات

```
472
```

نتيجة التنفيذ

```
Code 472 -> 274
```

المخطط الإنسيابي (Organigramme)



تمرين 1.22 مجموع التحقق لرقم الطلبية

تجارة إلكترونية ★★ متوسط

رقم كل طلبية مكون من **أربعة أرقام**. لاكتشاف أخطاء الإدخال نحسب **مجموع أرقامه** ثم رقم التحقق `S % 10`. اكتب برنامجًا يقوم بذلك.

خطوات الحل

1) نستخرج الأرقام الأربعة بالقسمة على 1000 و100 و10 وبباقي القسمة على 10.
2) نجمعها في `digitSum`.
3) رقم التحقق هو `digitSum % 10`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int order, d1, d2, d3, d4, digitSum;
    scanf("%d", &order);
    d1 = order / 1000;            // رقم الآلاف
    d2 = (order / 100) % 10;      // رقم المئات
    d3 = (order / 10) % 10;       // رقم العشرات
    d4 = order % 10;              // رقم الآحاد
    digitSum = d1 + d2 + d3 + d4;    // مجموع الأرقام
    printf("Digit sum = %d\n", digitSum);
    printf("Check digit = %d\n", digitSum % 10);
    return 0;
}
```

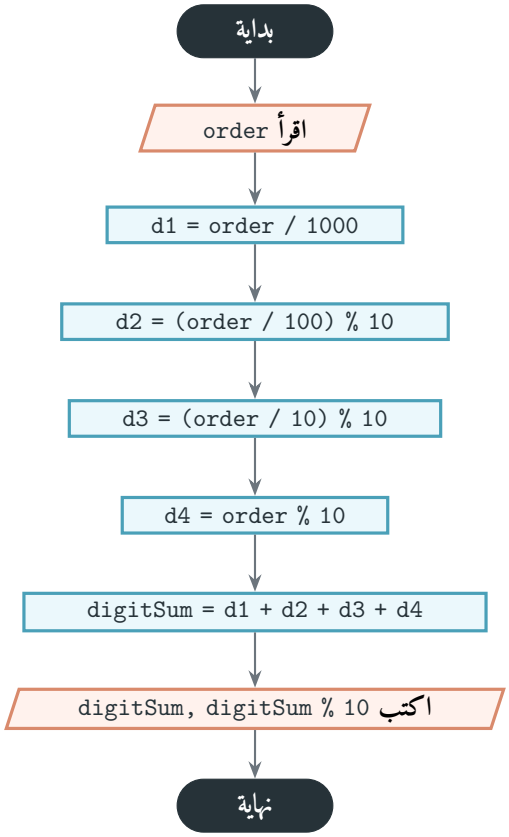
المدخلات

```
5837
```

نتيجة التنفيذ

```
Digit sum = 23
Check digit = 3
```

المخطط الإنسيابي (Organigramme)



تمرين 1.23 تقريب السعر إلى منزلتين

تجارة إلكترونية ★★ متوسط

يُنتج حساب الضرائب أحيانًا أسعارًا مثل 1234.5678. اكتب برنامجًا يقرب سعرًا إلى **منزلتين عشريتين** دون دالة round، ثم إلى **أقرب 10 دنانير**.

خطوات الحل

1) نضرب في 100 ونضيف 0.5 ثم نحول إلى int (يحذف الجزء العشري).
2) نقسم على 100.0 لنرجع إلى الدينار.
3) لأقرب 10: ((int)(price / 10 + 0.5)) * 10.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, r2;
    int r10;
    scanf("%lf", &price);
    // إضافة 0.5 ثم الحذف بالتحويل إلى int = تقريب لأقرب عدد صحيح
    r2 = (int)(price * 100 + 0.5) / 100.0;
    r10 = (int)(price / 10 + 0.5) * 10;      // تقريب لأقرب 10 دنانير
    printf("Rounded (2 decimals) = %.4f\n", r2);
    printf("Rounded (10 DA) = %d\n", r10);
    return 0;
}
```

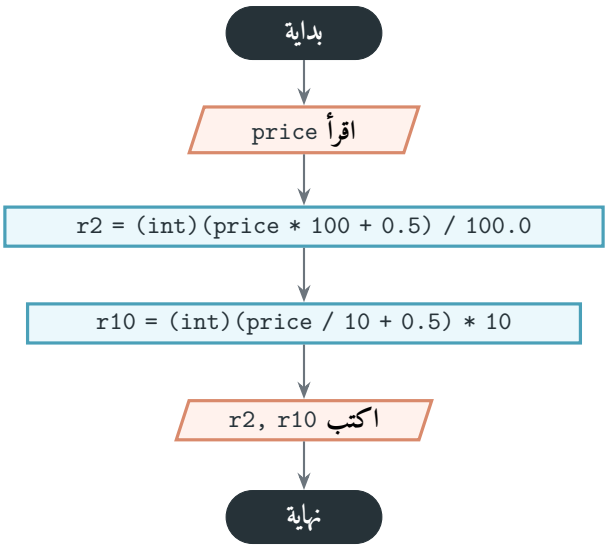
المدخلات

```
1234.5678
```

نتيجة التنفيذ

```
Rounded (2 decimals) = 1234.5700
Rounded (10 DA) = 1230
```

المخطط الإنسيابي (Organigramme)



تمرين 1.24 مؤشرات الحملة الإعلانية

علم البيانات ★★ متوسط

حملة إعلانية حققت عددًا من **مرات الظهور والنقرات** وكلفت مبلغًا معينًا. اكتب برنامجًا يحسب نسبة النقر CTR وتكلفة النقرة CPC وتكلفة الألف ظهور CPM.

خطوات الحل

1) `CTR = clicks / impressions * 100`.
2) `CPC = cost / clicks`.
3) `CPM = cost / impressions * 1000`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double imp, clicks, cost, ctr, cpc, cpm;
    scanf("%lf %lf %lf", &imp, &clicks, &cost);
    ctr = clicks / imp * 100;      // نسبة النقر بالمئة
    cpc = cost / clicks;           // تكلفة النقرة الواحدة
    cpm = cost / imp * 1000;       // تكلفة كل ألف ظهور
    printf("CTR = %.2f %%\n", ctr);
    printf("CPC = %.2f DA\n", cpc);
    printf("CPM = %.2f DA\n", cpm);
    return 0;
}
```

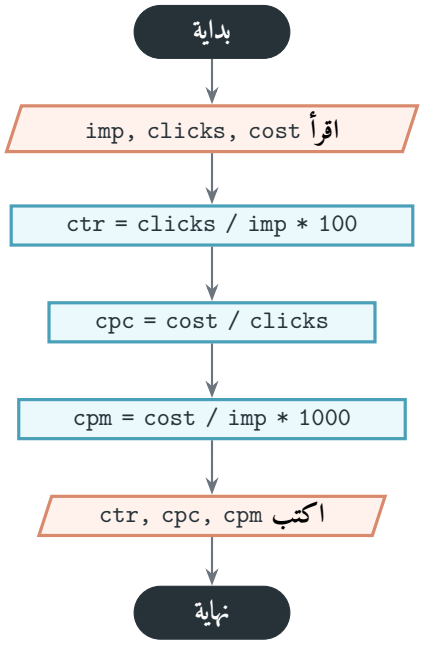
المدخلات

```
120000 3000 45000
```

نتيجة التنفيذ

```
CTR = 2.50 %
CPC = 15.00 DA
CPM = 375.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.25 القيمة الدائمة للزبون

علم البيانات ★★ متوسط

القيمة الدائمة للزبون CLV تقدر بـ: متوسط السلة × عدد الطلبيات في السنة × عدد سنوات الوفاء × هامش الربح. اكتب برنامجًا يحسبها.

خطوات الحل

1) نقرأ basket وfreq وyears وmargin.
2) رقم الأعمال المتوقع: rev = basket * freq * years.
3) clv = rev * margin.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double basket, margin, rev, clv;
    int freq, years;
    scanf("%lf %d %d %lf", &basket, &freq, &years, &margin);
    rev = basket * freq * years;  // رقم الأعمال طوال مدة الوفاء
    clv = rev * margin;           // الربح الذي يحققه الزبون
    printf("Expected revenue = %.2f DA\n", rev);
    printf("CLV = %.2f DA\n", clv);
    return 0;
}
```

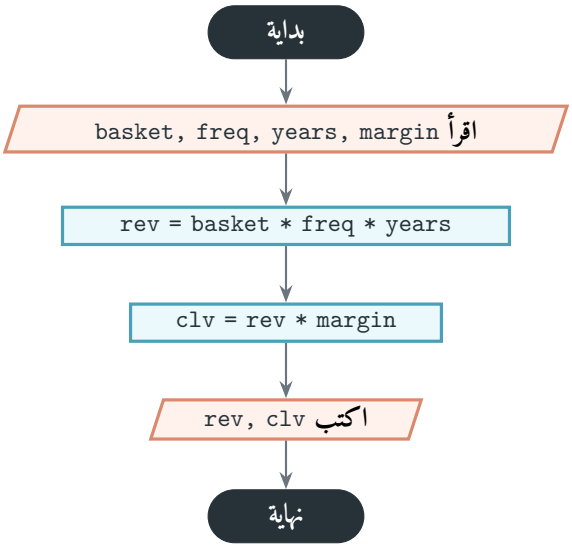
المدخلات

```
3200 6 4 0.25
```

نتيجة التنفيذ

```
Expected revenue = 76800.00 DA
CLV = 19200.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.26 متوسط قيمة الطلبية

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ رقم الأعمال (عدد صحيح) وعدد الطلبيات، ثم يعرض متوسط قيمة الطلبية مرة بقسمة صحيحة ومرة بقسمة حقيقية، ليوضح الفرق بينهما.

خطوات الحل

1) القسمة بين عددين صحيحين تحذف الجزء العشري: `rev / nbOrders`.
2) للحصول على قسمة حقيقية نحول: `(double)rev / nbOrders`.
3) الفرق بين النتيجتين هو الجزء الضائع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int rev, nbOrders, quotient;
    double avg;
    scanf("%d %d", &rev, &nbOrders);
    quotient = rev / nbOrders;                  // قسمة صحيحة: الجزء العشري يضيع
    avg = (double)rev / nbOrders;               // قسمة حقيقية
    printf("Integer division = %d\n", quotient);
    printf("Real division    = %.4f\n", avg);
    printf("Lost part        = %.4f\n", avg - quotient);
    return 0;
}
```

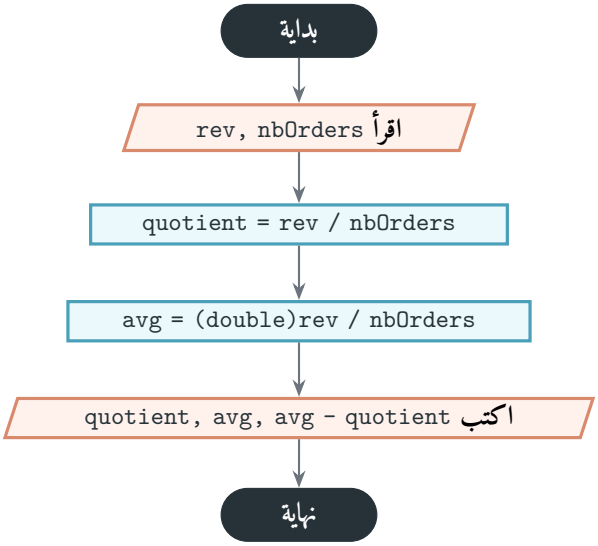
المدخلات

```
98765 23
```

نتيجة التنفيذ

```
Integer division = 4294
Real division    = 4294.1304
Lost part        = 0.1304
```

المخطط الإنسيابي (Organigramme)



تمرين 1.27 أيام تغطية المخزون

تجارة إلكترونية ★★ متوسط

مخزون منتج في بداية الأسبوع `stock`. بيعت منه `sold` وحدة ووصلت شحنة من `rec` وحدة. اكتب برنامجًا يحسب المخزون الجديد وعدد الأيام الكاملة التي يغطيها إذا كان البيع اليومي `daily`.

خطوات الحل

1) المخزون الجديد: `stock = stock - sold + rec`.
2) أيام التغطية: `days = stock / daily` (قسمة صحيحة).
3) الوحدات المتبقية بعد آخر يوم كامل: `stock % daily`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int stock, sold, rec, daily, days, rest;
    scanf("%d %d %d %d", &stock, &sold, &rec, &daily);
    stock = stock - sold + rec;          // تحديث المخزون
    days = stock / daily;                // عدد الأيام الكاملة
    rest = stock % daily;                // ما يبقى بعدها
    printf("New stock = %d\n", stock);
    printf("Covers %d full days (%d left)\n", days, rest);
    return 0;
}
```

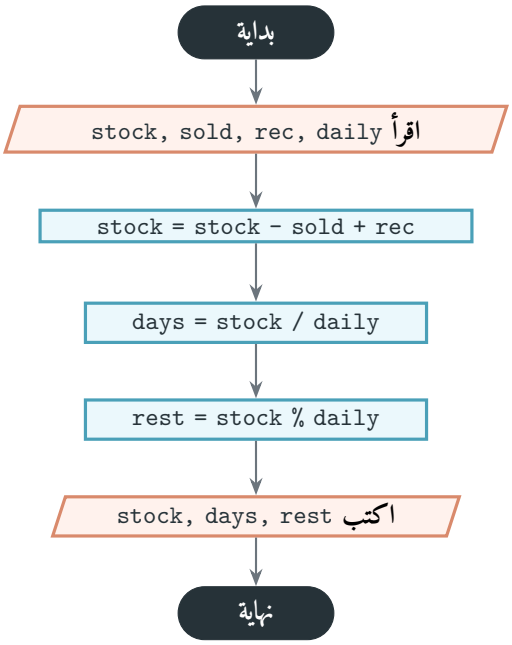
المدخلات

```
340 215 500 45
```

نتيجة التنفيذ

```
New stock = 625
Covers 13 full days (40 left)
```

المخطط الإنسيابي (Organigramme)



تمرين 1.28 التنبؤ بنموذج خطي

علم البيانات ★★ متوسط

درّب محلل بيانات نموذجًا خطيًا يتنبأ بالمبيعات y بدلالة ميزانية الإعلان x (بالآلاف): y = slope*x + intercept. اكتب برنامجًا يقرأ slope وintercept وميزانيتين ويعرض التنبؤين والفرق بينهما.

خطوات الحل

1) نقرأ معاملي النموذج slope وintercept.
2) نقرأ الميزانيتين x1 وx2.
3) نحسب y1 = slope*x1 + intercept وy2 = slope*x2 + intercept والفرق y2 - y1.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double slope, intercept, x1, x2, y1, y2;
    scanf("%lf %lf", &slope, &intercept);      // معاملا النموذج
    scanf("%lf %lf", &x1, &x2);    // ميزانيتان
    y1 = slope * x1 + intercept;               // التنبؤ الأول
    y2 = slope * x2 + intercept;               // التنبؤ الثاني
    printf("Prediction 1 = %.1f\n", y1);
    printf("Prediction 2 = %.1f\n", y2);
```

```c
    printf("Gain = %.1f\n", y2 - y1);   // = slope × (x2 - x1)
    return 0;
}
```

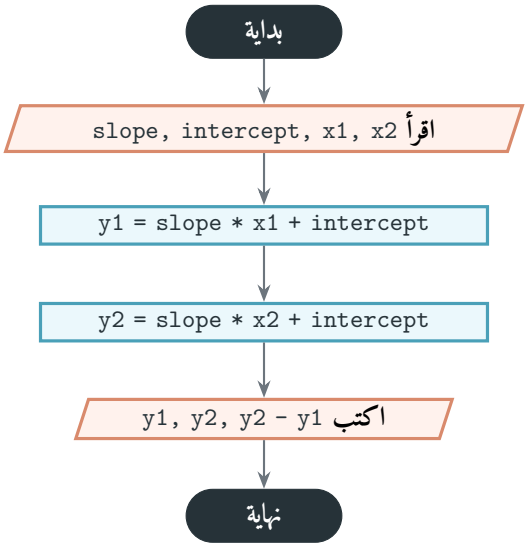
المدخلات

```
3.2 150
20 35
```

نتيجة التنفيذ

```
Prediction 1 = 214.0
Prediction 2 = 262.0
Gain = 48.0
```

المخطط الإنسيابي (Organigramme)



تمرين 1.29 مستقيم يمر بنقطتين

علم البيانات متوسط

لاحظنا أن سعر 100 دج يحقق 900 مبيعة، وسعر 150 دج يحقق 650 مبيعة. اكتب برنامجًا يقرأ نقطتين `(x1, y1)` و`(x2, y2)` ويحسب **الميل** و**الجزء المقطوع** للمستقيم المار بهما، ثم يتنبأ بالمبيعات عند سعر 120.

خطوات الحل

1) الميل: `slope = (y2 - y1) / (x2 - x1)`.
2) الجزء المقطوع: `intercept = y1 - slope * x1`.
3) التنبؤ: `sales = slope * 120 + intercept`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, slope, intercept, sales;
    scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2);
    slope = (y2 - y1) / (x2 - x1);     // الميل: تغير sales لكل وحدة من x
```

```c
    intercept = y1 - slope * x1;              // من intercept + slope*x1 = y1
    sales = slope * 120 + intercept;              // التنبؤ عند السعر 120
    printf("Slope a = %.2f\n", slope);
    printf("Intercept b = %.2f\n", intercept);
    printf("Sales at 120 DA = %.0f\n", sales);
    return 0;
}
```

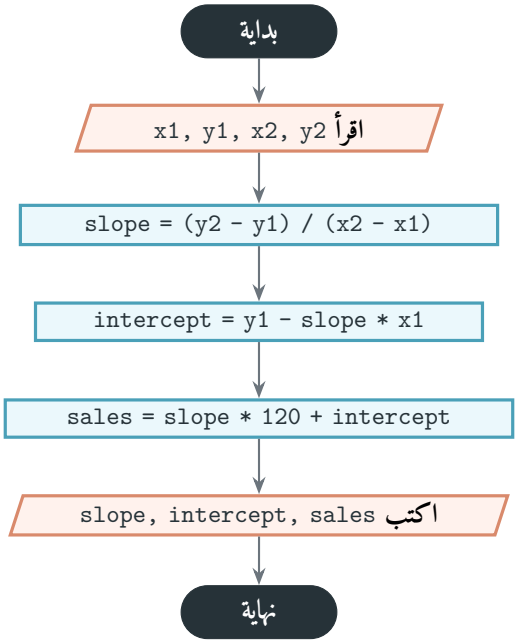
المدخلات

```
100 900
150 650
```

نتيجة التنفيذ

```
Slope a = -5.00
Intercept b = 1400.00
Sales at 120 DA = 800
```

المخطط الإنسيابي (Organigramme)



تمرين 1.30 حصص الأصناف من رقم الأعمال

علم البيانات متوسط

حقق متجر رقم أعمال في ثلاثة أصناف: إلكترونيات، ملابس، كتب. اكتب برنامجًا يحسب حصة كل صنف بالمئة، ويتحقق أن مجموع الحصص 100%.

خطوات الحل

1) المجموع: `total = electronics + clothing + books`.
2) حصة كل صنف: `/ total * 100` .
3) نعرض مجموع الحصص للتحقق.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double electronics, clothing, books, total, pe, pc, pb;
    scanf("%lf %lf %lf", &electronics, &clothing, &books);
    total = electronics + clothing + books;                رقم الأعمال الكلي //
    pe = electronics / total * 100;                حصة الإلكترونيات //
    pc = clothing / total * 100;                حصة الملابس //
    pb = books / total * 100;                حصة الكتب //
    printf("Electronics: %.1f %%\n", pe);
    printf("Clothing:    %.1f %%\n", pc);
    printf("Books:       %.1f %%\n", pb);
    printf("Check sum:   %.1f %%\n", pe + pc + pb);
    return 0;
}
```

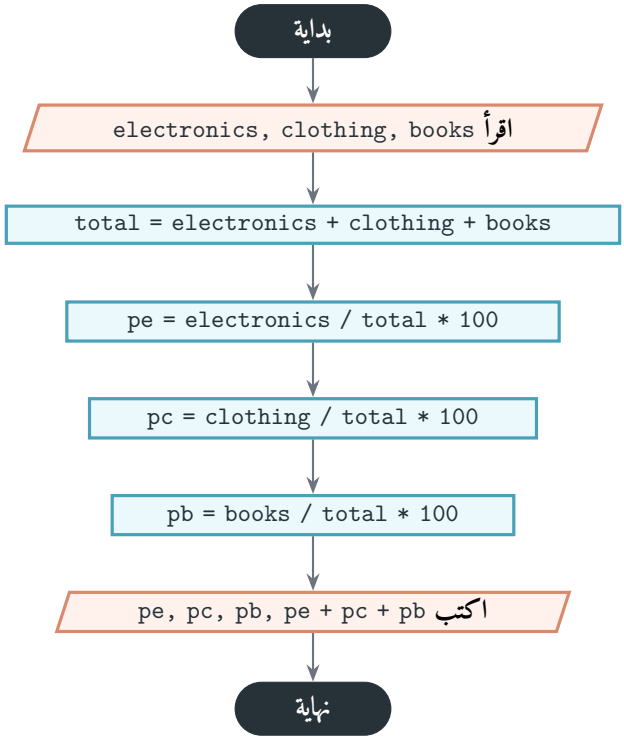
المدخلات

```
540000 310000 150000
```

نتيجة التنفيذ

```
Electronics: 54.0 %
Clothing:    31.0 %
Books:       15.0 %
Check sum:   100.0 %
```

المخطط الإنسيابي (Organigramme)



تمرين 1.31 الطابع الزمني إلى ساعة

علم البيانات ★★ متوسط

تسجل قاعدة البيانات وقت كل نقرة بعدد الثواني منذ منتصف الليل. اكتب برنامجًا يحول هذا العدد إلى الصيغة `hh:mm:ss`.

خطوات الحل

1) الساعات: `hours = timestamp / 3600`.
2) الدقائق: `minutes = (timestamp % 3600) / 60`.
3) الثواني: `seconds = timestamp % 60`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int timestamp, hours, minutes, seconds;
    scanf("%d", &timestamp);              // ثوانٍ منذ منتصف الليل
    hours = timestamp / 3600;                  // 3600 ثانية في الساعة
    minutes = (timestamp % 3600) / 60;           // ما تبقى من الساعة بالدقائق
    seconds = timestamp % 60;                    // الثواني المتبقية
    printf("%d s -> %02d:%02d:%02d\n", timestamp, hours, minutes, seconds);
    return 0;
}
```

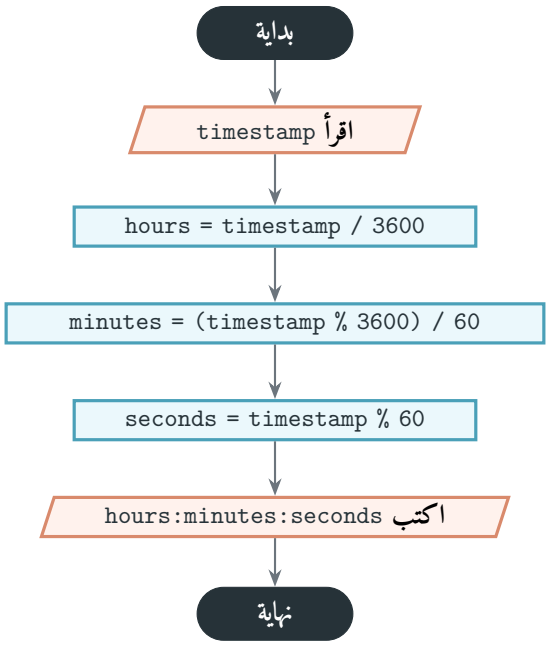
المدخلات

```
53917
```

نتيجة التنفيذ

```
53917 s -> 14:58:37
```

المخطط الإنسيابي (Organigramme)



تمرين 1.32 فاتورة كاملة — تجارة إلكترونية — ★★ متوسط

تحتوي سلة على ثلاثة منتجات (سعر وكمية لكل منها). يضاف الرسم 19% ثم مصاريف شحن ثابتة 600 دج. اكتب برنامجًا يعرض المجموع دون رسم، الرسم، والمبلغ النهائي.

خطوات الحل

1) نقرأ السعر والكمية لكل منتج.
2) `ht = p1*q1 + p2*q2 + p3*q3`.
3) `tax = ht * TVA` ثم `total = ht + tax + SHIP`.

البرنامج بلغة C

```c
#include <stdio.h>

#define TVA  0.19                 // نسبة الرسم
#define SHIP 600.0                // مصاريف الشحن الثابتة

int main(void) {
    double p1, p2, p3, ht, tax, total;
    int q1, q2, q3;
    scanf("%lf %d", &p1, &q1);    // المنتج الأول
    scanf("%lf %d", &p2, &q2);    // المنتج الثاني
    scanf("%lf %d", &p3, &q3);    // المنتج الثالث
    ht = p1 * q1 + p2 * q2 + p3 * q3;   // مجموع الأسطر
    tax = ht * TVA;
    total = ht + tax + SHIP;
    printf("Subtotal HT = %10.2f\n", ht);
    printf("Tax 19%%     = %10.2f\n", tax);
    printf("Shipping    = %10.2f\n", SHIP);
    printf("TOTAL       = %10.2f DA\n", total);
    return 0;
}
```

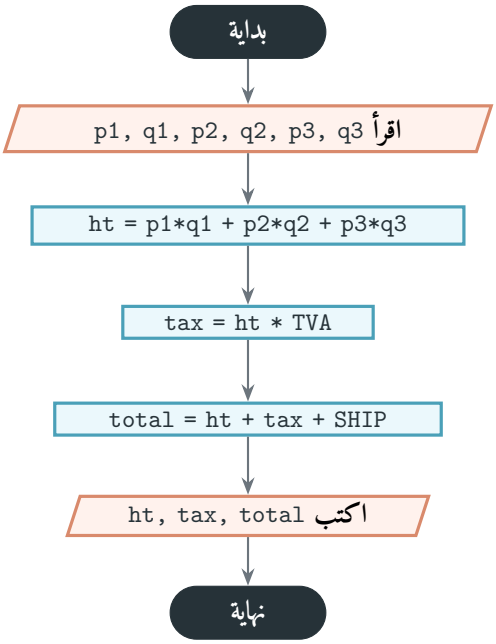
المدخلات

```
1200 2
450 4
3999 1
```

نتيجة التنفيذ

```
Subtotal HT =    8199.00
Tax 19%     =    1557.81
Shipping    =     600.00
TOTAL       =   10356.81 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.33 مقارنة سعر الكيلوغرام

تجارة إلكترونية ★★ متوسط

يعرض المتجر نفس القهوة في عبوتين: 250 غ بسعر p1 و1 كغ بسعر p2. اكتب برنامجًا يحسب سعر الكيلوغرام لكل عبوة والفرق بينهما والتوفير بالمئة عند شراء العبوة الكبيرة.

خطوات الحل

1) سعر الكيلوغرام: price / grams * 1000.
2) الفرق: k1 - k2.
3) التوفير بالمئة: (k1 - k2) / k1 * 100.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double g1, p1, g2, p2, k1, k2, save;
    scanf("%lf %lf", &g1, &p1);   // الوزن بالغرام والسعر للعبوة 1
    scanf("%lf %lf", &g2, &p2);   // العبوة 2
    k1 = p1 / g1 * 1000;          // سعر الكيلوغرام للعبوة 1
    k2 = p2 / g2 * 1000;          // سعر الكيلوغرام للعبوة 2
    save = (k1 - k2) / k1 * 100;  // التوفير النسبي
    printf("Pack 1: %.2f DA/kg\n", k1);
    printf("Pack 2: %.2f DA/kg\n", k2);
    printf("Difference: %.2f DA/kg (%.1f %% saved)\n", k1 - k2, save);
    return 0;
}
```

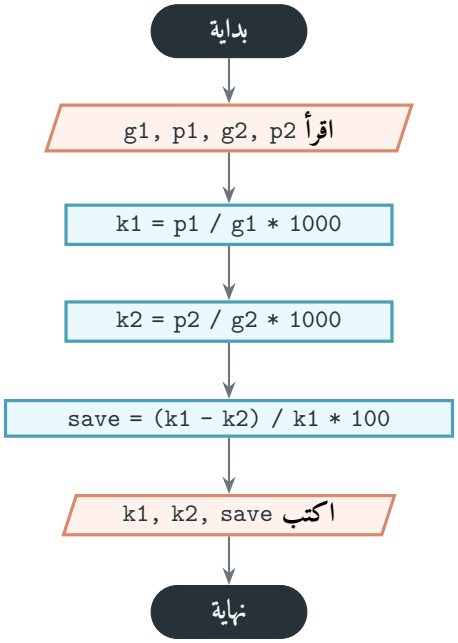
المدخلات

```
250 380
1000 1350
```

نتيجة التنفيذ

```
Pack 1: 1520.00 DA/kg
Pack 2: 1350.00 DA/kg
Difference: 170.00 DA/kg (11.2 % saved)
```

المخطط الإنسيابي (Organigramme)



تمرين 1.34 نقاط الوفاء

تجارة إلكترونية ★★ متوسط

يحصل الزبون على **نقطة وفاء** عن كل 100 دج مشتريات كاملة، وكل 50 نقطة تساوي قسيمة بـ 500 دج. اكتب برنامجًا يحسب عدد النقاط والمبلغ الناقص للنقطة التالية وعدد القسائم.

خطوات الحل

1) النقاط: `pts = amount / 100`.
2) الناقص للنقطة التالية: `100 - amount % 100`.
3) القسائم: `pts / 50` وقيمتها `500` ×.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int amount, pts, missing, vouchers;
    scanf("%d", &amount);
    pts = amount / 100;              // نقطة لكل 100 دج كاملة
    missing = 100 - amount % 100;    // ما ينقص للوصول إلى المئة التالية
    vouchers = pts / 50;             // قسيمة لكل 50 نقطة
    printf("Points = %d\n", pts);
```

```
    printf("Next point in %d DA\n", missing);
    printf("Vouchers = %d (%d DA)\n", vouchers, vouchers * 500);
    return 0;
}
```

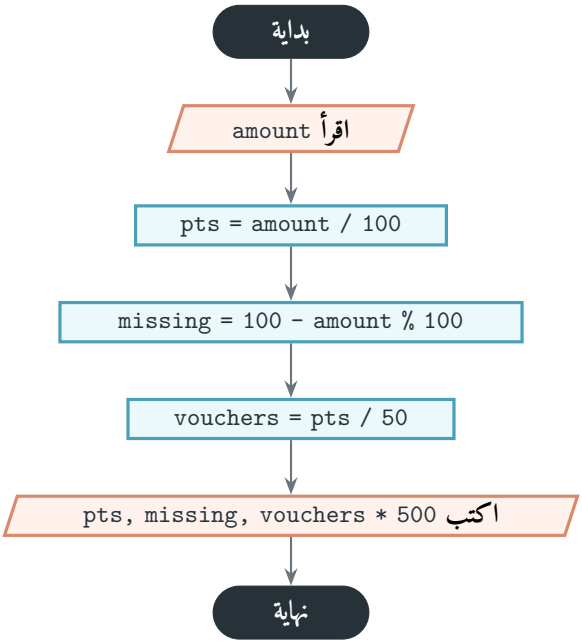
المدخلات

```
12370
```

نتيجة التنفيذ

```
Points = 123
Next point in 30 DA
Vouchers = 2 (1000 DA)
```

المخطط الإنسيابي (Organigramme)



تمرين 1.35 متوسط مربع الخطأ

علم البيانات ★★ متوسط

تنبأ نموذج بمبيعات ثلاثة أيام. اكتب برنامجًا يقرأ لكل يوم القيمة الحقيقية y والمتوقعة p، ويحسب الأخطاء ومتوسط مربع الخطأ MSE.

خطوات الحل

1) الخطأ لكل يوم: e = y - p.
2) نجمع مربعات الأخطاء.
3) MSE = / 3.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double y1, p1, y2, p2, y3, p3, e1, e2, e3, mse;
```

```
    scanf("%lf %lf", &y1, &p1);
    scanf("%lf %lf", &y2, &p2);
    scanf("%lf %lf", &y3, &p3);
    e1 = y1 - p1;                  // خطأ اليوم 1
    e2 = y2 - p2;                  // خطأ اليوم 2
    e3 = y3 - p3;                  // خطأ اليوم 3
    // التربيع يجعل كل الأخطاء موجبة ويعاقب الأخطاء الكبيرة
    mse = (e1 * e1 + e2 * e2 + e3 * e3) / 3;
    printf("Errors: %.1f %.1f %.1f\n", e1, e2, e3);
    printf("MSE = %.2f\n", mse);
    return 0;
}
```

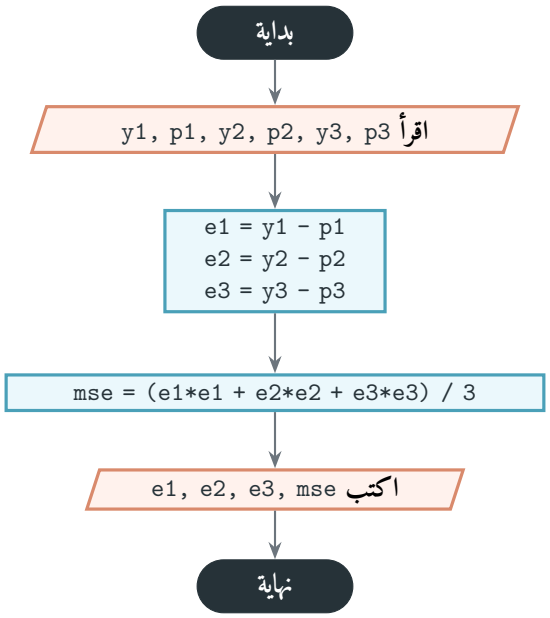
المدخلات

```
100 92
80 85
120 111
```

نتيجة التنفيذ

```
Errors: 8.0 -5.0 9.0
MSE = 56.67
```

المخطط الإنسيابي (Organigramme)



تمرين 1.36 السعر بعد التضخم

علم البيانات ★★ متوسط

سعر منتج اليوم `price`، ونسبة التضخم السنوية `inflation` بالمئة. اكتب برنامجًا يحسب سعره المتوقع بعد سنتين، وكذلك **القيمة الحقيقية** لمبلغ 10000 دج بعد سنتين (أي قسمته على معامل التضخم مرتين).

خطوات الحل

1) المعامل: `factor = 1 + inflation / 100`.
2) السعر بعد سنتين: `price * factor * factor`.
3) القيمة الحقيقية: `10000 / (factor * factor)`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, inflation, factor, p2, real;
    scanf("%lf %lf", &price, &inflation);
    factor = 1 + inflation / 100;              // معامل التضخم السنوي
    p2 = price * factor * factor;              // سنتان = ضرب مرتين
    real = 10000 / (factor * factor);          // القدرة الشرائية لـ 10000 دج
    printf("Price in 2 years = %.2f DA\n", p2);
    printf("Real value of 10000 DA = %.2f DA\n", real);
    return 0;
}
```

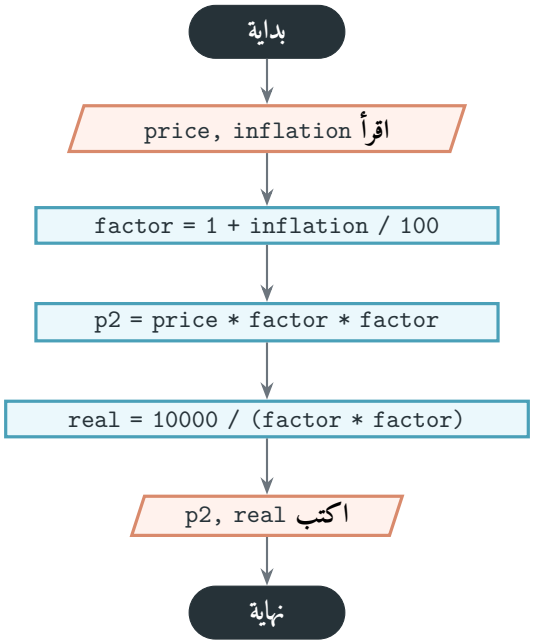
المدخلات

```
4500 9
```

نتيجة التنفيذ

```
Price in 2 years = 5346.45 DA
Real value of 10000 DA = 8416.80 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.37 تبديل دون متغير مساعد

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يبدل كميتي مخزون في مستودعين `stockA` و`stockB` **دون استعمال متغير ثالث**، باستعمال الجمع والطرح فقط.

خطوات الحل

1) `stockA = stockA + stockB` (stockA تحتوي المجموع).
2) `stockB = stockA - stockB` (stockB تأخذ القيمة القديمة لـ stockA).
3) `stockA = stockA - stockB` (stockA تأخذ القيمة القديمة لـ stockB).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int stockA, stockB;
    scanf("%d %d", &stockA, &stockB);
    printf("Before: a = %d, b = %d\n", stockA, stockB);
    stockA = stockA + stockB;        // stockA = a0 + b0
    stockB = stockA - stockB;        // stockB = a0 + b0 - b0 = a0
    stockA = stockA - stockB;        // stockA = a0 + b0 - a0 = b0
    printf("After:  a = %d, b = %d\n", stockA, stockB);
    return 0;
}
```

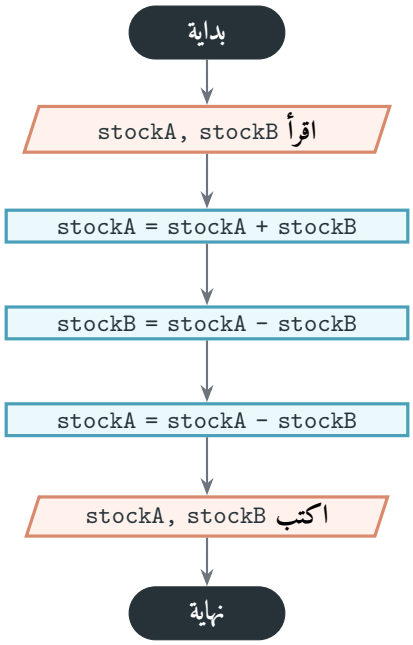
المدخلات

```
75 320
```

نتيجة التنفيذ

```
Before: a = 75, b = 320
After:  a = 320, b = 75
```

المخطط الإنسيابي (Organigramme)



تمرين 1.38 المبلغ بالسنتيم تجارة إلكترونية متوسط

لتجنب أخطاء الأعداد الحقيقية، يخزن نظام الدفع المبالغ بالسنتيم كأعداد صحيحة. اكتب برنامجًا يقرأ مبلغين بالسنتيم ويعرض مجموعهما بالصيغة DA.cc.

خطوات الحل

1) نجمع المبلغين كأعداد صحيحة: totalCents = centsA + centsB.
2) الدينار: totalCents / 100، السنتيم: totalCents % 100.
3) نعرض السنتيم برقمين %02d.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int centsA, centsB, totalCents;
    scanf("%d %d", &centsA, &centsB);        // مبلغان بالسنتيم
    totalCents = centsA + centsB;                    // جمع دقيق بلا أخطاء تقريب
    // 02d% تضمن ظهور الصفر: 5 سنتيم تكتب 05
    printf("Total = %d.%02d DA\n", totalCents / 100, totalCents % 100);
    return 0;
}
```

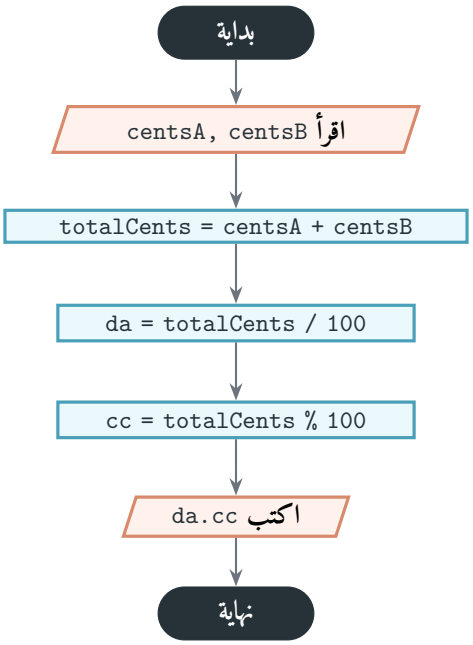
المدخلات

```
159990 45025
```

نتيجة التنفيذ

```
Total = 2050.15 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.39 رموز الأصناف والحروف

تجارة إلكترونية ⭐⭐ متوسط

يرمز لكل صنف بحرف كبير (A، B، ...). اكتب برنامجًا يقرأ رمز صنف ويعرض شفرته ASCII، والحرف التالي، وترتيب الصنف (A هو 1)، والحرف الصغير الموافق دون دالة `tolower`.

خطوات الحل

1) الحرف في C عدد صحيح، فنعرضه بـ `%d` لنحصل على شفرته.
2) التالي: `category + 1`، الترتيب: `category - 'A' + 1`.
3) الحرف الصغير: `category + ('a' - 'A')` لأن الفرق ثابت (32).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char category, next, low;
    int rank;
    scanf(" %c", &category);
    next = category + 1;                // الحرف الموالي في جدول ASCII
    rank = category - 'A' + 1;          // المسافة عن A
    low = category + ('a' - 'A');       // الفرق بين الصغير والكبير ثابت
    printf("Code ASCII = %d\n", category);
    printf("Next category = %c\n", next);
    printf("Rank = %d\n", rank);
    printf("Lowercase = %c\n", low);
    return 0;
}
```

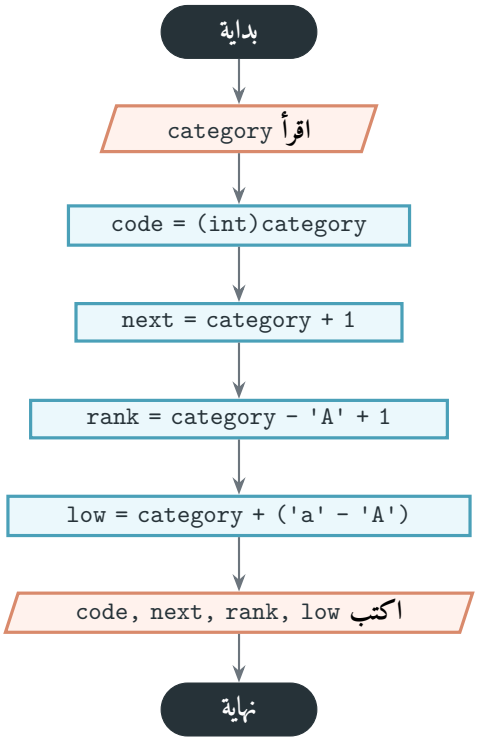
المدخلات

```
G
```

نتيجة التنفيذ

```
Code ASCII = 71
Next category = H
Rank = 7
Lowercase = g
```

المخطط الإنسيابي (Organigramme)



تمرين 1.40 أسبوع ويوم التسليم

تجارة إلكترونية ★★ متوسط

يُرقَّم كل يوم من السنة من 1 إلى 365، واليوم 1 يوم أحد. تُطلب سلعة في اليوم `orderDay` وتُسلَّم بعد `delay` يومًا. اكتب برنامجًا يعرض رقم يوم التسليم ورقم أسبوعه ورقم يومه في الأسبوع (0 = أحد).

خطوات الحل

1) يوم التسليم: `deliveryDay = orderDay + delay`.
2) الأسبوع: `(deliveryDay - 1) / 7 + 1`.
3) يوم الأسبوع: `(deliveryDay - 1) % 7`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int orderDay, delay, deliveryDay, week, wd;
    scanf("%d %d", &orderDay, &delay);
    deliveryDay = orderDay + delay;                 // رقم يوم التسليم في السنة
    week = (deliveryDay - 1) / 7 + 1;       // نطرح 1 لأن الترقيم يبدأ من 1
    wd = (deliveryDay - 1) % 7;             // 0 أحد، 1 إثنين، ...
    printf("Delivery day = %d\n", deliveryDay);
    printf("Week = %d, weekday = %d\n", week, wd);
    return 0;
}
```

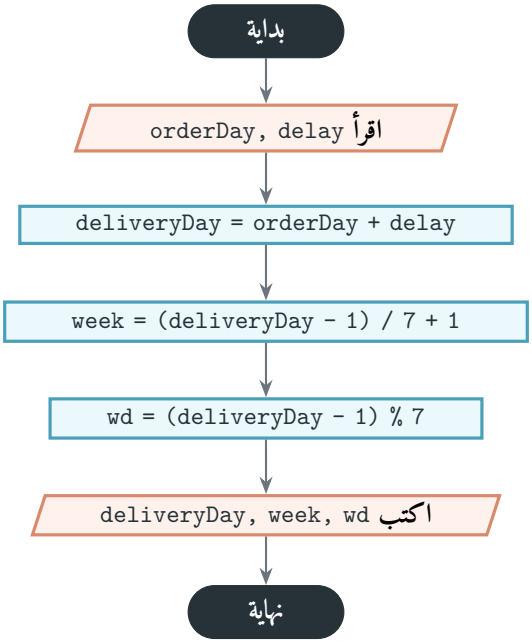
المدخلات

```
58 17
```

نتيجة التنفيذ

```
Delivery day = 75
Week = 11, weekday = 4
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 1.41 الأكبر بين سعرين دون شرط

تجارة إلكترونية ★★★ صعب

اكتب برنامجًا يحدد أكبر وأصغر سعرين صحيحين دون استعمال `if` ولا العامل الشرطي. استعمل الصيغة `max = (priceA + priceB + |priceA - priceB|) / 2` واحسب القيمة المطلقة بعمليات على البتات.

خطوات الحل

1) `diff = priceA - priceB`.
2) قناع الإشارة: `mask = diff >> 31` يساوي 0 إذا كان `diff` موجبًا و-1 إذا كان سالبًا.
3) القيمة المطلقة: `ad = (diff ^ mask) - mask`.
4) `max = (priceA + priceB + ad) / 2` و`min = (priceA + priceB - ad) / 2`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int priceA, priceB, diff, mask, ad, max, min;
    scanf("%d %d", &priceA, &priceB);
    diff = priceA - priceB;
    mask = diff >> 31;              // كل البتات 1 إذا كان diff سالبًا، وإلا 0
    ad = (diff ^ mask) - mask;      // إذا كان سالبًا: عكس البتات ثم +1 = -diff
    max = (priceA + priceB + ad) / 2;       // الأكبر
    min = (priceA + priceB - ad) / 2;       // الأصغر
    printf("|a - b| = %d\n", ad);
    printf("Max = %d, Min = %d\n", max, min);
    return 0;
}
```

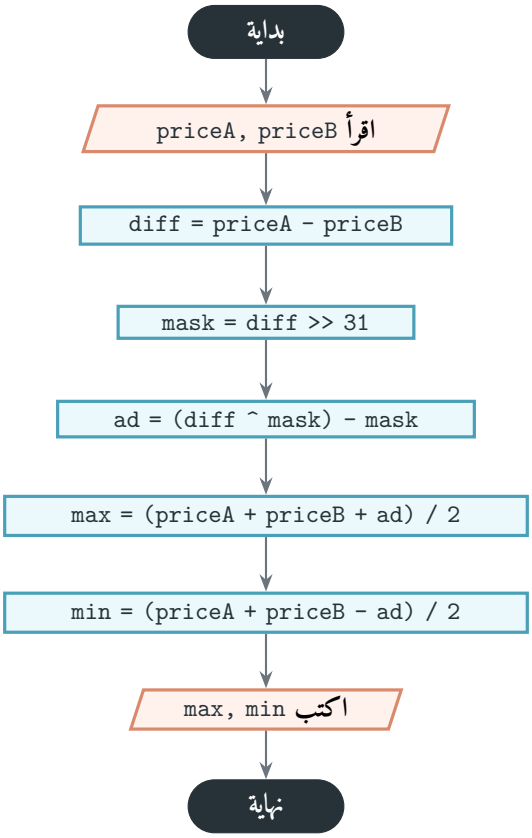
المدخلات

```
1750 2380
```

نتيجة التنفيذ

```
|a - b| = 630
Max = 2380, Min = 1750
```

المخطط الإنسيابي (Organigramme)



تمرين 1.42 الإنحراف المعياري بطريقة نيوتن

علم البيانات ★★★ صعب

لدينا تقييمات ثلاثة منتجات. اكتب برنامجًا يحسب **التباين** ثم **الانحراف المعياري** دون دالة `sqrt`، باستعمال 6 تكرارات مكتوبة يدويًا لطريقة نيوتن: `root = (root + variance / root) / 2`.

خطوات الحل

1) نحسب المتوسط `mean` ثم التباين `variance`.
2) نبدأ بتقدير أولي `root = variance` (أو 1 إذا كان صغيرًا).
3) نطبق صيغة نيوتن ست مرات، فيقترب `root` بسرعة من الجذر.
4) نتحقق بحساب `root * root`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double rating1, rating2, rating3, mean, variance, root;
    scanf("%lf %lf %lf", &rating1, &rating2, &rating3);
    mean = (rating1 + rating2 + rating3) / 3;                          // المتوسط
    variance = ((rating1 - mean) * (rating1 - mean) + (rating2 - mean) * (rating2 - mean)
       + (rating3 - mean) * (rating3 - mean)) / 3;                     // التباين
    root = variance + 1;                                               // تقدير أولي أكبر من الجذر
    // كل تكرار يضاعف تقريبًا عدد الأرقام الصحيحة
```

```
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    printf("Variance = %.6f\n", variance);
    printf("Std dev  = %.6f\n", root);
    printf("Check r*r = %.6f\n", root * root);      // يجب أن يساوي التباين
    return 0;
}
```

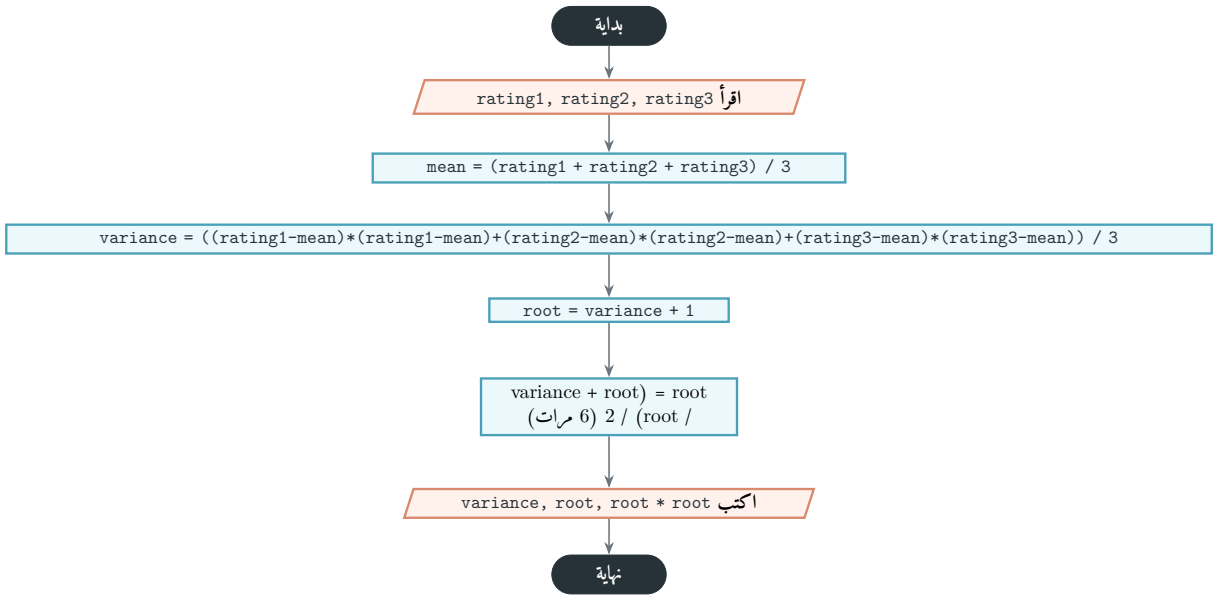
المدخلات

```
3.5 4.8 4.1
```

نتيجة التنفيذ

```
Variance = 0.282222
Std dev  = 0.531246
Check r*r = 0.282222
```

المخطط الإنسيابي (Organigramme)



تمرين 1.43 التغاير بين السعر والمبيعات

علم البيانات ★★★ صعب

لدينا ثلاث ملاحظات (سعر، مبيعات). اكتب برنامجًا يحسب التغاير cov بين المتغيرين ويستنتج اتجاه العلاقة بطباعة إشارة التغاير، دون شرط: نطبع cov > 0 كعدد (1 أو 0).

خطوات الحل

1) نحسب المتوسطين mx وmy.
2) التغاير: متوسط الجداءات (xi - mx) * (yi - my).
3) في C يعطي التعبير المنطقي cov > 0 القيمة 1 أو 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, x3, y3, mx, my, cov;
    int pos;
    scanf("%lf %lf %lf %lf %lf %lf", &x1, &y1, &x2, &y2, &x3, &y3);
    mx = (x1 + x2 + x3) / 3;       // متوسط الأسعار
    my = (y1 + y2 + y3) / 3;       // متوسط المبيعات
    // التغاير: هل يتحرك المتغيران في نفس الاتجاه؟
    cov = ((x1 - mx) * (y1 - my) + (x2 - mx) * (y2 - my)
         + (x3 - mx) * (y3 - my)) / 3;
    pos = cov > 0;                 // تعبير منطقي: 1 صحيح، 0 خطأ
    printf("Mean price = %.1f, mean sales = %.1f\n", mx, my);
    printf("Covariance = %.2f\n", cov);
    printf("Positive relation? %d\n", pos);
    return 0;
}
```

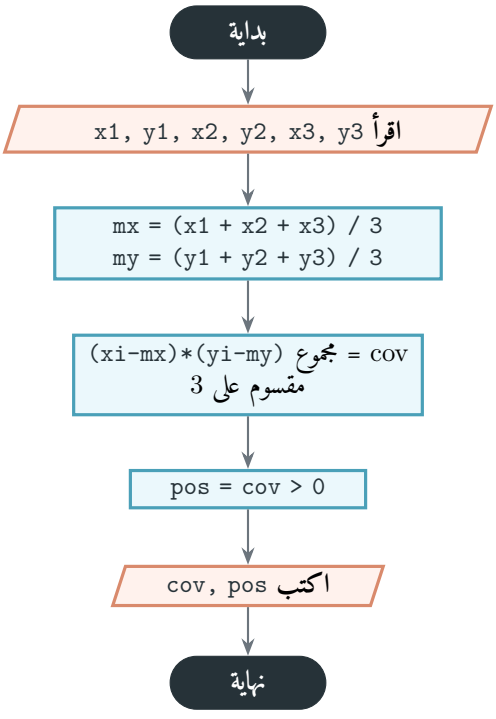
المدخلات

```
100 520
150 410
200 260
```

نتيجة التنفيذ

```
Mean price = 150.0, mean sales = 396.7
Covariance = -4333.33
Positive relation? 0
```

المخطط الإنسيابي (Organigramme)



تمرين 1.44 انحدار خطي بالمربعات الصغرى

علم البيانات صعب ★★★

اكتب برنامجًا يحسب معاملي مستقيم الانحدار `y = slope*x + intercept` بطريقة المربعات الصغرى لثلاث نقاط (ميزانية إعلان، مبيعات): `slope = (nbPoints*Sxy - Sx*Sy) / (nbPoints*Sxx - Sx*Sx)` و`intercept = (Sy - slope*Sx) / nbPoints`.

خطوات الحل

1) نحسب المجاميع `Sx` و`Sy` و`Sxy` و`Sxx`.
2) نطبق صيغة الميل `slope` ثم الجزء المقطوع `intercept`.
3) نتحقق بالتنبؤ عند `x = 3`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, x3, y3, Sx, Sy, Sxy, Sxx, slope, intercept;
    const int nbPoints = 3;                     // عدد النقاط
    scanf("%lf %lf %lf %lf %lf %lf", &x1, &y1, &x2, &y2, &x3, &y3);
    Sx = x1 + x2 + x3;                          // مجموع x
    Sy = y1 + y2 + y3;                          // مجموع y
    Sxy = x1 * y1 + x2 * y2 + x3 * y3;          // مجموع الجداءات
    Sxx = x1 * x1 + x2 * x2 + x3 * x3;          // مجموع المربعات
    slope = (nbPoints * Sxy - Sx * Sy) / (nbPoints * Sxx - Sx * Sx);   // الميل
    intercept = (Sy - slope * Sx) / nbPoints;                  // الجزء المقطوع
    printf("y = %.4f x + %.4f\n", slope, intercept);
    printf("Prediction at x = 3: %.4f\n", slope * 3 + intercept);
    return 0;
}
```

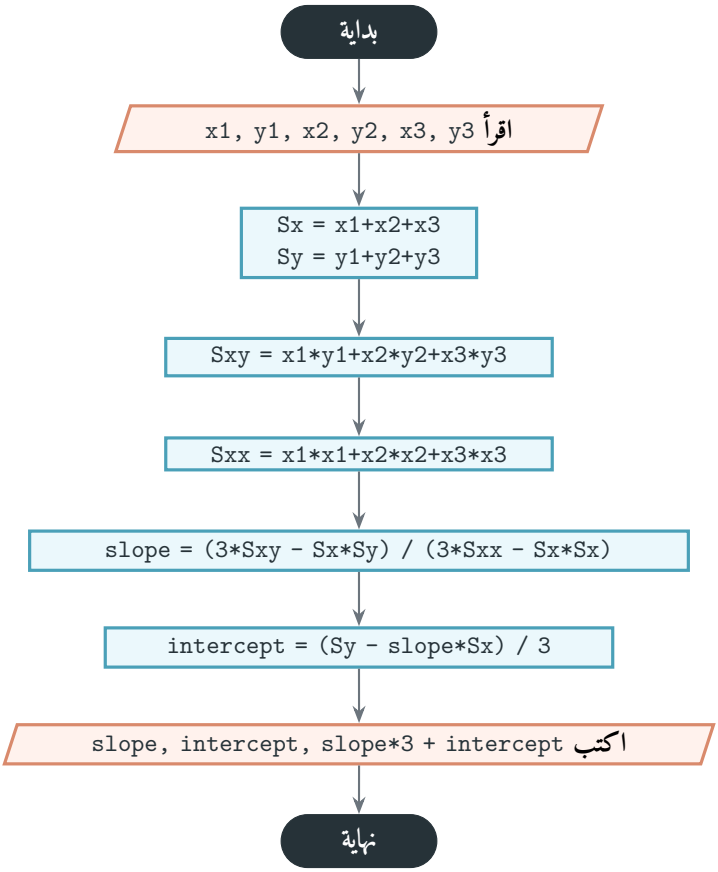
المدخلات

```
1 3
2 5
4 9
```

نتيجة التنفيذ

```
y = 2.0000 x + 1.0000
Prediction at x = 3: 7.0000
```

المخطط الإنسيابي (Organigramme)



تمرين 1.45 مدة المشاهدة الكلية

علم البيانات صعب

تجمع منصة فيديوهات المنتجات مدة المشاهدة الكلية بالثواني (عدد كبير). اكتب برنامجًا يحولها إلى **أيام وساعات ودقائق وثوانٍ**، ويحسب متوسط المشاهدة لكل زائر بالدقائق والثواني. استعمل النوع `long`.

خطوات الحل

1) الأيام: `totalSec / 86400`، ثم `totalSec % 86400`.
2) الساعات من الباقي: `/ 3600`، ثم الدقائق `/ 60`، ثم الثواني `% 60`.
3) متوسط الزائر: `avg = totalSec / visitors` ثم نفككه إلى دقائق وثوانٍ.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long totalSec, rest, days, hours, minutes, seconds, visitors, avg;
    scanf("%ld %ld", &totalSec, &visitors);      // الثواني الكلية، عدد الزوار
    days = totalSec / 86400;  rest = totalSec % 86400; // 86400 ثانية في اليوم
    hours = rest / 3600;    rest = rest % 3600;  // الساعات من الباقي
    minutes = rest / 60;      seconds = rest % 60;     // الدقائق والثواني
    avg = totalSec / visitors;                   // متوسط الزائر بالثواني
```

```
    printf("Total: %ld d %ld h %ld min %ld s\n", days, hours, minutes, seconds);
    printf("Per visitor: %ld min %ld s\n", avg / 60, avg % 60);
    return 0;
}
```

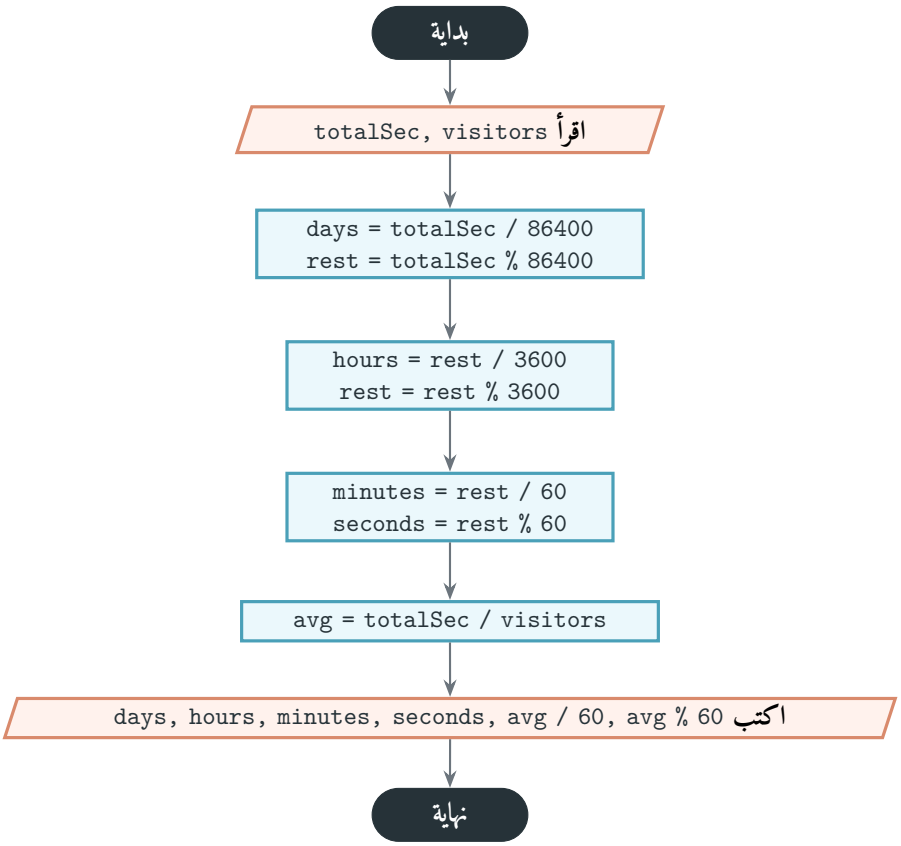
المدخلات

```
987654 1520
```

نتيجة التنفيذ

```
Total: 11 d 10 h 20 min 54 s
Per visitor: 10 min 49 s
```

المخطط الإنسيابي (Organigramme)



تمرين 1.46 خصائص المنتج بالبتات

تجارة إلكترونية صعب

تخزن خصائص منتج في عدد صحيح بين 0 و15، كل بت يمثل خاصية: البت 0 = متوفر، البت 1 = مخفض، البت 2 = شحن مجاني، البت 3 = جديد. اكتب برنامجًا يستخرج قيمة كل خاصية (0 أو 1) ويعرض التمثيل الثنائي.

خطوات الحل

1) البت `k` هو `(flags / 2^k) % 2`، أو `(flags >> k) & 1`.
2) نستخرج البتات الأربعة.
3) التمثيل الثنائي يُكتب من البت 3 إلى البت 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int flags, b0, b1, b2, b3;
    scanf("%d", &flags);
    b0 = flags & 1;              // البت 0: متوفر
    b1 = (flags >> 1) & 1;       // إزاحة لليمين ثم نأخذ آخر بت
    b2 = (flags >> 2) & 1;       // البت 2: شحن مجاني
    b3 = (flags >> 3) & 1;       // البت 3: جديد
    printf("Binary: %d%d%d%d\n", b3, b2, b1, b0);
    printf("In stock: %d\nOn sale: %d\n", b0, b1);
    printf("Free shipping: %d\nNew: %d\n", b2, b3);
    return 0;
}
```

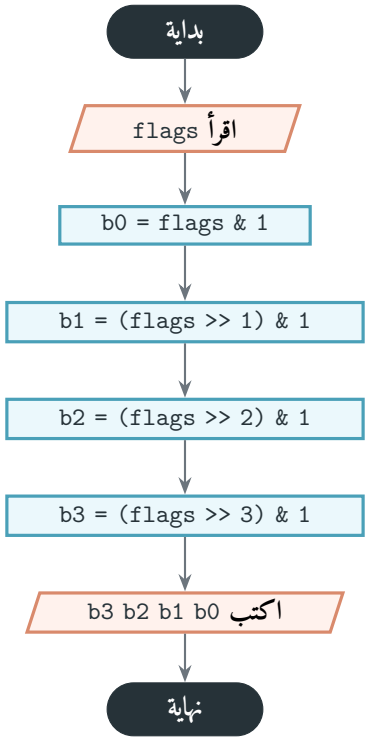
المدخلات

```
13
```

نتيجة التنفيذ

```
Binary: 1101
In stock: 1
On sale: 0
Free shipping: 1
New: 1
```

المخطط الإنسيابي (Organigramme)



تمرين 1.47 المتوسط المتحرك الأسي علم البيانات صعب

لتنعيم سلسلة أسعار نستعمل **المتوسط المتحرك الأسي**: `ema = alpha * p + (1 - alpha) * ema`، مع `ema` يبدأ بالسعر الأول. اكتب برنامجًا يطبقه على أربعة أسعار ويعرض القيمة بعد كل سعر.

خطوات الحل

1) نقرأ `alpha` ثم الأسعار الأربعة.
2) `ema = p1`.
3) نحدّث `ema` بكل سعر جديد بالصيغة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double alpha, p1, p2, p3, p4, ema;
    scanf("%lf", &alpha);                 // معامل التنعيم بين 0 و1
    scanf("%lf %lf %lf %lf", &p1, &p2, &p3, &p4);
    ema = p1;                             // البداية: أول سعر
    printf("t1: %.2f\n", ema);
    ema = alpha * p2 + (1 - alpha) * ema;     // وزن alpha للسعر الجديد
    printf("t2: %.2f\n", ema);
    ema = alpha * p3 + (1 - alpha) * ema;
    printf("t3: %.2f\n", ema);
    ema = alpha * p4 + (1 - alpha) * ema;
    printf("t4: %.2f\n", ema);
    return 0;
}
```

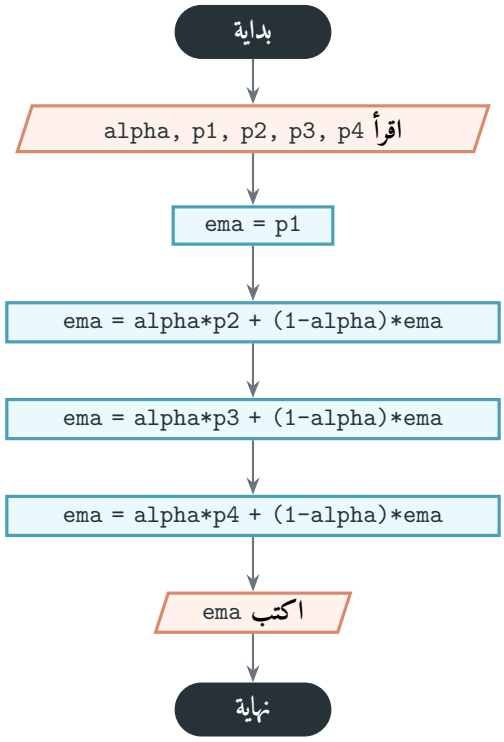
المدخلات

```
0.5
100 120 90 130
```

نتيجة التنفيذ

```
t1: 100.00
t2: 110.00
t3: 100.00
t4: 115.00
```

المخطط الإنسيابي (Organigramme)



تمرين 1.48 رقم التحقق لرمز EAN-8

تجارة إلكترونية صعب

رمز الأعمدة EAN-8 يتكون من 7 أرقام ورقم تحقق. نضرب الأرقام في المواضع الفردية (1، 3، 5، 7) في 3 ونجمعها مع أرقام المواضع الزوجية، ورقم التحقق `(10 - weightedSum % 10) % 10`. اكتب برنامجًا يحسبه لعدد من 7 أرقام.

خطوات الحل

1) نستخرج الأرقام السبعة من اليسار إلى اليمين بالقسمة على قوى 10.
2) `weightedSum = 3*(d1 + d3 + d5 + d7) + (d2 + d4 + d6)`.
3) `check = (10 - weightedSum % 10) % 10`، و% 10 الأخيرة تحول 10 إلى 0.
4) الرمز الكامل: `code * 10 + check`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long code;
    int d1, d2, d3, d4, d5, d6, d7, weightedSum, check;
    scanf("%ld", &code);
    d1 = code / 1000000;              // الرقم الأول من اليسار
    d2 = (code / 100000) % 10;
    d3 = (code / 10000) % 10;
    d4 = (code / 1000) % 10;
    d5 = (code / 100) % 10;
    d6 = (code / 10) % 10;
```

```c
    d7 = code % 10;                    // الرقم السابع
    weightedSum = 3 * (d1 + d3 + d5 + d7) + (d2 + d4 + d6);    // المجموع الموزون
    check = (10 - weightedSum % 10) % 10;    // ما يكمل weightedSum إلى مضاعف 10
    printf("Weighted sum = %d\n", weightedSum);
    printf("Check digit = %d\n", check);
    printf("EAN-8 = %ld\n", code * 10 + check);
    return 0;
}
```

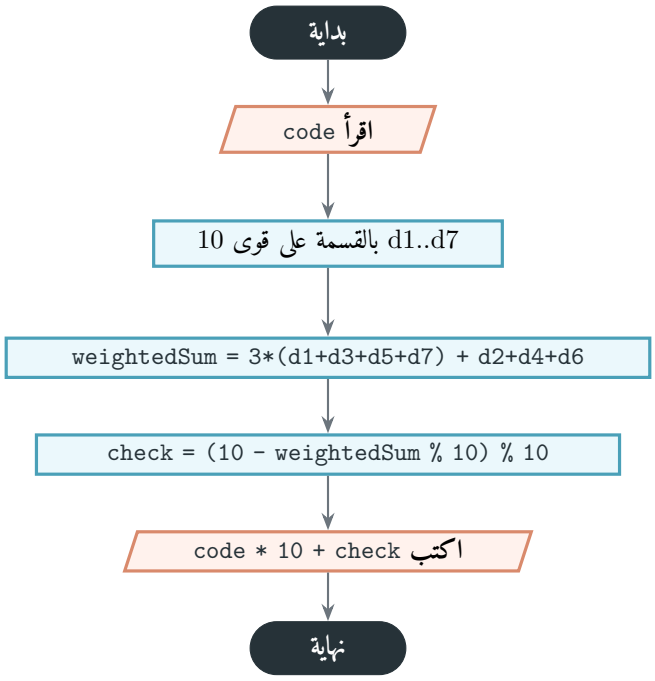
المدخلات

```
9638507
```

نتيجة التنفيذ

```
Weighted sum = 86
Check digit = 4
EAN-8 = 96385074
```

المخطط الإنسيابي (Organigramme)



تمرين 1.49 مرونة الطلب السعرية

علم البيانات ★★★ صعب

تقيس مرونة الطلب تأثر المبيعات بتغير السعر. بطريقة نقطة المنتصف: اكتب `elasticity = ((Q2 - Q1) / ((Q1 + Q2) / 2)) / ((P2 - P1) / ((P1 + P2) / 2))`. برنامجًا يحسبها، ويحسب رقم الأعمال قبل التغيير وبعده.

خطوات الحل

1) نقرأ `(P1, Q1)` و`(P2, Q2)`.
2) التغير النسبي للكمية: `dq = (Q2 - Q1) / ((Q1 + Q2) / 2)`.

3) التغير النسبي للسعر: dp بنفس الطريقة.

4) elasticity = dq / dp، ورقم الأعمال P * Q.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double P1, Q1, P2, Q2, dq, dp, elasticity;
    scanf("%lf %lf %lf %lf", &P1, &Q1, &P2, &Q2);
    dq = (Q2 - Q1) / ((Q1 + Q2) / 2);      // التغير النسبي للكمية
    dp = (P2 - P1) / ((P1 + P2) / 2);      // التغير النسبي للسعر
    elasticity = dq / dp;                   // المرونة
    printf("Elasticity = %.3f\n", elasticity);
    printf("Revenue before = %.0f DA\n", P1 * Q1);
    printf("Revenue after  = %.0f DA\n", P2 * Q2);
    return 0;
}
```

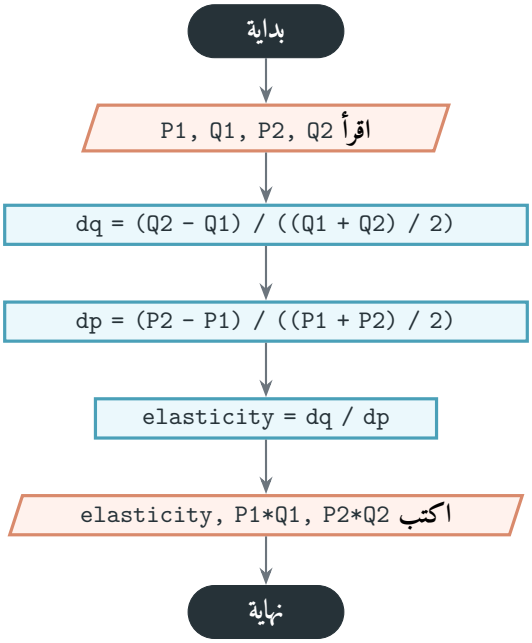
المدخلات

```
2000 500
2400 380
```

نتيجة التنفيذ

```
Elasticity = -1.500
Revenue before = 1000000 DA
Revenue after  = 912000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.50 تعبئة حاوية الشحن تجارة إلكترونية صعب ★★★

حاوية شحن أبعادها `contLength × contWidth × contHeight` (سم) ونريد ملأها بعلب أبعادها `boxLength × boxWidth × boxHeight` مرتبة في نفس الاتجاه. اكتب برنامجًا يحسب عدد العلب في كل بعد، والعدد الكلي، ونسبة الحجم المستغل.

خطوات الحل

1) عدد العلب في كل بعد: `contLength / boxLength` و`contWidth / boxWidth` و`contHeight / boxHeight` (قسمة صحيحة).
2) العدد الكلي: جداء الأعداد الثلاثة.
3) الحجم المستغل: `nbBoxes * boxLength * boxWidth * boxHeight`، والنسبة بالمئة من `contLength * contWidth * contHeight`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long contLength, contWidth, contHeight, boxLength, boxWidth, boxHeight, nx, ny, nz, nbBoxes, used;
    double pct;
    scanf("%ld %ld %ld", &contLength, &contWidth, &contHeight);      // أبعاد الحاوية
    scanf("%ld %ld %ld", &boxLength, &boxWidth, &boxHeight);      // أبعاد العلبة
    nx = contLength / boxLength;                    // علب على الطول
    ny = contWidth / boxWidth;                    // علب على العرض
    nz = contHeight / boxHeight;                    // طبقات على الارتفاع
    nbBoxes = nx * ny * nz;                // العدد الكلي
    used = nbBoxes * boxLength * boxWidth * boxHeight;          // الحجم المشغول بالعلب
    pct = (double)used / (contLength * contWidth * contHeight) * 100;
    printf("Grid: %ld x %ld x %ld\n", nx, ny, nz);
    printf("Boxes = %ld\n", nbBoxes);
    printf("Volume used = %.2f %%\n", pct);
    return 0;
}
```

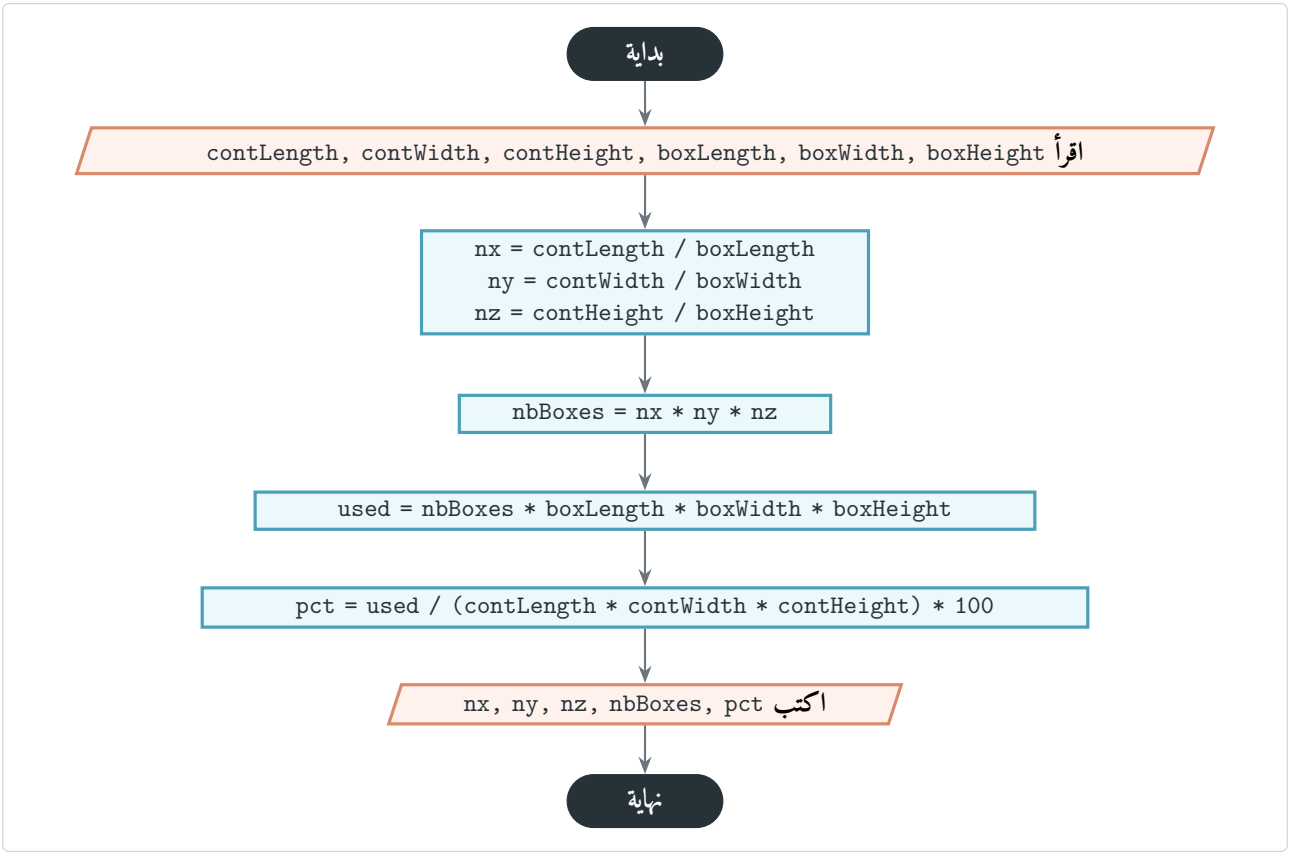
المدخلات

```
600 240 250
40 30 50
```

نتيجة التنفيذ

```
Grid: 15 x 8 x 5
Boxes = 600
Volume used = 100.00 %
```

المخطط الإنسيابي (Organigramme)

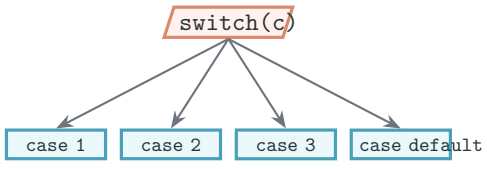


الفصل 2

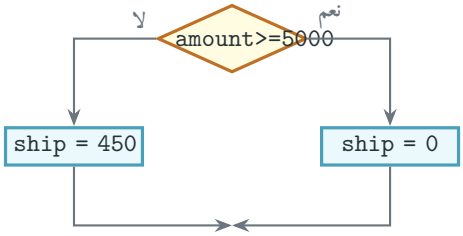
البنيات الشرطية

تعريفات

- **البنية الشرطية**: تنفذ مجموعة تعليمات **فقط إذا** تحقق شرط (if).
- **البديل**: else ينفذ عندما يكون الشرط خاطئًا، وelse if يسمح بسلسلة من الحالات.
- **الاختيار المتعدد**: switch يقارن متغيرًا صحيحًا أو حرفًا بقيم ثابتة (case) وbreak ينهي الحالة.
- **الشرط**: تعبير منطقي قيمته 1 (صحيح) أو 0 (خطأ)، يُبنى بـ == != < <= > >= ويُركب بـ ! || &&.



switch



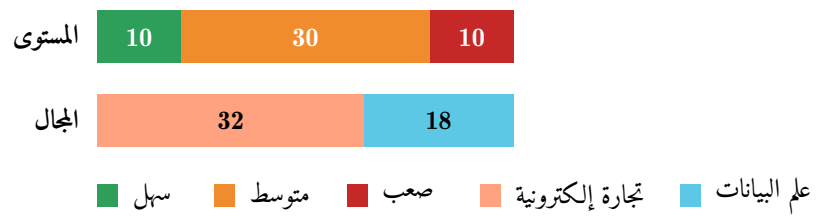
if ... else

A	B	A && B	A || B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

أخطاء شائعة

if (x = 5) إسناد وليس مقارنة (اكتب ==)، ونسيان break في switch يجعل التنفيذ ينزل إلى الحالة الموالية.

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 2.1 الشحن المجاني

سهل | تجارة إلكترونية

يقدم متجر **الشحن المجاني** للطلبيات التي يبلغ مبلغها 5000 دج أو أكثر، وإلا تضاف مصاريف شحن 450 دج. اكتب برنامجًا يقرأ مبلغ السلة ويعرض المبلغ النهائي.

خطوات الحل

1) نقرأ المبلغ `amount`.
2) إذا كان `amount >= 5000` فالشحن 0، وإلا 450.
3) نعرض الشحن والمبلغ النهائي.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, ship;
    scanf("%lf", &amount);
    if (amount >= 5000)
        ship = 0;                   // الشحن مجاني
    else
        ship = 450;                 // مصاريف الشحن العادية
    printf("Shipping = %.2f DA\n", ship);
    printf("To pay = %.2f DA\n", amount + ship);
    return 0;
}
```

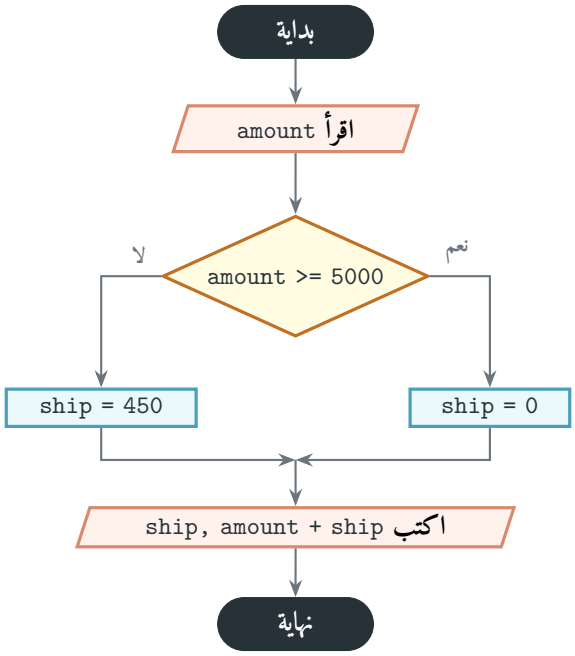
المدخلات

```
4200
```

نتيجة التنفيذ

```
Shipping = 450.00 DA
To pay = 4650.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.2 توفر المنتج في المخزون

سهل تجارة إلكترونية

يطلب زبون كمية `quantity` من منتج مخزونه `stock`. اكتب برنامجًا يعرض **"Order accepted"** إذا كان المخزون كافيًا مع المخزون المتبقي، وإلا يعرض الكمية الناقصة.

خطوات الحل

1) نقرأ المخزون `stock` والكمية المطلوبة `quantity`.
2) إذا كان `quantity <= stock`: نقبل الطلب ونعرض `stock - quantity`.
3) وإلا: نعرض النقص `quantity - stock`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int stock, quantity;
    scanf("%d %d", &stock, &quantity);        // المخزون والكمية المطلوبة
    if (quantity <= stock) {
        printf("Order accepted\n");
        printf("Remaining stock = %d\n", stock - quantity);
    } else {
        // المخزون غير كافٍ
        printf("Not enough stock: missing %d units\n", quantity - stock);
    }
    return 0;
}
```

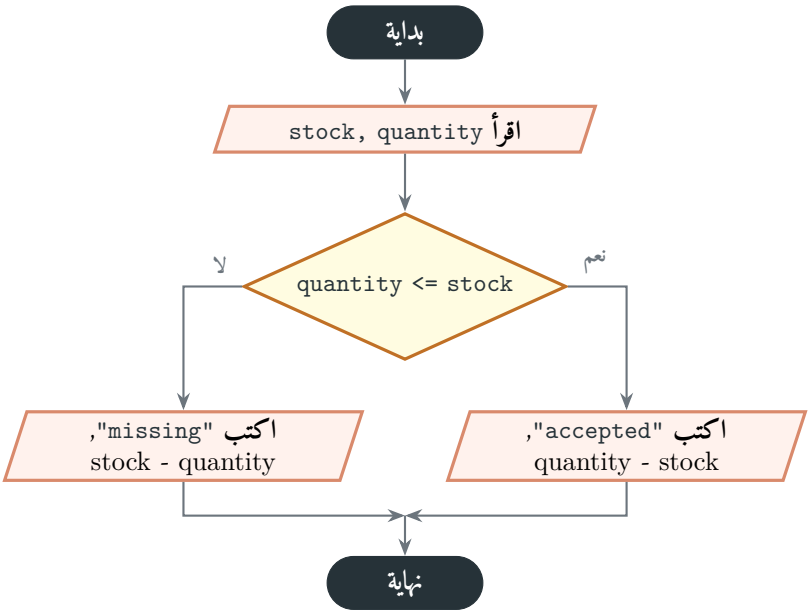
المدخلات

```
12 8
```

نتيجة التنفيذ

```
Order accepted
Remaining stock = 4
```

المخطط الإنسيابي (Organigramme)



تمرين 2.3 رقم طلبية زوجي أم فردي

تجارة إلكترونية
سهل

لتوزيع العمل بين مستودعين، تُرسل الطلبيات ذات الرقم **الزوجي** إلى المستودع A والفردية إلى المستودع B. اكتب برنامجًا يحدد المستودع.

خطوات الحل

1) نقرأ رقم الطلبية `orderId`.
2) العدد زوجي إذا كان `orderId % 2 == 0`.
3) نعرض اسم المستودع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int orderId;
    scanf("%d", &orderId);
    if (orderId % 2 == 0)                // باقي القسمة على 2 يساوي 0: زوجي
        printf("Order %d -> warehouse A\n", orderId);
    else
        printf("Order %d -> warehouse B\n", orderId);
    return 0;
}
```

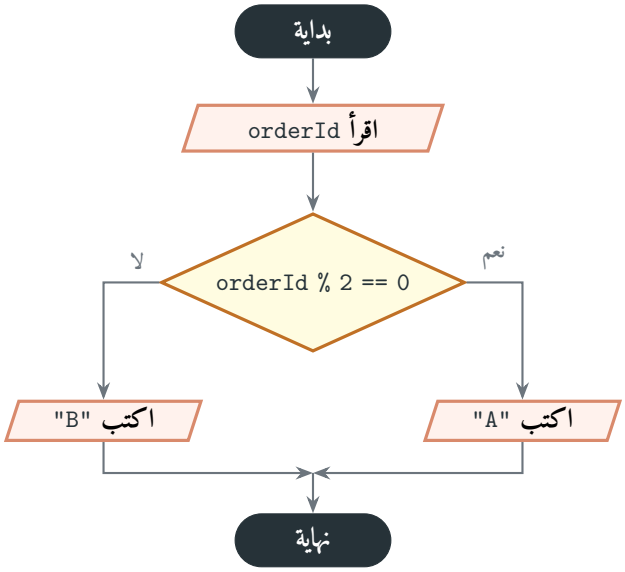
المدخلات

```
10457
```

نتيجة التنفيذ

```
Order 10457 -> warehouse B
```

المخطط الإنسيابي (Organigramme)



تمرين 2.4 الأرخص بين عرضين

تجارة إلكترونية
سهل

يعرض بائعان نفس المنتج بسعرين p1 وp2. اكتب برنامجًا يحدد البائع الأرخص والفرق بين السعرين، مع حالة التساوي.

خطوات الحل

1) نقرأ السعرين.
2) نقارن بـ if / else if / else لمعالجة الحالات الثلاث.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double p1, p2;
    scanf("%lf %lf", &p1, &p2);
    if (p1 < p2)
        printf("Seller 1 is cheaper by %.2f DA\n", p2 - p1);
    else if (p2 < p1)              // نختبر الحالة الثانية فقط إذا فشلت الأولى
        printf("Seller 2 is cheaper by %.2f DA\n", p1 - p2);
    else
        printf("Same price\n");    // لم يبق إلا التساوي
    return 0;
}
```

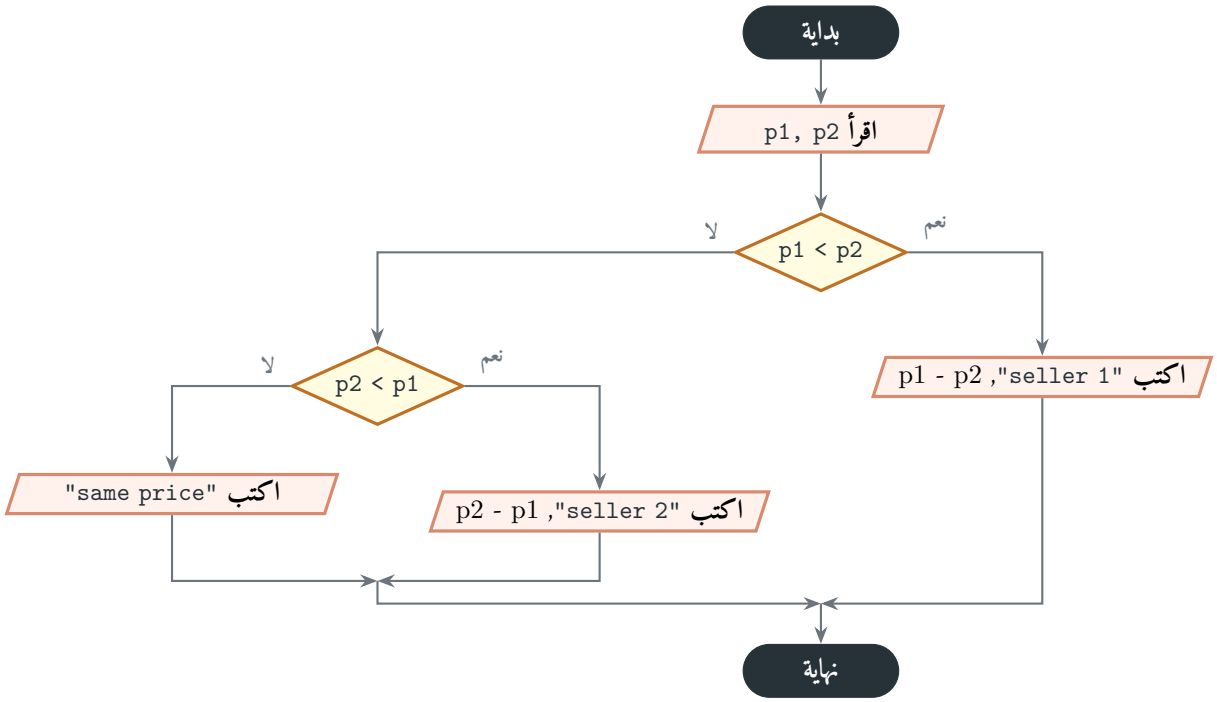
المدخلات

```
15400 14950
```

نتيجة التنفيذ

```
Seller 2 is cheaper by 450.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.5 التحقق من التقييم

علم البيانات
سهل

قبل حفظ تقييم في قاعدة البيانات، يجب التأكد أنه بين 1 و5. اكتب برنامجًا يقرأ تقييمًا ويعرض صالح أو غير صالح.

خطوات الحل

1) نقرأ التقييم `rating`.
2) الشرط المركب: `rating >= 1 && rating <= 5`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int rating;
    scanf("%d", &rating);
    // الشرطان معًا يجب أن يتحققا : &&
    if (rating >= 1 && rating <= 5)
        printf("Rating %d: valid\n", rating);
    else
        printf("Rating %d: invalid (must be 1..5)\n", rating);
    return 0;
}
```

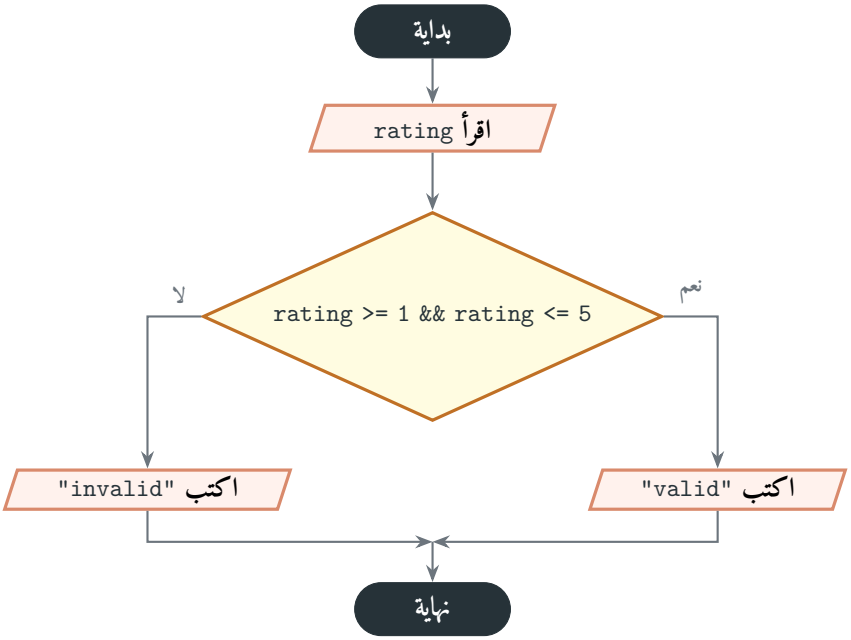
المدخلات

```
6
```

نتيجة التنفيذ

```
Rating 6: invalid (must be 1..5)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.6 ربح أم خسارة

تجارة إلكترونية سهل

اكتب برنامجًا يقرأ رقم الأعمال الشهري والتكاليف، ويعرض ربح أو خسارة أو توازن مع القيمة.

خطوات الحل

1) نحسب `result = revenue - cost`.
2) إذا كان `result > 0` ربح، إذا كان `result < 0` خسارة، وإلا توازن.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double revenue, cost, result;
    scanf("%lf %lf", &revenue, &cost);
    result = revenue - cost;              // النتيجة الصافية
    if (result > 0)
        printf("Profit: %.2f DA\n", result);
    else if (result < 0)
        printf("Loss: %.2f DA\n", -result); // نعرض الخسارة كعدد موجب
    else
        printf("Break-even\n");
    return 0;
}
```

```
14 }
```

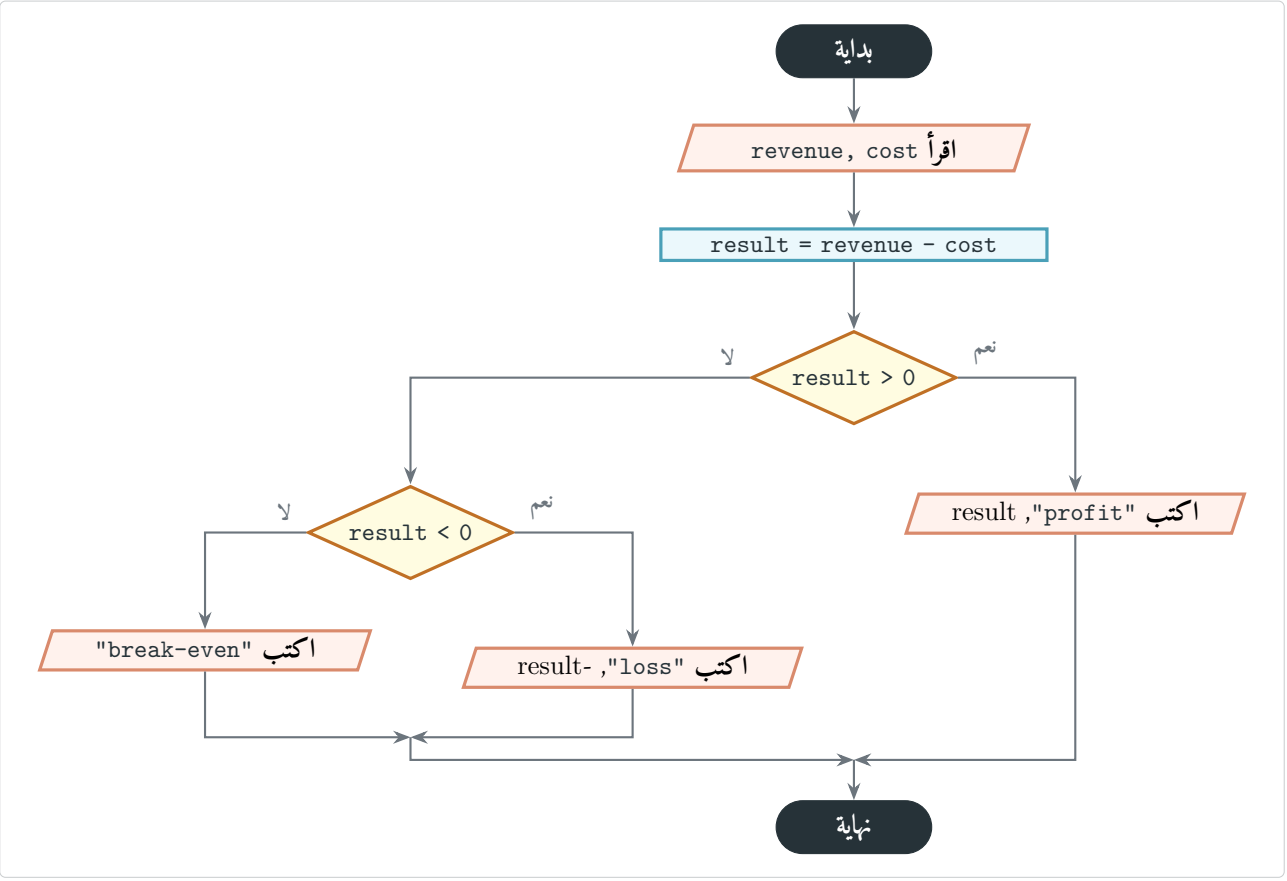
المدخلات

```
380000 412500
```

نتيجة التنفيذ

```
Loss: 32500.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.7 رمز القسيمة

تجارة إلكترونية سهل

القسيمة الصالحة لهذا الشهر رمزها **2026**، وتمنح تخفيضًا بـ 15%. اكتب برنامجًا يقرأ مبلغ السلة والرمز الذي أدخله الزبون، ويطبق التخفيض إن كان الرمز صحيحًا.

خطوات الحل

1) نقرأ المبلغ والرمز.
2) إذا كان `code == CODE` نضرب المبلغ في `0.85`.
3) نعرض المبلغ النهائي.

البرنامج بلغة C

```c
#include <stdio.h>

#define CODE 2026                   // رمز القسيمة الصالح

int main(void) {
    double amount;
    int code;
    scanf("%lf %d", &amount, &code);
    if (code == CODE) {             // == للمقارنة، و = للإسناد
        amount = amount * 0.85;     // تخفيض 15%
        printf("Coupon accepted\n");
    }
    printf("To pay = %.2f DA\n", amount);
    return 0;
}
```

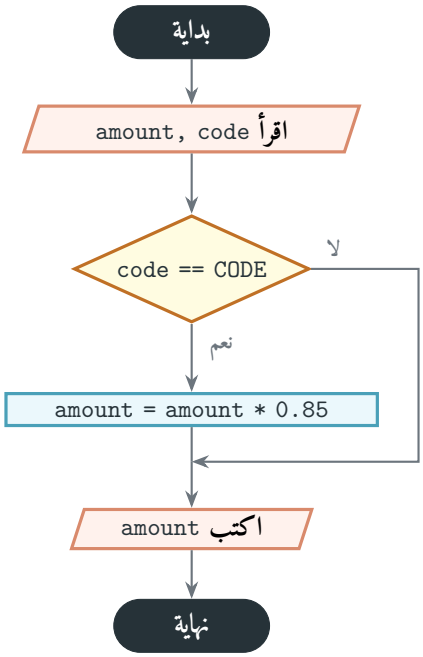
المدخلات

```
6400 2026
```

نتيجة التنفيذ

```
Coupon accepted
To pay = 5440.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.8 بيع منتج محدد بالسن

تجارة إلكترونية سهل

بعض المنتجات (مثل ألعاب الفيديو المصنفة) لا تباع لمن هم دون 16 سنة. اكتب برنامجًا يقرأ سنة ميلاد الزبون والسنة الحالية، ويحدد إن كان يُسمح له بالشراء.

خطوات الحل

1) السن: `age = year - birth`.
2) إذا كان `age >= 16` نسمح، وإلا نرفض ونعرض عدد السنوات الباقية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int birth, year, age;
    scanf("%d %d", &birth, &year);
    age = year - birth;                // السن التقريبية
    if (age >= 16)
        printf("Age %d: purchase allowed\n", age);
    else
        printf("Age %d: refused (%d years left)\n", age, 16 - age);
    return 0;
}
```

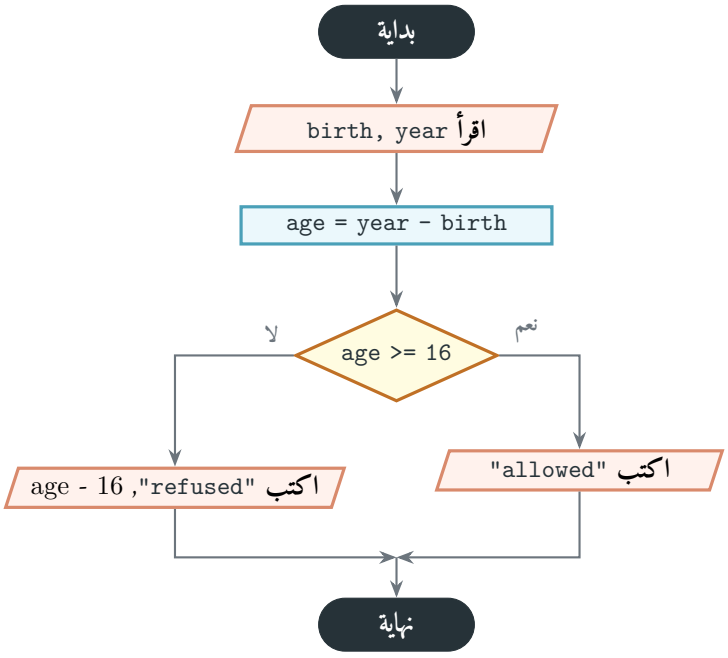
المدخلات

```
2011 2026
```

نتيجة التنفيذ

```
Age 15: refused (1 years left)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.9 طريقة الدفع

تجارة إلكترونية سهل

يختار الزبون طريقة الدفع برقم: 1 بطاقة بنكية، 2 الدفع عند الاستلام، 3 حساب بريدي جاري. اكتب برنامجًا يعرض اسم الطريقة والمصاريف الإضافية (0، 200، 50 دج) باستعمال `switch`.

خطوات الحل

1) نقرأ الاختيار choice.
2) switch (choice) مع حالة لكل رقم وbreak في نهايتها.
3) الحالة default تعالج الأرقام غير الصحيحة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int choice;
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            printf("Bank card, fee = 0 DA\n");
            break;                  // يمنع تنفيذ الحالات الموالية break
        case 2:
            printf("Cash on delivery, fee = 200 DA\n");
            break;
        case 3:
            printf("CCP account, fee = 50 DA\n");
            break;
        default:                    // أي قيمة أخرى
            printf("Invalid choice\n");
    }
    return 0;
}
```

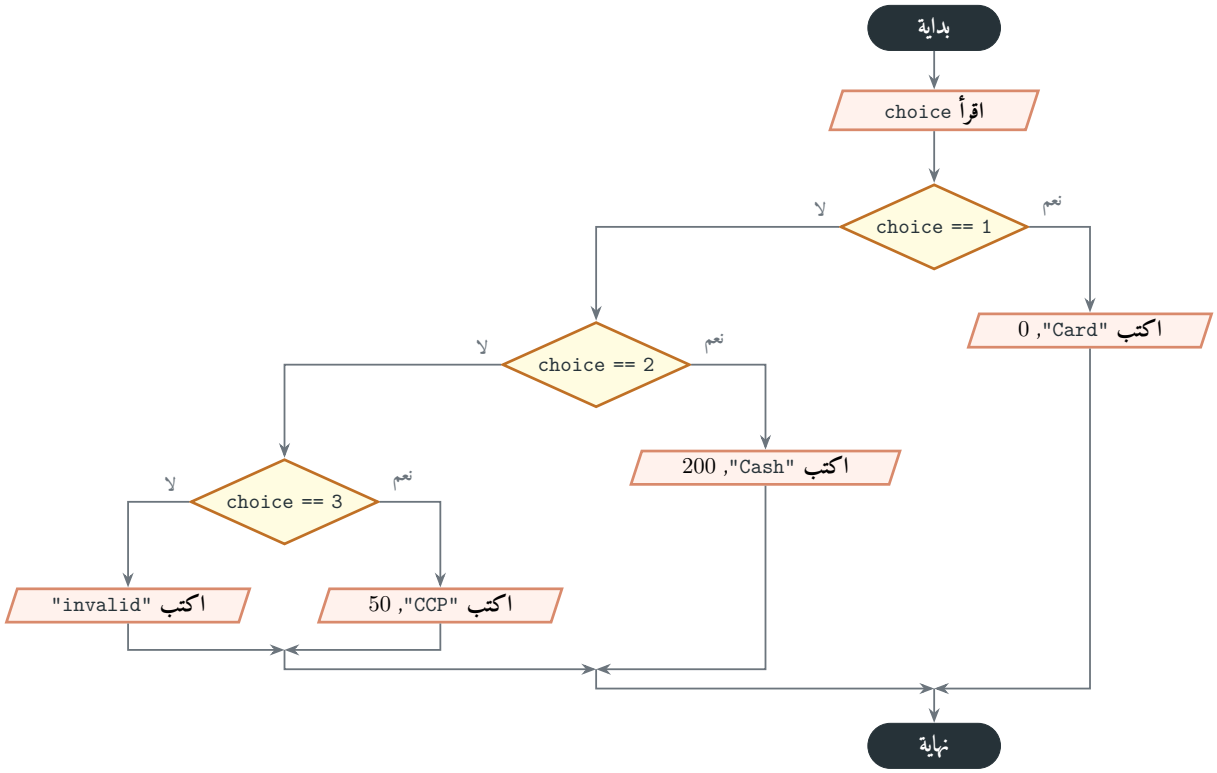
المدخلات

```
2
```

نتيجة التنفيذ

```
Cash on delivery, fee = 200 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.10 نجاح تجربة A/B

علم البيانات
سهل

في تجربة A/B نعتبر النسخة الجديدة ناجحة إذا تجاوز معدل تحويلها 3.5%. اكتب برنامجًا يقرأ عدد الزوار وعدد المشتريات للنسخة الجديدة ويحكم عليها.

خطوات الحل

1) نحسب المعدل `rate = buys / visitors * 100`.
2) نقارنه بالعتبة `3.5`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double visitors, buys, rate;
    scanf("%lf %lf", &visitors, &buys);
    rate = buys / visitors * 100;      // معدل التحويل بالمئة
    printf("Conversion = %.2f %%\n", rate);
    if (rate > 3.5)
        printf("New version: SUCCESS\n");
    else
        printf("New version: FAILURE\n");
    return 0;
}
```

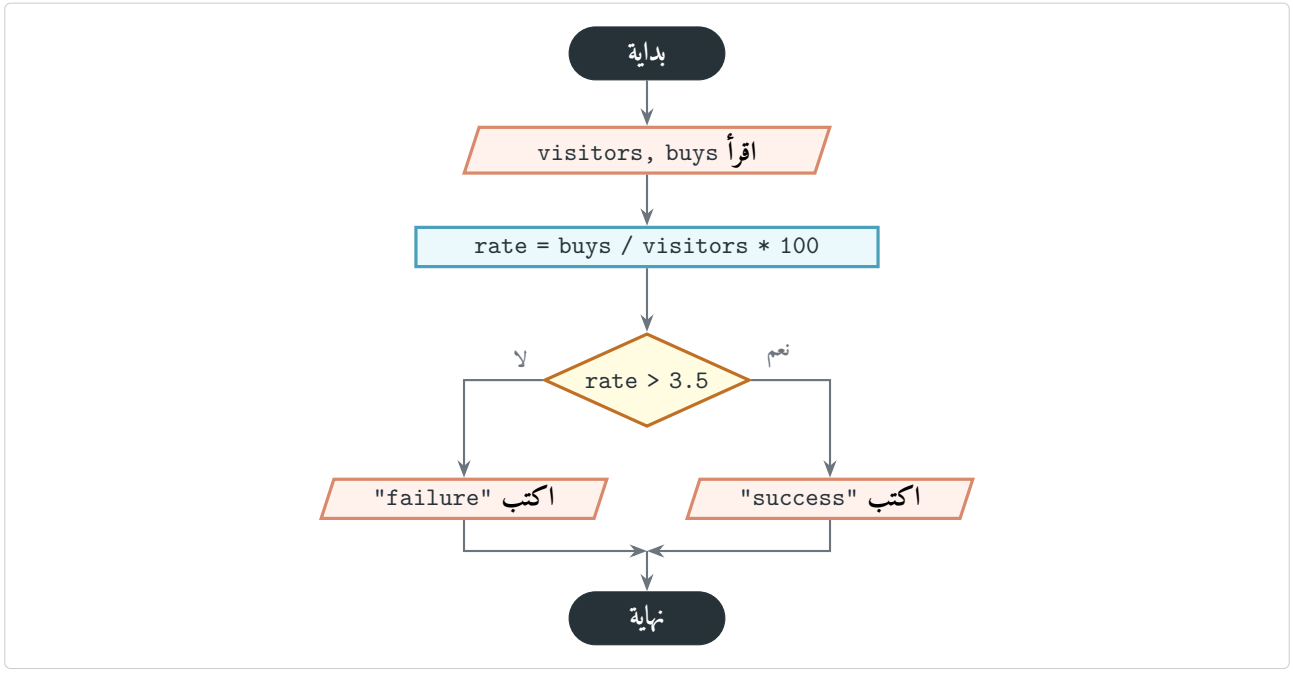
المدخلات

```
2400 91
```

نتيجة التنفيذ

```
Conversion = 3.79 %
New version: SUCCESS
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 2.11 تخفيض حسب الشرائح

تجارة إلكترونية ★★ متوسط

يطبق متجر تخفيضًا حسب مبلغ السلة: أقل من 2000 دج لا تخفيض، من 2000 إلى أقل من 10000 تخفيض 5%، من 10000 إلى أقل من 50000 تخفيض 10%، و50000 فأكثر تخفيض 15%. اكتب برنامجًا يحسب المبلغ النهائي.

خطوات الحل

1) نرتب الشروط من الأكبر إلى الأصغر في سلسلة `if / else if`.
2) كل فرع يحدد النسبة `rate`.
3) المبلغ النهائي: `amount * (1 - rate / 100)`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, rate, final;
    scanf("%lf", &amount);
    // نبدأ بأعلى شريحة: كل else if تعني أن الشروط السابقة فشلت
    if (amount >= 50000)
        rate = 15;
    else if (amount >= 10000)
        rate = 10;
    else if (amount >= 2000)
        rate = 5;
    else
        rate = 0;
    final = amount * (1 - rate / 100);
    printf("Discount rate = %.0f %%\n", rate);
    printf("Final amount = %.2f DA\n", final);
    return 0;
}
```

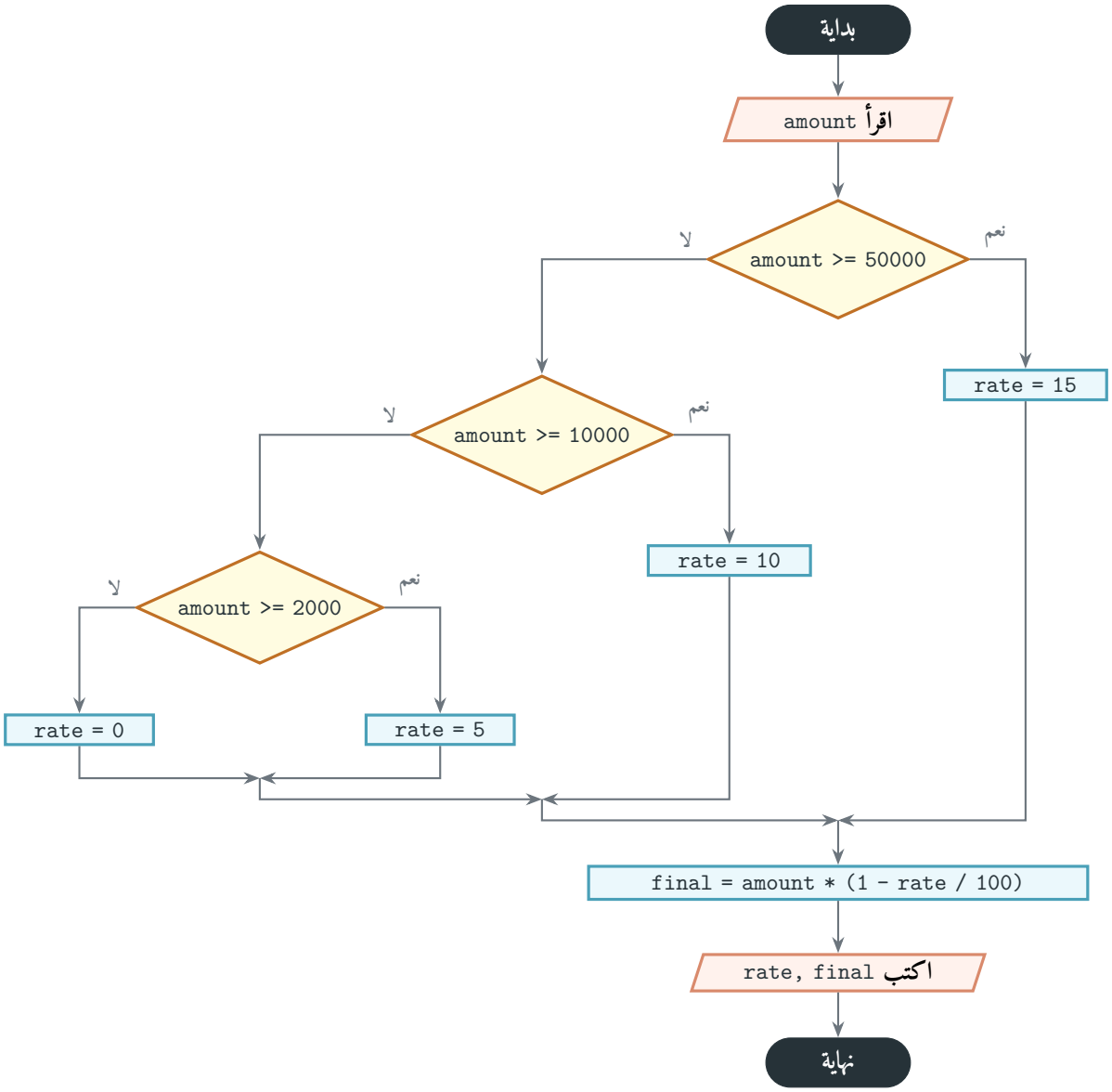
المدخلات

```
23500
```

نتيجة التنفيذ

```
Discount rate = 10 %
Final amount = 21150.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.12 تكلفة الشحن حسب الوزن

تجارة إلكترونية ★★ متوسط

تحسب شركة التوصيل التكلفة حسب الوزن: حتى 1 كغ: 400 دج، حتى 5 كغ: 700 دج، حتى 20 كغ: 1200 دج، وفوق ذلك 1200 دج + 80 دج لكل كيلوغرام إضافي. وتضاعف التكلفة إذا كان التوصيل سريعًا. اكتب البرنامج.

خطوات الحل

1) نقرأ الوزن `weight` ونوع التوصيل `fast` (1 سريع، 0 عادي).
2) نحدد التكلفة حسب الشريحة.
3) في الشريحة الأخيرة: `1200 + (weight - 20) * 80`.
4) إذا كان `fast == 1` نضاعف التكلفة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double weight, cost;
    int fast;
    scanf("%lf %d", &weight, &fast);        // الوزن ونوع التوصيل
    if (weight <= 1)
        cost = 400;
    else if (weight <= 5)
        cost = 700;
    else if (weight <= 20)
        cost = 1200;
    else
        cost = 1200 + (weight - 20) * 80;  // 80 دج لكل كغ فوق 20
    if (fast == 1)
        cost = cost * 2;                   // التوصيل السريع مضاعف
    printf("Shipping cost = %.2f DA\n", cost);
    return 0;
}
```

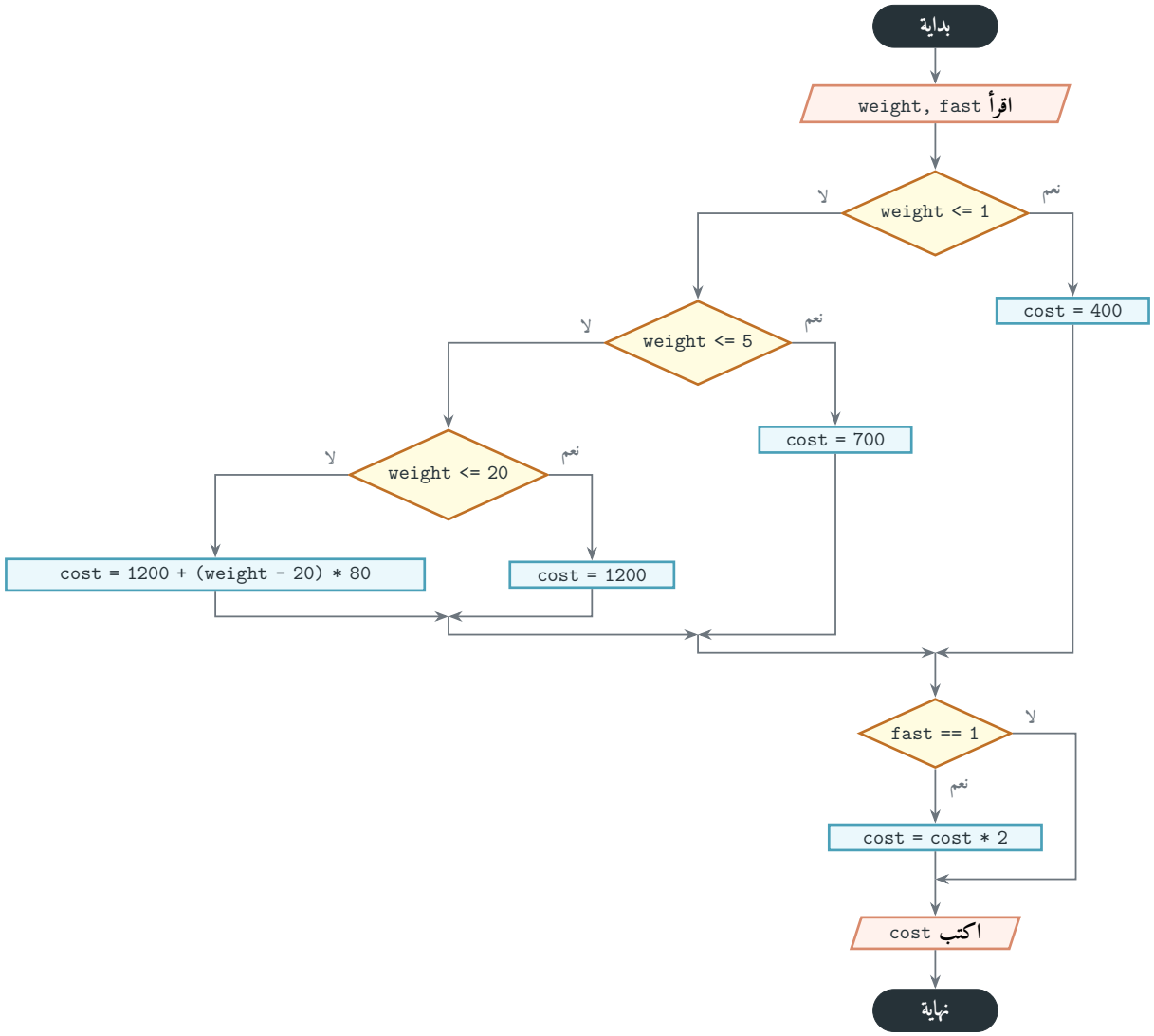
المدخلات

```
27.5 1
```

نتيجة التنفيذ

```
Shipping cost = 3600.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.13 أفضل تقييم بين ثلاثة منتجات

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ متوسطات تقييم ثلاثة منتجات ويعرض **رقم المنتج الأفضل** وتقييمه. في حالة التساوي نختار المنتج ذا الرقم الأصغر.

خطوات الحل

1) نفترض أن الأول هو الأفضل: `best = 1` و`max = ratingA`.
2) نقارن `ratingB` بـ `max`: إذا كان أكبر **تمامًا** نحدّث.
3) نفعل نفس الشيء مع `ratingC`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double ratingA, ratingB, ratingC, max;
    int best;
```

```
    scanf("%lf %lf %lf", &ratingA, &ratingB, &ratingC);
    max = ratingA;                      // نفترض أن الأول هو الأفضل
    best = 1;
    if (ratingB > max) {                // > تمامًا: عند التساوي نحتفظ بالأصغر رقمًا
        max = ratingB;
        best = 2;
    }
    if (ratingC > max) {
        max = ratingC;
        best = 3;
    }
    printf("Best product: #%d (%.1f stars)\n", best, max);
    return 0;
}
```

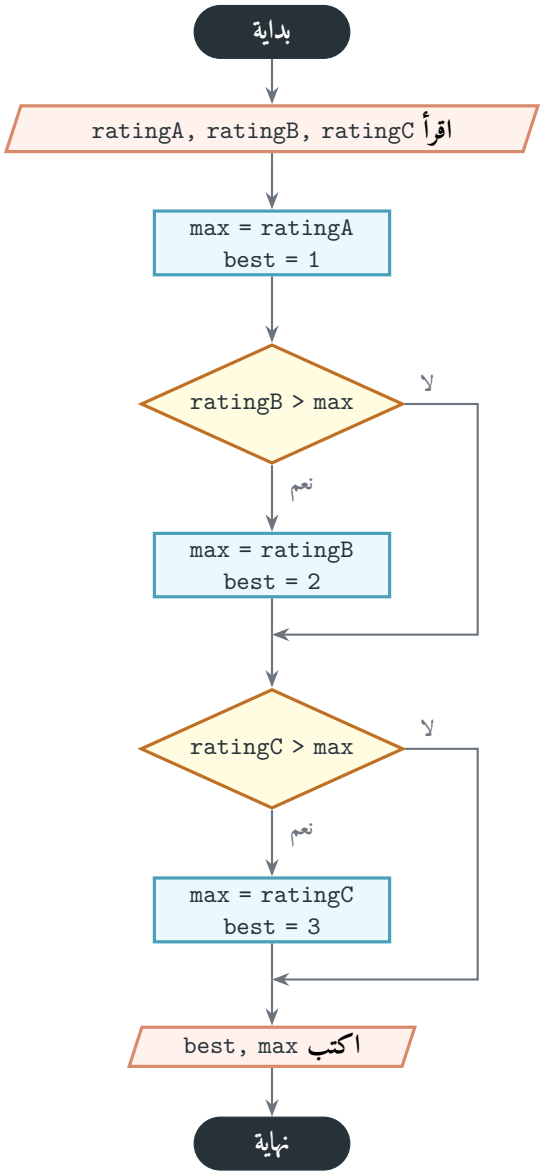
المدخلات

```
4.3 4.7 4.7
```

نتيجة التنفيذ

```
Best product: #2 (4.7 stars)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.14 ترتيب ثلاثة أسعار

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يقرأ ثلاثة أسعار ويعرضها **مرتبة تصاعديًا** باستعمال ثلاث مقارنات وتبديلات فقط.

خطوات الحل

1) إذا كان `price1 > price2` نبدل `price1` و`price2`.
2) إذا كان `price2 > price3` نبدل `price2` و`price3` (الآن `price3` هو الأكبر).
3) إذا كان `price1 > price2` نبدل مرة أخرى.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```
    double price1, price2, price3, tmp;
    scanf("%lf %lf %lf", &price1, &price2, &price3);
    if (price1 > price2) { tmp = price1; price1 = price2; price2 = tmp; }   // الآن price1 => price2
    if (price2 > price3) { tmp = price2; price2 = price3; price3 = tmp; }   // صعد إلى الأكبر price3
    if (price1 > price2) { tmp = price1; price1 = price2; price2 = tmp; }   // رتب الاثنين الأولين من جديد
    printf("Sorted: %.0f %.0f %.0f\n", price1, price2, price3);
    return 0;
}
```

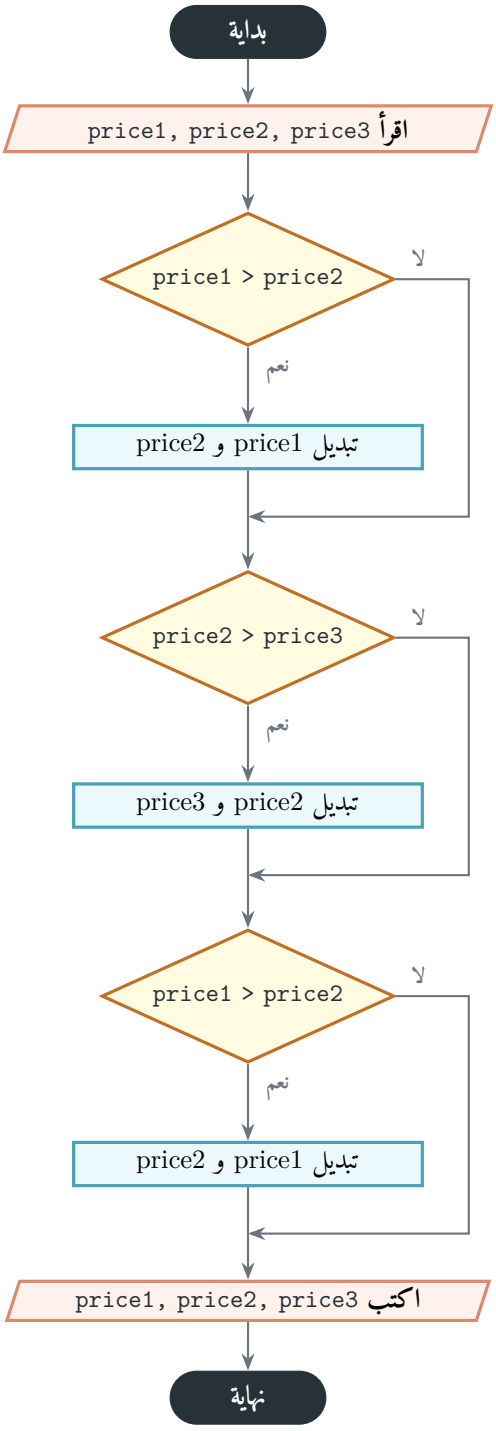
المدخلات

```
950 320 610
```

نتيجة التنفيذ

```
Sorted: 320 610 950
```

المخطط الإنسيابي (Organigramme)



تمرين 2.15 السنة الكبيسة

تجارة إلكترونية ★★ متوسط

لحساب تواريخ التسليم في فبراير نحتاج معرفة إن كانت السنة **كبيسة**: تقبل القسمة على 4 ولا تقبل القسمة على 100، أو تقبل القسمة على 400. اكتب برنامجًا يحدد ذلك ويعرض عدد أيام فبراير.

خطوات الحل

1) الشرط: `(year % 4 == 0 && year % 100 != 0) || year % 400 == 0`.

2) إذا تحقق: فبراير 29 يومًا، وإلا 28.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int year, feb;
    scanf("%d", &year);
    // يكفي أن يتحقق أحد الطرفين : ||
    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
        feb = 29;
    else
        feb = 28;
    printf("%d: %s year, February has %d days\n",
           year, feb == 29 ? "leap" : "common", feb);
    return 0;
}
```

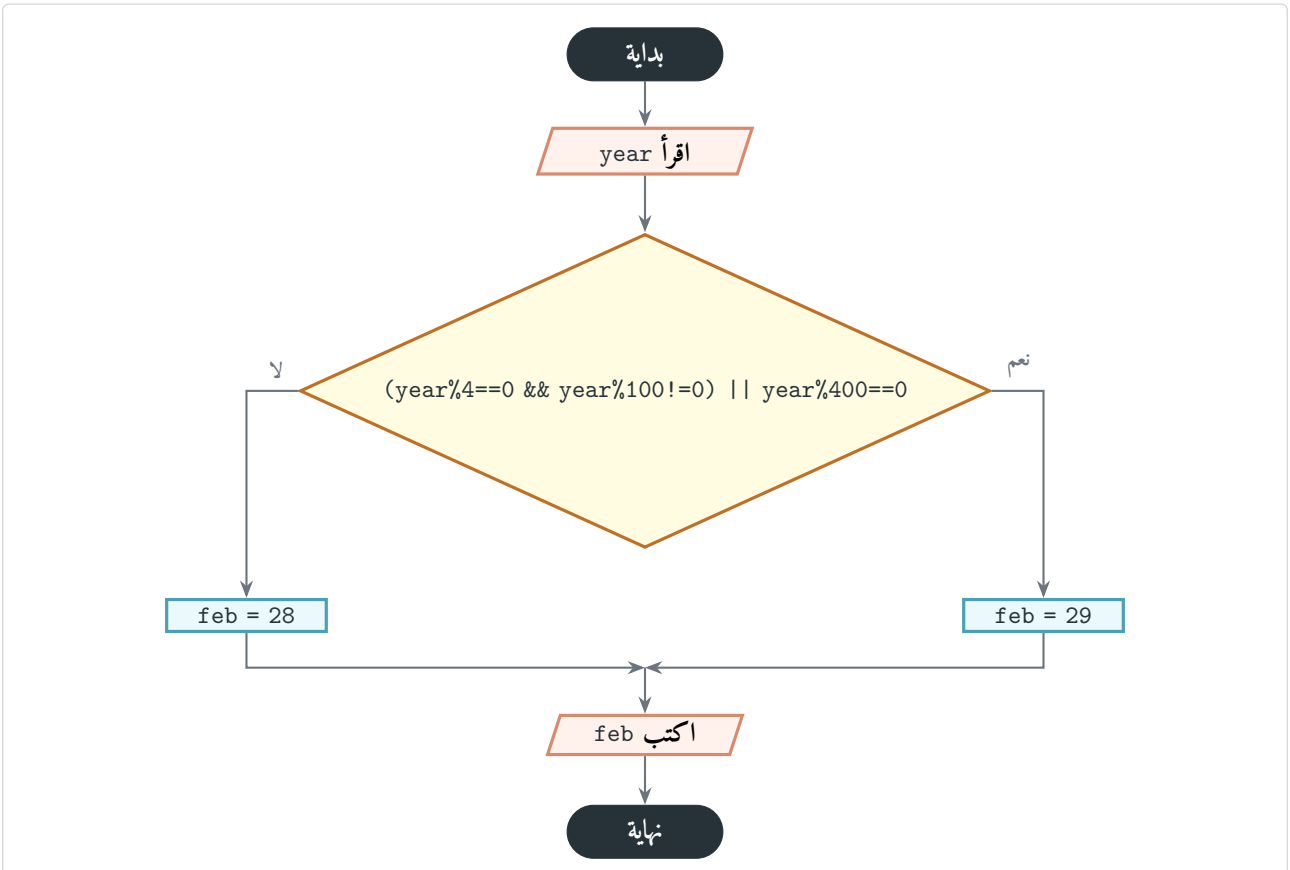
المدخلات

```
2028
```

نتيجة التنفيذ

```
2028: leap year, February has 29 days
```

المخطط الإنسيابي (Organigramme)



تمرين 2.16 عدد أيام الشهر

تجارة إلكترونية ⭐⭐ متوسط

لحساب مدة اشتراك شهري، اكتب برنامجًا يقرأ رقم الشهر والسنة ويعرض **عدد أيامه** باستعمال `switch` مع تجميع الحالات المتشابهة.

خطوات الحل

1) الأشهر 1، 3، 5، 7، 8، 10، 12 فيها 31 يومًا: حالات متتالية دون `break`.
2) الأشهر 4، 6، 9، 11 فيها 30 يومًا.
3) الشهر 2 يعتمد على السنة الكبيسة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int month, year, days;
    scanf("%d %d", &month, &year);
    switch (month) {
        case 1: case 3: case 5: case 7:
        case 8: case 10: case 12:      // عدة حالات تشترك في نفس الكود
            days = 31;
            break;
        case 4: case 6: case 9: case 11:
            days = 30;
            break;
        case 2:
            if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
                days = 29;                // سنة كبيسة
            else
                days = 28;
            break;
        default:
            days = 0;                     // شهر غير صالح
    }
    if (days == 0)
        printf("Invalid month\n");
    else
        printf("Month %d/%d has %d days\n", month, year, days);
    return 0;
}
```

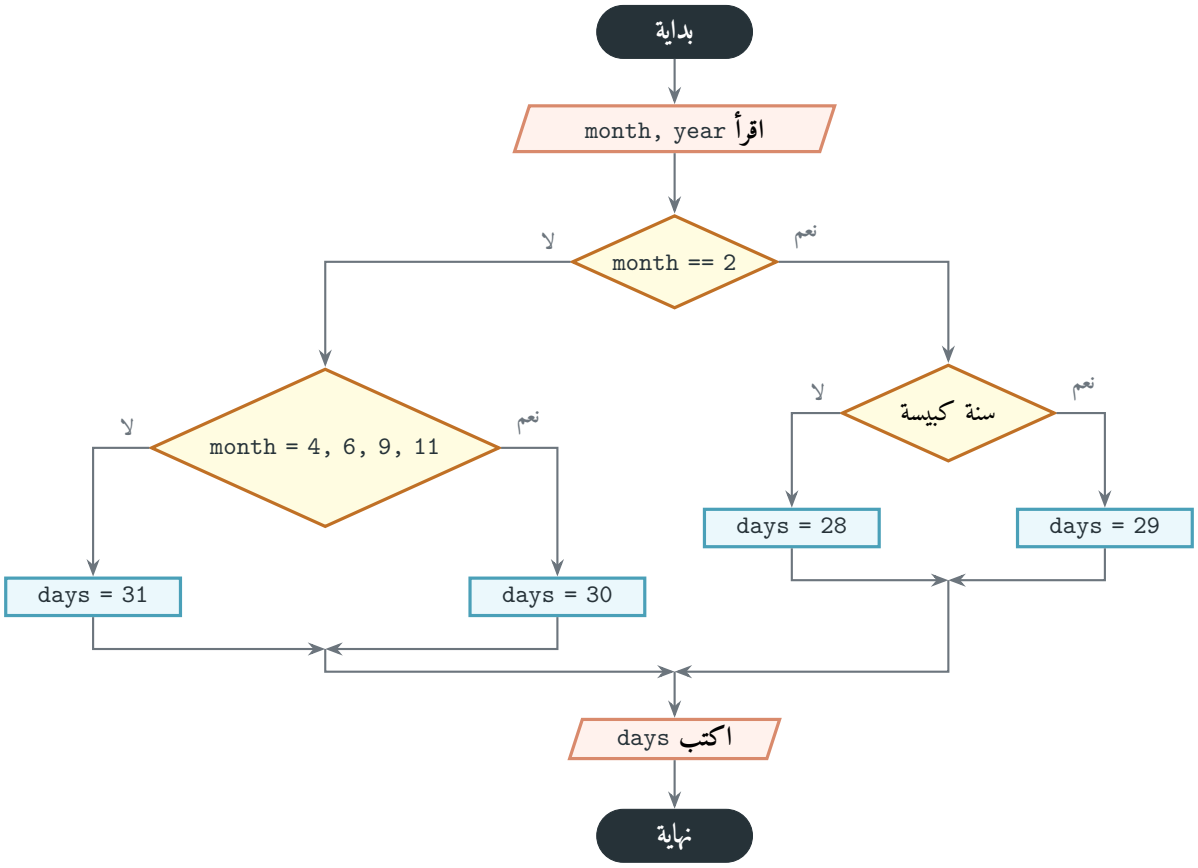
المدخلات

```
4 2026
```

نتيجة التنفيذ

```
Month 4/2026 has 30 days
```

المخطط الإنسيابي (Organigramme)



تمرين 2.17 تصنيف الزبائن حسب الإنفاق

علم البيانات ★★ متوسط

يصنف قسم التسويق الزبائن حسب إنفاقهم السنوي وعدد طلبياتهم: **VIP** إذا تجاوز الإنفاق 200000 وعدد الطلبيات 20، **وفي** إذا تجاوز الإنفاق 50000 أو الطلبيات 10، **عادي** إذا كانت له طلبية واحدة على الأقل، وإلا **غير نشط**.

خطوات الحل

1) نقرأ الإنفاق `spend` وعدد الطلبيات `nbOrders`.
2) نختبر الشروط من الأقوى إلى الأضعف.
3) نستعمل `&&` للفئة الأولى و`||` للثانية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double spend;
    int nbOrders;
    scanf("%lf %d", &spend, &nbOrders);
    if (spend > 200000 && nbOrders > 20)        // الشرطان معًا
        printf("Segment: VIP\n");
    else if (spend > 50000 || nbOrders > 10)    // يكفي أحدهما
        printf("Segment: Loyal\n");
```

```
    else if (nbOrders >= 1)
        printf("Segment: Regular\n");
    else
        printf("Segment: Inactive\n");
    return 0;
}
```

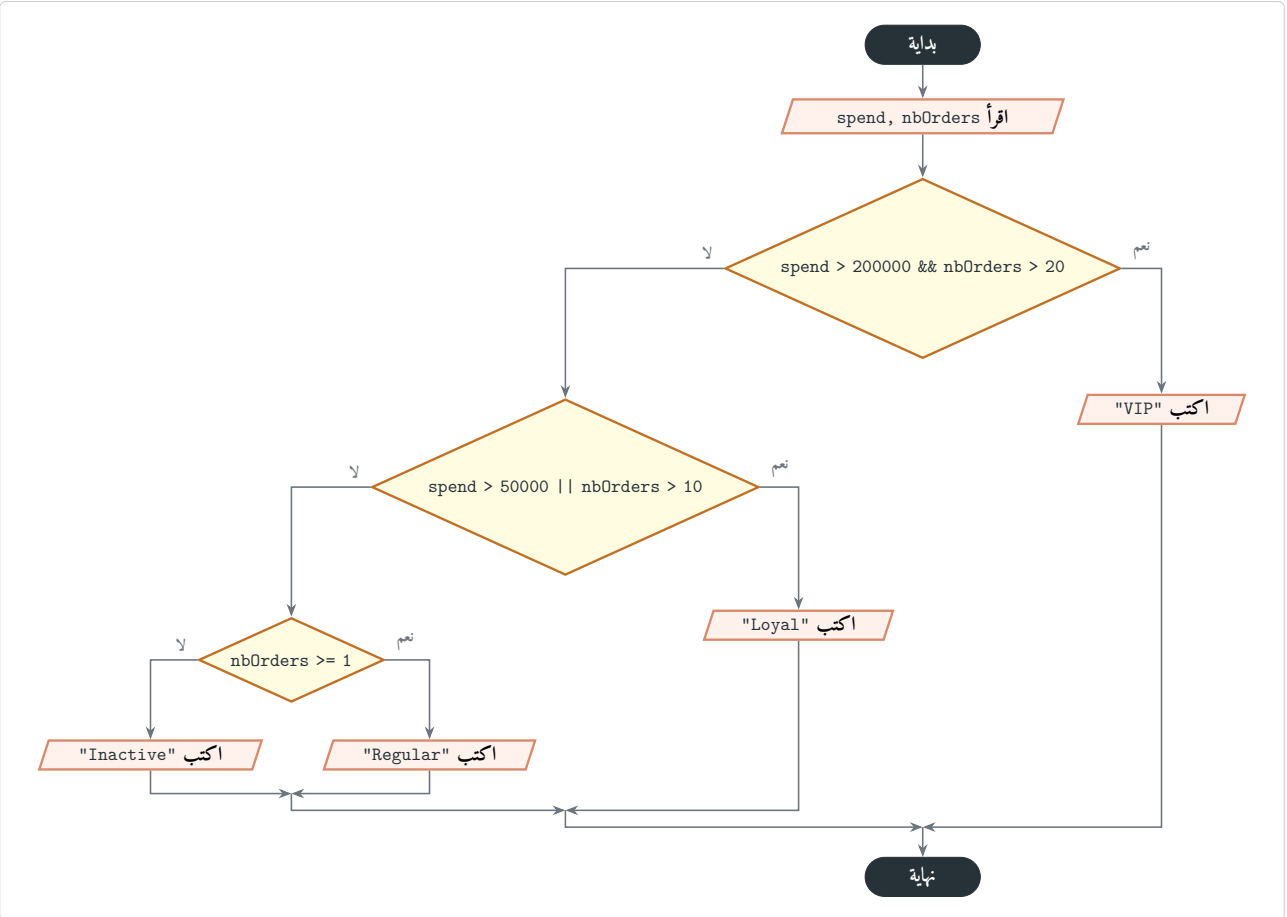
المدخلات

```
85000 7
```

نتيجة التنفيذ

```
Segment: Loyal
```

المخطط الإنسيابي (Organigramme)



تمرين 2.18 الرسم حسب صنف المنتج

تجارة إلكترونية ★★ متوسط

تختلف نسبة الرسم حسب صنف المنتج: `'A'` مواد غذائية أساسية 0%، `'B'` أدوية 9%، `'C'` بقية المنتجات 19%. اكتب برنامجًا يقرأ الصنف والسعر ويعرض السعر مع الرسم، ويقبل الحرف كبيرًا أو صغيرًا.

خطوات الحل

1) نقرأ الحرف `cat` والسعر.

2) في switch نضع case 'A': case 'a': معًا.

3) نحسب price * (1 + taxRate).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char cat;
    double price, taxRate;
    scanf(" %c %lf", &cat, &price);
    switch (cat) {
        case 'A': case 'a': taxRate = 0;    break;   // مواد أساسية
        case 'B': case 'b': taxRate = 0.09; break;   // أدوية
        case 'C': case 'c': taxRate = 0.19; break;   // بقية المنتجات
        default:            taxRate = -1;            // صنف غير معروف
    }
    if (taxRate >= 0)
        printf("Category %c: %.2f DA with tax\n", cat, price * (1 + taxRate));
    else
        printf("Unknown category\n");
    return 0;
}
```

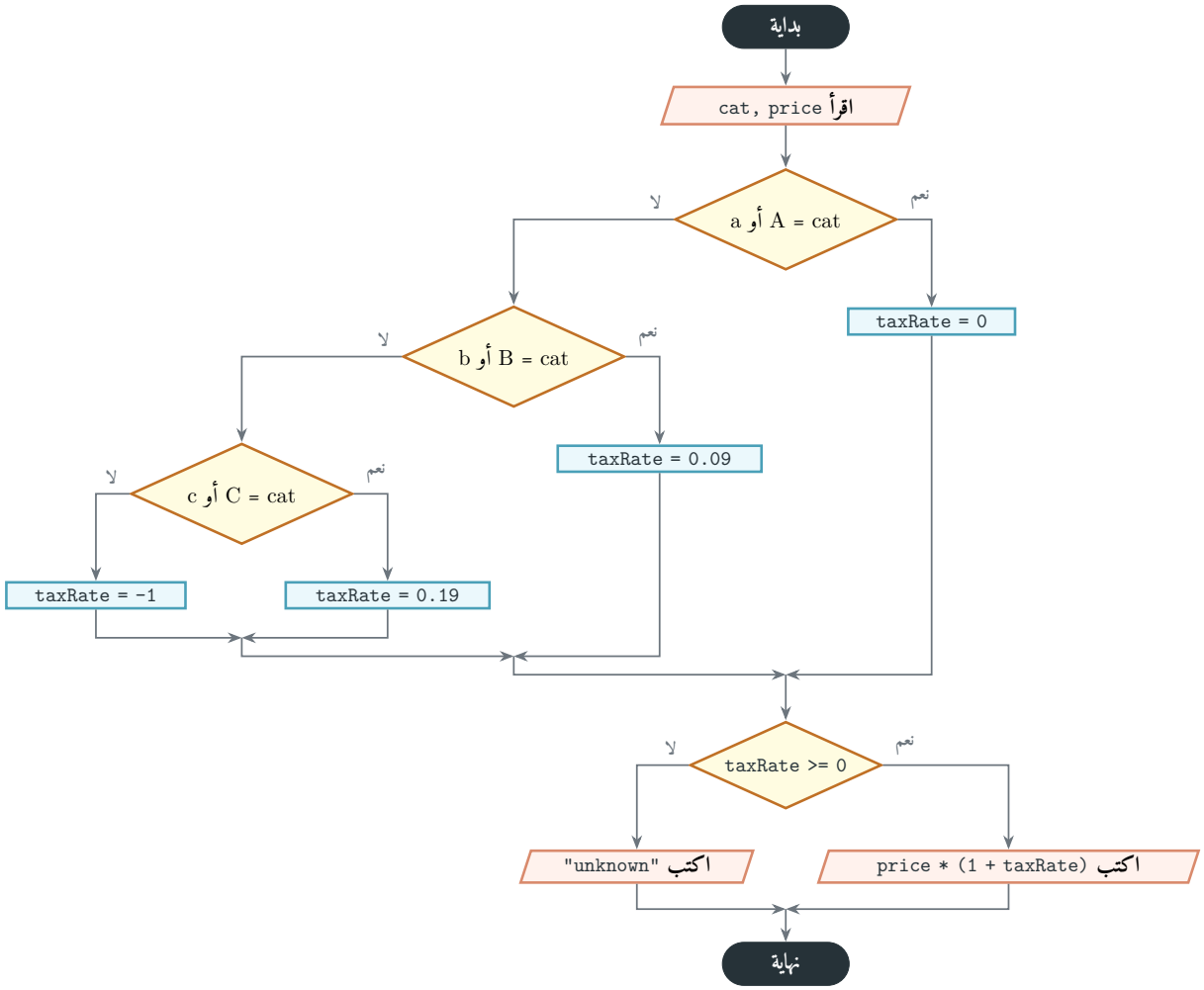
المدخلات

```
c 12000
```

نتيجة التنفيذ

```
Category c: 14280.00 DA with tax
```

المخطط الإنسيابي (Organigramme)



تمرين 2.19 كشف القيم الشاذة

علم البيانات ★★ متوسط

نعتبر مبلغ طلبية **شاذًا** إذا كانت درجته المعيارية أكبر من 3 أو أصغر من -3، **مشبوهًا** إذا كانت قيمتها المطلقة بين 2 و3، **وعاديًا** في غير ذلك. اكتب برنامجًا يقرأ المبلغ والمتوسط والانحراف المعياري ويصنفه.

خطوات الحل

1) نحسب `zscore = (amount - mean) / std`.
2) نحسب القيمة المطلقة يدويًا: إذا كان `zscore < 0` فـ `az = -zscore`.
3) نقارن `az` بالعتبتين 3 و2.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, mean, std, zscore, az;
    scanf("%lf %lf %lf", &amount, &mean, &std);
    zscore = (amount - mean) / std;          // الدرجة المعيارية
```

```c
    az = zscore;
    if (az < 0)                    // القيمة المطلقة دون دالة fabs
        az = -az;
    printf("z = %.2f -> ", zscore);
    if (az > 3)
        printf("outlier\n");
    else if (az >= 2)
        printf("suspicious\n");
    else
        printf("normal\n");
    return 0;
}
```

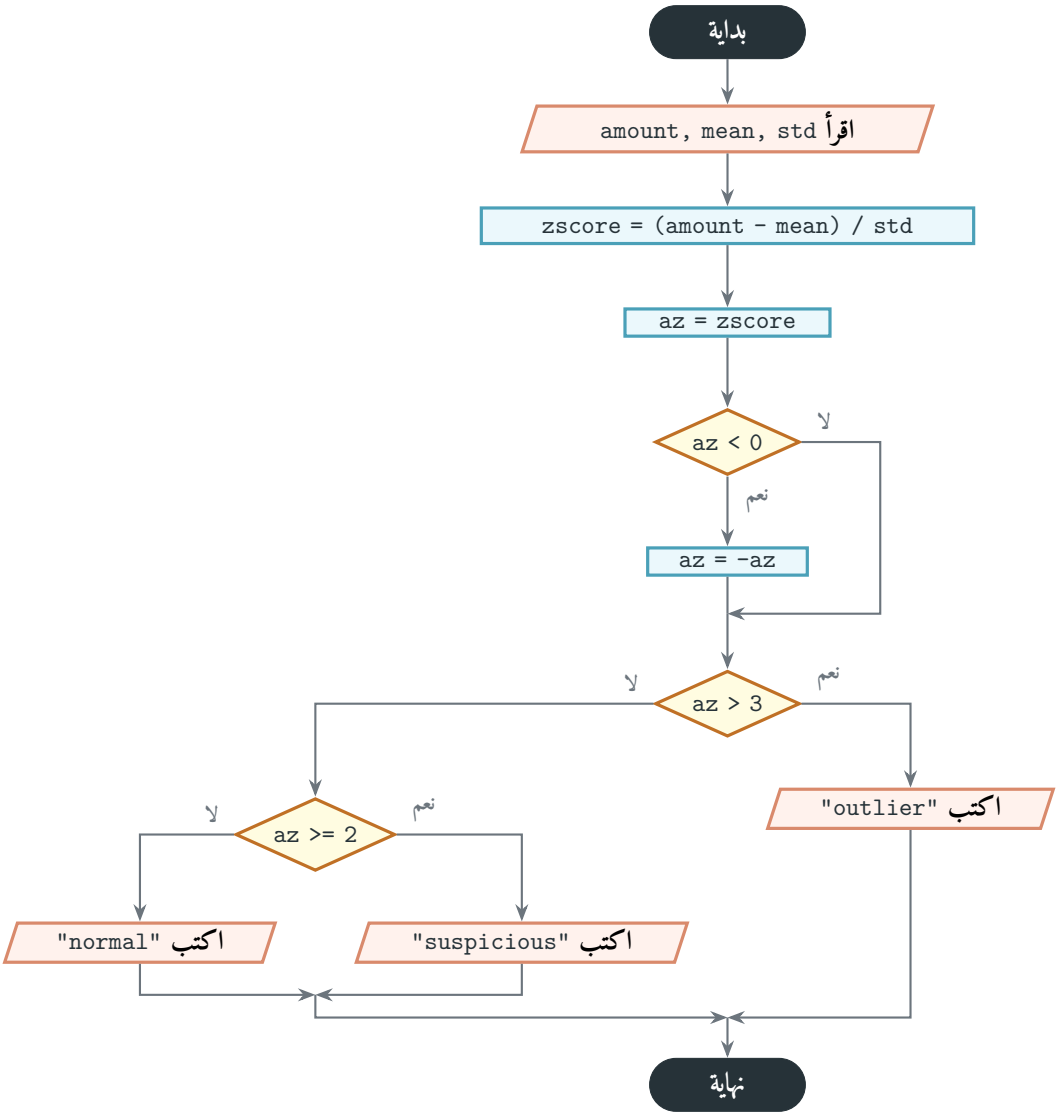
المدخلات

```
15500 4800 3100
```

نتيجة التنفيذ

```
z = 3.45 -> outlier
```

المخطط الإنسيابي (Organigramme)



تمرين 2.20 هيمنة منتج على آخر

علم البيانات ⭐⭐ متوسط

نقول إن منتجًا يهيمن على آخر إذا كان سعره أقل أو يساويه وتقييمه أكبر أو يساويه، مع تفوق في أحدهما على الأقل. اكتب برنامجًا يقرأ (سعر، تقييم) لمنتجين ويحدد علاقة الهيمنة بينهما.

خطوات الحل

1) d1: المنتج 1 يهيمن إذا كان `p1 <= p2 && r1 >= r2 && (p1 < p2 || r1 > r2)`.
2) d2 بنفس الطريقة بتبديل الأدوار.
3) إذا لم يهيمن أي منهما فالمنتجان غير قابلين للمقارنة (أو متساويان).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double p1, r1, p2, r2;
    scanf("%lf %lf %lf %lf", &p1, &r1, &p2, &r2);
    // ليس أسوأ في أي معيار، وأفضل في معيار واحد على الأقل
    if (p1 <= p2 && r1 >= r2 && (p1 < p2 || r1 > r2))
        printf("Product 1 dominates product 2\n");
    else if (p2 <= p1 && r2 >= r1 && (p2 < p1 || r2 > r1))
        printf("Product 2 dominates product 1\n");
    else
        printf("No dominance (trade-off)\n");
    return 0;
}
```

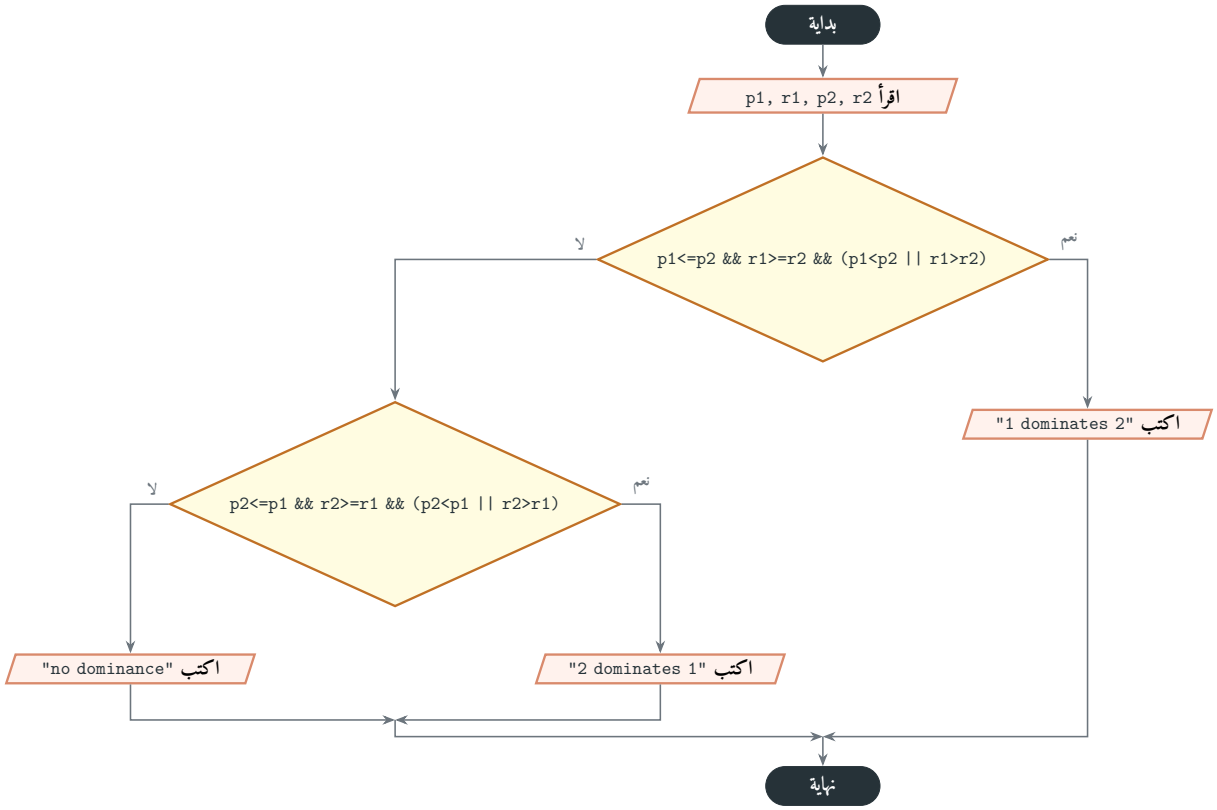
المدخلات

```
3200 4.5
3500 4.1
```

نتيجة التنفيذ

```
Product 1 dominates product 2
```

المخطط الإنسيابي (Organigramme)



تمرين 2.21 التقييم إلى نجوم

علم البيانات ★★ متوسط

يُعرض متوسط التقييم (عدد حقيقي بين 0 و5) على شكل نجوم: نقربه لأقرب نصف نجمة. اكتب برنامجًا يقرأ المتوسط ويعرض عدد النجوم الكاملة وهل توجد نصف نجمة، ثم وصفًا: ممتاز (4.5 فأكثر)، جيد (3.5 فأكثر)، متوسط (2.5 فأكثر)، ضعيف.

خطوات الحل

1) نقرب لأقرب نصف: `halves = (int)(avg * 2 + 0.5)` (عدد الأنصاف).
2) النجوم الكاملة: `halves / 2`، ونصف نجمة إذا كان `halves % 2 == 1`.
3) الوصف بسلسلة `if / else if` على القيمة المقربة `halves / 2.0`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double avg, rounded;
    int halves, full, half;
    scanf("%lf", &avg);
    halves = (int)(avg * 2 + 0.5);      // عدد أنصاف النجوم بعد التقريب
    full = halves / 2;                  // النجوم الكاملة
    half = halves % 2;                  // 1 إذا بقي نصف نجمة
    rounded = halves / 2.0;                     // القيمة المقربة
    printf("%.2f -> %d full star(s)", avg, full);
    if (half == 1)
```

```c
        printf(" + half star");
    printf(" (%.1f)\n", rounded);
    if (rounded >= 4.5)       printf("Excellent\n");
    else if (rounded >= 3.5)  printf("Good\n");
    else if (rounded >= 2.5)  printf("Average\n");
    else                      printf("Poor\n");
    return 0;
}
```

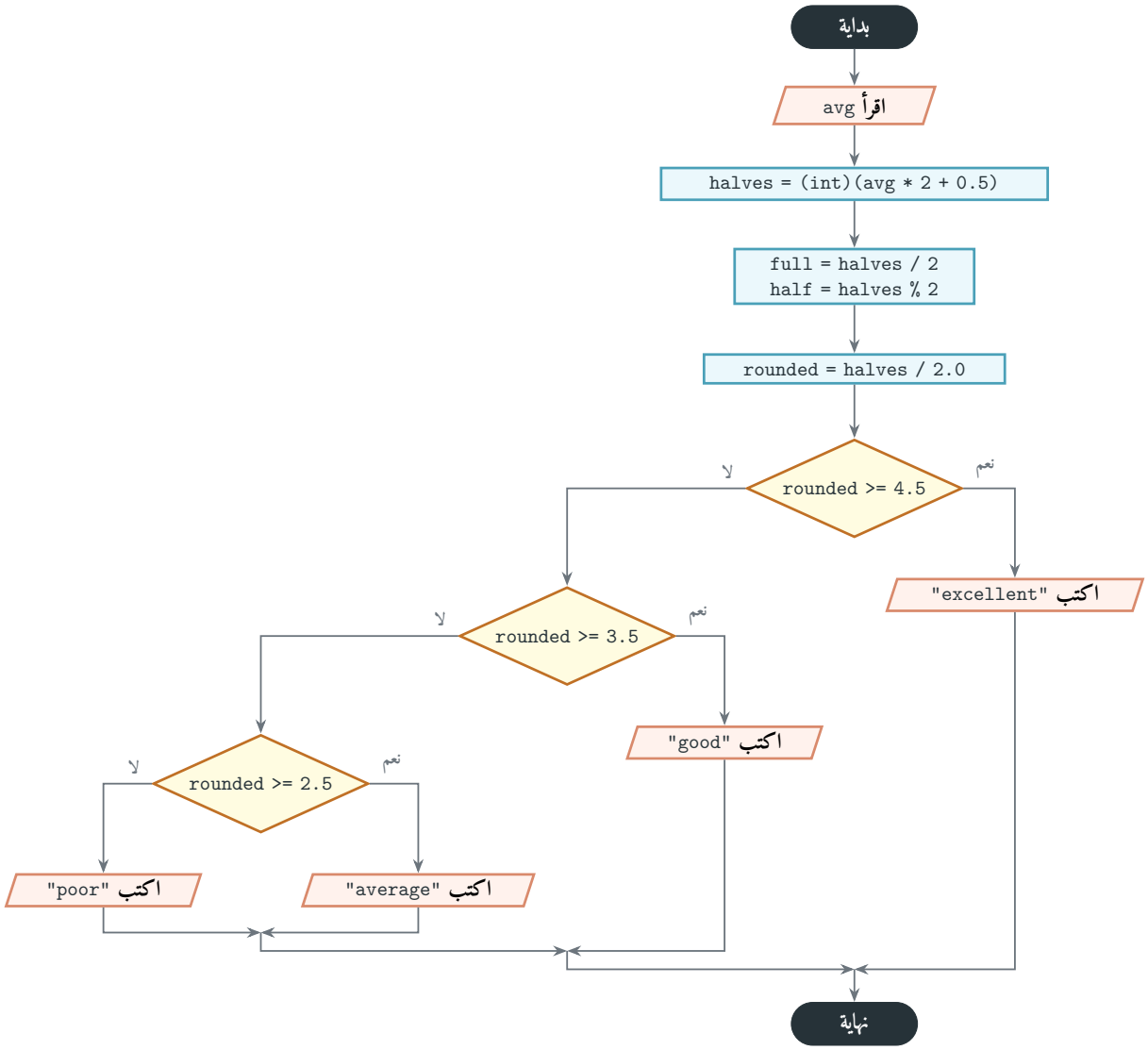
المدخلات

```
3.83
```

نتيجة التنفيذ

```
3.83 -> 4 full star(s) (4.0)
Good
```

المخطط الإنسيابي (Organigramme)



تمرين 2.22 نقطة التعادل

تجارة إلكترونية ★★ متوسط

بائع لديه تكاليف ثابتة `fixedCost` (كراء، إعلانات)، وتكلفة متغيرة `varCost` لكل وحدة، وسعر بيع `price`. **نقطة التعادل** هي أصغر عدد وحدات يغطي التكاليف: `fixedCost / (price - varCost)` مقربًا إلى الأعلى. اكتب برنامجًا يحسبها، مع معالجة الحالة التي يكون فيها السعر غير كافٍ.

خطوات الحل

1) الهامش للوحدة: `margin = price - varCost`.
2) إذا كان `margin <= 0` فلا يمكن تحقيق ربح أبدًا.
3) وإلا: `units = fixedCost / margin`، وإذا كان هناك باقٍ نضيف 1.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long fixedCost, varCost, price, margin, units;
    scanf("%ld %ld %ld", &fixedCost, &varCost, &price);
    margin = price - varCost;                    // ما تربحه في كل وحدة
    if (margin <= 0) {
        printf("Price too low: never profitable\n");
    } else {
        units = fixedCost / margin;              // قسمة صحيحة
        if (fixedCost % margin != 0)
            units = units + 1;                   // تقريب إلى الأعلى
        printf("Margin per unit = %ld DA\n", margin);
        printf("Break-even = %ld units\n", units);
    }
    return 0;
}
```

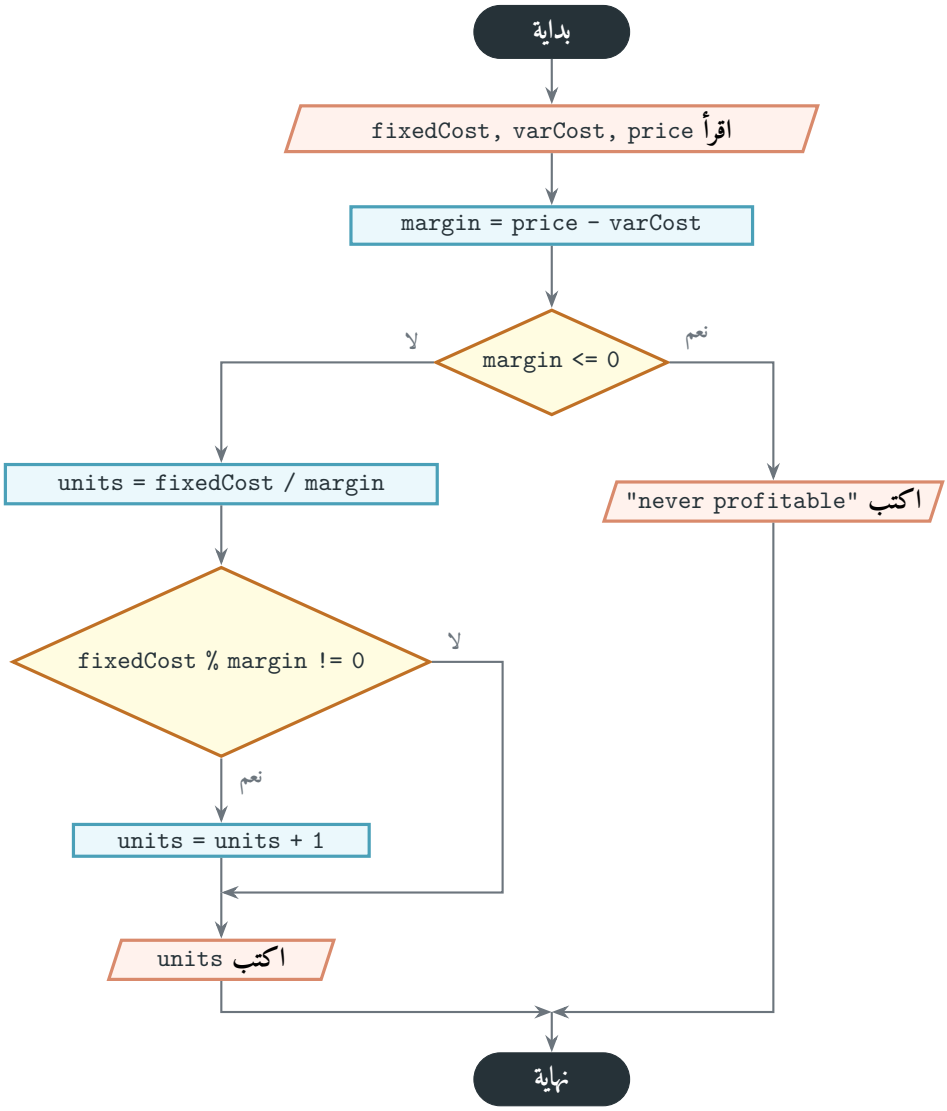
المدخلات

```
120000 650 1100
```

نتيجة التنفيذ

```
Margin per unit = 450 DA
Break-even = 267 units
```

المخطط الإنسيابي (Organigramme)



تمرين 2.23 صحة تاريخ الطلبية

تجارة إلكترونية ★★ متوسط

يدخل الزبون تاريخ استلام مرغوب `jj mm aaaa`. اكتب برنامجًا يتحقق من **صحة التاريخ** (الشهر بين 1 و12، واليوم بين 1 وعدد أيام الشهر مع مراعاة السنة الكبيسة).

خطوات الحل

1) نتحقق أولًا من الشهر.
2) نحسب عدد أيام الشهر `max` (31 أو 30 أو 28/29).
3) التاريخ صحيح إذا كان `1 <= day <= max`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```c
    int day, month, year, max;
    scanf("%d %d %d", &day, &month, &year);
    if (month < 1 || month > 12) {
        printf("Invalid month\n");
    } else {
        if (month == 2) {
            if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
                max = 29;
            else
                max = 28;
        } else if (month == 4 || month == 6 || month == 9 || month == 11) {
            max = 30;
        } else {
            max = 31;
        }
        if (day >= 1 && day <= max)       // اليوم داخل حدود الشهر
            printf("%02d/%02d/%d is valid\n", day, month, year);
        else
            printf("Invalid day: month %d has %d days\n", month, max);
    }
    return 0;
}
```

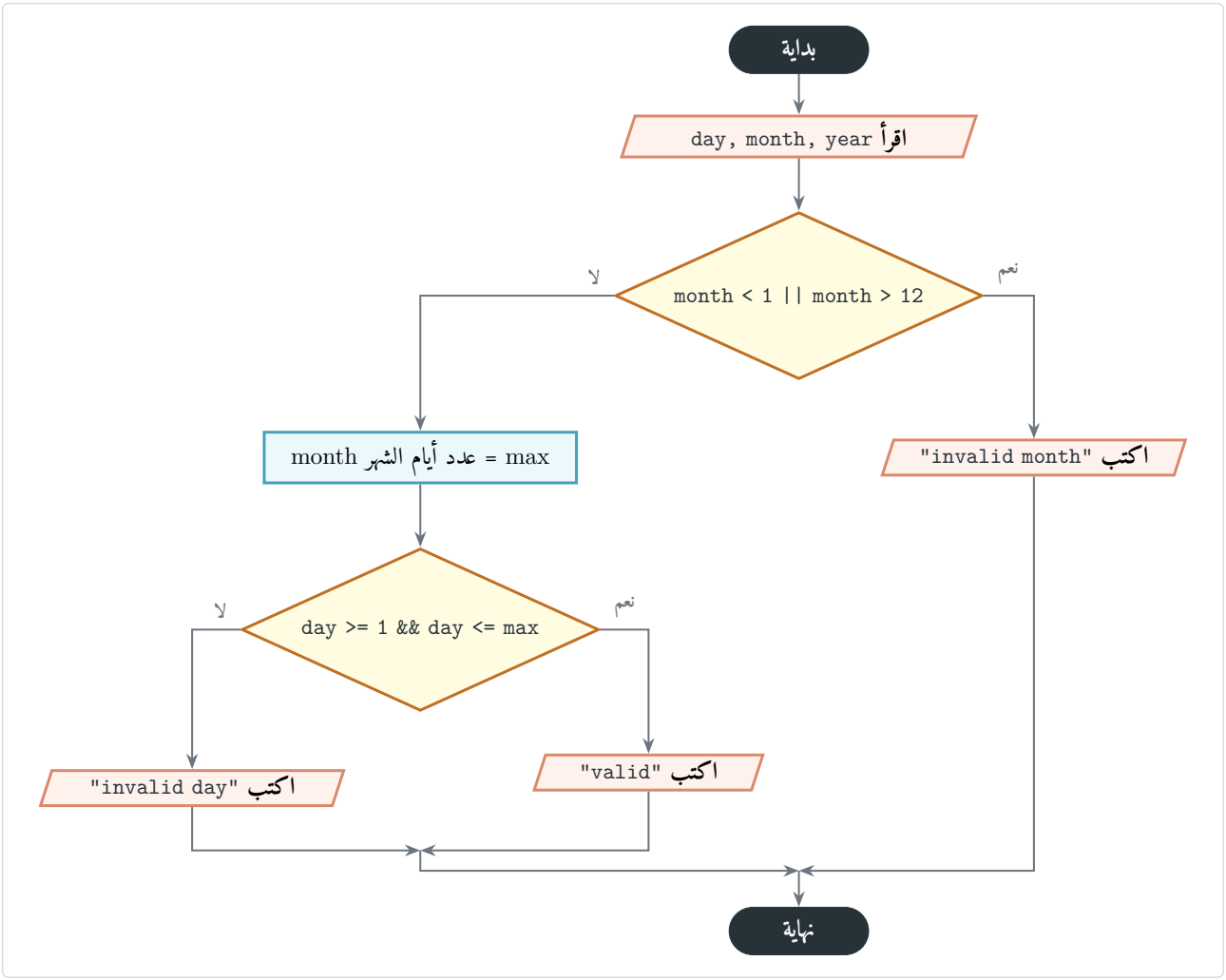
المدخلات

```
29 2 2027
```

نتيجة التنفيذ

```
Invalid day: month 2 has 28 days
```

المخطط الإنسيابي (Organigramme)



تمرين 2.24 اليوم الموالي

تجارة إلكترونية ★★ متوسط

التوصيل يتم دائمًا في اليوم الموالي للطلب. اكتب برنامجًا يقرأ تاريخ الطلب (صحيحًا) ويعرض تاريخ التوصيل، مع الانتقال إلى الشهر والسنة التالية عند الحاجة.

خطوات الحل

1) نحسب عدد أيام الشهر `max`.
2) إذا كان `day < max` نزيد اليوم فقط.
3) وإلا: اليوم 1، والشهر يزيد؛ وإذا كان الشهر 12 يصبح 1 وتزيد السنة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int day, month, year, max;
    scanf("%d %d %d", &day, &month, &year);
    if (month == 2)
        max = ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) ? 29 : 28;
```

```
    else if (month == 4 || month == 6 || month == 9 || month == 11)
        max = 30;
    else
        max = 31;
    if (day < max) {
        day = day + 1;                  // نفس الشهر
    } else {
        day = 1;                       // بداية شهر جديد
        if (month == 12) {
            month = 1;                  // بداية سنة جديدة
            year = year + 1;
        } else {
            month = month + 1;
        }
    }
    printf("Delivery date: %02d/%02d/%d\n", day, month, year);
    return 0;
}
```

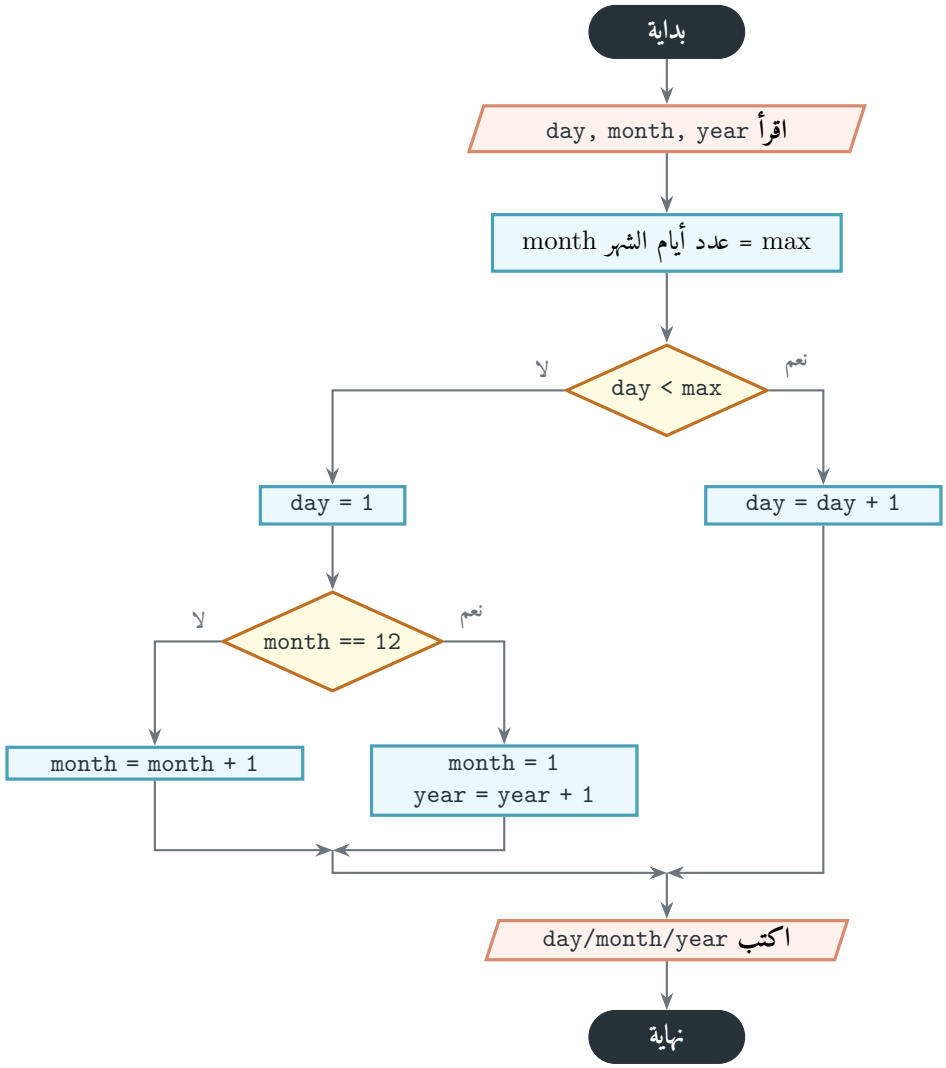
المدخلات

```
31 12 2026
```

نتيجة التنفيذ

```
Delivery date: 01/01/2027
```

المخطط الإنسيابي (Organigramme)



تمرين 2.25 خلية مصفوفة الإلتباس

علم البيانات ★★ متوسط

نموذج يتنبأ هل سيلغي الزبون طلبيته (1) أم لا (0). اكتب برنامجًا يقرأ القيمة **الحقيقية** والقيمة **المتنبأ بها** ويحدد نوع النتيجة: TP أو TN أو FP أو FN مع شرح.

خطوات الحل

1) إذا تساوت القيمتان فالتنبؤ صحيح: TP إذا كانت 1 وTN إذا كانت 0.
2) وإلا فهو خطأ: FP إذا تنبأ بـ 1، وFN إذا تنبأ بـ 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int actual, pred;
    scanf("%d %d", &actual, &pred);
    if (actual == pred) {            // تنبؤ صحيح
```

```c
        if (pred == 1)
            printf("TP: cancellation correctly detected\n");
        else
            printf("TN: normal order correctly detected\n");
    } else {                            // تنبؤ خاطئ
        if (pred == 1)
            printf("FP: false alarm\n");
        else
            printf("FN: missed cancellation\n");
    }
    return 0;
}
```

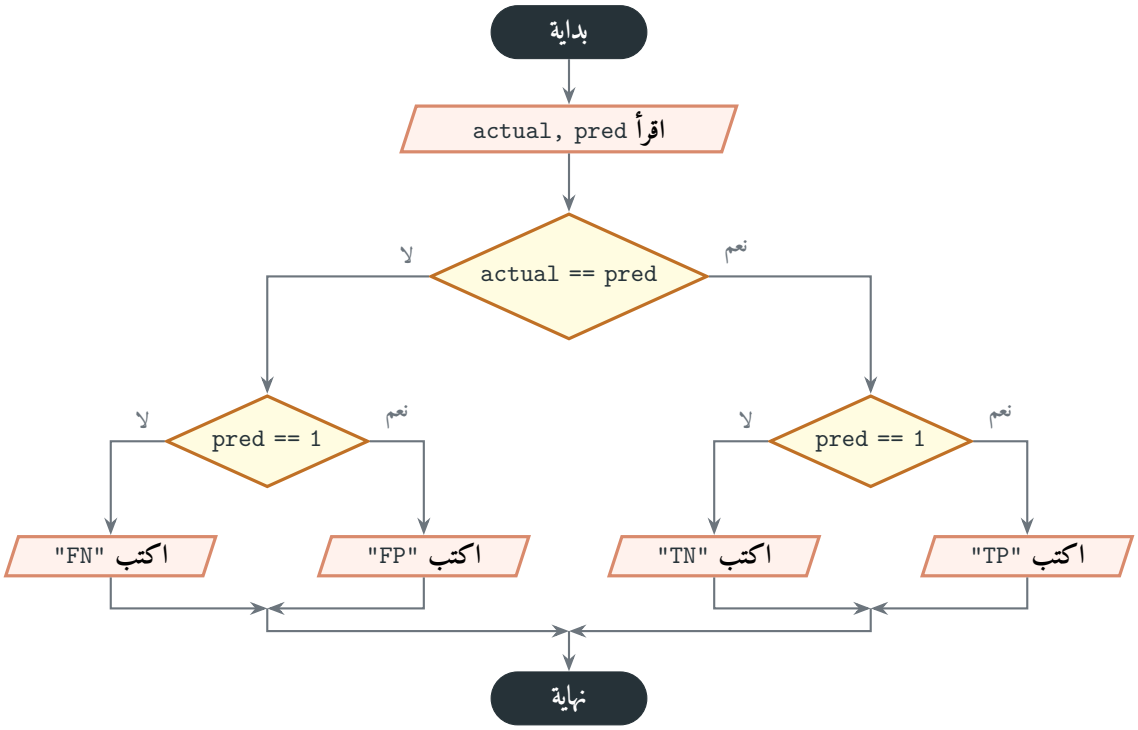
المدخلات

```
0 1
```

نتيجة التنفيذ

```
FP: false alarm
```

المخطط الإنسيابي (Organigramme)



تمرين 2.26 قاعدة توصية

علم البيانات ★★ متوسط

استخرجنا من بيانات المشتريات القاعدة: من اشترى **هاتفًا** ولم يشترِ **غلافًا** نقترح عليه غلافًا؛ من اشترى غلافًا وشاحنًا معًا نقترح سماعات؛ وإلا لا توصية. اكتب برنامجًا يقرأ ثلاث قيم (0 أو 1) للهاتف والغلاف والشاحن ويعرض التوصية.

خطوات الحل

1) نقرأ phone وcover وcharger.
2) القاعدة الأولى: phone && !cover.
3) القاعدة الثانية: cover && charger.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int phone, cover, charger;
    scanf("%d %d %d", &phone, &cover, &charger);
    // 0 يساوي cover صحيح إذا كان cover! تعني النفي: !
    if (phone && !cover)
        printf("Recommend: phone cover\n");
    else if (cover && charger)
        printf("Recommend: earphones\n");
    else
        printf("No recommendation\n");
    return 0;
}
```

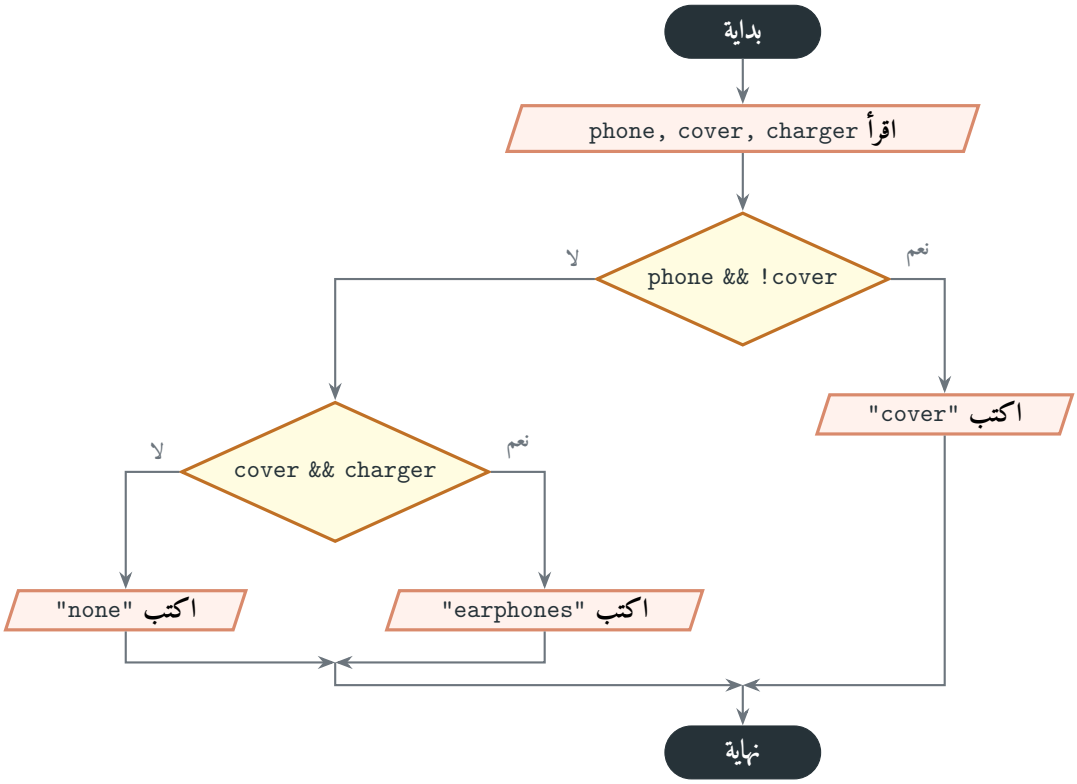
المدخلات

```
1 0 1
```

نتيجة التنفيذ

```
Recommend: phone cover
```

المخطط الإنسيابي (Organigramme)



تمرين 2.27 منطقة التوصيل

تجارة إلكترونية ★★ متوسط

تقسم شركة التوصيل الولايات حسب رقمها: الولاية 16 (الجزائر) منطقة 0 بـ 300 دج، الولايات 9 و35 و42 منطقة 1 بـ 500 دج، الولايات من 1 إلى 30 الباقية منطقة 2 بـ 700 دج، ومن 31 إلى 58 منطقة 3 بـ 900 دج. اكتب برنامجًا يحدد المنطقة والسعر.

خطوات الحل

1) نتحقق أولًا من صحة الرقم (1 إلى 58).
2) نختبر الحالات الخاصة قبل المجالات العامة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int wilaya, zone, price;
    scanf("%d", &wilaya);
    if (wilaya < 1 || wilaya > 58) {
        printf("Invalid wilaya code\n");
    } else {
        // الحالات الخاصة أولًا، ثم المجالات العامة
        if (wilaya == 16) {
            zone = 0; price = 300;
        } else if (wilaya == 9 || wilaya == 35 || wilaya == 42) {
            zone = 1; price = 500;
        } else if (wilaya <= 30) {
            zone = 2; price = 700;
        } else {
            zone = 3; price = 900;
        }
        printf("Wilaya %d: zone %d, %d DA\n", wilaya, zone, price);
    }
    return 0;
}
```

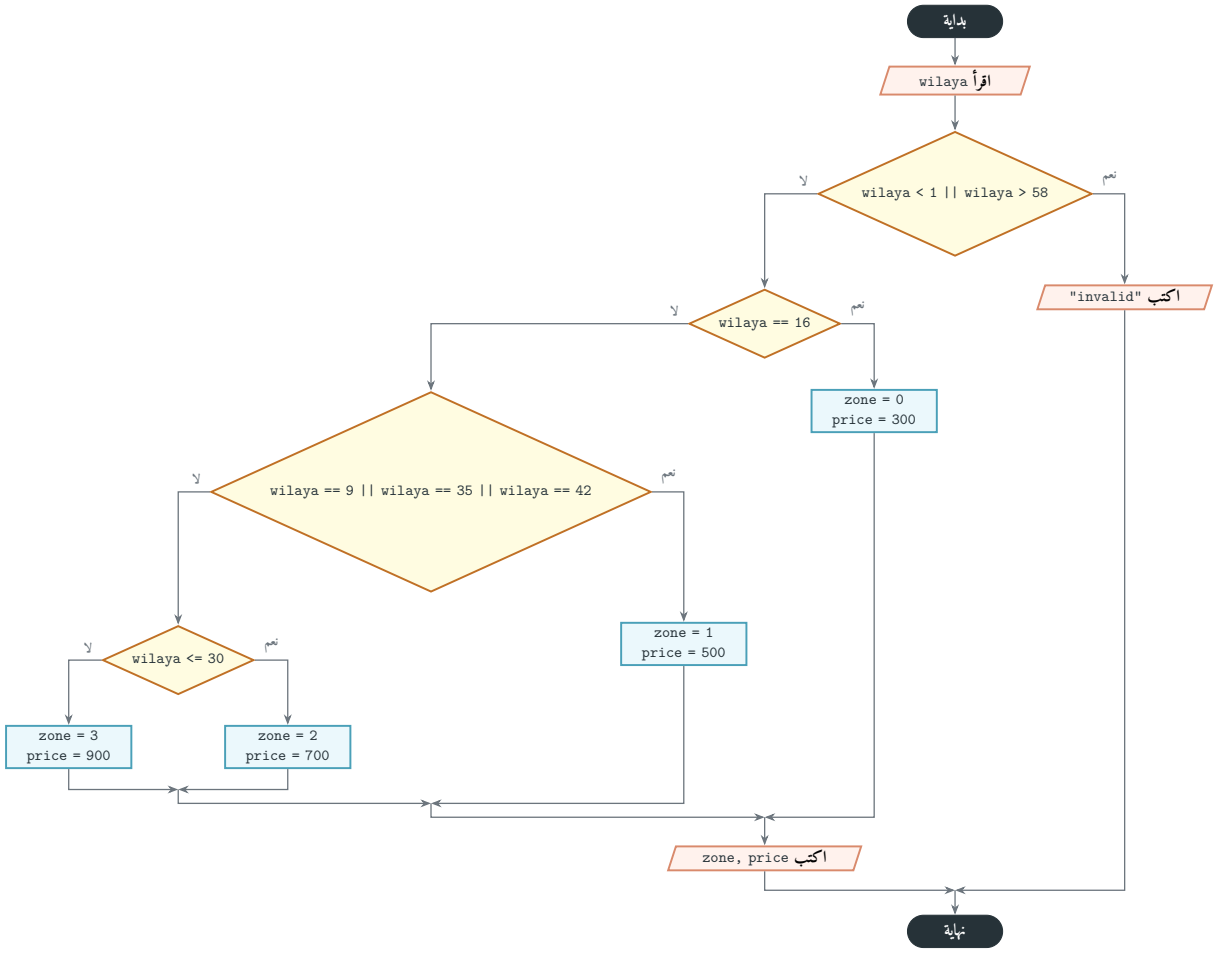
المدخلات

```
35
```

نتيجة التنفيذ

```
Wilaya 35: zone 1, 500 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.28 عرض 3 بثمن 2

تجارة إلكترونية ★★ متوسط

عرض ترويجي: كل **3 وحدات** من نفس المنتج يدفع الزبون ثمن 2 فقط. وإذا اشترى 10 وحدات أو أكثر يحصل أيضًا على تخفيض إضافي 5% على المبلغ. اكتب برنامجًا يحسب المبلغ والتوفير.

خطوات الحل

1) الوحدات المجانية: `free = quantity / 3`.
2) المبلغ: `(quantity - free) * price`.
3) إذا كان `quantity >= 10` نطبق 5% إضافية.
4) التوفير: `quantity * price - total`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int quantity, free;
    double price, total;
    scanf("%d %lf", &quantity, &price);
    free = quantity / 3;                    // وحدة مجانية لكل 3
```

```c
    total = (quantity - free) * price;
    if (quantity >= 10)
        total = total * 0.95;          // تخفيض إضافي للكميات الكبيرة
    printf("Free units = %d\n", free);
    printf("Total = %.2f DA\n", total);
    printf("Saved = %.2f DA\n", quantity * price - total);
    return 0;
}
```

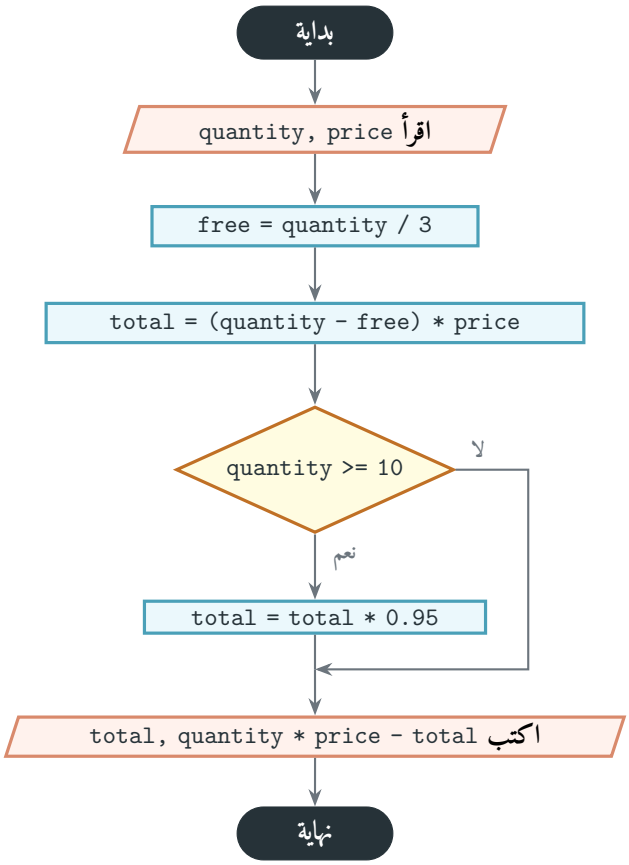
المدخلات

```
11 850
```

نتيجة التنفيذ

```
Free units = 3
Total = 6460.00 DA
Saved = 2890.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.29 تداخل فترتي ترويج

تجارة إلكترونية ★★ متوسط

لدينا حملتان ترويجيتان، لكل منهما يوم بداية ويوم نهاية (أرقام أيام السنة). اكتب برنامجًا يحدد إن كانت الحملتان **متداخلتين**، ويعرض فترة التداخل وطولها.

خطوات الحل

1) بداية التداخل: الأكبر بين البدايتين `overlapStart`.
2) نهاية التداخل: الأصغر بين النهايتين `overlapEnd`.
3) التداخل موجود إذا كان `overlapStart <= overlapEnd`، وطوله `overlapEnd - overlapStart + 1`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int a1, b1, a2, b2, overlapStart, overlapEnd;
    scanf("%d %d %d %d", &a1, &b1, &a2, &b2);
    overlapStart = (a1 > a2) ? a1 : a2;       // البداية الأكبر
    overlapEnd = (b1 < b2) ? b1 : b2;         // النهاية الأصغر
    if (overlapStart <= overlapEnd)
        printf("Overlap: days %d to %d (%d days)\n", overlapStart, overlapEnd, overlapEnd - overlapStart
    else
        printf("No overlap\n");
    return 0;
}
```

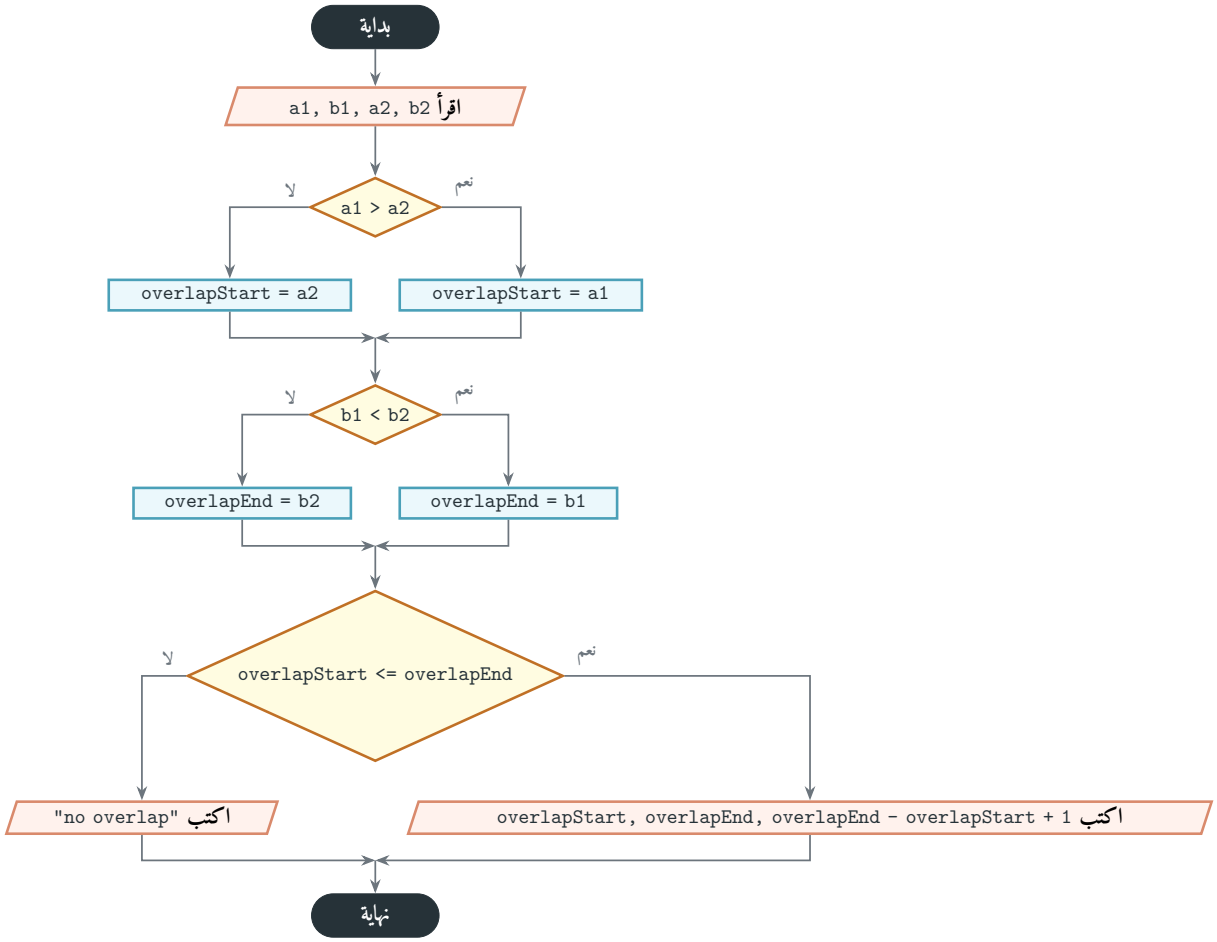
المدخلات

```
40 75
60 90
```

نتيجة التنفيذ

```
Overlap: days 60 to 75 (16 days)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.30 آلة حاسبة للأسعار

تجارة إلكترونية ★★ متوسط

يستعمل موظف المخزن آلة حاسبة بسيطة: يقرأ عددين وعاملًا (+ أو - أو * أو / أو % للنسبة المئوية: a% من b). اكتب البرنامج باستعمال `switch` مع منع القسمة على صفر.

خطوات الحل

1) نقرأ `left` ثم العامل `op` ثم `right`.
2) لكل عامل حالة في `switch`.
3) في حالة `/` نتحقق أن `right != 0`.
4) `%` تعطي `left * right / 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double left, right, result = 0;
    char op;
    int ok = 1;                         // 1 إذا كانت العملية صالحة
    scanf("%lf %c %lf", &left, &op, &right);
```

```c
    switch (op) {
        case '+': result = left + right; break;
        case '-': result = left - right; break;
        case '*': result = left * right; break;
        case '/':
            if (right == 0) ok = 0;        // لا نقسم على صفر
            else result = left / right;
            break;
        case '%': result = left * right / 100; break;    // right بالمئة من left
        default: ok = 0;
    }
    if (ok)
        printf("%.2f %c %.2f = %.2f\n", left, op, right, result);
    else
        printf("Error: invalid operation\n");
    return 0;
}
```

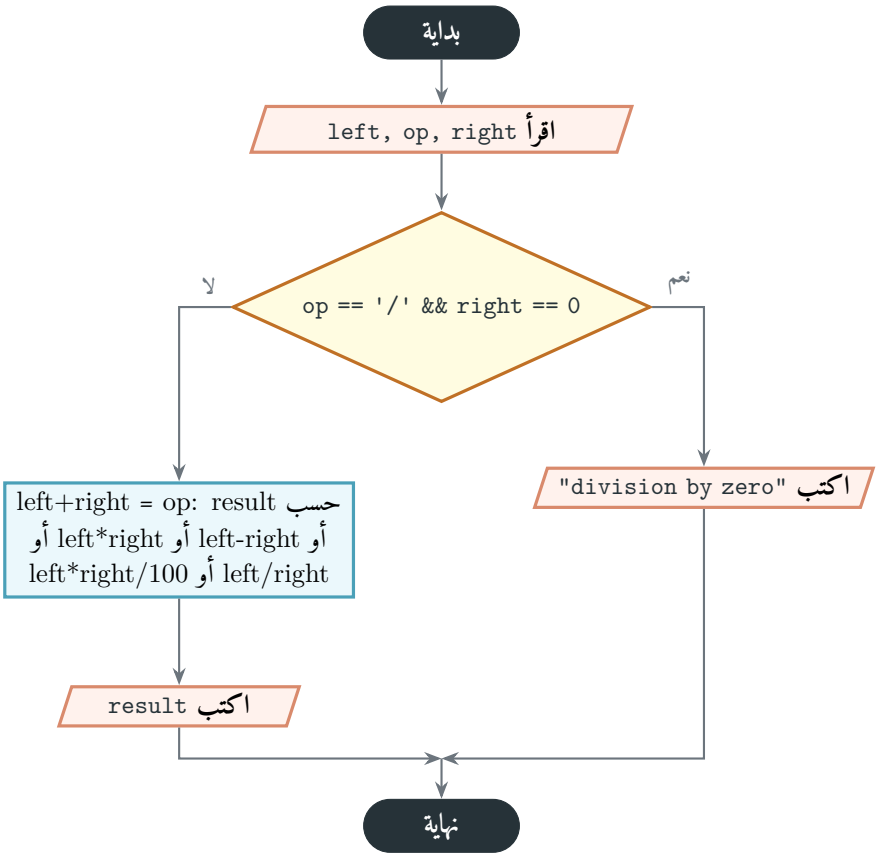
المدخلات

```
15 % 8400
```

نتيجة التنفيذ

```
15.00 % 8400.00 = 1260.00
```

المخطط الإنسيابي (Organigramme)



تمرين 2.31 تأجيل التوصيل في عطلة نهاية الأسبوع — تجارة إلكترونية — ★★ متوسط

لا توصيل يومي **الجمعة والسبت**. أيام الأسبوع مرقمة من 0 (الأحد) إلى 6 (السبت). اكتب برنامجًا يقرأ يوم الطلب ومدة التحضير بالأيام، ويحدد يوم التوصيل؛ إذا وقع في الجمعة أو السبت يؤجَّل إلى الأحد.

خطوات الحل

1) يوم التوصيل: `deliveryDay = (day + prep) % 7`.
2) إذا كان `deliveryDay == 5` نضيف يومين، وإذا كان `deliveryDay == 6` نضيف يومًا، ثم `% 7`.
3) نعرض اسم اليوم باستعمال `switch`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int day, prep, deliveryDay;
    scanf("%d %d", &day, &prep);
    deliveryDay = (day + prep) % 7;            // اليوم الطبيعي للتوصيل
    if (deliveryDay == 5 || deliveryDay == 6) {          // الجمعة أو السبت
        printf("Weekend: postponed to Sunday\n");
        deliveryDay = 0;
    }
    printf("Delivery day: ");
    switch (deliveryDay) {
        case 0: printf("Sunday\n"); break;
        case 1: printf("Monday\n"); break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n"); break;
    }
    return 0;
}
```

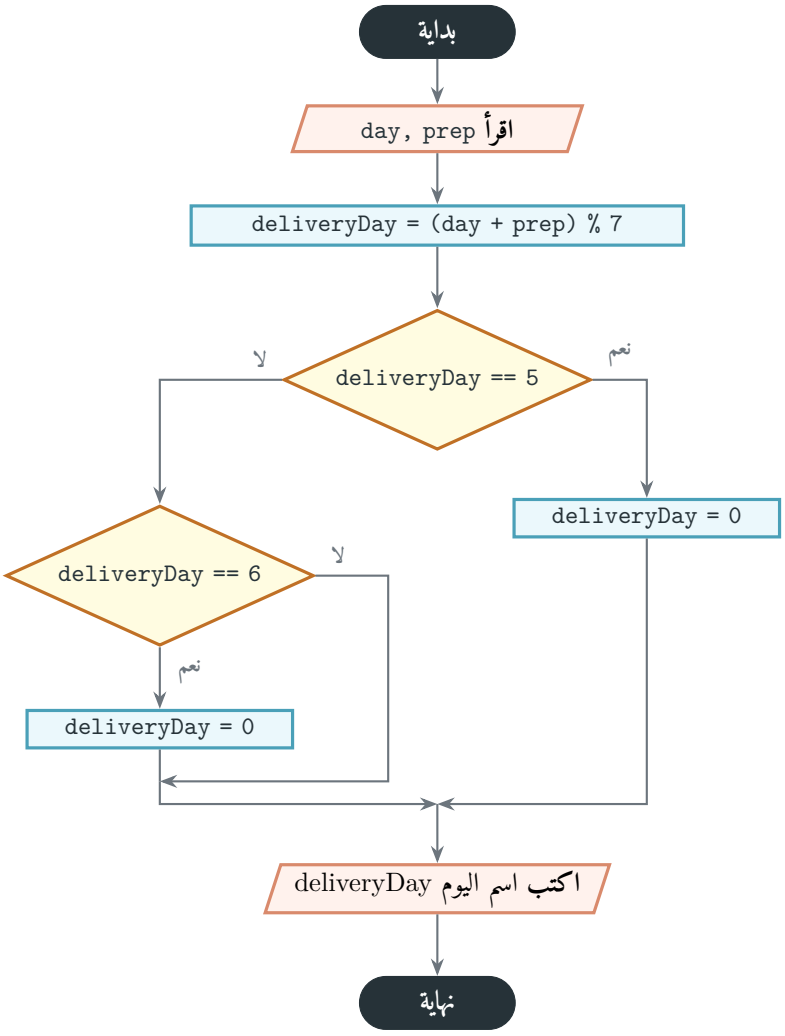
المدخلات

```
3 2
```

نتيجة التنفيذ

```
Weekend: postponed to Sunday
Delivery day: Sunday
```

المخطط الإنسيابي (Organigramme)



تمرين 2.32 صلاحية البطاقة البنكية

تجارة إلكترونية ★★ متوسط

قبل قبول الدفع نتحقق أن البطاقة **غير منتهية الصلاحية**: تاريخ انتهائها (شهر، سنة) يجب أن يكون بعد التاريخ الحالي أو في نفس الشهر. اكتب برنامجًا يقرأ التاريخين ويحكم، ويعرض عدد الأشهر المتبقية.

خطوات الحل

1) نحول كل تاريخ إلى عدد أشهر: `y * 12 + m`.
2) البطاقة صالحة إذا كان `exp >= now`.
3) الأشهر المتبقية: `exp - now`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int em, ey, cm, cy, exp, now;
    scanf("%d %d", &em, &ey);          // تاريخ الانتهاء
```

```c
    scanf("%d %d", &cm, &cy);          // التاريخ الحالي
    // تحويل التاريخ إلى عدد أشهر يسهل المقارنة
    exp = ey * 12 + em;
    now = cy * 12 + cm;
    if (exp >= now)
        printf("Card valid (%d months left)\n", exp - now);
    else
        printf("Card expired\n");
    return 0;
}
```

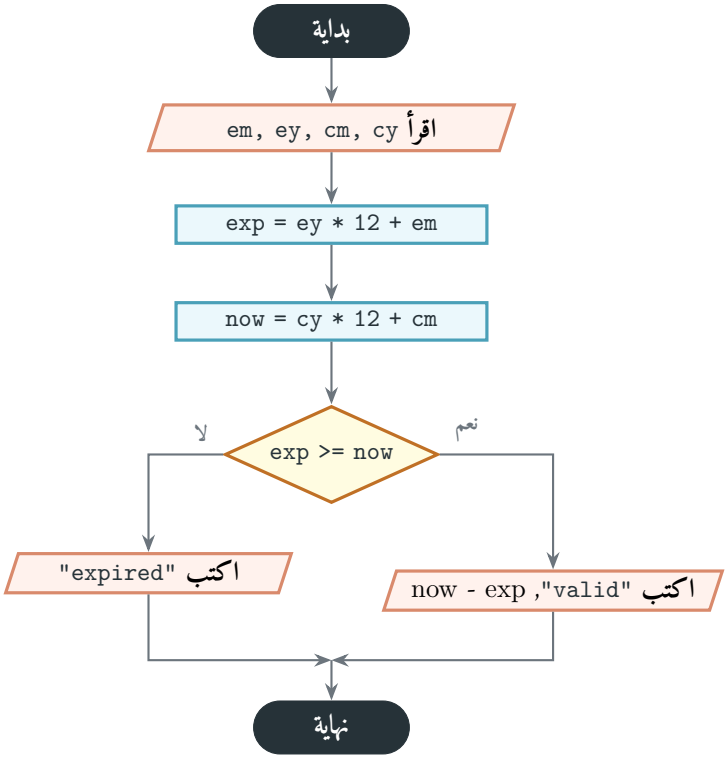
المدخلات

```
3 2027
11 2026
```

نتيجة التنفيذ

```
Card valid (4 months left)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.33 هدية حسب حجم السلة

تجارة إلكترونية ★★ متوسط

تصنف السلات حسب عدد القطع: صغيرة (1-3)، متوسطة (4-9)، كبيرة (10 فأكثر). الهدية: لا شيء للصغيرة، كيس قماشي للمتوسطة، وللكبيرة كيس + قسيمة 500 دج إذا تجاوز المبلغ 20000 دج. اكتب برنامجًا يعرض الصنف والهدية.

خطوات الحل

1) نحدد الصنف بالمجالات.
2) في صنف الكبيرة نضيف شرطًا داخليًا على المبلغ (شرط متداخل).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbItems;
    double amount;
    scanf("%d %lf", &nbItems, &amount);
    if (nbItems >= 10) {
        printf("Large basket: tote bag");
        if (amount > 20000)           // شرط متداخل
            printf(" + 500 DA voucher");
        printf("\n");
    } else if (nbItems >= 4) {
        printf("Medium basket: tote bag\n");
    } else {
        printf("Small basket: no gift\n");
    }
    return 0;
}
```

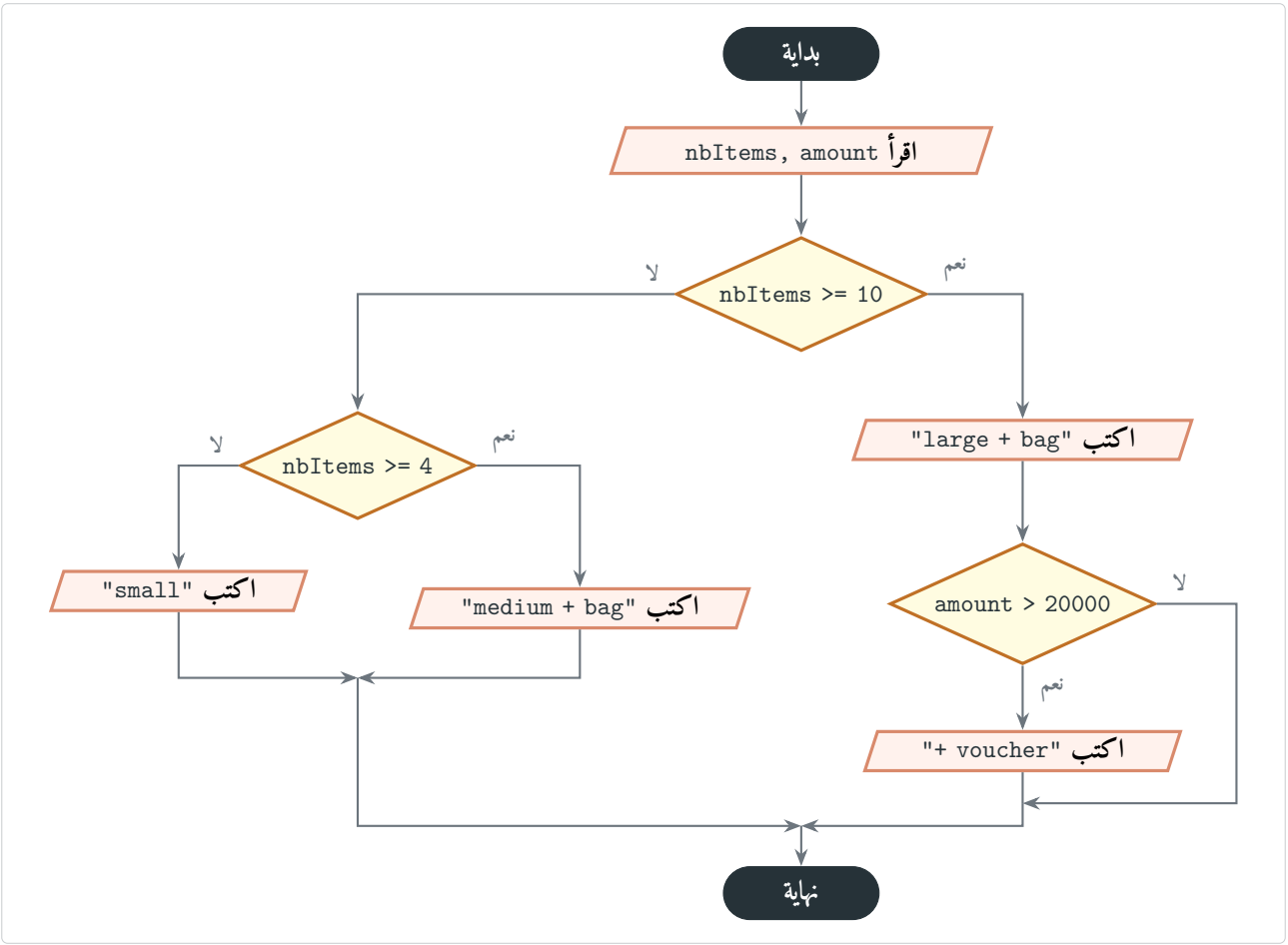
المدخلات

```
12 18500
```

نتيجة التنفيذ

```
Large basket: tote bag
```

المخطط الإنسيابي (Organigramme)



تمرين 2.34 تطور مؤشر الأداء

علم البيانات ★★ متوسط

نقارن قيمة مؤشر (مثل عدد الطلبيات) بين أسبوعين. نعتبر التغير **مستقرًا** إذا كانت النسبة بين -2% و+2%، وإلا **ارتفاعًا** أو **انخفاضًا**؛ ونعتبره **حادًا** إذا تجاوز 20% بالقيمة المطلقة. اكتب البرنامج.

خطوات الحل

1) نسبة التغير: `change = (now - before) / before * 100`.
2) القيمة المطلقة `ac` يدويًا.
3) إذا كان `ac <= 2` مستقر، وإلا نحدد الاتجاه، ثم نختبر `ac > 20`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double before, now, change, ac;
    scanf("%lf %lf", &before, &now);
    change = (now - before) / before * 100;     // التغير بالمئة
    ac = (change < 0) ? -change : change;                // القيمة المطلقة
    printf("Change = %.1f %% : ", change);
    if (ac <= 2) {
```

```c
        printf("stable\n");
    } else {
        if (change > 0) printf("increase");
        else        printf("decrease");
        if (ac > 20)
            printf(" (sharp!)");          // تنبيه للتغير الكبير
        printf("\n");
    }
    return 0;
}
```

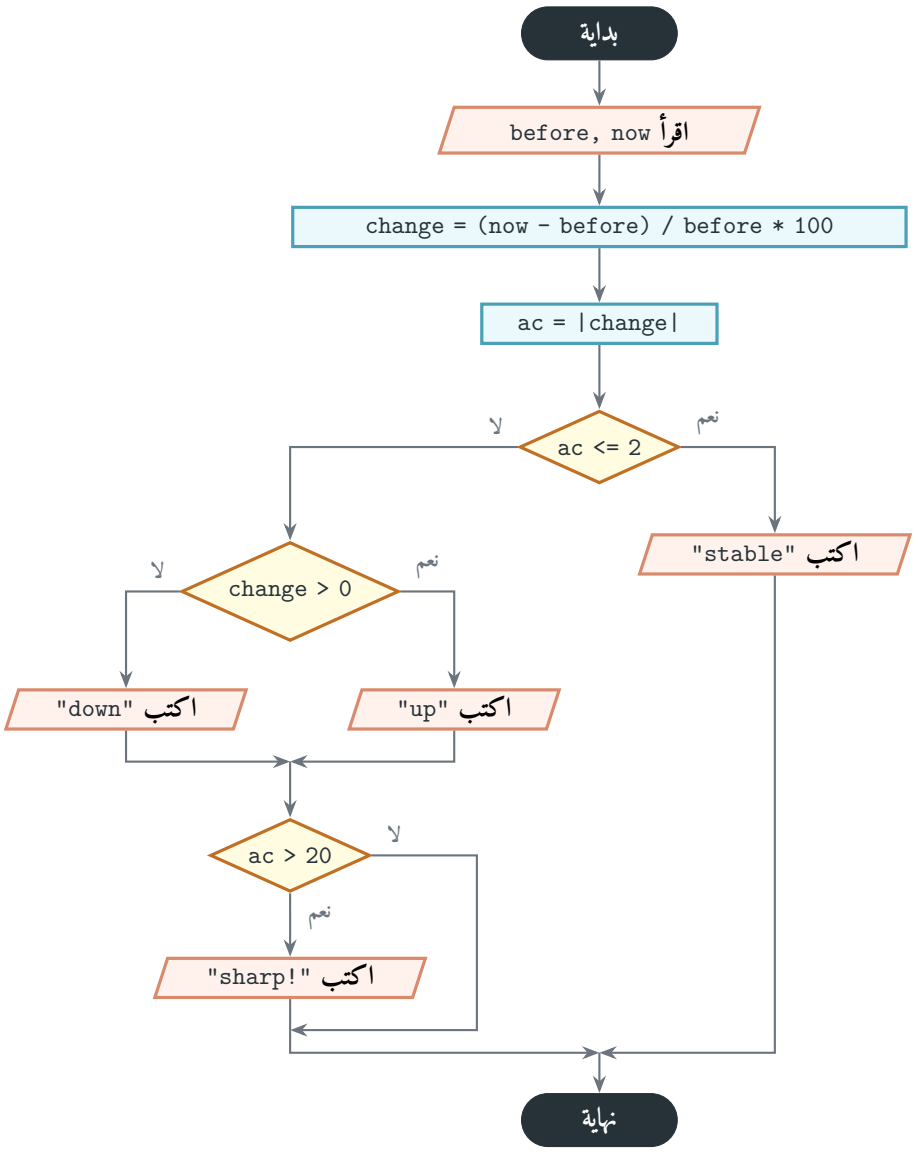
المدخلات

```
1250 980
```

نتيجة التنفيذ

```
Change = -21.6 % : decrease (sharp!)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.35 السعر الأمثل

علم البيانات ★★ متوسط

قدّر محلل أن الطلب يتبع Q = demandMax - slope*price. رقم الأعمال revenue = price * (demandMax - slope*price) يبلغ أقصاه عند price* = demandMax / (2b). لكن السعر يجب أن يبقى بين pmin وpmax. اكتب برنامجًا يحسب السعر الأمثل المسموح ورقم الأعمال الموافق.

خطوات الحل

1) price = demandMax / (2 * slope).
2) إذا كان price < pmin نأخذ pmin، وإذا كان price > pmax نأخذ pmax.
3) revenue = price * (demandMax - slope * price).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double demandMax, slope, pmin, pmax, price, revenue;
    scanf("%lf %lf %lf %lf", &demandMax, &slope, &pmin, &pmax);
    price = demandMax / (2 * slope);                    // قمة القطع المكافئ
    printf("Unconstrained optimum = %.2f DA\n", price);
    if (price < pmin)
        price = pmin;                                   // نحصر السعر في المجال المسموح
    else if (price > pmax)
        price = pmax;
    revenue = price * (demandMax - slope * price);
    printf("Chosen price = %.2f DA\n", price);
    printf("Revenue = %.2f DA\n", revenue);
    return 0;
}
```

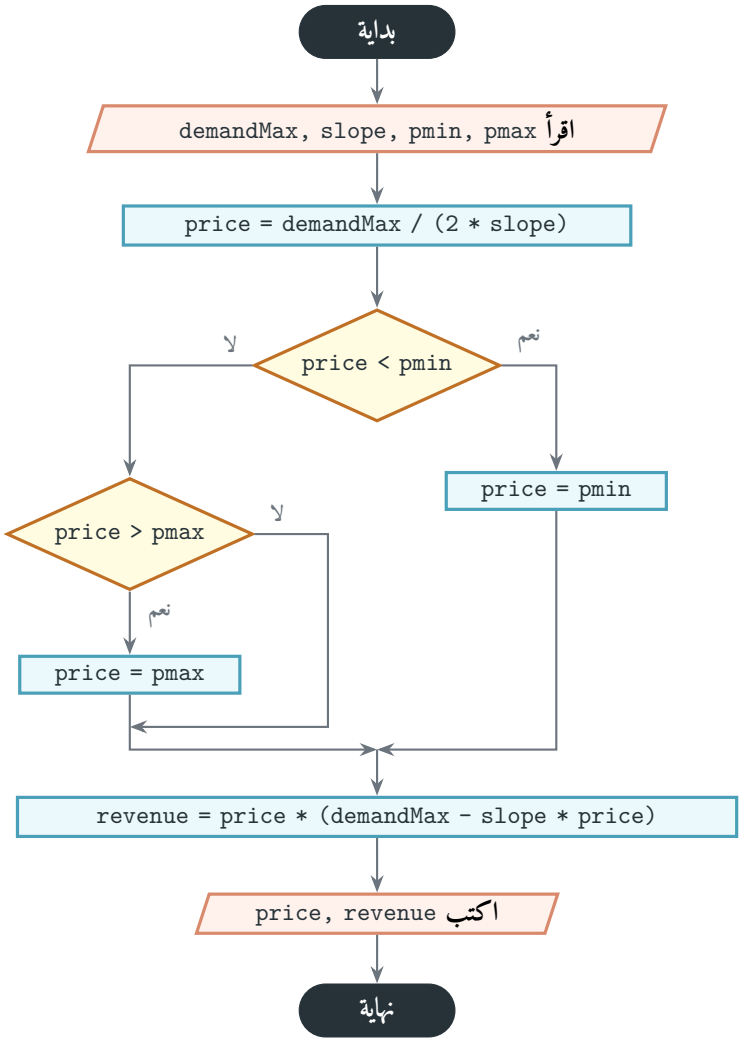
المدخلات

```
5000 2 1500 3000
```

نتيجة التنفيذ

```
Unconstrained optimum = 1250.00 DA
Chosen price = 1500.00 DA
Revenue = 3000000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.36 الوسيط بين ثلاث قيم

علم البيانات ★★ متوسط

الوسيط أقل تأثرًا بالقيم الشاذة من المتوسط. اكتب برنامجًا يقرأ ثلاثة مبالغ طلبيات ويعرض وسيطها (القيمة الوسطى) ومتوسطها للمقارنة، دون ترتيب كامل.

خطوات الحل

amount3 : `(amount2 <= amount1 && amount1 <= amount3) || (amount3 <= amount1 && amount1 <= amount2)`.

2) نفس الاختبار لـ `amount2`، وإلا فالوسيط `amount3`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount1, amount2, amount3, med;
    scanf("%lf %lf %lf", &amount1, &amount2, &amount3);
    if ((amount2 <= amount1 && amount1 <= amount3) || (amount3 <= amount1 && amount1 <= amount2))
        med = amount1;                          // amount1 بين الاثنين الآخرين
```

```
    else if ((amount1 <= amount2 && amount2 <= amount3) || (amount3 <= amount2 && amount2 <= amount1))
        med = amount2;
    else
        med = amount3;
    printf("Median = %.2f\n", med);
    printf("Mean   = %.2f (pulled by the outlier)\n", (amount1 + amount2 + amount3) / 3);
    return 0;
}
```

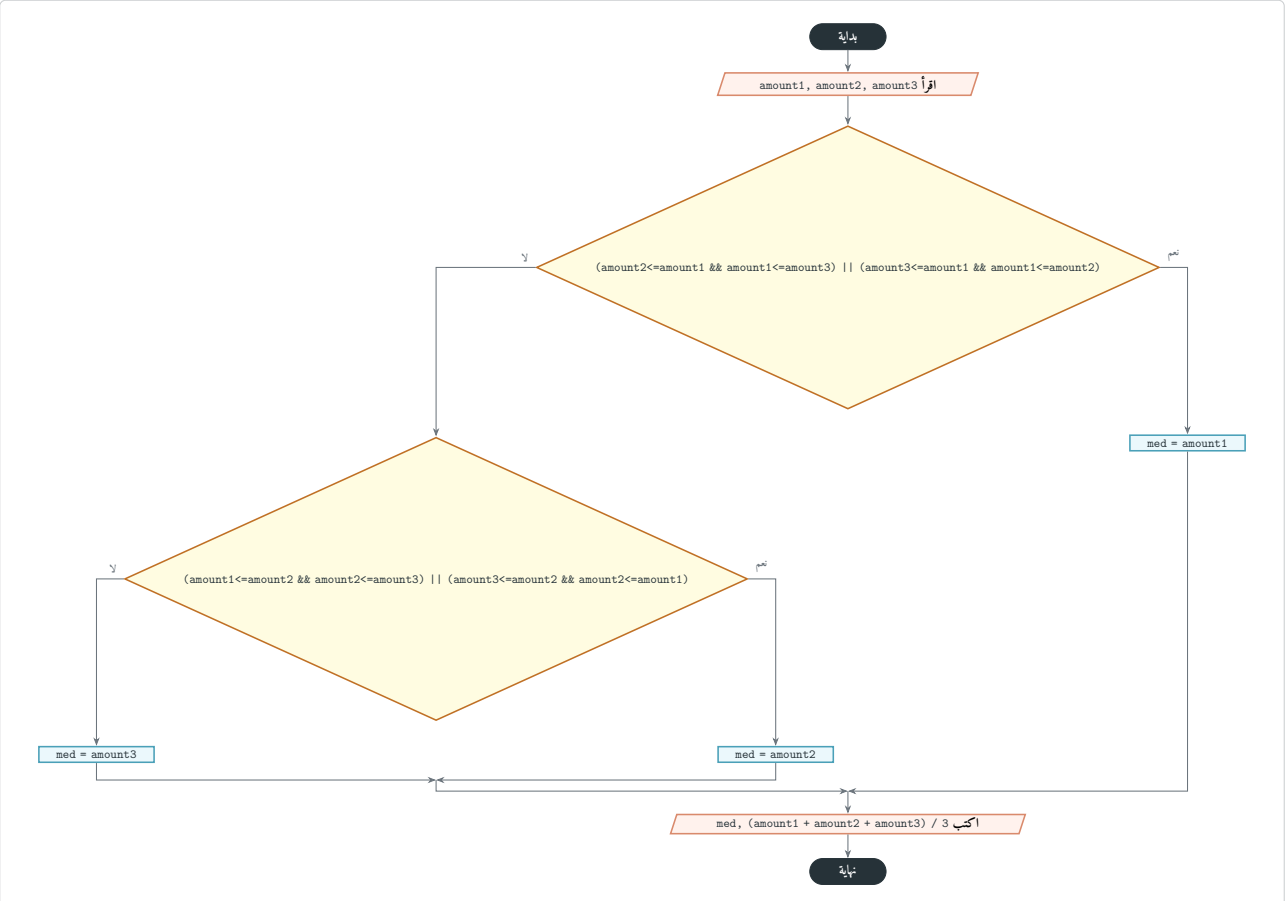
المدخلات

```
1200 98000 1500
```

نتيجة التنفيذ

```
Median = 1500.00
Mean   = 33566.67 (pulled by the outlier)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.37 تسعير حسب ساعة الطلب

تجارة إلكترونية ★★ متوسط

تطبق منصة توصيل الوجبات أسعارًا حسب الساعة: من 11 إلى 13 ومن 19 إلى 21 (ساعات الذروة) زيادة 20%، من 15 إلى 17 (ساعة سعيدة) تخفيض 25%، ومن 0 إلى 6 زيادة ليلية 30%. اكتب برنامجًا يقرأ الساعة والسعر.

خطوات الحل

1) نحدد المعامل `factor` حسب المجال.
2) مجالا الذروة يُجمعان بـ `||`.
3) السعر النهائي `price * factor`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int hour;
    double price, factor;
    scanf("%d %lf", &hour, &price);
    if ((hour >= 11 && hour <= 13) || (hour >= 19 && hour <= 21))
        factor = 1.2;                       // ساعات الذروة
    else if (hour >= 15 && hour <= 17)
        factor = 0.75;                      // الساعة السعيدة
    else if (hour >= 0 && hour <= 6)
        factor = 1.3;                       // الليل
    else
        factor = 1.0;                       // السعر العادي
    printf("Hour %d: factor %.2f -> %.2f DA\n", hour, factor, price * factor);
    return 0;
}
```

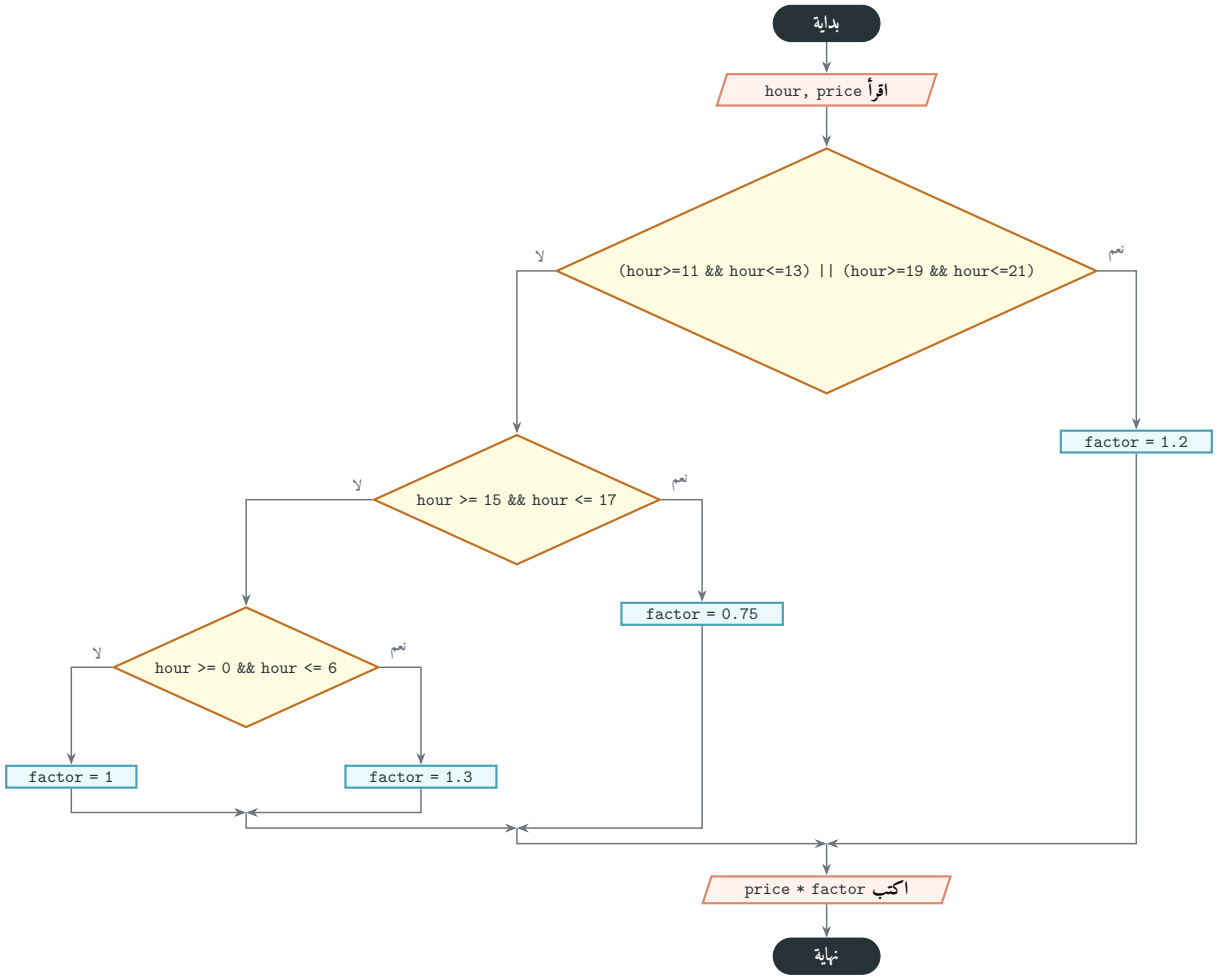
المدخلات

```
20 1400
```

نتيجة التنفيذ

```
Hour 20: factor 1.20 -> 1680.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.38 تصنيف المشاعر

علم البيانات

متوسط

يعطي نموذج تحليل المشاعر لكل تعليق درجة بين -1 و1 ودرجة ثقة بين 0 و1. اكتب برنامجًا يصنف التعليق: إذا كانت الثقة أقل من 0.6 فهو **غير مؤكد**، وإلا: إيجابي (درجة > 0.25)، سلبي (< -0.25)، أو محايد.

خطوات الحل

1) نختبر الثقة أولًا: إنها شرط مسبق.
2) ثم نصنف حسب الدرجة بثلاث حالات.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double score, conf;
    scanf("%lf %lf", &score, &conf);
    if (conf < 0.6) {
        printf("Uncertain (confidence %.2f)\n", conf);
    } else if (score > 0.25) {
        printf("Positive review\n");
```

```c
    } else if (score < -0.25) {
        printf("Negative review\n");       // يرسل إلى خدمة الزبائن
    } else {
        printf("Neutral review\n");
    }
    return 0;
}
```

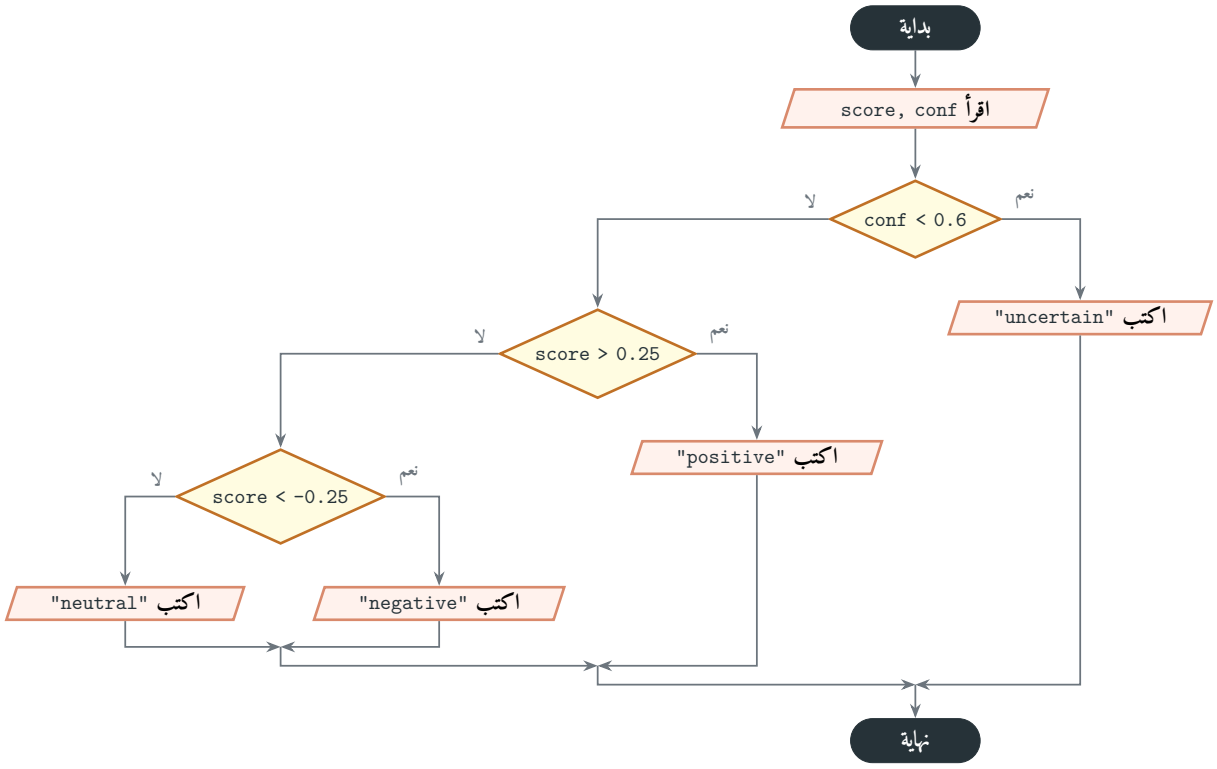
المدخلات

```
-0.62 0.81
```

نتيجة التنفيذ

```
Negative review
```

المخطط الإنسيابي (Organigramme)



تمرين 2.39 كشف الإحتيال بالقواعد

علم البيانات متوسط

نعطي كل عملية دفع **درجة خطر**: +40 إذا تجاوز المبلغ 100000 دج، +30 إذا كان البلد مختلفًا عن بلد الحساب، +20 إذا كانت الساعة بين 0 و5، +25 إذا كان الحساب عمره أقل من 7 أيام. العملية **مرفوضة** إذا بلغت الدرجة 70، و**للمراجعة** من 40. اكتب البرنامج.

خطوات الحل

1) نقرأ المبلغ، `foreign` (0 أو 1)، الساعة، وعمر الحساب.
2) كل قاعدة شرط مستقل `if` يضيف نقاطًا (ليست `else if`).

3) القرار النهائي حسب المجموع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount;
    int foreign, hour, age, risk = 0;
    scanf("%lf %d %d %d", &amount, &foreign, &hour, &age);
    // شروط مستقلة: كل قاعدة تضيف نقاطها
    if (amount > 100000)          risk = risk + 40;
    if (foreign == 1)             risk = risk + 30;
    if (hour >= 0 && hour <= 5)   risk = risk + 20;
    if (age < 7)                  risk = risk + 25;
    printf("Risk score = %d -> ", risk);
    if (risk >= 70)
        printf("REJECTED\n");
    else if (risk >= 40)
        printf("MANUAL REVIEW\n");
    else
        printf("ACCEPTED\n");
    return 0;
}
```

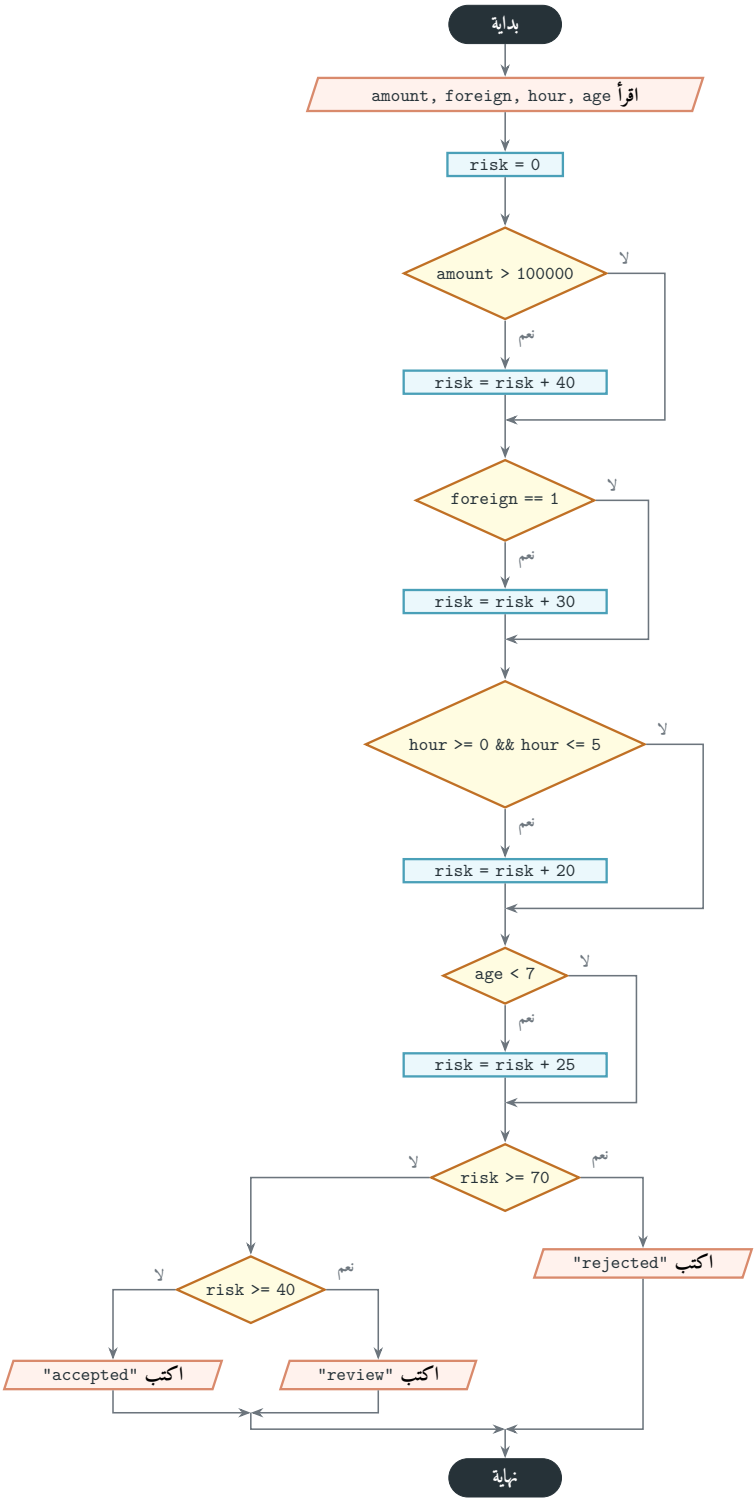
المدخلات

```
125000 1 3 30
```

نتيجة التنفيذ

```
Risk score = 90 -> REJECTED
```

المخطط الإنسيابي (Organigramme)



تمرين 2.40 الفائز في تجربة A/B

علم البيانات ★★ متوسط

نسختان A وB من صفحة منتج. اكتب برنامجًا يقرأ الزوار والمشتريات لكل نسخة، ويعلن الفائز إذا كان الفرق في معدل التحويل **0.5 نقطة على الأقل** وكان لكل نسخة 1000 زائر على الأقل؛ وإلا "غير حاسم".

خطوات الحل

1) نحسب المعدلين ra وrb.
2) شرط حجم العينة: na >= 1000 && nb >= 1000.
3) الفرق gap = ra - rb، ونقارن gap >= 0.5 أو gap <= -0.5.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double na, ca, nb, cb, ra, rb, gap;
    scanf("%lf %lf %lf %lf", &na, &ca, &nb, &cb);
    ra = ca / na * 100;                    // معدل تحويل النسخة A
    rb = cb / nb * 100;                    // معدل تحويل النسخة B
    printf("A: %.2f %%  B: %.2f %%\n", ra, rb);
    if (na < 1000 || nb < 1000) {
        printf("Sample too small\n");      // لا نحكم على عينة صغيرة
    } else {
        gap = ra - rb;
        if (gap >= 0.5)       printf("Winner: A\n");
        else if (gap <= -0.5) printf("Winner: B\n");
        else                  printf("Inconclusive\n");
    }
    return 0;
}
```

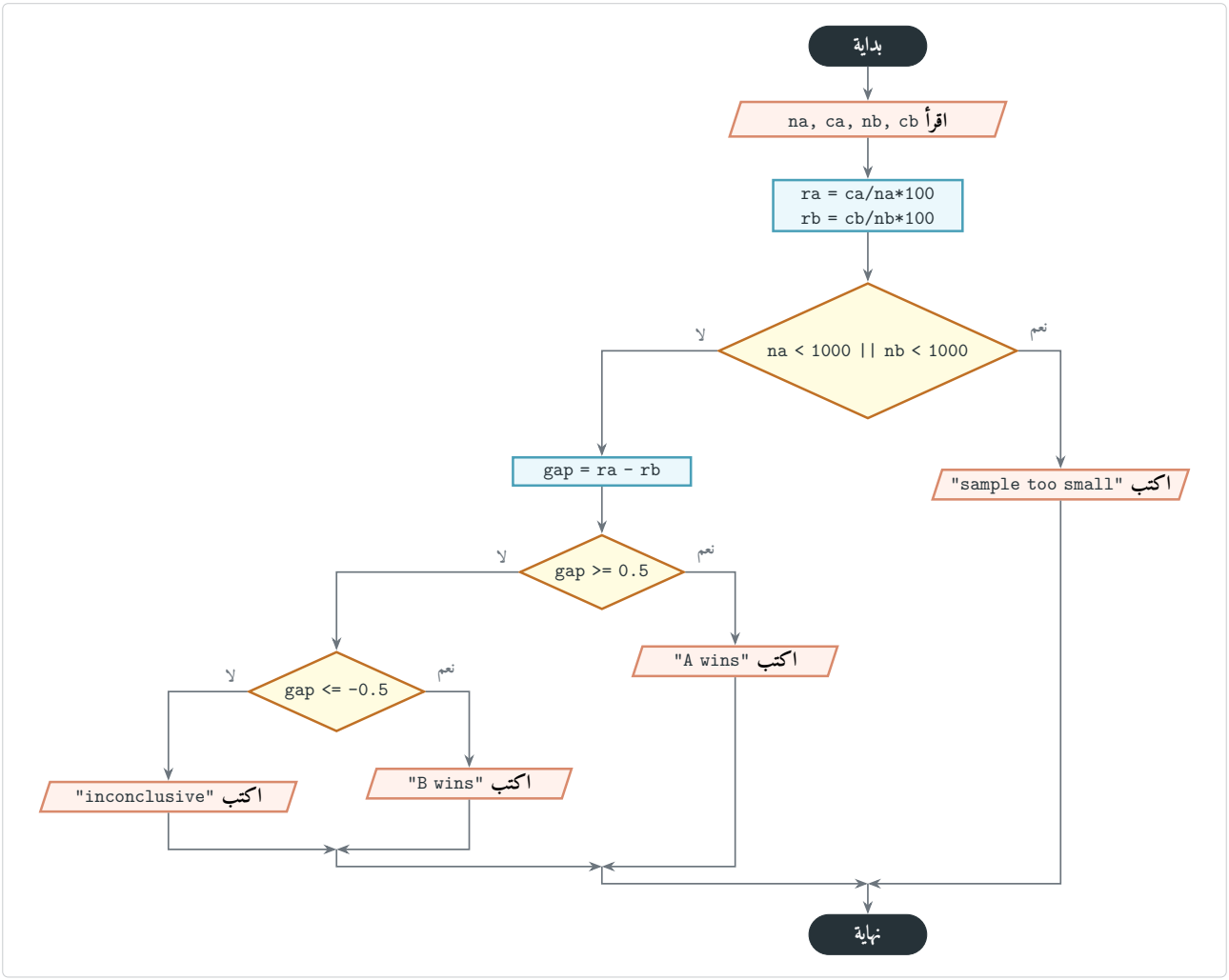
المدخلات

```
1500 60
1450 41
```

نتيجة التنفيذ

```
A: 4.00 %  B: 2.83 %
Winner: A
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 2.41 يوم الأسبوع لتاريخ ما

تجارة إلكترونية ★★★ صعب

لتخطيط الحملات نحتاج يوم الأسبوع لأي تاريخ. اكتب برنامجًا يطبق صيغة زيلر: نعتبر جانفي وفيفري الشهرين 13 و14 من السنة السابقة، ثم `weekday = (day + 13(month+1)/5 + yearInCentury + yearInCentury/4 + century/4 + 5J) % 7` حيث `yearInCentury = year % 100` و`century = year / 100`، و`weekday = 0` يعني السبت.

خطوات الحل

1) إذا كان `month < 3`: `month = month + 12` و`year = year - 1`.
2) نحسب `yearInCentury` و`century` ثم `weekday` بقسمات صحيحة.
3) نحول `weekday` إلى اسم يوم بـ `switch` (0 سبت، 1 أحد، ...).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int day, month, year, yearInCentury, century, weekday;
    scanf("%d %d %d", &day, &month, &year);
    printf("%02d/%02d/%d is a ", day, month, year);
    if (month < 3) {              // جانفي وفيفري: شهرا 13 و14 من السنة السابقة
        month = month + 12;
        year = year - 1;
    }
    yearInCentury = year % 100;              // السنة داخل القرن
    century = year / 100;                    // القرن
    weekday = (day + 13 * (month + 1) / 5 + yearInCentury + yearInCentury / 4 + century / 4 + 5 * centu
    switch (weekday) {
        case 0: printf("Saturday\n"); break;
        case 1: printf("Sunday\n"); break;
        case 2: printf("Monday\n"); break;
        case 3: printf("Tuesday\n"); break;
        case 4: printf("Wednesday\n"); break;
        case 5: printf("Thursday\n"); break;
        default: printf("Friday\n");
    }
    return 0;
}
```

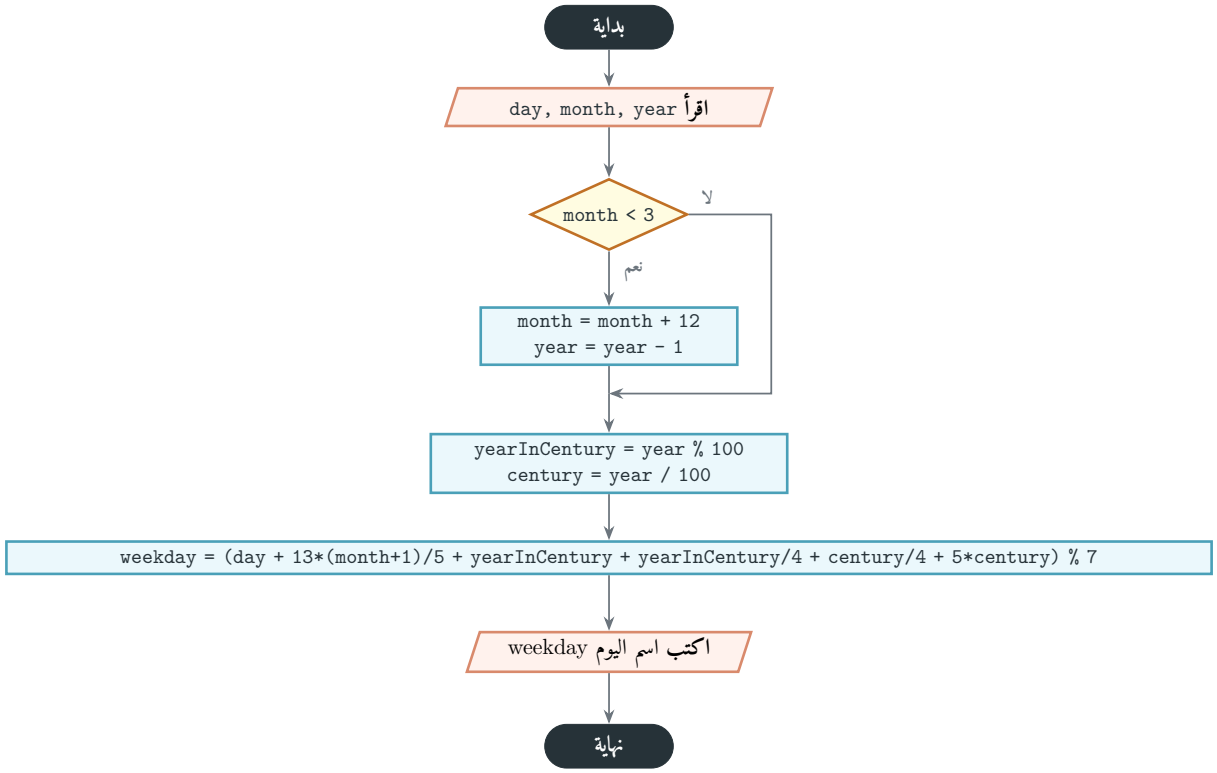
المدخلات

```
1 11 2026
```

نتيجة التنفيذ

```
01/11/2026 is a Sunday
```

المخطط الإنسيابي (Organigramme)



تمرين 2.42 شجرة قرار للتنبؤ بمغادرة الزبون

علم البيانات ★★★ صعب

درّبنا **شجرة قرار** للتنبؤ بمغادرة الزبون (churn). القواعد: إذا مرّ أكثر من 60 يومًا على آخر طلبية: إذا كان عدد الشكاوى ≥ 2 فـ "مغادر" (احتمال 0.85)، وإلا إذا كان الإنفاق < 10000 فـ "مغادر" (0.65) وإلا "باقٍ" (0.40). إذا مر 60 يومًا أو أقل: إذا كان الإنفاق > 50000 فـ "باقٍ" (0.05)، وإلا إذا كانت الشكاوى ≥ 3 فـ "مغادر" (0.55) وإلا "باقٍ" (0.15). اكتب البرنامج.

خطوات الحل

1) نقرأ الأيام منذ آخر طلبية، عدد الشكاوى، والإنفاق.
2) نترجم كل عقدة في الشجرة إلى `if` وكل فرع إلى `else`.
3) كل ورقة تعطي الصنف والاحتمال.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int days, compl;
    double spend, proba;
    scanf("%d %d %lf", &days, &compl, &spend);
    if (days > 60) {                    // العقدة الجذر
        if (compl >= 2)
            proba = 0.85;
        else if (spend < 10000)
            proba = 0.65;
        else
```

```c
            proba = 0.40;
    } else {
        if (spend > 50000)
            proba = 0.05;
        else if (compl >= 3)
            proba = 0.55;
        else
            proba = 0.15;
    }
    // نصنف الزبون مغادرًا إذا تجاوز الاحتمال 0.5
    printf("Churn probability = %.2f\n", proba);
    if (proba >= 0.5)
        printf("Prediction: will churn -> send offer\n");
    else
        printf("Prediction: will stay\n");
    return 0;
}
```

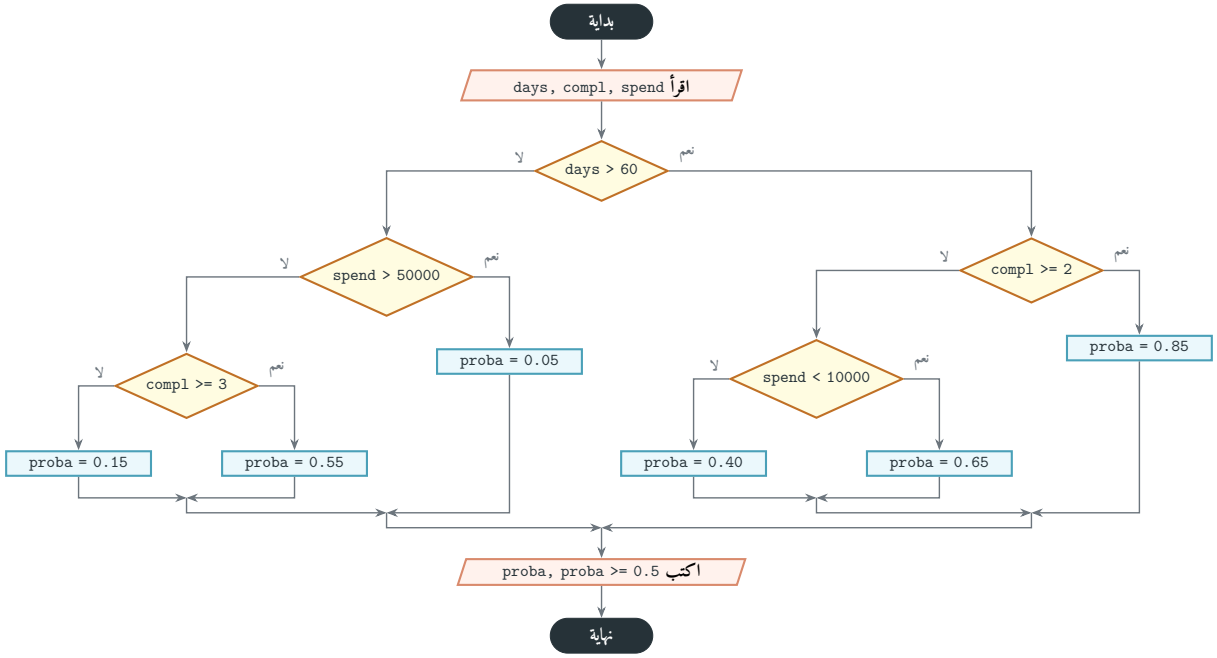
المدخلات

```
75 1 15000
```

نتيجة التنفيذ

```
Churn probability = 0.40
Prediction: will stay
```

المخطط الإنسيابي (Organigramme)



تمرين 2.43 ترتيب أربعة أسعار بشبكة مقارنات

تجارة إلكترونية ★★★ صعب

اكتب برنامجًا يرتب أربعة أسعار تصاعديًا باستعمال **شبكة ترتيب** ثابتة من 5 مقارنات وتبديلات فقط: `(price1,price2)`، `(price3,price4)`، `(price1,price3)`، `(price2,price4)`، `(price2,price3)`. ثم اعرض أيضًا الوسيط (متوسط القيمتين الوسطيتين).

خطوات الحل

1) كل مقارنة `(x,y)`: إذا كان `x > y` نبدلهما.
2) بعد `(price1,price2)` و`(price3,price4)`: كل زوج مرتب.
3) `(price1,price3)` يضع الأصغر في `price1`، و`(price2,price4)` يضع الأكبر في `price4`.
4) `(price2,price3)` يرتب القيمتين الوسطيتين.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price1, price2, price3, price4, tmp;
    scanf("%lf %lf %lf %lf", &price1, &price2, &price3, &price4);
    if (price1 > price2) { tmp = price1; price1 = price2; price2 = tmp; }   // رتب الزوج الأول
    if (price3 > price4) { tmp = price3; price3 = price4; price4 = tmp; }   // رتب الزوج الثاني
    if (price1 > price3) { tmp = price1; price1 = price3; price3 = tmp; }   // أصبح الأصغر price1
    if (price2 > price4) { tmp = price2; price2 = price4; price4 = tmp; }   // أصبح الأكبر price4
    if (price2 > price3) { tmp = price2; price2 = price3; price3 = tmp; }   // رتب الوسطيين
    printf("Sorted: %.0f %.0f %.0f %.0f\n", price1, price2, price3, price4);
    printf("Median = %.1f\n", (price2 + price3) / 2);
    return 0;
}
```

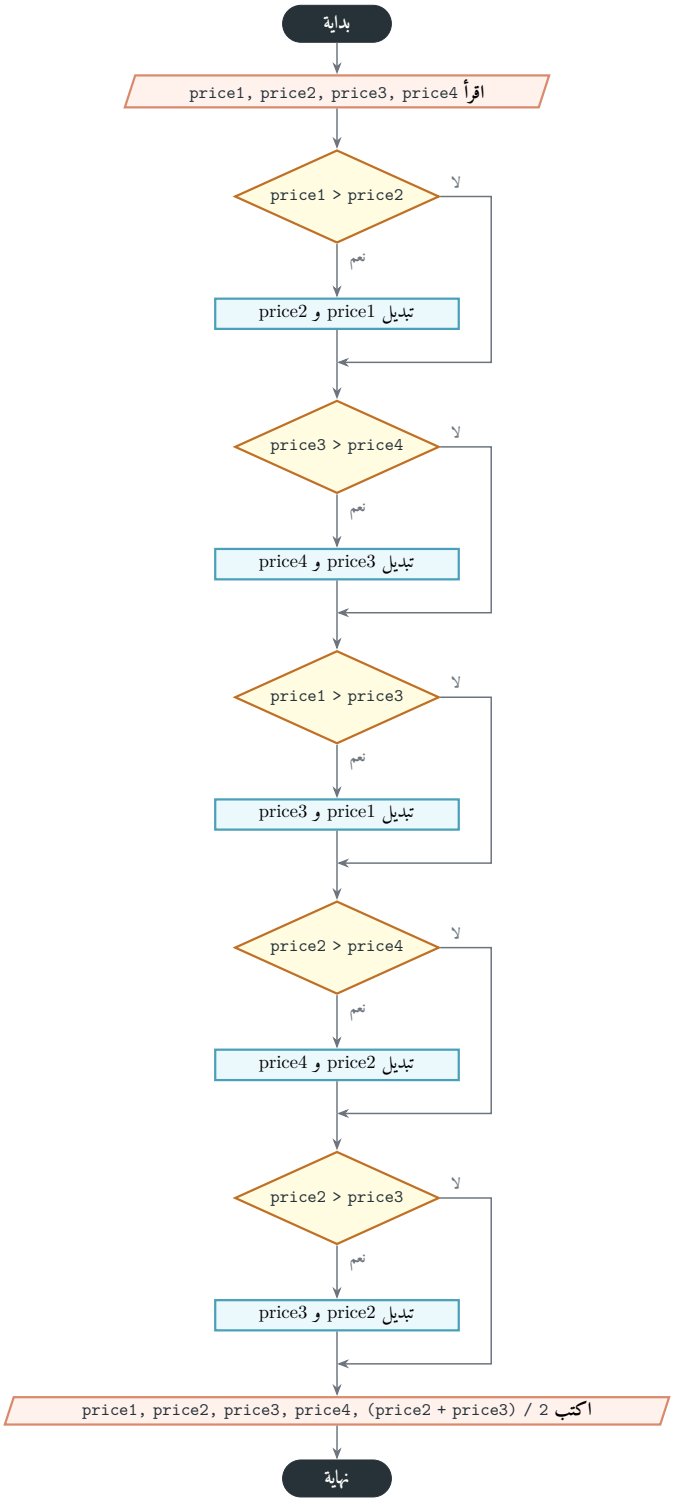
المدخلات

```
780 120 455 300
```

نتيجة التنفيذ

```
Sorted: 120 300 455 780
Median = 377.5
```

المخطط الإنسيابي (Organigramme)



تمرين 2.44 الضريبة التصاعدية على دخل البائع

تجارة إلكترونية صعب

يدفع البائع على المنصة ضريبة **تصاعدية** على دخله السنوي: 0% حتى 240000، 23% للجزء بين 240000 و480000، 27% للجزء بين 480000 و960000، 30% للجزء بين 960000 و1920000، و33% لما فوق ذلك. اكتب برنامجًا يحسب الضريبة والنسبة الفعلية.

خطوات الحل

1) كل شريحة تُطبق نسبتها **على الجزء الواقع داخلها فقط**.
2) نبدأ من الشريحة العليا: إذا كان `rest > 1920000` نضيف `(rest - 1920000) * 0.33` ونضع `rest = 1920000`.
3) نكرر نزولًا حتى الشريحة الأولى.
4) النسبة الفعلية: `tax / income * 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double income, rest, tax = 0;
    scanf("%lf", &income);
    rest = income;                          // الجزء الذي لم يُضرب بعد
    if (rest > 1920000) {                   // الجزء فوق 1920000
        tax = tax + (rest - 1920000) * 0.33;
        rest = 1920000;
    }
    if (rest > 960000) {
        tax = tax + (rest - 960000) * 0.30;
        rest = 960000;
    }
    if (rest > 480000) {
        tax = tax + (rest - 480000) * 0.27;
        rest = 480000;
    }
    if (rest > 240000)                      // الشريحة الأولى معفاة
        tax = tax + (rest - 240000) * 0.23;
    printf("Tax = %.2f DA\n", tax);
    if (income > 0)
        printf("Effective rate = %.2f %%\n", tax / income * 100);
    return 0;
}
```

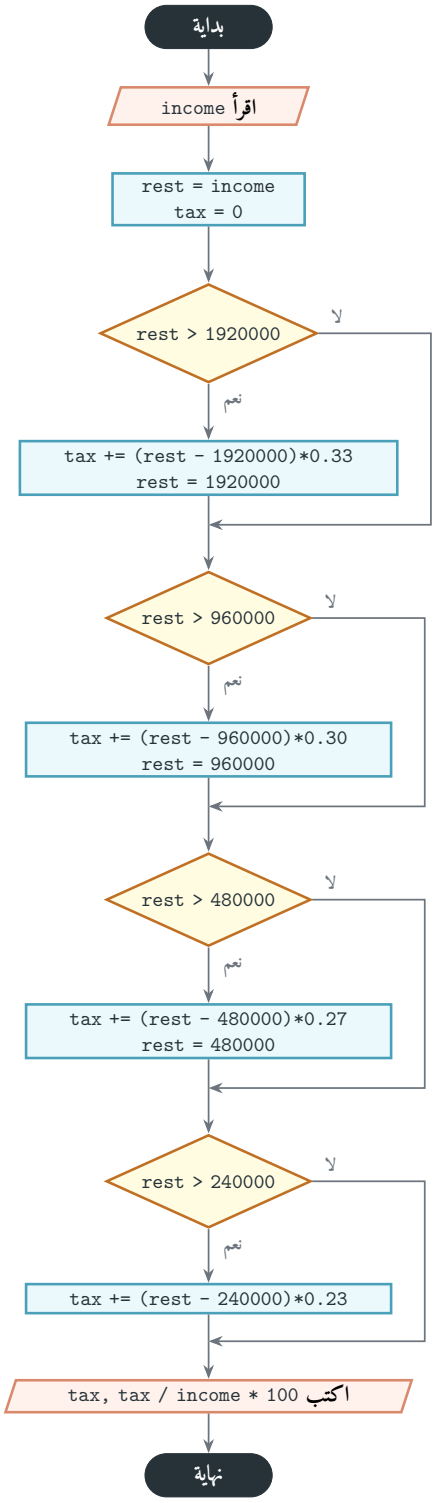
المدخلات

```
1350000
```

نتيجة التنفيذ

```
Tax = 301800.00 DA
Effective rate = 22.36 %
```

المخطط الإنسيابي (Organigramme)



تمرين 2.45 عدد الأيام بين تاريخين

تجارة إلكترونية ★★★ صعب

لحساب مدة بقاء طلبية في المستودع، اكتب برنامجًا يقرأ تاريخين من **نفس السنة** ويحسب عدد الأيام بينهما. نحسب رقم كل تاريخ في السنة باستعمال `switch` دون `break` لتجميع أيام الأشهر السابقة.

خطوات الحل

1) رقم اليوم في السنة: `n = d +`.

2) `switch (m)` يبدأ من الشهر السابق وينزل (fall-through) جامعًا أيام كل شهر.

3) نضيف يومًا إذا كانت السنة كبيسة والشهر بعد فيفري.

4) الفرق `n2 - n1`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int year, d1, m1, d2, m2, n1, n2, leap, diff;
    scanf("%d %d %d %d %d", &year, &d1, &m1, &d2, &m2);
    leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;

    n1 = d1;
    switch (m1 - 1) {          // نجمع كل الأشهر السابقة :break دون
        case 11: n1 += 30;     // نوفمبر
        // fall through
        case 10: n1 += 31;
        // fall through
        case 9:  n1 += 30;
        // fall through
        case 8:  n1 += 31;
        // fall through
        case 7:  n1 += 31;
        // fall through
        case 6:  n1 += 30;
        // fall through
        case 5:  n1 += 31;
        // fall through
        case 4:  n1 += 30;
        // fall through
        case 3:  n1 += 31;
        // fall through
        case 2:  n1 += 28 + leap;   // فيفري
        // fall through
        case 1:  n1 += 31;          // جانفي
    }
    n2 = d2;
    switch (m2 - 1) {          // نفس الطريقة للتاريخ الثاني
        case 11: n2 += 30;
        // fall through
        case 10: n2 += 31;
        // fall through
        case 9:  n2 += 30;
        // fall through
        case 8:  n2 += 31;
        // fall through
        case 7:  n2 += 31;
        // fall through
        case 6:  n2 += 30;
        // fall through
        case 5:  n2 += 31;
        // fall through
```

```c
        case 4:  n2 += 30;
        // fall through
        case 3:  n2 += 31;
        // fall through
        case 2:  n2 += 28 + leap;
        // fall through
        case 1:  n2 += 31;
    }
    diff = n2 - n1;
    if (diff < 0)
        diff = -diff;
    printf("Day numbers: %d and %d\n", n1, n2);
    printf("Days between = %d\n", diff);
    return 0;
}
```

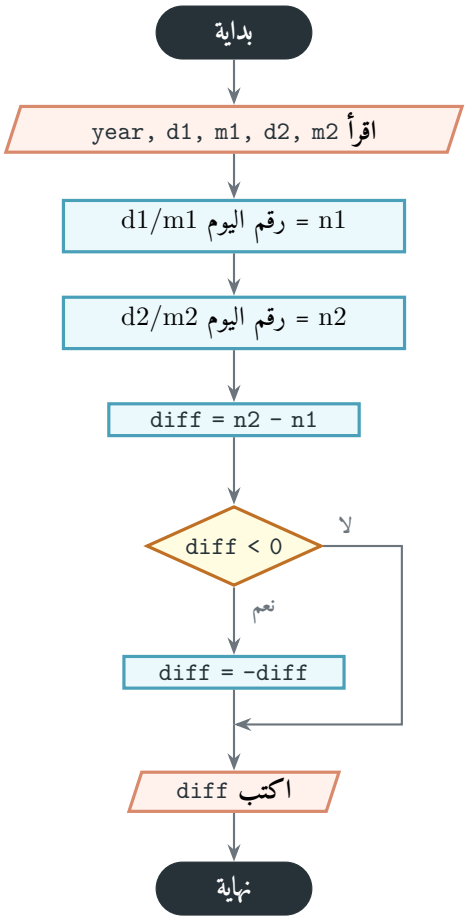
المدخلات

```
2026
15 1
3 4
```

نتيجة التنفيذ

```
Day numbers: 15 and 93
Days between = 78
```

المخطط الإنسيابي (Organigramme)



تمرين 2.46 اختيار شركة الشحن الأرخص

تجارة إلكترونية ★★★ صعب

ثلاث شركات شحن بتسعيرات مختلفة لطرد وزنه `weight` كغ ومسافته `km`: الشركة 1: `300 + 25*weight + 2*km`، الشركة 2: `500 + 15*weight` ولا تقبل أكثر من 30 كغ، الشركة 3: `0.9 * (200 + 40*weight)` إذا كانت المسافة أقل من 100 كم، وإلا `200 + 40*weight + 3*km`. اكتب برنامجًا يحسب التكاليف ويختار الأرخص بين الشركات المتاحة.

خطوات الحل

1) نحسب `c1`.
2) `c2` فقط إذا كان `weight <= 30`، وإلا نعتبرها غير متاحة (`ok2 = 0`).
3) `c3` حسب المسافة.
4) نختار الأرخص مع تجاهل الشركة غير المتاحة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double weight, km, c1, c2, c3, min;
    int ok2, best;
    scanf("%lf %lf", &weight, &km);
    c1 = 300 + 25 * weight + 2 * km;
    ok2 = (weight <= 30);                  // الشركة 2 لها حد للوزن
    c2 = 500 + 15 * weight;
    if (km < 100)
        c3 = 0.9 * (200 + 40 * weight);    // تخفيض للمسافات القصيرة
    else
        c3 = 200 + 40 * weight + 3 * km;
    printf("Carrier 1: %.2f DA\n", c1);
    if (ok2) printf("Carrier 2: %.2f DA\n", c2);
    else     printf("Carrier 2: not available\n");
    printf("Carrier 3: %.2f DA\n", c3);
    best = 1;  min = c1;                   // نبدأ بالشركة 1
    if (ok2 && c2 < min) { best = 2; min = c2; }
    if (c3 < min)        { best = 3; min = c3; }
    printf("Cheapest: carrier %d (%.2f DA)\n", best, min);
    return 0;
}
```

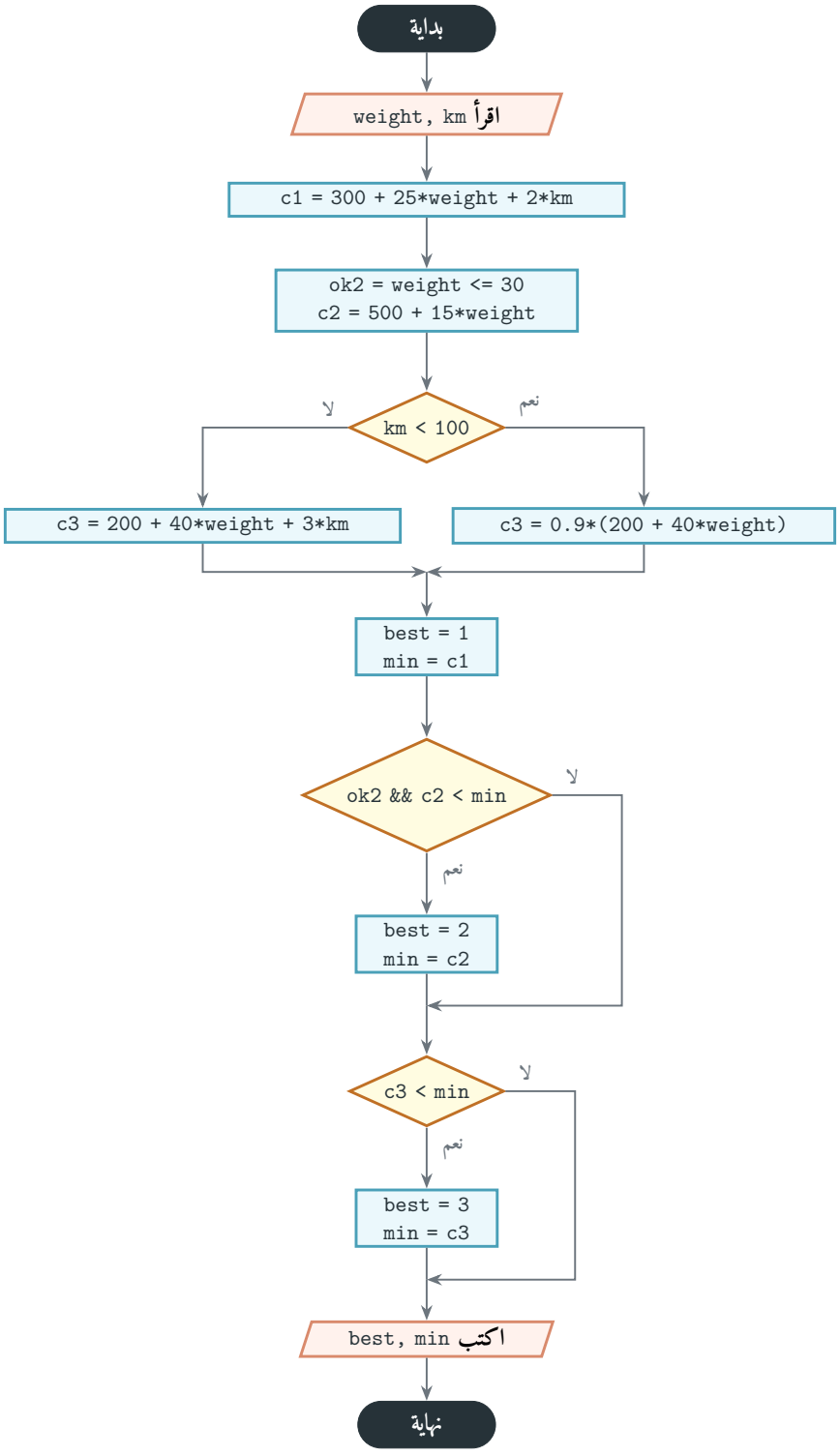
المدخلات

```
12 350
```

نتيجة التنفيذ

```
Carrier 1: 1300.00 DA
Carrier 2: 680.00 DA
Carrier 3: 1730.00 DA
Cheapest: carrier 2 (680.00 DA)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.47 هل العنوان داخل منطقة التوصيل؟

تجارة إلكترونية ★★★ صعب

يخدم مستودع منطقتين: **دائرة** مركزها المستودع `(cx, cy)` ونصف قطرها `radius` كم، و**مستطيل** صناعي `[x1, x2] × [y1, y2]`. اكتب برنامجًا يقرأ إحداثيات الزبون ويحدد المنطقة التي يقع فيها (أو كلتيهما أو لا شيء)، دون استعمال الجذر التربيعي.

خطوات الحل

1) داخل الدائرة: $(custX-cx)^2 + (custY-cy)^2 <= radius^2$ (نقارن المربعات لتجنب الجذر).
2) داخل المستطيل: $x1 <= custX <= x2$ و$y1 <= custY <= y2$.
3) نجمع النتيجتين لتحديد الحالات الأربع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double cx, cy, radius, x1, x2, y1, y2, custX, custY, d2;
    int inC, inR;
    scanf("%lf %lf %lf", &cx, &cy, &radius);           // الدائرة
    scanf("%lf %lf %lf %lf", &x1, &x2, &y1, &y2); // المستطيل
    scanf("%lf %lf", &custX, &custY);                    // الزبون
    d2 = (custX - cx) * (custX - cx) + (custY - cy) * (custY - cy);
    inC = (d2 <= radius * radius);              // مقارنة المربعات بدل الجذر
    inR = (custX >= x1 && custX <= x2 && custY >= y1 && custY <= y2);
    printf("Squared distance = %.1f (limit %.1f)\n", d2, radius * radius);
    if (inC && inR)
        printf("Inside both zones\n");
    else if (inC)
        printf("Inside the circle only\n");
    else if (inR)
        printf("Inside the industrial rectangle only\n");
    else
        printf("Out of delivery zone\n");
    return 0;
}
```

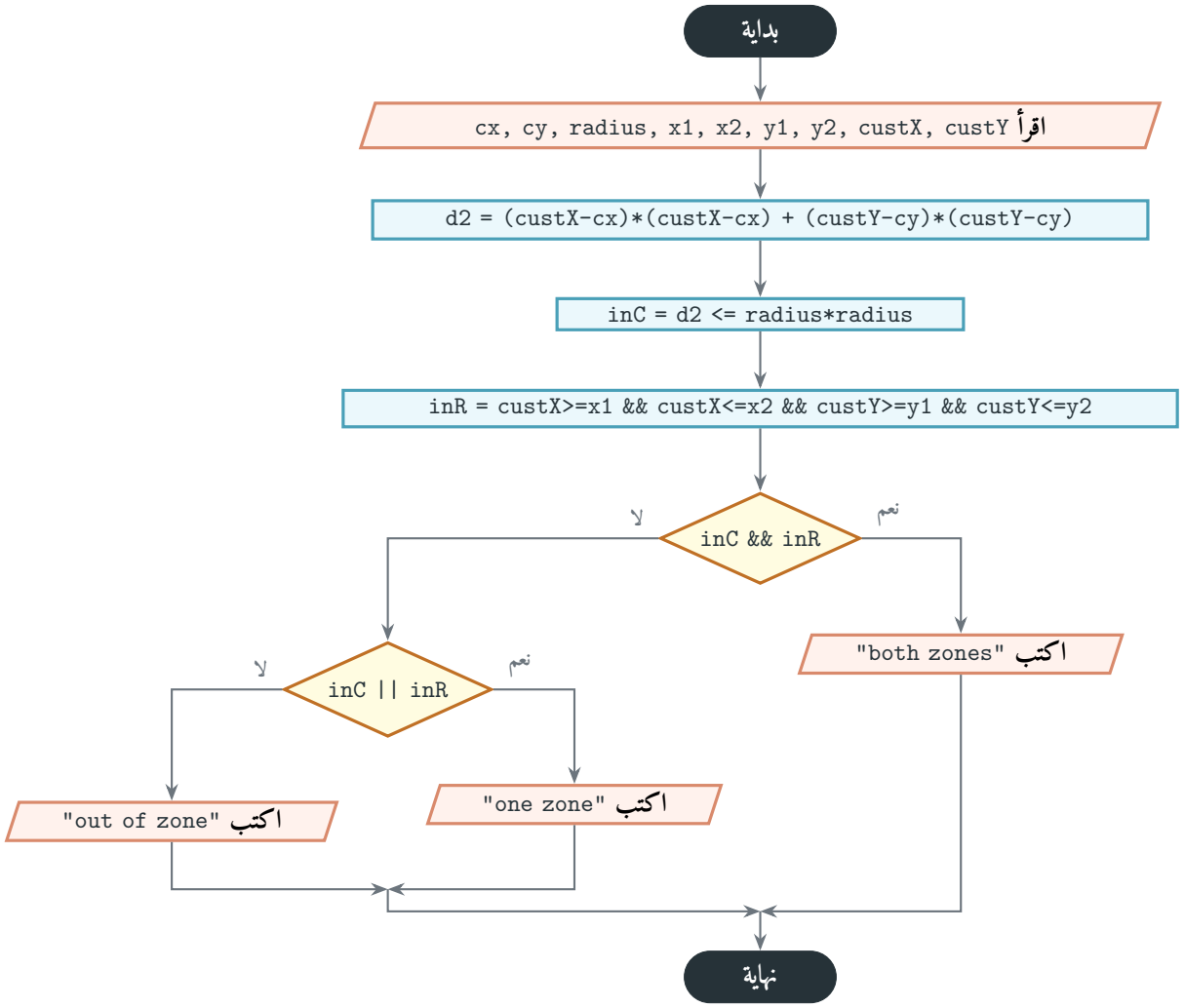
المدخلات

```
0 0 10
12 20 -3 4
13 2
```

نتيجة التنفيذ

```
Squared distance = 173.0 (limit 100.0)
Inside the industrial rectangle only
```

المخطط الإنسيابي (Organigramme)



تمرين 2.48 أقرب جار لتصنيف زبون جديد

علم البيانات ★★★ صعب

لدينا أربعة زبائن معروفين، لكل منهم (عدد الزيارات، متوسط السلة بالآلاف) وصنف: الأول والثاني "مقتصد" (0)، الثالث والرابع "مبذر" (1). البيانات ثابتة في البرنامج: `(2,1.5)`، `(4,2)`، `(9,7.5)`، `(11,6)`. اكتب برنامجًا يقرأ زبونًا جديدًا ويصنفه بطريقة **أقرب جار** (1-NN) باستعمال مربع المسافة.

خطوات الحل

1) نحسب مربع المسافة إلى كل زبون من الأربعة.
2) نحتفظ بأصغر مسافة `best` وصنفها `cls`، ونحدّثهما عند كل مسافة أصغر.
3) نعرض الصنف المتنبأ به.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double visits, basket, d1, d2, d3, d4, best;
```

```c
    int cls, who;
    scanf("%lf %lf", &visits, &basket);
    // مربع المسافة إلى كل زبون معروف (بيانات التدريب)
    d1 = (visits - 2) * (visits - 2) + (basket - 1.5) * (basket - 1.5);
    d2 = (visits - 4) * (visits - 4) + (basket - 2) * (basket - 2);
    d3 = (visits - 9) * (visits - 9) + (basket - 7.5) * (basket - 7.5);
    d4 = (visits - 11) * (visits - 11) + (basket - 6) * (basket - 6);
    printf("d2: %.2f %.2f %.2f %.2f\n", d1, d2, d3, d4);
    best = d1; cls = 0; who = 1;          // نبدأ بالجار الأول
    if (d2 < best) { best = d2; cls = 0; who = 2; }
    if (d3 < best) { best = d3; cls = 1; who = 3; }
    if (d4 < best) { best = d4; cls = 1; who = 4; }
    printf("Nearest neighbour: customer %d\n", who);
    if (cls == 0) printf("Predicted class: thrifty\n");
    else          printf("Predicted class: big spender\n");
    return 0;
}
```

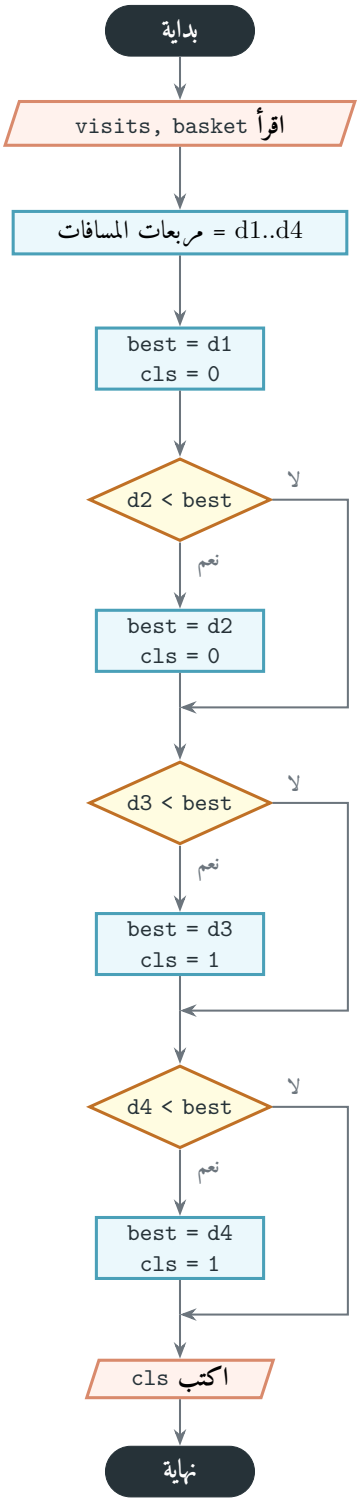
المدخلات

```
7 5
```

نتيجة التنفيذ

```
d2: 37.25 18.00 10.25 17.00
Nearest neighbour: customer 3
Predicted class: big spender
```

المخطط الإنسيابي (Organigramme)



تمرين 2.49 محرك التسعير الكامل

تجارة إلكترونية ★★★ صعب

اكتب برنامج **محرك تسعير** يقرأ: مبلغ السلة، مستوى الوفاء (0 عادي، 1 فضي، 2 ذهبي)، رمز قسيمة (0 لا شيء، 1 = -10%، 2 = -1000 دج إذا تجاوزت السلة 8000)، والولاية (16 أو غيرها). القواعد: الوفاء يعطي 0% أو 3% أو 7%؛ **مجموع التخفيضات لا يتجاوز 30%** من السلة؛ ثم الرسم 19%؛ ثم الشحن: مجاني للذهبي أو إذا تجاوز المبلغ 10000، وإلا 300 للولاية 16 و700 لغيرها.

خطوات الحل

1) نحسب تخفيض الوفاء dl بـ switch.
2) نحسب تخفيض القسيمة dc مع شرطها.
3) نحصر المجموع: إذا كان dl + dc > 0.3 * cart نجعله 0.3 * cart.
4) نطبق الرسم ثم نحدد الشحن.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double cart, dl, dc, disc, net, ship;
    int level, coupon, wilaya;
    scanf("%lf %d %d %d", &cart, &level, &coupon, &wilaya);
    switch (level) {                        // تخفيض الوفاء
        case 1:  dl = cart * 0.03; break; // فضي
        case 2:  dl = cart * 0.07; break; // ذهبي
        default: dl = 0;
    }
    dc = 0;                                 // تخفيض القسيمة
    if (coupon == 1)
        dc = cart * 0.10;
    else if (coupon == 2 && cart > 8000)
        dc = 1000;
    disc = dl + dc;
    if (disc > 0.3 * cart)                  // سقف التخفيضات 30%
        disc = 0.3 * cart;
    net = (cart - disc) * 1.19;             // الرسم بعد التخفيض
    if (level == 2 || net > 10000)
        ship = 0;
    else if (wilaya == 16)
        ship = 300;
    else
        ship = 700;
    printf("Discounts = %.2f DA\n", disc);
    printf("Net with tax = %.2f DA\n", net);
    printf("Shipping = %.2f DA\n", ship);
    printf("TOTAL = %.2f DA\n", net + ship);
    return 0;
}
```

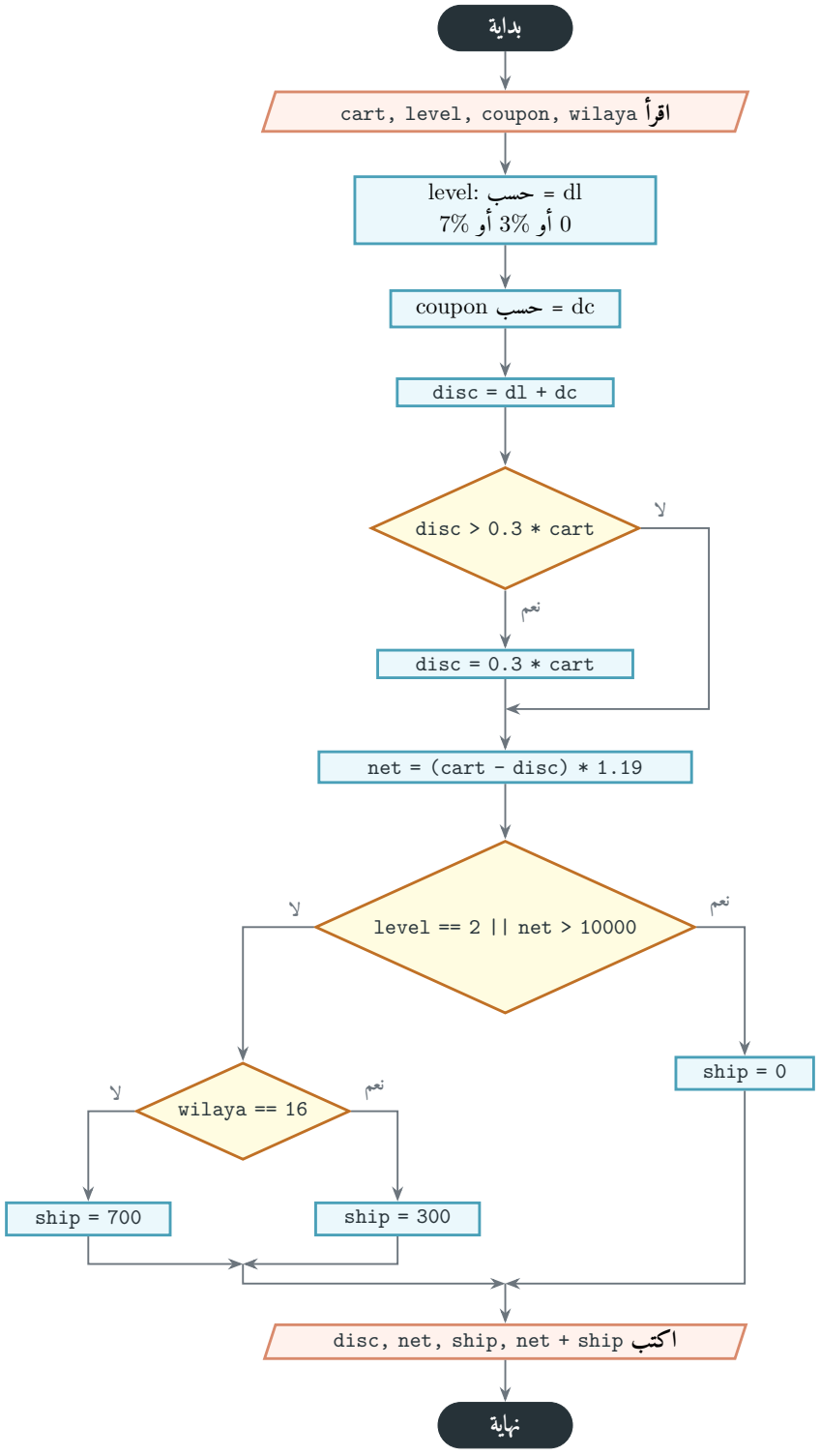
المدخلات

```
12500 2 1 31
```

نتيجة التنفيذ

```
Discounts = 2125.00 DA
Net with tax = 12346.25 DA
Shipping = 0.00 DA
TOTAL = 12346.25 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.50 توازن العرض والطلب

علم البيانات ★★★ صعب

في سوق منتج: الطلب `Qd = demandBase - demandSlope*price` والعرض `Qs = supplyBase + supplySlope*price`. سعر التوازن يحقق `Qd = Qs` أي `price* = (demandBase - supplyBase) / (demandSlope + supplySlope)`. تفرض الدولة أحيانًا

سعراً أدنى `floor`. اكتب برنامجًا يحسب التوازن، وإذا كان السعر الأدنى أعلى من `price*` يُباع بالسعر الأدنى ويظهر **فائض** `Qs - Qd`. تحقق أيضًا أن التوازن ممكن (`price* > 0` و`Q* > 0`).

خطوات الحل

1) `price = (demandBase - supplyBase) / (demandSlope + supplySlope)` و`quantity = demandBase - demandSlope * price`.
2) إذا كان `price <= 0` أو `quantity <= 0`: لا يوجد توازن ذو معنى.
3) إذا كان `floor > price`: نحسب `Qd` و`Qs` عند `floor` والفائض.
4) وإلا السعر الأدنى غير مؤثر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double demandBase, demandSlope, supplyBase, supplySlope, floor_p, price, quantity, qd, qs;
    scanf("%lf %lf %lf %lf", &demandBase, &demandSlope, &supplyBase, &supplySlope);
    scanf("%lf", &floor_p);                 // السعر الأدنى المفروض
    price = (demandBase - supplyBase) / (demandSlope + supplySlope);                 // من supplySlope*price + supplyBase = demandSlope*price - demandBase
    quantity = demandBase - demandSlope * price;
    if (price <= 0 || quantity <= 0) {
        printf("No meaningful equilibrium\n");
    } else {
        printf("Equilibrium: p* = %.2f DA, q* = %.2f units\n", price, quantity);
        if (floor_p > price) {                  // السعر الأدنى مؤثر
            qd = demandBase - demandSlope * floor_p;          // الطلب ينخفض
            qs = supplyBase + supplySlope * floor_p;          // العرض يرتفع
            printf("At floor %.2f: demand %.2f, supply %.2f\n", floor_p, qd, qs);
            printf("Surplus = %.2f units\n", qs - qd);
        } else {
            printf("Price floor not binding\n");
        }
    }
    return 0;
}
```

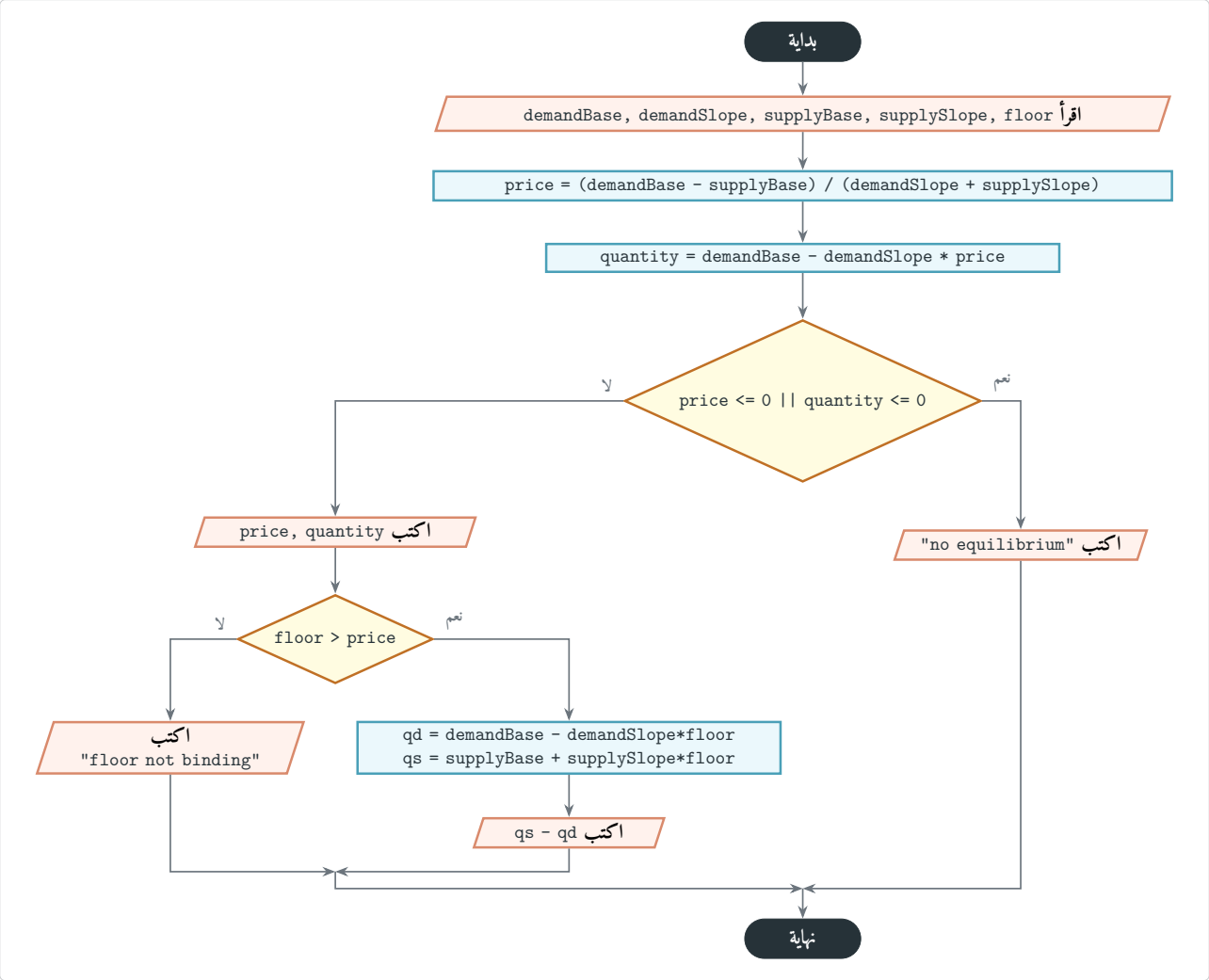
المدخلات

```
1000 4 100 2
180
```

نتيجة التنفيذ

```
Equilibrium: p* = 150.00 DA, q* = 400.00 units
At floor 180.00: demand 280.00, supply 460.00
Surplus = 180.00 units
```

المخطط الإنسيابي (Organigramme)

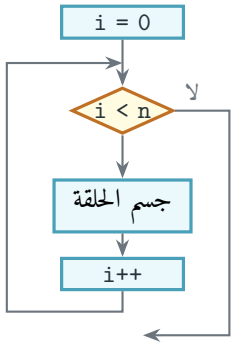


الفصل 3

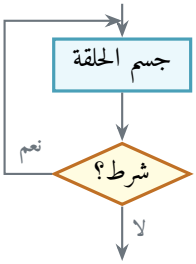
الحلقات التكرارية

تعريفات

- **الحلقة التكرارية**: تعيد تنفيذ مجموعة تعليمات ما دام شرط ما محققًا.
- `for`: عندما يكون **عدد التكرارات معروفًا** مسبقًا (تهيئة، شرط، تحديث).
- `while`: الشرط يُختبر **قبل** كل دورة، فقد لا تُنفذ الحلقة أبدًا.
- `do ... while`: الشرط يُختبر **بعد** الدورة، فتُنفذ **مرة واحدة على الأقل**.
- **المجمِّع والعداد**: متغير يُهيأ قبل الحلقة (`sum = 0`، `count = 0`) ويُحدَّث داخلها.



for / while



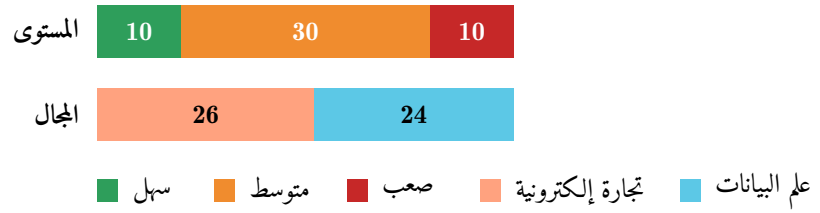
do ... while

الحلقة	متى نستعملها؟	مثال من التمارين
`for`	عدد معروف من التكرارات	المرور على n سعرًا
`while`	التوقف عند تحقق شرط	الادخار حتى بلوغ السعر
`do while`	تنفيذ مرة على الأقل	إعادة طلب إدخال صالح

الحلقة اللانهائية

إذا لم يتغير متغير الشرط داخل الحلقة فلن تتوقف أبدًا. تأكد دائمًا أن كل دورة **تقرّب** الحلقة من نهايتها.

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 3.1 مجموع أسعار السلة

تجارة إلكترونية ⭐ سهل

اكتب برنامجًا يقرأ عدد المنتجات `nbItems` ثم أسعارها، ويعرض المجموع باستعمال حلقة `for`.

خطوات الحل

1) نقرأ `nbItems` ونهيئ `sum = 0`.
2) حلقة `for` من `item = 1` إلى `nbItems`: نقرأ سعرًا ونضيفه.
3) نعرض المجموع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbItems, item;
    double price, sum = 0;          // المجمّع يبدأ من الصفر
    scanf("%d", &nbItems);
    for (item = 1; item <= nbItems; item++) {   // nbItems تكرارًا بالضبط
        scanf("%lf", &price);
        sum = sum + price;          // نضيف السعر الحالي
    }
    printf("Basket total = %.2f DA\n", sum);
    return 0;
}
```

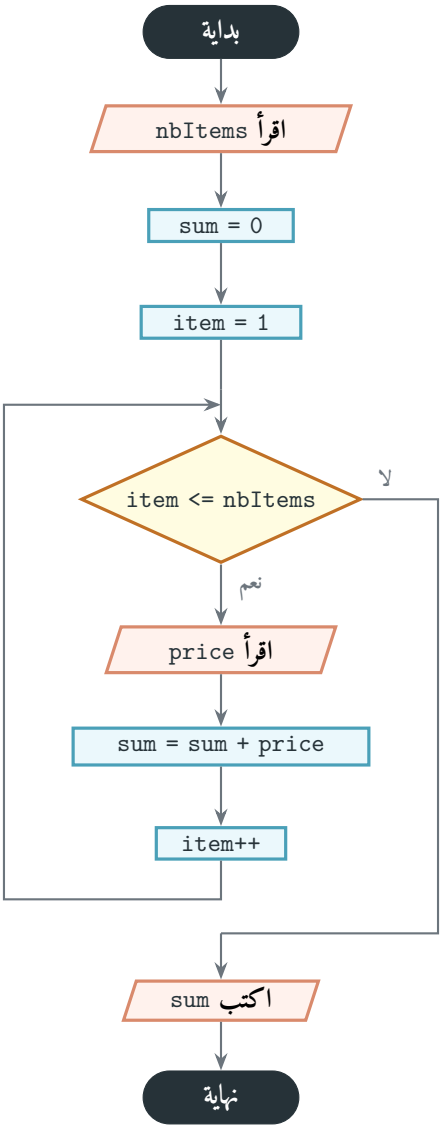
المدخلات

```
5
1200 450 3999 89.5 700
```

نتيجة التنفيذ

```
Basket total = 6438.50 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.2 عدد الطلبيات الكبيرة

علم البيانات ⭐ سهل

اكتب برنامجًا يقرأ nbOrders مبالغ طلبيات ويعد كم منها يتجاوز 10000 دج، ثم يعرض نسبتها.

خطوات الحل

1) عداد count = 0.
2) لكل مبلغ: إذا كان > 10000 نزيد العداد.
3) النسبة: count * 100.0 / nbOrders.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbOrders, index, count = 0;
```

```c
    double amount;
    scanf("%d", &nbOrders);
    for (index = 0; index < nbOrders; index++) {
        scanf("%lf", &amount);
        if (amount > 10000)
            count++;                // طلبية كبيرة: نزيد العداد
    }
    printf("Large orders: %d / %d (%.1f %%)\n", count, nbOrders, count * 100.0 / nbOrders);
    return 0;
}
```

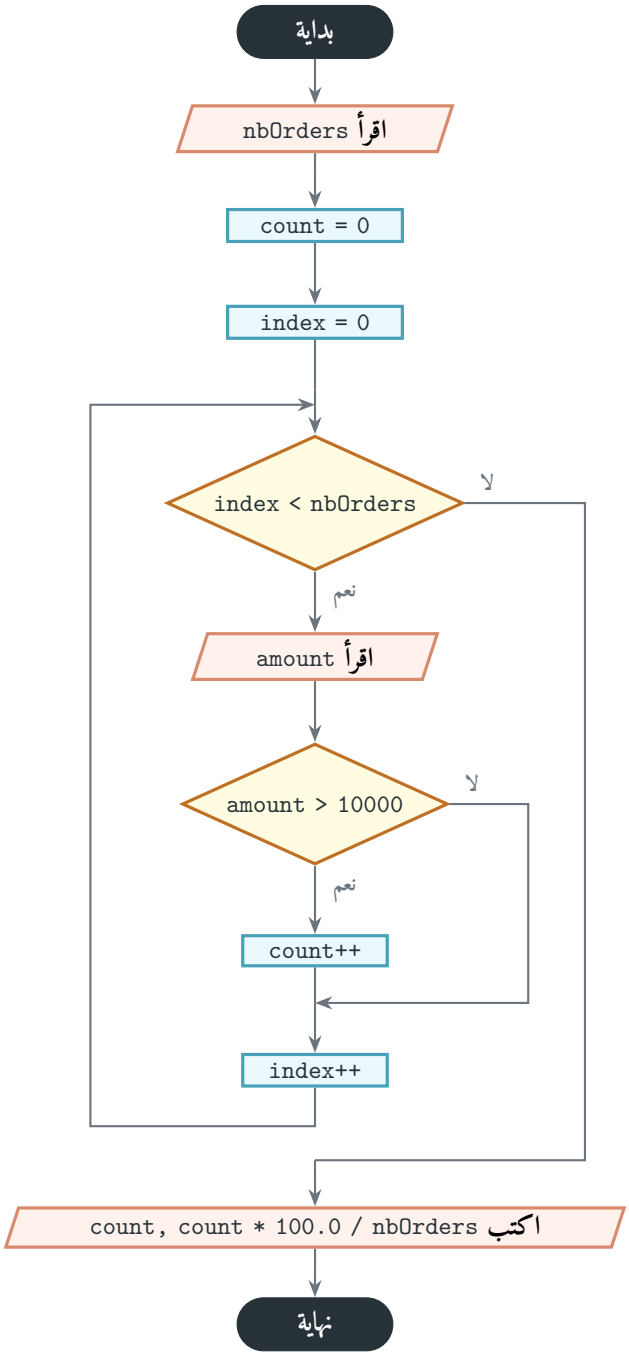
⌨ المدخلات

```
8
2500 12000 800 15600 9999 10001 4300 22000
```

>_ نتيجة التنفيذ

```
Large orders: 4 / 8 (50.0 %)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.3 متوسط التقييمات

علم البيانات ⭐ سهل

اكتب برنامجًا يقرأ عدد التقييمات `nbReviews` ثم التقييمات (أعداد صحيحة من 1 إلى 5) ويحسب **المتوسط**.

خطوات الحل

1) نجمع التقييمات في `sum`.
2) المتوسط `(double)sum / nbReviews`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbReviews, index, rating, sum = 0;
    scanf("%d", &nbReviews);
    for (index = 0; index < nbReviews; index++) {
        scanf("%d", &rating);
        sum += rating;                  // rating + sum = sum تعني rating =+ sum
    }
    printf("Average = %.2f\n", (double)sum / nbReviews);
    return 0;
}
```

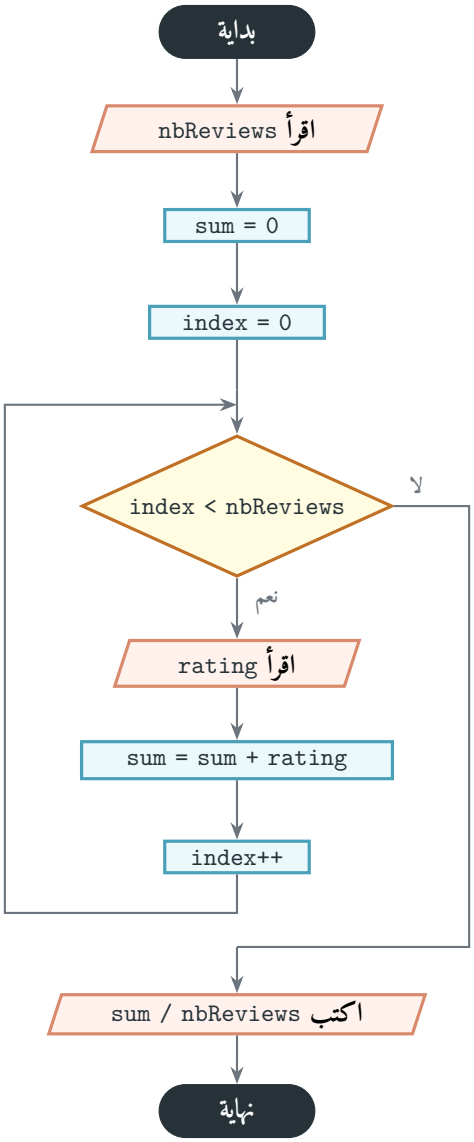
المدخلات

```
7
5 4 4 3 5 2 4
```

نتيجة التنفيذ

```
Average = 3.86
```

المخطط الإنسيابي (Organigramme)



تمرين 3.4 جدول الأسعار حسب الكمية

تجارة إلكترونية ⭐ سهل

يعرض المتجر جدولًا يبين ثمن 1 إلى 10 وحدات من منتج سعره مقروء. اكتب البرنامج.

خطوات الحل

1) حلقة `for` على الكمية `qty` من 1 إلى 10.
2) نعرض `qty` و`qty * price` في كل سطر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int qty;
    double price;
```

```c
    scanf("%lf", &price);
    printf("Qty | Price\n");
    for (qty = 1; qty <= 10; qty++)                    // سطر لكل كمية
        printf("%3d | %8.2f\n", qty, qty * price);
    return 0;
}
```

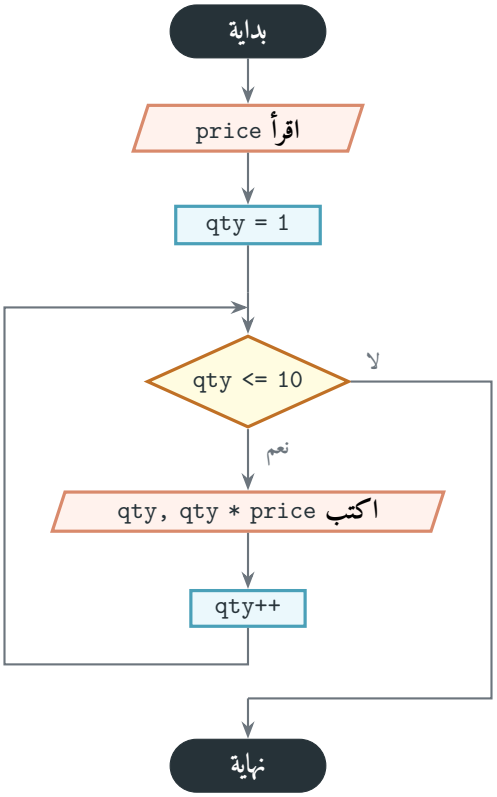
المدخلات

```
275
```

نتيجة التنفيذ

```
Qty | Price
  1 |   275.00
  2 |   550.00
  3 |   825.00
  4 |  1100.00
  5 |  1375.00
  6 |  1650.00
  7 |  1925.00
  8 |  2200.00
  9 |  2475.00
 10 |  2750.00
```

المخطط الإنسيابي (Organigramme)



تمرين 3.5 العد التنازلي للتخفيضات

تجارة إلكترونية
سهل

تبدأ تخفيضات "الجمعة السوداء" بعد hours ساعة. اكتب برنامجًا يعرض عدًا تنازليًا بخطوة 6 ساعات باستعمال حلقة while حتى يصل إلى 0 أو أقل.

خطوات الحل

1) ما دام hours > 0 نعرض hours ثم نطرح 6.
2) عند الخروج من الحلقة نعرض رسالة البداية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int hours;
    scanf("%d", &hours);
    while (hours > 0) {                    // الشرط يُختبر قبل كل دورة
        printf("%d hours left\n", hours);
        hours = hours - 6;
    }
    printf("Black Friday sale started!\n");
    return 0;
}
```

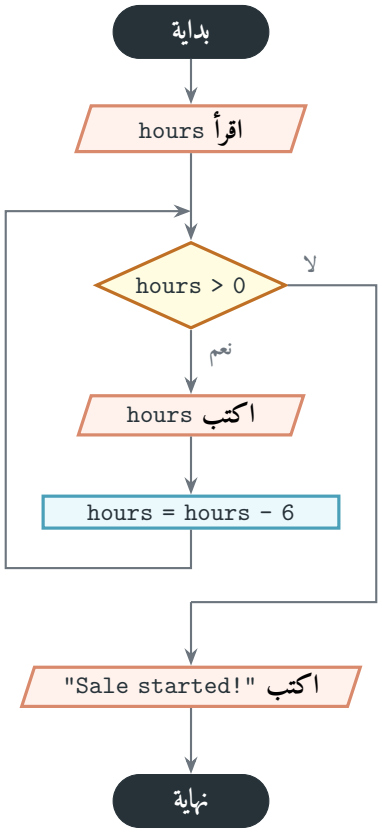
المدخلات

```
27
```

نتيجة التنفيذ

```
27 hours left
21 hours left
15 hours left
9 hours left
3 hours left
Black Friday sale started!
```

المخطط الإنسيابي (Organigramme)



تمرين 3.6 الإدخال حتى الصفر

تجارة إلكترونية
سهل

يُدخل أمين الصندوق أسعار المنتجات واحدًا واحدًا ولا يعرف عددها مسبقًا، ويُنهي بإدخال **0**. اكتب برنامجًا يحسب المجموع وعدد المنتجات.

خطوات الحل

1) نقرأ السعر الأول قبل الحلقة.
2) ما دام `price != 0`: نضيفه ونزيد العداد ثم نقرأ السعر التالي.
3) القيمة 0 **قيمة حارسة** تنهي الإدخال.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, sum = 0;
    int nbItems = 0;
    scanf("%lf", &price);            // القراءة الأولى قبل الحلقة
    while (price != 0) {             // 0 قيمة حارسة: نهاية الإدخال
        sum += price;
        nbItems++;
        scanf("%lf", &price);        // نقرأ السعر التالي
    }
```

```
    printf("%d items, total = %.2f DA\n", nbItems, sum);
    return 0;
}
```

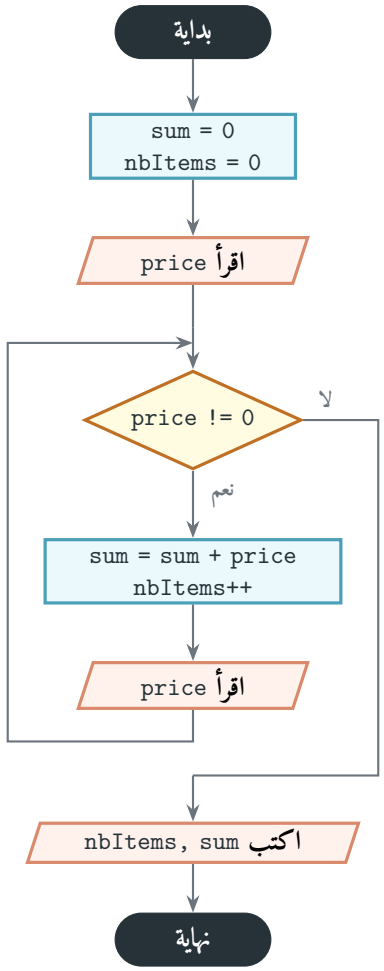
المدخلات

```
350 1200 75 990 0
```

نتيجة التنفيذ

```
4 items, total = 2615.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.7 أغلى منتج

تجارة إلكترونية سهل

اكتب برنامجًا يقرأ `nbPrices` أسعار ويعرض أكبرها ورقم ترتيبه.

خطوات الحل

1) نقرأ السعر الأول ونعتبره الأكبر: `max = price` و`pos = 1`.
2) لكل سعر موالٍ: إذا كان أكبر من `max` نحدّث `max` و`pos`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPrices, index, pos;
    double price, max;
    scanf("%d %lf", &nbPrices, &price);
    max = price;                        // أول سعر هو الأكبر مبدئيًا
    pos = 1;
    for (index = 2; index <= nbPrices; index++) {
        scanf("%lf", &price);
        if (price > max) {              // وجدنا سعرًا أكبر
            max = price;
            pos = index;
        }
    }
    printf("Most expensive: %.2f DA (item #%d)\n", max, pos);
    return 0;
}
```

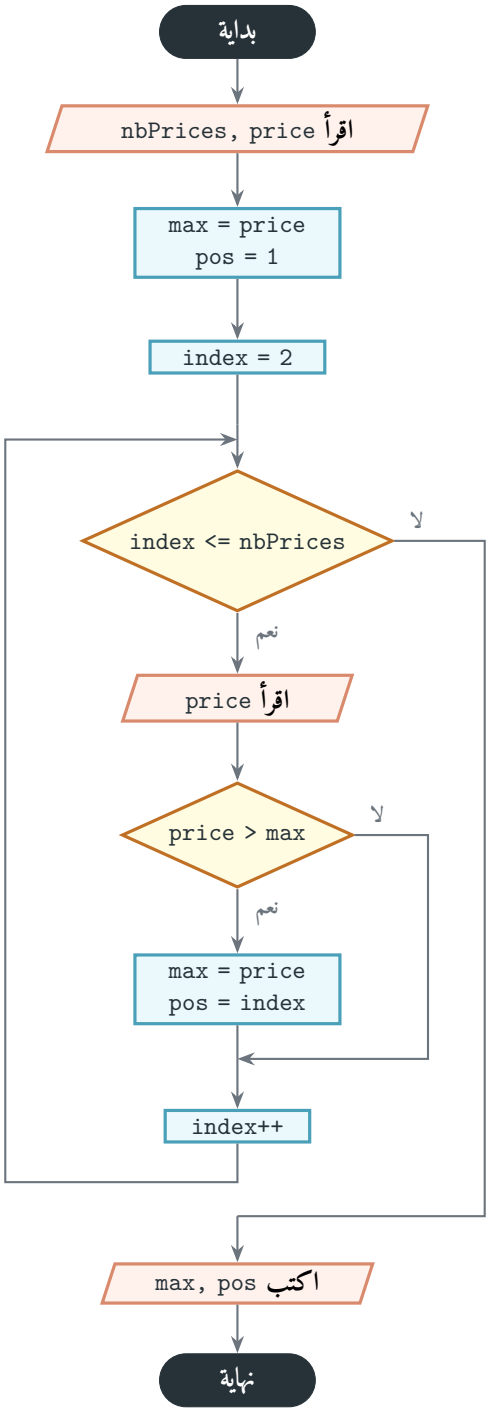
المدخلات

```
6
450 1290 300 1290 870 99
```

نتيجة التنفيذ

```
Most expensive: 1290.00 DA (item #2)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.8 أرقام الطلبيات الزوجية

تجارة إلكترونية | سهل

اكتب برنامجًا يعرض كل أرقام الطلبيات **الزوجية** بين `first` و`last` (ضمنًا) ويعد عددها.

خطوات الحل

1) حلقة من `first` إلى `last`.
2) نعرض الرقم إذا كان `orderId % 2 == 0`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int first, last, orderId, count = 0;
    scanf("%d %d", &first, &last);
    for (orderId = first; orderId <= last; orderId++) {
        if (orderId % 2 == 0) {         // رقم زوجي
            printf("%d ", orderId);
            count++;
        }
    }
    printf("\nEven order numbers: %d\n", count);
    return 0;
}
```

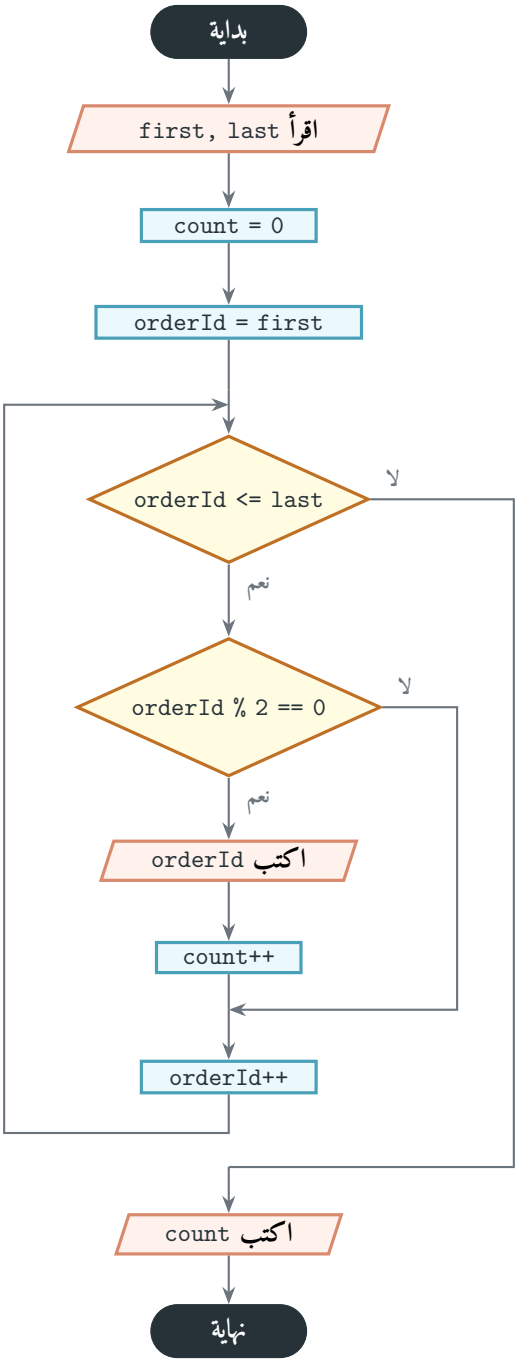
المدخلات

```
1001 1015
```

نتيجة التنفيذ

```
1002 1004 1006 1008 1010 1012 1014
Even order numbers: 7
```

المخطط الإنسيابي (Organigramme)



تمرين 3.9 الإدخار لشراء حاسوب

تجارة إلكترونية سهل

يريد طالب شراء حاسوب سعره `price`. يدّخر كل شهر مبلغًا `monthly`. اكتب برنامجًا يحسب **عدد الأشهر** اللازمة باستعمال `while`.

خطوات الحل

1) `saved = 0` و`months = 0`.
2) ما دام `saved < price`: نضيف `monthly` ونزيد الأشهر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, monthly, saved = 0;
    int months = 0;
    scanf("%lf %lf", &price, &monthly);
    while (saved < price) {        // لا نعرف عدد التكرارات مسبقًا
        saved += monthly;
        months++;
    }
    printf("Months needed: %d (saved %.2f DA)\n", months, saved);
    return 0;
}
```

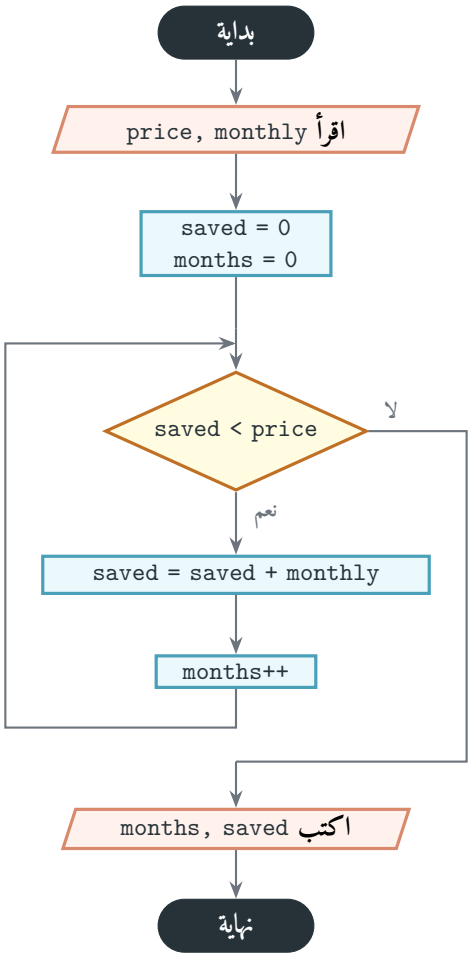
المدخلات

```
95000 12500
```

نتيجة التنفيذ

```
Months needed: 8 (saved 100000.00 DA)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.10 إعادة طلب تقييم صالح

سهل | علم البيانات

اكتب برنامجًا يطلب من الزبون تقييمًا بين 1 و5، ويعيد الطلب ما دام الإدخال غير صالح، باستعمال `do ... while`.

خطوات الحل

1) `do`: نقرأ التقييم.
2) `while (rating < 1 || rating > 5)`: نعيد إذا كان خارج المجال.
3) الحلقة تنفذ مرة واحدة على الأقل.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int rating, tries = 0;
    do {
        printf("Rating (1-5): ");
        scanf("%d", &rating);
        tries++;
        if (rating < 1 || rating > 5)
            printf("invalid!\n");
    } while (rating < 1 || rating > 5);      // نعيد ما دام الإدخال خاطئًا
    printf("Accepted rating %d after %d tries\n", rating, tries);
    return 0;
}
```

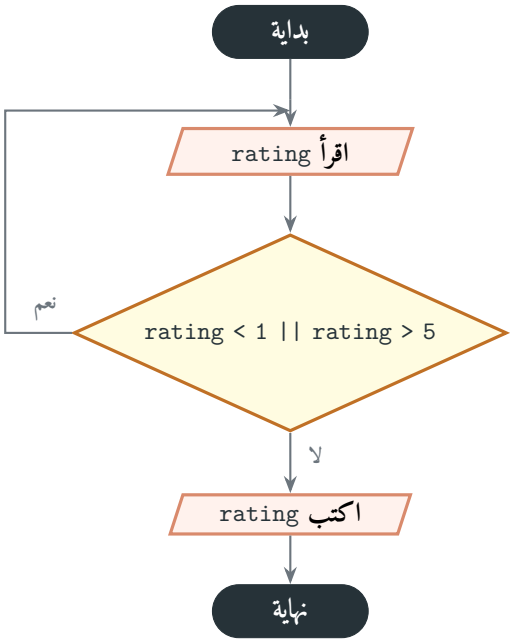
المدخلات

```
7 0 4
```

نتيجة التنفيذ

```
Rating (1-5): invalid!
Rating (1-5): invalid!
Rating (1-5): Accepted rating 4 after 3 tries
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 3.11 الأصغر والأكبر والمدى

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ أوقات تحميل nbPages صفحات (بالملي ثانية) ويعرض **الأصغر والأكبر والمدى** (الفرق بينهما) والمتوسط، في مرور واحد على البيانات.

خطوات الحل

1) القيمة الأولى تهيئ min وmax وsum.
2) في كل تكرار نحدّث الثلاثة.
3) المدى: max - min.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPages, index;
    double loadTime, min, max, sum;
    scanf("%d %lf", &nbPages, &loadTime);
    min = max = sum = loadTime;          // التهيئة بالقيمة الأولى
    for (index = 2; index <= nbPages; index++) {
        scanf("%lf", &loadTime);
        sum += loadTime;
        if (loadTime < min) min = loadTime;     // تحديث الأصغر
        if (loadTime > max) max = loadTime;     // تحديث الأكبر
    }
    printf("Min = %.0f ms, Max = %.0f ms\n", min, max);
    printf("Range = %.0f ms, Mean = %.1f ms\n", max - min, sum / nbPages);
    return 0;
}
```

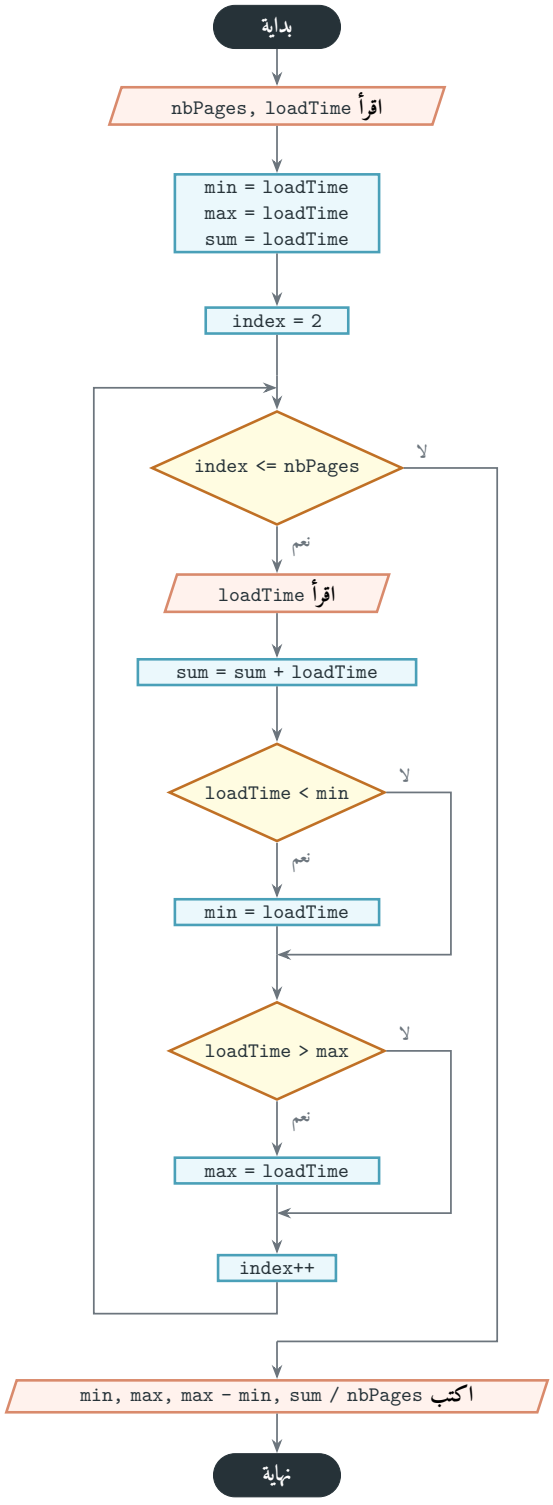
المدخلات

```
7
320 280 910 450 300 275 610
```

نتيجة التنفيذ

```
Min = 275 ms, Max = 910 ms
Range = 635 ms, Mean = 449.3 ms
```

المخطط الإنسيابي (Organigramme)



تمرين 3.12 المتوسط والتباين في مرور واحد

علم البيانات

متوسط

لحساب التباين دون تخزين البيانات نستعمل الصيغة: $var = (\Sigma sales^2)/nbDays - mean^2$. اكتب برنامجًا يقرأ nbDays قيم مبيعات يومية ويحسب المتوسط والتباين في مرور واحد.

خطوات الحل

1) نجمع `sum = Σsales` و`s2 = Σsales²` في نفس الحلقة.
2) `mean = sum / nbDays`.
3) `var = s2 / nbDays - mean * mean`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day;
    double sales, sum = 0, s2 = 0, mean, var;
    scanf("%d", &nbDays);
    for (day = 0; day < nbDays; day++) {
        scanf("%lf", &sales);
        sum += sales;                   // مجموع القيم
        s2 += sales * sales;            // مجموع المربعات
    }
    mean = sum / nbDays;
    var = s2 / nbDays - mean * mean;    // متوسط المربعات ناقص مربع المتوسط
    printf("Mean = %.2f\n", mean);
    printf("Variance = %.2f\n", var);
    return 0;
}
```

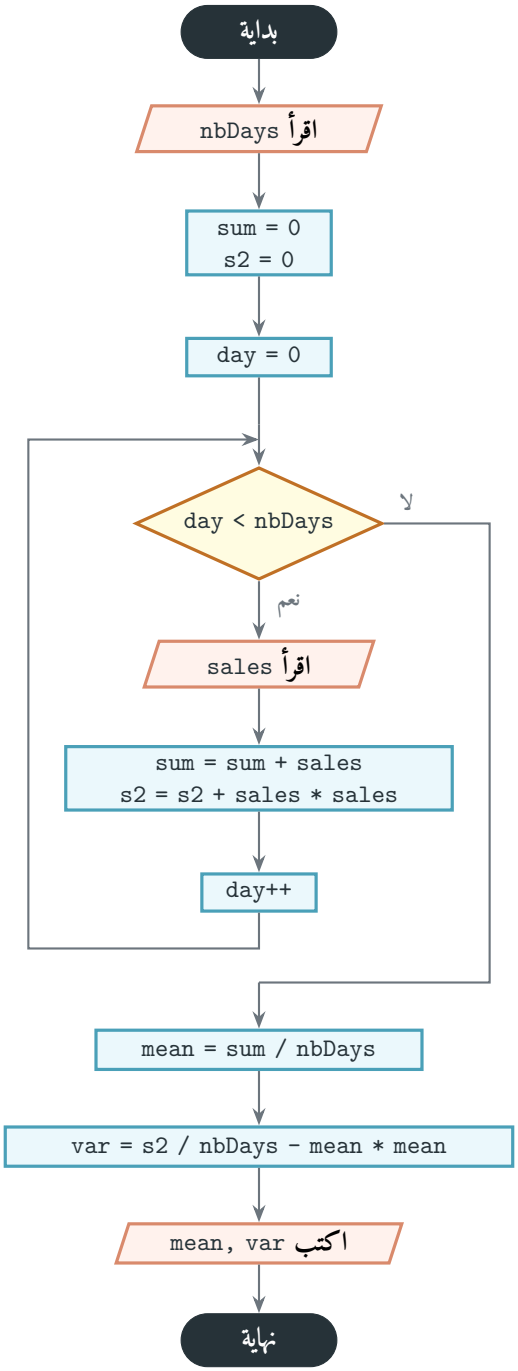
المدخلات

```
6
120 135 98 160 142 115
```

نتيجة التنفيذ

```
Mean = 128.33
Variance = 400.22
```

المخطط الإنسيابي (Organigramme)



تمرين 3.13 الجذر التربيعي بطريقة نيوتن

علم البيانات ★★ متوسط

نحتاج الانحراف المعياري، أي **جذر** التباين، دون `sqrt`. اكتب برنامجًا يحسب الجذر بطريقة نيوتن `root = (root + value / root) / 2` ويكرر **حتى** يصبح الفرق بين تقديرين متتاليين أقل من `1e-9`، ويعرض عدد التكرارات.

خطوات الحل

1) نبدأ بـ `root = value` (أو 1 إذا كان `value < 1`).

2) do: نحفظ old = root ثم نحسب root الجديد.
3) نكرر ما دام |root - old| > 1e-9.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double value, root, old, diff;
    int it = 0;
    scanf("%lf", &value);
    root = (value > 1) ? value : 1;          // تقدير أولي موجب
    do {
        old = root;
        root = (root + value / root) / 2;      // متوسط root و value/root
        it++;
        diff = root - old;
        if (diff < 0) diff = -diff;   // القيمة المطلقة
    } while (diff > 1e-9);
    printf("sqrt(%.2f) = %.9f\n", value, root);
    printf("Iterations: %d\n", it);
    return 0;
}
```

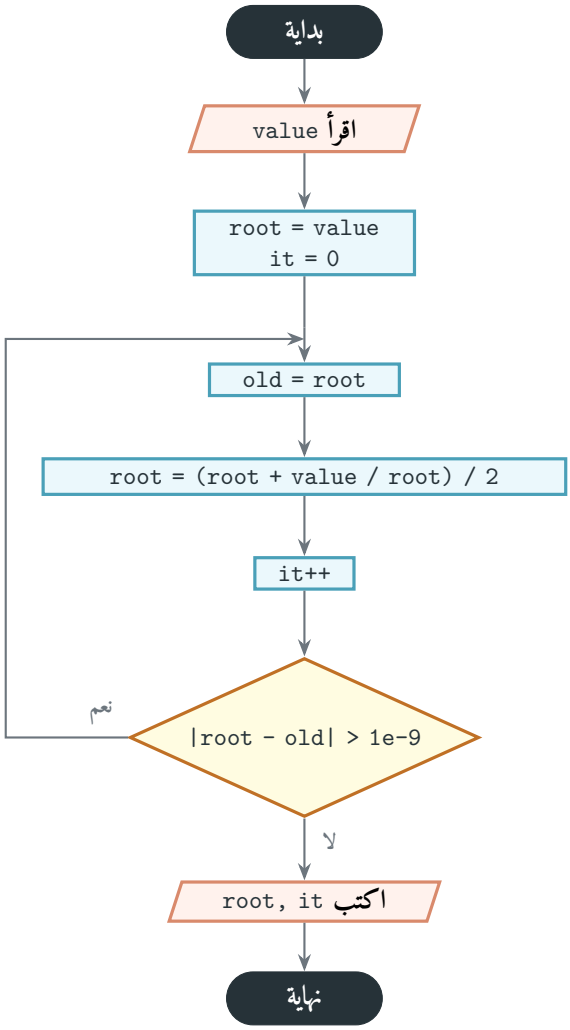
المدخلات

```
547.25
```

نتيجة التنفيذ

```
sqrt(547.25) = 23.393375131
Iterations: 10
```

المخطط الإنسيابي (Organigramme)



تمرين 3.14 ترتيبات عرض المنتجات

تجارة إلكترونية ★★ متوسط

في واجهة الصفحة الرئيسية `slots` خانات ونريد اختيار وترتيب منتجات من بين `nbProducts`. عدد **الترتيبات** هو $\text{nbProducts} \times (\text{nbProducts}-1) \times \ldots \times (\text{nbProducts}-\text{slots}+1) = \text{nbProducts}! \,/\, (\text{nbProducts}-\text{slots})!$.
اكتب برنامجًا يحسبه بحلقة، ويحسب أيضًا `nbProducts!`.

خطوات الحل

1) `arr = 1`، ثم نضرب في `nbProducts`، `nbProducts-1`، ... `slots` مرات.
2) `fact = 1` ثم نضرب في 1، 2، ...، `nbProducts`.
3) نستعمل `long long` لأن الأعداد تكبر بسرعة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbProducts, slots, step;
    long long arr = 1, fact = 1;  // long long لتجنب تجاوز السعة
```

```c
    scanf("%d %d", &nbProducts, &slots);
    for (step = 0; step < slots; step++)
        arr *= (nbProducts - step);          // (عوامل slots) ... × (nbProducts-1) × nbProducts
    for (step = 2; step <= nbProducts; step++)
        fact *= step;                        // nbProducts!
    printf("Arrangements A(%d,%d) = %lld\n", nbProducts, slots, arr);
    printf("%d! = %lld\n", nbProducts, fact);
    return 0;
}
```

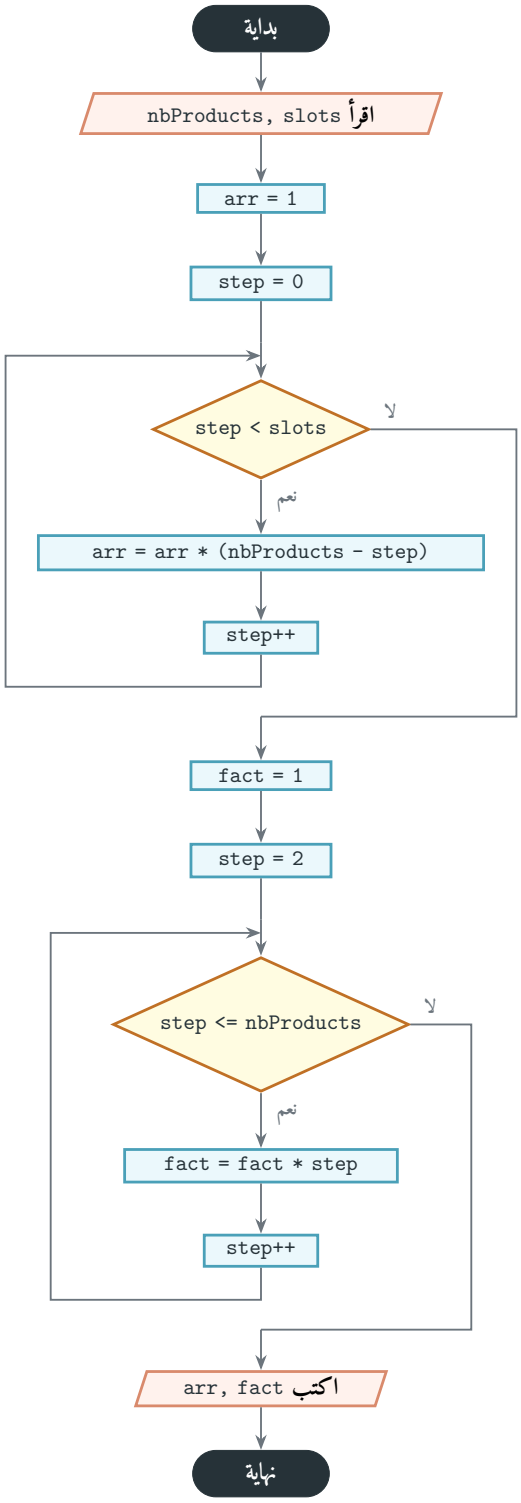
المدخلات

```
10 4
```

نتيجة التنفيذ

```
Arrangements A(10,4) = 5040
10! = 3628800
```

المخطط الإنسيابي (Organigramme)



تمرين 3.15 عدد الحزم الممكنة

تجارة إلكترونية ★★ متوسط

يريد المتجر تكوين حزم من size منتجات مختلفة من بين nbProducts (الترتيب غير مهم). عدد التوفيقات C(nbProducts,size). اكتب برنامجًا يحسبه بحلقة واحدة دون حساب العامليات الكاملة: C = C * (nbProducts - size + step) / step لكل step من 1 إلى size.

خطوات الحل

1) `bundles = 1`.
2) لكل `step` من 1 إلى `size`: نضرب أولًا ثم نقسم؛ القسمة دائمًا صحيحة بالضبط.
3) هذه الطريقة تتجنب الأعداد الضخمة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbProducts, size, step;
    long long bundles = 1;
    scanf("%d %d", &nbProducts, &size);
    for (step = 1; step <= size; step++) {
        // بعد هذه الخطوة bundles = C(nbProducts-size+step, step)، عدد صحيح دائمًا
        bundles = bundles * (nbProducts - size + step) / step;
    }
    printf("Bundles C(%d,%d) = %lld\n", nbProducts, size, bundles);
    return 0;
}
```

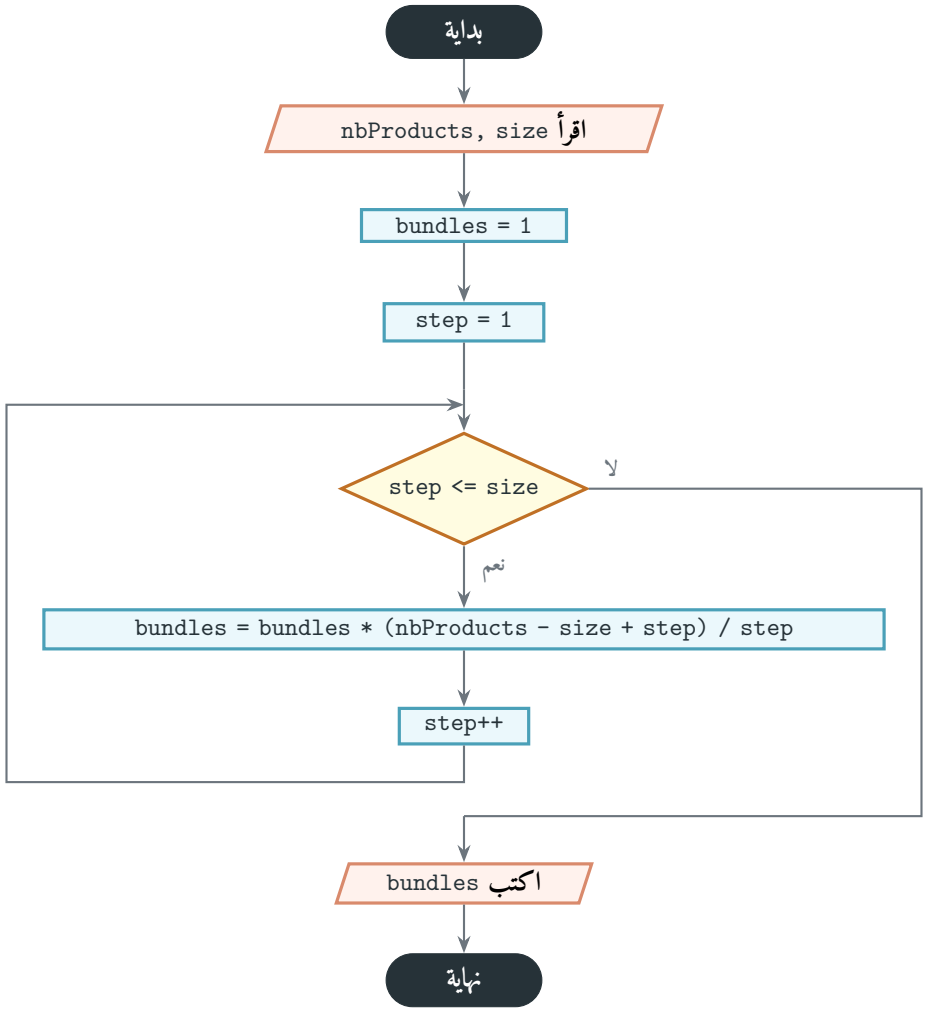
المدخلات

```
20 5
```

نتيجة التنفيذ

```
Bundles C(20,5) = 15504
```

المخطط الإنسيابي (Organigramme)



تمرين 3.16 أكبر قاسم مشترك لأحجام العبوات

تجارة إلكترونية ★★ متوسط

يصل منتج في كراتين من packA وحدة ومنتج آخر في كراتين من packB وحدة. لتكوين رفوف متساوية دون باقٍ نحتاج **أكبر قاسم مشترك**. اكتب برنامجًا يحسبه بخوارزمية **إقليدس**.

خطوات الحل

1) ما دام packB != 0: rest = packA % packB ثم packA = packB وpackB = rest.
2) عند التوقف، packA هو القاسم المشترك الأكبر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int packA, packB, rest, a0, b0;
    scanf("%d %d", &packA, &packB);
    a0 = packA; b0 = packB;              // نحفظ القيم الأصلية للعرض
    while (packB != 0) {
```

```c
        rest = packA % packB;               // pgcd(packA, packB) = pgcd(packB, packA % packB)
        packA = packB;
        packB = rest;
    }
    printf("GCD(%d, %d) = %d\n", a0, b0, packA);
    printf("Shelves of %d units: %d + %d\n", packA, a0 / packA, b0 / packA);
    return 0;
}
```

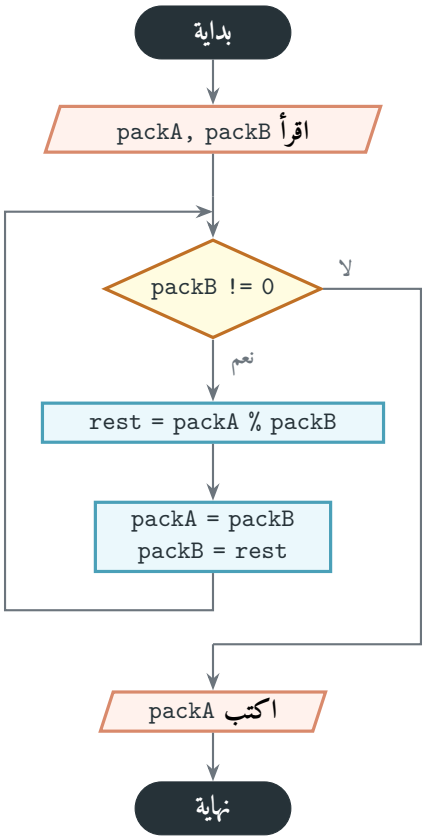
المدخلات

```
84 120
```

نتيجة التنفيذ

```
GCD(84, 120) = 12
Shelves of 12 units: 7 + 10
```

المخطط الإنسيابي (Organigramme)



تمرين 3.17 تزامن دورات التموين

تجارة إلكترونية ★★ متوسط

يُموَّن منتج كل cycleA أيام ومنتج آخر كل cycleB أيام، وقد مُوِّنا معًا اليوم. بعد كم يومًا يُموَّنان معًا من جديد؟ اكتب برنامجًا يحسب المضاعف المشترك الأصغر lcm = cycleA / gcd * cycleB.

خطوات الحل

1) نحسب `gcd(cycleA, cycleB)` بخوارزمية إقليدس على نسختين من المتغيرين.

2) `lcm = cycleA / g * cycleB` (نقسم أولًا لتجنب تجاوز السعة).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int cycleA, cycleB, gcdA, gcdB, rest, lcm;
    scanf("%d %d", &cycleA, &cycleB);
    gcdA = cycleA; gcdB = cycleB;                 // نسختان لكي لا نفقد cycleA و cycleB
    while (gcdB != 0) {
        rest = gcdA % gcdB;
        gcdA = gcdB;
        gcdB = rest;
    }                                 // gcdA = pgcd(cycleA, cycleB)
    lcm = cycleA / gcdA * cycleB;
    printf("Both restocked again after %d days\n", lcm);
    return 0;
}
```

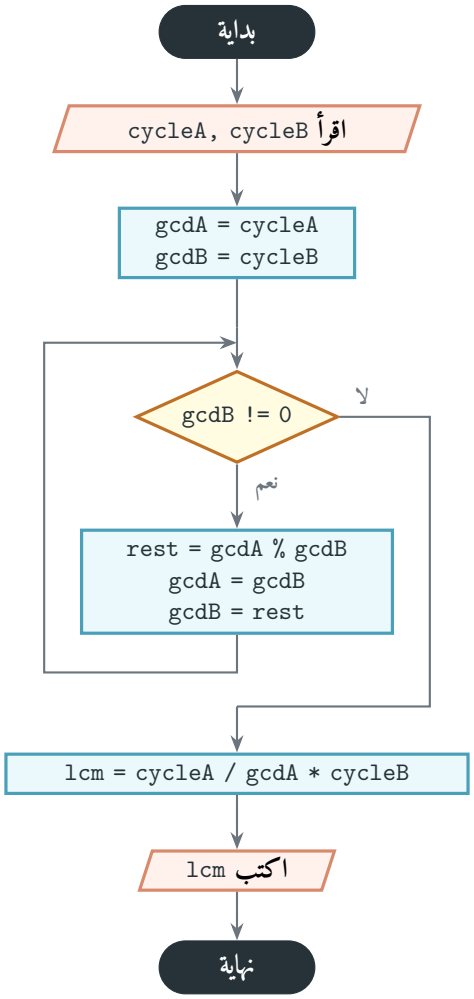
المدخلات

```
6 15
```

نتيجة التنفيذ

```
Both restocked again after 30 days
```

المخطط الإنسيابي (Organigramme)



تمرين 3.18 هل رقم المنتج أولي؟

تجارة إلكترونية ★★ متوسط

لتوزيع المنتجات على خوادم التخزين نستعمل أرقامًا أولية. اكتب برنامجًا يتحقق إن كان عدد ما أوليًا، باختبار القواسم من 2 حتى `divisor * divisor <= number` فقط.

خطوات الحل

1) إذا كان `number < 2` فهو ليس أوليًا.
2) نجرب القواسم `divisor` ما دام `divisor * divisor <= number`، نتوقف عند أول قاسم.
3) إذا لم نجد قاسمًا فهو أولي.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int number, divisor, prime;
    scanf("%d", &number);
    prime = (number >= 2);              // 0 و 1 ليسا أوليين
```

```c
    // يكفي البحث حتى الجذر: إذا كان number = a × b فأحدهما <= الجذر
    for (divisor = 2; prime && divisor * divisor <= number; divisor++) {
        if (number % divisor == 0)
            prime = 0;              // وجدنا قاسمًا
    }
    if (prime)
        printf("%d is prime\n", number);
    else
        printf("%d is not prime\n", number);
    return 0;
}
```

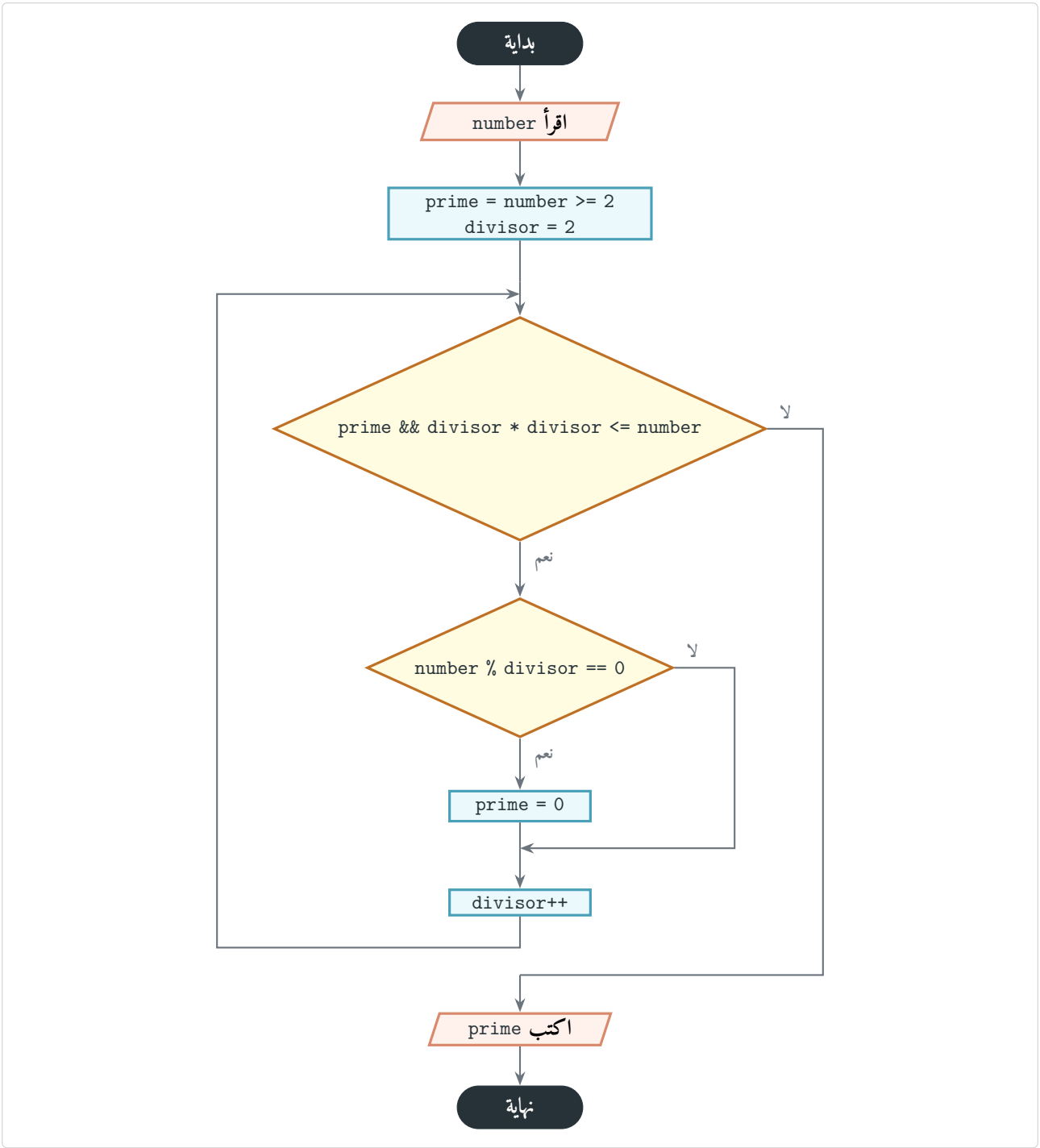
المدخلات

```
7919
```

نتيجة التنفيذ

```
7919 is prime
```

المخطط الإنسيابي (Organigramme)



تمرين 3.19 أرقام رقم الطلبية

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يقرأ رقم طلبية (عدد صحيح موجب طويل) ويحسب **عدد أرقامه** و**مجموعها** و**أكبر رقم** فيه، بحلقة تقسم على 10.

خطوات الحل

1) ما دام `number > 0`: الرقم الأخير `digit = number % 10` ثم `number = number / 10`.
2) نزيد العداد ونضيف `digit` للمجموع ونحدّث الأكبر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long number;
    int digit, count = 0, sum = 0, big = 0;
    scanf("%ld", &number);
    do {                            // do-while: الرقم 0 له رقم واحد
        digit = number % 10;                // آخر رقم
        count++;
        sum += digit;
        if (digit > big) big = digit;
        number = number / 10;               // نحذف آخر رقم
    } while (number > 0);
    printf("Digits = %d, sum = %d, largest = %d\n", count, sum, big);
    return 0;
}
```

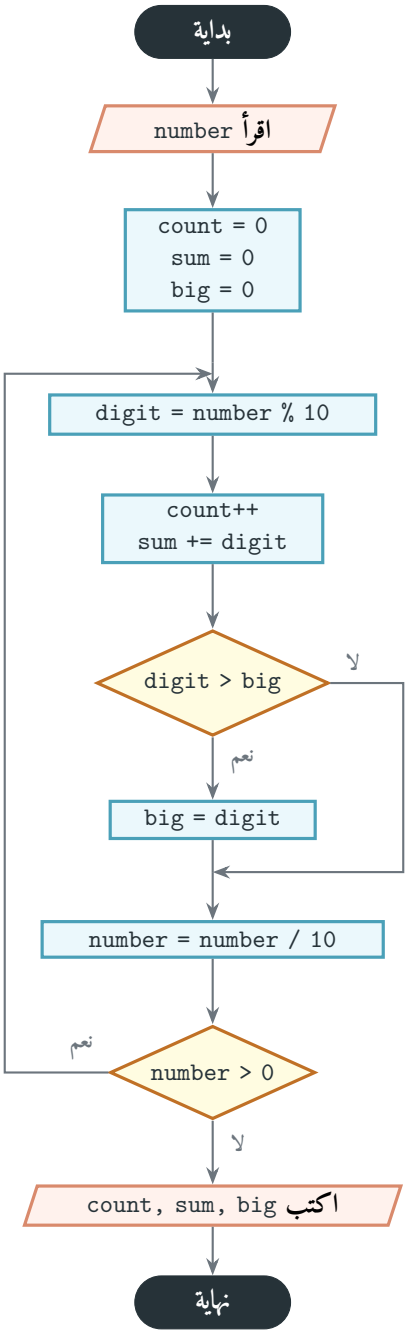
المدخلات

```
90417253
```

نتيجة التنفيذ

```
Digits = 8, sum = 31, largest = 9
```

المخطط الإنسيابي (Organigramme)



تمرين 3.20 رمز تخفيض متناظر

تجارة إلكترونية ★★ متوسط

رموز التخفيض الخاصة هي أعداد **متناظرة** (تُقرأ بنفس الطريقة من اليسار واليمين مثل 12321). اكتب برنامجًا يعكس العدد بحلقة ويتحقق إن كان متناظرًا.

خطوات الحل

1) `rev = 0`، وما دام `code > 0`: `rev = rev * 10 + code % 10` ثم `code /= 10`.
2) نقارن `rev` بالعدد الأصلي.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long code, work, rev = 0;
    scanf("%ld", &code);
    work = code;                          // نعمل على نسخة
    while (work > 0) {
        rev = rev * 10 + work % 10;  // نلصق آخر رقم في نهاية rev
        work = work / 10;
    }
    printf("Reversed: %ld\n", rev);
    if (rev == code)
        printf("%ld is a valid palindrome code\n", code);
    else
        printf("%ld is not a palindrome\n", code);
    return 0;
}
```

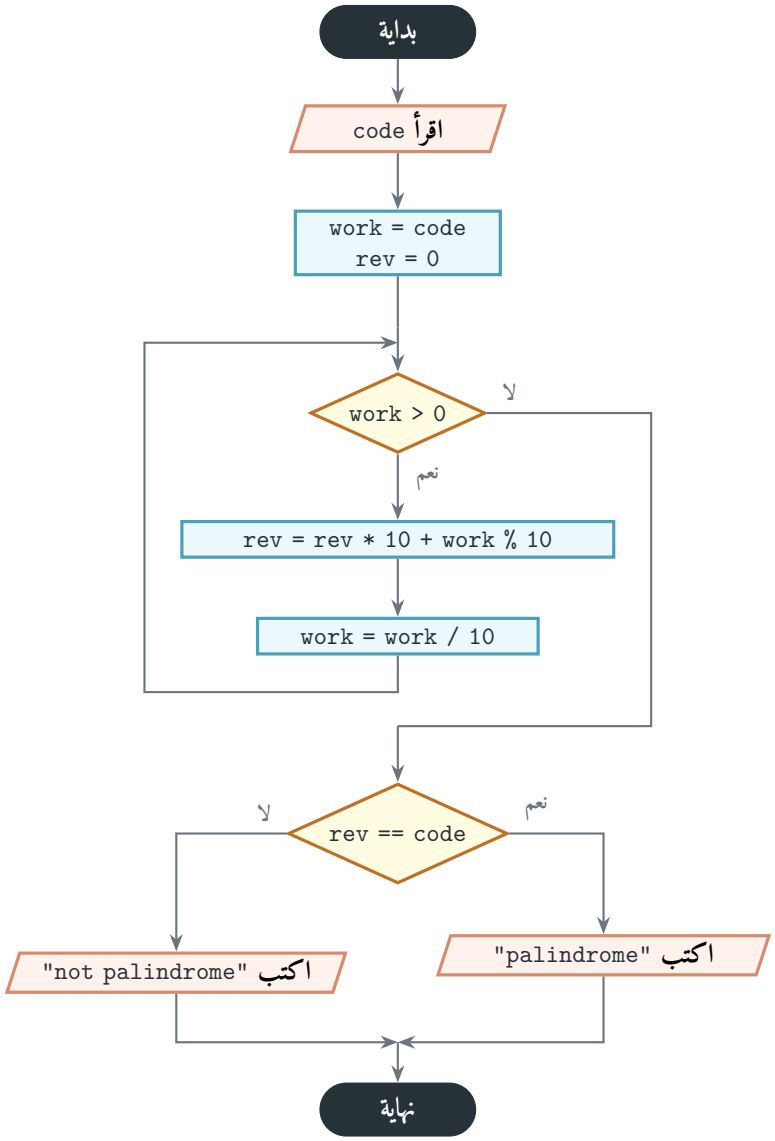
المدخلات

```
45654
```

نتيجة التنفيذ

```
Reversed: 45654
45654 is a valid palindrome code
```

المخطط الإنسيابي (Organigramme)



تمرين 3.21 نمو المستخدمين بمتتالية فيبوناتشي

علم البيانات ★★ متوسط

في نموذج نمو مبسط، عدد المستخدمين الجدد كل شهر يساوي مجموع الشهرين السابقين (متتالية **فيبوناتشي**)، بدءًا من 1 و1. اكتب برنامجًا يعرض الأشهر حتى يتجاوز العدد الشهري `target`، مع المجموع التراكمي.

خطوات الحل

1) `prev = 1`، `current = 1` (الشهران الأولان).
2) في كل دورة: `next = prev + current` ثم نزيح `prev = current` و`current = next`.
3) نتوقف عندما يتجاوز العدد الهدف.

البرنامج بلغة C

```c
#include <stdio.h>

```

```c
int main(void) {
    long prev = 1, current = 1, next, total = 2, target;
    int month = 2;
    scanf("%ld", &target);
    printf("Month 1: 1\nMonth 2: 1\n");
    while (current <= target) {
        next = prev + current;                // الشهر الجديد = مجموع الشهرين السابقين
        prev = current;                       // نزيح النافذة
        current = next;
        total += next;
        month++;
        printf("Month %d: %ld\n", month, next);
    }
    printf("Total users after %d months: %ld\n", month, total);
    return 0;
}
```

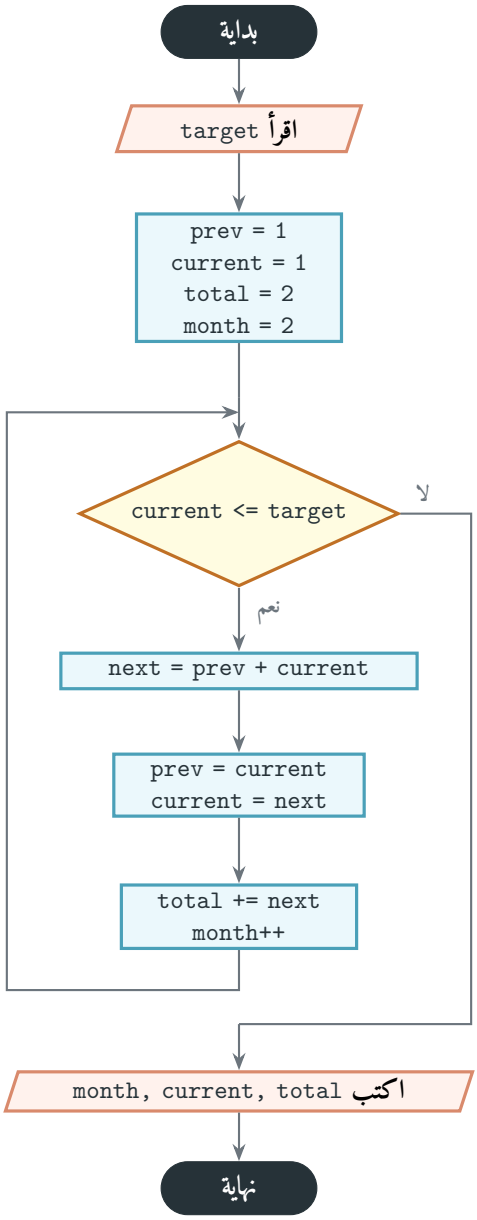
المدخلات

```
500
```

نتيجة التنفيذ

```
Month 1: 1
Month 2: 1
Month 3: 2
Month 4: 3
Month 5: 5
Month 6: 8
Month 7: 13
Month 8: 21
Month 9: 34
Month 10: 55
Month 11: 89
Month 12: 144
Month 13: 233
Month 14: 377
Month 15: 610
Total users after 15 months: 1596
```

المخطط الإنسيابي (Organigramme)



تمرين 3.22 الفائدة المركبة على عدة سنوات

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب قيمة استثمار `capital` بعد `years` سنوات بنسبة فائدة سنوية `rate`%، ويعرض القيمة في نهاية كل سنة، دون `pow`.

خطوات الحل

1) `factor = 1 + rate / 100`.
2) حلقة على السنوات: `capital = capital * factor` ثم العرض.

البرنامج بلغة C

```c
#include <stdio.h>

```

```c
int main(void) {
    double capital, rate, factor;
    int years, year;
    scanf("%lf %lf %d", &capital, &rate, &years);
    factor = 1 + rate / 100;
    for (year = 1; year <= years; year++) {
        capital = capital * factor;                // القوة تُبنى بالضرب المتكرر
        printf("Year %d: %.2f DA\n", year, capital);
    }
    return 0;
}
```

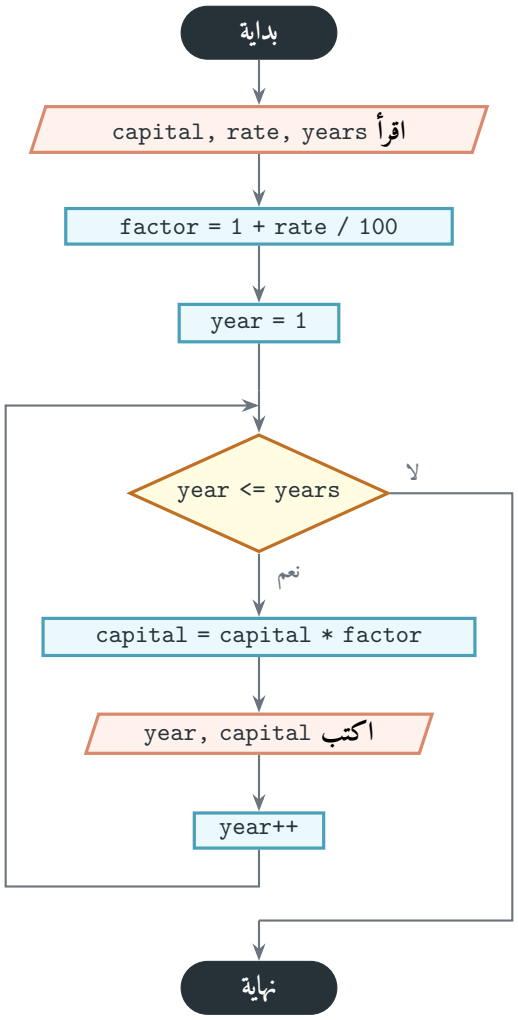
المدخلات

```
100000 6 5
```

نتيجة التنفيذ

```
Year 1: 106000.00 DA
Year 2: 112360.00 DA
Year 3: 119101.60 DA
Year 4: 126247.70 DA
Year 5: 133822.56 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.23 جدول تسديد الشراء بالتقسيط

تجارة إلكترونية ★★ متوسط

اشترى زبون هاتفًا بالتقسيط: المبلغ balance، فائدة شهرية rate% على الباقي، وقسط شهري ثابت installment. اكتب برنامجًا يعرض لكل شهر الفائدة والباقي، حتى السداد الكامل، مع مجموع الفوائد.

خطوات الحل

1) ما دام الباقي balance > 0: الفائدة interest = balance * rate / 100.
2) balance = balance + interest - installment؛ إذا أصبح سالبًا فالقسط الأخير أصغر.
3) نجمع الفوائد.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double balance, rate, installment, interest, totI = 0;
    int month = 0;
    scanf("%lf %lf %lf", &balance, &rate, &installment);
    printf("Month | Interest | Remaining\n");
    while (balance > 0) {
        interest = balance * rate / 100;          // فائدة الشهر على الباقي
        totI += interest;
        balance = balance + interest - installment;          // نضيف الفائدة ونطرح القسط
        month++;
        if (balance < 0) balance = 0;          // القسط الأخير يسدد ما تبقى فقط
        printf("%5d | %8.2f | %9.2f\n", month, interest, balance);
    }
    printf("Total interest = %.2f DA\n", totI);
    return 0;
}
```

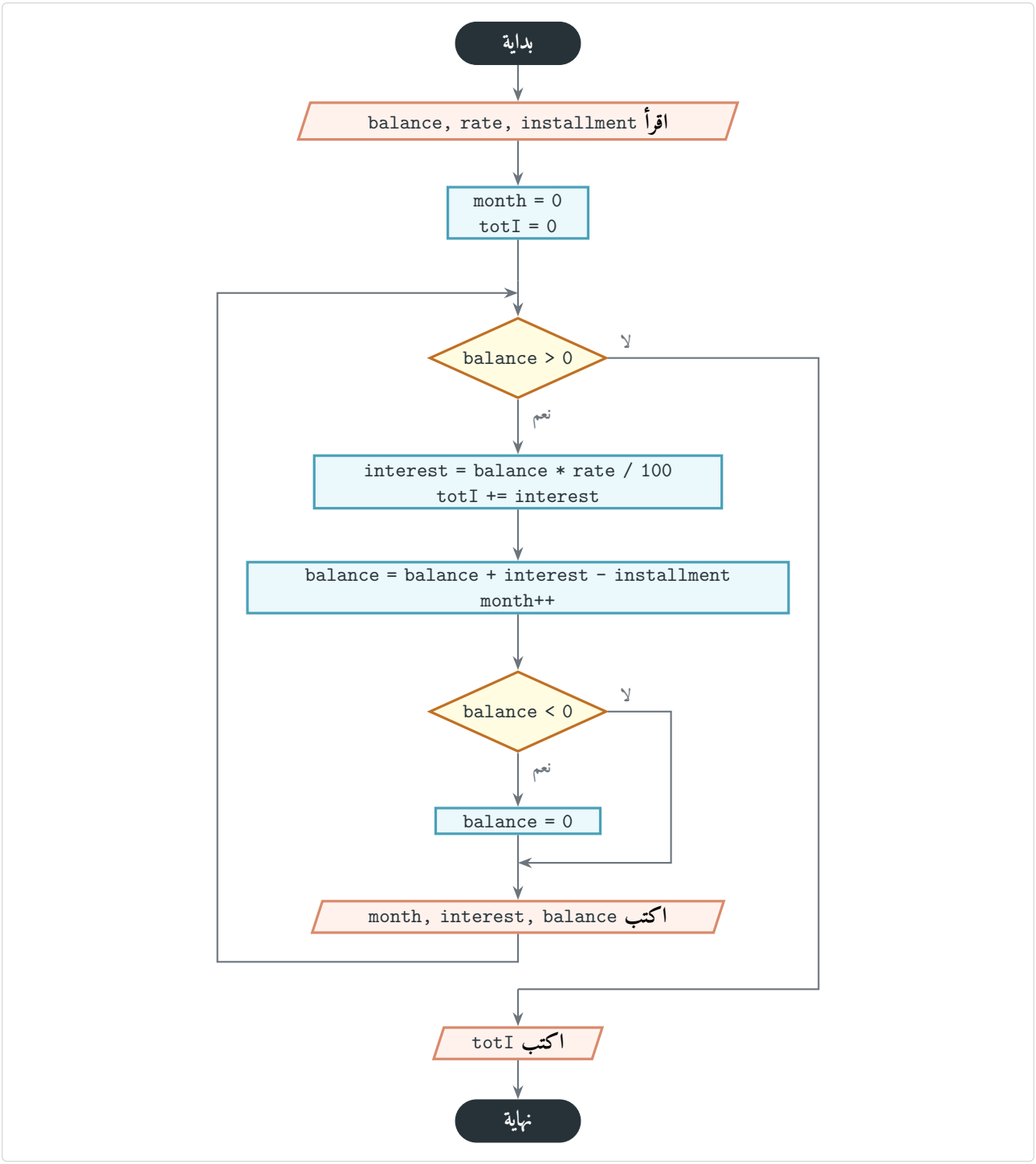
المدخلات

```
60000 1.5 11000
```

نتيجة التنفيذ

```
Month | Interest | Remaining
    1 |   900.00 |  49900.00
    2 |   748.50 |  39648.50
    3 |   594.73 |  29243.23
    4 |   438.65 |  18681.88
    5 |   280.23 |   7962.10
    6 |   119.43 |      0.00
Total interest = 3081.54 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.24 أطول فترة نمو متواصل

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ مبيعات `nbDays` يومًا متتالية ويحدد **أطول سلسلة** من الأيام التي ارتفعت فيها المبيعات مقارنة باليوم السابق، ويوم نهايتها.

خطوات الحل

1) نحتفظ بالقيمة السابقة `prev`.

2) إذا كان `sales > prev` نزيد طول السلسلة الحالية `cur`، وإلا نعيدها إلى 0.

3) نحدّث الأطول `best` ويوم نهايته.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day, cur = 0, best = 0, end = 1;
    double prev, sales;
    scanf("%d %lf", &nbDays, &prev);
    for (day = 2; day <= nbDays; day++) {
        scanf("%lf", &sales);
        if (sales > prev)
            cur++;                  // السلسلة تستمر
        else
            cur = 0;                // السلسلة انقطعت
        if (cur > best) {
            best = cur;
            end = day;
        }
        prev = sales;                   // اليوم الحالي يصبح السابق
    }
    printf("Longest growth streak: %d days (ending day %d)\n", best, end);
    return 0;
}
```

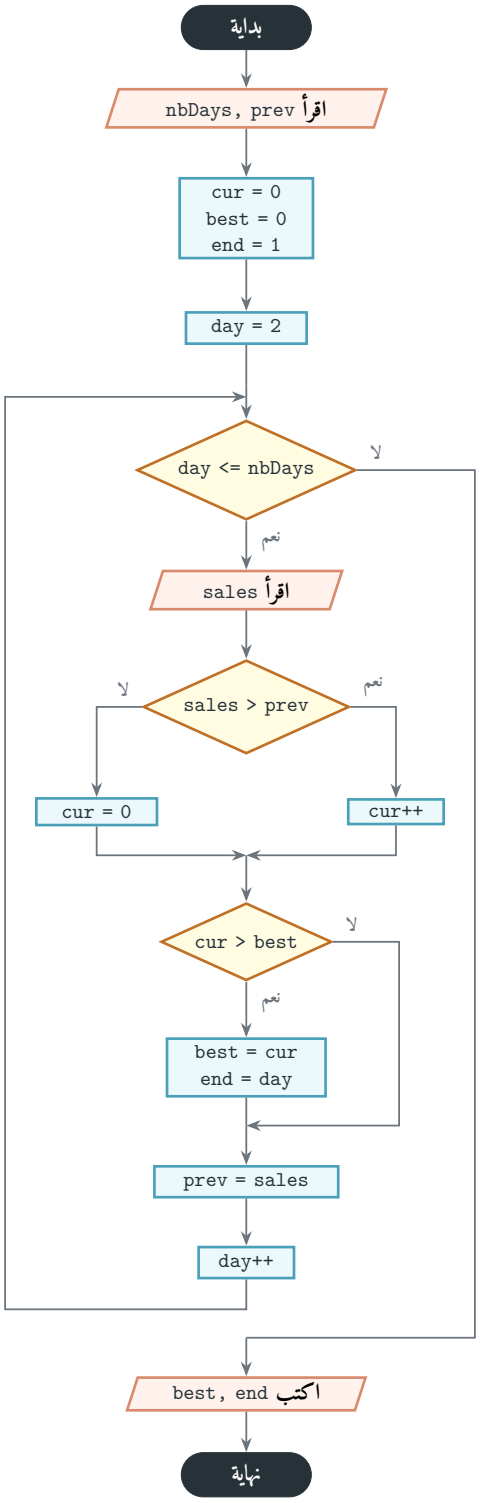
المدخلات

```
10
50 55 60 58 61 64 70 75 72 80
```

نتيجة التنفيذ

```
Longest growth streak: 4 days (ending day 8)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.25 المتوسط المتحرك لثلاثة أيام

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ مبيعات `nbDays` يومًا ويعرض ابتداءً من اليوم الثالث **المتوسط المتحرك** لآخر 3 أيام، دون جداول: نحتفظ بآخر قيمتين في متغيرين.

خطوات الحل

1) نقرأ أول قيمتين في `p2` (قبل الأخيرة) و`p1` (الأخيرة).
2) لكل قيمة جديدة `sales`: المتوسط `(p2 + p1 + sales) / 3`.
3) نزيح النافذة: `p2 = p1` ثم `p1 = sales`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day;
    double p2, p1, sales;
    scanf("%d %lf %lf", &nbDays, &p2, &p1);     // أول يومين
    for (day = 3; day <= nbDays; day++) {
        scanf("%lf", &sales);
        printf("Day %d: MA3 = %.2f\n", day, (p2 + p1 + sales) / 3);
        p2 = p1;                    // إزاحة النافذة بمقدار يوم
        p1 = sales;
    }
    return 0;
}
```

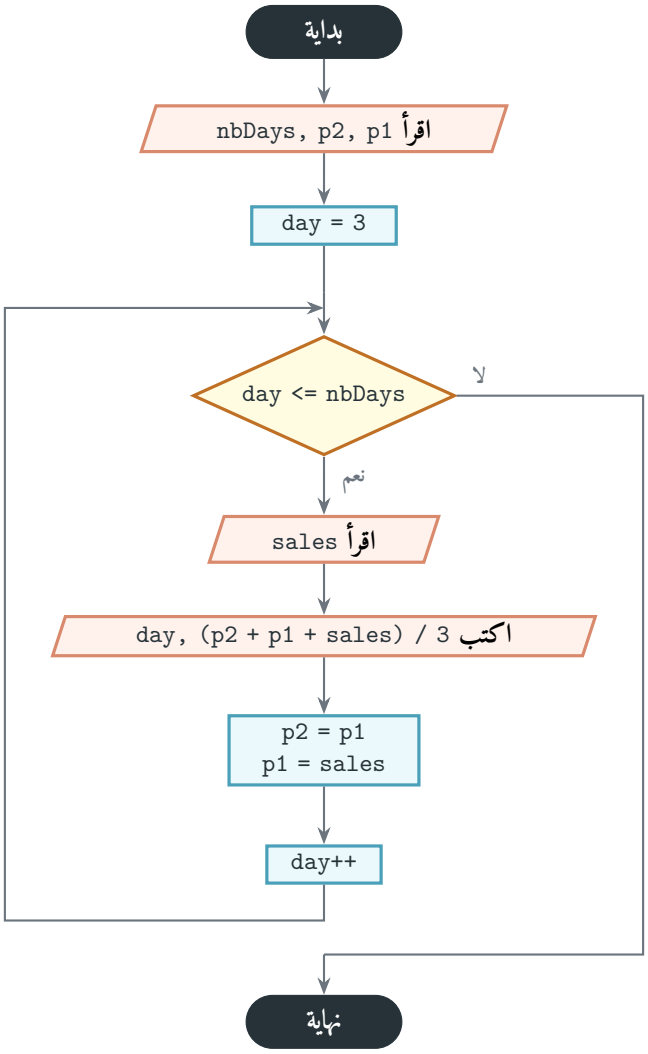
المدخلات

```
7
100 130 115 160 140 170 150
```

نتيجة التنفيذ

```
Day 3: MA3 = 115.00
Day 4: MA3 = 135.00
Day 5: MA3 = 138.33
Day 6: MA3 = 156.67
Day 7: MA3 = 153.33
```

المخطط الإنسيابي (Organigramme)



تمرين 3.26 مدرج تكراري للتقييمات

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ nbReviews تقييمات (1 إلى 5) ويعد تكرار كل قيمة في خمسة عدادات، ثم يرسم **مدرجًا تكراريًا** نصيًا بالرمز * لكل نجمة.

خطوات الحل

1) خمسة عدادات c1 إلى c5 نزيدها بـ switch.
2) لرسم السطر stars نستعمل حلقة داخلية تطبع * عددًا من المرات يساوي العداد.
3) لتجنب تكرار الكود نختار العداد في كل دورة بـ switch.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbReviews, index, rating, stars, star, bar;
    int c1 = 0, c2 = 0, c3 = 0, c4 = 0, c5 = 0;
```

```c
    scanf("%d", &nbReviews);
    for (index = 0; index < nbReviews; index++) {
        scanf("%d", &rating);
        switch (rating) {              // نزيد عداد القيمة المقروءة
            case 1: c1++; break;
            case 2: c2++; break;
            case 3: c3++; break;
            case 4: c4++; break;
            case 5: c5++; break;
        }
    }
    for (stars = 5; stars >= 1; stars--) {    // من 5 نجوم إلى نجمة واحدة
        switch (stars) {
            case 5: bar = c5; break;
            case 4: bar = c4; break;
            case 3: bar = c3; break;
            case 2: bar = c2; break;
            default: bar = c1;
        }
        printf("%d stars | ", stars);
        for (star = 0; star < bar; star++)   // حلقة داخلية: طول العمود
            printf("*");
        printf(" (%d)\n", bar);
    }
    return 0;
}
```

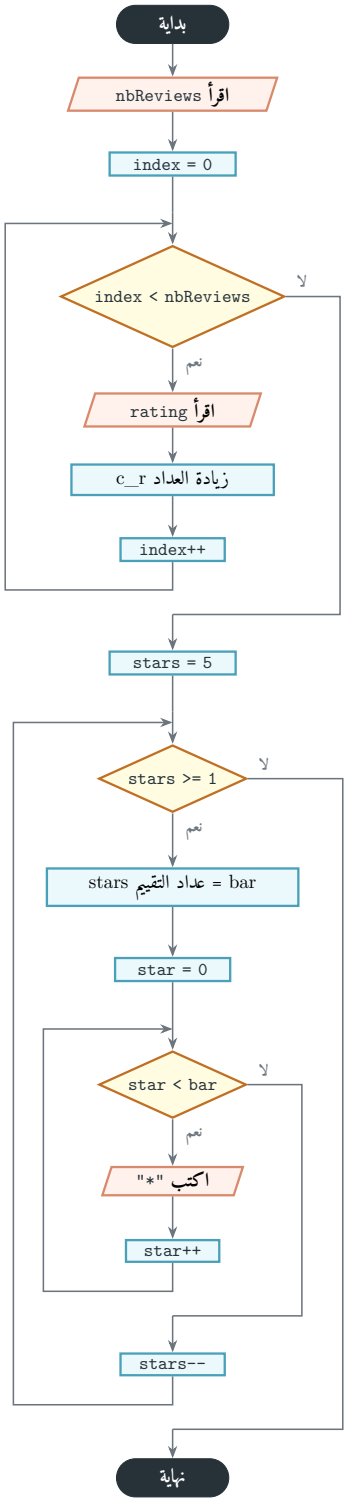
المدخلات

```
15
5 4 5 3 5 4 4 2 5 1 4 5 3 4 5
```

نتيجة التنفيذ

```
5 stars | ****** (6)
4 stars | ***** (5)
3 stars | ** (2)
2 stars | * (1)
1 stars | * (1)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.27 رقم المنتج بالنظام الثنائي

تجارة إلكترونية ★★ متوسط

تخزن بطاقات RFID أرقام المنتجات بالنظام الثنائي. اكتب برنامجًا يحول عددًا عشريًا إلى الثنائي دون جداول: نبني الناتج كعدد يُقرأ بالأرقام 0 و1، ونعد البتات التي قيمتها 1.

خطوات الحل

1) ما دام `number > 0`: البت `bit = number % 2` ثم `number = number / 2`.
2) البتات تخرج من اليمين إلى اليسار، فنضعها في الموضع `place` (1، 10، 100، ...): `bin = bin + bit * place`.
3) نضاعف `place` في 10 في كل دورة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int number, n0, bit, ones = 0;
    long long bin = 0, place = 1;
    scanf("%d", &number);
    n0 = number;
    while (number > 0) {
        bit = number % 2;              // البت الأقل وزنًا
        ones += bit;
        bin = bin + bit * place;       // نضعه في موضعه العشري
        place = place * 10;
        number = number / 2;           // ننتقل إلى البت الموالي
    }
    printf("%d in binary = %lld (%d ones)\n", n0, bin, ones);
    return 0;
}
```

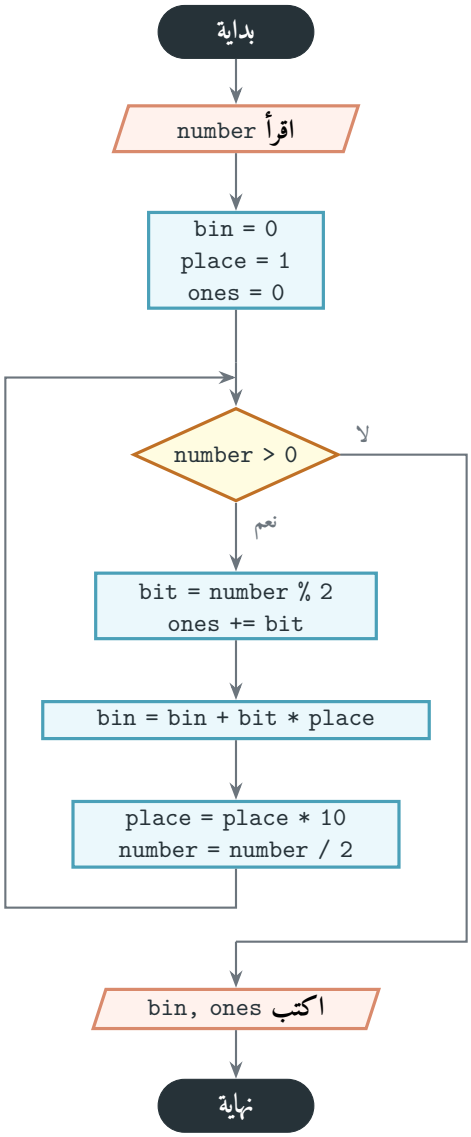
المدخلات

```
202
```

نتيجة التنفيذ

```
202 in binary = 11001010 (4 ones)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.28 شبكة التخفيضات

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يعرض جدولًا ثنائيًا: الأسطر كميات من 1 إلى 5، والأعمدة نسب تخفيض 0% و10% و20% و30%، وكل خانة تحتوي ثمن الكمية بعد التخفيض لسعر وحدة مقروء. استعمل **حلقتين متداخلتين**.

خطوات الحل

1) الحلقة الخارجية على الكمية `qty`.
2) الحلقة الداخلية على النسبة `discount` من 0 إلى 30 بخطوة 10.
3) الخانة: `qty * price * (100 - discount) / 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```c
    double price;
    int qty, discount;
    scanf("%lf", &price);
    printf("qty |     0%%    10%%    20%%    30%%\n");
    for (qty = 1; qty <= 5; qty++) {            // سطر لكل كمية
        printf("%3d |", qty);
        for (discount = 0; discount <= 30; discount += 10)     // عمود لكل نسبة
            printf(" %6.0f", qty * price * (100 - discount) / 100);
        printf("\n");
    }
    return 0;
}
```

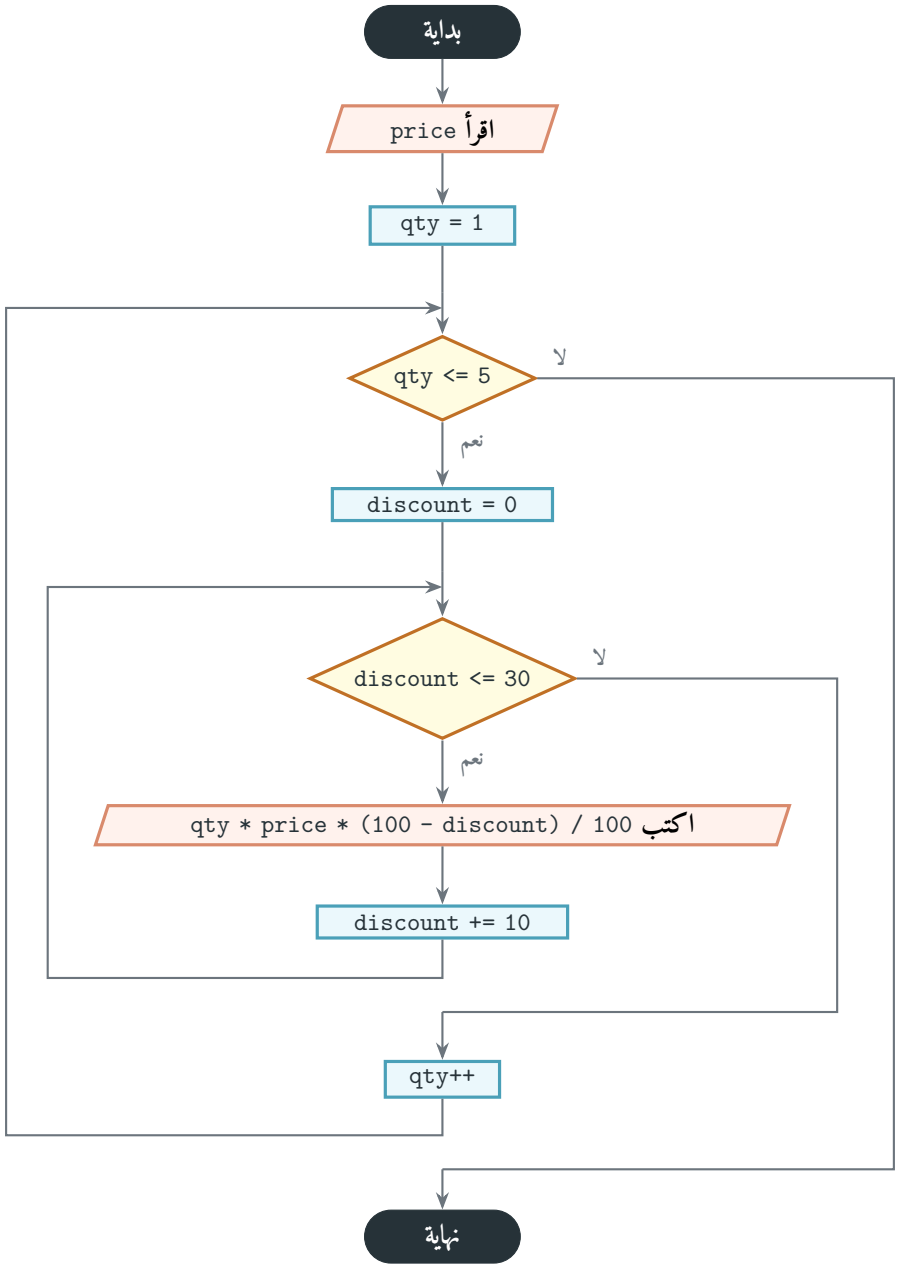
المدخلات

```
1500
```

نتيجة التنفيذ

```
qty |     0%    10%    20%    30%
  1 |   1500   1350   1200   1050
  2 |   3000   2700   2400   2100
  3 |   4500   4050   3600   3150
  4 |   6000   5400   4800   4200
  5 |   7500   6750   6000   5250
```

المخطط الإنسيابي (Organigramme)



تمرين 3.29 عد الزبائن في كل شريحة

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ الإنفاق السنوي لـ `nbCustomers` زبونًا ويعد الزبائن في كل شريحة: أقل من 20000، من 20000 إلى أقل من 100000، و100000 فأكثر، مع متوسط إنفاق كل شريحة.

خطوات الحل

1) ثلاثة عدادات وثلاثة مجاميع.
2) في كل دورة نحدد الشريحة بـ `if / else if`.
3) المتوسط لكل شريحة إذا كان عددها غير صفري.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbCustomers, index, n1 = 0, n2 = 0, n3 = 0;
    double spend, s1 = 0, s2 = 0, s3 = 0;
    scanf("%d", &nbCustomers);
    for (index = 0; index < nbCustomers; index++) {
        scanf("%lf", &spend);
        if (spend < 20000)       { n1++; s1 += spend; }   // صغار
        else if (spend < 100000) { n2++; s2 += spend; }   // متوسطون
        else                     { n3++; s3 += spend; }   // كبار
    }
    printf("Low:    %d customers", n1);
    if (n1 > 0) printf(", avg %.0f", s1 / n1);   // تجنب القسمة على 0
    printf("\nMedium: %d customers", n2);
    if (n2 > 0) printf(", avg %.0f", s2 / n2);
    printf("\nHigh:   %d customers", n3);
    if (n3 > 0) printf(", avg %.0f", s3 / n3);
    printf("\n");
    return 0;
}
```

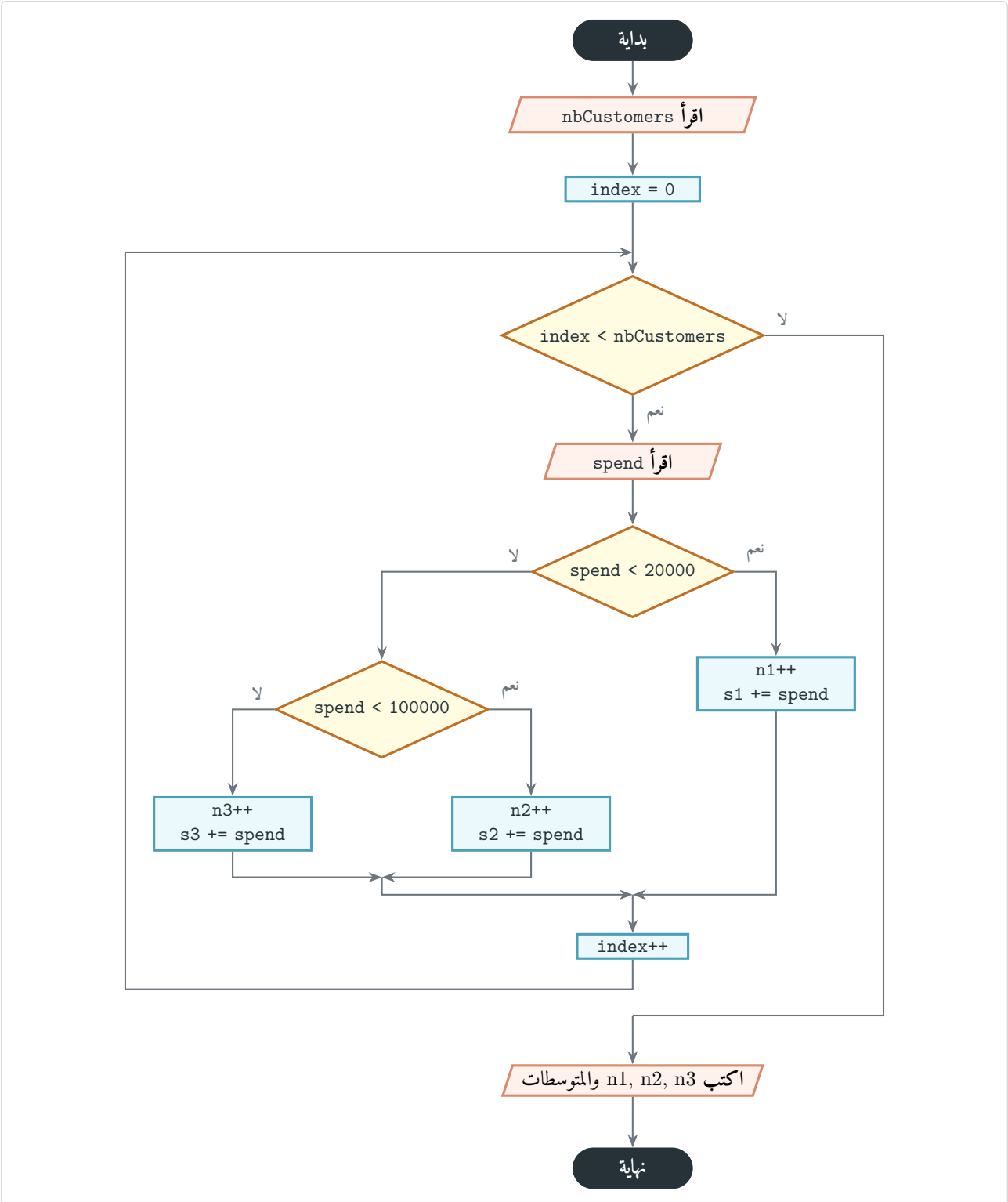
المدخلات

```
9
15000 120000 45000 8000 99000 250000 30000 61000 19999
```

نتيجة التنفيذ

```
Low:    3 customers, avg 14333
Medium: 4 customers, avg 58750
High:   2 customers, avg 185000
```

المخطط الإنسيابي (Organigramme)



تمرين 3.30 محاكاة نفاد المخزون

تجارة إلكترونية

متوسط

مخزون منتج `stock` وحدة. الطلب اليومي يبدأ بـ `initDemand` وحدة ويرتفع بنسبة `growth`% كل يوم (منتج رائج). اكتب برنامجًا يحاكي الأيام حتى **نفاد المخزون**، ويعرض يوم النفاد والطلب غير الملبى في آخر يوم.

خطوات الحل

1) ما دام `stock > 0`: نزيد اليوم، الطلب اليوم `dem` (مقرب لعدد صحيح).
2) إذا كان `dem >= stock` فالمخزون ينفد اليوم والناقص `dem - stock`.
3) وإلا `stock = stock - dem` ثم `dem = dem * (1 + growth / 100)`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double stock, initDemand, growth, dem;
    int day = 0;
    scanf("%lf %lf %lf", &stock, &initDemand, &growth);
    dem = initDemand;
    while (stock > 0) {
        day++;
        dem = (int)(dem + 0.5);    // الطلب عدد صحيح من الوحدات
        if (dem >= stock) {            // آخر يوم: لا يكفي المخزون
            printf("Stock out on day %d (unmet demand: %.0f)\n", day, dem - stock);
            stock = 0;
        } else {
            stock = stock - dem;
            printf("Day %2d: sold %3.0f, left %4.0f\n", day, dem, stock);
            dem = dem * (1 + growth / 100);   // الطلب يرتفع
        }
    }
    return 0;
}
```

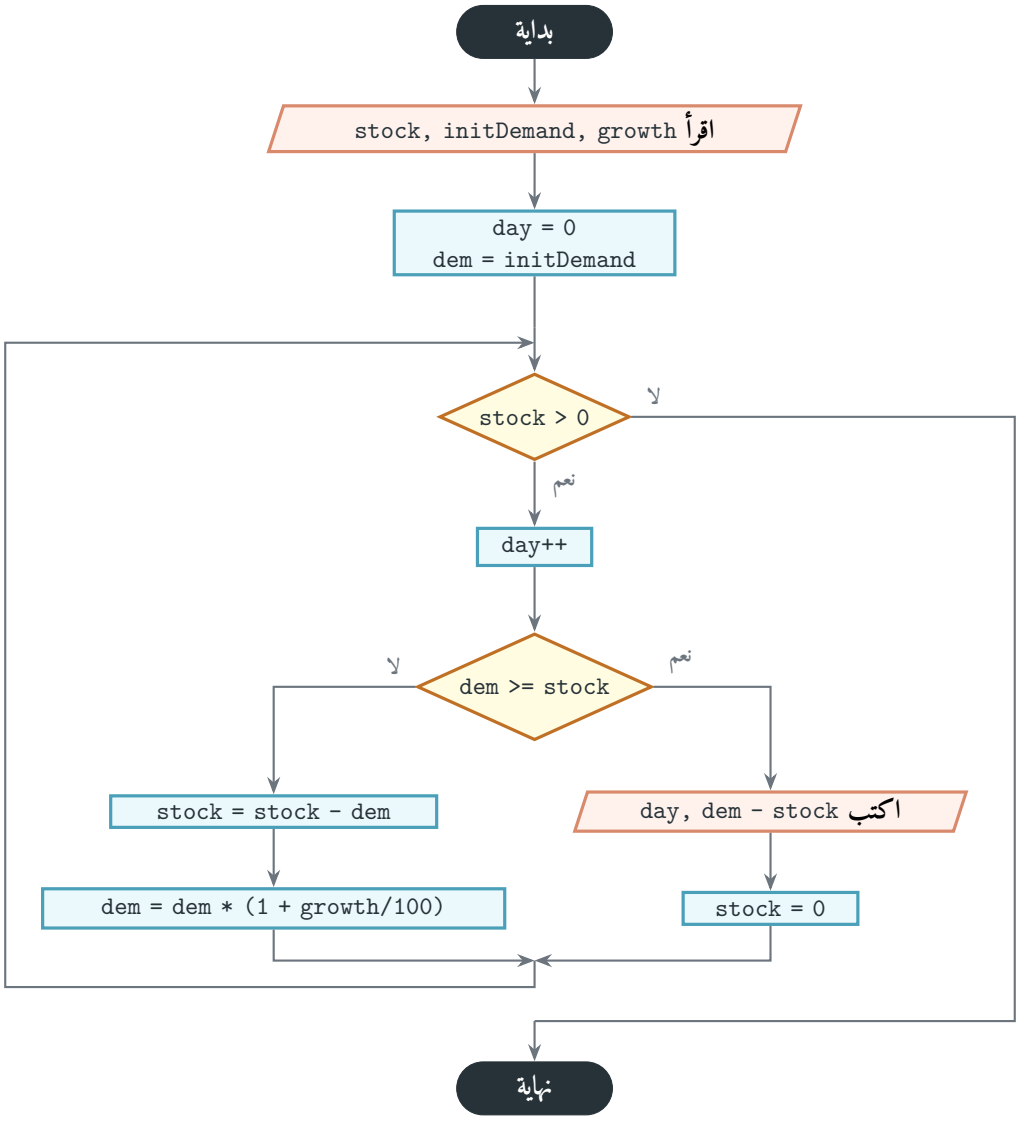
المدخلات

```
1000 40 10
```

نتيجة التنفيذ

```
Day  1: sold  40, left  960
Day  2: sold  44, left  916
Day  3: sold  48, left  868
Day  4: sold  53, left  815
Day  5: sold  58, left  757
Day  6: sold  64, left  693
Day  7: sold  70, left  623
Day  8: sold  77, left  546
Day  9: sold  85, left  461
Day 10: sold  94, left  367
Day 11: sold 103, left  264
Day 12: sold 113, left  151
Day 13: sold 124, left   27
Stock out on day 14 (unmet demand: 109)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.31 التنعيم الأسي لسلسلة زمنية

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ معامل التنعيم `alpha` وعدد الأيام `nbDays` ثم المبيعات، ويعرض لكل يوم القيمة المنعّمة `smooth = alpha * sales + (1 - alpha) * smooth` والتنبؤ لليوم الموالي.

خطوات الحل

1) القيمة الأولى تهيّئ `smooth`.
2) لكل قيمة موالية نطبق الصيغة.
3) التنبؤ لليوم `nbDays + 1` هو آخر قيمة `smooth`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```c
    double alpha, sales, smooth;
    int nbDays, day;
    scanf("%lf %d %lf", &alpha, &nbDays, &sales);
    smooth = sales;                                  // نبدأ بالملاحظة الأولى
    printf("Day 1: %.2f\n", smooth);
    for (day = 2; day <= nbDays; day++) {
        scanf("%lf", &sales);
        smooth = alpha * sales + (1 - alpha) * smooth;  // مزيج من الجديد والقديم
        printf("Day %d: %.2f\n", day, smooth);
    }
    printf("Forecast day %d: %.2f\n", nbDays + 1, smooth);
    return 0;
}
```

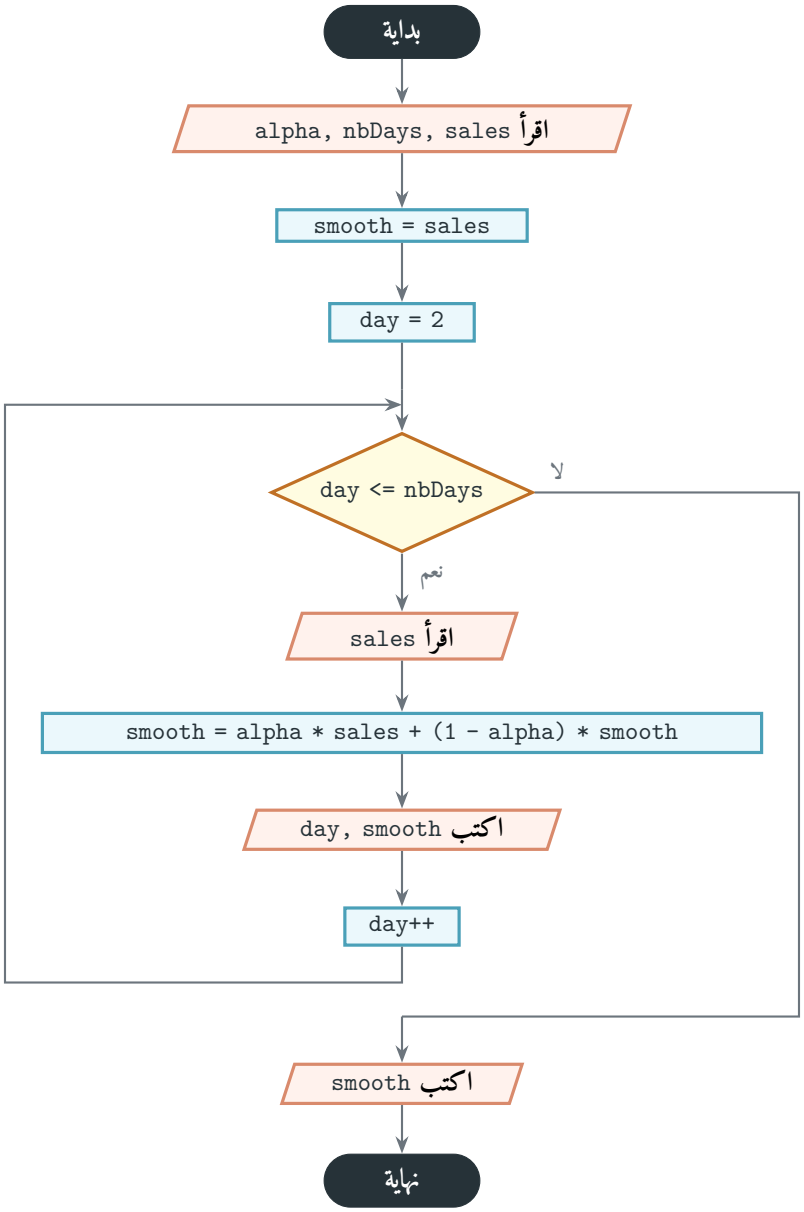
المدخلات

```
0.3 6
200 260 210 300 280 330
```

نتيجة التنفيذ

```
Day 1: 200.00
Day 2: 218.00
Day 3: 215.60
Day 4: 240.92
Day 5: 252.64
Day 6: 275.85
Forecast day 7: 275.85
```

المخطط الإنسيابي (Organigramme)



تمرين 3.32 الإنحدار الخطي على n نقطة

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ nbPoints نقطة (ميزانية الإعلان بالآلاف، المبيعات) ويحسب مستقيم **المربعات الصغرى** sales = slope budget + intercept في مرور واحد بجمع Σbudget وΣsales وΣxy و$\Sigma budget^2$.

خطوات الحل

1) نجمع المجاميع الأربعة في حلقة واحدة.
2) $slope = (nbPoints\ \Sigma xy - \Sigma budget\ \Sigma sales) / (nbPoints\ \Sigma budget^2 - (\Sigma budget)^2)$.
3) $intercept = (\Sigma sales - slope\ \Sigma budget) / nbPoints$.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPoints, index;
    double budget, sales, sx = 0, sy = 0, sxy = 0, sxx = 0, slope, intercept;
    scanf("%d", &nbPoints);
    for (index = 0; index < nbPoints; index++) {
        scanf("%lf %lf", &budget, &sales);
        sx += budget;  sy += sales;        // المجاميع البسيطة
        sxy += budget * sales;             // مجموع الجداءات
        sxx += budget * budget;             // مجموع المربعات
    }
    slope = (nbPoints * sxy - sx * sy) / (nbPoints * sxx - sx * sx);
    intercept = (sy - slope * sx) / nbPoints;
    printf("Sales = %.3f * budget + %.3f\n", slope, intercept);
    printf("Forecast for budget 8: %.1f\n", slope * 8 + intercept);
    return 0;
}
```

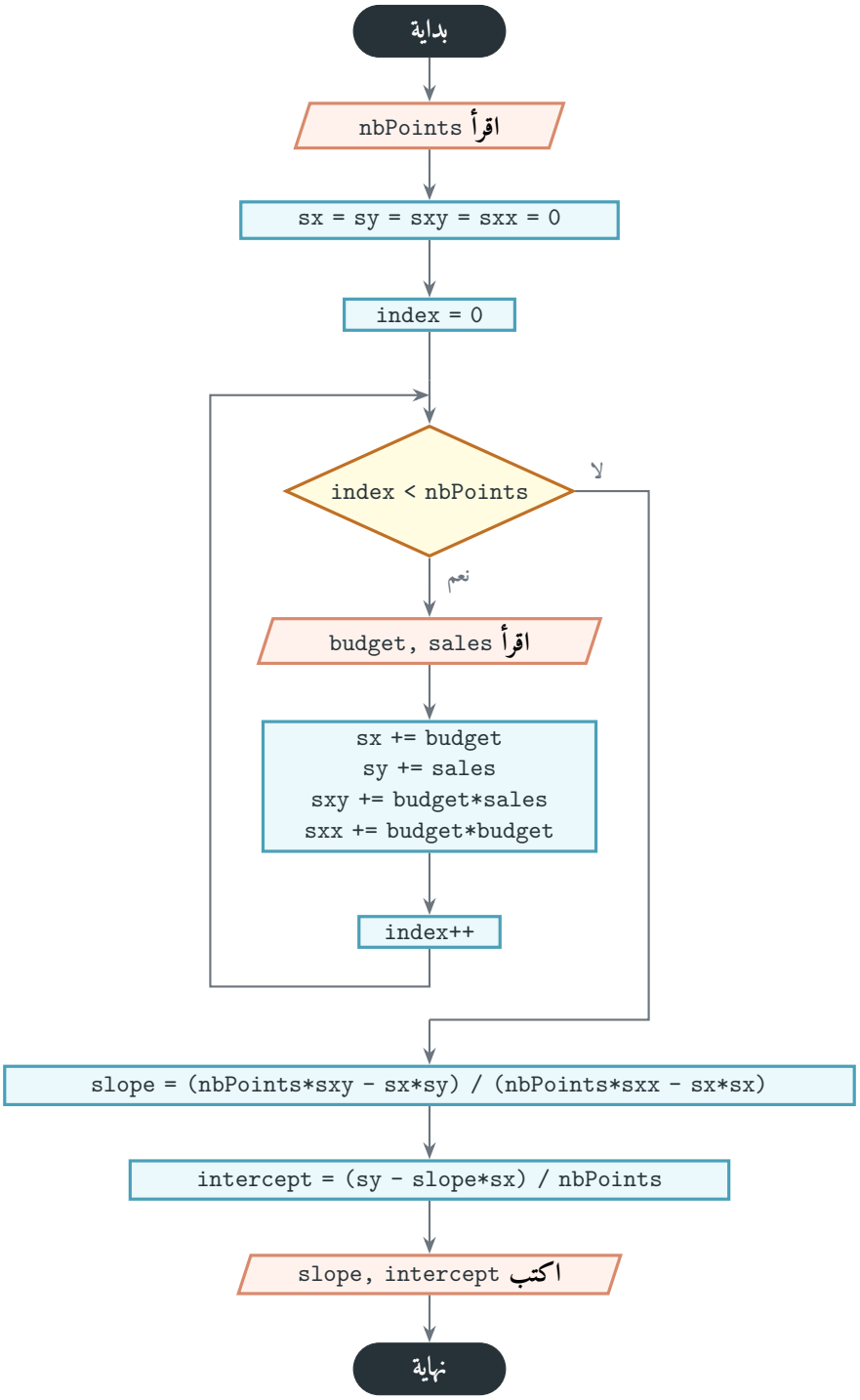
المدخلات

```
6
1 52
2 61
3 74
4 79
5 93
6 101
```

نتيجة التنفيذ

```
Sales = 9.886 * budget + 42.067
Forecast for budget 8: 121.2
```

المخطط الإنسيابي (Organigramme)



تمرين 3.33 معامل الإرتباط

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب **معامل ارتباط بيرسون** بين الوقت المقضي في الموقع (دقائق) ومبلغ الشراء لـ زبون: nbCustomers
، r = (nbCustomers Σxy - Σminutes Σamount) / sqrt((nbCustomers Σminutes² - (Σminutes)²)(nbCustomer
مع حساب الجذر بطريقة نيوتن.

خطوات الحل

1) نجمع $\Sigma minutes$، $\Sigma amount$، Σxy، $\Sigma minutes^2$، $\Sigma amount^2$.
2) البسط num والمقدار تحت الجذر den2.
3) الجذر بحلقة نيوتن (40 تكرارًا كافية).
4) r = num / den.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbCustomers, index;
    double minutes, amount, sx = 0, sy = 0, sxy = 0, sxx = 0, syy = 0;
    double num, den2, den;
    scanf("%d", &nbCustomers);
    for (index = 0; index < nbCustomers; index++) {
        scanf("%lf %lf", &minutes, &amount);
        sx += minutes; sy += amount; sxy += minutes * amount;
        sxx += minutes * minutes; syy += amount * amount;
    }
    num = nbCustomers * sxy - sx * sy;
    den2 = (nbCustomers * sxx - sx * sx) * (nbCustomers * syy - sy * sy);
    den = den2;                          // الجذر بطريقة نيوتن
    for (index = 0; index < 40; index++)
        den = (den + den2 / den) / 2;
    printf("Pearson r = %.4f\n", num / den);
    return 0;
}
```

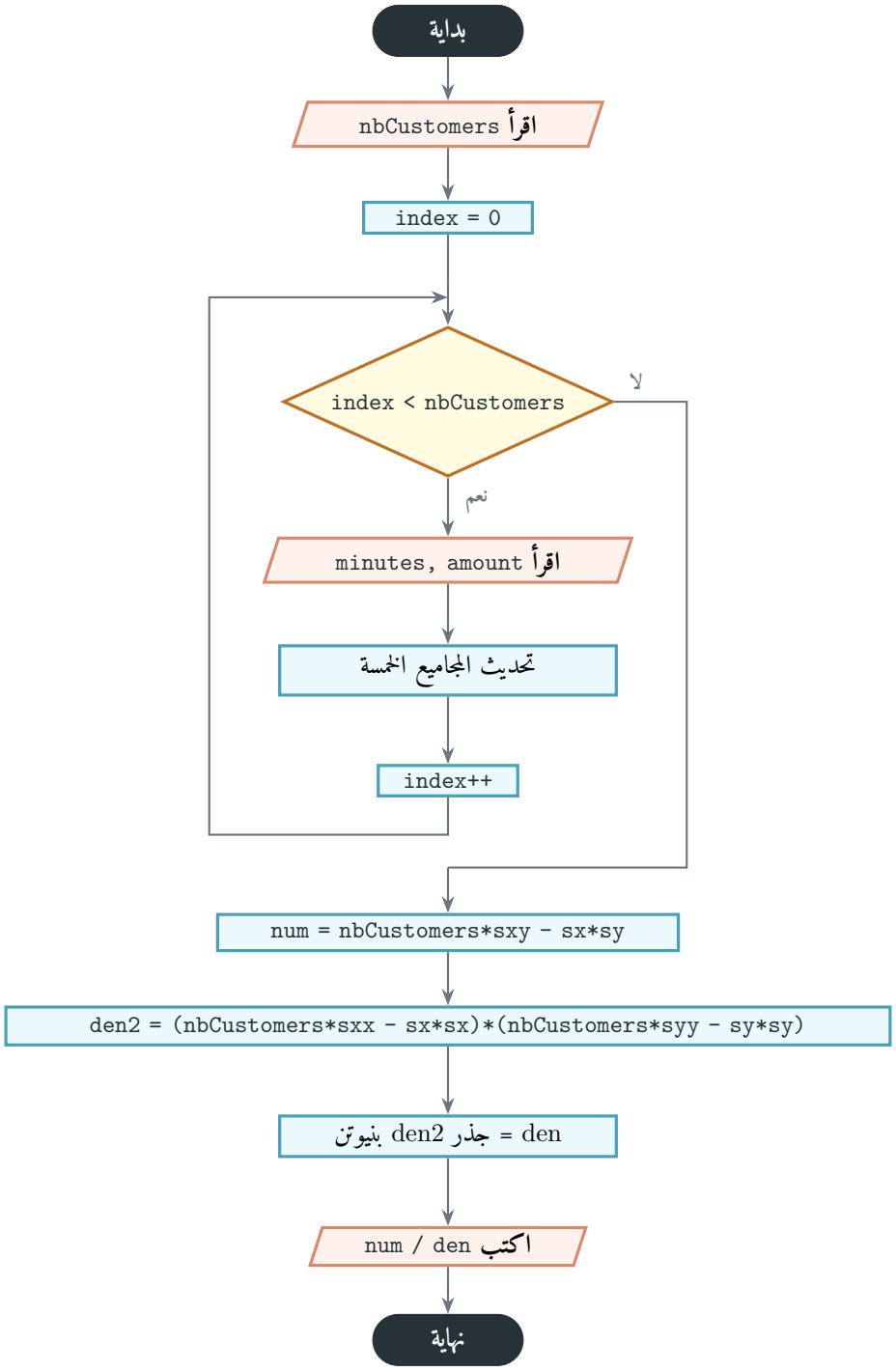
المدخلات

```
6
3 500
8 2100
5 1200
12 3900
2 300
9 2600
```

نتيجة التنفيذ

```
Pearson r = 0.9950
```

المخطط الإنسيابي (Organigramme)



تمرين 3.34 ثاني أغلى منتج

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يقرأ `nbPrices` أسعار **مختلفة** ويجد **ثاني أكبر** سعر في مرور واحد، دون ترتيب ودون جداول.

خطوات الحل

1) نحتفظ بـ `first` (الأكبر) و`second` (الثاني)، مهيأين بقيمة صغيرة جدًا.

2) إذا كان `price > first`: الأكبر القديم يصبح الثاني و`first = price`.

3) وإلا إذا كان `price > second`: `second = price`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPrices, index;
    double price, first = -1, second = -1;     // الأسعار موجبة
    scanf("%d", &nbPrices);
    for (index = 0; index < nbPrices; index++) {
        scanf("%lf", &price);
        if (price > first) {
            second = first;        // الأكبر القديم ينزل إلى المرتبة الثانية
            first = price;
        } else if (price > second) {
            second = price;
        }
    }
    printf("Highest = %.0f, second = %.0f\n", first, second);
    return 0;
}
```

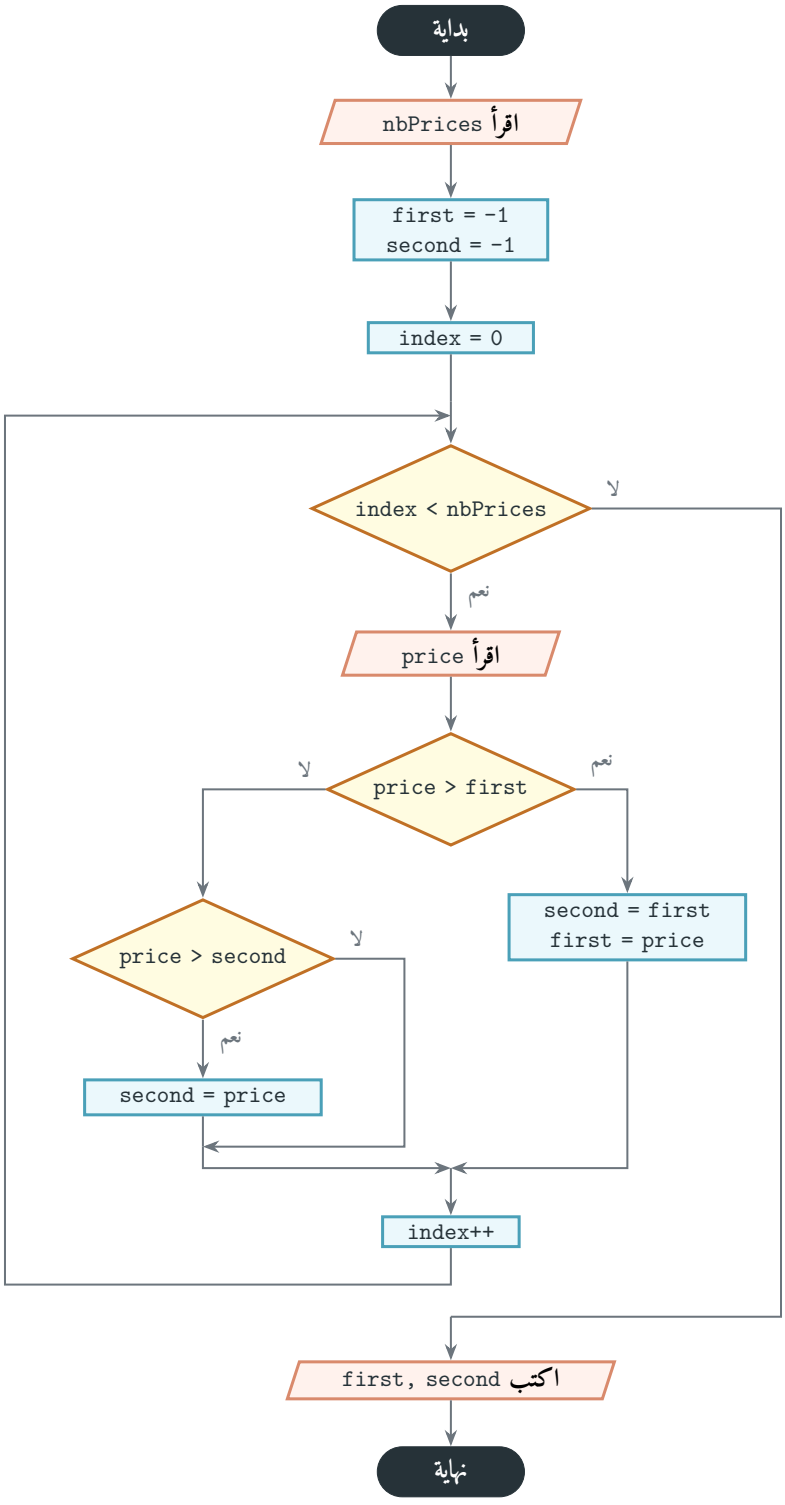
المدخلات

```
7
450 1290 300 2200 870 1990 99
```

نتيجة التنفيذ

```
Highest = 2200, second = 1990
```

المخطط الإنسيابي (Organigramme)



تمرين 3.35 هرم مستويات الوفاء

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يرسم هرمًا من `levels` أسطر يمثل مستويات الوفاء: السطر `row` يحتوي `levels - row` فراغًا ثم `2i - 1` نجمة، ثم يعرض رقم المستوى.

خطوات الحل

1) حلقة خارجية على الأسطر `row` من 1 إلى `levels`.
2) حلقة داخلية أولى للفراغات، وثانية للنجوم.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int levels, row, col;
    scanf("%d", &levels);
    for (row = 1; row <= levels; row++) {
        for (col = 1; col <= levels - row; col++)      // فراغات للتوسيط
            printf(" ");
        for (col = 1; col <= 2 * row - 1; col++)  // عدد فردي من النجوم
            printf("*");
        printf("  level %d\n", levels - row + 1);
    }
    return 0;
}
```

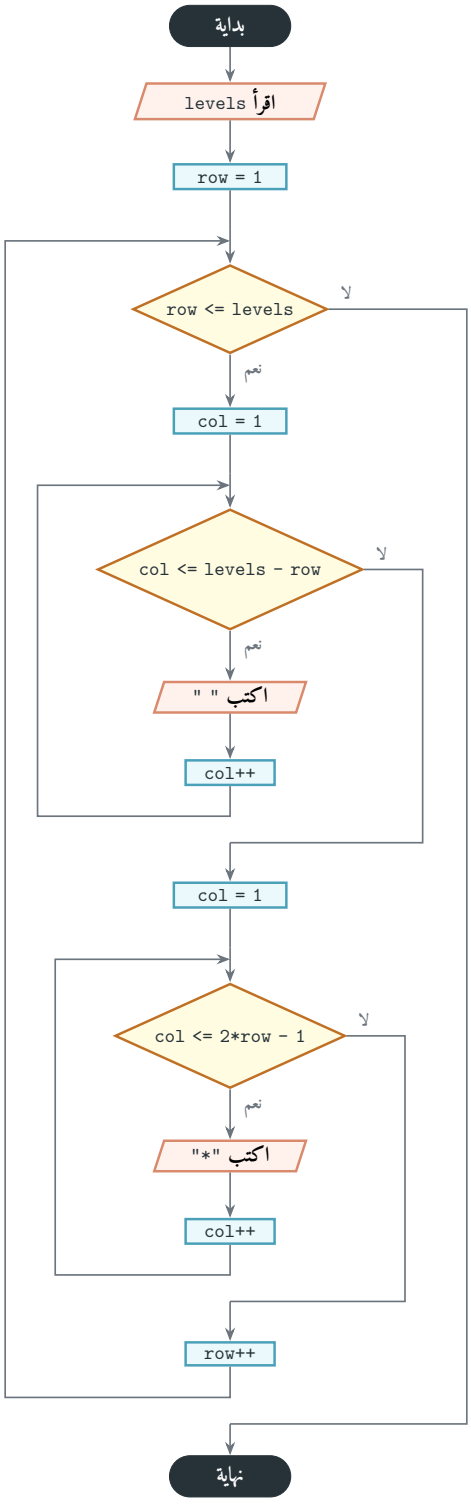
المدخلات

```
5
```

نتيجة التنفيذ

```
    *  level 5
   ***  level 4
  *****  level 3
 *******  level 2
*********  level 1
```

المخطط الإنسيابي (Organigramme)



تمرين 3.36 سعر التوازن بالتنصيف

علم البيانات ★★ متوسط

الطلب على منتج `D(p) = 5000 / p` (غير خطي) والعرض `S(p) = 2p - 20`. اكتب برنامجًا يجد سعر التوازن (حيث `D(p) - S(p) = 0`) بطريقة **التنصيف** في المجال `[lo, hi]` بدقة 0.001.

خطوات الحل

1) `gap(p) = 5000 / p - (2p - 20)` موجبة عند `lo` وسالبة عند `hi`.
2) ما دام `hi - lo > 0.001`: `mid = (lo + hi) / 2`.
3) إذا كان `gap(mid) > 0` فالجذر على اليمين: `lo = mid`، وإلا `hi = mid`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double lo, hi, mid, gap;
    int it = 0;
    scanf("%lf %lf", &lo, &hi);
    while (hi - lo > 0.001) {
        mid = (lo + hi) / 2;
        gap = 5000 / mid - (2 * mid - 20);  // الطلب ناقص العرض
        if (gap > 0)
            lo = mid;                       // الطلب أكبر: السعر يجب أن يرتفع
        else
            hi = mid;
        it++;                               // كل دورة تقسم المجال على 2
    }
    mid = (lo + hi) / 2;
    printf("Equilibrium price = %.3f DA (%d iterations)\n", mid, it);
    printf("Quantity = %.2f\n", 5000 / mid);
    return 0;
}
```

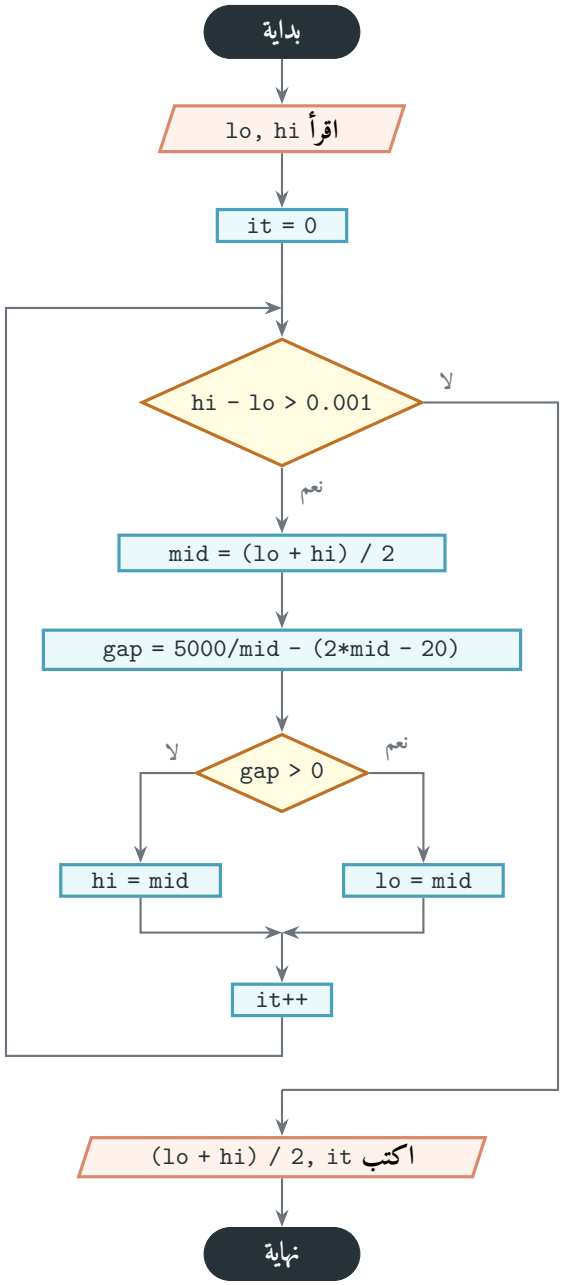
المدخلات

```
10 100
```

نتيجة التنفيذ

```
Equilibrium price = 55.250 DA (17 iterations)
Quantity = 90.50
```

المخطط الإنسيابي (Organigramme)



تمرين 3.37 — الفائدة المستمرة بسلسلة تايلور

علم البيانات — متوسط

مع الفائدة المستمرة يصبح المبلغ `capital × e^(rt)`. اكتب برنامجًا يحسب `e^exponent` بجمع حدود سلسلة تايلور `1 + exponent + exponent²/2! + exponent³/3! + ...` حتى يصبح الحد أصغر من `1e-12`، دون حساب العامليات منفصلة.

خطوات الحل

1) `exponent = rate * years`، `term = 1`، `sum = 1`، `term_k = 0`.
2) الحد التالي من السابق: `term = term * exponent / term_k`.
3) نتوقف عندما يصبح `term < 1e-12`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double capital, rate, years, exponent, term = 1, sum = 1;
    int term_k = 0;
    scanf("%lf %lf %lf", &capital, &rate, &years);
    exponent = rate * years;
    do {
        term_k++;
        term = term * exponent / term_k;       // من الحد السابق! term_k / term_k^exponent
        sum += term;
    } while (term > 1e-12);
    printf("e^%.2f = %.10f (%d terms)\n", exponent, sum, term_k);
    printf("Continuous: %.2f DA\n", capital * sum);
    return 0;
}
```

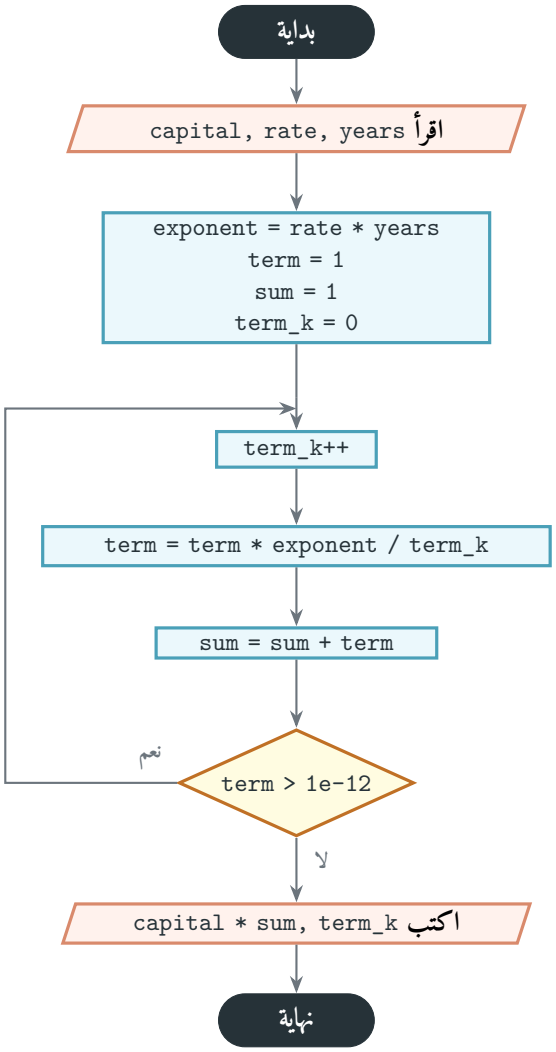
المدخلات

```
50000 0.05 10
```

نتيجة التنفيذ

```
e^0.50 = 1.6487212707 (12 terms)
Continuous: 82436.06 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.38 القفزات المفاجئة في الزيارات

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ عدد الزيارات لـ nbHours ساعة ويكشف **القفزات**: كل ساعة تختلف زياراتها عن الساعة السابقة بأكثر من 50%. نعرض رقم الساعة ونسبة التغير.

خطوات الحل

1) نحتفظ بالقيمة السابقة.
2) `change = (visits - prev) / prev * 100` ثم قيمتها المطلقة.
3) إذا كانت `> 50` نعرض تنبيهًا ونعد القفزات.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbHours, hour, jumps = 0;
    double prev, visits, change, ac;
```

```c
    scanf("%d %lf", &nbHours, &prev);
    for (hour = 2; hour <= nbHours; hour++) {
        scanf("%lf", &visits);
        change = (visits - prev) / prev * 100;  // التغير النسبي
        ac = (change < 0) ? -change : change;
        if (ac > 50) {
            printf("Hour %d: jump of %+.1f %%\n", hour, change);
            jumps++;
        }
        prev = visits;
    }
    printf("Total jumps: %d\n", jumps);
    return 0;
}
```

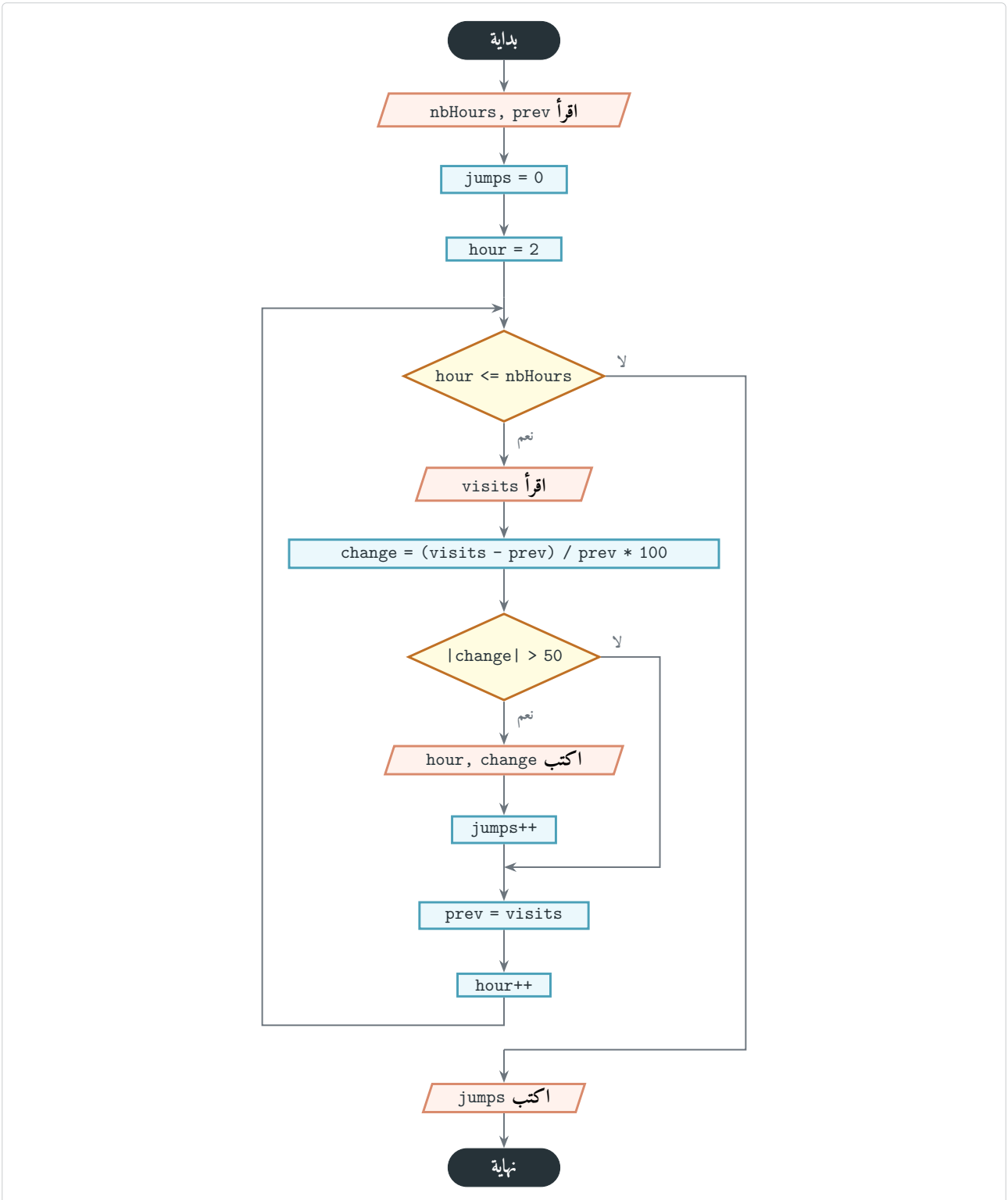
المدخلات

```
8
400 420 900 880 300 310 650 640
```

نتيجة التنفيذ

```
Hour 3: jump of +114.3 %
Hour 5: jump of -65.9 %
Hour 7: jump of +109.7 %
Total jumps: 3
```

المخطط الإنسيابي (Organigramme)



تمرين 3.39 تخمين السعر بالبحث الثنائي

تجارة إلكترونية ★★ متوسط

في لعبة ترويجية يخفي المتجر سعراً بين 1 و100000. اكتب برنامجًا يقرأ السعر المخفي ثم يجعل الحاسوب يخمنه بالبحث الثنائي: في كل محاولة يقترح المنتصف ويُقال له "أكبر" أو "أصغر". اعرض المحاولات.

خطوات الحل

1) `lo = 1`، `hi = 100000`.
2) `guess = (lo + hi) / 2`؛ إذا كان أصغر من السعر: `lo = guess + 1`، إذا كان أكبر: `hi = guess - 1`.
3) نتوقف عند الإصابة؛ عدد المحاولات لا يتجاوز 17.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int secret, lo = 1, hi = 100000, guess, tries = 0;
    scanf("%d", &secret);
    do {
        guess = (lo + hi) / 2;     // نجرب منتصف المجال
        tries++;
        printf("Try %2d: %6d -> ", tries, guess);
        if (guess < secret) {
            printf("higher\n");
            lo = guess + 1;        // نتخلص من النصف الأيسر
        } else if (guess > secret) {
            printf("lower\n");
            hi = guess - 1;        // نتخلص من النصف الأيمن
        } else {
            printf("found!\n");
        }
    } while (guess != secret);
    printf("Found in %d tries\n", tries);
    return 0;
}
```

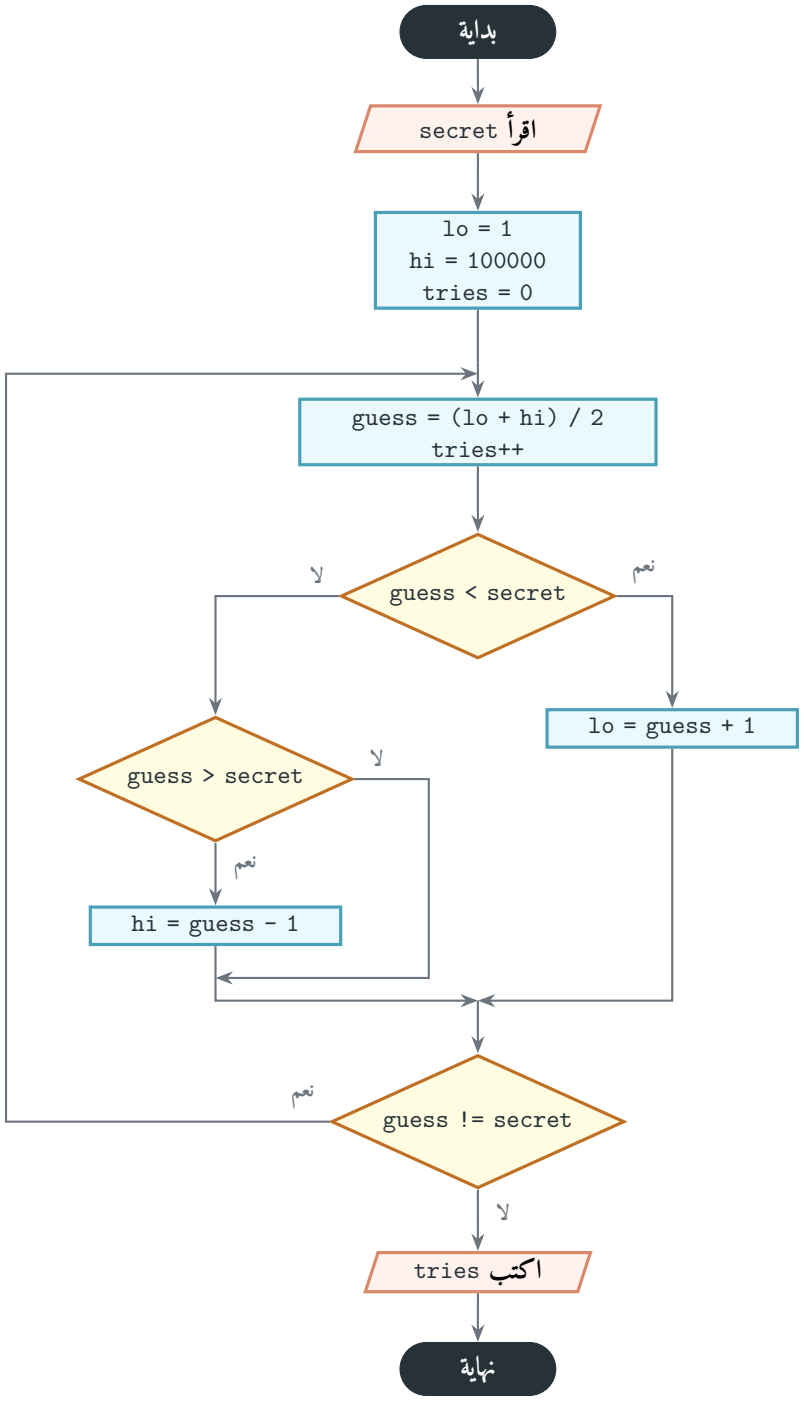
المدخلات

```
73421
```

نتيجة التنفيذ

```
Try  1:  50000 -> higher
Try  2:  75000 -> lower
Try  3:  62500 -> higher
Try  4:  68750 -> higher
Try  5:  71875 -> higher
Try  6:  73437 -> lower
Try  7:  72656 -> higher
Try  8:  73046 -> higher
Try  9:  73241 -> higher
Try 10:  73339 -> higher
Try 11:  73388 -> higher
Try 12:  73412 -> higher
Try 13:  73424 -> lower
Try 14:  73418 -> higher
Try 15:  73421 -> found!
Found in 15 tries
```

المخطط الإنسيابي (Organigramme)



تمرين 3.40 يوم بلوغ هدف رقم الأعمال

علم البيانات ★★ متوسط

هدف الشهر رقم أعمال `target`. اكتب برنامجًا يقرأ مبيعات الأيام (30 يومًا كحد أقصى) ويتوقف فور بلوغ الهدف باستعمال `break`، ويعرض اليوم؛ وإذا لم يُبلغ الهدف يعرض الناقص.

خطوات الحل

1) نجمع المبيعات في `cum`.

2) إذا كان `cum >= target` نحفظ اليوم ونخرج بـ `break`.

3) بعد الحلقة نميز الحالتين بالمتغير `reached`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double target, sales, cum = 0;
    int maxDays, day, reached = 0;
    scanf("%lf %d", &target, &maxDays);
    for (day = 1; day <= maxDays; day++) {
        if (scanf("%lf", &sales) != 1)  // نهاية البيانات المتوفرة
            break;
        cum += sales;
        if (cum >= target) {
            reached = day;
            break;                      // نخرج من الحلقة فورًا
        }
    }
    if (reached)
        printf("Target reached on day %d (%.0f DA)\n", reached, cum);
    else
        printf("Target missed by %.0f DA\n", target - cum);
    return 0;
}
```

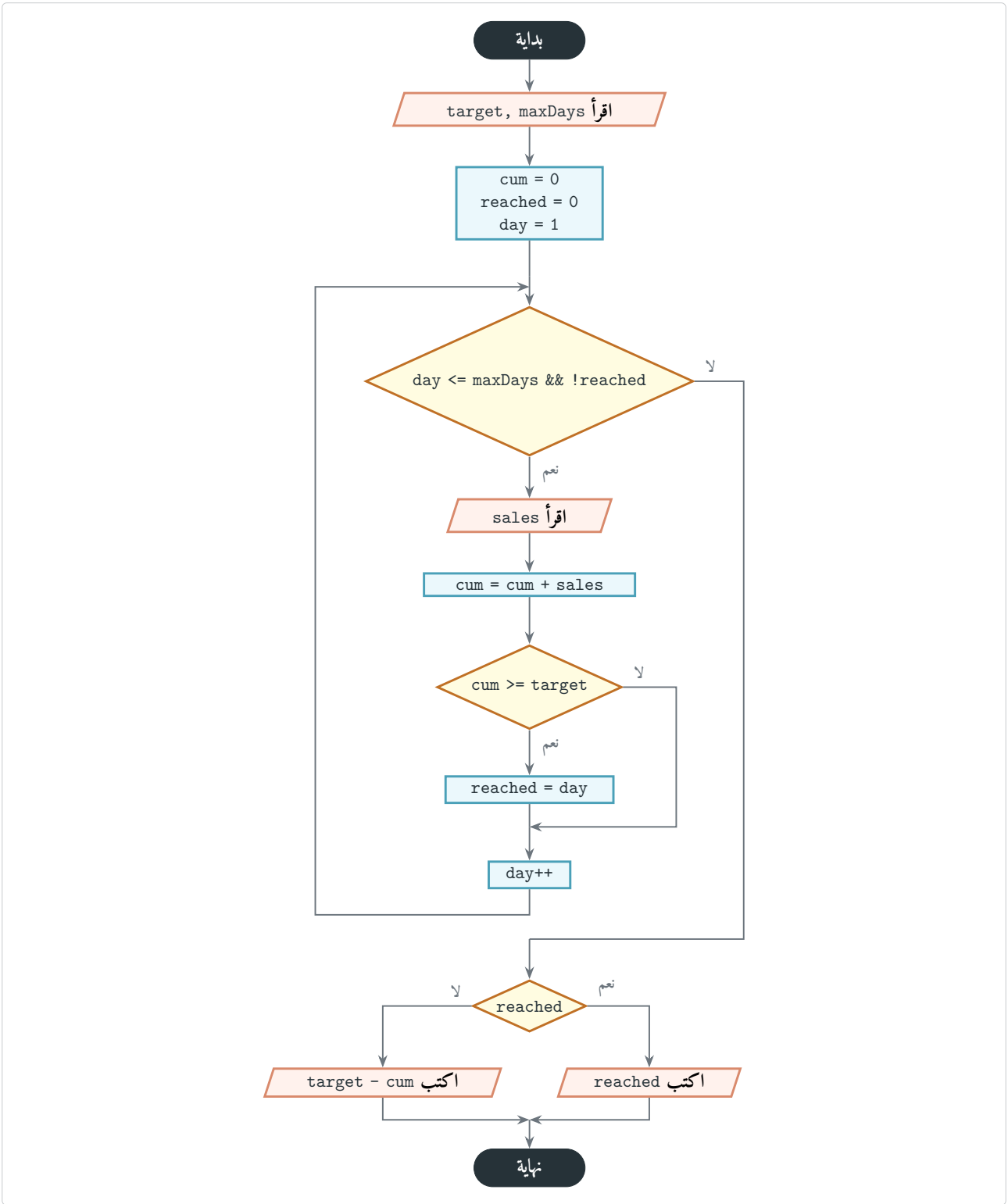
المدخلات

```
100000 30
8500 12000 9900 15000 7000 11000 13500 9000 14000 6000 3000 2000
```

نتيجة التنفيذ

```
Target reached on day 10 (105900 DA)
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 3.41 الأرقام الأولية حتى N

تجارة إلكترونية صعب

اكتب برنامجًا يعرض كل الأعداد الأولية الأصغر من أو تساوي `limit` (لاستعمالها كأرقام خوادم)، ويعد عددها ومجموعها، باستعمال حلقتين متداخلتين واختبار القواسم الفردية فقط.

خطوات الحل

1) العدد 2 أولي، ثم نختبر الأعداد الفردية فقط `number = 3, 5, 7, ...`.
2) لكل `number`: حلقة داخلية على القواسم الفردية `divisor` ما دام `divisor * divisor <= number`.
3) نتوقف عند أول قاسم (`isp = 0`).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int limit, number, divisor, isp, count = 0, sum = 0;
    scanf("%d", &limit);
    if (limit >= 2) {                    // 2 هو العدد الأولي الزوجي الوحيد
        printf("2 ");
        count = 1; sum = 2;
    }
    for (number = 3; number <= limit; number += 2) {     // الأعداد الفردية فقط
        isp = 1;
        for (divisor = 3; isp && divisor * divisor <= number; divisor += 2)
            if (number % divisor == 0)
                isp = 0;                 // له قاسم: ليس أوليًا
        if (isp) {
            printf("%d ", number);
            count++;
            sum += number;
        }
    }
    printf("\n%d primes, sum = %d\n", count, sum);
    return 0;
}
```

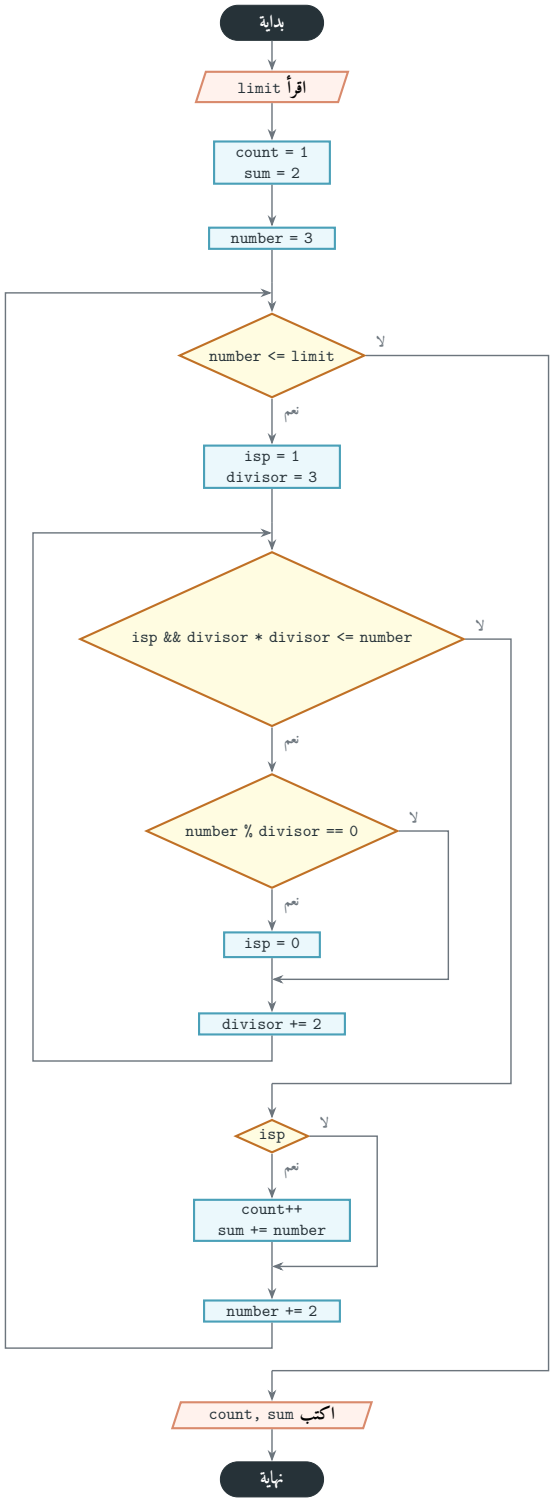
المدخلات

```
100
```

نتيجة التنفيذ

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
25 primes, sum = 1060
```

المخطط الإنسيابي (Organigramme)



تمرين 3.42 التجميع بـ K-Means في بعد واحد

علم البيانات ★★★ صعب

نريد تقسيم 20 زبونًا إلى **مجموعتين** حسب إنفاقهم بخوارزمية **K-Means**. الإنفاق مولَّد بصيغة: `spend(customer) = (customer * 37) % 50 + (customer % 2) * 60` لـ `customer` من 0 إلى 19 (لا نخزنها، نعيد حسابها). نبدأ بمركزين `c1` و`c2` مقروءين ونكرر: نسند كل نقطة لأقرب مركز ثم نحسب المراكز الجديدة كمتوسطات، حتى الاستقرار.

خطوات الحل

1) في كل تكرار: نمر على النقاط، إذا كانت `|spend - c1| <= |spend - c2|` تنتمي للمجموعة 1.
2) نجمع مجموع وعدد كل مجموعة، ثم المركز الجديد = المتوسط.
3) نتوقف عندما لا يتغير المركزان (أو بعد 20 تكرارًا).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double c1, c2, m1, m2, s1, s2, spend, d1, d2;
    int customer, n1, n2, it = 0, changed = 1;
    scanf("%lf %lf", &c1, &c2);           // المركزان الابتدائيان
    while (changed && it < 20) {
        s1 = s2 = 0; n1 = n2 = 0;
        for (customer = 0; customer < 20; customer++) {
            spend = (customer * 37) % 50 + (customer % 2) * 60;   // إنفاق الزبون customer
            d1 = spend - c1; if (d1 < 0) d1 = -d1;  // المسافة إلى c1
            d2 = spend - c2; if (d2 < 0) d2 = -d2;  // المسافة إلى c2
            if (d1 <= d2) { s1 += spend; n1++; }    // أقرب إلى c1
            else          { s2 += spend; n2++; }
        }
        m1 = (n1 > 0) ? s1 / n1 : c1;   // المركز الجديد = المتوسط
        m2 = (n2 > 0) ? s2 / n2 : c2;
        changed = (m1 != c1 || m2 != c2);
        c1 = m1; c2 = m2;
        it++;
        printf("Iter %d: c1 = %.2f (%d), c2 = %.2f (%d)\n", it, c1, n1, c2, n2);
    }
    printf("Converged after %d iterations\n", it);
    return 0;
}
```

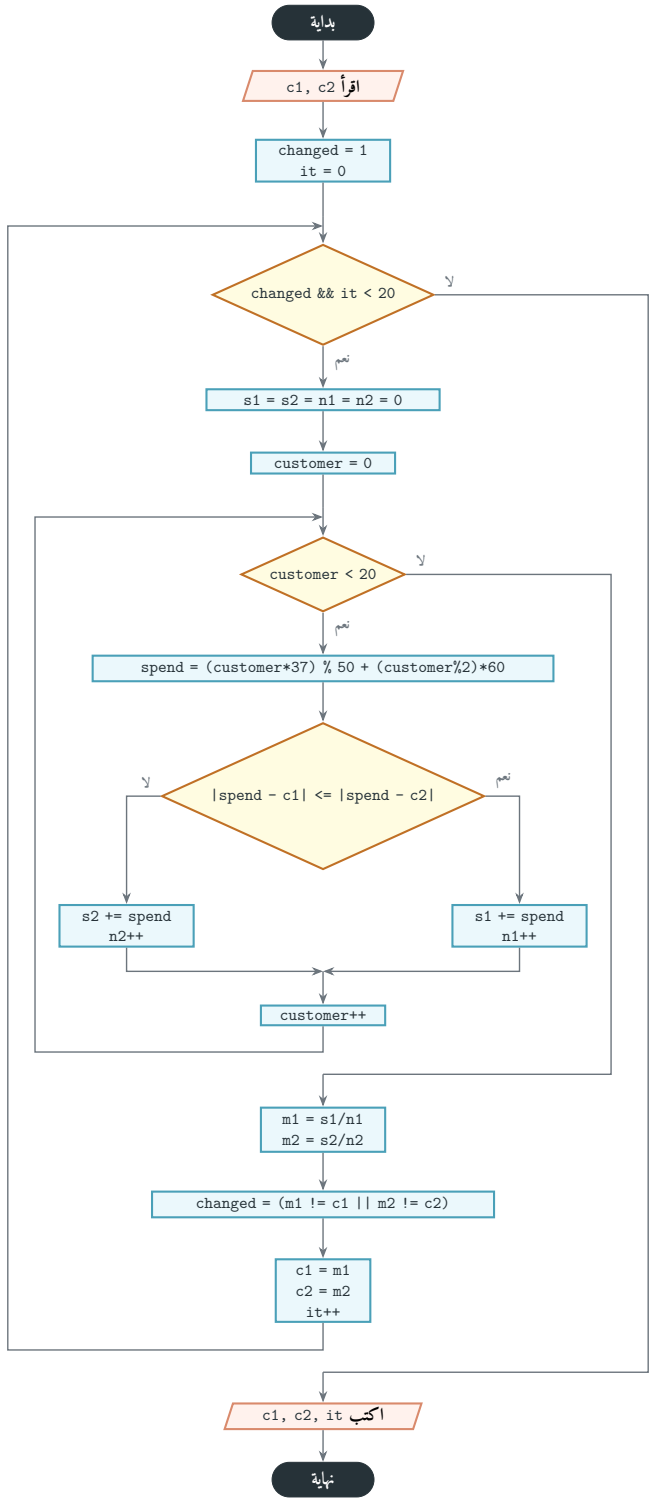
المدخلات

```
10 20
```

نتيجة التنفيذ

```
Iter 1: c1 = 0.00 (1), c2 = 56.84 (19)
Iter 2: c1 = 16.67 (6), c2 = 70.00 (14)
Iter 3: c1 = 20.29 (7), c2 = 72.15 (13)
Iter 4: c1 = 25.78 (9), c2 = 77.09 (11)
Iter 5: c1 = 28.00 (10), c2 = 80.00 (10)
Iter 6: c1 = 28.00 (10), c2 = 80.00 (10)
Converged after 6 iterations
```

المخطط الإنسيابي (Organigramme)



تمرين 3.43 محاكاة مونت كارلو بمولد عشوائي يدوي — علم البيانات — صعب ★★★

دون الدالة `rand`، نبني **مولد أعداد شبه عشوائية خطيًا** (LCG): `seed = (seed * 1103515245 + 12345) % 2147483648`، و`uniform = seed / 2147483648.0` عدد بين 0 و1. كل زبون يشتري 1 إلى 5 قطع (`(1 + (int)(uniform * 5))`) وكل قطعة سعرها بين 500 و3000. اكتب برنامجًا يحاكي `nbBaskets` سلة ويقدر **احتمال** أن تتجاوز السلة 6000 دج ومتوسط

السلة.

خطوات الحل

1) نستعمل `unsigned long long` للمولد لتجنب تجاوز السعة.
2) لكل سلة: نولد عدد القطع، ثم لكل قطعة نولد سعراً `500 + uniform * 2500`.
3) نعد السلات التي تتجاوز 6000 ونجمع المبالغ.
4) الاحتمال = `count / nbBaskets`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    unsigned long long seed;
    int nbBaskets, basketNo, item, nbItems, big = 0;
    double uniform, basket, total = 0;
    scanf("%llu %d", &seed, &nbBaskets);
    for (basketNo = 0; basketNo < nbBaskets; basketNo++) {
        seed = (seed * 1103515245ULL + 12345ULL) % 2147483648ULL;
        uniform = seed / 2147483648.0;          // عدد شبه عشوائي في [0, 1)
        nbItems = 1 + (int)(uniform * 5);       // عدد القطع: 1 إلى 5
        basket = 0;
        for (item = 0; item < nbItems; item++) {
            seed = (seed * 1103515245ULL + 12345ULL) % 2147483648ULL;
            uniform = seed / 2147483648.0;
            basket += 500 + uniform * 2500;     // سعر بين 500 و 3000
        }
        total += basket;
        if (basket > 6000)
            big++;
    }
    printf("P(basket > 6000) ~ %.4f\n", (double)big / nbBaskets);
    printf("Mean basket ~ %.2f DA\n", total / nbBaskets);
    return 0;
}
```

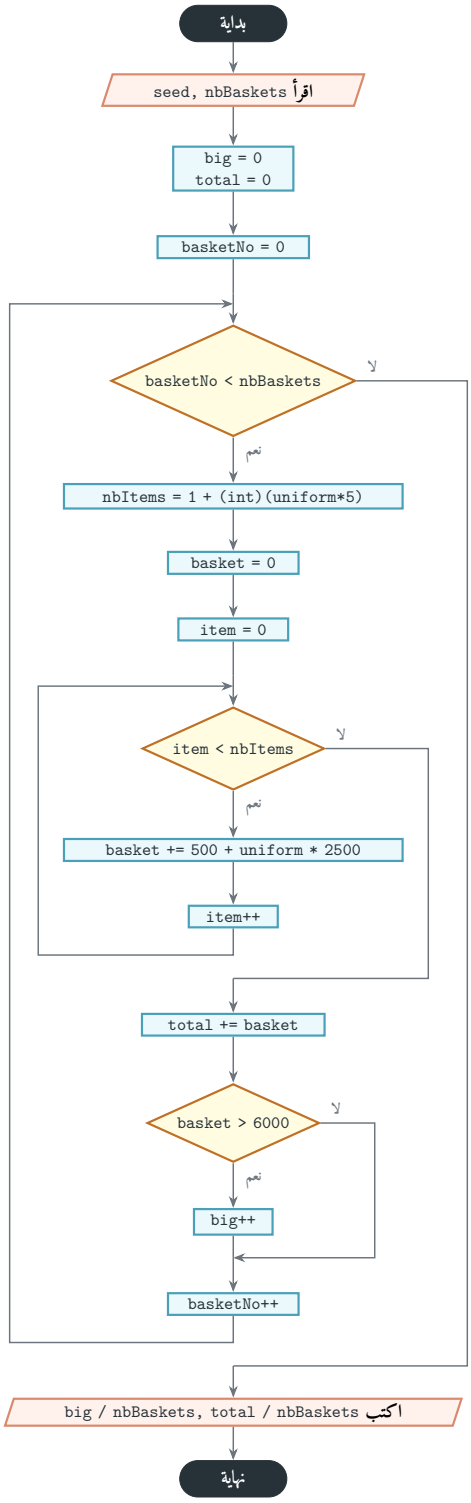
المدخلات

```
2026 20000
```

نتيجة التنفيذ

```
P(basket > 6000) ~ 0.3942
Mean basket ~ 5239.69 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.44 الإنحدار بالنزول التدريجي

علم البيانات ★★★ صعب

بدل الصيغة المغلقة، ندرب نموذج `target = weight feature + bias` بخوارزمية النزول التدريجي (Gradient Descent). البيانات: `feature(index) = index` و`target(index) = 3i + 7 + ((index * 7) % 5 - 2)` لـ `index` من 0 إلى 9. في كل حقبة نحسب المشتقتين `dw = (2/nbPoints) Σ (ŷ - target) feature` و`db = (2/nbPoints) Σ (ŷ - target)` ثم

`weight -= lr * dw` و`bias -= lr * db`. اكتب برنامجًا يدرب النموذج ويعرض الخطأ.

خطوات الحل

1) نهيئ `weight = 0` و`bias = 0`.
2) لكل حقبة: نمر على النقاط ونجمع التدرجين والخطأ التربيعي.
3) نحدّث المعاملين بمعدل التعلم `lr`.
4) نعرض التقدم كل 500 حقبة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double lr, weight = 0, bias = 0, dw, db, mse, feature, target, err;
    int epochs, epoch, index, nbPoints = 10;
    scanf("%lf %d", &lr, &epochs);
    for (epoch = 1; epoch <= epochs; epoch++) {
        dw = db = mse = 0;
        for (index = 0; index < nbPoints; index++) {
            feature = index;
            target = 3 * index + 7 + ((index * 7) % 5 - 2);  // بيانات مع ضجيج
            err = weight * feature + bias - target;          // خطأ التنبؤ
            dw += err * feature;                             // مساهمة النقطة في التدرج
            db += err;
            mse += err * err;
        }
        weight -= lr * 2 * dw / nbPoints;     // خطوة في عكس اتجاه التدرج
        bias -= lr * 2 * db / nbPoints;
        if (epoch % 500 == 0)
            printf("Epoch %4d: w = %.4f, b = %.4f, MSE = %.4f\n", epoch, weight, bias, mse / nbPoints);
    }
    return 0;
}
```

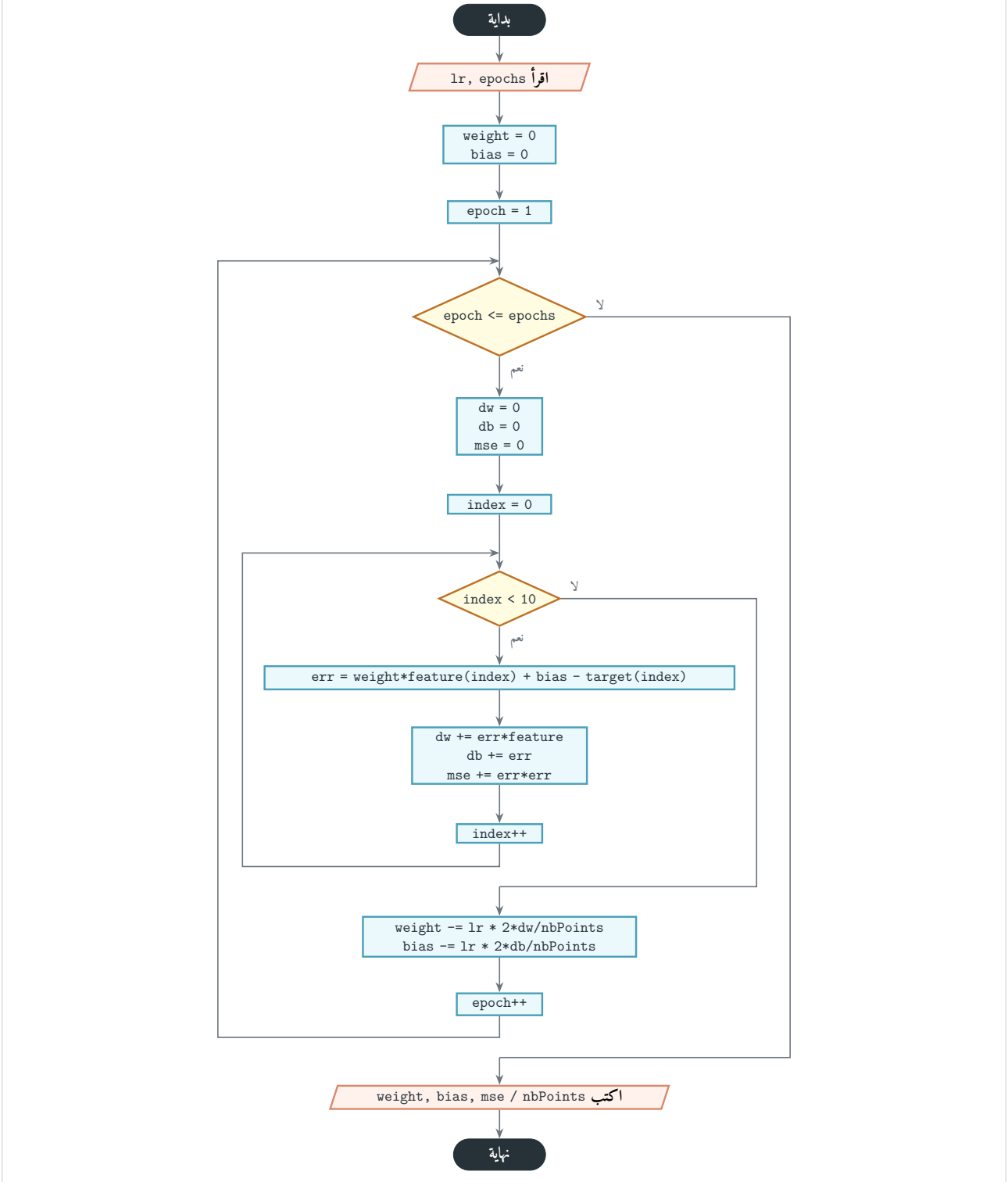
المدخلات

```
0.01 3000
```

نتيجة التنفيذ

```
Epoch  500: w = 3.1758, b = 6.1123, MSE = 1.9131
Epoch 1000: w = 3.1244, b = 6.4344, MSE = 1.8789
Epoch 1500: w = 3.1214, b = 6.4534, MSE = 1.8788
Epoch 2000: w = 3.1212, b = 6.4545, MSE = 1.8788
Epoch 2500: w = 3.1212, b = 6.4545, MSE = 1.8788
Epoch 3000: w = 3.1212, b = 6.4545, MSE = 1.8788
```

المخطط الإنسيابي (Organigramme)



تمرين 3.45 التحقق من رقم البطاقة بخوارزمية Luhn

تجارة إلكترونية ★★★ صعب

قبل إرسال الدفع للبنك نتحقق من رقم البطاقة بخوارزمية **Luhn**: من اليمين، نضاعف رقمًا من كل اثنين (الثاني، الرابع، ...)، وإذا تجاوز الناتج 9 نطرح 9، ثم نجمع كل الأرقام؛ الرقم صالح إذا كان المجموع يقبل القسمة على 10. اكتب البرنامج لرقم من 16 خانة.

خطوات الحل

1) نقرأ الرقم في `unsigned long long`.
2) حلقة: `digit = number % 10` ثم `number /= 10`، مع موضع `pos` يبدأ من 1 من اليمين.
3) إذا كان `pos` زوجيًا: `digit = 2d` ثم `digit -= 9` إذا تجاوز 9.
4) نجمع، ونتحقق من `sum % 10 == 0`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    unsigned long long number;
    int digit, sum = 0, pos = 1;
    scanf("%llu", &number);
    while (number > 0) {
        digit = number % 10;                // نقرأ الأرقام من اليمين
        number = number / 10;
        if (pos % 2 == 0) {                 // رقم من كل اثنين
            digit = 2 * digit;
            if (digit > 9)
                digit = digit - 9;          // = مجموع رقمي الناتج
        }
        sum += digit;
        pos++;
    }
    printf("Luhn sum = %d\n", sum);
    if (sum % 10 == 0)
        printf("Card number is valid\n");
    else
        printf("Card number is INVALID\n");
    return 0;
}
```

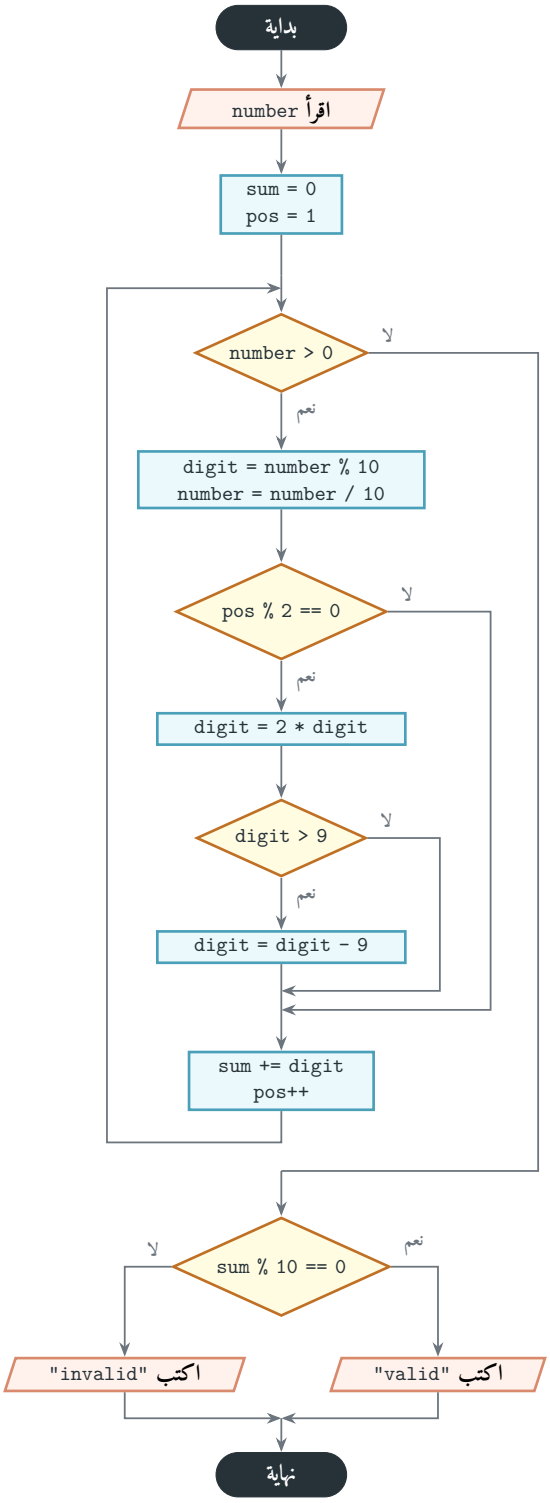
المدخلات

```
4539578763621486
```

نتيجة التنفيذ

```
Luhn sum = 80
Card number is valid
```

المخطط الإنسيابي (Organigramme)



تمرين 3.46 مثلث باسكال لعدد الحزم

تجارة إلكترونية ★★★ صعب

يعطي **مثلث باسكال** في السطر `row` والعمود `col` عدد الحزم `C(row,col)`. اكتب برنامجًا يعرض الأسطر من 0 إلى `maxRow`، ويحسب كل عنصر من سابقه في نفس السطر: `C(row,col) = C(row,col-1) * (row - col + 1) / col`، ثم يعرض مجموع كل سطر (يساوي `2^row`).

خطوات الحل

1) حلقة خارجية على السطر `row`.
2) في كل سطر: `coef = 1`، وحلقة داخلية تحدّث `coef` وتطبعه.
3) نجمع عناصر السطر للتحقق.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int maxRow, row, col;
    long coef, rowSum;
    scanf("%d", &maxRow);
    for (row = 0; row <= maxRow; row++) {
        coef = 1; rowSum = 1;
        printf("n=%2d: 1", row);
        for (col = 1; col <= row; col++) {
            coef = coef * (row - col + 1) / col;  // العنصر من سابقه، القسمة دقيقة
            rowSum += coef;
            printf(" %ld", coef);
        }
        printf("  (sum %ld)\n", rowSum);   // 2 = أس row
    }
    return 0;
}
```

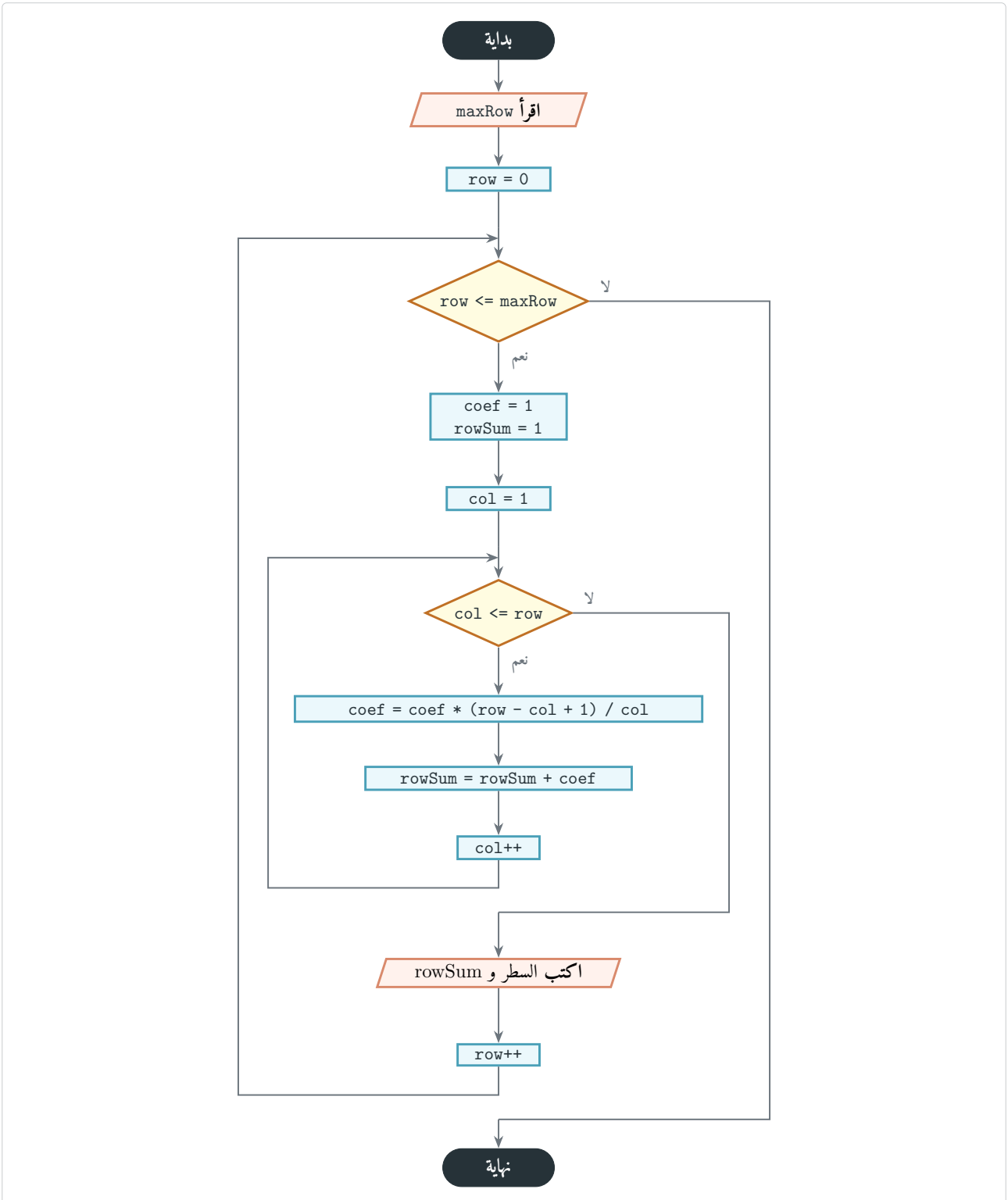
المدخلات

```
10
```

نتيجة التنفيذ

```
n= 0: 1  (sum 1)
n= 1: 1 1  (sum 2)
n= 2: 1 2 1  (sum 4)
n= 3: 1 3 3 1  (sum 8)
n= 4: 1 4 6 4 1  (sum 16)
n= 5: 1 5 10 10 5 1  (sum 32)
n= 6: 1 6 15 20 15 6 1  (sum 64)
n= 7: 1 7 21 35 35 21 7 1  (sum 128)
n= 8: 1 8 28 56 70 56 28 8 1  (sum 256)
n= 9: 1 9 36 84 126 126 84 36 9 1  (sum 512)
n=10: 1 10 45 120 210 252 210 120 45 10 1  (sum 1024)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.47 التحويل إلى النظام الست عشري

تجارة إلكترونية ★★★ صعب

تُعرض أرقام الشحنات بالنظام **الست عشري** (0-9 ثم A-F). اكتب برنامجًا يحول عددًا موجبًا إلى الست عشري دون جداول ودون `printf("%X")`: نحدد أولًا أكبر قوة للعدد 16 لا تتجاوز العدد، ثم نستخرج الأرقام من اليسار إلى اليمين.

خطوات الحل

1) `power = 1`، وما دام `power * 16 <= number`: `power *= 16`.
2) ما دام `power > 0`: الرقم `digit = number / power`، ثم `number %= power` و`power /= 16`.
3) نطبع `'0' + digit` إذا كان `digit < 10`، وإلا `'A' + digit - 10`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    unsigned long long number, power = 1;
    int digit;
    scanf("%llu", &number);
    printf("%llu = 0x", number);
    while (power * 16 <= number)           // أكبر قوة لـ 16 لا تتجاوز number
        power *= 16;
    while (power > 0) {
        digit = number / power;                // الرقم الحالي من اليسار
        number = number % power;
        if (digit < 10)
            putchar('0' + digit);     // 0..9
        else
            putchar('A' + digit - 10);// A..F
        power /= 16;
    }
    printf("\n");
    return 0;
}
```

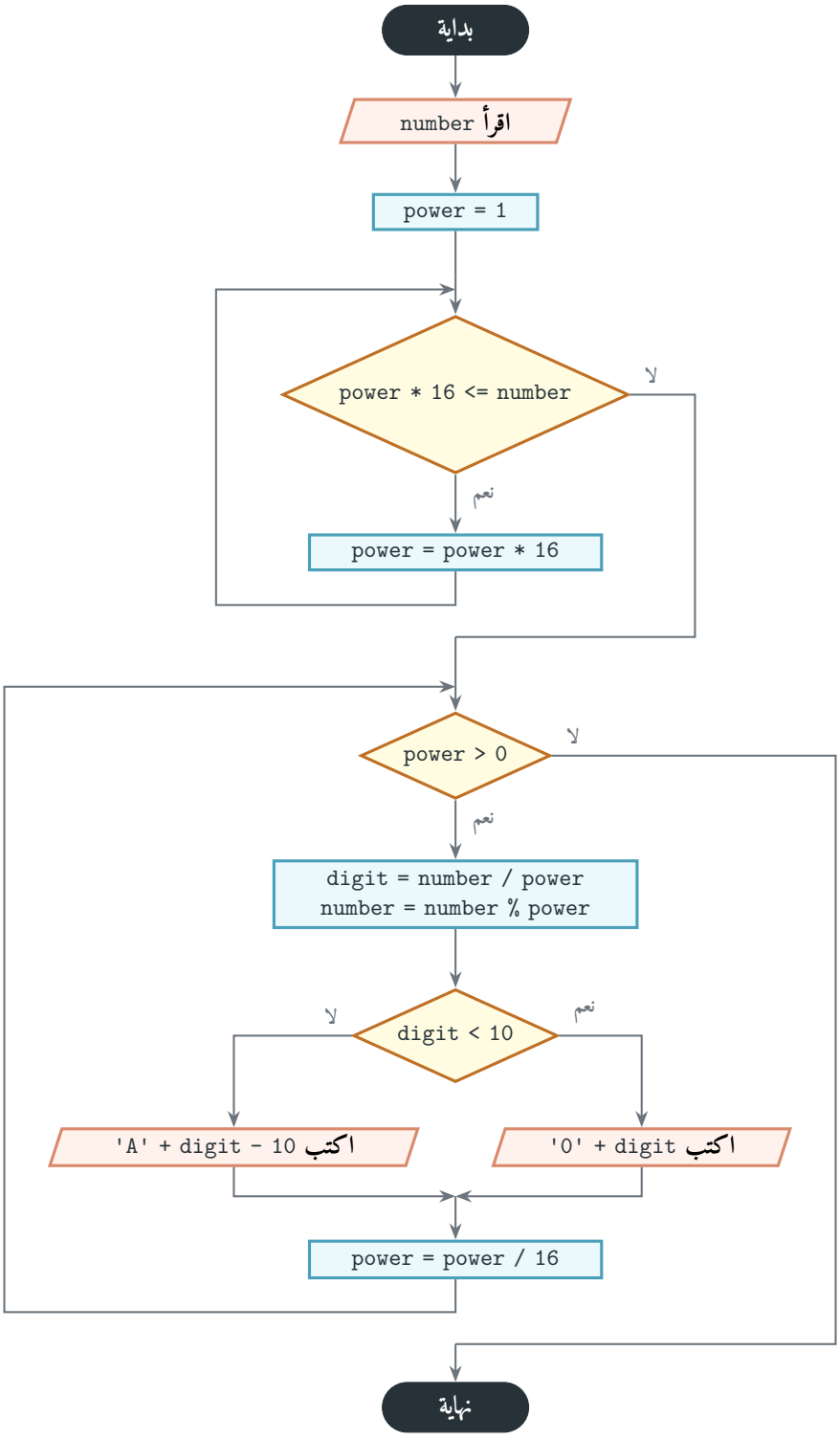
المدخلات

```
3054198966
```

نتيجة التنفيذ

```
3054198966 = 0xB60B60B6
```

المخطط الإنسيابي (Organigramme)



تمرين 3.48 الجذر الصحيح بالبحث الثنائي

علم البيانات ★★★ صعب

لتنظيم صور المنتجات في **شبكة مربعة**، نحتاج أكبر عدد صحيح `r` بحيث `r² <= number`. اكتب برنامجًا يجده بالبحث الثنائي (دون أعداد حقيقية)، ثم يحدد إن كان `number` مربعًا كاملًا، وأبعاد الشبكة اللازمة `r × c`.

خطوات الحل

1) `lo = 0`، `hi = number` (أو مليارين كحد أقصى).
2) `mid = lo + (hi - lo) / 2`؛ إذا كان `mid * mid <= number` فـ `ans = mid` و`lo = mid + 1`، وإلا `hi = mid - 1`.
3) عدد الأعمدة: `c = (number + r - 1) / r`، وإذا لم يكن مربعًا كاملًا نستعمل `r + 1` أسطر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long long number, lo, hi, mid, ans = 0;
    scanf("%lld", &number);
    lo = 0;
    hi = (number < 3000000000LL) ? number : 3000000000LL;  // حد أعلى آمن للمربع
    while (lo <= hi) {
        mid = lo + (hi - lo) / 2;      // يتجنب تجاوز السعة عند الجمع
        if (mid * mid <= number) {
            ans = mid;                 // مرشح صالح، نبحث عن أكبر
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    printf("isqrt(%lld) = %lld\n", number, ans);
    if (ans * ans == number)
        printf("Perfect square: grid %lld x %lld\n", ans, ans);
    else
        printf("Not a square: grid %lld x %lld\n", ans + 1, (number + ans) / (ans + 1));
    return 0;
}
```

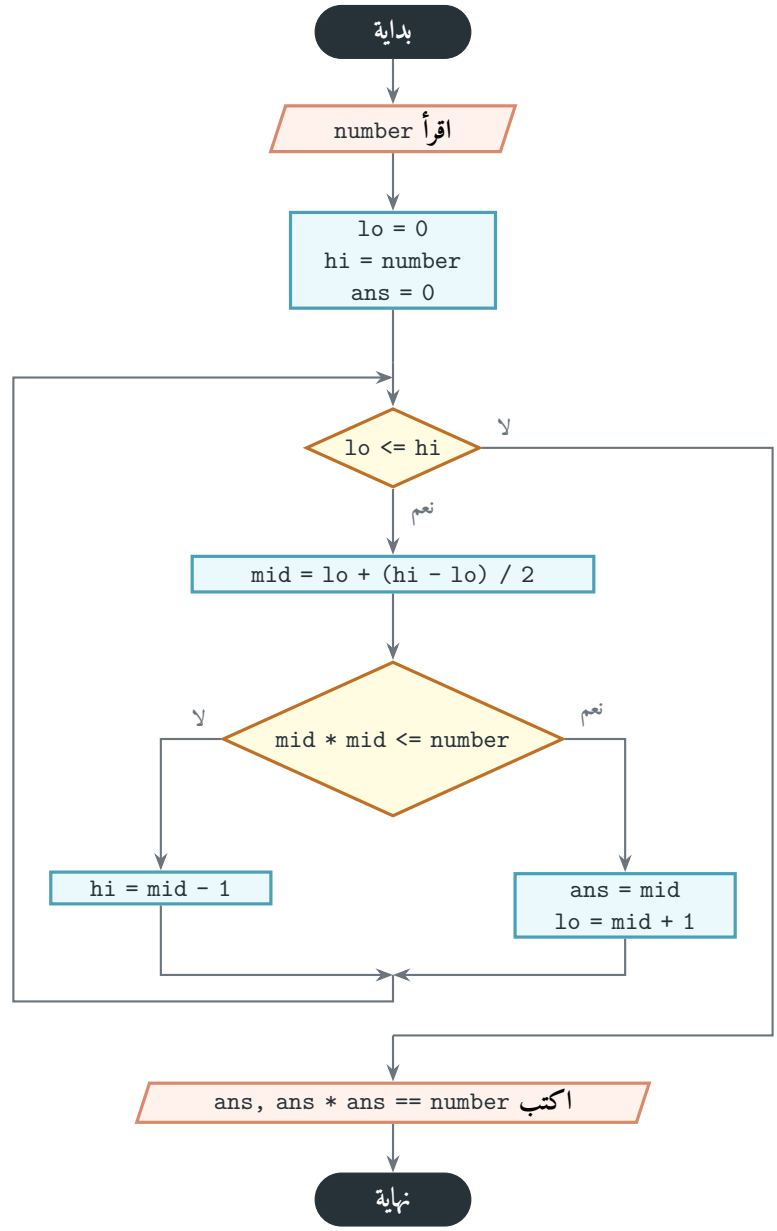
المدخلات

```
1000000007
```

نتيجة التنفيذ

```
isqrt(1000000007) = 31622
Not a square: grid 31623 x 31623
```

المخطط الإنسيابي (Organigramme)



تمرين 3.49 محاكاة سياسة التموين (s, S)

تجارة إلكترونية ★★★ صعب

سياسة (s, S): في نهاية كل يوم، إذا نزل المخزون تحت `minLevel` نطلب ما يرفعه إلى `maxLevel`، ويصل الطلب بعد يومين. الطلب اليومي عشوائي بين 0 و39 بمولد LCG (`seed = (seed * 75) % 65537`, `demand = seed % 40`). التكاليف: 2 دج تخزين لكل وحدة في اليوم، 50 دج لكل وحدة ناقصة، و500 دج لكل طلبية. اكتب برنامجًا يحاكي 60 يومًا ويعرض التكاليف وعدد أيام النفاد.

خطوات الحل

1) متغيرات: `stock`، كمية الطلب المعلق `pending`، وأيام الانتظار `wait`.
2) كل يوم: نستقبل الطلب إذا حان وقته، نولد الطلب، نبيع ما نستطيع ونحسب النقص.
3) تكلفة التخزين على المخزون المتبقي.

4) إذا كان `stock < minLevel` ولا يوجد طلب معلق: نطلب `maxLevel - stock`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long minLevel, maxLevel, seed, stock, pending = 0, demand;
    int day, wait = 0, outdays = 0, orders = 0;
    double hold = 0, shortc = 0, orderc = 0;
    scanf("%ld %ld %ld", &minLevel, &maxLevel, &seed);
    stock = maxLevel;
    for (day = 1; day <= 60; day++) {
        if (pending > 0) {            // طلبية في الطريق
            wait--;
            if (wait == 0) {          // وصلت
                stock += pending;
                pending = 0;
            }
        }
        seed = (seed * 75) % 65537;   // مولد عشوائي يدوي
        demand = seed % 40;                // طلب اليوم 0..39
        if (demand > stock) {              // نفاد: الطلب أكبر من المخزون
            shortc += 50.0 * (demand - stock);
            stock = 0;
            outdays++;
        } else {
            stock -= demand;
        }
        hold += 2.0 * stock;          // تكلفة التخزين اليومية
        if (stock < minLevel && pending == 0) {
            pending = maxLevel - stock;       // نطلب ما يكمل إلى maxLevel
            wait = 2;
            orders++;
            orderc += 500;
        }
    }
    printf("Orders placed: %d\n", orders);
    printf("Stock-out days: %d\n", outdays);
    printf("Holding = %.0f, shortage = %.0f, ordering = %.0f\n", hold, shortc, orderc);
    printf("Total cost = %.0f DA\n", hold + shortc + orderc);
    return 0;
}
```

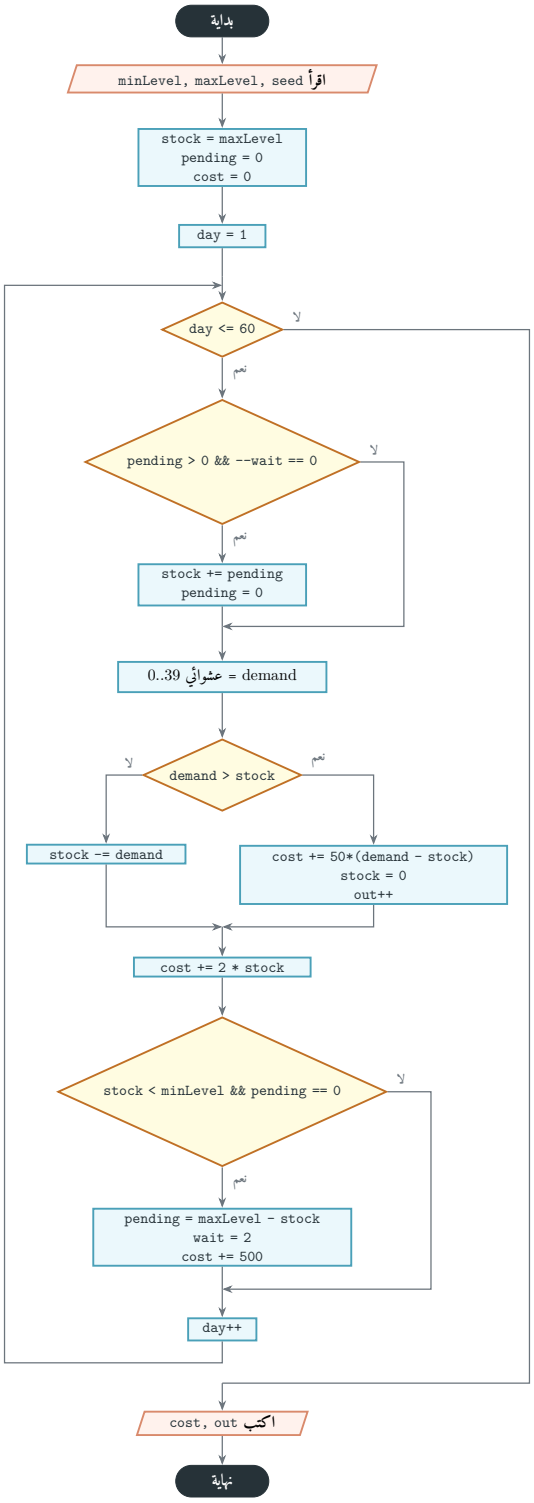
المدخلات

```
60 150 7
```

نتيجة التنفيذ

```
Orders placed: 10
Stock-out days: 0
Holding = 9132, shortage = 0, ordering = 5000
Total cost = 14132 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.50 أفضل يوم للشراء والبيع

علم البيانات ★★★ صعب

يتابع تاجر سعر منتج يوميًا لمدة `nbDays` يومًا ويريد **الشراء مرة والبيع مرة** لاحقًا لتحقيق أكبر ربح. اكتب برنامجًا يحل المسألة في **مرور واحد**: نحتفظ بأدنى سعر رأيناه حتى الآن وبيوم حدوثه.

خطوات الحل

1) `minP` = أدنى سعر حتى اليوم، و`minDay` يومه.
2) كل يوم: الربح الممكن إذا بعنا اليوم هو `price - minP`.
3) إذا كان أكبر من `best` نحفظه مع يومي الشراء والبيع.
4) ثم نحدّث `minP` إذا كان سعر اليوم أصغر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day, minDay = 1, buy = 0, sell = 0;
    double price, minP, best = 0;
    scanf("%d %lf", &nbDays, &price);
    minP = price;
    for (day = 2; day <= nbDays; day++) {
        scanf("%lf", &price);
        if (price - minP > best) {    // البيع اليوم أفضل من كل ما سبق
            best = price - minP;
            buy = minDay;
            sell = day;
        }
        if (price < minP) {           // سعر شراء أفضل للأيام القادمة
            minP = price;
            minDay = day;
        }
    }
    if (best > 0)
        printf("Buy day %d, sell day %d: profit %.0f DA\n", buy, sell, best);
    else
        printf("No profitable trade\n");
    return 0;
}
```

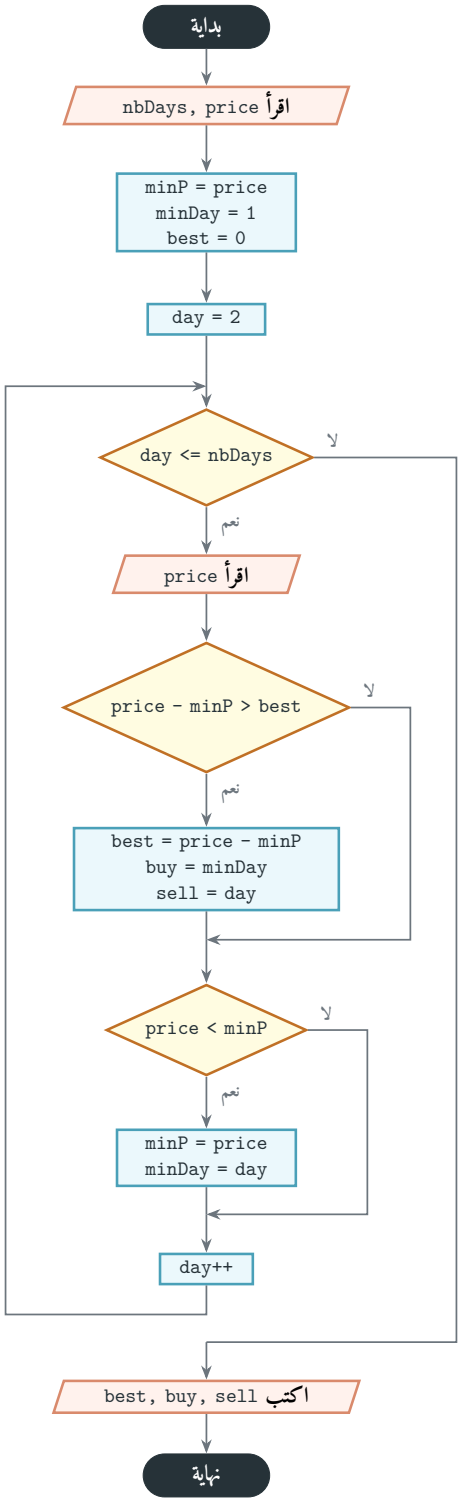
المدخلات

```
10
320 280 300 250 310 390 360 240 330 370
```

نتيجة التنفيذ

```
Buy day 4, sell day 6: profit 140 DA
```

المخطط الإنسيابي (Organigramme)



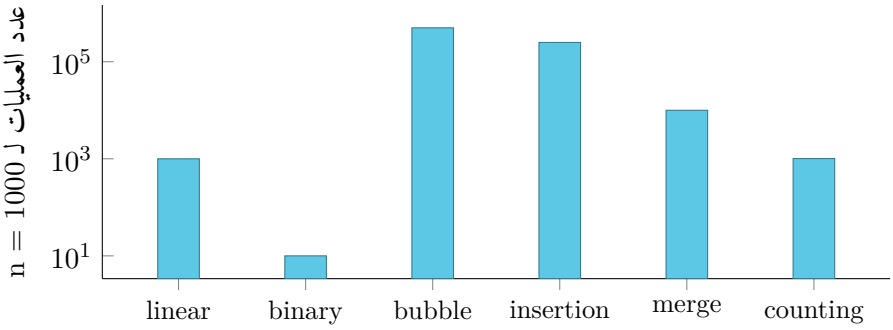
الفصل 4

الجداول أحادية البعد

تعريفات

- **الجدول**: مجموعة عناصر من نفس النوع متجاورة في الذاكرة، لها اسم واحد، ونصل إلى كل عنصر بفهرسه.
- الفهرس في C يبدأ من 0 وينتهي عند `SIZE - 1`.
- **التصريح**: `double prices[50];` **والتهيئة**: `int stock[4] = {5, 0, 12, 7};`
- **الحجم المستعمل** `nbItems` قد يكون أصغر من **السعة** `MAX`.



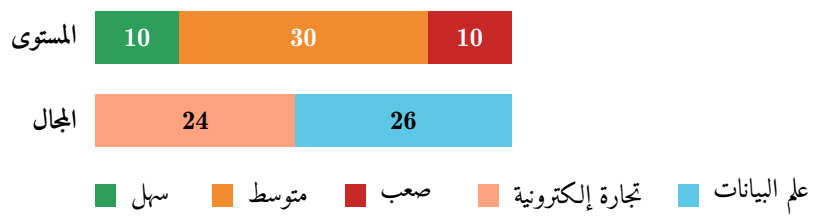


مقارنة تقريبية لتكلفة خوارزميات البحث والترتيب المستعملة في هذا الفصل (سلم لوغاريتمي).

تجاوز حدود الجدول

C لا يتحقق من الفهرس: الكتابة في `prices[50]` لجدول من 50 خانة تفسد الذاكرة دون أي رسالة خطأ.

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 4.1 عرض كتالوج الأسعار

تجارة إلكترونية سهل

اكتب برنامجًا يقرأ أسعار nbPrices منتجات (nbPrices <= 50) في جدول، ثم يعرض كل سعر مع فهرسه.

خطوات الحل

1) نصرح بجدول double prices[50].
2) حلقة أولى للقراءة: scanf("%lf", &prices[index]).
3) حلقة ثانية للعرض.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 50                  // السعة القصوى للجدول

int main(void) {
    double prices[MAX];         // جدول من 50 عددًا حقيقيًا
    int nbPrices, index;
    scanf("%d", &nbPrices);
    for (index = 0; index < nbPrices; index++)
        scanf("%lf", &prices[index]);      // &prices[index] عنوان الخانة index
    for (index = 0; index < nbPrices; index++)
        printf("T[%d] = %.2f DA\n", index, prices[index]);
    return 0;
}
```

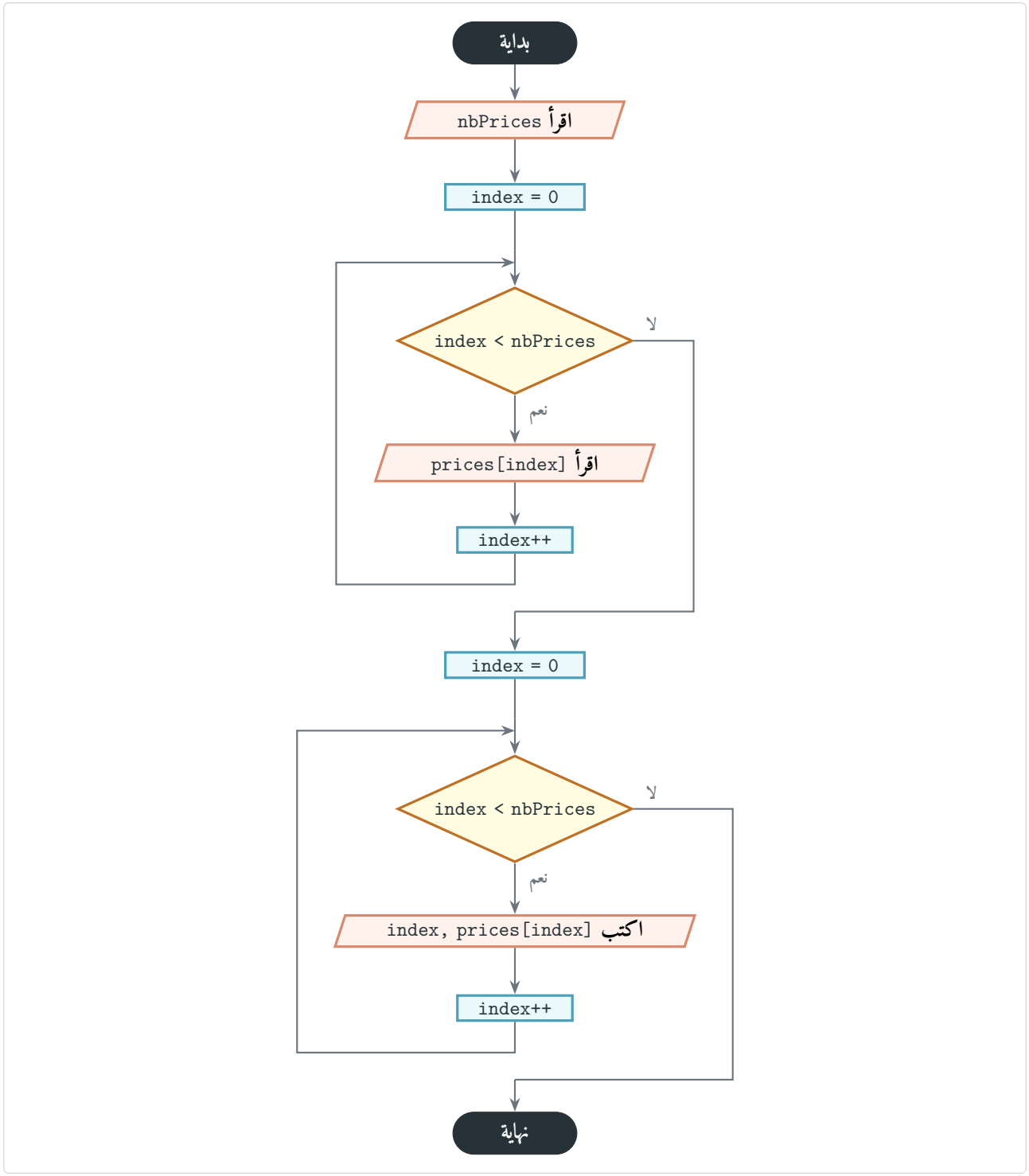
المدخلات

```
5
1200 450 3999 89.5 700
```

نتيجة التنفيذ

```
T[0] = 1200.00 DA
T[1] = 450.00 DA
T[2] = 3999.00 DA
T[3] = 89.50 DA
T[4] = 700.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.2 مبيعات الأسبوع

علم البيانات سهل

مبيعات أسبوع مخزنة في جدول مهيأ مباشرة في البرنامج. اكتب برنامجًا يحسب **المجموع** و**المتوسط** وعدد الأيام التي تجاوزت المتوسط.

خطوات الحل

1) نهيئ الجدول `{...}` بسبع قيم.

2) حلقة أولى للمجموع، ثم المتوسط.

3) حلقة ثانية للعد: نحتاج المتوسط أولًا، لذلك نمر على الجدول مرتين.

البرنامج بلغة C

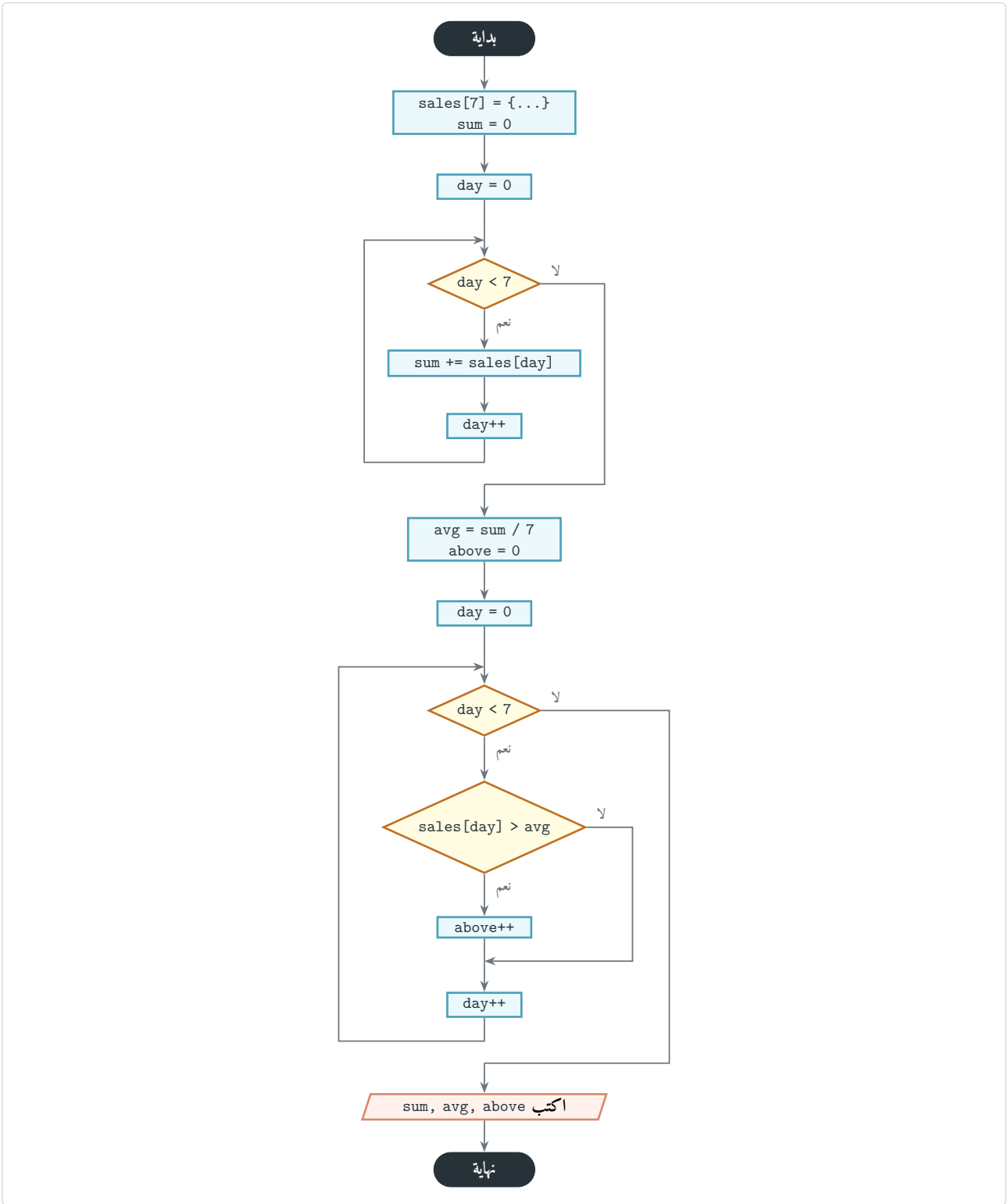
```c
#include <stdio.h>

int main(void) {
    double sales[7] = {120, 95, 140, 180, 160, 210, 75};   // مبيعات 7 أيام
    double sum = 0, avg;
    int day, above = 0;
    for (day = 0; day < 7; day++)
        sum += sales[day];
    avg = sum / 7;
    // مرور ثانٍ: لم يكن المتوسط معروفًا في المرور الأول
    for (day = 0; day < 7; day++)
        if (sales[day] > avg)
            above++;
    printf("Total = %.0f, average = %.2f\n", sum, avg);
    printf("Days above average: %d\n", above);
    return 0;
}
```

نتيجة التنفيذ

```
Total = 980, average = 140.00
Days above average: 3
```

المخطط الإنسيابي (Organigramme)



تمرين 4.3 أكثر منتج مبيعًا

تجارة إلكترونية ⭐ سهل

يحتوي الجدول `sold` عدد الوحدات المباعة لكل منتج (الفهرس هو رقم المنتج). اكتب برنامجًا يجد **أكثر منتج مبيعًا** وفهرسه.

خطوات الحل

1) `best = 0` (فهرس الأكبر مبدئيًا).
2) لكل `index` من 1: إذا كان `sold[index] > sold[best]` فـ `best = index`.
3) نحفظ الفهرس فقط، فالقيمة هي `sold[best]`.

البرنامج بلغة C

```c
#include <stdio.h>

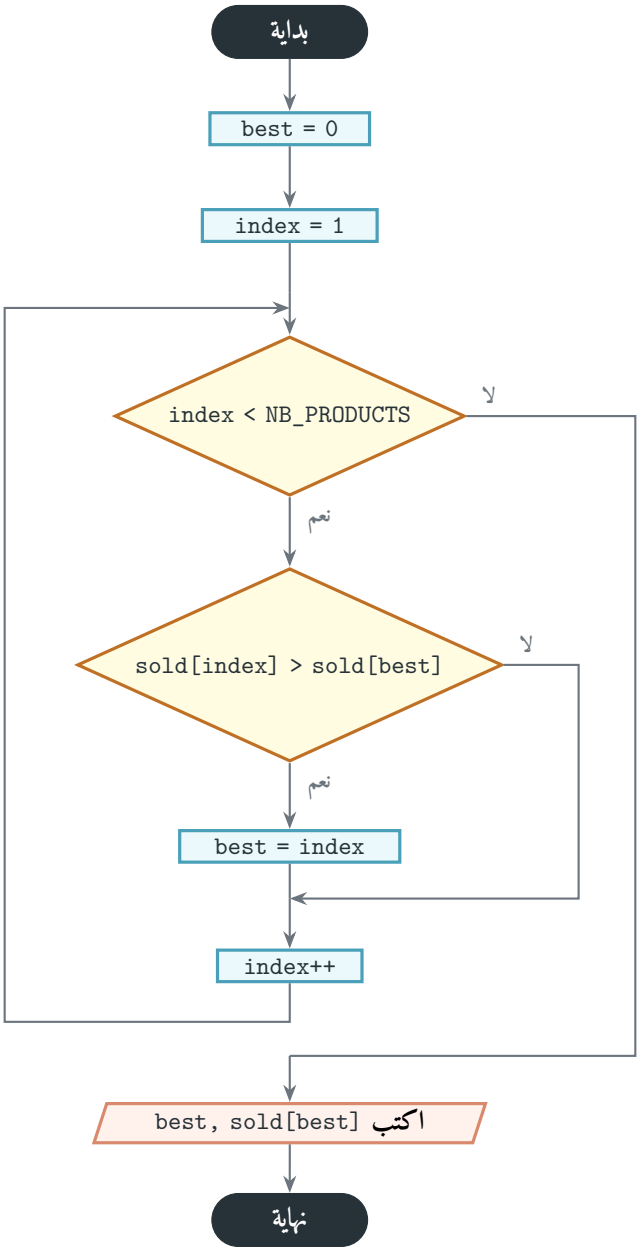
#define NB_PRODUCTS 8

int main(void) {
    int sold[NB_PRODUCTS] = {34, 120, 56, 98, 143, 12, 77, 143};
    int index, best = 0;              // نحفظ الفهرس، لا القيمة
    for (index = 1; index < NB_PRODUCTS; index++)
        if (sold[index] > sold[best])
            best = index;
    printf("Best seller: product #%d with %d units\n", best, sold[best]);
    return 0;
}
```

نتيجة التنفيذ

```
Best seller: product #4 with 143 units
```

المخطط الإنسيابي (Organigramme)



تمرين 4.4 المنتجات النافدة

تجارة إلكترونية
سهل

اكتب برنامجًا يعرض أرقام المنتجات التي مخزونها **0** من جدول المخزون، ويعد عددها.

خطوات الحل

1) نمر على الجدول.
2) إذا كان `stock[index] == 0` نعرض `index` ونزيد العداد.

البرنامج بلغة C

```c
#include <stdio.h>

```

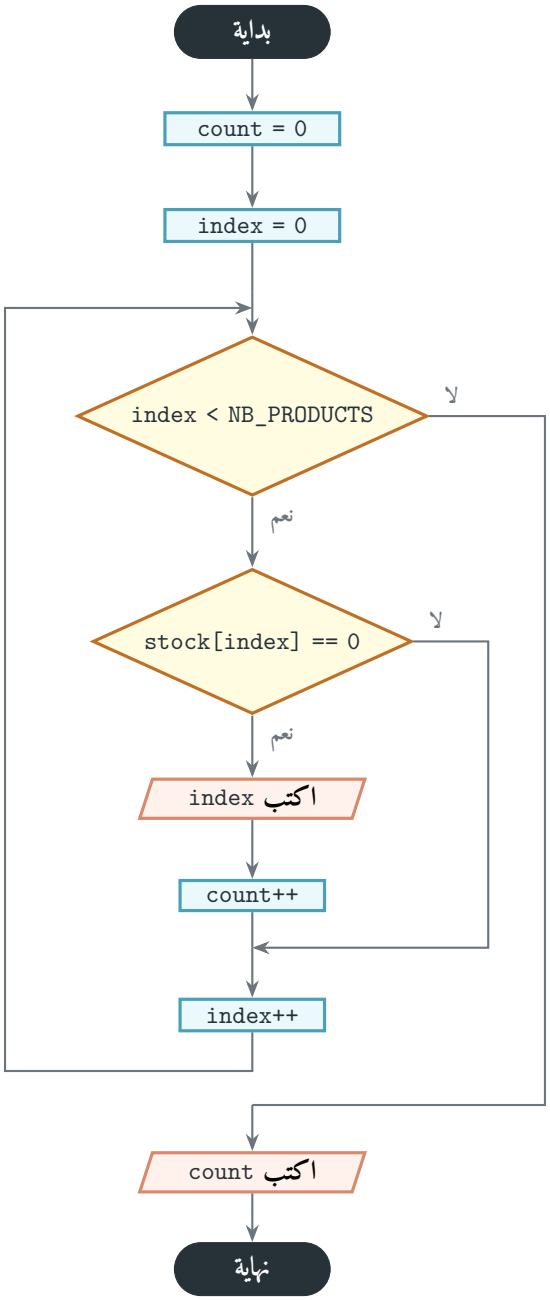
```c
#define NB_PRODUCTS 10

int main(void) {
    int stock[NB_PRODUCTS] = {5, 0, 12, 0, 7, 3, 0, 22, 1, 9};
    int index, count = 0;
    printf("Out of stock:");
    for (index = 0; index < NB_PRODUCTS; index++) {
        if (stock[index] == 0) {      // منتج نافد
            printf(" #%d", index);
            count++;
        }
    }
    printf("\n%d products to reorder\n", count);
    return 0;
}
```

نتيجة التنفيذ

```
Out of stock: #1 #3 #6
3 products to reorder
```

المخطط الإنسيابي (Organigramme)



تمرين 4.5 رفع كل الأسعار

تجارة إلكترونية

سهل

بسبب ارتفاع تكاليف النقل، يرفع المتجر **كل** الأسعار بنسبة `percent`%. اكتب برنامجًا يعدل الجدول **في مكانه** ثم يعرضه.

خطوات الحل

1) نقرأ النسبة `percent`.
2) لكل خانة: `prices[index] = prices[index] * (1 + percent / 100)`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_PRICES 5

int main(void) {
    double prices[NB_PRICES] = {1000, 2500, 499, 12000, 75};
    double percent;
    int index;
    scanf("%lf", &percent);
    for (index = 0; index < NB_PRICES; index++)
        prices[index] = prices[index] * (1 + percent / 100);  // تعديل الخانة في مكانها
    for (index = 0; index < NB_PRICES; index++)
        printf("%.2f ", prices[index]);
    printf("\n");
    return 0;
}
```

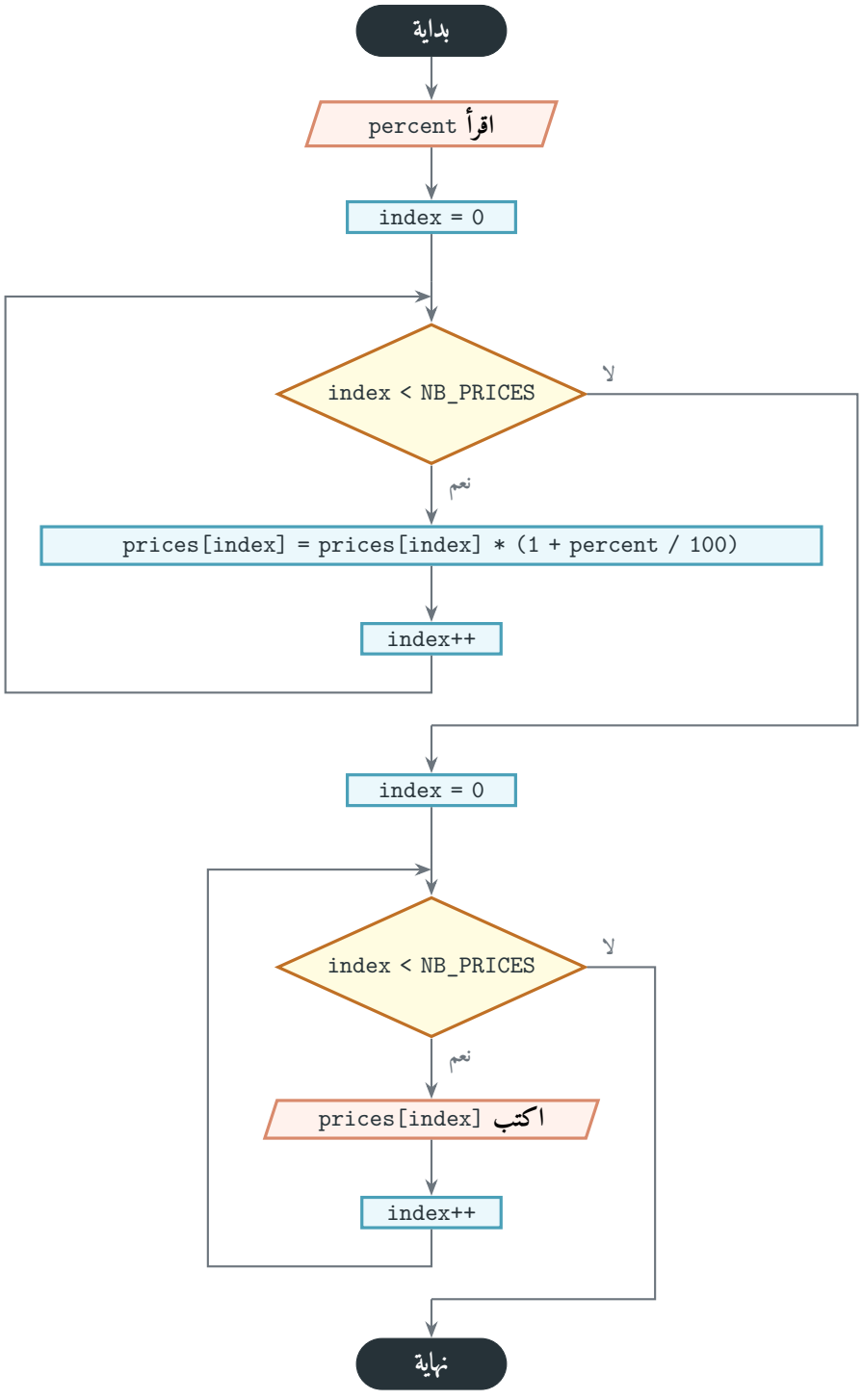
المدخلات

```
8
```

نتيجة التنفيذ

```
1080.00 2700.00 538.92 12960.00 81.00
```

المخطط الإنسيابي (Organigramme)



تمرين 4.6 آخر الطلبيات أولًا

تجارة إلكترونية ⭐ سهل

تحفظ لوحة التحكم أرقام الطلبيات بترتيب وصولها. اكتب برنامجًا يقرأ `nbOrders` رقمًا ويعرضها **من الأحدث إلى الأقدم**.

خطوات الحل

1) نقرأ الأرقام في الجدول.

2) نعرضها بحلقة تنازلية من `nbOrders - 1` إلى 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int orders[100], nbOrders, index;
    scanf("%d", &nbOrders);
    for (index = 0; index < nbOrders; index++)
        scanf("%d", &orders[index]);
    printf("Latest first:");
    for (index = nbOrders - 1; index >= 0; index--)  // من الفهرس الأخير إلى 0
        printf(" %d", orders[index]);
    printf("\n");
    return 0;
}
```

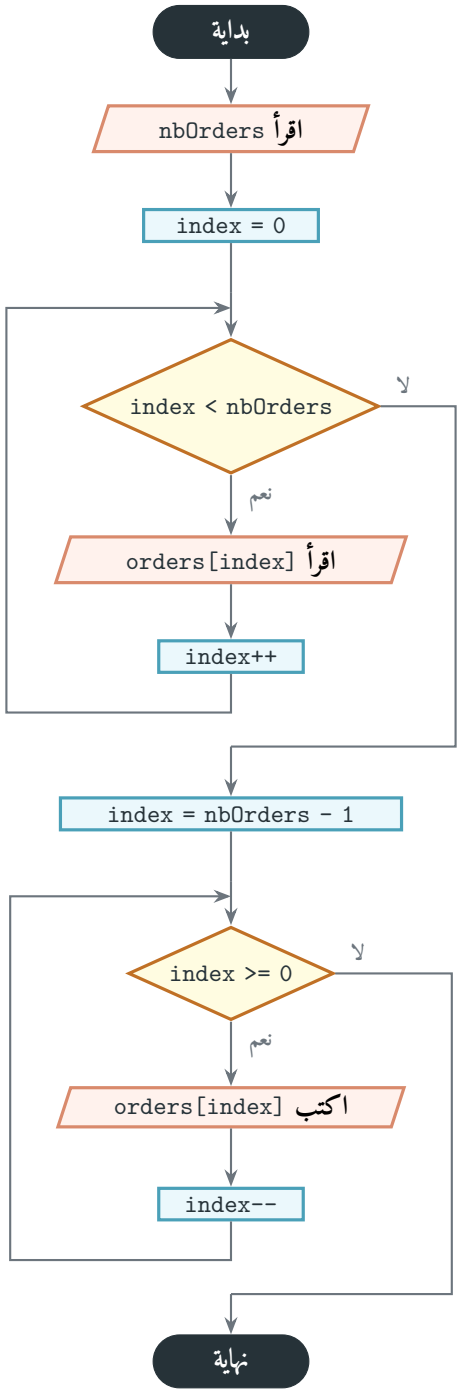
المدخلات

```
6
1001 1002 1005 1009 1010 1014
```

نتيجة التنفيذ

```
Latest first: 1014 1010 1009 1005 1002 1001
```

المخطط الإنسيابي (Organigramme)



تمرين 4.7 البحث عن منتج

تجارة إلكترونية ⭐ سهل

اكتب برنامجًا يبحث عن رقم منتج في جدول الأرقام بحثًا تسلسليًا، ويعرض موضعه أو رسالة "غير موجود".

خطوات الحل

1) `pos = -1` يعني "لم نجده بعد".
2) نمر على الجدول ما دام `pos == -1`؛ عند التساوي نحفظ الموضع.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_IDS 8

int main(void) {
    int ids[NB_IDS] = {1203, 4410, 7781, 5512, 3001, 9045, 2210, 6600};
    int target, index, pos = -1;           // -1: غير موجود
    scanf("%d", &target);
    for (index = 0; index < NB_IDS && pos == -1; index++)  // نتوقف عند أول تطابق
        if (ids[index] == target)
            pos = index;
    if (pos >= 0)
        printf("Product %d found at index %d\n", target, pos);
    else
        printf("Product %d not found\n", target);
    return 0;
}
```

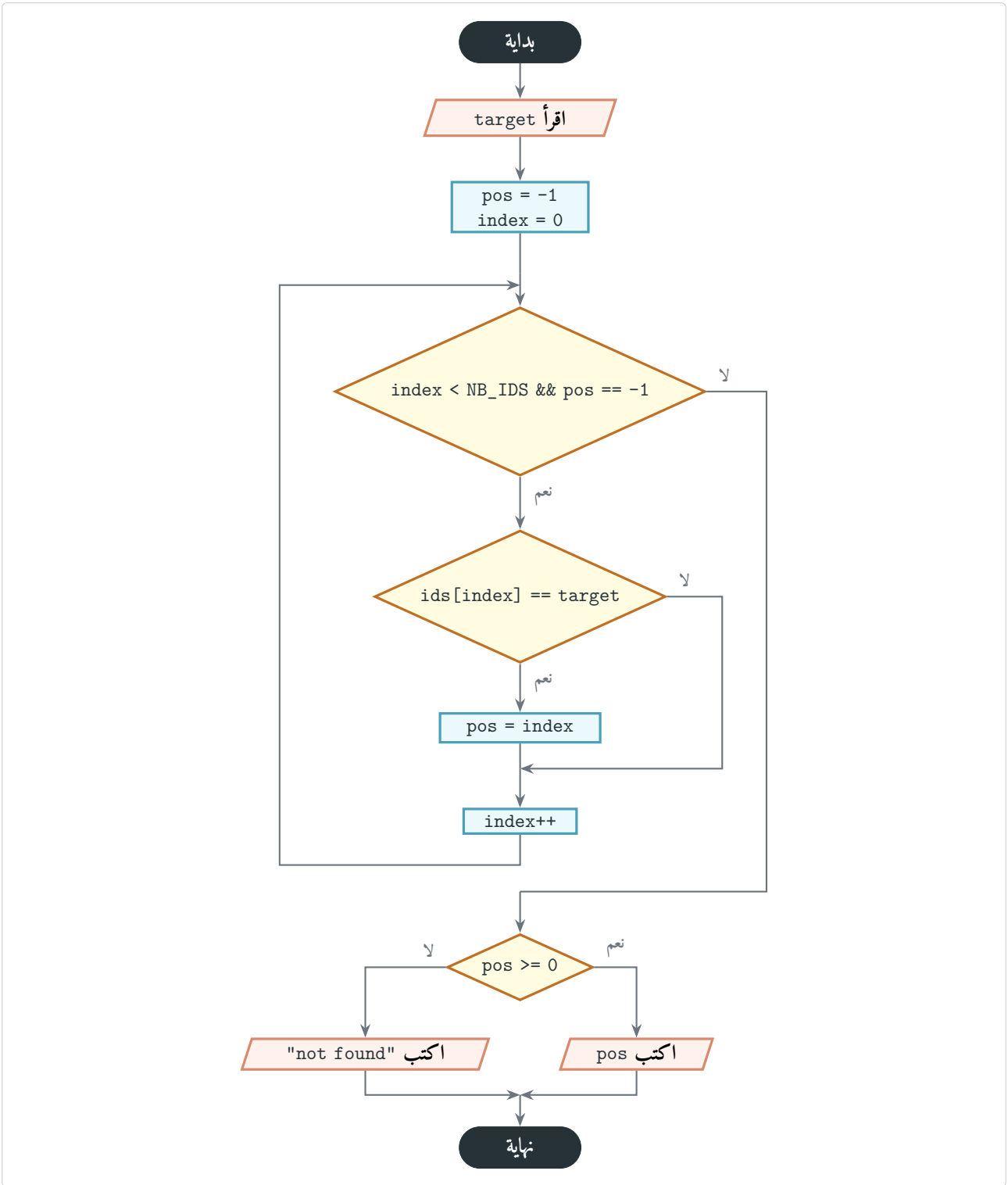
المدخلات

```
5512
```

نتيجة التنفيذ

```
Product 5512 found at index 3
```

المخطط الإنسيابي (Organigramme)



تمرين 4.8 نسخ المنتجات الفاخرة

تجارة إلكترونية | سهل

اكتب برنامجًا ينسخ الأسعار التي تتجاوز 5000 دج من جدول `prices` إلى جدول جديد `premium` (قسم المنتجات الفاخرة)، ثم يعرض `premium`.

خطوات الحل

1) `nbPremium = 0` عدد العناصر في `premium`.
2) إذا كان `prices[index] > 5000`: `premium[nbPremium] = prices[index]` ثم `nbPremium++`.
3) `nbPremium` في النهاية هو حجم `premium`.

البرنامج بلغة C

```c
#include <stdio.h>

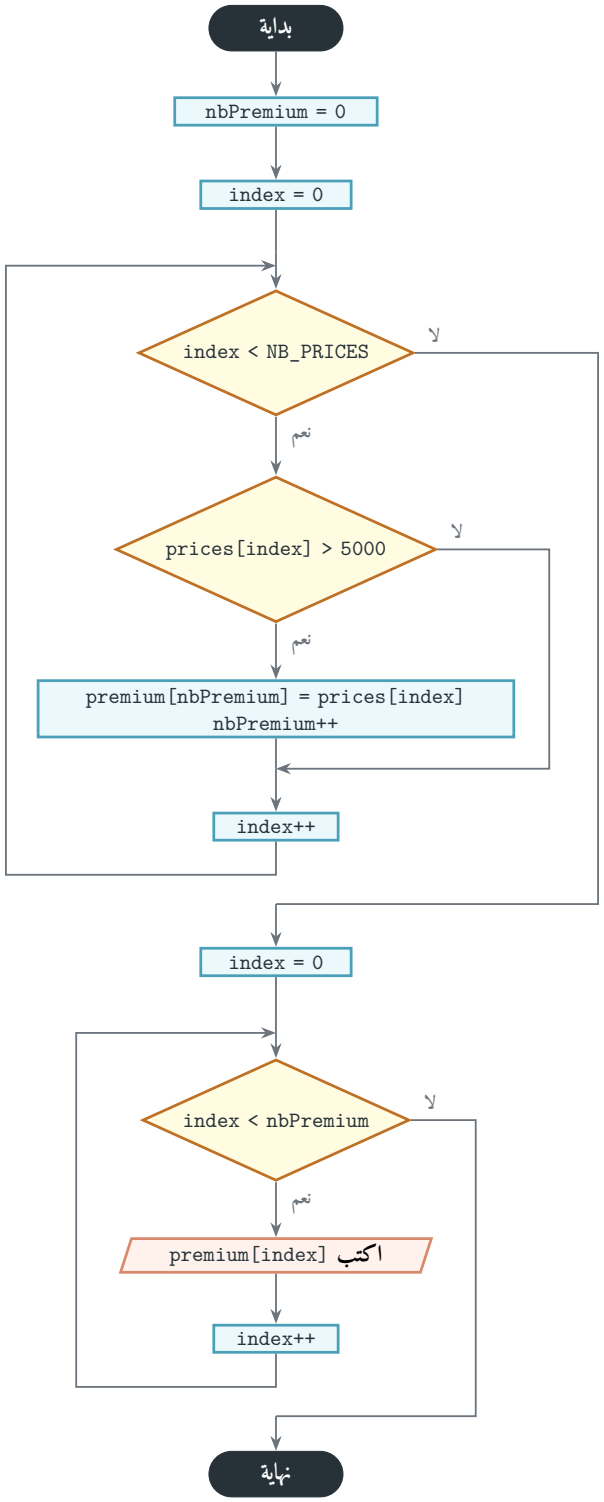
#define NB_PRICES 8

int main(void) {
    double prices[NB_PRICES] = {1200, 8900, 450, 15000, 5000, 7200, 300, 22000};
    double premium[NB_PRICES];                  // في أسوأ الحالات كل العناصر
    int index, nbPremium = 0;                   // nbPremium: عدد العناصر المنسوخة
    for (index = 0; index < NB_PRICES; index++)
        if (prices[index] > 5000)
            premium[nbPremium++] = prices[index];        // نسخ ثم نزيد nbPremium
    printf("%d premium products:", nbPremium);
    for (index = 0; index < nbPremium; index++)
        printf(" %.0f", premium[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
4 premium products: 8900 15000 7200 22000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.9 نسبة التقييمات الإيجابية

علم البيانات
سهل

اكتب برنامجًا يحسب نسبة التقييمات **الإيجابية** (4 أو 5 نجوم) في جدول من التقييمات.

خطوات الحل

1) نعد العناصر التي تحقق `ratings[index] >= 4`.
2) النسبة: `pos * 100.0 / NB_REVIEWS`.

البرنامج بلغة C

```c
#include <stdio.h>

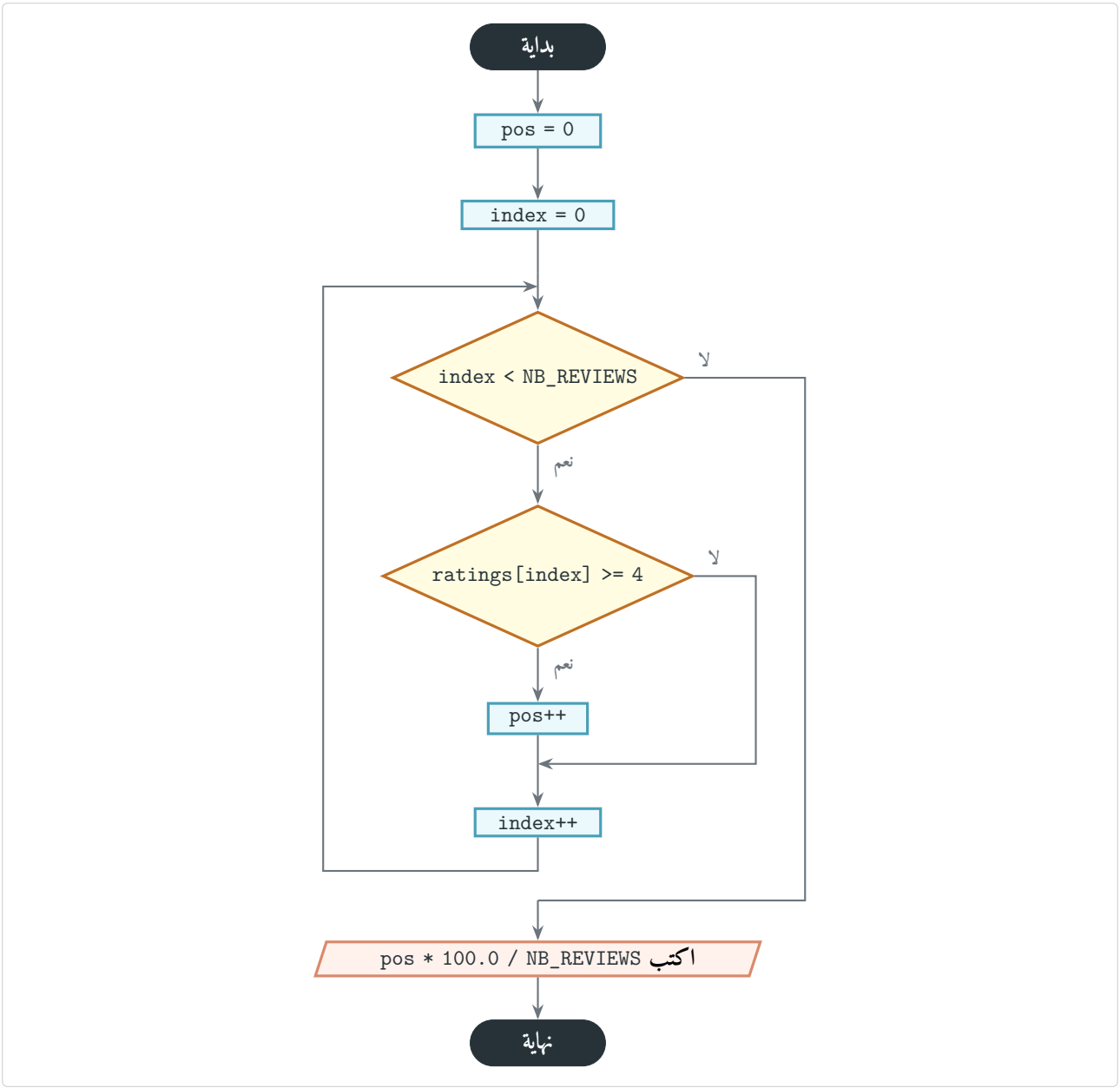
#define NB_REVIEWS 12

int main(void) {
    int ratings[NB_REVIEWS] = {5, 4, 3, 5, 2, 4, 4, 1, 5, 3, 4, 5};
    int index, pos = 0;
    for (index = 0; index < NB_REVIEWS; index++)
        if (ratings[index] >= 4)
            pos++;
    printf("Positive reviews: %d / %d = %.1f %%\n", pos, NB_REVIEWS, pos * 100.0 / NB_REVIEWS);
    return 0;
}
```

نتيجة التنفيذ

```
Positive reviews: 8 / 12 = 66.7 %
```

المخطط الإنسيابي (Organigramme)



تمرين 4.10 الفاتورة من جدولين

تجارة إلكترونية | سهل

جدول `price` يحتوي أسعار المنتجات وجدول `qty` الكميات المطلوبة (نفس الفهرس = نفس المنتج). اكتب برنامجًا يعرض مبلغ كل سطر والمجموع.

خطوات الحل

1) لكل `index`: `line = price[index] * qty[index]`.
2) تجاهل الأسطر ذات الكمية 0.
3) نجمع المبالغ.

البرنامج بلغة C

```c
#include <stdio.h>

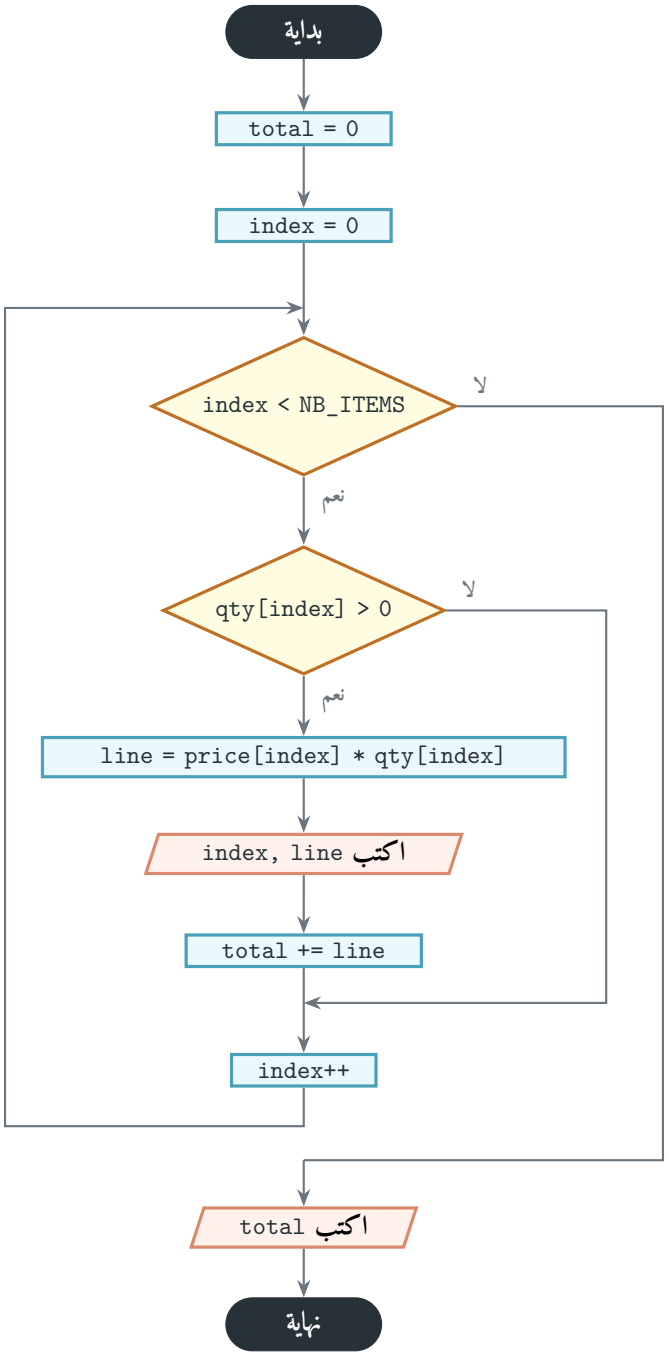
#define NB_ITEMS 5

int main(void) {
    double price[NB_ITEMS] = {350, 1200, 80, 4500, 999};
    int qty[NB_ITEMS] = {2, 0, 10, 1, 3};    // جدولان متوازيان
    double total = 0, line;
    int index;
    for (index = 0; index < NB_ITEMS; index++) {
        if (qty[index] > 0) {
            line = price[index] * qty[index];
            printf("Item %d: %d x %.2f = %.2f\n", index, qty[index], price[index], line);
            total += line;
        }
    }
    printf("TOTAL = %.2f DA\n", total);
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 2 x 350.00 = 700.00
Item 2: 10 x 80.00 = 800.00
Item 3: 1 x 4500.00 = 4500.00
Item 4: 3 x 999.00 = 2997.00
TOTAL = 8997.00 DA
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 4.11 عكس الجدول في مكانه

علم البيانات ★★ متوسط

لعرض سلسلة زمنية من الأحدث إلى الأقدم، اكتب برنامجًا **يعكس** جدولًا في مكانه (دون جدول ثانٍ) بتبديل العناصر المتناظرة.

خطوات الحل

1) `left` من البداية و`right` من النهاية.
2) ما دام `left < right`: نبدل `series[left]` و`series[right]` ثم `left++` و`right--`.

البرنامج بلغة C

```c
#include <stdio.h>

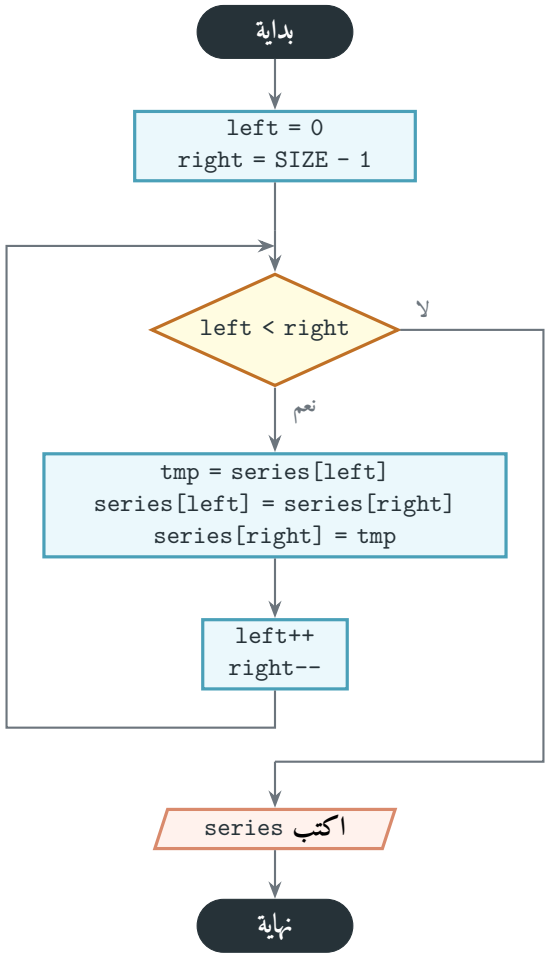
#define SIZE 7

int main(void) {
    int series[SIZE] = {10, 20, 30, 40, 50, 60, 70};
    int left = 0, right = SIZE - 1, tmp;
    while (left < right) {              // نتوقف عند المنتصف
        tmp = series[left];             // تبديل الطرفين
        series[left] = series[right];
        series[right] = tmp;
        left++;
        right--;
    }
    for (left = 0; left < SIZE; left++)
        printf("%d ", series[left]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
70 60 50 40 30 20 10
```

المخطط الإنسيابي (Organigramme)



تمرين 4.12 تدوير جدول المناوبات

تجارة إلكترونية ★★ متوسط

جدول يحتوي أرقام عمال التوصيل حسب أيام الأسبوع. كل أسبوع **يدور** الجدول بخطوة `steps` نحو اليسار (العامل الأول يصبح في الآخر). اكتب البرنامج.

خطوات الحل

1) تدوير بخطوة واحدة: نحفظ `shifts[0]`، نزيح الباقي نحو اليسار، ونضع المحفوظ في الآخر.
2) نكرر ذلك `steps` مرة (أو `steps % NB_DAYS` مرة).

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_DAYS 7

int main(void) {
    int shifts[NB_DAYS] = {11, 12, 13, 14, 15, 16, 17};
    int steps, step, index, first;
    scanf("%d", &steps);
    for (step = 0; step < steps % NB_DAYS; step++) {      // تدوير NB_DAYS مرة يعيد الجدول كما كان
        first = shifts[0];
```

```
11          for (index = 0; index < NB_DAYS - 1; index++)
12              shifts[index] = shifts[index + 1];          // إزاحة نحو اليسار
13          shifts[NB_DAYS - 1] = first;
14      }
15      printf("Rotated:");
16      for (index = 0; index < NB_DAYS; index++)
17          printf(" %d", shifts[index]);
18      printf("\n");
19      return 0;
20  }
```

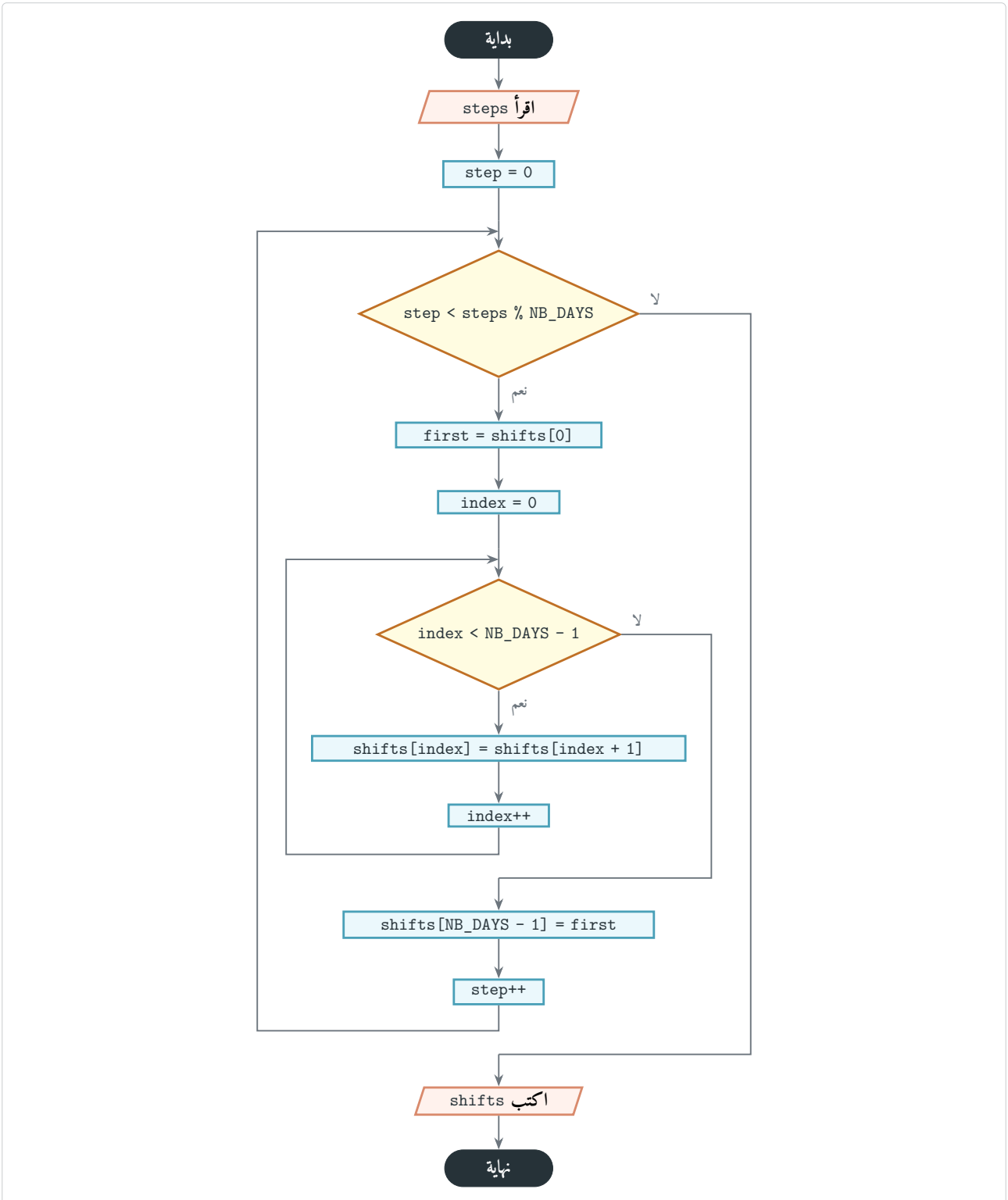
المدخلات

```
2
```

نتيجة التنفيذ

```
Rotated: 13 14 15 16 17 11 12
```

المخطط الإنسيابي (Organigramme)



تمرين 4.13 إدراج منتج جديد في موضع

تجارة إلكترونية ★★ متوسط

قائمة المنتجات المميزة مخزنة في جدول. اكتب برنامجًا يدرج منتجًا جديدًا في الموضع `pos` مع إزاحة العناصر الموالية نحو اليمين.

خطوات الحل

1) نتحقق أن `0 <= pos <= size` وأن الجدول غير ممتلئ.
2) نزيح من الآخر: `featured[index] = featured[index - 1]` لـ `index` من `size` نزولًا إلى `pos + 1`.
3) `featured[pos] = newId` ثم `size++`.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 10

int main(void) {
    int featured[MAX] = {101, 205, 330, 412, 560};
    int size = 5, pos, newId, index;          // size: عدد العناصر المستعملة
    scanf("%d %d", &pos, &newId);
    if (pos < 0 || pos > size || size == MAX) {
        printf("Insertion impossible\n");
        return 0;
    }
    // نبدأ من الآخر لكي لا نمسح أي عنصر
    for (index = size; index > pos; index--)
        featured[index] = featured[index - 1];
    featured[pos] = newId;
    size++;
    for (index = 0; index < size; index++)
        printf("%d ", featured[index]);
    printf("\n");
    return 0;
}
```

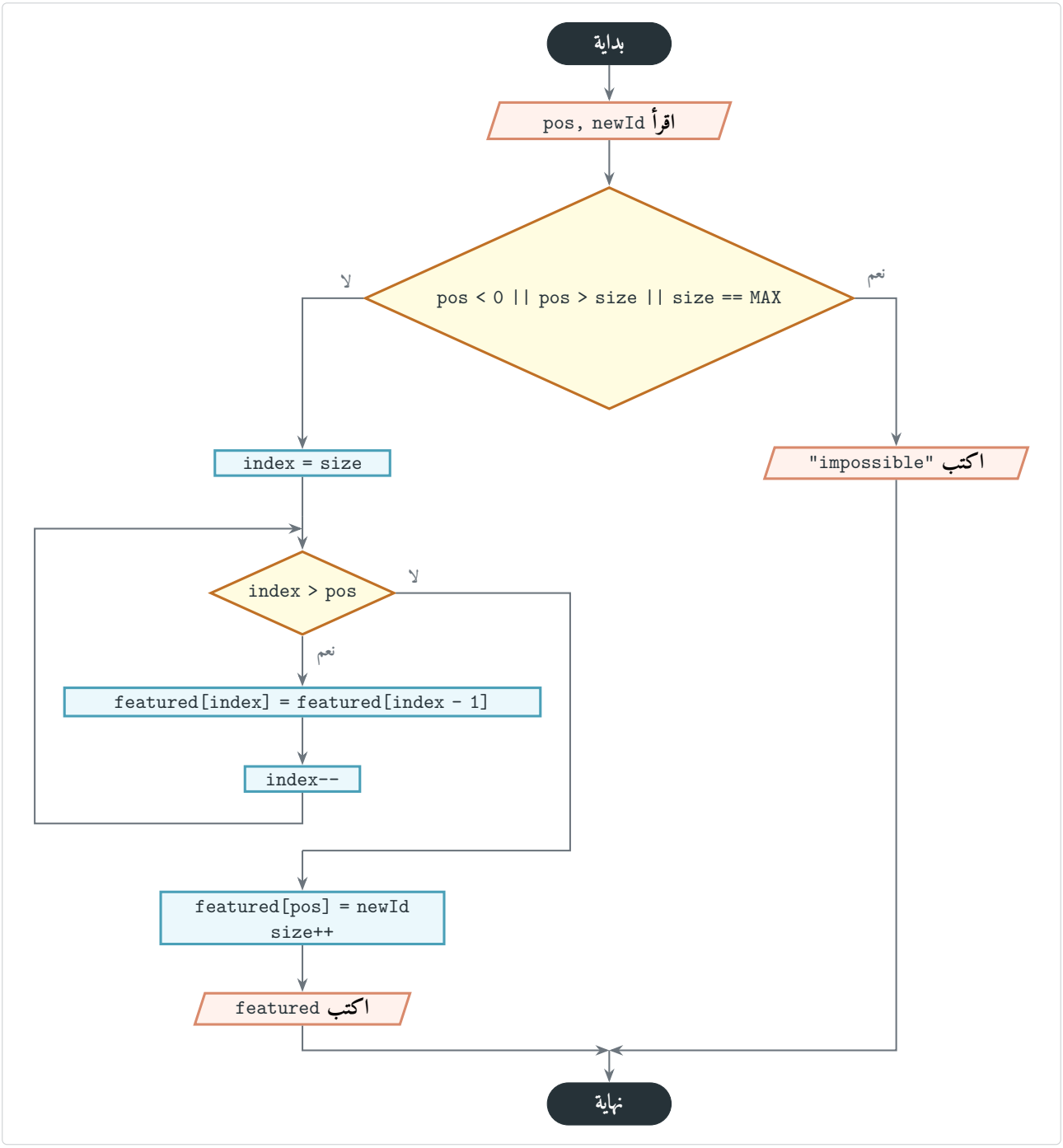
المدخلات

```
2 777
```

نتيجة التنفيذ

```
101 205 777 330 412 560
```

المخطط الإنسيابي (Organigramme)



تمرين 4.14 حذف منتج متوقف

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يحذف من الجدول المنتج ذي الرقم `removedId` (إن وُجد) بإزاحة العناصر الموالية نحو اليسار، ثم يعرض الجدول الجديد.

خطوات الحل

1) نبحث عن موضع `removedId`.

2) إذا وُجد في `pos`: `products[index] = products[index + 1]` لـ `index` من `pos` إلى `size - 2`، ثم `size--`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int products[10] = {101, 205, 330, 412, 560, 618};
    int size = 6, removedId, index, pos = -1;
    scanf("%d", &removedId);
    for (index = 0; index < size; index++)          // البحث عن الموضع
        if (products[index] == removedId) { pos = index; break; }
    if (pos == -1) {
        printf("%d not found\n", removedId);
    } else {
        for (index = pos; index < size - 1; index++)
            products[index] = products[index + 1];          // نسد الفراغ
        size--;                                 // الجدول فقد عنصراً //
        for (index = 0; index < size; index++)
            printf("%d ", products[index]);
        printf("\n");
    }
    return 0;
}
```

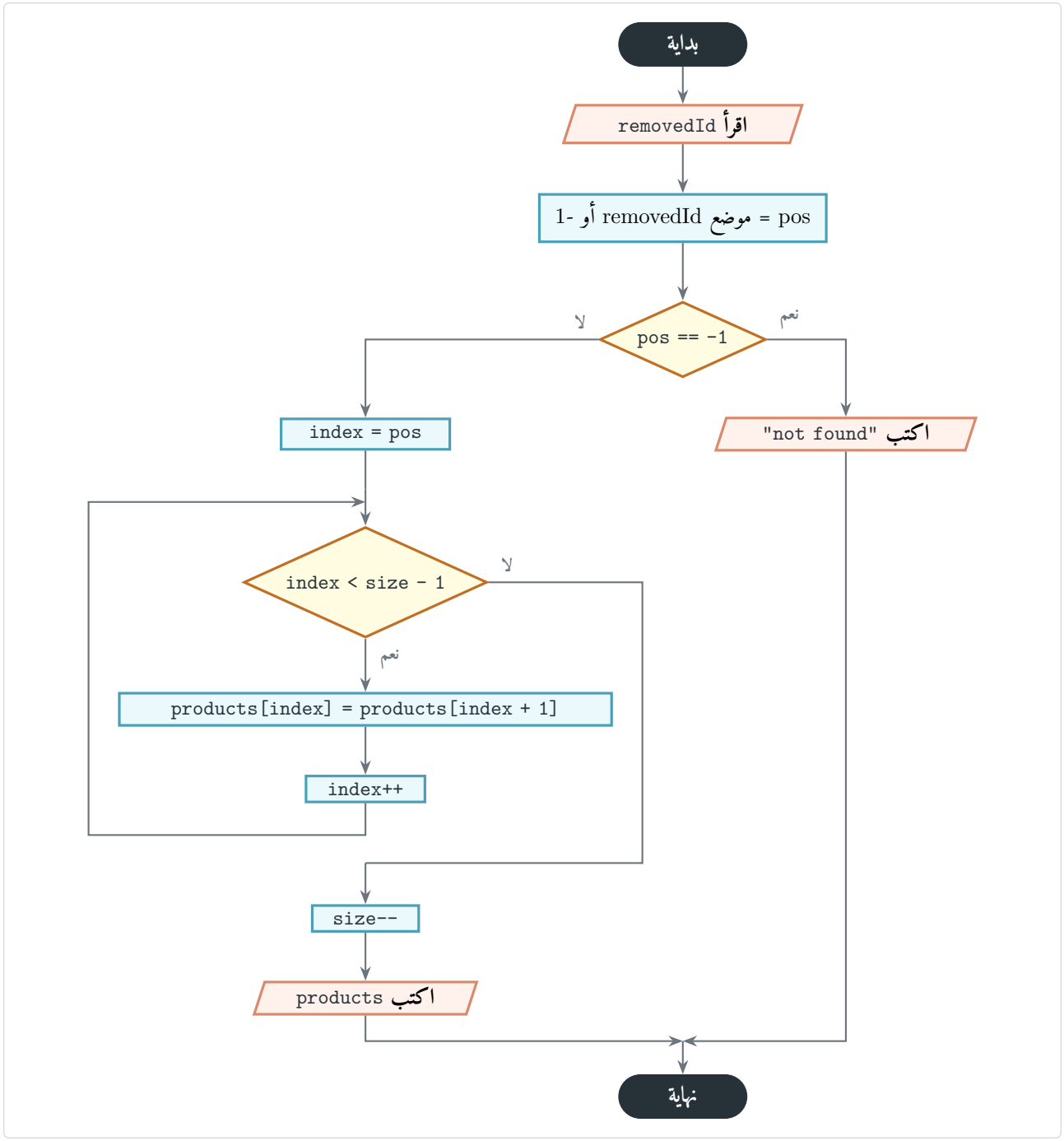
المدخلات

```
330
```

نتيجة التنفيذ

```
101 205 412 560 618
```

المخطط الإنسيابي (Organigramme)



تمرين 4.15 الترتيب بالفقاعات

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يرتب أسعار المنتجات **تصاعديًا** بخوارزمية **الفقاعات**، مع التوقف المبكر إذا لم يحدث أي تبديل في دورة كاملة.

خطوات الحل

1) في كل دورة نقارن كل عنصرين متجاورين ونبدلهما إذا كانا غير مرتبين.
2) بعد الدورة الأولى يصل الأكبر إلى الآخر.
3) `swapped = 0` في بداية الدورة، إذا بقي 0 فالجدول مرتب.

البرنامج بلغة C

```c
#include <stdio.h>

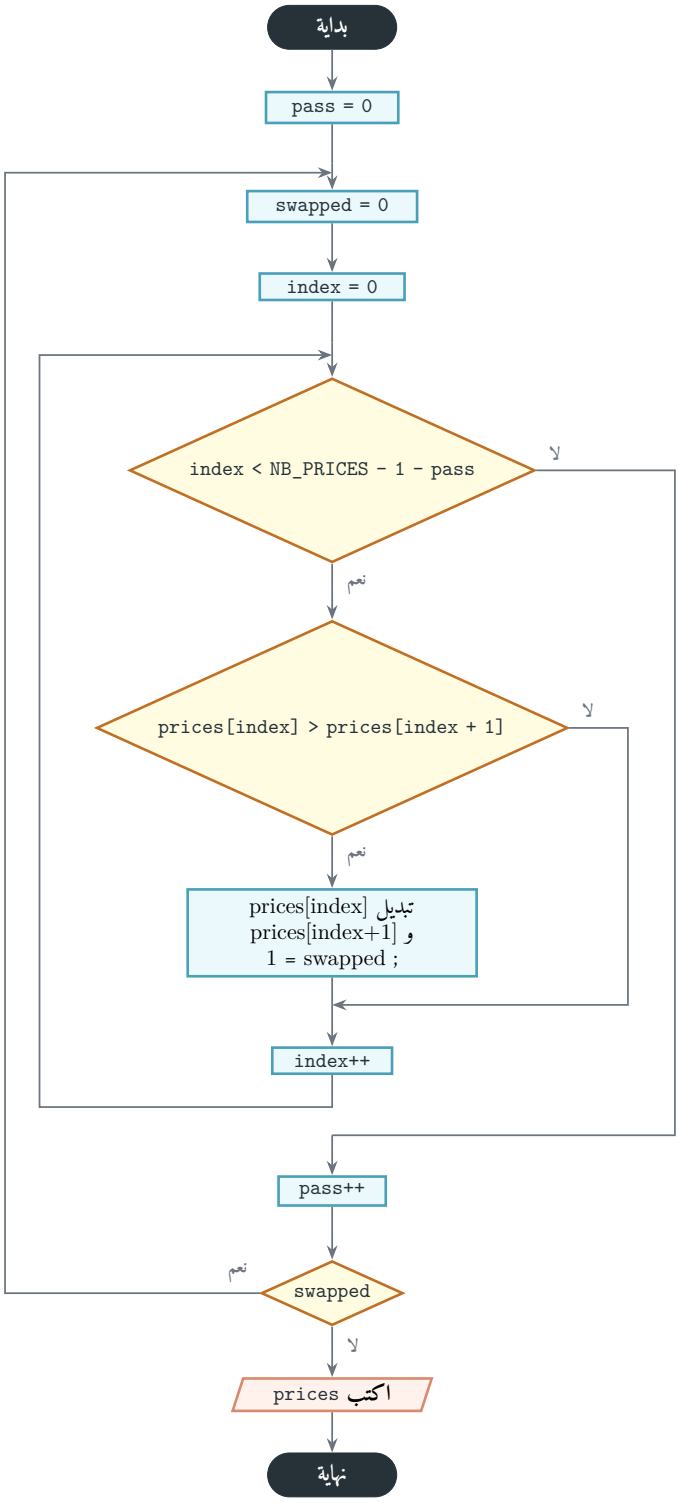
#define NB_PRICES 8

int main(void) {
    double prices[NB_PRICES] = {890, 120, 450, 3000, 75, 640, 1200, 300};
    double tmp;
    int index, pass = 0, swapped;
    do {
        swapped = 0;
        // آخر pass عناصر مرتبة سلفًا
        for (index = 0; index < NB_PRICES - 1 - pass; index++) {
            if (prices[index] > prices[index + 1]) {
                tmp = prices[index]; prices[index] = prices[index + 1]; prices[index + 1] = tmp;
                swapped = 1;
            }
        }
        pass++;
    } while (swapped);              // لا تبديل = الجدول مرتب
    printf("Sorted in %d passes:", pass);
    for (index = 0; index < NB_PRICES; index++)
        printf(" %.0f", prices[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Sorted in 6 passes: 75 120 300 450 640 890 1200 3000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.16 الترتيب بالإنتقاء

علم البيانات ★★ متوسط

اكتب برنامجًا يرتب عدد زيارات صفحات الموقع تنازليًا بخوارزمية الانتقاء: في كل خطوة نبحث عن أكبر عنصر في الجزء غير المرتب ونضعه في بدايته.

خطوات الحل

1) لكل index من 0 إلى NB_PAGES - 2: نبحث عن فهرس الأكبر maxPos بين index و NB_PAGES - 1.
2) نبدل visits[index] و visits[maxPos].
3) عدد التبديلات لا يتجاوز NB_PAGES - 1.

البرنامج بلغة C

```c
#include <stdio.h>

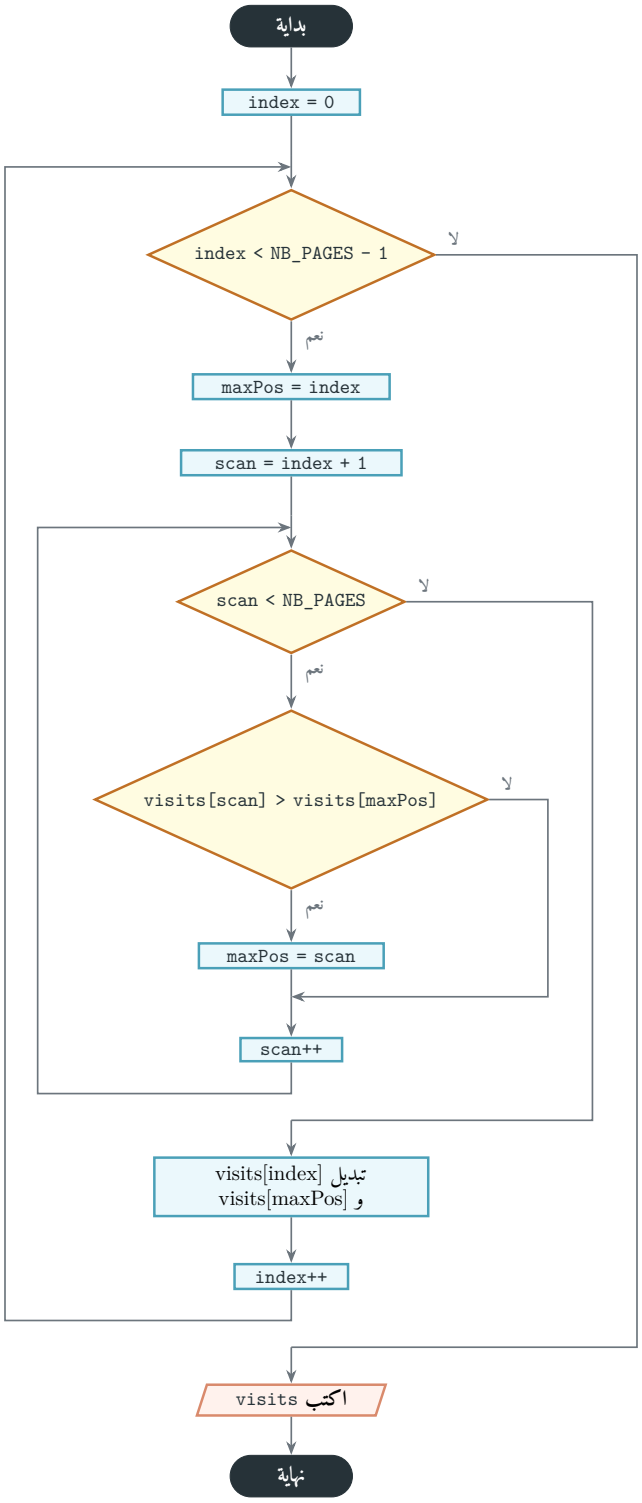
#define NB_PAGES 8

int main(void) {
    int visits[NB_PAGES] = {340, 1200, 85, 960, 1200, 410, 77, 2300};
    int index, scan, maxPos, tmp;
    for (index = 0; index < NB_PAGES - 1; index++) {
        maxPos = index;                    // فهرس الأكبر في visits[index..N-1]
        for (scan = index + 1; scan < NB_PAGES; scan++)
            if (visits[scan] > visits[maxPos])
                maxPos = scan;
        tmp = visits[index]; visits[index] = visits[maxPos]; visits[maxPos] = tmp;  // نضعه في الموضع index
    }
    printf("Pages by visits:");
    for (index = 0; index < NB_PAGES; index++)
        printf(" %d", visits[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Pages by visits: 2300 1200 1200 960 410 340 85 77
```

المخطط الإنسيابي (Organigramme)



تمرين 4.17 الترتيب بالإدراج

تجارة إلكترونية ★★ متوسط

تصل الطلبيات بأوقات غير مرتبة (بالدقائق). اكتب برنامجًا يرتبها بخوارزمية **الإدراج**: نأخذ كل عنصر وندرجه في مكانه في الجزء المرتب على يساره.

خطوات الحل

1) لكل index من 1: key = times[index].
2) نزيح العناصر الأكبر من key نحو اليمين: scan من index - 1 نزولًا.
3) نضع key في times[scan + 1].

البرنامج بلغة C

```c
#include <stdio.h>

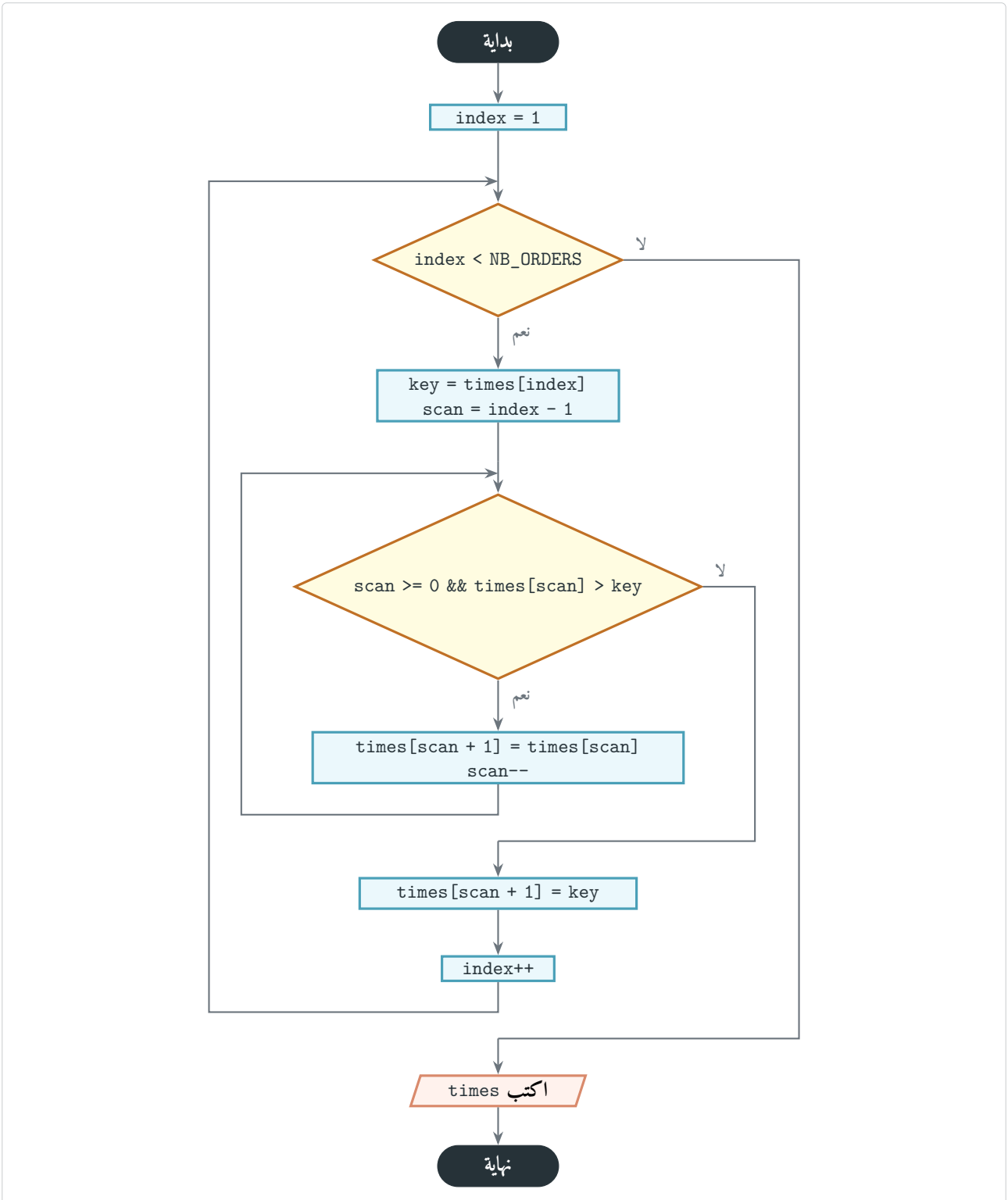
#define NB_ORDERS 8

int main(void) {
    int times[NB_ORDERS] = {615, 540, 720, 480, 600, 545, 900, 510};
    int index, scan, key;
    for (index = 1; index < NB_ORDERS; index++) {
        key = times[index];                 // العنصر المراد إدراجه
        scan = index - 1;
        while (scan >= 0 && times[scan] > key) {
            times[scan + 1] = times[scan];      // نزيح الأكبر نحو اليمين
            scan--;
        }
        times[scan + 1] = key;              // الفراغ الذي بقي هو مكانه
    }
    for (index = 0; index < NB_ORDERS; index++)
        printf("%02d:%02d ", times[index] / 60, times[index] % 60);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
08:00 08:30 09:00 09:05 10:00 10:15 12:00 15:00
```

المخطط الإنسيابي (Organigramme)



تمرين 4.18 البحث الثنائي في كتالوج مرتب

تجارة إلكترونية

متوسط

أرقام المنتجات في الكتالوج مرتبة تصاعديًا. اكتب برنامجًا يبحث عن رقم بحثًا ثنائيًا ويعرض عدد المقارنات.

خطوات الحل

1) `lo = 0`، `hi = NB_IDS - 1`.
2) `mid = (lo + hi) / 2`: إذا كان `catalog[mid] == target` وجدناه.
3) إذا كان `catalog[mid] < target` نبحث يميناً `lo = mid + 1`، وإلا يساراً `hi = mid - 1`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_IDS 12

int main(void) {
    int catalog[NB_IDS] = {1001, 1203, 2210, 3001, 3550, 4410, 5512,
                6600, 7781, 8120, 9045, 9900};      // مرتب
    int target, lo = 0, hi = NB_IDS - 1, mid, pos = -1, cmp = 0;
    scanf("%d", &target);
    while (lo <= hi && pos == -1) {
        mid = (lo + hi) / 2;
        cmp++;
        if (catalog[mid] == target)
            pos = mid;
        else if (catalog[mid] < target)
            lo = mid + 1;          // target في النصف الأيمن
        else
            hi = mid - 1;          // target في النصف الأيسر
    }
    if (pos >= 0)
        printf("Found at index %d after %d comparisons\n", pos, cmp);
    else
        printf("Not found (%d comparisons)\n", cmp);
    return 0;
}
```

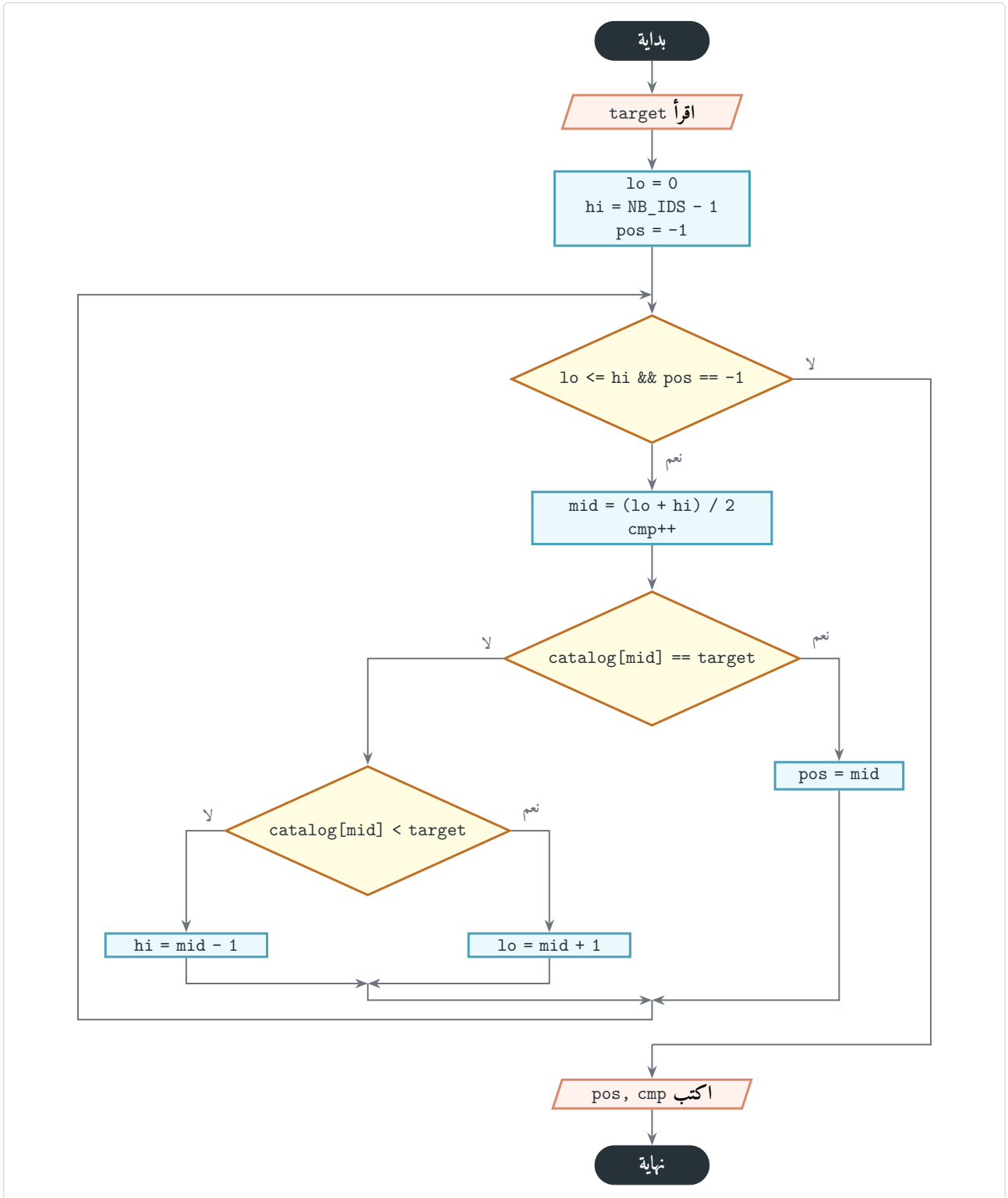
المدخلات

```
7781
```

نتيجة التنفيذ

```
Found at index 8 after 2 comparisons
```

المخطط الإنسيابي (Organigramme)



تمرين 4.19 وسيط أسعار السوق

علم البيانات ★★ متوسط

لتحديد سعر عادل لمنتج نجمع أسعار المنافسين ونأخذ **الوسيط**. اكتب برنامجًا يقرأ `nbPrices` أسعار، يرتبها، ثم يحسب الوسيط (العنصر الأوسط، أو متوسط العنصرين الأوسطين إذا كان `nbPrices` زوجيًا).

خطوات الحل

1) نقرأ الأسعار ونرتبها بالإدراج.
2) إذا كان `nbPrices` فرديًا: `prices[nbPrices / 2]`.
3) وإلا: `(prices[nbPrices/2 - 1] + prices[nbPrices/2]) / 2`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double prices[100], key, med;
    int nbPrices, index, scan;
    scanf("%d", &nbPrices);
    for (index = 0; index < nbPrices; index++)
        scanf("%lf", &prices[index]);
    for (index = 1; index < nbPrices; index++) {          // ترتيب بالإدراج
        key = prices[index];
        for (scan = index - 1; scan >= 0 && prices[scan] > key; scan--)
            prices[scan + 1] = prices[scan];
        prices[scan + 1] = key;
    }
    if (nbPrices % 2 == 1)
        med = prices[nbPrices / 2];                   // العنصر الأوسط
    else
        med = (prices[nbPrices / 2 - 1] + prices[nbPrices / 2]) / 2;
    printf("Median price = %.2f DA\n", med);
    return 0;
}
```

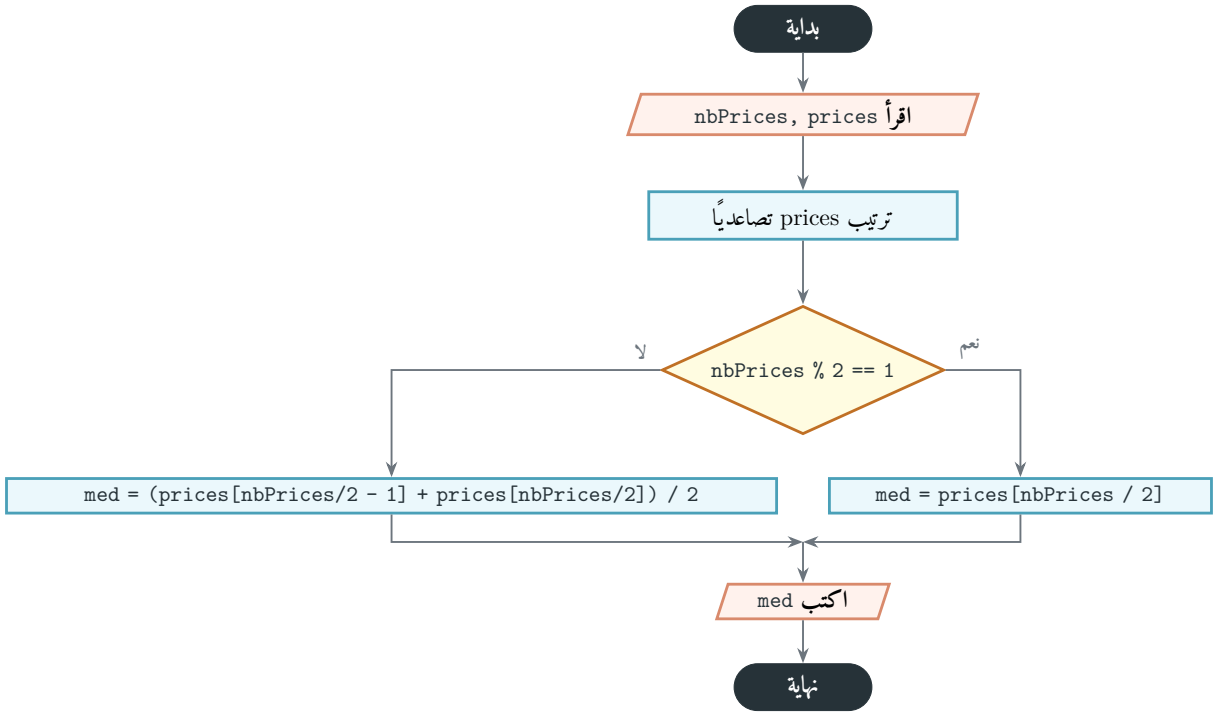
المدخلات

```
8
2500 2390 2700 9900 2450 2600 2550 1200
```

نتيجة التنفيذ

```
Median price = 2525.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.20 المنوال بجدول عدادات

علم البيانات ★★ متوسط

اكتب برنامجًا يجد **المنوال** (القيمة الأكثر تكرارًا) في جدول تقييمات من 1 إلى 5، باستعمال **جدول عدادات** `cnt[6]` حيث `cnt[value]` عدد مرات ظهور `value`.

خطوات الحل

1) نهيئ `cnt` بأصفار: `int cnt[6] = {0};`.
2) لكل تقييم `ratings`: `cnt[ratings]++` (القيمة نفسها فهرس).
3) المنوال هو الفهرس ذو أكبر عداد.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_REVIEWS 15

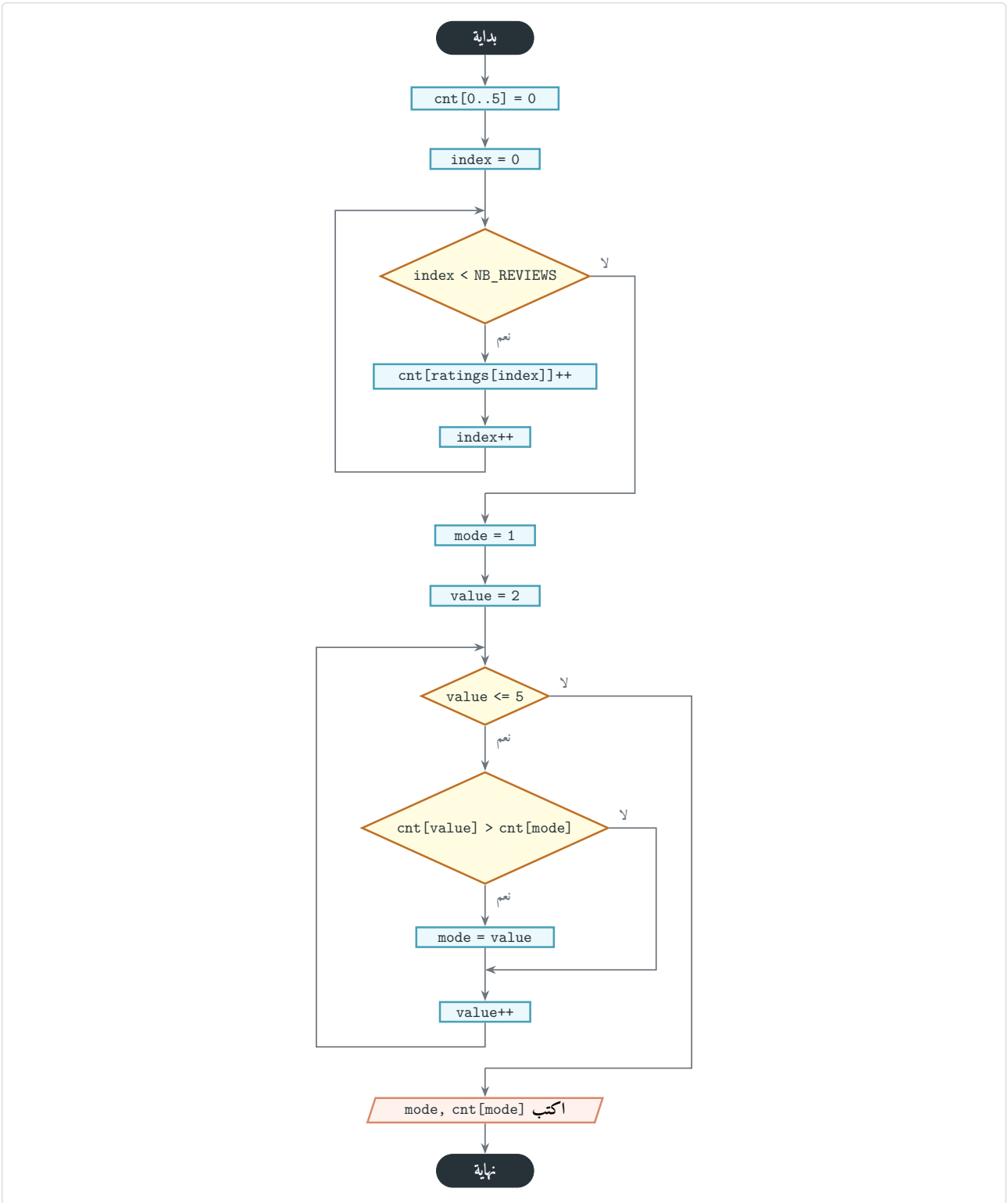
int main(void) {
    int ratings[NB_REVIEWS] = {4, 5, 3, 4, 4, 2, 5, 4, 1, 3, 4, 5, 5, 4, 3};
    int cnt[6] = {0};              // كل الخانات تبدأ من 0
    int index, value, mode = 1;
    for (index = 0; index < NB_REVIEWS; index++)
        cnt[ratings[index]]++;             // التقييم نفسه هو الفهرس
    for (value = 2; value <= 5; value++)
        if (cnt[value] > cnt[mode])
            mode = value;
    for (value = 1; value <= 5; value++)
        printf("%d stars: %d\n", value, cnt[value]);
```

```
16      printf("Mode = %d (%d times)\n", mode, cnt[mode]);
17      return 0;
18  }
```

نتيجة التنفيذ

```
1 stars: 1
2 stars: 1
3 stars: 3
4 stars: 6
5 stars: 4
Mode = 4 (6 times)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.21 توزيع الطلبيات حسب الساعة

علم البيانات ★★ متوسط

لدينا ساعات 20 طلبية (من 0 إلى 23). اكتب برنامجًا يحسب عدد الطلبيات في كل ساعة بجدول `cnt[24]`، ويعرض الساعات غير الفارغة مع شريط نجوم، ثم **ساعة الذروة**.

خطوات الحل

1) `cnt[hour]++` لكل طلبية.
2) نعرض فقط الساعات التي `cnt[hour] > 0`.
3) ساعة الذروة: فهرس أكبر عداد.

البرنامج بلغة C

```c
#include <stdio.h>

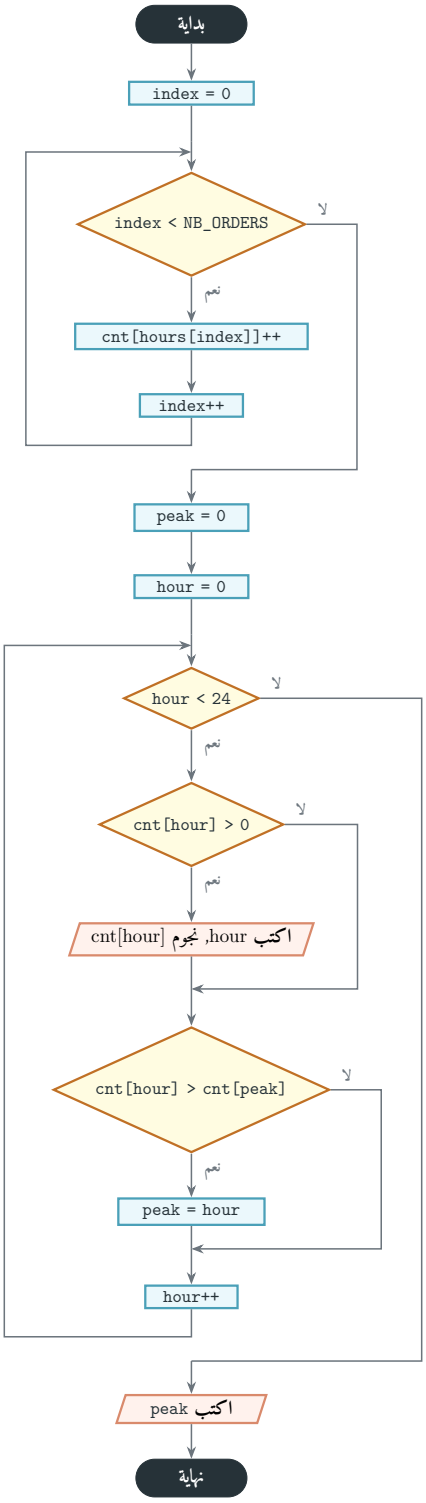
#define NB_ORDERS 20

int main(void) {
    int hours[NB_ORDERS] = {9, 12, 13, 20, 21, 12, 20, 22, 8, 13,
                    20, 21, 12, 19, 20, 23, 13, 20, 10, 21};
    int cnt[24] = {0};
    int index, hour, bar, peak = 0;
    for (index = 0; index < NB_ORDERS; index++)
        cnt[hours[index]]++;
    for (hour = 0; hour < 24; hour++) {
        if (cnt[hour] > 0) {
            printf("%02dh | ", hour);
            for (bar = 0; bar < cnt[hour]; bar++)
                printf("#");
            printf(" %d\n", cnt[hour]);
        }
        if (cnt[hour] > cnt[peak])
            peak = hour;
    }
    printf("Peak hour: %02dh\n", peak);
    return 0;
}
```

نتيجة التنفيذ

```
08h | # 1
09h | # 1
10h | # 1
12h | ### 3
13h | ### 3
19h | # 1
20h | ##### 5
21h | ### 3
22h | # 1
23h | # 1
Peak hour: 20h
```

المخطط الإنسيابي (Organigramme)



تمرين 4.22 حذف المكرر من جدول مرتب

علم البيانات ★★ متوسط

في سجل مرتب لأرقام الزبائن الذين زاروا الموقع تتكرر بعض الأرقام. اكتب برنامجًا يحذف **المكرر** في مكانه ويعرض عدد الزبائن المختلفين.

خطوات الحل

1) بما أن الجدول مرتب، المكرر متجاور.
2) `nbUnique = 1` (العنصر الأول محفوظ دائمًا).
3) لكل `index` من 1: إذا كان `visitors[index] != visitors[nbUnique - 1]` نكتب `visitors[nbUnique] = visitors[index]` و`nbUnique++`.

البرنامج بلغة C

```c
#include <stdio.h>

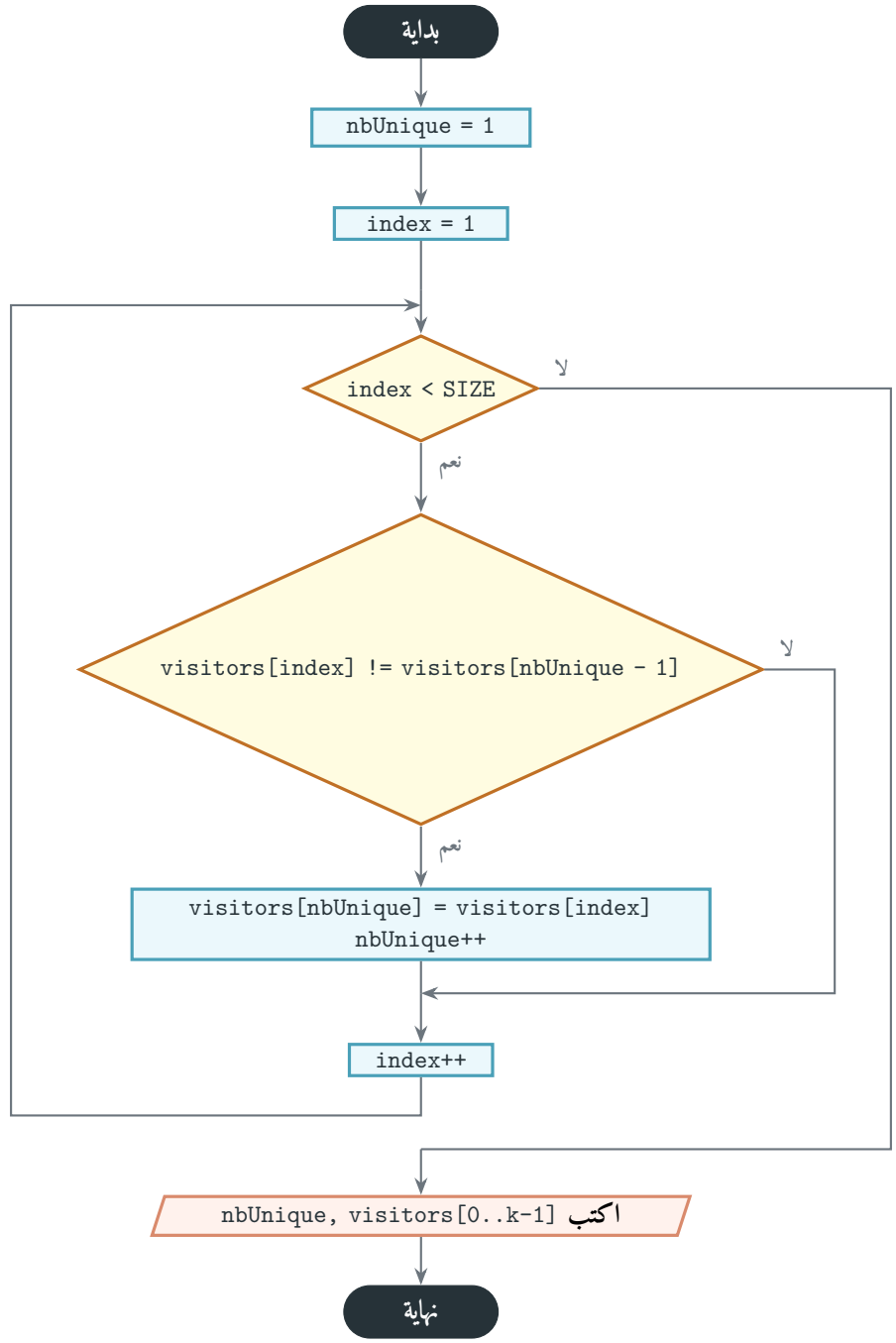
#define SIZE 12

int main(void) {
    int visitors[SIZE] = {101, 101, 105, 107, 107, 107, 110, 112, 112, 115, 115, 120};
    int index, nbUnique = 1;                // nbUnique: حجم الجزء دون تكرار
    for (index = 1; index < SIZE; index++)
        if (visitors[index] != visitors[nbUnique - 1])      // مختلف عن آخر عنصر محفوظ
            visitors[nbUnique++] = visitors[index];
    printf("%d unique customers:", nbUnique);
    for (index = 0; index < nbUnique; index++)
        printf(" %d", visitors[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
7 unique customers: 101 105 107 110 112 115 120
```

المخطط الإنسيابي (Organigramme)



تمرين 4.23 دمج قائمتي أسعار مرتبتين

تجارة إلكترونية ★★ متوسط

لدى مستودعين قائمتا أسعار **مرتبتان**. اكتب برنامجًا **يدمجهما** في قائمة واحدة مرتبة في مرور واحد (دون إعادة ترتيب).

خطوات الحل

1) مؤشران `posA` على `listA` و`posB` على `listB`.
2) نأخذ الأصغر بين `listA[posA]` و`listB[posB]` ونضعه في `merged[posC]`.
3) عند نفاد إحدى القائمتين ننسخ بقية الأخرى.

البرنامج بلغة C

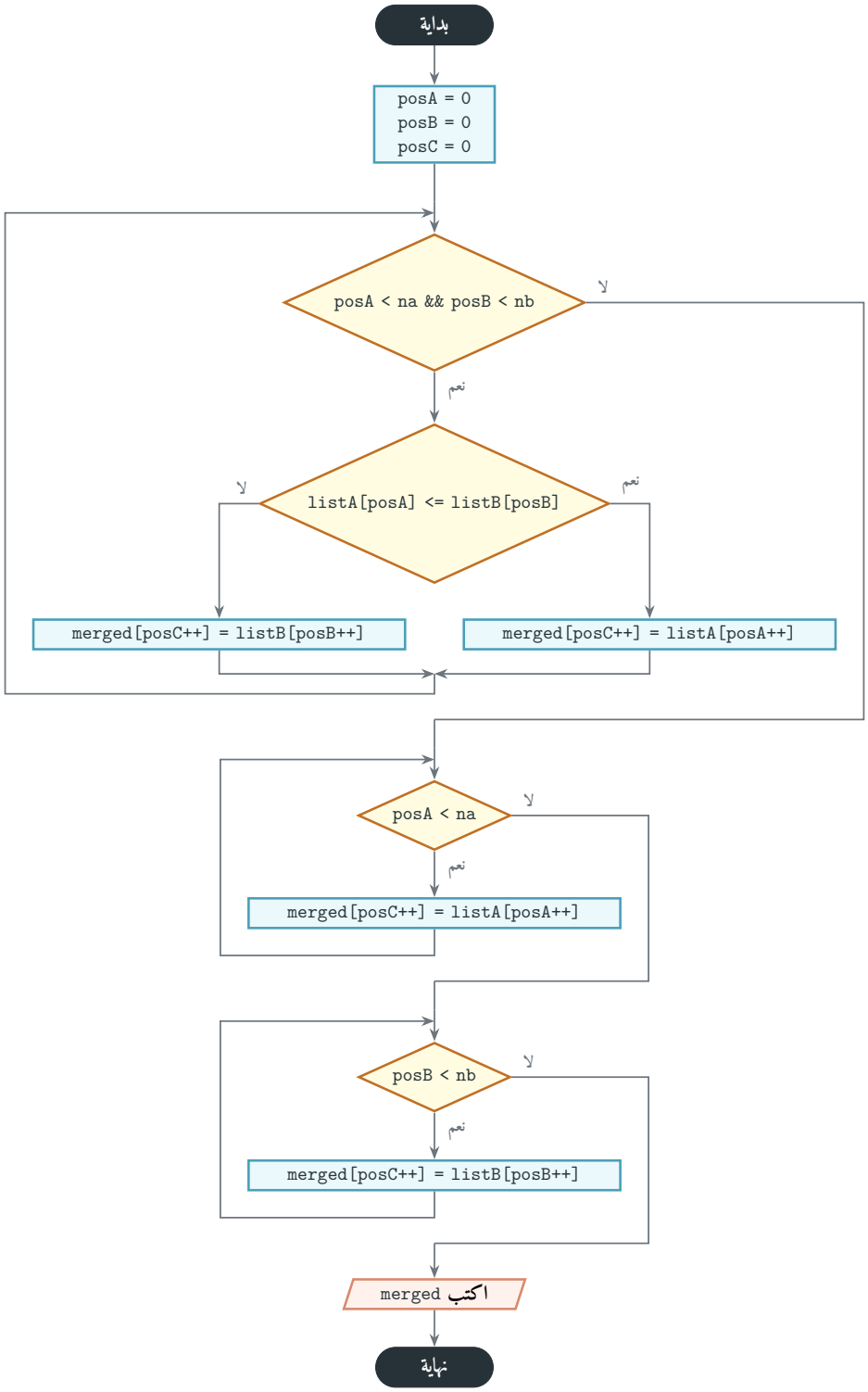
```c
#include <stdio.h>

int main(void) {
    int listA[5] = {120, 450, 800, 1500, 3000};
    int listB[4] = {99, 500, 820, 2500};
    int merged[9], na = 5, nb = 4, posA = 0, posB = 0, posC = 0;
    while (posA < na && posB < nb) {
        if (listA[posA] <= listB[posB])
            merged[posC++] = listA[posA++];      // الأصغر من listA
        else
            merged[posC++] = listB[posB++];      // الأصغر من listB
    }
    while (posA < na) merged[posC++] = listA[posA++];   // بقية listA
    while (posB < nb) merged[posC++] = listB[posB++];   // بقية listB
    for (posA = 0; posA < posC; posA++)
        printf("%d ", merged[posA]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
99 120 450 500 800 820 1500 2500 3000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.24 المجاميع التراكمية واستعلام الفترات

علم البيانات ★★ متوسط

تطلب لوحة التحكم مرارًا مجموع المبيعات بين يومين `from` و`to`. بدل جمع الفترة في كل مرة، نحسب مرة واحدة جدول **المجاميع التراكمية** `prefix[d] = sales[0] + ... + sales[d-1]`، فيصبح مجموع الفترة `prefix[to + 1] - prefix[from]`.
اكتب البرنامج.

خطوات الحل

1) `prefix[0] = 0` ثم `prefix[day + 1] = prefix[day] + sales[day]`.
2) لكل استعلام: `prefix[to + 1] - prefix[from]` في عملية واحدة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_DAYS 10

int main(void) {
    int sales[NB_DAYS] = {120, 95, 140, 180, 160, 210, 75, 130, 150, 90};
    int prefix[NB_DAYS + 1];          // خانة إضافية للمجموع الفارغ
    int day, query, from, to;
    prefix[0] = 0;
    for (day = 0; day < NB_DAYS; day++)
        prefix[day + 1] = prefix[day] + sales[day];   // تراكم
    for (query = 0; query < 2; query++) {
        scanf("%d %d", &from, &to);
        // مجموع الأيام from..to دون حلقة
        printf("Sales days %d..%d = %d\n", from, to, prefix[to + 1] - prefix[from]);
    }
    return 0;
}
```

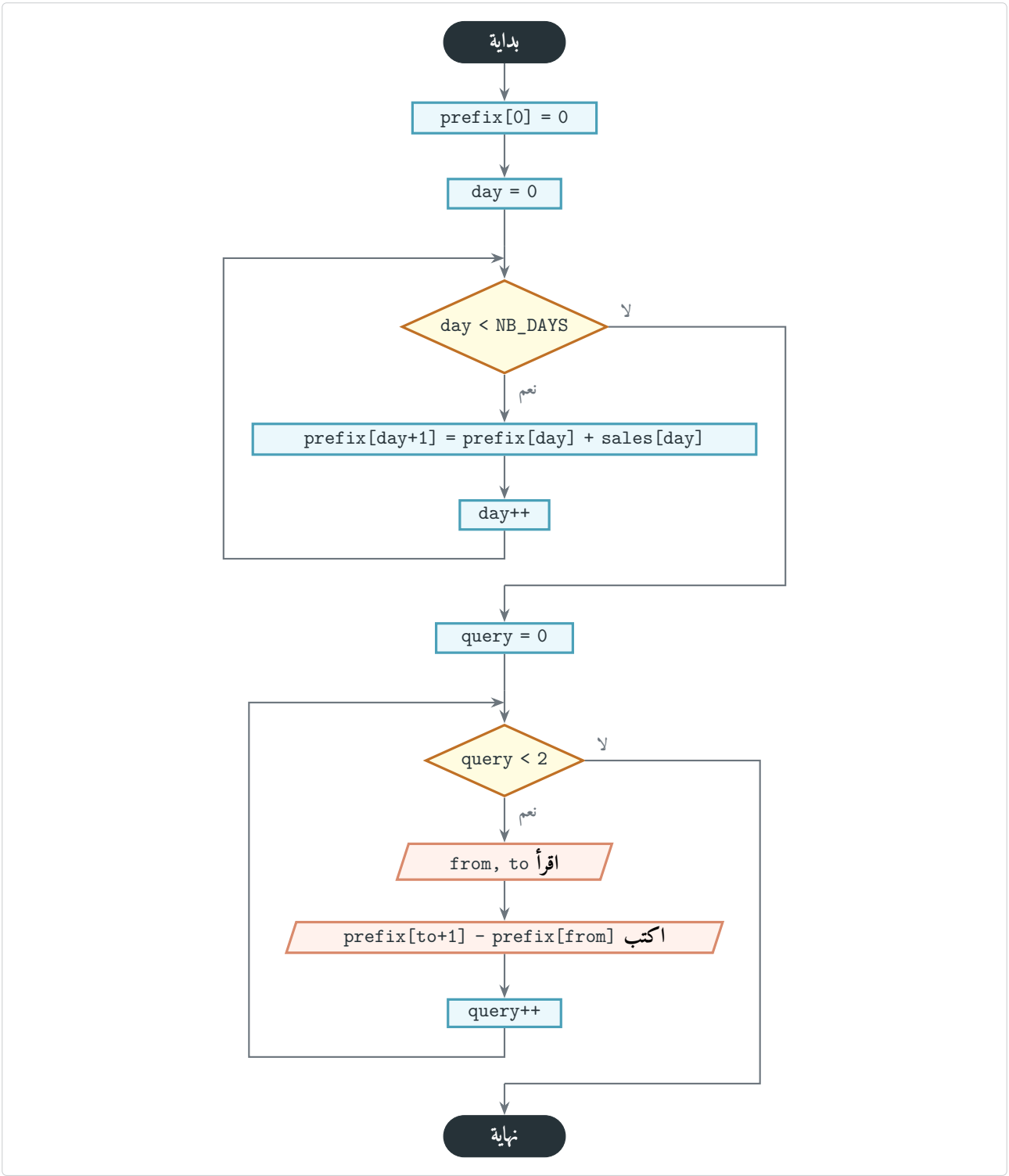
المدخلات

```
2 5
0 9
```

نتيجة التنفيذ

```
Sales days 2..5 = 690
Sales days 0..9 = 1350
```

المخطط الإنسيابي (Organigramme)



تمرين 4.25 المتوسط المتحرك بنافذة k

علم البيانات متوسط

اكتب برنامجًا يحسب **المتوسط المتحرك** لمبيعات يومية بنافذة طولها `window` يومًا. لتجنب إعادة الجمع، ننزلق بالنافذة: نضيف اليوم الجديد ونطرح اليوم الذي خرج منها.

خطوات الحل

1) نحسب مجموع أول `window` يومًا.
2) لكل يوم `day >= window`: `windowSum += sales[day] - sales[day - window]`.
3) المتوسط `windowSum / window`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_DAYS 10

int main(void) {
    double sales[NB_DAYS] = {100, 130, 115, 160, 140, 170, 150, 190, 210, 180};
    double windowSum = 0;
    int window, day;
    scanf("%d", &window);
    for (day = 0; day < window; day++)
        windowSum += sales[day];          // النافذة الأولى
    printf("Days 1-%d: %.2f\n", window, windowSum / window);
    for (day = window; day < NB_DAYS; day++) {
        // اليوم الجديد يدخل، واليوم الأقدم يخرج
        windowSum += sales[day] - sales[day - window];
        printf("Days %d-%d: %.2f\n", day - window + 2, day + 1, windowSum / window);
    }
    return 0;
}
```

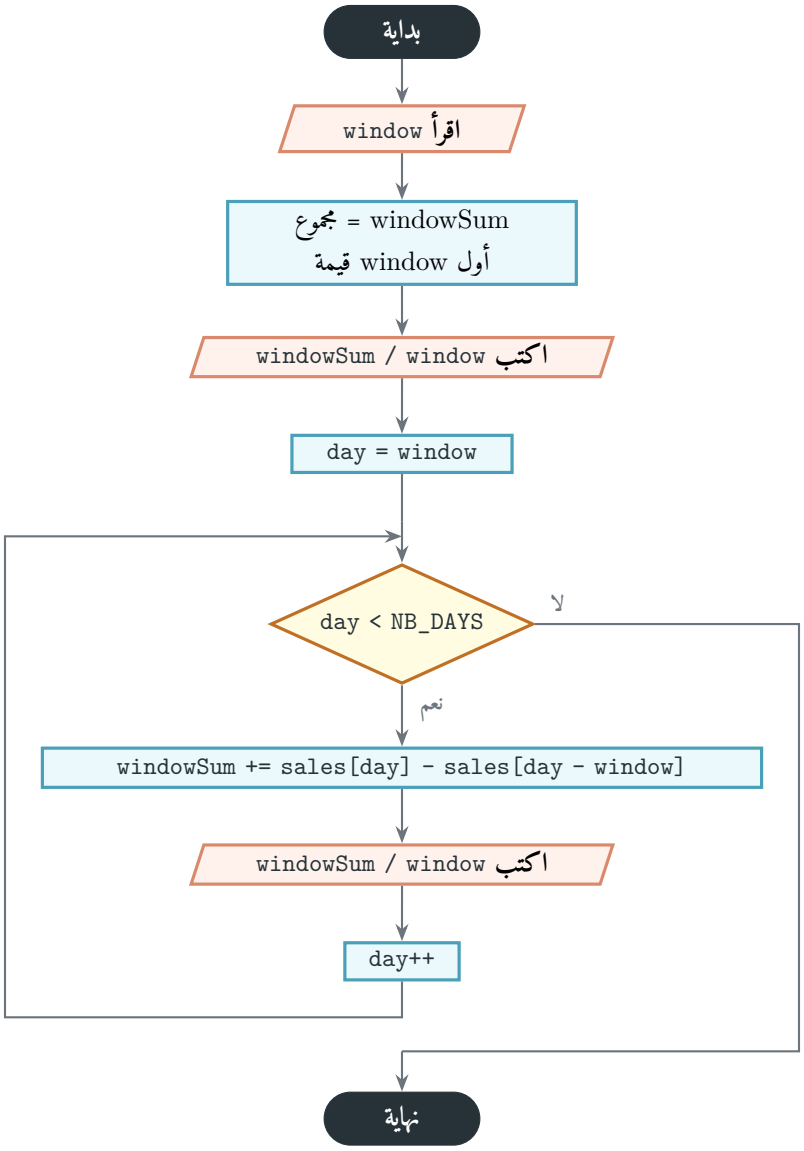
المدخلات

```
4
```

نتيجة التنفيذ

```
Days 1-4: 126.25
Days 2-5: 136.25
Days 3-6: 146.25
Days 4-7: 155.00
Days 5-8: 162.50
Days 6-9: 180.00
Days 7-10: 182.50
```

المخطط الإنسيابي (Organigramme)



تمرين 4.26 تطبيع جدول الأسعار

علم البيانات ★★ متوسط

اكتب برنامجًا يحول كل الأسعار في جدول إلى المجال `[0, 1]` بطريقة **Min-Max**، بعد إيجاد الأصغر والأكبر في مرور أول.

خطوات الحل

1) المرور الأول: `minPrice` و`maxPrice`.
2) المرور الثاني: `scaled[index] = (prices[index] - minPrice) / (maxPrice - minPrice)`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_PRICES 7

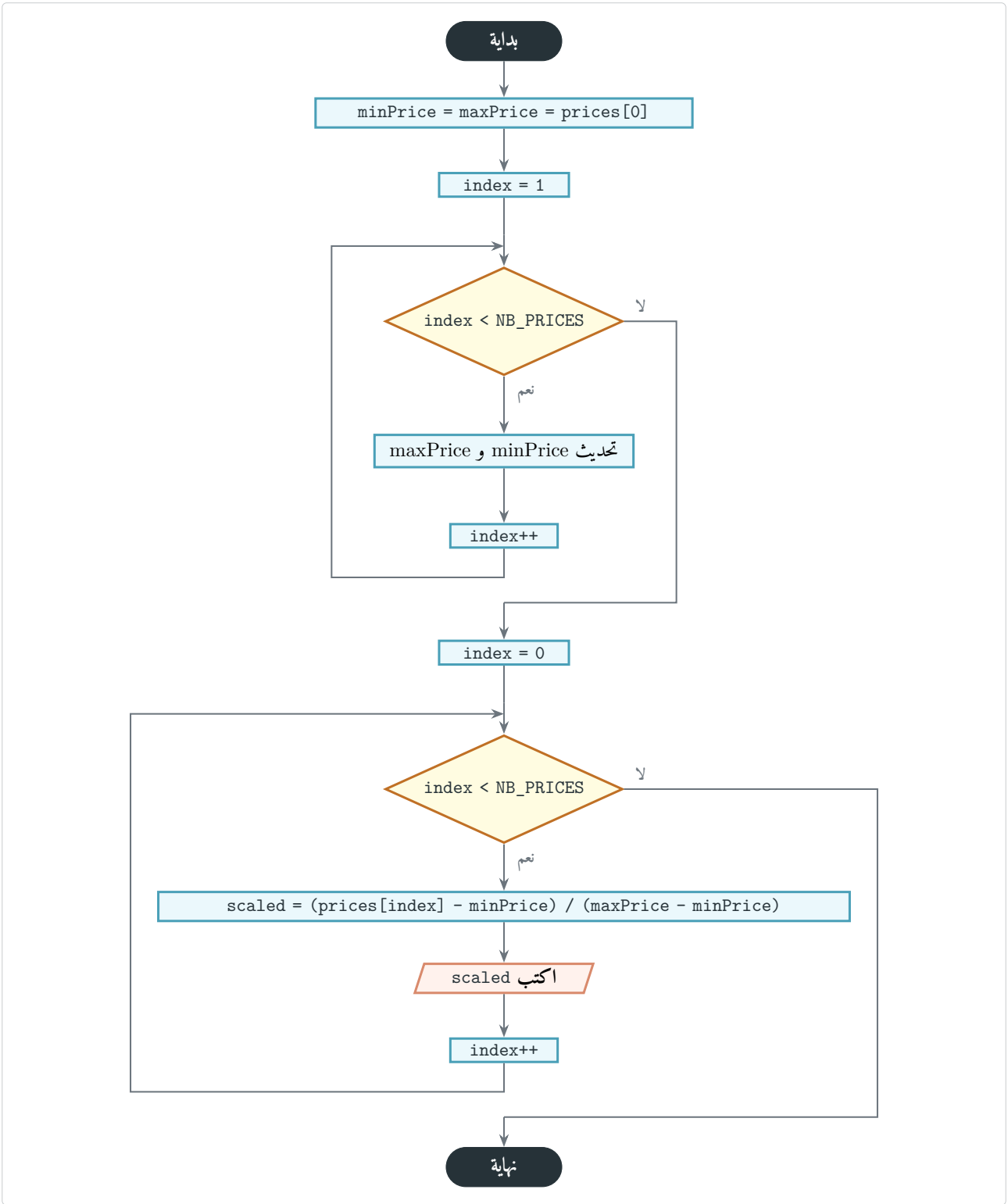
int main(void) {
```

```
    double prices[NB_PRICES] = {1200, 450, 3990, 89, 700, 2500, 1600};
    double minPrice = prices[0], maxPrice = prices[0];
    int index;
    for (index = 1; index < NB_PRICES; index++) {
        if (prices[index] < minPrice) minPrice = prices[index];
        if (prices[index] > maxPrice) maxPrice = prices[index];
    }
    for (index = 0; index < NB_PRICES; index++) {
        // 0 للأرخص و 1 للأغلى
        double scaled = (prices[index] - minPrice) / (maxPrice - minPrice);
        printf("%7.0f -> %.3f\n", prices[index], scaled);
    }
    return 0;
}
```

نتيجة التنفيذ

```
 1200 -> 0.285
  450 -> 0.093
 3990 -> 1.000
   89 -> 0.000
  700 -> 0.157
 2500 -> 0.618
 1600 -> 0.387
```

المخطط الإنسيابي (Organigramme)



تمرين 4.27 التوحيد المعياري للبيانات

علم البيانات ★★ متوسط

قبل تدريب نموذج نحول أعمار الزبائن إلى **درجات معيارية** `(x - mean) / std`. اكتب برنامجًا يحسب المتوسط والانحراف المعياري (الجذر بطريقة نيوتن) ثم الجدول المعياري، ويتحقق أن متوسطه ≈ 0.

خطوات الحل

1) المتوسط في مرور أول، ثم التباين في مرور ثانٍ.
2) الجذر بطريقة نيوتن.
3) `standard[index] = (ages[index] - mean) / std`.

البرنامج بلغة C

```c
#include <stdio.h>

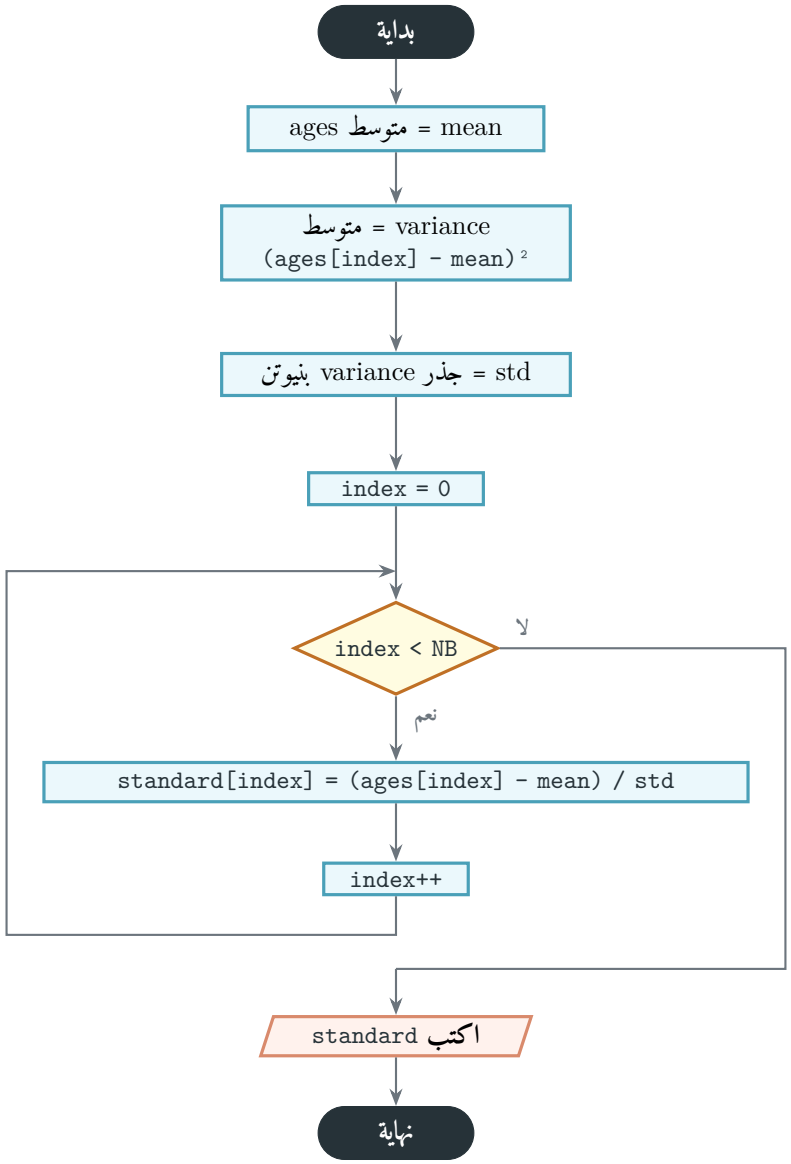
#define NB 8

int main(void) {
    double ages[NB] = {22, 35, 41, 28, 19, 55, 33, 47};
    double standard[NB], mean = 0, variance = 0, std, check = 0;
    int index, step;
    for (index = 0; index < NB; index++)
        mean += ages[index];
    mean /= NB;
    for (index = 0; index < NB; index++)
        variance += (ages[index] - mean) * (ages[index] - mean);
    variance /= NB;
    std = variance;                          // الجذر بطريقة نيوتن
    for (step = 0; step < 30; step++)
        std = (std + variance / std) / 2;
    for (index = 0; index < NB; index++) {
        standard[index] = (ages[index] - mean) / std;
        check += standard[index];
        printf("%4.0f -> %+.3f\n", ages[index], standard[index]);
    }
    printf("mean = %.2f, std = %.3f, new mean = %.6f\n", mean, std, check / NB);
    return 0;
}
```

نتيجة التنفيذ

```
  22 -> -1.130
  35 -> +0.000
  41 -> +0.522
  28 -> -0.609
  19 -> -1.391
  55 -> +1.739
  33 -> -0.174
  47 -> +1.043
mean = 35.00, std = 11.500, new mean = -0.000000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.28 المئين التسعون لزمن التوصيل

علم البيانات ★★ متوسط

تَعِد شركة التوصيل بأن 90% من الطلبيات تصل في أقل من مدة معينة. اكتب برنامجًا يرتب أزمنة التوصيل (بالساعات) ويحسب المئين 90 بطريقة الرتبة الأقرب: العنصر ذو الرتبة `ceil(0.9 × n)`.

خطوات الحل

1) نرتب الجدول (ترتيب بالإدراج).
2) الرتبة: `rank = (90 * n + 99) / 100` (تقريب للأعلى بأعداد صحيحة).
3) المئين هو `times[rank - 1]`.

البرنامج بلغة C

```c
#include <stdio.h>

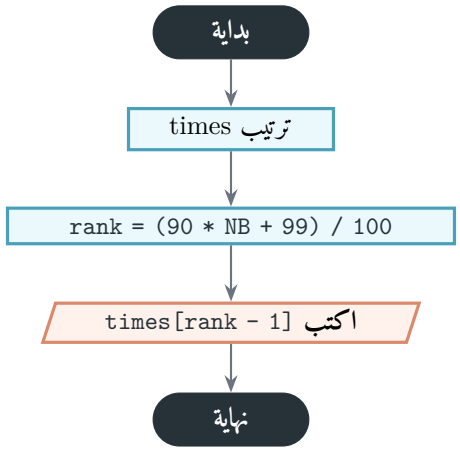
#define NB 15

int main(void) {
    int times[NB] = {24, 48, 30, 26, 72, 36, 28, 50, 25, 96, 30, 44, 27, 33, 60};
    int index, scan, key, rank;
    for (index = 1; index < NB; index++) {        // ترتيب بالإدراج
        key = times[index];
        for (scan = index - 1; scan >= 0 && times[scan] > key; scan--)
            times[scan + 1] = times[scan];
        times[scan + 1] = key;
    }
    rank = (90 * NB + 99) / 100;      // ceil(n × 0.9) بأعداد صحيحة
    printf("Sorted:");
    for (index = 0; index < NB; index++)
        printf(" %d", times[index]);
    printf("\nP90 = %d hours (rank %d of %d)\n", times[rank - 1], rank, NB);
    return 0;
}
```

نتيجة التنفيذ

```
Sorted: 24 25 26 27 28 30 30 33 36 44 48 50 60 72 96
P90 = 72 hours (rank 14 of 15)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.29 — منتجان بقيمة بطاقة الهدية

تجارة إلكترونية ★★ متوسط

يملك زبون **بطاقة هدية** بمبلغ `budget` ويريد شراء **منتجين** مجموع سعريهما يساوي المبلغ بالضبط. الأسعار مرتبة. اكتب برنامجًا يجد زوجًا بطريقة **المؤشرين** في مرور واحد.

خطوات الحل

1) `left = 0` و`right = NB - 1`.
2) إذا كان المجموع أصغر من المطلوب نزيد `left`، وإذا كان أكبر ننقص `right`.
3) نتوقف عند التساوي أو عندما يلتقي المؤشران.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 9

int main(void) {
    int prices[NB] = {450, 900, 1200, 1800, 2100, 2600, 3200, 3800, 4500};
    int budget, left = 0, right = NB - 1, sum, found = 0;
    scanf("%d", &budget);
    while (left < right && !found) {
        sum = prices[left] + prices[right];
        if (sum == budget)
            found = 1;
        else if (sum < budget)
            left++;                 // نحتاج مجموعًا أكبر
        else
            right--;                // نحتاج مجموعًا أصغر
    }
    if (found)
        printf("%d + %d = %d DA\n", prices[left], prices[right], budget);
    else
        printf("No pair matches %d DA\n", budget);
    return 0;
}
```

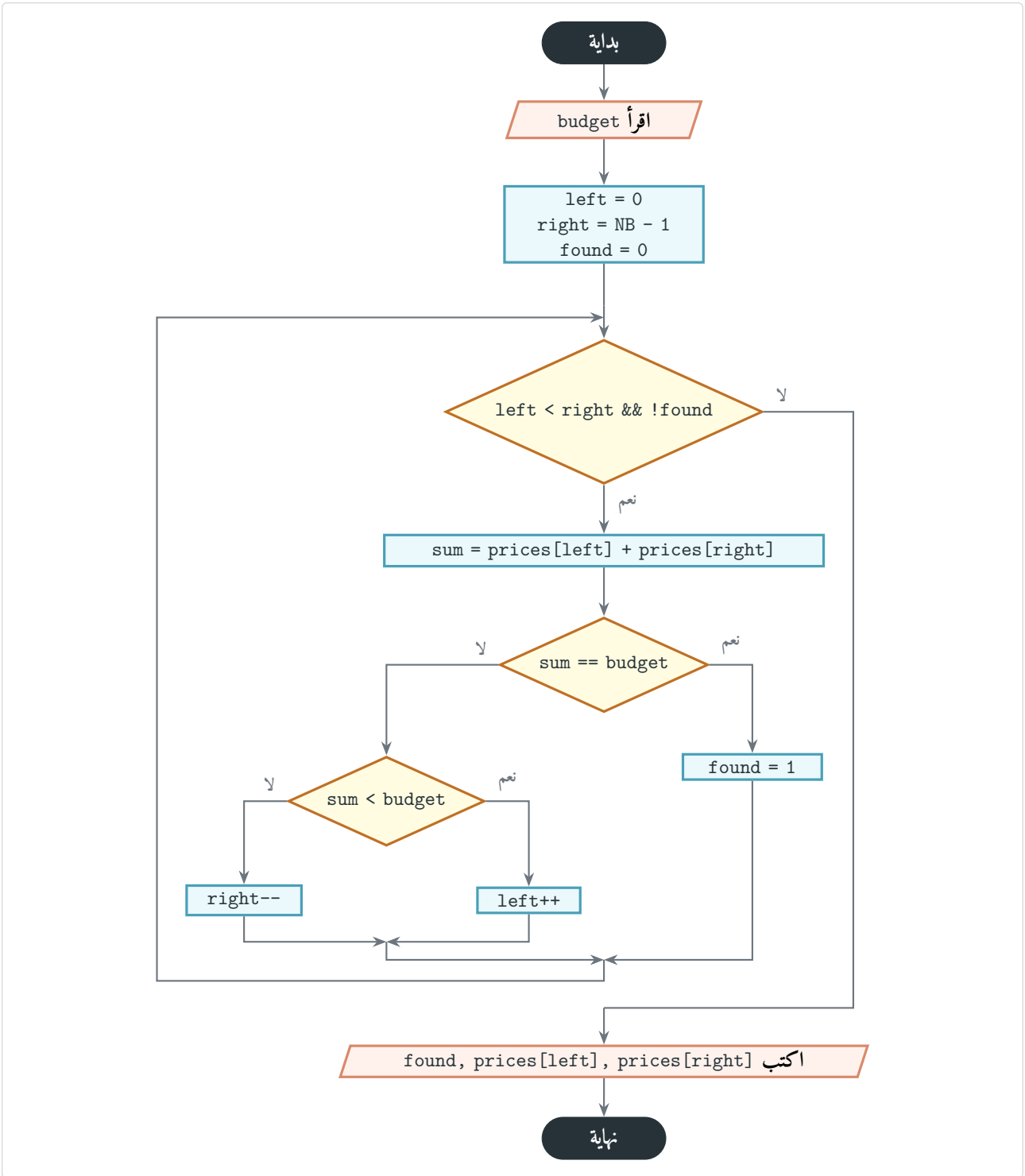
المدخلات

```
5000
```

نتيجة التنفيذ

```
1200 + 3800 = 5000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.30 التشابه بين زبونين

علم البيانات ★★ متوسط

نمثل كل زبون بجدول عدد مشترياته في 6 أصناف. اكتب برنامجًا يحسب **تشابه جيب التمام** `cos = (u·v) / (|u| |v|)` بين زبونين (الجذر بطريقة نيوتن). قيمة قريبة من 1 تعني ذوقًا متشابهًا.

خطوات الحل

1) الجداء السلمي: `dot = Σ first[k] * second[k]`.
2) المعيار: `normFirst = Σ first[k]²` ونفس الشيء للثاني.
3) `cosine = dot / (normFirst * normSecond)`.

البرنامج بلغة C

```c
#include <stdio.h>

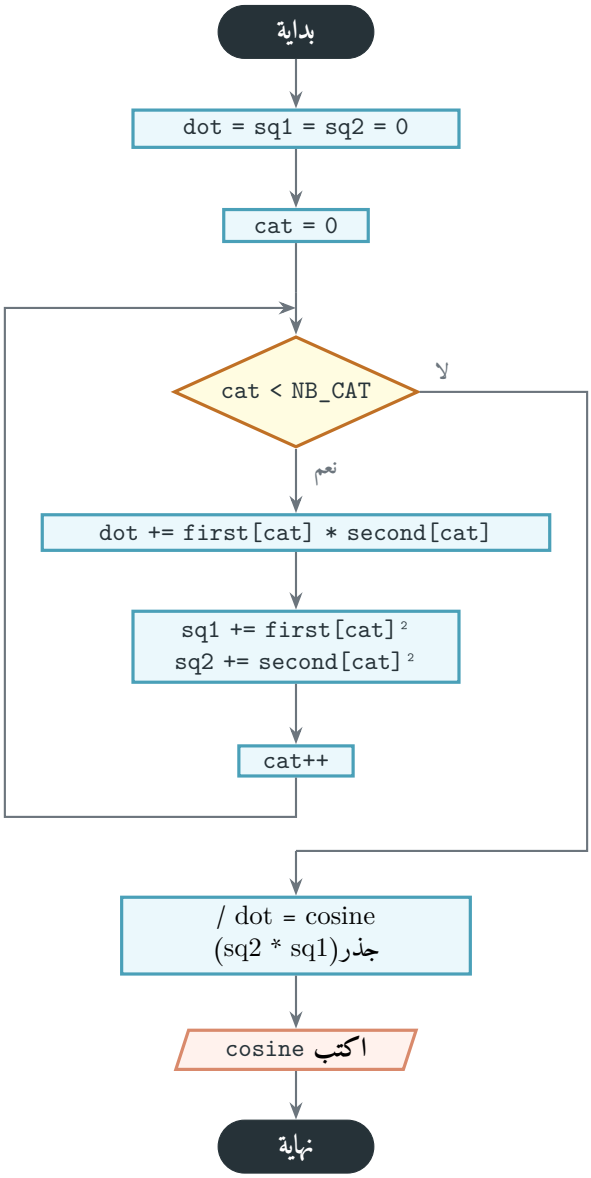
#define NB_CAT 6

int main(void) {
    double first[NB_CAT]  = {5, 0, 2, 8, 1, 0};   // مشتريات الزبون الأول
    double second[NB_CAT] = {4, 1, 3, 6, 0, 0};   // الزبون الثاني
    double dot = 0, sq1 = 0, sq2 = 0, product, root;
    int cat, step;
    for (cat = 0; cat < NB_CAT; cat++) {
        dot += first[cat] * second[cat];      // الجداء السلمي
        sq1 += first[cat] * first[cat];
        sq2 += second[cat] * second[cat];
    }
    product = sq1 * sq2;
    root = product;                           // جذر product بطريقة نيوتن
    for (step = 0; step < 40; step++)
        root = (root + product / root) / 2;
    printf("Dot = %.0f, norms product = %.4f\n", dot, root);
    printf("Cosine similarity = %.4f\n", dot / root);
    return 0;
}
```

نتيجة التنفيذ

```
Dot = 74, norms product = 76.3413
Cosine similarity = 0.9693
```

المخطط الإنسيابي (Organigramme)



تمرين 4.31 أكبر هبوط يومي

علم البيانات ★★ متوسط

اكتب برنامجًا يبني جدول **التغيرات اليومية** `diff[day] = sales[day + 1] - sales[day]` ثم يجد **أكبر هبوط** (أصغر قيمة سالبة) واليوم الذي حدث فيه.

خطوات الحل

1) جدول التغيرات أقصر بخانة واحدة.
2) نبحث عن أصغر عنصر في `diff`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 9
```

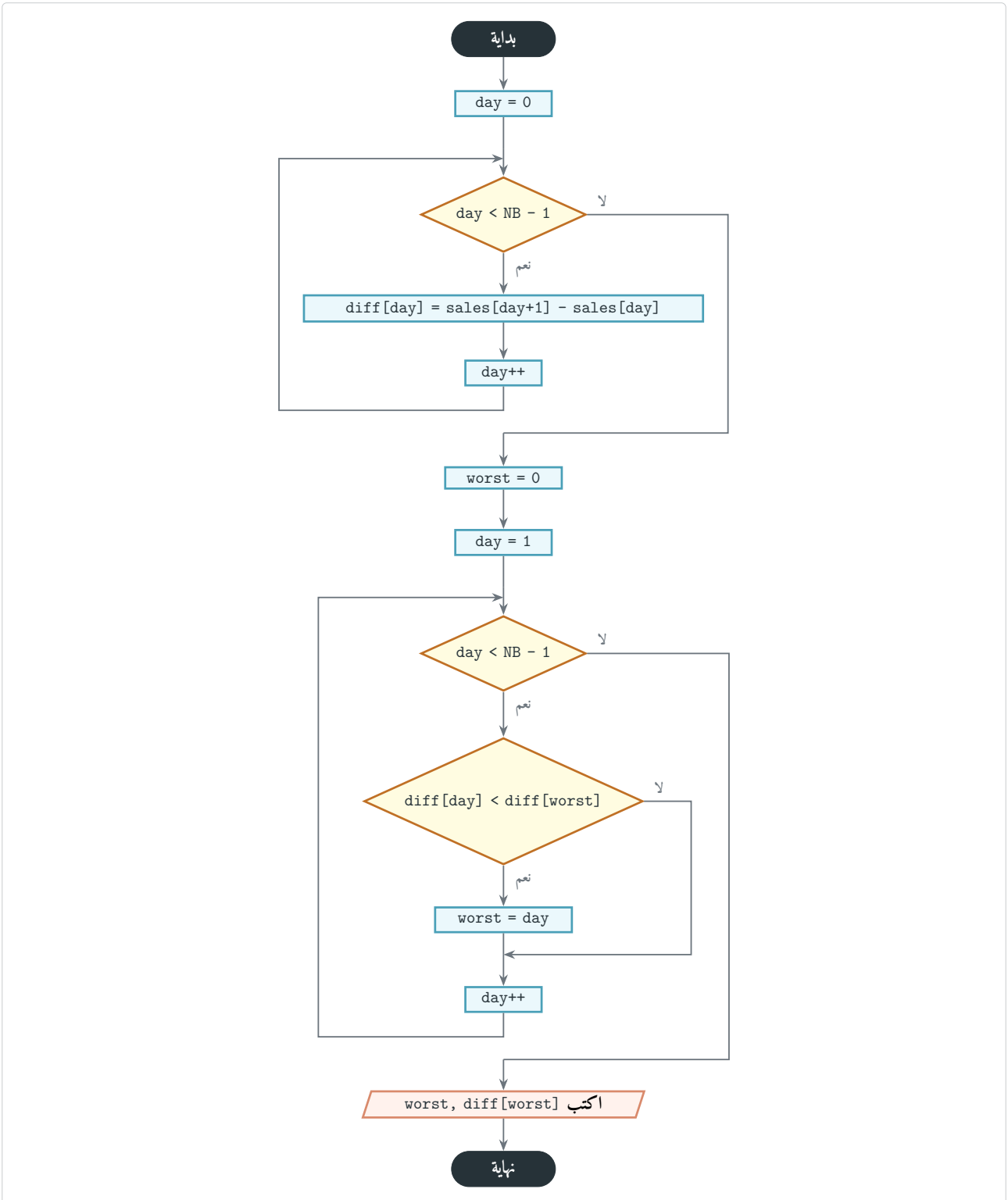
```c

int main(void) {
    int sales[NB] = {200, 230, 210, 260, 180, 190, 240, 150, 170};
    int diff[NB - 1], day, worst = 0;
    for (day = 0; day < NB - 1; day++) {
        diff[day] = sales[day + 1] - sales[day];  // تغير من يوم لآخر
        printf("%+d ", diff[day]);
    }
    for (day = 1; day < NB - 1; day++)
        if (diff[day] < diff[worst])
            worst = day;
    printf("\nBiggest drop: %d (from day %d to day %d)\n",
           diff[worst], worst + 1, worst + 2);
    return 0;
}
```

نتيجة التنفيذ

```
+30 -20 +50 -80 +10 +50 -90 +20
Biggest drop: -90 (from day 7 to day 8)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.32 القيم الشاذة بطريقة المدى الربيعي

علم البيانات ★★ متوسط

اكتب برنامجًا يكشف المبالغ **الشاذة** بطريقة **IQR**: نرتب البيانات، نأخذ الربيع الأول `Q1` (الرتبة $n/4$) والثالث `Q3` (الرتبة $3n/4$)، ثم كل قيمة خارج `[Q1 - 1.5 IQR, Q3 + 1.5 IQR]` شاذة.

خطوات الحل

1) ترتيب الجدول.
2) `q1 = values[NB / 4]` و`q3 = values[3 * NB / 4]` (تقريب بسيط).
3) `iqr = q3 - q1`، الحدان `low` و`high`.
4) نعرض القيم خارج الحدين.

البرنامج بلغة C

```c
#include <stdio.h>

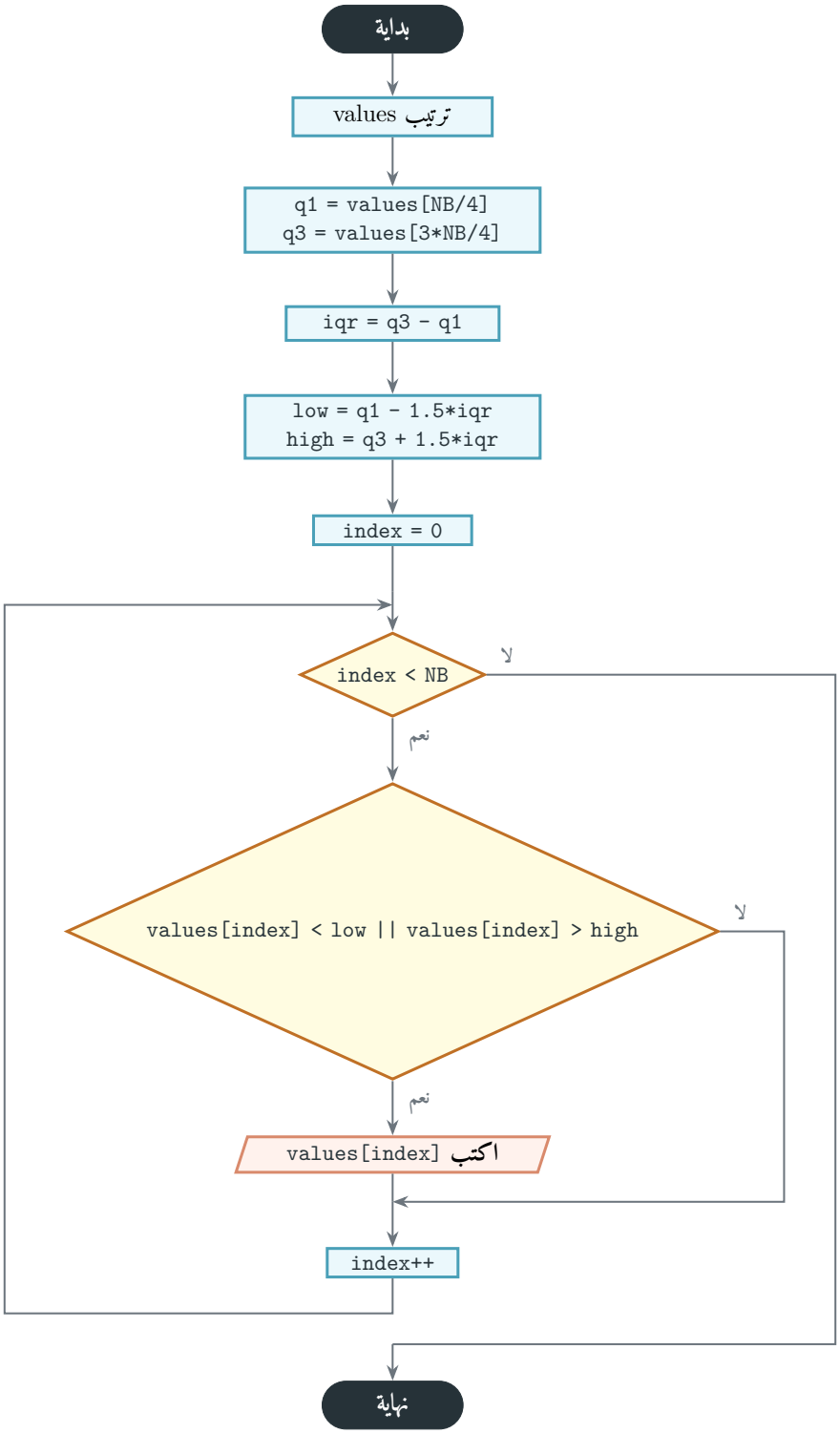
#define NB 12

int main(void) {
    double values[NB] = {3200, 2900, 3500, 41000, 3100, 2700,
                         3300, 150, 3600, 2800, 3400, 3000};
    double key, q1, q3, iqr, low, high;
    int index, scan;
    for (index = 1; index < NB; index++) {        // ترتيب بالإدراج
        key = values[index];
        for (scan = index - 1; scan >= 0 && values[scan] > key; scan--)
            values[scan + 1] = values[scan];
        values[scan + 1] = key;
    }
    q1 = values[NB / 4];                  // الربيع الأول
    q3 = values[3 * NB / 4];              // الربيع الثالث
    iqr = q3 - q1;
    low = q1 - 1.5 * iqr;
    high = q3 + 1.5 * iqr;
    printf("Q1 = %.0f, Q3 = %.0f, bounds = [%.0f, %.0f]\n", q1, q3, low, high);
    for (index = 0; index < NB; index++)
        if (values[index] < low || values[index] > high)
            printf("Outlier: %.0f\n", values[index]);
    return 0;
}
```

نتيجة التنفيذ

```
Q1 = 2900, Q3 = 3500, bounds = [2000, 4400]
Outlier: 150
Outlier: 41000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.33 نقل المنتجات النافدة إلى الآخر

تجارة إلكترونية ★★ متوسط

في صفحة النتائج نريد عرض المنتجات **المتوفرة أولًا** ثم النافدة (مخزون 0)، مع **الحفاظ على الترتيب الأصلي** داخل كل مجموعة. اكتب برنامجًا يعيد ترتيب جدولي الأرقام والمخزون في مكانهما.

خطوات الحل

1) مؤشر الكتابة `write = 0`.
2) نمر على الجدول: كل منتج متوفر ننقله إلى `write` ثم `write++`.
3) نحتاج جدولًا مؤقتًا للنافدة لنضعها بعد ذلك بنفس ترتيبها.

البرنامج بلغة C

```c
#include <stdio.h>

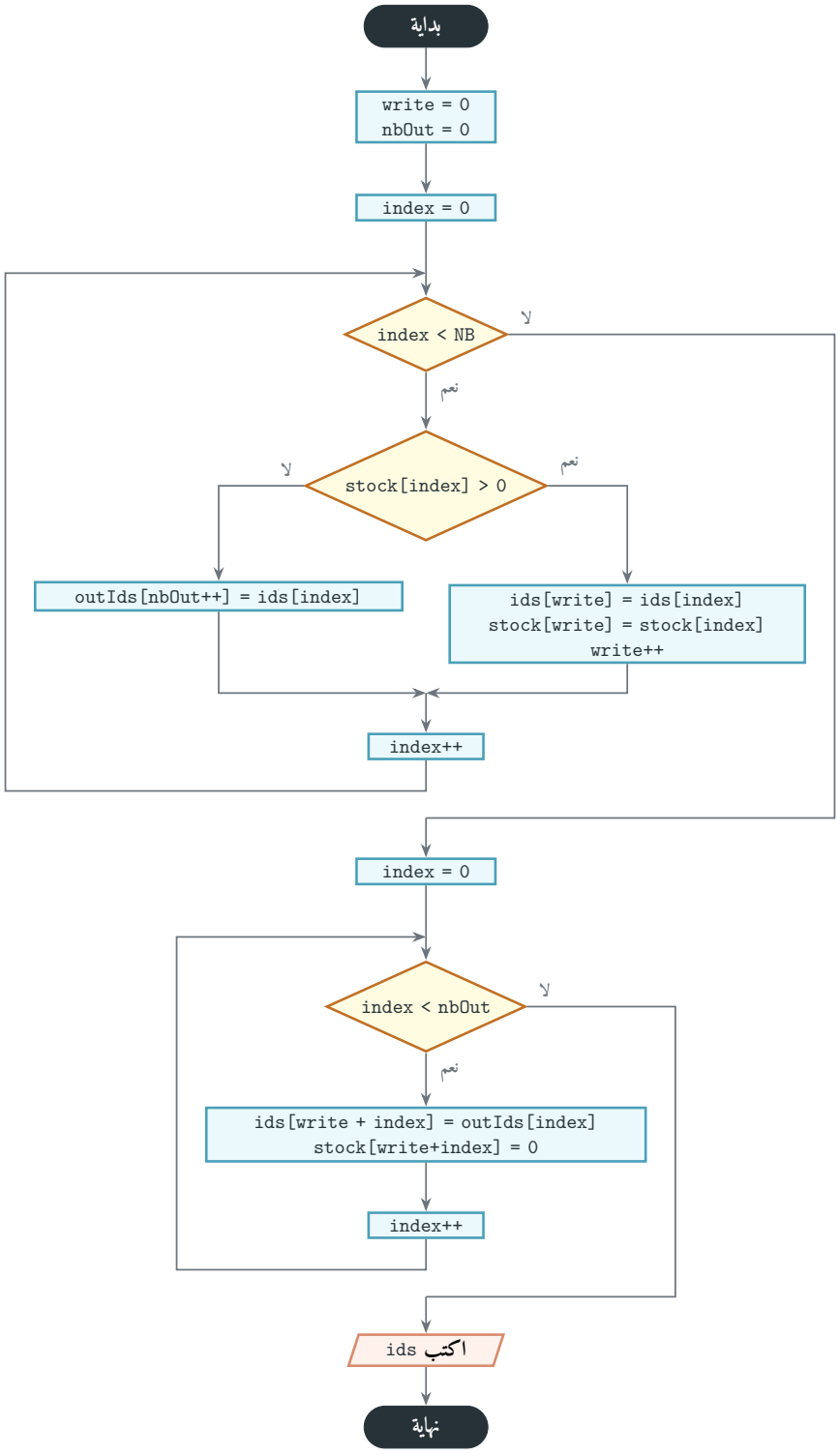
#define NB 8

int main(void) {
    int ids[NB]   = {11, 12, 13, 14, 15, 16, 17, 18};
    int stock[NB] = { 4,  0,  9,  0,  2,  7,  0,  1};
    int outIds[NB], write = 0, nbOut = 0, index;
    for (index = 0; index < NB; index++) {
        if (stock[index] > 0) {
            ids[write] = ids[index];      // دائمًا: لا نمسح شيئًا index => write
            stock[write] = stock[index];
            write++;
        } else {
            outIds[nbOut++] = ids[index]; // نحفظ النافدة جانبًا
        }
    }
    for (index = 0; index < nbOut; index++) {
        ids[write + index] = outIds[index];
        stock[write + index] = 0;
    }
    for (index = 0; index < NB; index++)
        printf("#%d(%d) ", ids[index], stock[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
#11(4) #13(9) #15(2) #16(7) #18(1) #12(0) #14(0) #17(0)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.34 غربال إراتوستينس

تجارة إلكترونية ★★ متوسط

لإنشاء أرقام أولية لجداول التجزئة في قاعدة البيانات، اكتب برنامجًا يطبق **غربال إراتوستينس** حتى `LIMIT`: جدول `isPrime` نشطب فيه مضاعفات كل عدد أولي.

خطوات الحل

1) نهيئ `isPrime[k] = 1` لكل `k >= 2`.
2) لكل `number` من 2 ما دام `number * number <= LIMIT`: إذا كان أوليًا نشطب مضاعفاته بدءًا من `number * number`.
3) نعرض الأعداد التي بقيت.

البرنامج بلغة C

```c
#include <stdio.h>

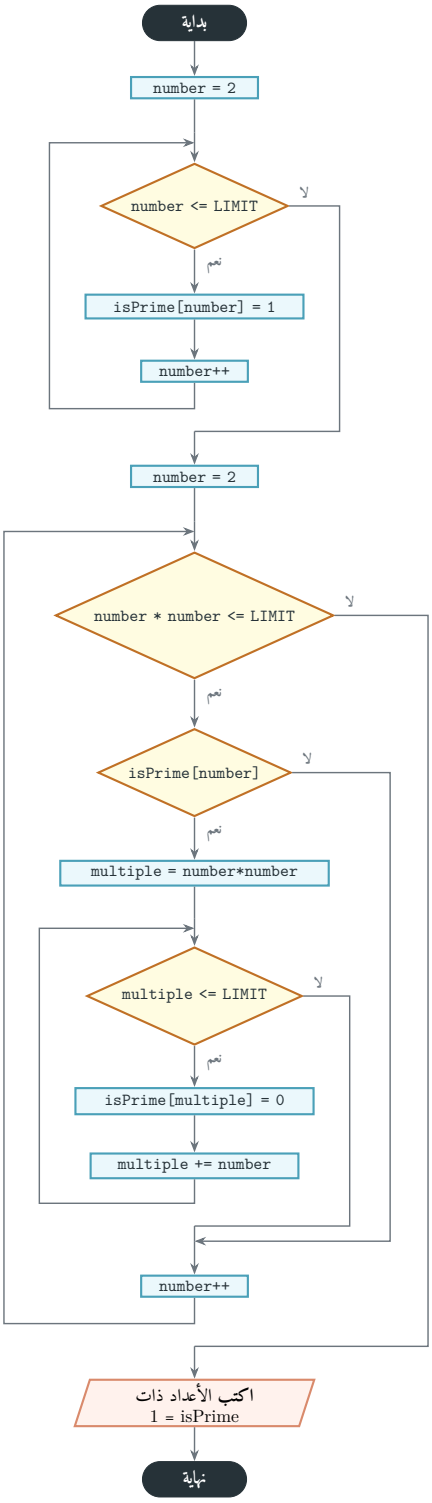
#define LIMIT 120

int main(void) {
    int isPrime[LIMIT + 1];
    int number, multiple, count = 0;
    for (number = 2; number <= LIMIT; number++)
        isPrime[number] = 1;              // كل عدد مرشح مبدئيًا
    for (number = 2; number * number <= LIMIT; number++)
        if (isPrime[number])
            // المضاعفات الأصغر من number² شُطبت سابقًا
            for (multiple = number * number; multiple <= LIMIT; multiple += number)
                isPrime[multiple] = 0;
    for (number = 2; number <= LIMIT; number++)
        if (isPrime[number]) {
            printf("%d ", number);
            count++;
        }
    printf("\n%d primes up to %d\n", count, LIMIT);
    return 0;
}
```

نتيجة التنفيذ

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113
30 primes up to 120
```

المخطط الإنسيابي (Organigramme)



تمرين 4.35 عدد المنتجات المختلفة في سلة

تجارة إلكترونية ★★ متوسط

سجل المشتريات غير مرتب ويحتوي أرقام منتجات متكررة. اكتب برنامجًا يعد المنتجات **المختلفة** ويعرض كل منتج مع عدد مرات شرائه، دون ترتيب الجدول.

خطوات الحل

1) لكل عنصر index: تحقق هل ظهر قبله (حلقة داخلية على 0..index-1).
2) إذا لم يظهر فهو جديد: نعده ونحسب تكراره في بقية الجدول.

البرنامج بلغة C

```c
#include <stdio.h>

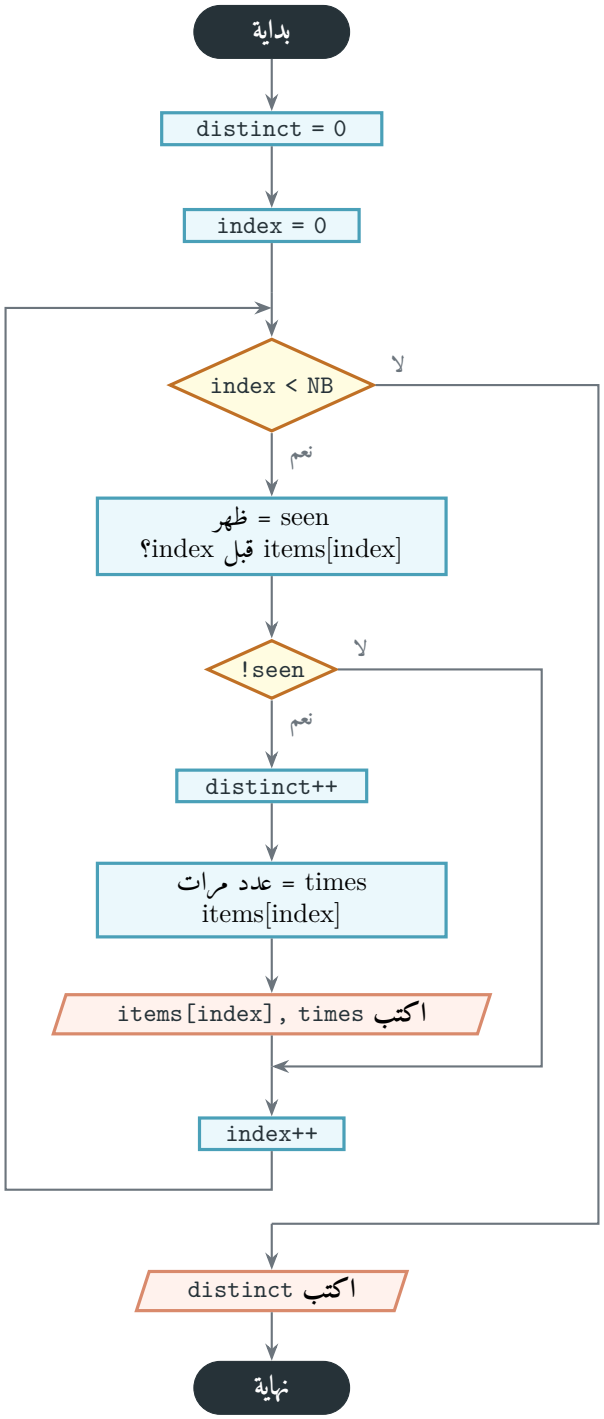
#define NB 12

int main(void) {
    int items[NB] = {204, 117, 204, 350, 117, 204, 980, 350, 411, 117, 204, 980};
    int index, scan, seen, times, distinct = 0;
    for (index = 0; index < NB; index++) {
        seen = 0;
        for (scan = 0; scan < index && !seen; scan++)
            if (items[scan] == items[index])
                seen = 1;                // عولج سابقًا
        if (!seen) {
            distinct++;
            times = 0;
            for (scan = index; scan < NB; scan++)
                if (items[scan] == items[index])
                    times++;
            printf("Product %d bought %d time(s)\n", items[index], times);
        }
    }
    printf("Distinct products: %d\n", distinct);
    return 0;
}
```

نتيجة التنفيذ

```
Product 204 bought 4 time(s)
Product 117 bought 3 time(s)
Product 350 bought 2 time(s)
Product 980 bought 2 time(s)
Product 411 bought 1 time(s)
Distinct products: 5
```

المخطط الإنسيابي (Organigramme)



تمرين 4.36 مطابقة جردين

تجارة إلكترونية ★★ متوسط

بعد الجرد السنوي نقارن جدول المخزون في النظام مع جدول العد الفعلي لنفس المنتجات. اكتب برنامجًا يعرض المنتجات المختلفة مع الفرق، ويحكم إن كان الجردان متطابقين.

خطوات الحل

1) نمر على الجدولين بنفس الفهرس.
2) إذا كان `system[index] != counted[index]` نعرض الفرق ونزيد عداد الاختلافات.

البرنامج بلغة C

```c
#include <stdio.h>

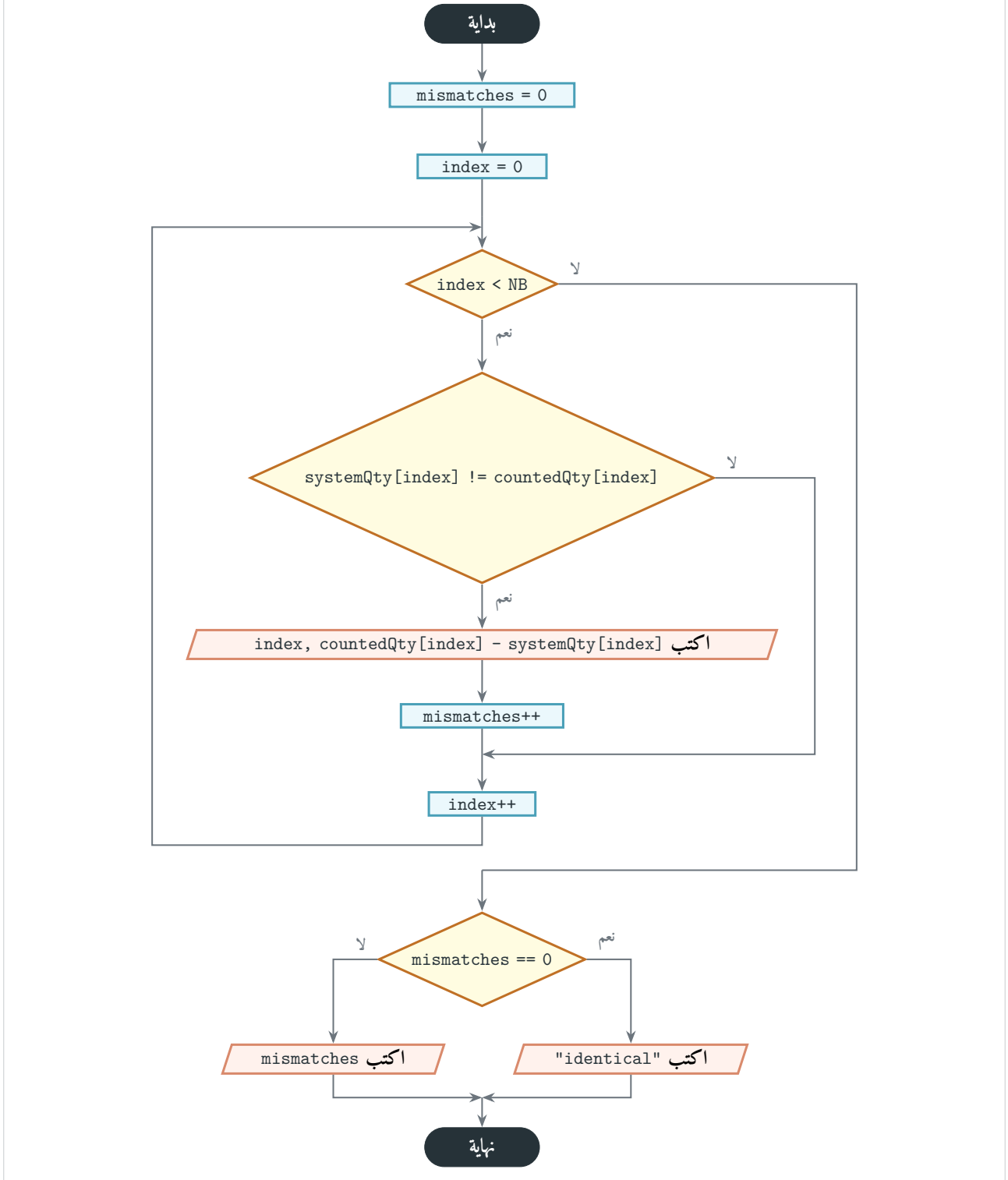
#define NB 8

int main(void) {
    int systemQty[NB]  = {40, 12, 7, 0, 25, 60, 3, 18};
    int countedQty[NB] = {40, 11, 7, 0, 27, 60, 3, 15};
    int index, mismatches = 0;
    for (index = 0; index < NB; index++) {
        if (systemQty[index] != countedQty[index]) {
            printf("Product %d: system %d, counted %d (%+d)\n", index,
                   systemQty[index], countedQty[index],
                   countedQty[index] - systemQty[index]);
            mismatches++;
        }
    }
    if (mismatches == 0)
        printf("Inventories are identical\n");
    else
        printf("%d mismatch(es) to investigate\n", mismatches);
    return 0;
}
```

نتيجة التنفيذ

```
Product 1: system 12, counted 11 (-1)
Product 4: system 25, counted 27 (+2)
Product 7: system 18, counted 15 (-3)
3 mismatch(es) to investigate
```

المخطط الإنسيابي (Organigramme)



تمرين 4.37 البتات في جدول

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يحول رقم شحنة إلى الثنائي بتخزين البتات في جدول (البت الأقل وزنًا أولًا)، ثم يعرضها بالترتيب الصحيح، ويعرض أيضًا عدد الخانات.

خطوات الحل

1) ما دام `number > 0`: `bits[nbBits++] = number % 2` و`number /= 2`.

2) العرض من `nbBits - 1` نزولًا إلى 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int bits[32], nbBits = 0, number, pos;
    scanf("%d", &number);
    printf("%d = ", number);
    do {
        bits[nbBits++] = number % 2;  // البتات من اليمين إلى اليسار
        number = number / 2;
    } while (number > 0);
    for (pos = nbBits - 1; pos >= 0; pos--)    // العرض بالترتيب المعاكس
        printf("%d", bits[pos]);
    printf(" (%d bits)\n", nbBits);
    return 0;
}
```

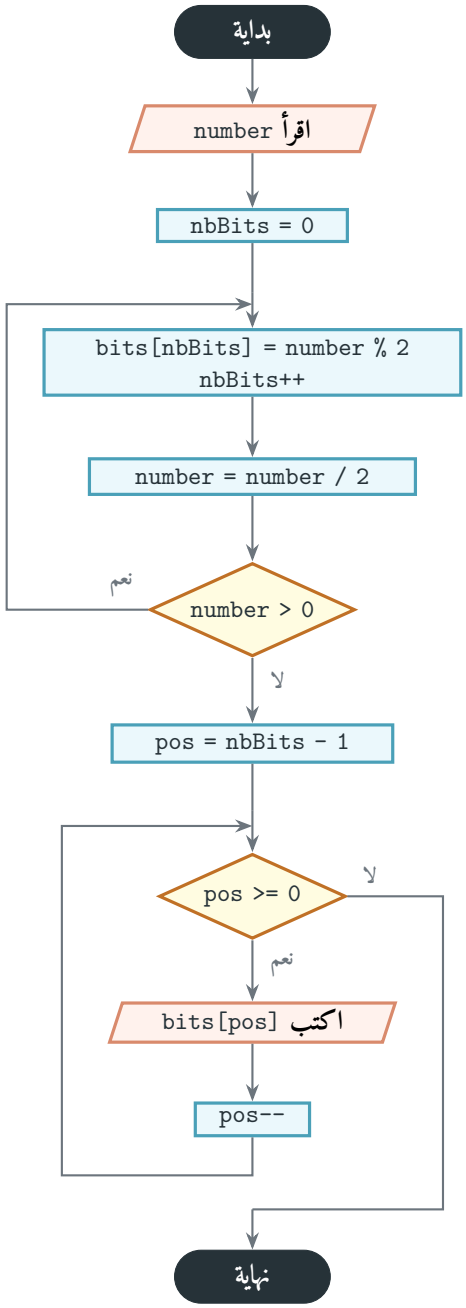
المدخلات

```
1453
```

نتيجة التنفيذ

```
1453 = 10110101101 (11 bits)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.38 جمع أعداد ضخمة

علم البيانات ★★ متوسط

رقم الأعمال السنوي لمنصة كبيرة يتجاوز سعة `long long`. نخزن العدد في **جدول من الأرقام** (الآحاد في الخانة 0). اكتب برنامجًا يجمع عددين من 25 رقمًا بخوارزمية الجمع المدرسية مع الاحتفاظ.

خطوات الحل

1) نقرأ كل عدد حرفًا بحرف بـ `scanf(" %c")` ونحول `ch - '0'`.
2) نخزن معكوسًا: الحرف الأول هو أعلى رقم.

3) لكل موضع: `sum = numA[pos] + numB[pos] + carry`، الرقم `sum % 10` والاحتفاظ `sum / 10`.

البرنامج بلغة C

```c
#include <stdio.h>

#define DIGITS 25

int main(void) {
    int numA[DIGITS], numB[DIGITS], result[DIGITS + 1];
    int pos, sum, carry = 0;
    char digit;
    for (pos = DIGITS - 1; pos >= 0; pos--) {      // الآحاد في الخانة 0
        scanf(" %c", &digit);
        numA[pos] = digit - '0';       // تحويل الحرف إلى رقم
    }
    for (pos = DIGITS - 1; pos >= 0; pos--) {
        scanf(" %c", &digit);
        numB[pos] = digit - '0';
    }
    for (pos = 0; pos < DIGITS; pos++) {
        sum = numA[pos] + numB[pos] + carry;
        result[pos] = sum % 10;        // الرقم المكتوب
        carry = sum / 10;              // الاحتفاظ للموضع التالي
    }
    result[DIGITS] = carry;
    pos = (carry > 0) ? DIGITS : DIGITS - 1;    // نتجاهل الصفر البادئ
    printf("Sum = ");
    for (; pos >= 0; pos--)
        printf("%d", result[pos]);
    printf("\n");
    return 0;
}
```

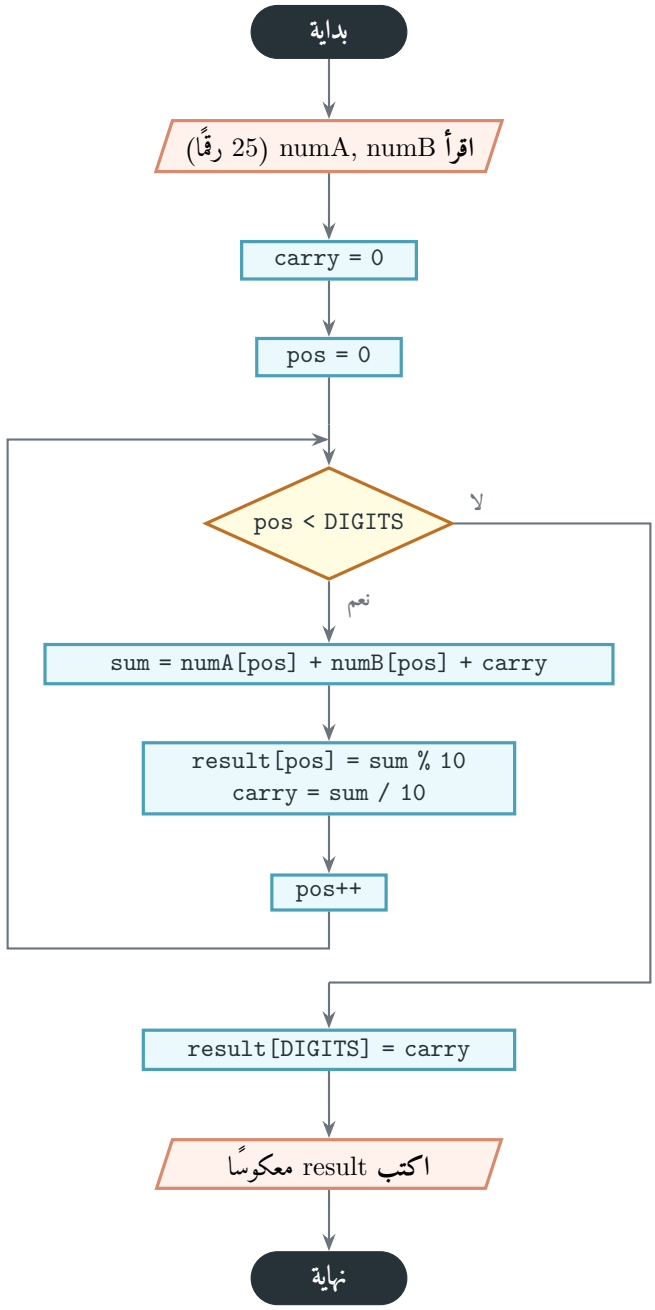
المدخلات

```
987654321098765432109876
123456789012345678901234
```

نتيجة التنفيذ

```
Sum = 1111111110111111111110111110
```

المخطط الإنسيابي (Organigramme)



تمرين 4.39 أطول فترة صعود في جدول

علم البيانات ★★ متوسط

اكتب برنامجًا يجد في جدول أسعار سهم شركة تجارة إلكترونية **أطول فترة متصلة** من الارتفاع المتتالي، ويعرض فهرسي بدايتها ونهايتها وقيمها.

خطوات الحل

1) `start` بداية الفترة الحالية.
2) إذا لم يكن `price[index] > price[index - 1]` تبدأ فترة جديدة عند `index`.
3) إذا كان طول الفترة الحالية `index - start + 1` أكبر من الأفضل نحفظها.

البرنامج بلغة C

```c
#include <stdio.h>

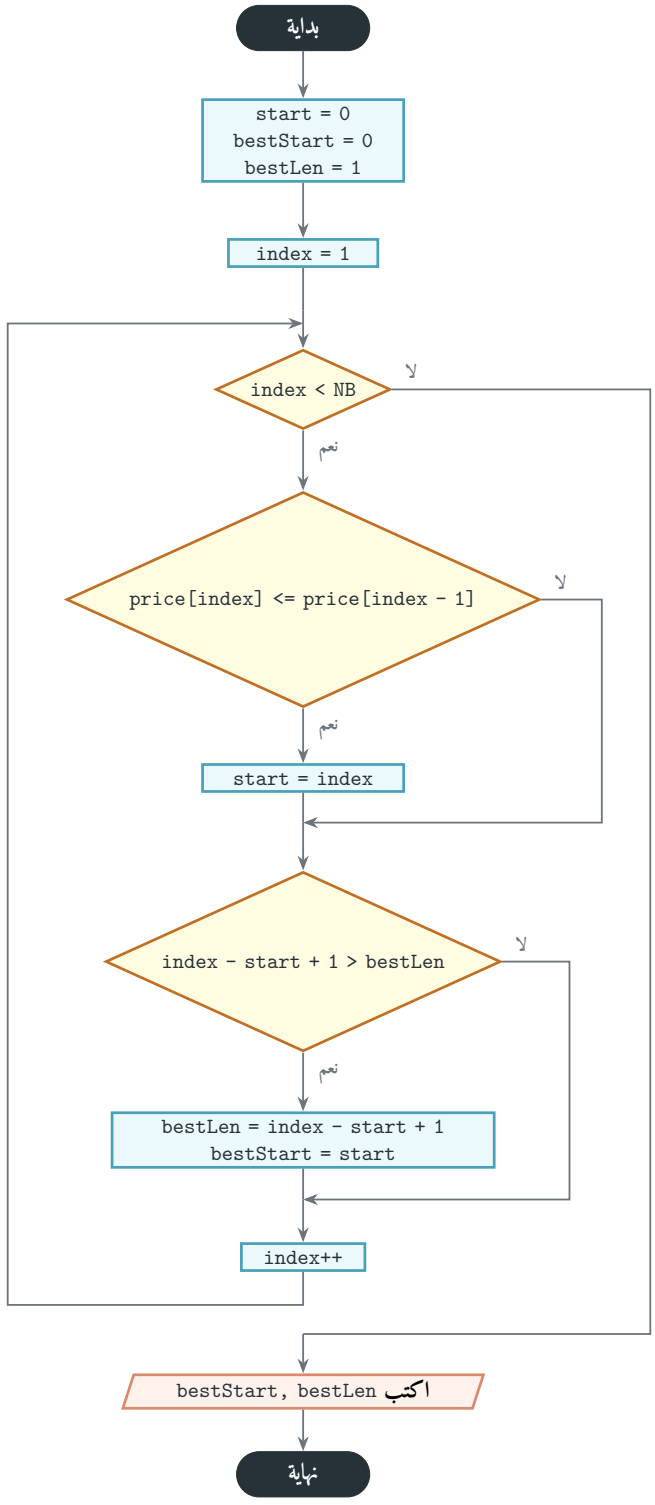
#define NB 12

int main(void) {
    double price[NB] = {50, 52, 51, 53, 55, 58, 60, 57, 59, 61, 62, 60};
    int index, start = 0, bestStart = 0, bestLen = 1;
    for (index = 1; index < NB; index++) {
        if (price[index] <= price[index - 1])
            start = index;                    // الصعود انقطع
        if (index - start + 1 > bestLen) {
            bestLen = index - start + 1;
            bestStart = start;
        }
    }
    printf("Longest rise: index %d to %d (%d values):",
           bestStart, bestStart + bestLen - 1, bestLen);
    for (index = bestStart; index < bestStart + bestLen; index++)
        printf(" %.0f", price[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Longest rise: index 2 to 6 (5 values): 51 53 55 58 60
```

المخطط الإنسيابي (Organigramme)



تمرين 4.40 الدرجة الموزونة للمنتج

علم البيانات ★★ متوسط

لترتيب نتائج البحث نعطي كل منتج **درجة** = الجداء السلمي بين جدول خصائصه (التقييم، المبيعات المطبعة، الهامش، التوفر) وجدول **الأوزان** `{0.4, 0.3, 0.2, 0.1}`. اكتب برنامجًا يحسب درجات 4 منتجات ويعرض الأفضل.

خطوات الحل

1) الخصائص في جدول أحادي واحد: المنتج `product` خصائصه من `product * 4` إلى `product * 4 + 3`.
2) الدرجة: `Σ weights[feat] * features[product * 4 + feat]`.
3) نحتفظ بالأفضل.

البرنامج بلغة C

```c
#include <stdio.h>

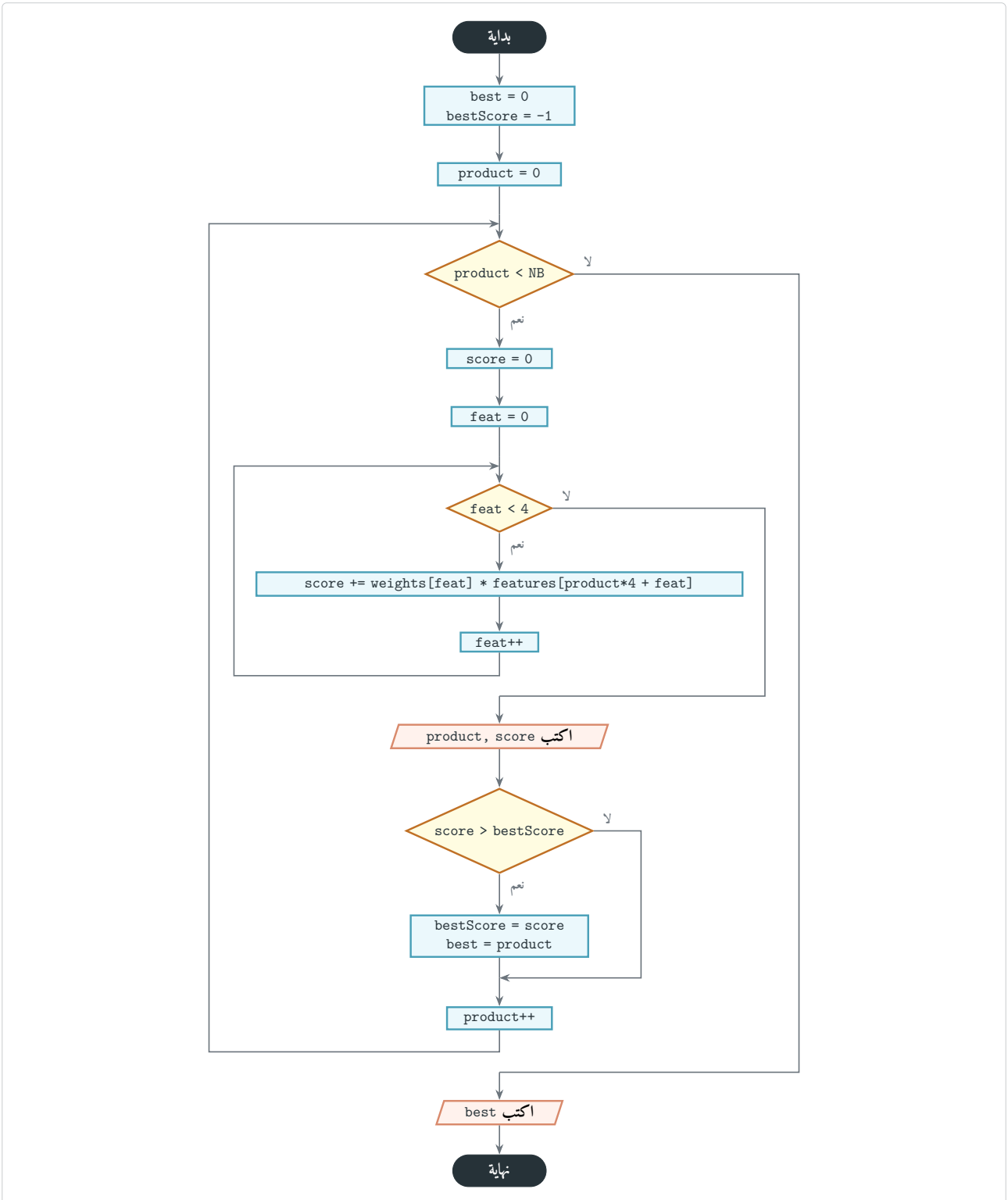
#define NB 4
#define NB_FEAT 4

int main(void) {
    double weights[NB_FEAT] = {0.4, 0.3, 0.2, 0.1};
    // كل 4 قيم متتالية = منتج: تقييم، مبيعات، هامش، توفر (بين 0 و 1)
    double features[NB * NB_FEAT] = {0.90, 0.40, 0.30, 1.0,
                                     0.70, 0.95, 0.50, 1.0,
                                     0.85, 0.60, 0.80, 0.0,
                                     0.60, 0.30, 0.90, 1.0};
    double score, bestScore = -1;
    int product, feat, best = 0;
    for (product = 0; product < NB; product++) {
        score = 0;
        for (feat = 0; feat < NB_FEAT; feat++)
            score += weights[feat] * features[product * NB_FEAT + feat];
        printf("Product %d: score %.3f\n", product, score);
        if (score > bestScore) {
            bestScore = score;
            best = product;
        }
    }
    printf("Top result: product %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Product 0: score 0.640
Product 1: score 0.765
Product 2: score 0.680
Product 3: score 0.610
Top result: product 1
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 4.41 أفضل فترة ربح متتالية (خوارزمية كادان)

علم البيانات ★★★ صعب

جدول `profit` يحتوي الربح الصافي اليومي لحملة إعلانية (قد يكون سالبًا). اكتب برنامجًا يجد **الفترة المتصلة** ذات **أكبر مجموع** بخوارزمية **كادان** في مرور واحد، مع يومي بدايتها ونهايتها.

خطوات الحل

1) `current` أفضل مجموع لفترة تنتهي عند اليوم الحالي.
2) إذا كان `current < 0` فمن الأفضل البدء من جديد: `current = profit[day]` و`start = day`.
3) وإلا: `current += profit[day]`.
4) إذا كان `current > best` نحفظ الفترة.

البرنامج بلغة C

```c
#include <stdio.h>

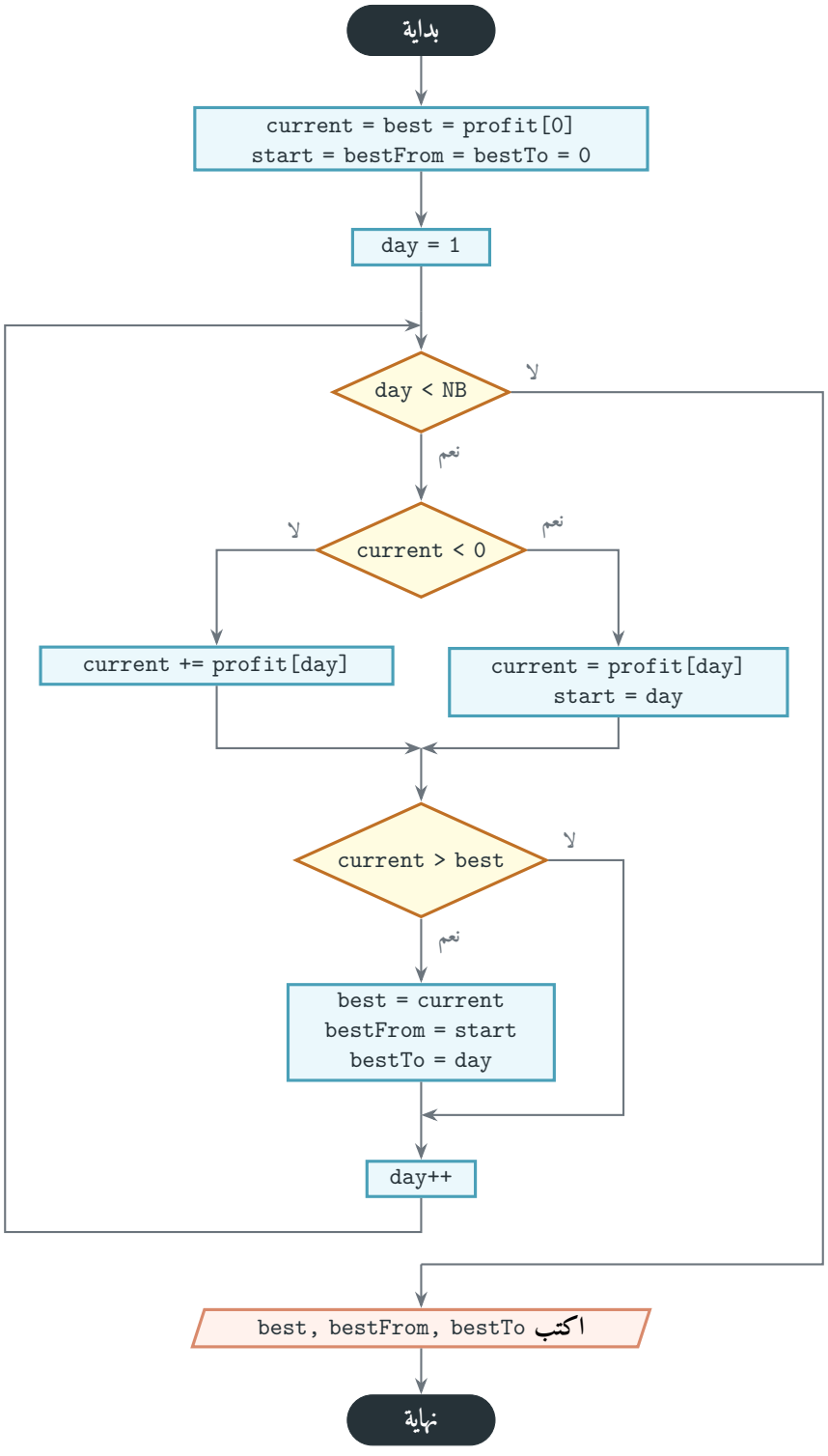
#define NB 12

int main(void) {
    int profit[NB] = {-200, 150, 300, -120, 400, -500, 100, 250, -50, 300, -400, 80};
    int current = profit[0], best = profit[0];
    int start = 0, bestFrom = 0, bestTo = 0, day;
    for (day = 1; day < NB; day++) {
        if (current < 0) {          // ماضٍ سالب يضر: نبدأ فترة جديدة
            current = profit[day];
            start = day;
        } else {
            current += profit[day];   // نمدد الفترة الحالية
        }
        if (current > best) {
            best = current;
            bestFrom = start;
            bestTo = day;
        }
    }
    printf("Best period: day %d to day %d, profit = %d DA\n",
           bestFrom + 1, bestTo + 1, best);
    return 0;
}
```

نتيجة التنفيذ

```
Best period: day 2 to day 10, profit = 830 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.42 الترتيب السريع دون عودية

تجارة إلكترونية ★★★ صعب

اكتب برنامجًا يرتب أسعار 12 منتجًا بخوارزمية **الترتيب السريع** (Quick Sort) دون استعمال الدوال العودية: نستعمل **مكدسًا يدويًا** (جدولين `stackLow` و`stackHigh`) لحفظ الأجزاء التي تنتظر الترتيب. التقسيم بطريقة Lomuto مع آخر عنصر كمحور.

خطوات الحل

1) ندفع المجال `[0, NB-1]` في المكدس.
2) ما دام المكدس غير فارغ: نسحب مجالًا `[low, high]`.
3) التقسيم: كل عنصر أصغر من المحور ينقل إلى اليسار، ثم يوضع المحور في مكانه `pivotPos`.
4) ندفع الجزأين `[low, pivotPos-1]` و`[pivotPos+1, high]` إذا كان طولهما أكبر من 1.

البرنامج بلغة C

```c
#include <stdio.h>

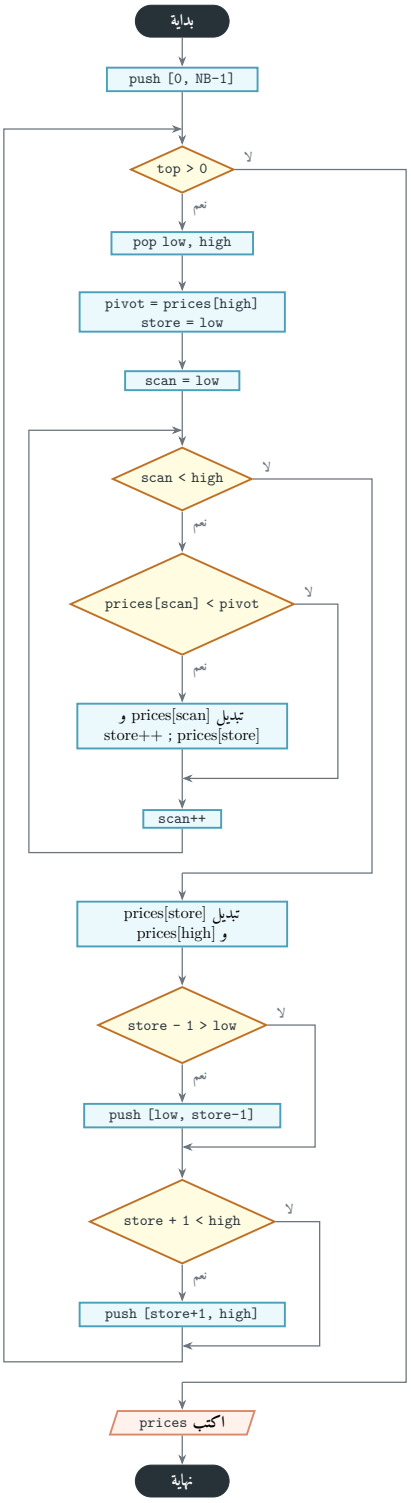
#define NB 12

int main(void) {
    int prices[NB] = {890, 120, 450, 3000, 75, 640, 1200, 300, 999, 15, 2200, 640};
    int stackLow[NB], stackHigh[NB], top = 0;
    int low, high, pivot, store, scan, tmp, index;
    stackLow[top] = 0; stackHigh[top] = NB - 1; top++;   // المجال الكامل
    while (top > 0) {
        top--;                                  // نسحب مجالًا
        low = stackLow[top];
        high = stackHigh[top];
        pivot = prices[high];                   // المحور: آخر عنصر
        store = low;                            // حد منطقة "الأصغر من المحور"
        for (scan = low; scan < high; scan++) {
            if (prices[scan] < pivot) {
                tmp = prices[scan]; prices[scan] = prices[store]; prices[store] = tmp;
                store++;
            }
        }
        tmp = prices[store]; prices[store] = prices[high]; prices[high] = tmp;
        // المحور الآن في مكانه النهائي store
        if (store - 1 > low)  { stackLow[top] = low;       stackHigh[top] = store - 1; top++; }
        if (store + 1 < high) { stackLow[top] = store + 1; stackHigh[top] = high;      top++; }
    }
    for (index = 0; index < NB; index++)
        printf("%d ", prices[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
15 75 120 300 450 640 640 890 999 1200 2200 3000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.43 الترتيب بالدمج من الأسفل إلى الأعلى

علم البيانات ★★★ صعب

اكتب برنامجًا يرتب أزمنة استجابة الخادم بخوارزمية **الترتيب بالدمج** في نسختها التكرارية (Bottom-Up): ندمج أجزاء بطول 1 ثم 2 ثم 4 ... مستعملين جدولًا مساعدًا، ونعرض الجدول بعد كل مرحلة.

خطوات الحل

1) `width` يبدأ من 1 ويتضاعف.
2) لكل زوج من الأجزاء المتجاورة `[left, mid)` و`[mid, right)`: ندمجهما في `buffer`.
3) ننسخ `buffer` إلى الجدول الأصلي بعد كل مرحلة.

البرنامج بلغة C

```c
#include <stdio.h>

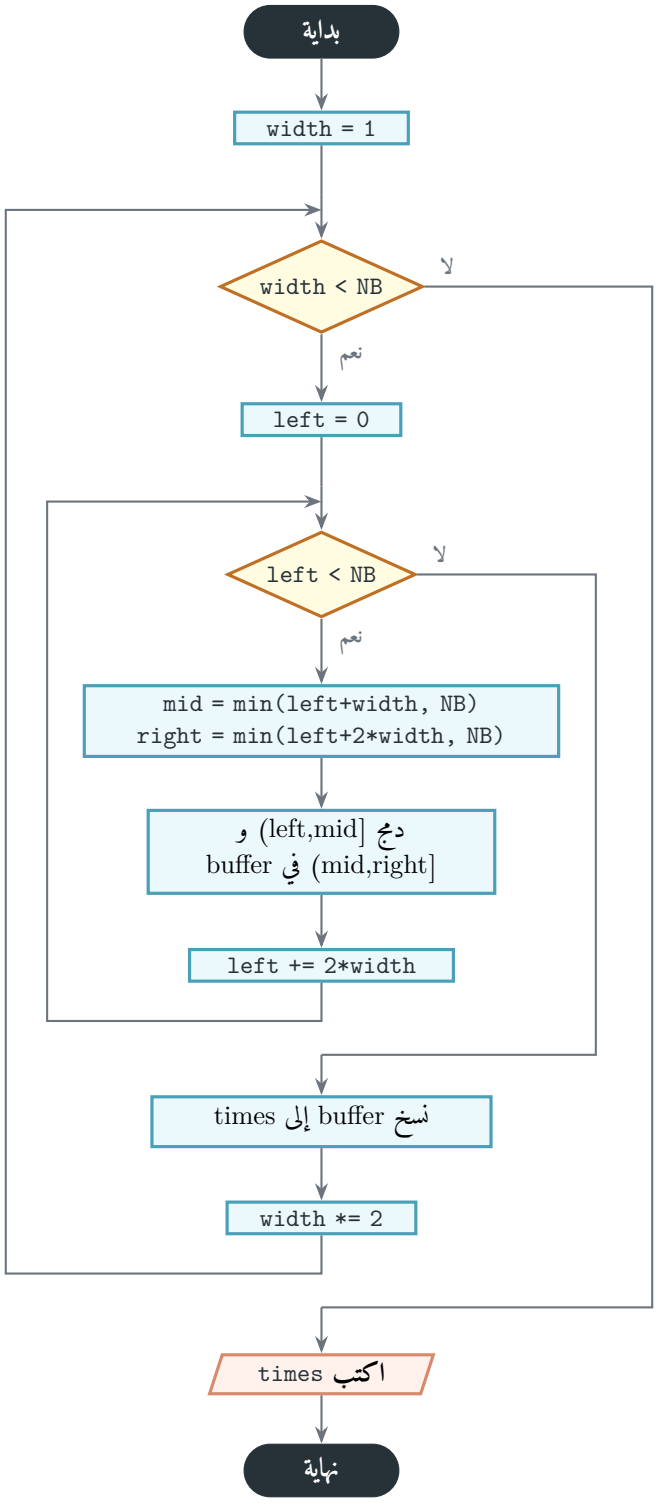
#define NB 10

int main(void) {
    int times[NB] = {120, 45, 300, 80, 95, 15, 210, 60, 33, 150};
    int buffer[NB];
    int width, left, mid, right, posA, posB, out, index;
    for (width = 1; width < NB; width *= 2) {          // طول الأجزاء المرتبة
        for (left = 0; left < NB; left += 2 * width) {
            mid = (left + width < NB) ? left + width : NB;
            right = (left + 2 * width < NB) ? left + 2 * width : NB;
            posA = left; posB = mid; out = left;
            while (posA < mid && posB < right)         // دمج جزأين مرتبين
                buffer[out++] = (times[posA] <= times[posB]) ? times[posA++] : times[posB++];
            while (posA < mid)   buffer[out++] = times[posA++];
            while (posB < right) buffer[out++] = times[posB++];
        }
        for (index = 0; index < NB; index++)
            times[index] = buffer[index];
        printf("width %d:", width);
        for (index = 0; index < NB; index++)
            printf(" %d", times[index]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
width 1: 45 120 80 300 15 95 60 210 33 150
width 2: 45 80 120 300 15 60 95 210 33 150
width 4: 15 45 60 80 95 120 210 300 33 150
width 8: 15 33 45 60 80 95 120 150 210 300
```

المخطط الإنسيابي (Organigramme)



تمرين 4.44 الترتيب بالعد لمليون تقييم

علم البيانات

صعب ★★★

لدينا عدد كبير من تقييمات المنتجات (0 إلى 10). الترتيب بالمقارنات بطيء، بينما **الترتيب بالعد** (Counting Sort) خطي. اكتب برنامجًا يولد 1000000 تقييم بمولد LCG يدوي، يعدها في جدول `count[11]`، ثم يحسب **الوسيط والمئين 90** مباشرة من العدادات دون تخزين الملايين.

خطوات الحل

1) مولد: `seed = (seed * 1103515245 + 12345) % 2^31`، والتقييم `(seed / 65536) % 11`.
2) `count[rating]++`.
3) نمر على العدادات تراكميًا: الوسيط هو أول قيمة يبلغ عندها التراكم `NB / 2`.

البرنامج بلغة C

```c
#include <stdio.h>

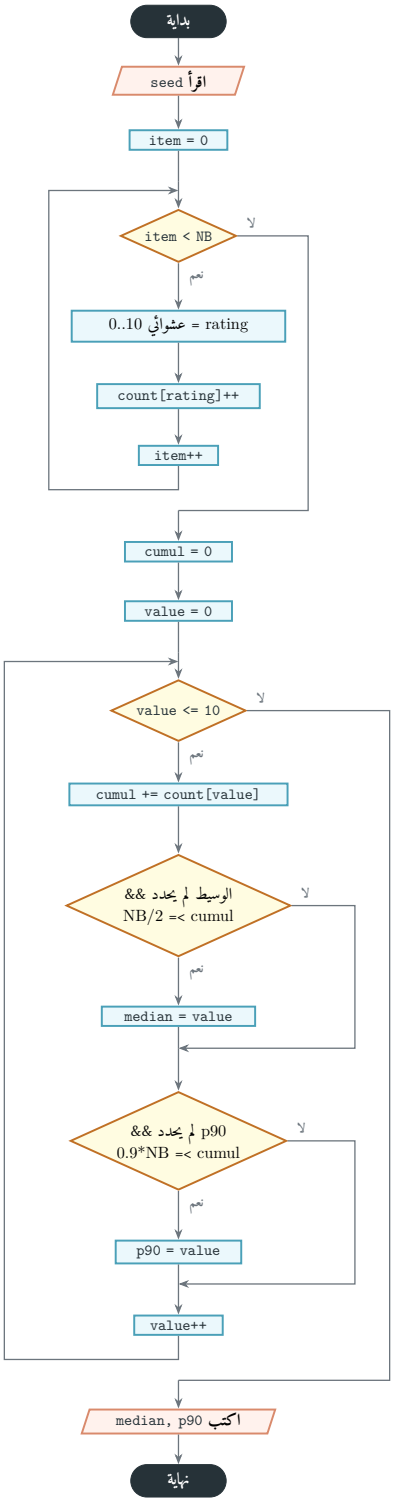
#define NB 1000000L

int main(void) {
    unsigned long long seed;
    long count[11] = {0}, cumul = 0, item;
    int value, rating, median = -1, p90 = -1;
    scanf("%llu", &seed);
    for (item = 0; item < NB; item++) {
        seed = (seed * 1103515245ULL + 12345ULL) % 2147483648ULL;
        rating = (int)((seed / 65536) % 11);  // تقييم من 0 إلى 10
        count[rating]++;                      // لا نخزن التقييم، فقط نعده
    }
    for (value = 0; value <= 10; value++) {
        cumul += count[value];                // كم تقييمًا <= value
        printf("%2d: %ld\n", value, count[value]);
        if (median < 0 && cumul >= NB / 2) median = value;
        if (p90 < 0 && cumul >= NB * 9 / 10) p90 = value;
    }
    printf("Median = %d, P90 = %d\n", median, p90);
    return 0;
}
```

نتيجة التنفيذ

```
 0: 90820
 1: 91304
 2: 91289
 3: 90572
 4: 91026
 5: 91112
 6: 90611
 7: 90940
 8: 90589
 9: 90691
10: 91046
Median = 5, P90 = 9
```

المخطط الإنسيابي (Organigramme)



تمرين 4.45 تصنيف زبون بـ k أقرب جيران

علم البيانات ★★★ صعب

لدينا 10 زبائن معروفين (عدد الزيارات الشهرية، متوسط السلة بالآلاف) وصنف كل منهم: 1 = سيشتري الاشتراك المميز، 0 = لا. اكتب برنامجًا يصنف زبونًا جديدًا بخوارزمية k-NN مع `k = 3`: نحسب المسافات، نختار الثلاثة الأقرب (انتقاء جزئي)، ونصوت.

خطوات الحل

1) `dist[index]` = مربع المسافة إلى كل زبون.
2) لاختيار الأقرب 3 مرات: نبحث عن أصغر مسافة غير مستعملة ونعلّمها `used[index] = 1`.
3) نجمع أصوات الصنف 1، والقرار بالأغلبية.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 10
#define NB_NEIGHBOURS 3

int main(void) {
    double visitsData[NB] = {1, 2, 3, 8, 9, 7, 5, 10, 4, 6};
    double basketData[NB] = {1.0, 1.5, 2.0, 6.0, 5.5, 4.0, 3.5, 7.0, 2.5, 5.0};
    int label[NB]         = {0, 0, 0, 1, 1, 1, 0, 1, 0, 1};
    double dist[NB], visits, basket, dv, db;
    int used[NB] = {0}, index, round, nearest, votes = 0;
    scanf("%lf %lf", &visits, &basket);
    for (index = 0; index < NB; index++) {
        dv = visits - visitsData[index];
        db = basket - basketData[index];
        dist[index] = dv * dv + db * db;      // مربع المسافة يكفي للمقارنة
    }
    for (round = 0; round < NB_NEIGHBOURS; round++) {     // نختار الأقرب NB_NEIGHBOURS مرات
        nearest = -1;
        for (index = 0; index < NB; index++)
            if (!used[index] && (nearest < 0 || dist[index] < dist[nearest]))
                nearest = index;
        used[nearest] = 1;
        votes += label[nearest];
        printf("Neighbour %d: customer %d (d2 = %.2f, class %d)\n",
               round + 1, nearest, dist[nearest], label[nearest]);
    }
    if (votes * 2 > NB_NEIGHBOURS)                        // الأغلبية
        printf("Prediction: premium subscriber\n");
    else
        printf("Prediction: standard\n");
    return 0;
}
```

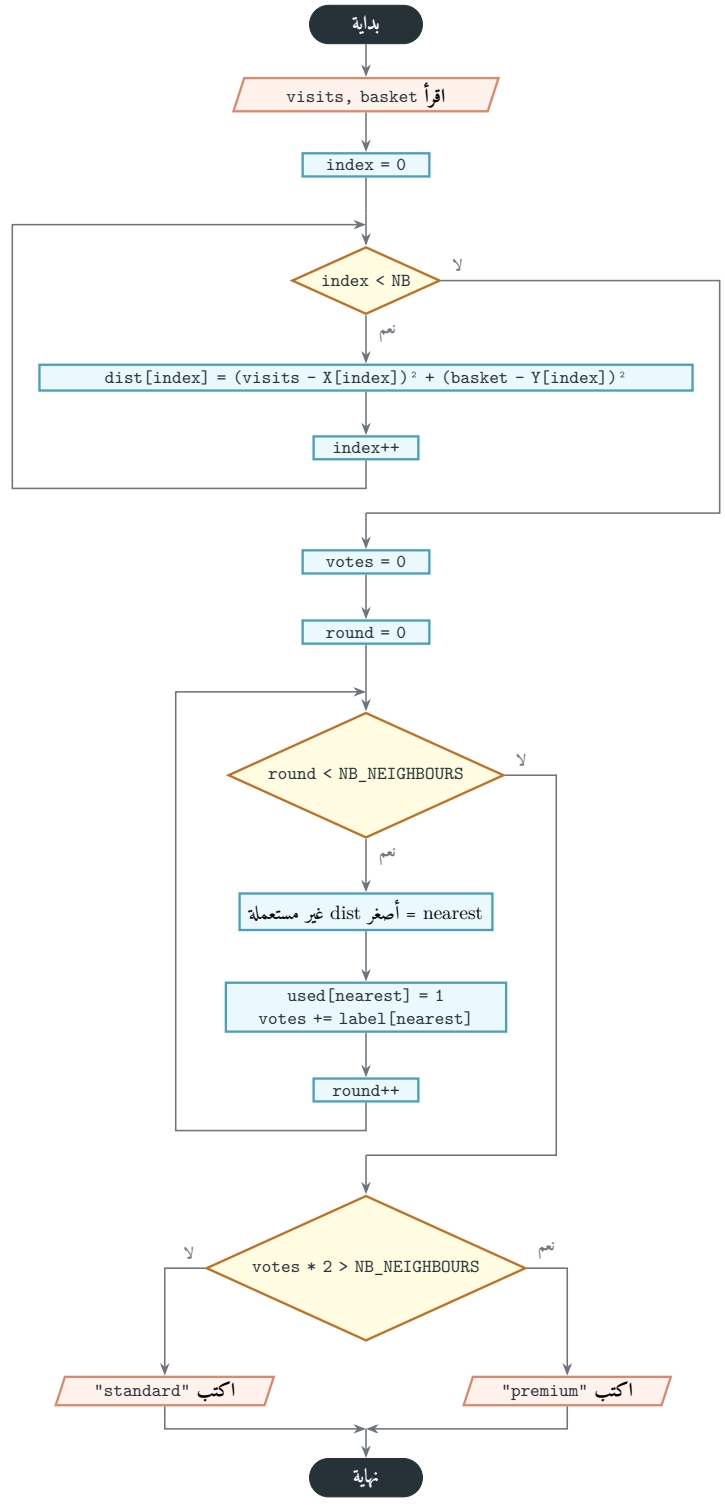
المدخلات

```
6 4.5
```

نتيجة التنفيذ

```
Neighbour 1: customer 9 (d2 = 0.25, class 1)
Neighbour 2: customer 5 (d2 = 1.25, class 1)
Neighbour 3: customer 6 (d2 = 2.00, class 0)
Prediction: premium subscriber
```

المخطط الإنسيابي (Organigramme)



تمرين 4.46 تجميع الزبائن إلى ثلاث فئات

علم البيانات ★★★ صعب

اكتب برنامجًا يطبق **K-Means** بثلاث مجموعات على إنفاق 15 زبونًا مخزن في جدول: المراكز الأولية هي أصغر قيمة، الوسط، وأكبر قيمة. نكرر الإسناد وإعادة الحساب حتى لا يتغير أي إسناد، ونعرض المراكز وأعضاء كل مجموعة.

خطوات الحل

1) `group[index]` رقم مجموعة كل زبون (يبدأ بـ -1).
2) الإسناد: أقرب مركز بالقيمة المطلقة للفرق.
3) `changed` يعد الزبائن الذين غيروا مجموعتهم؛ نتوقف عندما يساوي 0.
4) إعادة الحساب: كل مركز = متوسط أعضائه.

البرنامج بلغة C

```c
#include <stdio.h>

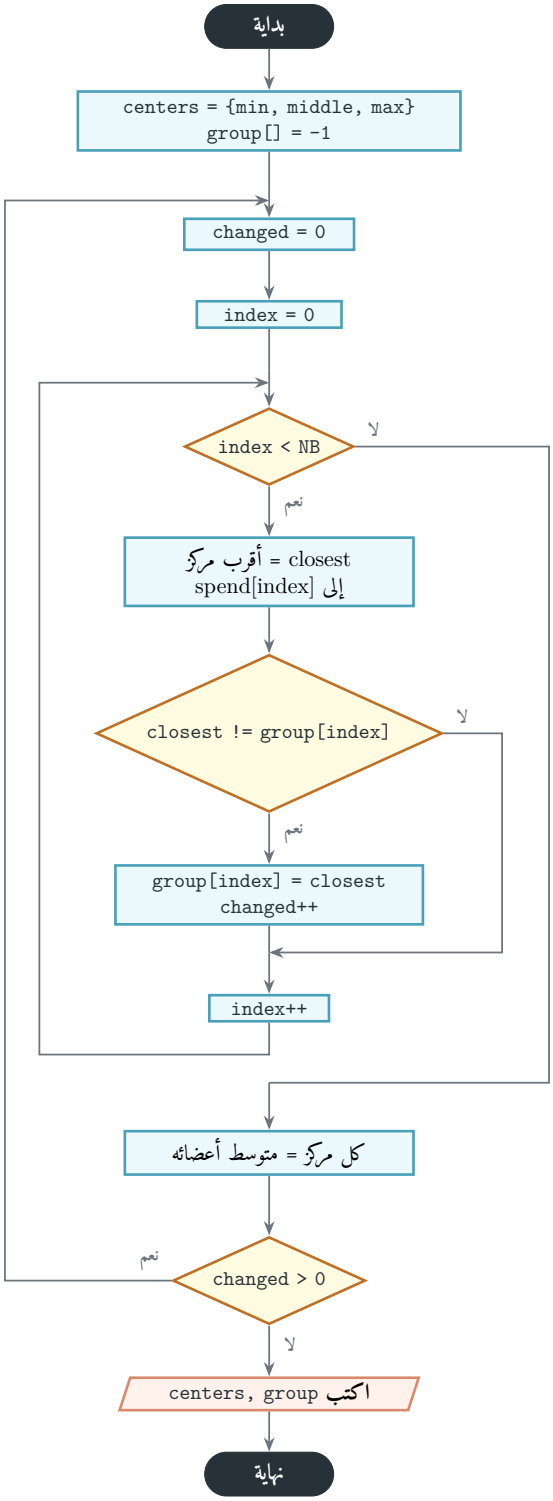
#define NB 15
#define NB_GROUPS 3

int main(void) {
    double spend[NB] = {12, 15, 90, 14, 52, 48, 95, 11, 55, 100, 47, 13, 88, 50, 92};
    double centers[NB_GROUPS] = {11, 50, 100};     // أصغر، وسط، أكبر
    double sum[NB_GROUPS], gap, bestGap;
    int group[NB], members[NB_GROUPS];
    int index, cluster, closest, changed, iteration = 0;
    for (index = 0; index < NB; index++)
        group[index] = -1;                 // لا مجموعة بعد
    do {
        changed = 0;
        for (index = 0; index < NB; index++) {       // مرحلة الإسناد
            closest = 0;
            bestGap = -1;
            for (cluster = 0; cluster < NB_GROUPS; cluster++) {
                gap = spend[index] - centers[cluster];
                if (gap < 0) gap = -gap;
                if (bestGap < 0 || gap < bestGap) { bestGap = gap; closest = cluster; }
            }
            if (closest != group[index]) { group[index] = closest; changed++; }
        }
        for (cluster = 0; cluster < NB_GROUPS; cluster++) { sum[cluster] = 0; members[cluster] = 0; }
        for (index = 0; index < NB; index++) {       // مرحلة إعادة الحساب
            sum[group[index]] += spend[index];
            members[group[index]]++;
        }
        for (cluster = 0; cluster < NB_GROUPS; cluster++)
            if (members[cluster] > 0)
                centers[cluster] = sum[cluster] / members[cluster];
        iteration++;
    } while (changed > 0);
    printf("Converged in %d iterations\n", iteration);
    for (cluster = 0; cluster < NB_GROUPS; cluster++) {
        printf("Group %d (center %.2f):", cluster, centers[cluster]);
        for (index = 0; index < NB; index++)
            if (group[index] == cluster)
                printf(" %.0f", spend[index]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Converged in 2 iterations
Group 0 (center 13.00): 12 15 14 11 13
Group 1 (center 50.40): 52 48 55 47 50
Group 2 (center 93.00): 90 95 100 88 92
```

المخطط الإنسيابي (Organigramme)



تمرين 4.47 اختيار الحملات الإعلانية بميزانية محدودة تجارة إلكترونية صعب ★★★

ميزانية التسويق BUDGET (بالآلاف). لكل حملة تكلفة وعائد متوقع، ولا يمكن تنفيذ الحملة إلا مرة واحدة. اكتب برنامجًا يجد أكبر عائد ممكن بالبرمجة الديناميكية (**مسألة حقيبة الظهر 0/1**) بجدول أحادي best[budget]، ثم يستنتج الحملات المختارة بجدول قرارات.

خطوات الحل

1) best[money] أكبر عائد بميزانية money باستعمال الحملات المعالجة حتى الآن.

لكل حملة نمر على الميزانيات **تنازليًا** لكي لا نستعمل الحملة مرتين: best[money] = max(best[money], best[money - cost] + gain).

3) نسجل take[campaign][money] في جدول مسطح لإعادة بناء الاختيار.

البرنامج بلغة C

```c
#include <stdio.h>

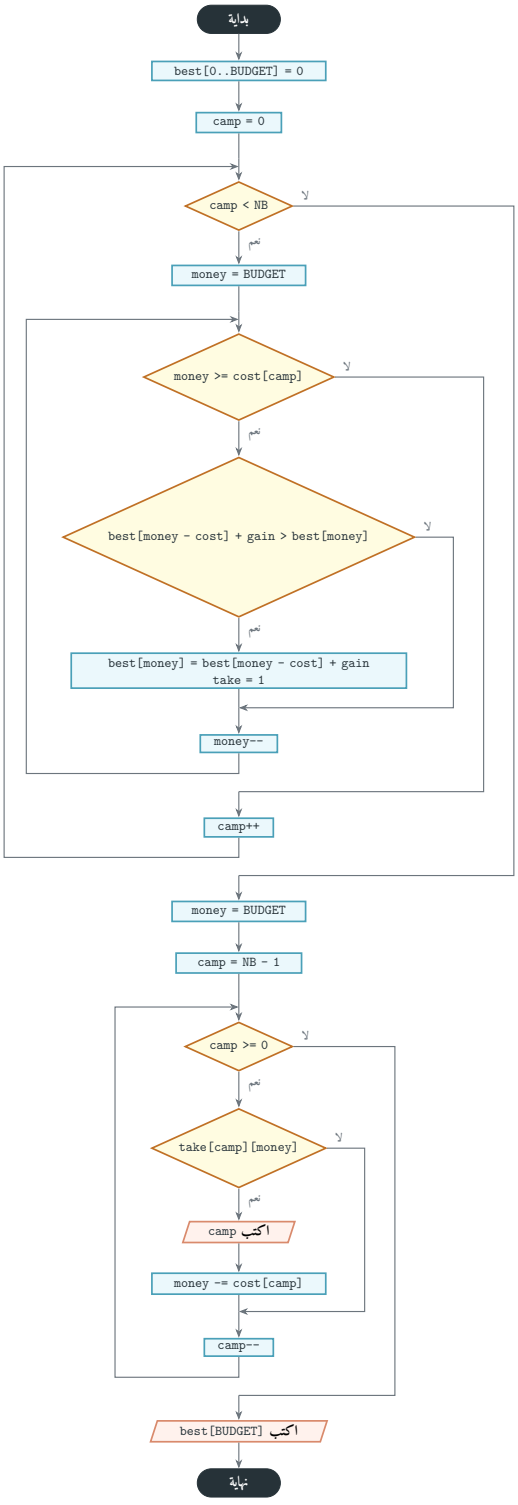
#define NB 6
#define BUDGET 20

int main(void) {
    int cost[NB] = {5, 8, 4, 10, 3, 7};        // بالآلاف
    int gain[NB] = {12, 20, 9, 25, 5, 14};     // العائد المتوقع
    int best[BUDGET + 1] = {0};
    int take[NB * (BUDGET + 1)] = {0};         // جدول قرارات مسطح
    int camp, money;
    for (camp = 0; camp < NB; camp++) {
        // تنازليًا: best[money - cost] ما زال يخص الحملات السابقة
        for (money = BUDGET; money >= cost[camp]; money--) {
            if (best[money - cost[camp]] + gain[camp] > best[money]) {
                best[money] = best[money - cost[camp]] + gain[camp];
                take[camp * (BUDGET + 1) + money] = 1;
            }
        }
    }
    printf("Max gain = %d with budget %d\n", best[BUDGET], BUDGET);
    money = BUDGET;
    printf("Campaigns:");
    for (camp = NB - 1; camp >= 0; camp--) {  // نعيد بناء الاختيار من الآخر
        if (take[camp * (BUDGET + 1) + money]) {
            printf(" #%d(cost %d)", camp, cost[camp]);
            money -= cost[camp];
        }
    }
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Max gain = 46 with budget 20
Campaigns: #3(cost 10) #2(cost 4) #0(cost 5)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.48 المنتجات التي تُشترى معًا

علم البيانات ★★★ صعب

نمثل كل سلة بقناع بتات: البت p يساوي 1 إذا احتوت السلة المنتج p (6 منتجات). اكتب برنامجًا يحسب لكل زوج من المنتجات **الدعم** (عدد السلات التي تحتوي الاثنين) ويعرض الأزواج التي يتجاوز دعمها 30%، مع **الثقة** `support(A,B) / support(A)` — أساس خوارزمية Apriori.

خطوات الحل

1) `single[p]` عدد السلات التي فيها المنتج p: نختبر `(basket >> p) & 1`.
2) لكل زوج `first < second`: نعد السلات التي فيها البتان.
3) نعرض الأزواج التي `pairCount * 100 / NB_BASKETS >= 30`.

البرنامج بلغة C

```c
#include <stdio.h>

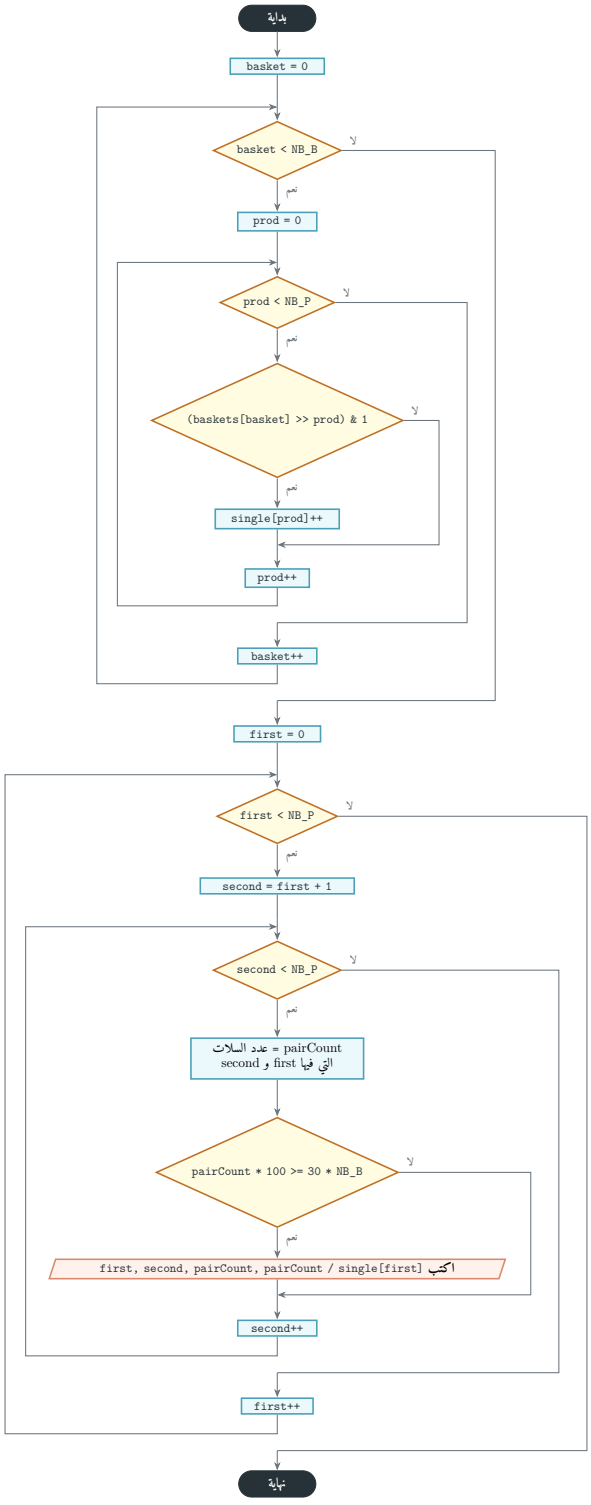
#define NB_B 10
#define NB_P 6

int main(void) {
    // 0 هاتف، 1 غلاف، 2 شاحن، 3 سماعات، 4 ساعة، 5 حاسوب
    int baskets[NB_B] = {0x07, 0x03, 0x0B, 0x01, 0x23, 0x0F, 0x30, 0x03, 0x13, 0x07};
    int single[NB_P] = {0};
    int basket, prod, first, second, pairCount, mask;
    for (basket = 0; basket < NB_B; basket++)
        for (prod = 0; prod < NB_P; prod++)
            if ((baskets[basket] >> prod) & 1)    // هل البت prod يساوي 1؟
                single[prod]++;
    for (first = 0; first < NB_P; first++) {
        for (second = first + 1; second < NB_P; second++) {
            mask = (1 << first) | (1 << second);  // البتان معًا
            pairCount = 0;
            for (basket = 0; basket < NB_B; basket++)
                if ((baskets[basket] & mask) == mask)
                    pairCount++;
            if (pairCount * 100 >= 30 * NB_B)
                printf("{%d,%d}: support %d%%, confidence %d->%d = %.2f\n",
                       first, second, pairCount * 100 / NB_B, first, second,
                       (double)pairCount / single[first]);
        }
    }
    return 0;
}
```

نتيجة التنفيذ

```
{0,1}: support 80%, confidence 0->1 = 0.89
{0,2}: support 30%, confidence 0->2 = 0.33
{1,2}: support 30%, confidence 1->2 = 0.38
```

المخطط الإنسيابي (Organigramme)



تمرين 4.49 معامل جيني لتركز الإنفاق

علم البيانات ★★★ صعب

يقيس **معامل جيني** مدى تركز الإنفاق عند قلة من الزبائن (0 = توزيع متساوٍ، 1 = زبون واحد ينفق كل شيء). بعد ترتيب الإنفاق تصاعديًا: `G = (2 Σ i·x_i) / (n Σ x_i) - (n + 1) / n` حيث `i` من 1 إلى `n`. اكتب برنامجًا يحسبه، ويحسب أيضًا حصة أكبر 20% من الزبائن.

خطوات الحل

1) ترتيب تصاعدي.
2) `weighted = Σ (rank + 1) * spend[rank]` و`total = Σ spend`.
3) نطبق الصيغة.
4) أكبر 20% هم آخر `NB / 5` عناصر بعد الترتيب.

البرنامج بلغة C

```c
#include <stdio.h>

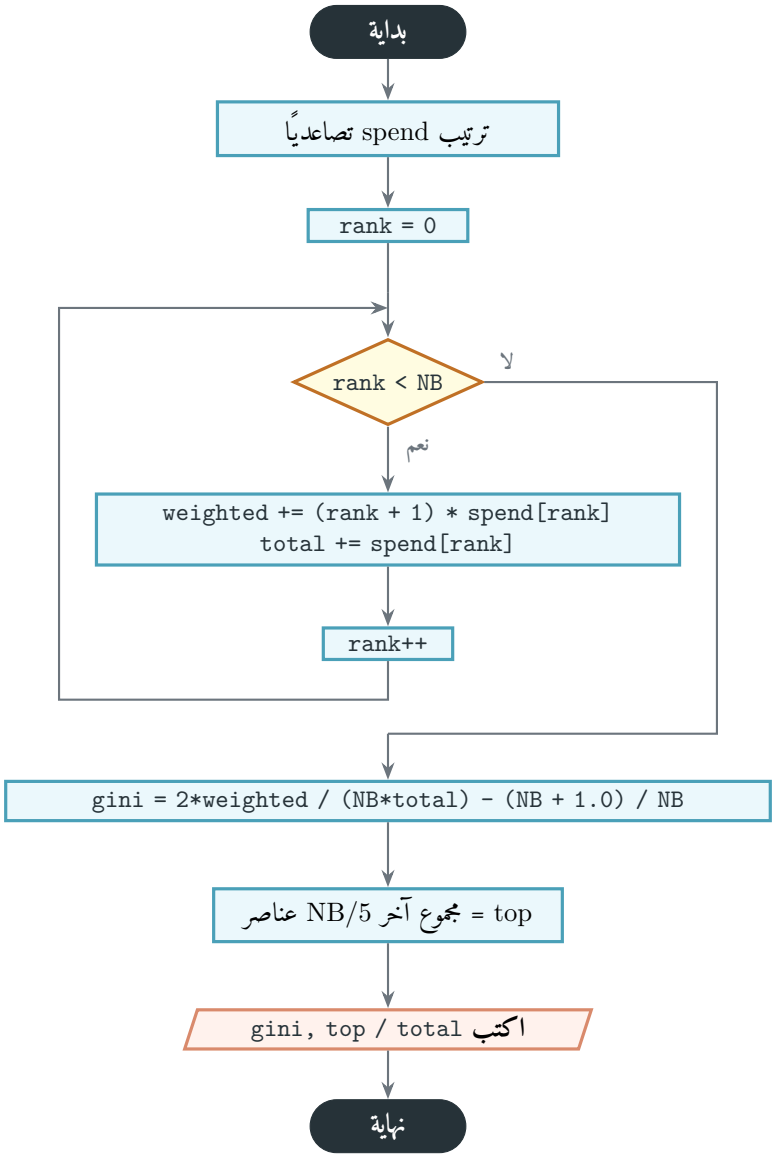
#define NB 10

int main(void) {
    double spend[NB] = {1200, 800, 45000, 3000, 950, 2200, 18000, 600, 1500, 7000};
    double key, weighted = 0, total = 0, gini, top = 0;
    int rank, scan;
    for (rank = 1; rank < NB; rank++) {            // ترتيب بالإدراج
        key = spend[rank];
        for (scan = rank - 1; scan >= 0 && spend[scan] > key; scan--)
            spend[scan + 1] = spend[scan];
        spend[scan + 1] = key;
    }
    for (rank = 0; rank < NB; rank++) {
        weighted += (rank + 1) * spend[rank];      // الرتبة تبدأ من 1
        total += spend[rank];
    }
    gini = 2 * weighted / (NB * total) - (NB + 1.0) / NB;
    for (rank = NB - NB / 5; rank < NB; rank++)    // أكبر 20%
        top += spend[rank];
    printf("Gini = %.3f\n", gini);
    printf("Top 20%% customers = %.1f%% of revenue\n", top * 100 / total);
    return 0;
}
```

نتيجة التنفيذ

```
Gini = 0.693
Top 20% customers = 78.5% of revenue
```

المخطط الإنسيابي (Organigramme)



تمرين 4.50 أطول فترة ضمن الميزانية

تجارة إلكترونية ★★★ صعب

يريد مسؤول الإعلانات تشغيل حملة في **أيام متتالية**، وتكلفة كل يوم معروفة. اكتب برنامجًا يجد **أطول** سلسلة أيام متتالية مجموع تكاليفها لا يتجاوز الميزانية، بطريقة **النافذة المنزلقة** في `O(n)`.

خطوات الحل

1) مؤشرا النافذة `left` و`right`، ومجموعها `windowCost`.
2) نوسع النافذة يمينًا بإضافة `cost[right]`.
3) ما دام المجموع يتجاوز الميزانية نقلصها من اليسار.
4) نحفظ أطول نافذة صالحة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 14

int main(void) {
    int cost[NB] = {300, 150, 200, 500, 100, 120, 90, 400, 80, 60, 250, 700, 50, 100};
    int budget, left = 0, right, windowCost = 0, bestLen = 0, bestLeft = 0, day;
    scanf("%d", &budget);
    for (right = 0; right < NB; right++) {
        windowCost += cost[right];          // نضيف يومًا من اليمين
        while (windowCost > budget) {       // تجاوزنا: نحذف من اليسار
            windowCost -= cost[left];
            left++;
        }
        if (right - left + 1 > bestLen) {
            bestLen = right - left + 1;
            bestLeft = left;
        }
    }
    printf("Longest campaign: %d days (day %d to %d):",
           bestLen, bestLeft + 1, bestLeft + bestLen);
    windowCost = 0;
    for (day = bestLeft; day < bestLeft + bestLen; day++) {
        printf(" %d", cost[day]);
        windowCost += cost[day];
    }
    printf(" = %d DA\n", windowCost);
    return 0;
}
```

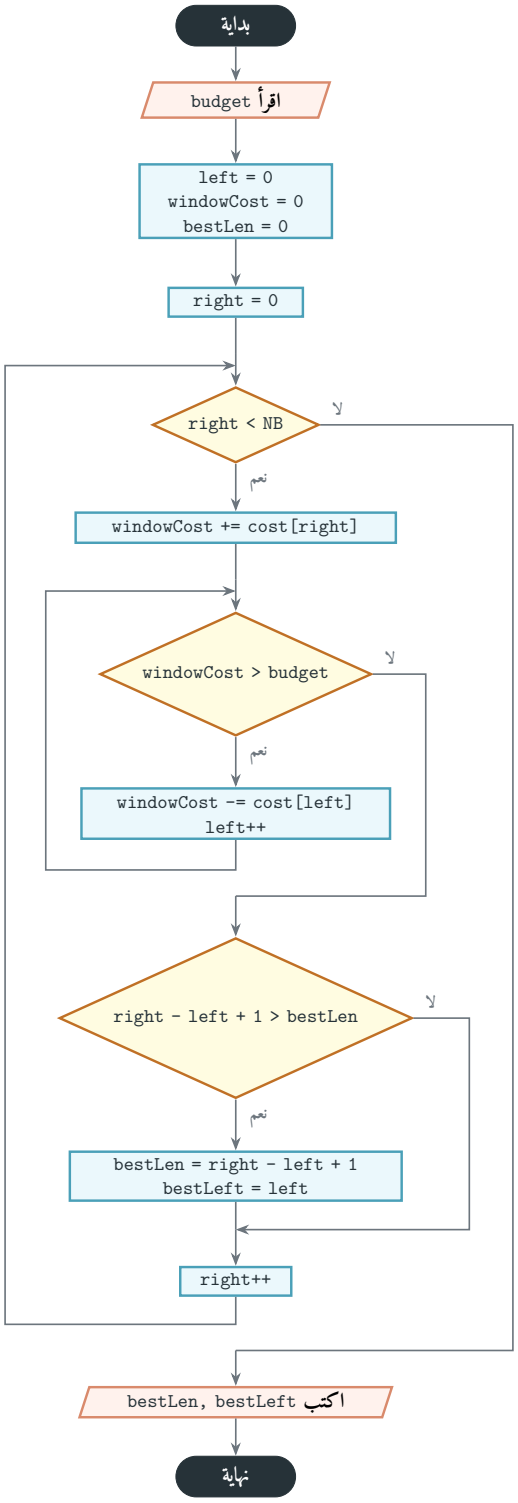
المدخلات

```
1000
```

نتيجة التنفيذ

```
Longest campaign: 6 days (day 5 to 10): 100 120 90 400 80 60 = 850 DA
```

المخطط الإنسيابي (Organigramme)



الفصل 5

الجداول ثنائية البعد

التمارين السهلة

تمرين 5.1 جدول مبيعات المتاجر

تجارة إلكترونية | سهل

مبيعات 3 متاجر خلال 4 أيام مخزنة في **مصفوفة** `sales[3][4]` (السطر = المتجر، العمود = اليوم). اكتب برنامجًا يعرضها على شكل جدول منسق.

خطوات الحل

1) نهيئ المصفوفة بأقواس متداخلة: سطر لكل متجر.
2) حلقة على الأسطر، وداخلها حلقة على الأعمدة.
3) نطبع سطرًا جديدًا بعد كل متجر.

البرنامج بلغة C

```c
#include <stdio.h>

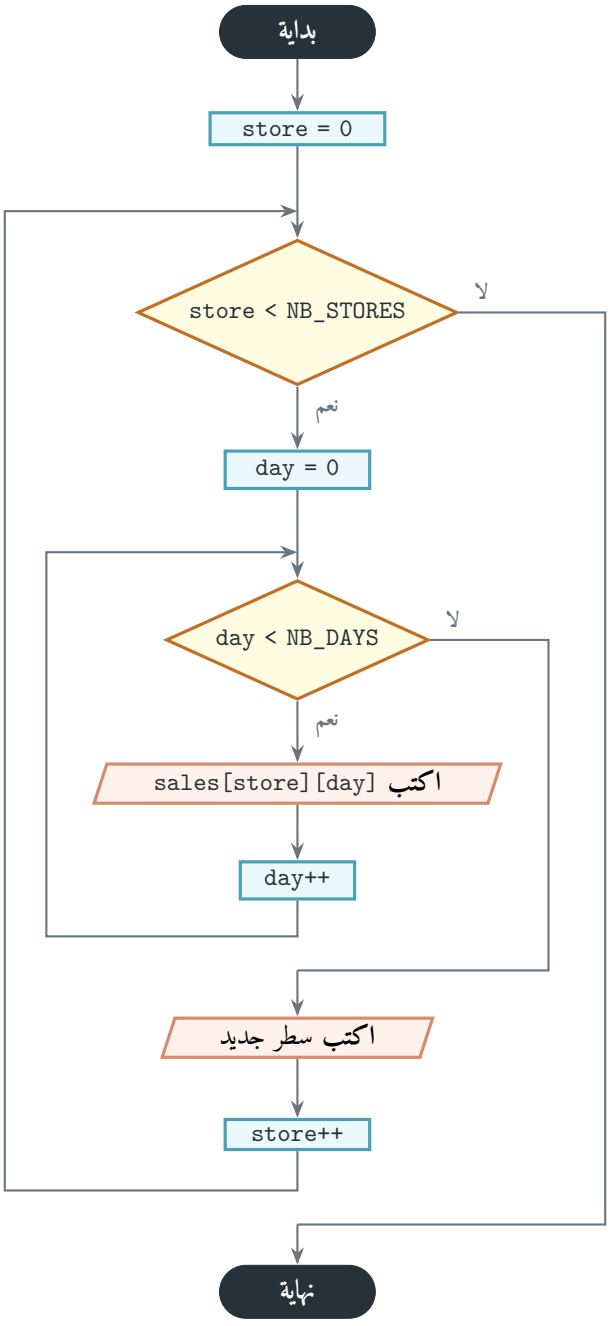
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {
        {120, 95, 140, 180},          // المتجر 0
        { 80, 60,  75,  90},          // المتجر 1
        {200, 210, 190, 250}          // المتجر 2
    };
    int store, day;
    printf("Store | D1  D2  D3  D4\n");
    for (store = 0; store < NB_STORES; store++) {      // الأسطر
        printf("%5d |", store);
        for (day = 0; day < NB_DAYS; day++)           // الأعمدة
            printf(" %3d", sales[store][day]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Store | D1  D2  D3  D4
    0 | 120  95 140 180
    1 |  80  60  75  90
    2 | 200 210 190 250
```

المخطط الإنسيابي (Organigramme)



تمرين 5.2 المجموع الكلي للمبيعات

علم البيانات سهل

اكتب برنامجًا يحسب مجموع كل خانات مصفوفة المبيعات (3 متاجر × 4 أيام) والمتوسط لكل خانة.

خطوات الحل

1) مجمّع واحد `total`.
2) حلقتان متداخلتان تضيفان كل خانة.
3) المتوسط: `total / (NB_STORES * NB_DAYS)`.

البرنامج بلغة C

```c
#include <stdio.h>

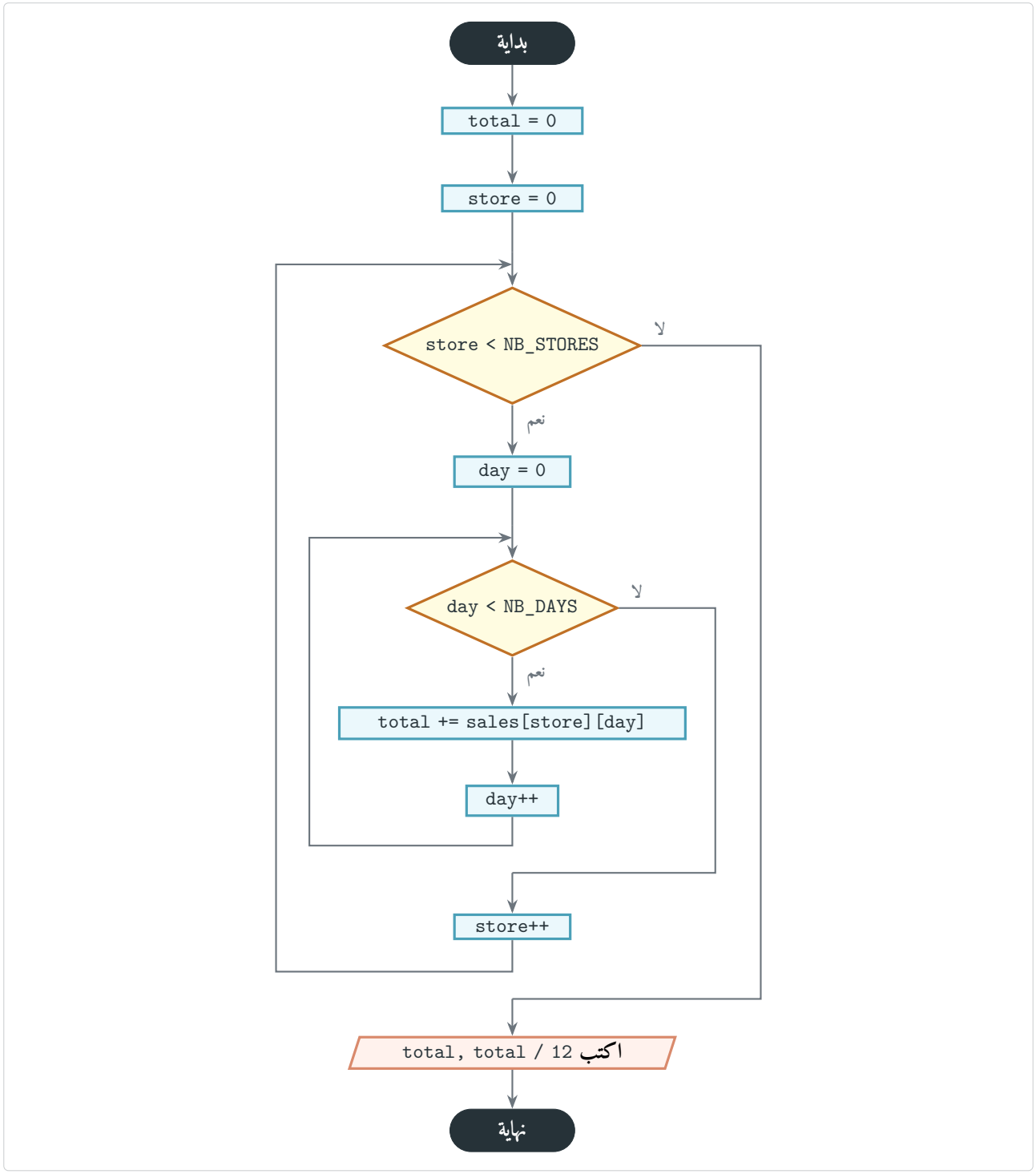
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, total = 0;
    for (store = 0; store < NB_STORES; store++)
        for (day = 0; day < NB_DAYS; day++)
            total += sales[store][day];   // كل خانة مرة واحدة
    printf("Total = %d\n", total);
    printf("Average per cell = %.2f\n", (double)total / (NB_STORES * NB_DAYS));
    return 0;
}
```

نتيجة التنفيذ

```
Total = 1690
Average per cell = 140.83
```

المخطط الإنسيابي (Organigramme)



تمرين 5.3 مجموع كل متجر

تجارة إلكترونية سهل

اكتب برنامجًا يحسب مجموع مبيعات **كل متجر** (مجموع كل سطر) ويخزنها في جدول `storeTotal`.

خطوات الحل

1) لكل سطر: نهيئ `storeTotal[store] = 0` ثم نجمع أعمدته.

البرنامج بلغة C

```c
#include <stdio.h>

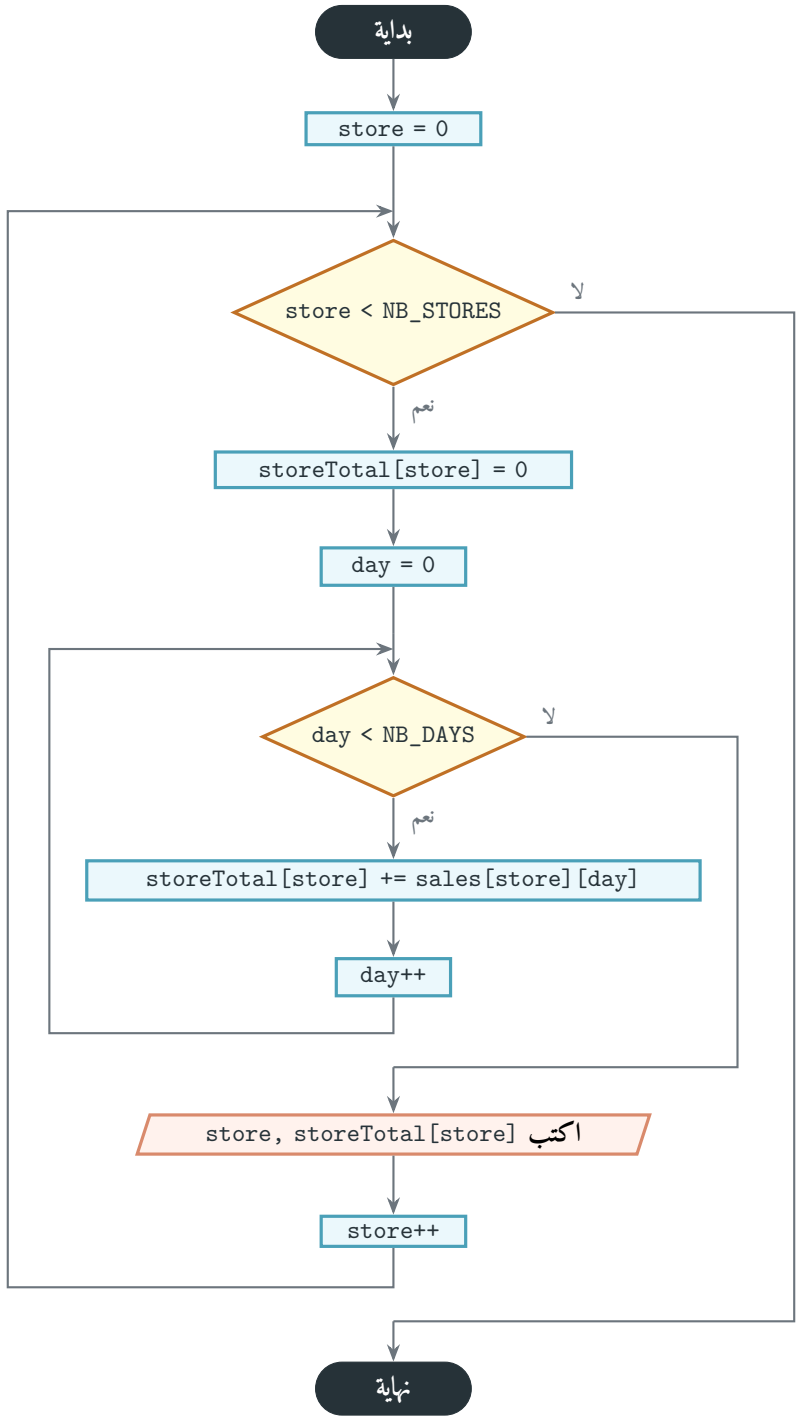
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int storeTotal[NB_STORES];
    int store, day;
    for (store = 0; store < NB_STORES; store++) {
        storeTotal[store] = 0;            // مجمّع خاص بكل سطر
        for (day = 0; day < NB_DAYS; day++)
            storeTotal[store] += sales[store][day];
        printf("Store %d: %d\n", store, storeTotal[store]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Store 0: 535
Store 1: 305
Store 2: 850
```

المخطط الإنسيابي (Organigramme)



تمرين 5.4 مجموع كل يوم

تجارة إلكترونية ⭐ سهل

اكتب برنامجًا يحسب مجموع مبيعات **كل يوم** في كل المتاجر (مجموع كل عمود). لاحظ أن الحلقة الخارجية تصبح على الأعمدة.

خطوات الحل

1) الحلقة الخارجية على `day`، والداخلية على `store`.
2) `dayTotal[day] += sales[store][day]`.

البرنامج بلغة C

```c
#include <stdio.h>

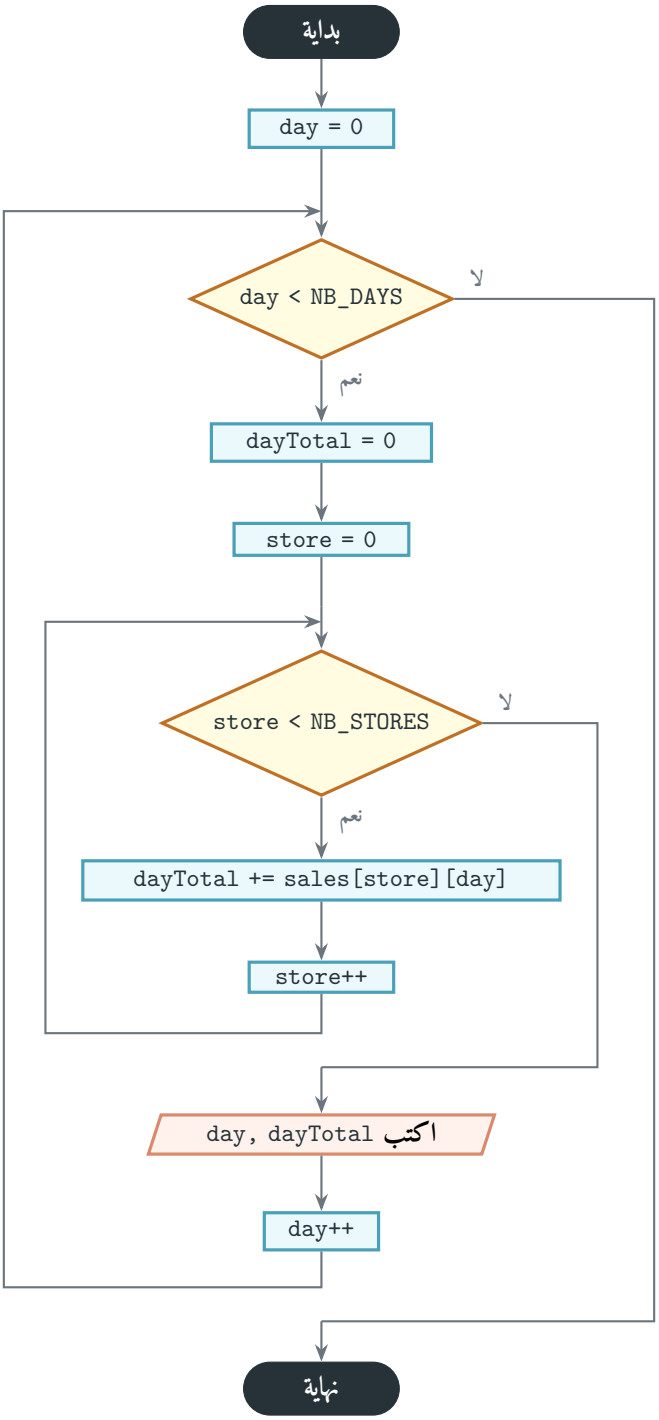
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, dayTotal;
    for (day = 0; day < NB_DAYS; day++) {          // الأعمدة في الخارج
        dayTotal = 0;
        for (store = 0; store < NB_STORES; store++)
            dayTotal += sales[store][day];
        printf("Day %d: %d\n", day + 1, dayTotal);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Day 1: 400
Day 2: 365
Day 3: 405
Day 4: 520
```

المخطط الإنسيابي (Organigramme)



تمرين 5.5 أفضل خانة في المصفوفة

علم البيانات ★ سهل

اكتب برنامجًا يجد أكبر قيمة مبيعات في المصفوفة مع المتجر واليوم الموافقين.

خطوات الحل

1) نحفظ موضع الأكبر `bestStore` و`bestDay` (نبدأ بالخانة `[0][0]`).
2) نقارن كل خانة بـ `sales[bestStore][bestDay]`.

البرنامج بلغة C

```c
#include <stdio.h>

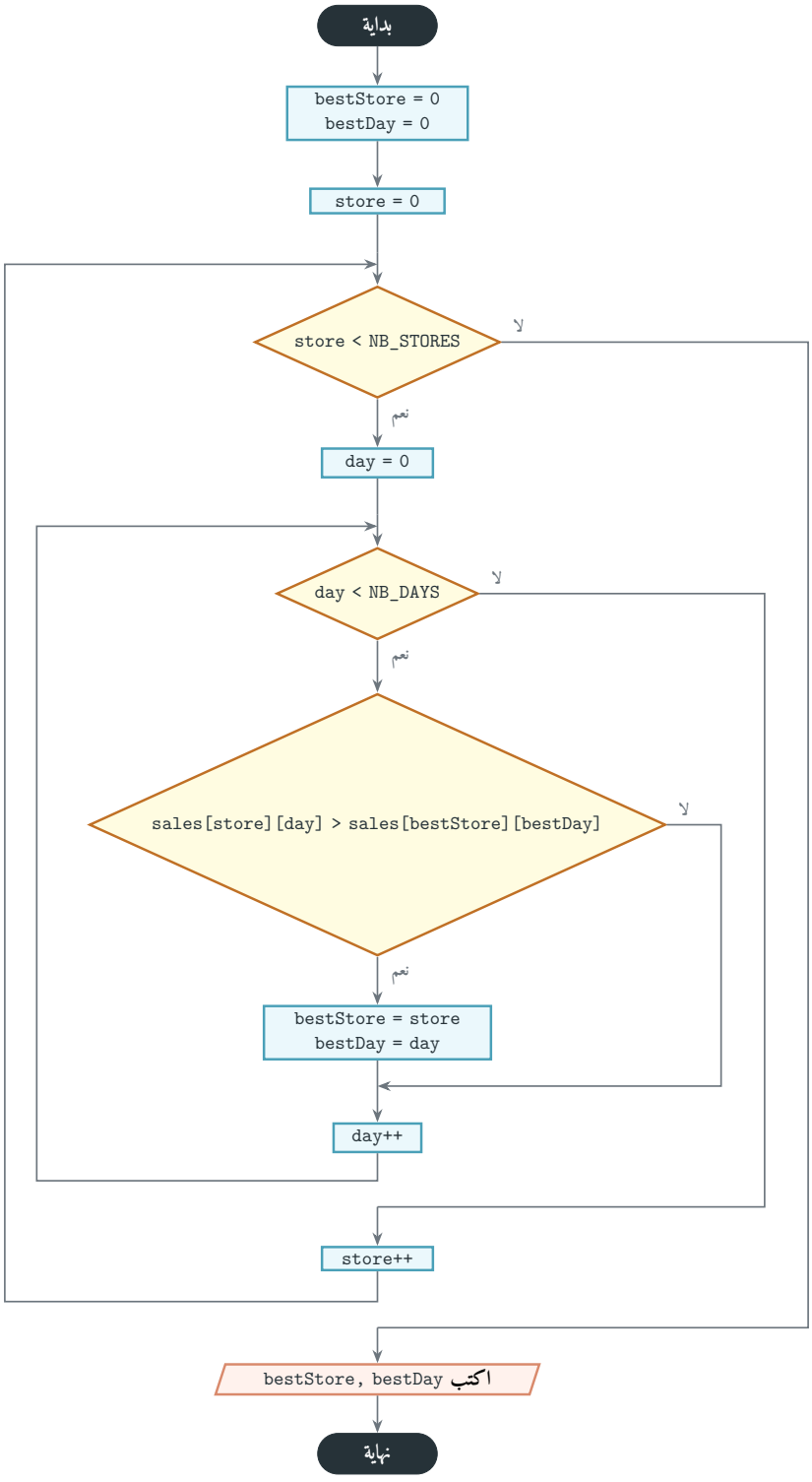
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, bestStore = 0, bestDay = 0;
    for (store = 0; store < NB_STORES; store++)
        for (day = 0; day < NB_DAYS; day++)
            if (sales[store][day] > sales[bestStore][bestDay]) {
                bestStore = store;        // نحفظ الإحداثيتين
                bestDay = day;
            }
    printf("Best: store %d, day %d -> %d\n", bestStore, bestDay + 1,
           sales[bestStore][bestDay]);
    return 0;
}
```

نتيجة التنفيذ

```
Best: store 2, day 4 -> 250
```

المخطط الإنسيابي (Organigramme)



تمرين 5.6 الرفوف الفارغة في المستودع

تجارة إلكترونية | سهل

مصفوفة `stock[4][5]` تمثل مخزون المنتجات (الأعمدة) في 4 مستودعات (الأسطر). اكتب برنامجًا يعرض إحداثيات كل خانة مخزونها 0 ويعد عددها.

خطوات الحل

1) حلقتان متداخلتان.
2) إذا كان `stock[depot][product] == 0` نعرض الإحداثيتين ونزيد العداد.

البرنامج بلغة C

```c
#include <stdio.h>

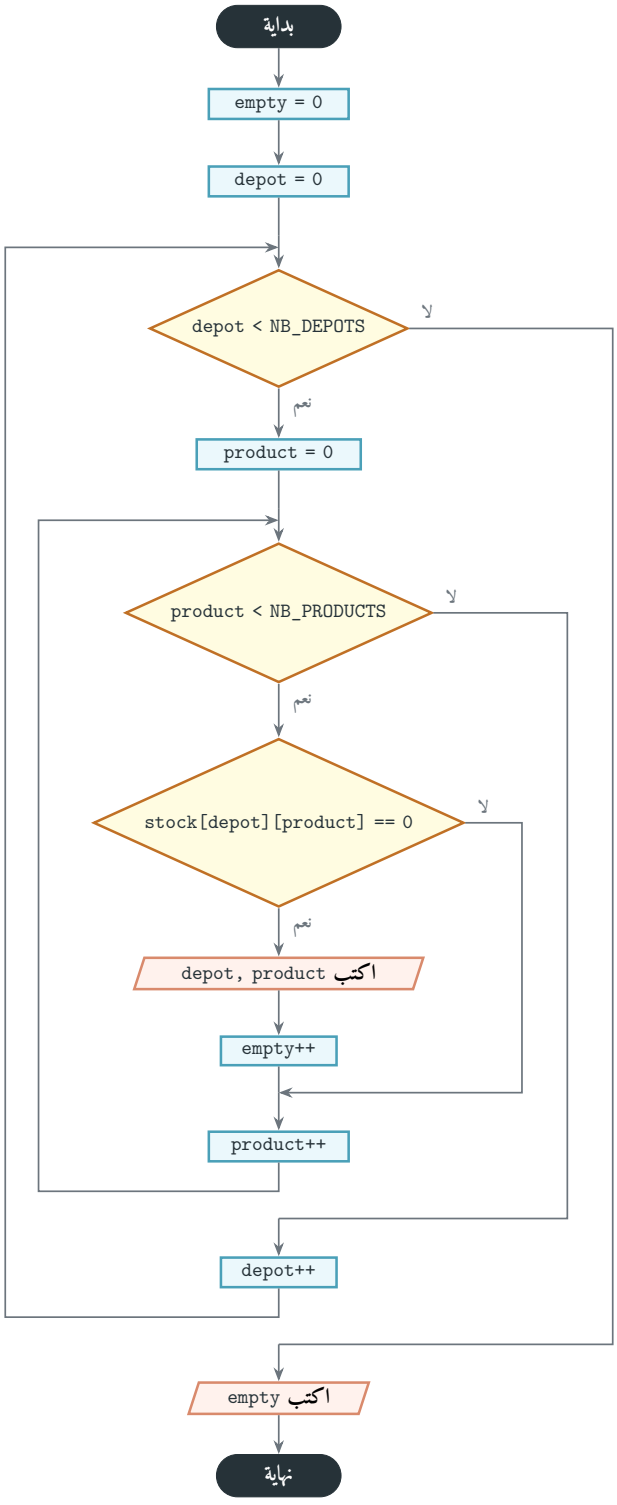
#define NB_DEPOTS 4
#define NB_PRODUCTS 5

int main(void) {
    int stock[NB_DEPOTS][NB_PRODUCTS] = {{5, 0, 12, 3, 8},
                                         {0, 4,  0, 9, 1},
                                         {7, 7,  2, 0, 6},
                                         {3, 0,  5, 5, 0}};
    int depot, product, empty = 0;
    for (depot = 0; depot < NB_DEPOTS; depot++)
        for (product = 0; product < NB_PRODUCTS; product++)
            if (stock[depot][product] == 0) {
                printf("Depot %d, product %d is empty\n", depot, product);
                empty++;
            }
    printf("%d empty slots\n", empty);
    return 0;
}
```

نتيجة التنفيذ

```
Depot 0, product 1 is empty
Depot 1, product 0 is empty
Depot 1, product 2 is empty
Depot 2, product 3 is empty
Depot 3, product 1 is empty
Depot 3, product 4 is empty
6 empty slots
```

المخطط الإنسيابي (Organigramme)



تمرين 5.7 تحديل كل الأسعار في المصفوفة

تجارة إلكترونية سهل

مصفوفة `prices[3][3]` تحتوي أسعار 3 منتجات (أسطر) بثلاثة أحجام (أعمدة). اكتب برنامجًا يطبق تخفيضًا `percent`% على كل الأسعار في مكانها ثم يعرضها.

خطوات الحل

1) نقرأ النسبة.
2) `prices[product][size] *= (1 - percent / 100)` لكل خانة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double prices[3][3] = {{1200, 1500, 1800},    // صغير، متوسط، كبير
                           { 450,  600,  750},
                           {3000, 3600, 4200}};
    double percent;
    int product, size;
    scanf("%lf", &percent);
    for (product = 0; product < 3; product++) {
        for (size = 0; size < 3; size++) {
            prices[product][size] *= 1 - percent / 100;    // التعديل في المكان
            printf("%8.2f", prices[product][size]);
        }
        printf("\n");
    }
    return 0;
}
```

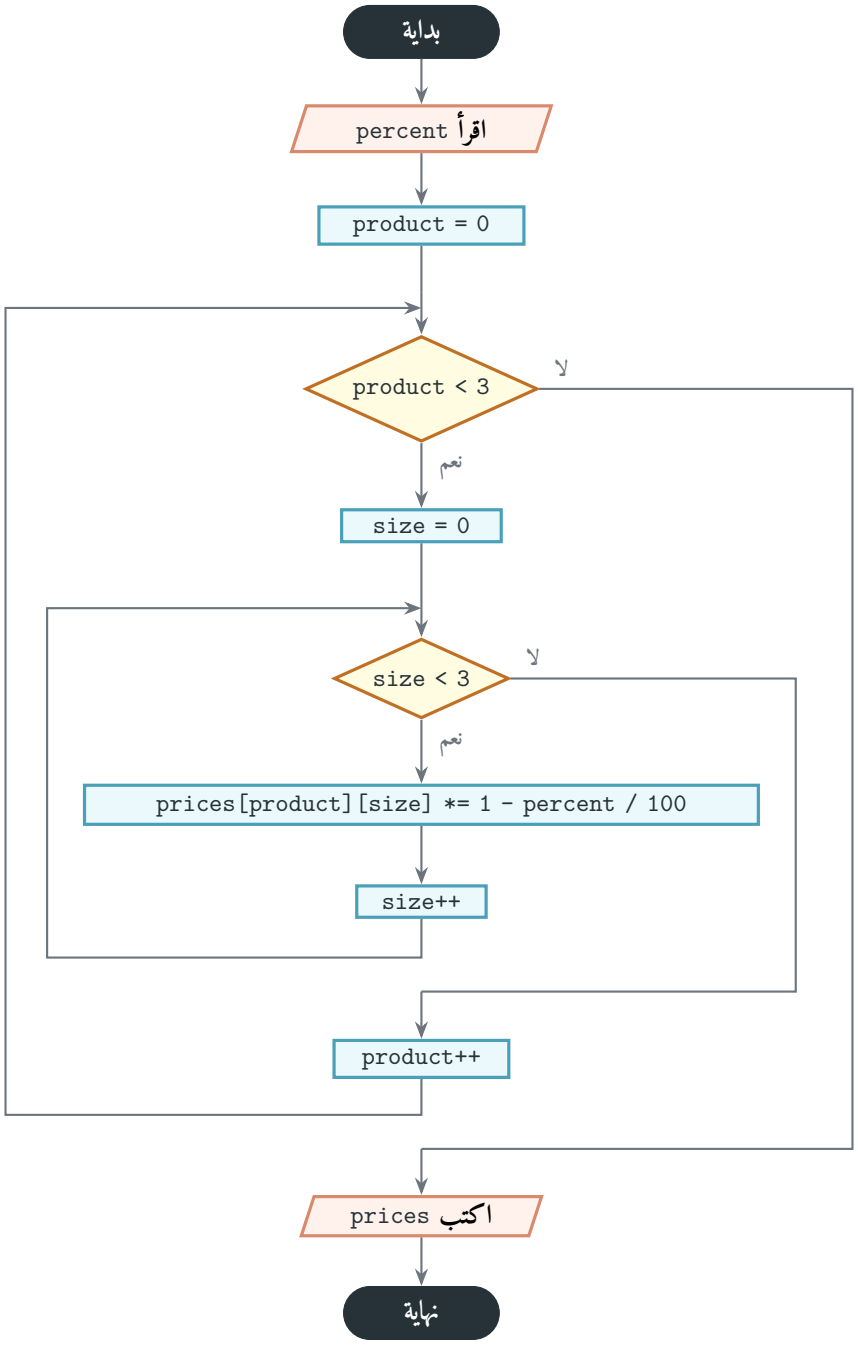
المدخلات

```
15
```

نتيجة التنفيذ

```
 1020.00 1275.00 1530.00
  382.50  510.00  637.50
2550.00 3060.00 3570.00
```

المخطط الإنسيابي (Organigramme)



تمرين 5.8 قراءة مصفوفة تقييمات

علم البيانات ⭐ سهل

اكتب برنامجًا يقرأ عدد الزبائن `nbUsers` وعدد المنتجات `nbItems` (لا يتجاوزان 10) ثم مصفوفة التقييمات، ويعرضها مع متوسط كل زبون.

خطوات الحل

1) نصرح بمصفوفة بالسعة القصوى `ratings[10][10]` ونستعمل جزءًا منها.
2) القراءة بحلقتين متداخلتين.
3) متوسط كل سطر عند العرض.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 10

int main(void) {
    int ratings[MAX][MAX];
    int nbUsers, nbItems, user, item, rowSum;
    scanf("%d %d", &nbUsers, &nbItems);
    for (user = 0; user < nbUsers; user++)
        for (item = 0; item < nbItems; item++)
            scanf("%d", &ratings[user][item]);     // عنوان الخانة
    for (user = 0; user < nbUsers; user++) {
        rowSum = 0;
        printf("User %d:", user);
        for (item = 0; item < nbItems; item++) {
            printf(" %d", ratings[user][item]);
            rowSum += ratings[user][item];
        }
        printf("  avg %.2f\n", (double)rowSum / nbItems);
    }
    return 0;
}
```

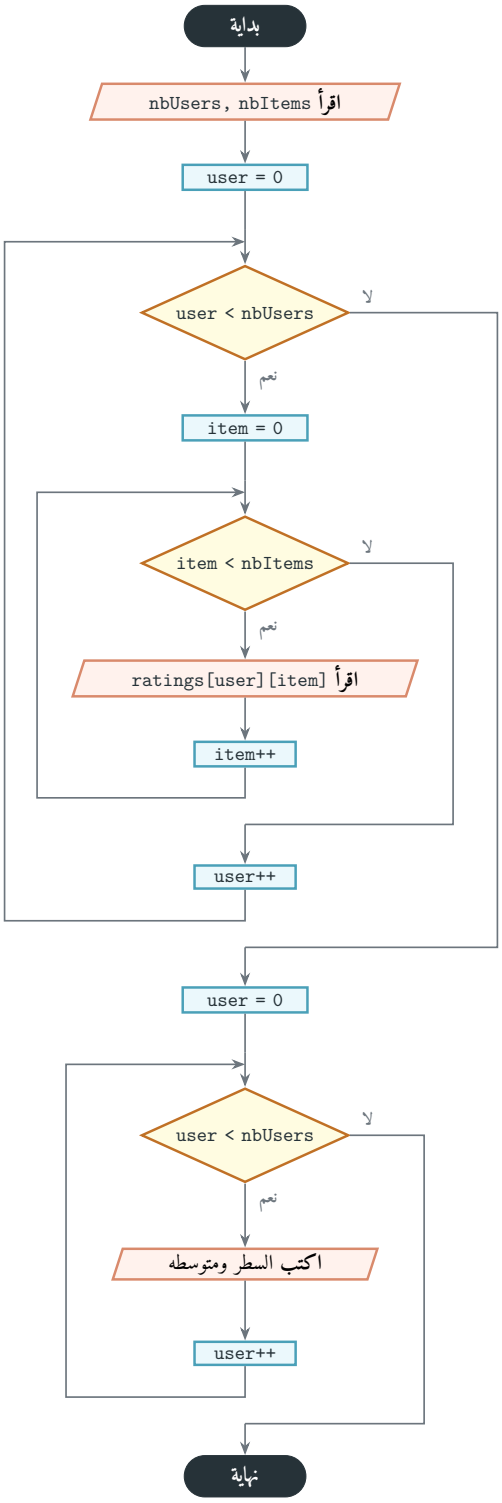
المدخلات

```
3 4
5 3 0 4
4 0 4 5
1 2 5 0
```

نتيجة التنفيذ

```
User 0: 5 3 0 4  avg 3.00
User 1: 4 0 4 5  avg 3.25
User 2: 1 2 5 0  avg 2.00
```

المخطط الإنسيابي (Organigramme)



تمرين 5.9 القطر الرئيسي لمصفوفة التحويلات

تجارة إلكترونية ★ سهل

مصفوفة مربعة `transfers[4][4]` تعطي عدد الطرود المرسلة من مستودع (سطر) إلى مستودع (عمود). القطر الرئيسي يمثل الطرود التي بقيت في نفس المستودع. اكتب برنامجًا يعرض القطر ومجموعه (الأثر) ومجموع الطرود المنقولة فعلًا.

خطوات الحل

1) عناصر القطر هي `transfers[depot][depot]`: حلقة واحدة تكفي.

2) المنقولة = المجموع الكلي − الأثر.

البرنامج بلغة C

```c
#include <stdio.h>

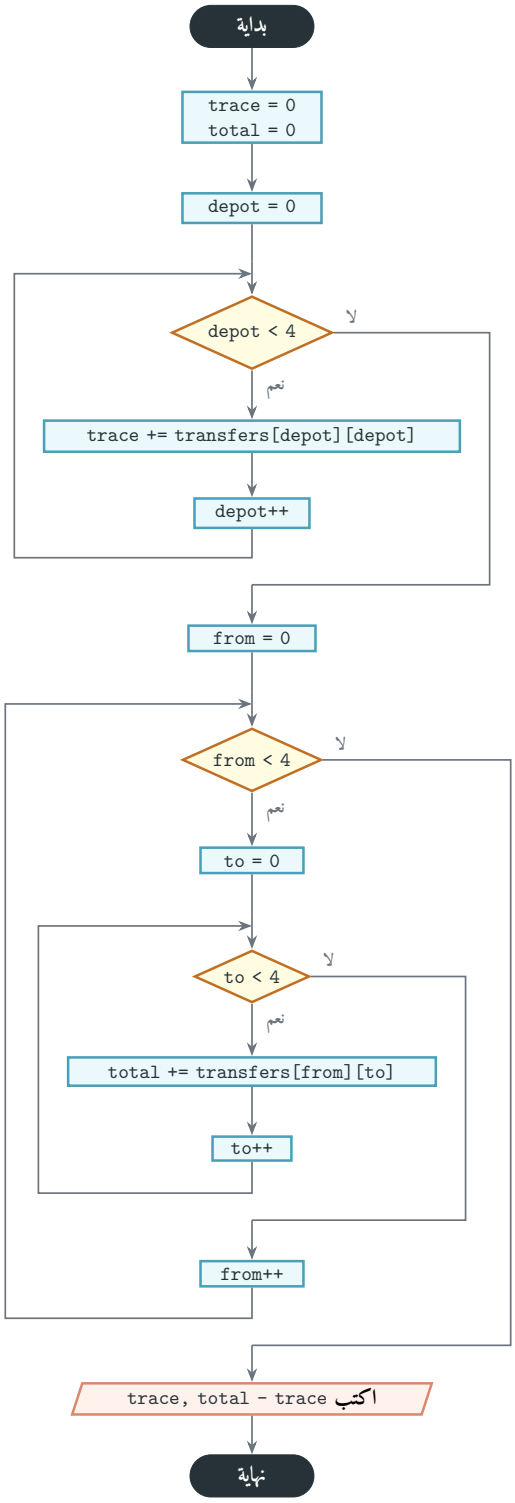
#define NB 4

int main(void) {
    int transfers[NB][NB] = {{30,  5,  2, 0},
                             { 4, 25,  6, 1},
                             { 0,  7, 40, 3},
                             { 2,  1,  5, 18}};
    int depot, from, to, trace = 0, total = 0;
    printf("Diagonal:");
    for (depot = 0; depot < NB; depot++) {
        printf(" %d", transfers[depot][depot]);   // السطر = العمود
        trace += transfers[depot][depot];
    }
    for (from = 0; from < NB; from++)
        for (to = 0; to < NB; to++)
            total += transfers[from][to];
    printf("\nKept = %d, moved = %d\n", trace, total - trace);
    return 0;
}
```

نتيجة التنفيذ

```
Diagonal: 30 25 40 18
Kept = 113, moved = 36
```

المخطط الإنسيابي (Organigramme)



تمرين 5.10 البحث عن منتج في الرفوف

تجارة إلكترونية ⭐ سهل

مصفوفة `shelves[3][4]` تحتوي رقم المنتج الموضوع في كل رف. اكتب برنامجًا يبحث عن منتج ويعرض **رقم الممر والرف** أو "غير موجود"، ويتوقف عند أول تطابق.

خطوات الحل

1) `found = 0`؛ الحلقتان تستمران ما دام `!found`.
2) عند التطابق نحفظ الموضع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int shelves[3][4] = {{101, 204, 330, 415},
                         {512, 618, 777, 801},
                         {905, 999, 120, 343}};
    int target, aisle, slot, found = 0, foundAisle = 0, foundSlot = 0;
    scanf("%d", &target);
    for (aisle = 0; aisle < 3 && !found; aisle++)          // نخرج من الحلقتين
        for (slot = 0; slot < 4 && !found; slot++)
            if (shelves[aisle][slot] == target) {
                found = 1;
                foundAisle = aisle;
                foundSlot = slot;
            }
    if (found)
        printf("Product %d: aisle %d, slot %d\n", target, foundAisle, foundSlot);
    else
        printf("Product %d not found\n", target);
    return 0;
}
```

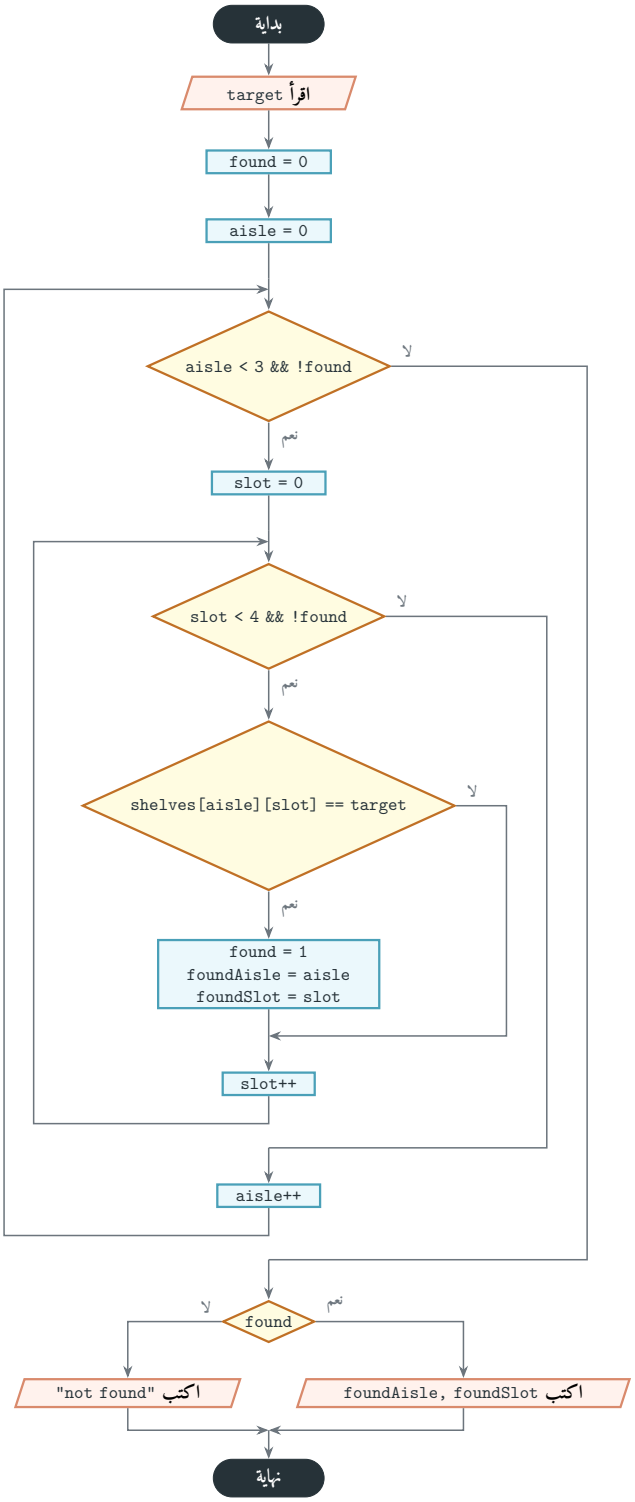
المدخلات

```
512
```

نتيجة التنفيذ

```
Product 512: aisle 1, slot 0
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 5.11 منقول المصفوفة

علم البيانات ⭐⭐ متوسط

مصفوفة `purchases[3][5]` (3 زبائن × 5 منتجات). يحتاج المحلل النظرة المعاكسة: منتجات × زبائن. اكتب برنامجًا يحسب المنقول `transposed[5][3]` ويعرضه.

خطوات الحل

1) `transposed[item][user] = purchases[user][item]`.
2) أبعاد المنقول معكوسة.

البرنامج بلغة C

```c
#include <stdio.h>

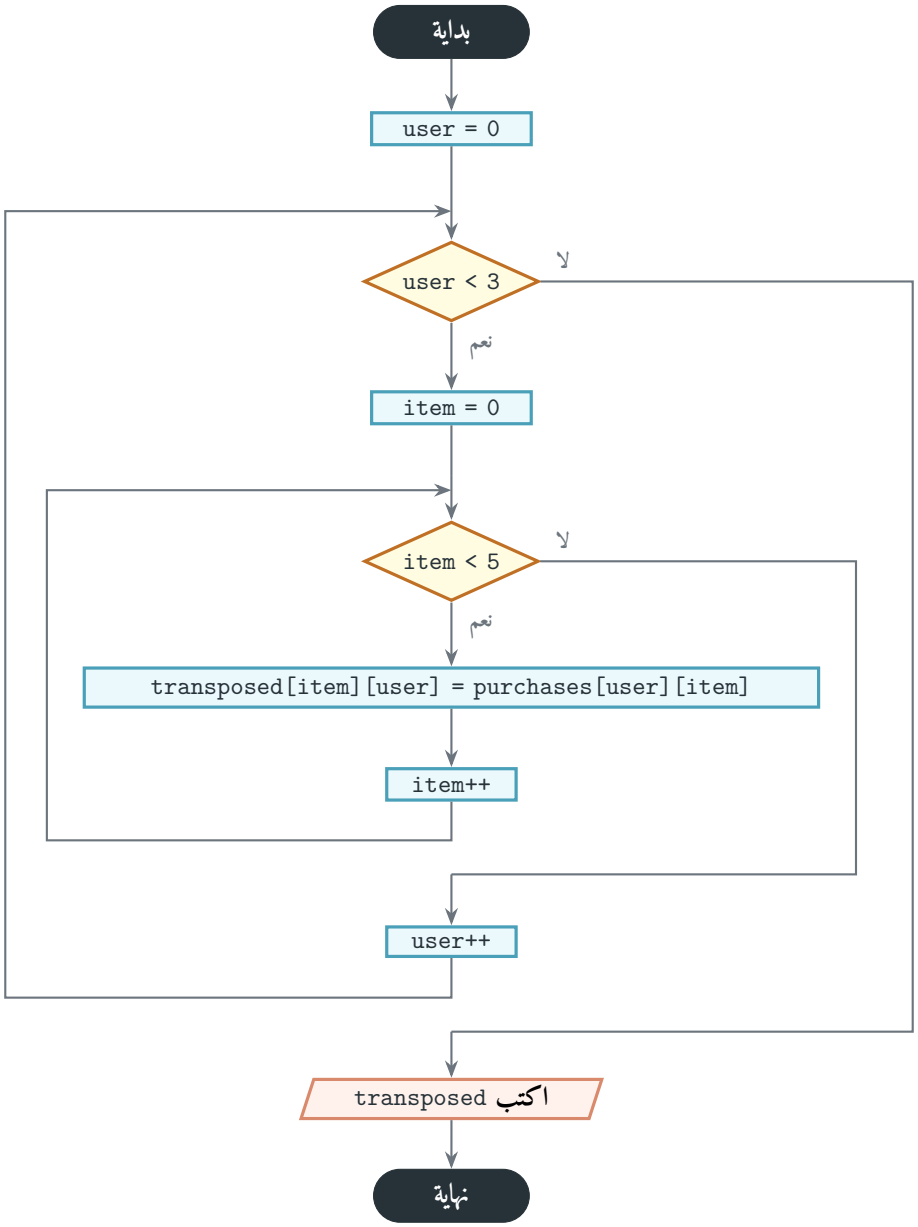
#define NB_USERS 3
#define NB_ITEMS 5

int main(void) {
    int purchases[NB_USERS][NB_ITEMS] = {{1, 0, 2, 0, 1},
                                         {0, 3, 0, 1, 0},
                                         {2, 1, 0, 0, 4}};
    int transposed[NB_ITEMS][NB_USERS];
    int user, item;
    for (user = 0; user < NB_USERS; user++)
        for (item = 0; item < NB_ITEMS; item++)
            transposed[item][user] = purchases[user][item];    // تبديل الفهرسين
    for (item = 0; item < NB_ITEMS; item++) {
        printf("Item %d:", item);
        for (user = 0; user < NB_USERS; user++)
            printf(" %d", transposed[item][user]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 1 0 2
Item 1: 0 3 1
Item 2: 2 0 0
Item 3: 0 1 0
Item 4: 1 0 4
```

المخطط الإنسيابي (Organigramme)



تمرين 5.12 جمع مبيعات شهرين

تجارة إلكترونية ★★ متوسط

لدينا مصفوفتا مبيعات (منتجات × متاجر) لشهري جانفي وفيفري. اكتب برنامجًا يحسب **مجموعهما** خانة بخانة، ثم **نسبة النمو** لكل خانة من الشهر الأول إلى الثاني.

خطوات الحل

1) `total[row][col] = jan[row][col] + feb[row][col]`.
2) `growth = (feb - jan) / jan * 100` إذا كان `jan > 0`.

البرنامج بلغة C

```c
#include <stdio.h>

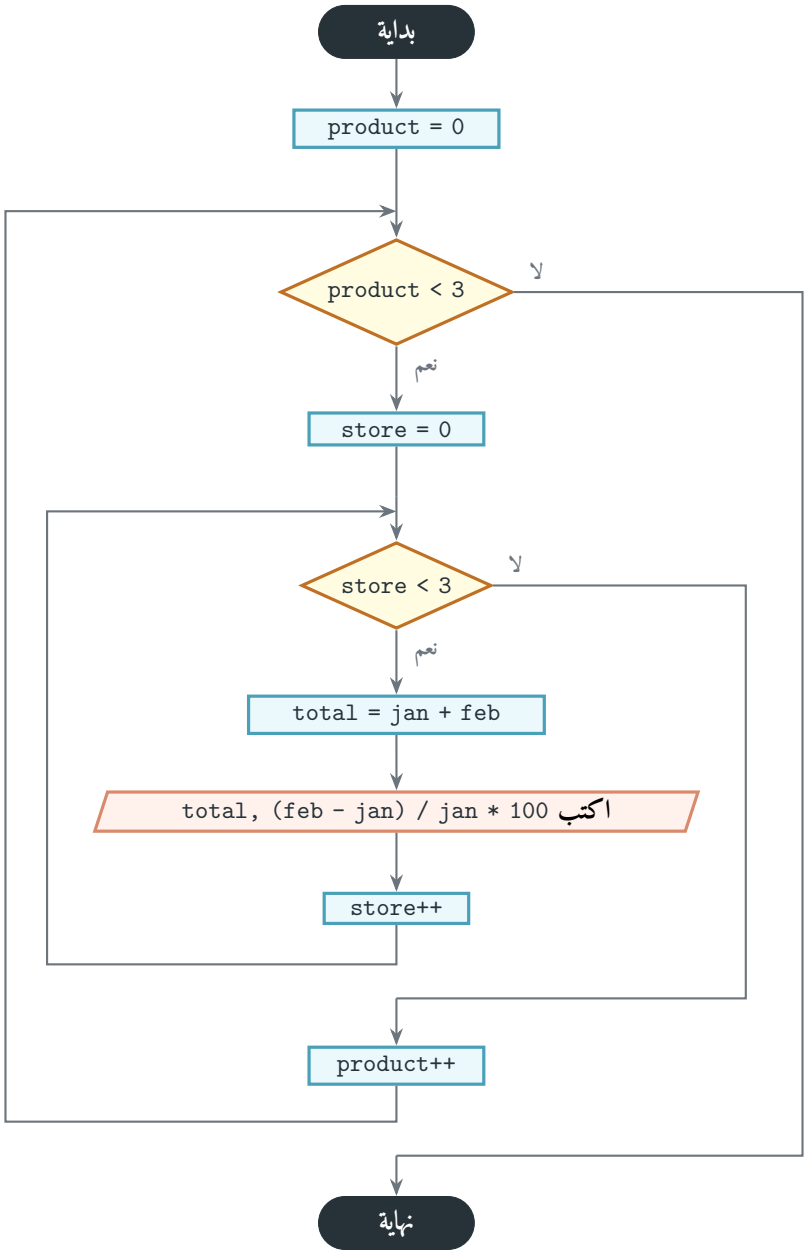
#define NB 3

int main(void) {
    int jan[NB][NB] = {{120, 80, 60}, {40, 55, 70}, {300, 210, 150}};
    int feb[NB][NB] = {{132, 70, 66}, {52, 55, 63}, {330, 240, 150}};
    int total[NB][NB], product, store;
    for (product = 0; product < NB; product++) {
        for (store = 0; store < NB; store++) {
            total[product][store] = jan[product][store] + feb[product][store];
            printf("%4d (%+5.1f%%)", total[product][store],
                   (feb[product][store] - jan[product][store]) * 100.0 / jan[product][store]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
252 (+10.0%) 150 (-12.5%) 126 (+10.0%)
 92 (+30.0%) 110 ( +0.0%) 133 (-10.0%)
630 (+10.0%) 450 (+14.3%) 300 ( +0.0%)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.13 جداء المصفوفات: كلفة الطلبيات عند الموردين

تجارة إلكترونية ★★ متوسط

quantities[2][3]: كمية كل منتج (3 منتجات) في كل طلبية (2 طلبيات). unitPrice[3][2]: سعر كل منتج عند كل مورد (2 موردين). اكتب برنامجًا يحسب **جداء المصفوفتين** cost[2][2]: كلفة كل طلبية عند كل مورد، ويحدد المورد الأرخص لكل طلبية.

خطوات الحل

1) $cost[order][supplier] = \Sigma\ quantities[order][prod] * unitPrice[prod][supplier]$.
2) ثلاث حلقات متداخلة: السطر، العمود، ثم المجموع.
3) الأرخص: مقارنة عمودي كل سطر.

البرنامج بلغة C

```c
#include <stdio.h>

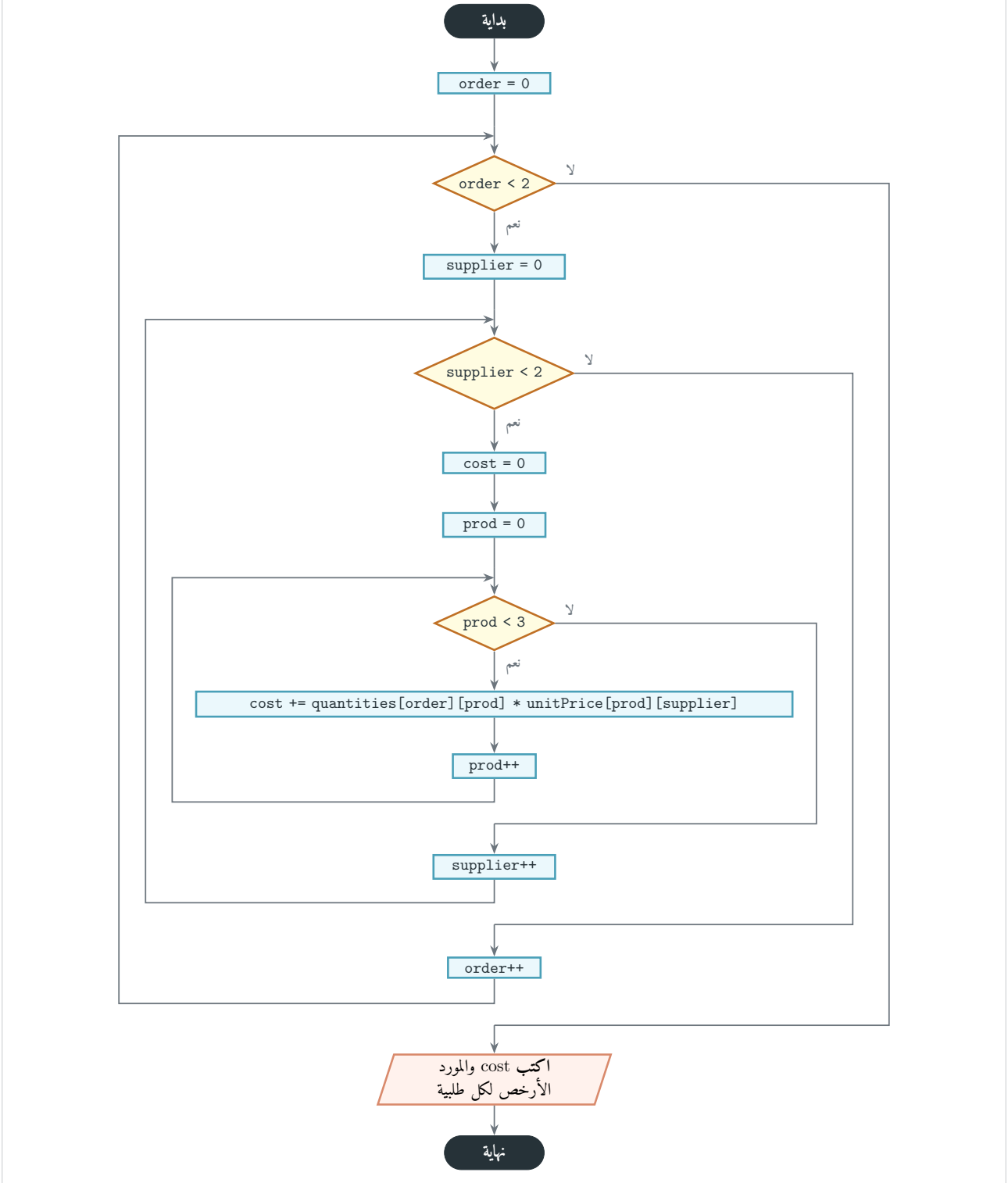
#define NB_ORDERS 2
#define NB_PRODUCTS 3
#define NB_SUPPLIERS 2

int main(void) {
    int quantities[NB_ORDERS][NB_PRODUCTS] = {{10, 5, 2}, {0, 20, 8}};
    int unitPrice[NB_PRODUCTS][NB_SUPPLIERS] = {{100, 95}, {40, 48}, {250, 230}};
    int cost[NB_ORDERS][NB_SUPPLIERS];
    int order, supplier, prod;
    for (order = 0; order < NB_ORDERS; order++) {
        for (supplier = 0; supplier < NB_SUPPLIERS; supplier++) {
            cost[order][supplier] = 0;
            for (prod = 0; prod < NB_PRODUCTS; prod++)   // سطر × عمود
                cost[order][supplier] += quantities[order][prod] * unitPrice[prod][supplier];
        }
        printf("Order %d: supplier A = %d, supplier B = %d -> choose %c\n", order,
               cost[order][0], cost[order][1], cost[order][0] <= cost[order][1] ? 'A' : 'B');
    }
    return 0;
}
```

نتيجة التنفيذ

```
Order 0: supplier A = 1700, supplier B = 1650 -> choose B
Order 1: supplier A = 2800, supplier B = 2800 -> choose A
```

المخطط الإنسيابي (Organigramme)



تمرين 5.14 فاتورة كل زبونٍ بجداء مصفوفة في شعاع

تجارة إلكترونية
متوسط

مصفوفة `cart[4][5]`: كمية كل منتج في سلة كل زبون. جدول `price[5]`: أسعار المنتجات. اكتب برنامجًا يحسب **جداء المصفوفة في الشعاع**: فاتورة كل زبون، والزبون الأكبر إنفاقًا.

خطوات الحل

1) `bill[customer] = Σ cart[customer][item] * price[item]`.
2) نحتفظ بأكبر فاتورة.

البرنامج بلغة C

```c
#include <stdio.h>

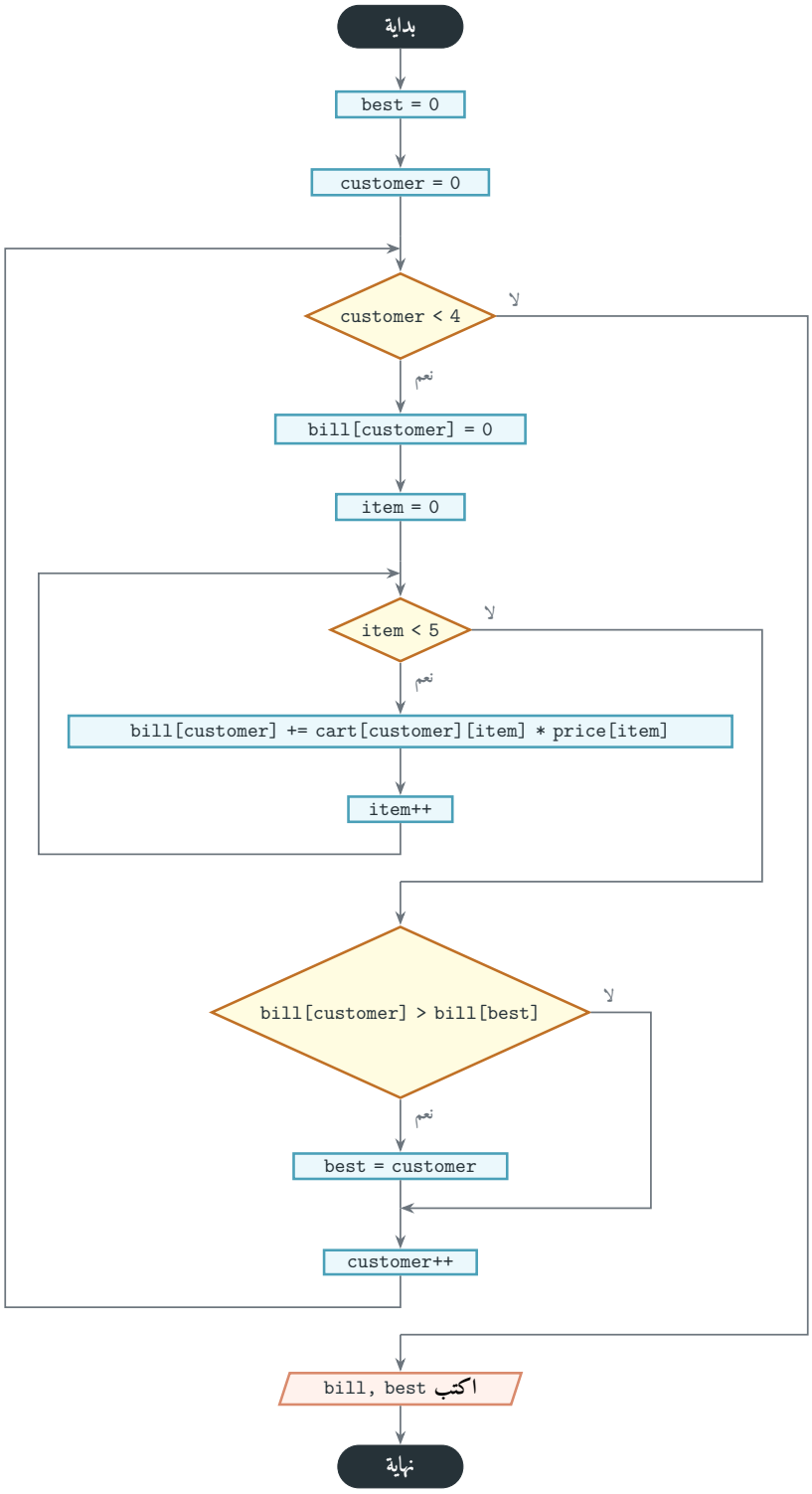
#define NB_CUST 4
#define NB_ITEMS 5

int main(void) {
    int cart[NB_CUST][NB_ITEMS] = {{1, 0, 2, 0, 1},
                                   {0, 3, 0, 1, 0},
                                   {2, 1, 0, 0, 4},
                                   {0, 0, 1, 2, 2}};
    double price[NB_ITEMS] = {1500, 350, 2200, 999, 120};
    double bill[NB_CUST];
    int customer, item, best = 0;
    for (customer = 0; customer < NB_CUST; customer++) {
        bill[customer] = 0;
        for (item = 0; item < NB_ITEMS; item++)
            bill[customer] += cart[customer][item] * price[item];  // كمية × سعر
        printf("Customer %d: %.2f DA\n", customer, bill[customer]);
        if (bill[customer] > bill[best])
            best = customer;
    }
    printf("Top spender: customer %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Customer 0: 6020.00 DA
Customer 1: 2049.00 DA
Customer 2: 3830.00 DA
Customer 3: 4438.00 DA
Top spender: customer 0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.15 أفضل متجر في كل يوم

علم البيانات ★★ متوسط

اكتب برنامجًا يحدد لكل يوم (عمود) **المتجر الأفضل** (السطر ذو الأكبر قيمة)، ثم يعد عدد الأيام التي فاز بها كل متجر.

خطوات الحل

1) لكل عمود: بحث عن فهرس السطر الأكبر `winner`.
2) `wins[winner]++`.

البرنامج بلغة C

```c
#include <stdio.h>

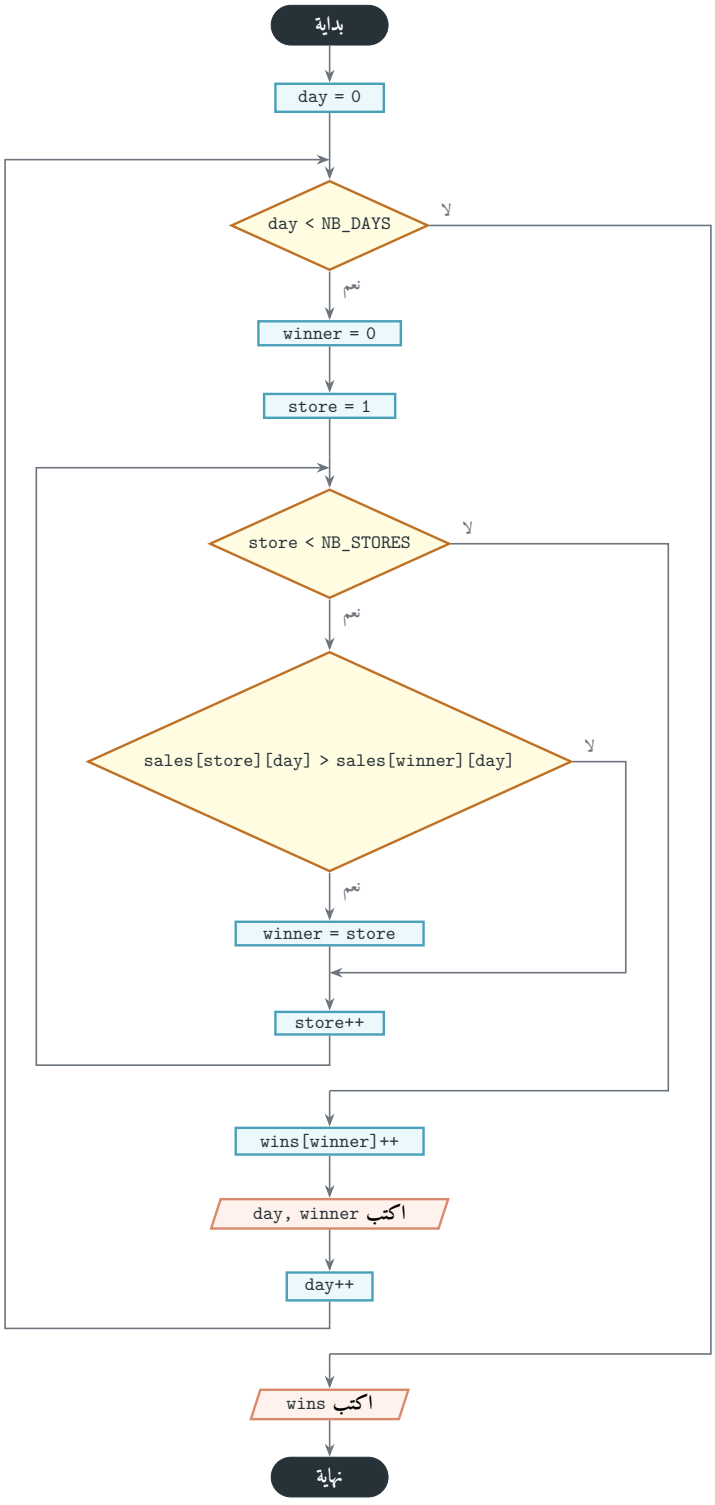
#define NB_STORES 3
#define NB_DAYS 6

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180, 90, 210},
                                     {130, 60, 175,  90, 95, 100},
                                     {100, 99, 150, 170, 80, 240}};
    int wins[NB_STORES] = {0};
    int day, store, winner;
    for (day = 0; day < NB_DAYS; day++) {
        winner = 0;                          // نبحث في العمود day
        for (store = 1; store < NB_STORES; store++)
            if (sales[store][day] > sales[winner][day])
                winner = store;
        wins[winner]++;
        printf("Day %d: store %d\n", day + 1, winner);
    }
    for (store = 0; store < NB_STORES; store++)
        printf("Store %d won %d day(s)\n", store, wins[store]);
    return 0;
}
```

نتيجة التنفيذ

```
Day 1: store 1
Day 2: store 2
Day 3: store 1
Day 4: store 0
Day 5: store 1
Day 6: store 2
Store 0 won 1 day(s)
Store 1 won 3 day(s)
Store 2 won 2 day(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.16 متوسط تقييم كل منتج دون الخانات الفارغة

علم البيانات ★★ متوسط

في مصفوفة التقييمات (زبائن × منتجات) القيمة **0** تعني "لم يقيّم". اكتب برنامجًا يحسب متوسط كل منتج على التقييمات الموجودة فقط، وعدد مقيّميه، ثم المنتج الأعلى تقييمًا.

خطوات الحل

1) لكل عمود: نجمع الخانات غير الصفرية ونعدها.
2) المتوسط `sum / count` إذا كان `count > 0`.

البرنامج بلغة C

```c
#include <stdio.h>

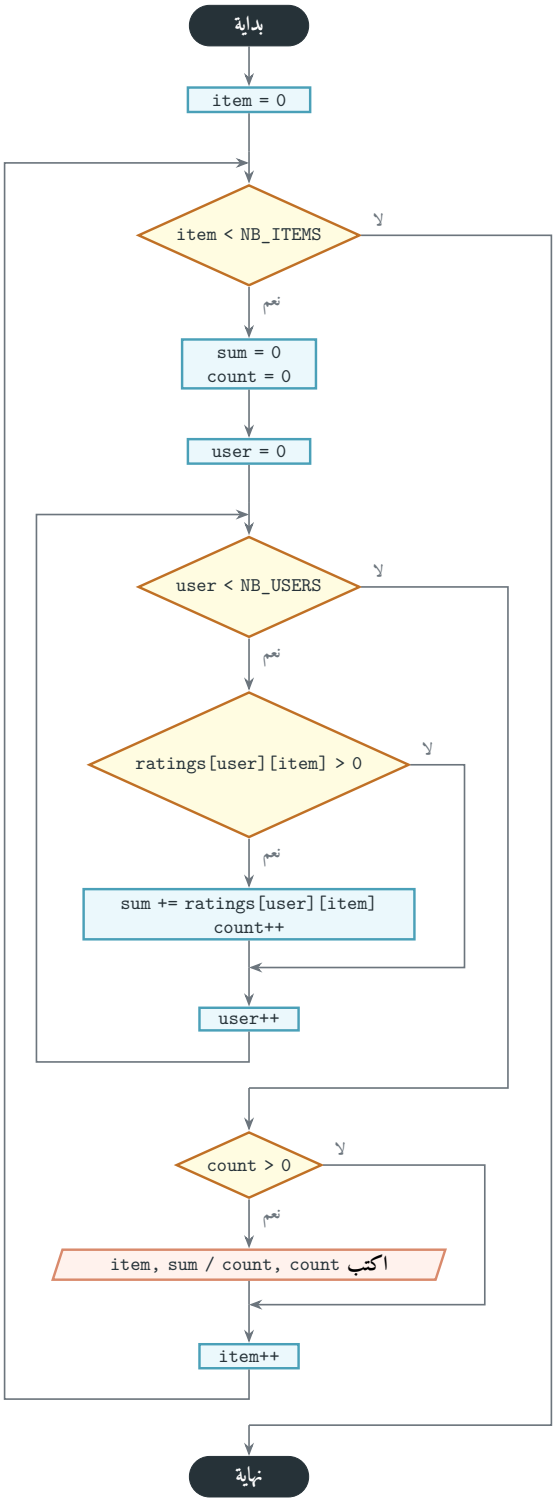
#define NB_USERS 5
#define NB_ITEMS 4

int main(void) {
    int ratings[NB_USERS][NB_ITEMS] = {{5, 3, 0, 4},
                                       {4, 0, 0, 5},
                                       {1, 2, 5, 0},
                                       {0, 3, 4, 4},
                                       {5, 0, 3, 0}};
    int user, item, sum, count, best = -1;
    double avg, bestAvg = 0;
    for (item = 0; item < NB_ITEMS; item++) {
        sum = 0;
        count = 0;
        for (user = 0; user < NB_USERS; user++)
            if (ratings[user][item] > 0) {    // نتجاهل الخانات الفارغة
                sum += ratings[user][item];
                count++;
            }
        if (count > 0) {
            avg = (double)sum / count;
            printf("Item %d: %.2f (%d ratings)\n", item, avg, count);
            if (avg > bestAvg) { bestAvg = avg; best = item; }
        }
    }
    printf("Best rated: item %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 3.75 (4 ratings)
Item 1: 2.67 (3 ratings)
Item 2: 4.00 (3 ratings)
Item 3: 4.33 (3 ratings)
Best rated: item 3
```

المخطط الإنسيابي (Organigramme)



تمرين 5.17 تطبيع أعمدة جدول الخصائص

علم البيانات ★★ متوسط

مصفوفة `features[6][3]`: لكل منتج (سطر) ثلاث خصائص (السعر، الوزن، المبيعات) بوحدات مختلفة. اكتب برنامجًا **يطبع** كل **عمود** على حدة إلى `[0, 1]` (Min-Max)، لأن نماذج التعلم الآلي حساسة لاختلاف المقاييس.

خطوات الحل

1) لكل عمود: نجد `colMin` و`colMax`.
2) ثم نحول كل خانة في العمود.

البرنامج بلغة C

```c
#include <stdio.h>

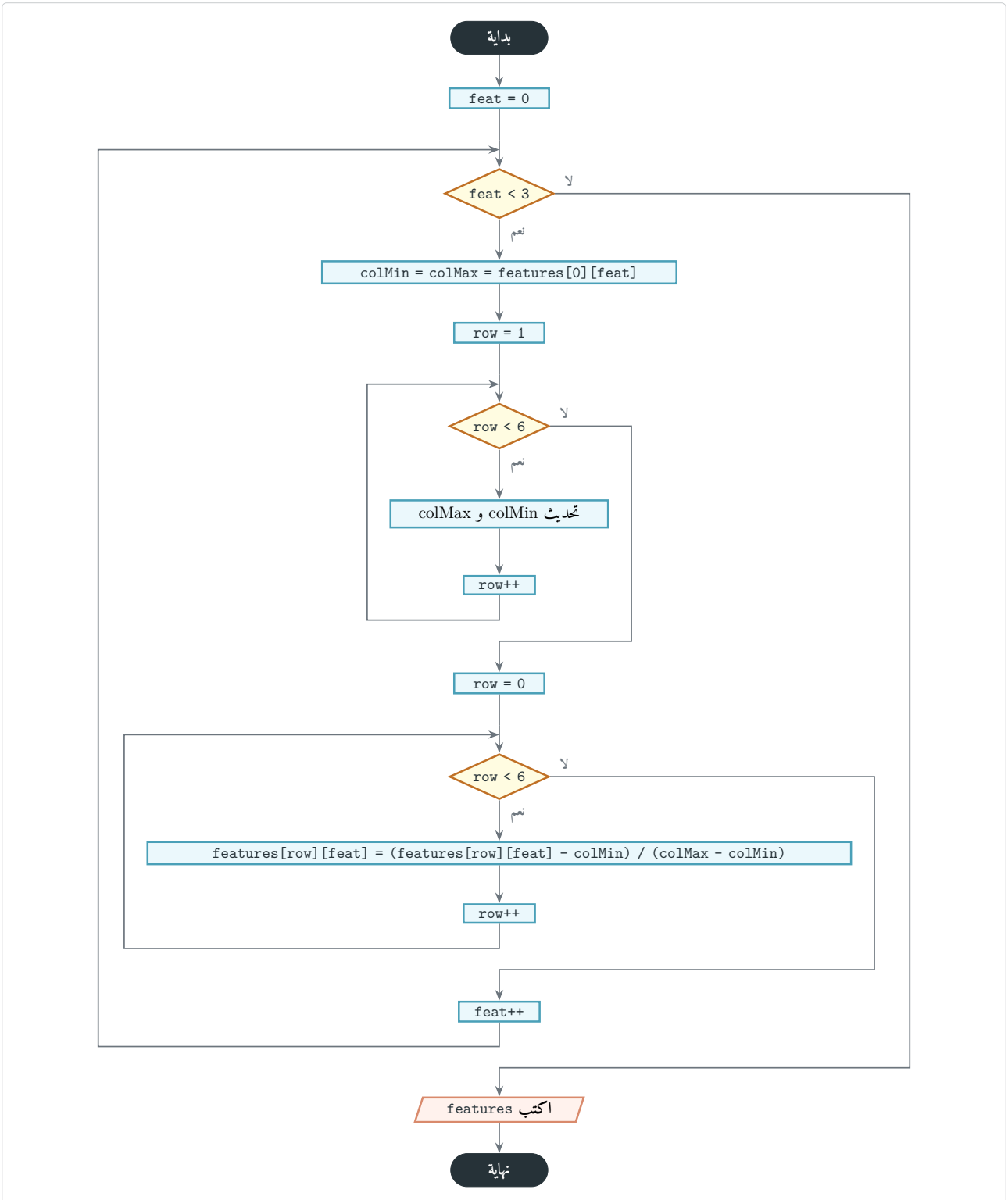
#define NB_ROWS 6
#define NB_FEAT 3

int main(void) {
    double features[NB_ROWS][NB_FEAT] = {{1200, 0.5, 340},
                                         {4500, 2.0,  80},
                                         { 300, 0.2, 900},
                                         {8000, 3.5,  45},
                                         {2500, 1.1, 210},
                                         { 950, 0.8, 500}};
    double colMin, colMax;
    int feat, row;
    for (feat = 0; feat < NB_FEAT; feat++) {          // عمود بعد عمود
        colMin = colMax = features[0][feat];
        for (row = 1; row < NB_ROWS; row++) {
            if (features[row][feat] < colMin) colMin = features[row][feat];
            if (features[row][feat] > colMax) colMax = features[row][feat];
        }
        for (row = 0; row < NB_ROWS; row++)
            features[row][feat] = (features[row][feat] - colMin) / (colMax - colMin);
    }
    printf(" price weight sales\n");
    for (row = 0; row < NB_ROWS; row++)
        printf(" %.2f  %.2f   %.2f\n", features[row][0], features[row][1], features[row][2]);
    return 0;
}
```

نتيجة التنفيذ

```
price weight sales
0.12  0.09   0.35
0.55  0.55   0.04
0.00  0.00   1.00
1.00  1.00   0.00
0.29  0.27   0.19
0.08  0.18   0.53
```

المخطط الإنسيابي (Organigramme)



تمرين 5.18 مصفوفة الإرتباط

علم البيانات ★★ متوسط

لثلاث خصائص لكل زبون (الزيارات، الدقائق، المبلغ) اكتب برنامجًا يحسب **مصفوفة الارتباط** `corr[3][3]` حيث `corr[a][b]` معامل بيرسون بين الخاصيتين، باستعمال الجذر بطريقة نيوتن.

خطوات الحل

1) نحسب متوسط كل عمود.
2) `cov[first][second] = Σ (x - mean)(y - mean)`.
3) `corr = cov[first][second] / (cov[first][first] * cov[second][second])`.

البرنامج بلغة C

```c
#include <stdio.h>

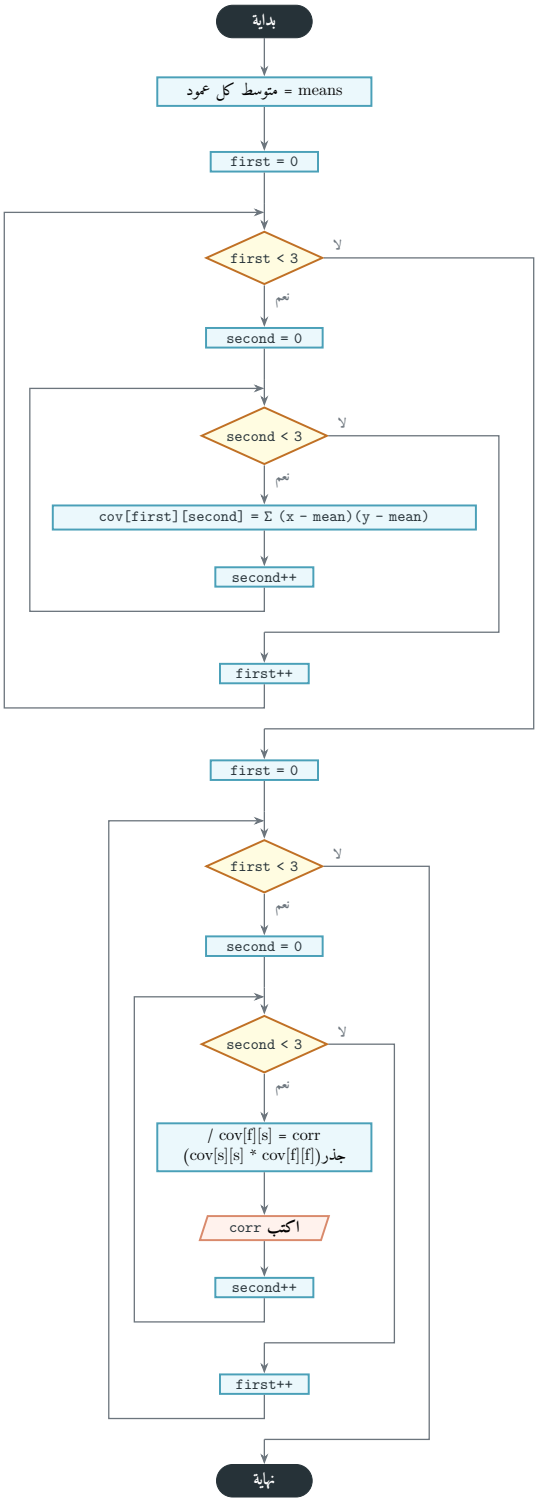
#define NB 6
#define NB_FEAT 3

int main(void) {
    double data[NB][NB_FEAT] = {{3, 12, 1500}, {8, 40, 5200}, {5, 22, 2400},
                                {1,  4,  300}, {9, 50, 6100}, {4, 25, 1900}};
    double mean[NB_FEAT] = {0}, cov[NB_FEAT][NB_FEAT], product, root;
    int row, first, second, step;
    for (row = 0; row < NB; row++)
        for (first = 0; first < NB_FEAT; first++)
            mean[first] += data[row][first] / NB;
    for (first = 0; first < NB_FEAT; first++)
        for (second = 0; second < NB_FEAT; second++) {
            cov[first][second] = 0;           // التغاير (دون القسمة على n)
            for (row = 0; row < NB; row++)
                cov[first][second] += (data[row][first] - mean[first])
                                    * (data[row][second] - mean[second]);
        }
    for (first = 0; first < NB_FEAT; first++) {
        for (second = 0; second < NB_FEAT; second++) {
            product = cov[first][first] * cov[second][second];
            root = product;                   // الجذر بطريقة نيوتن
            for (step = 0; step < 60; step++)
                root = (root + product / root) / 2;
            printf(" %6.3f", cov[first][second] / root);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
 1.000  0.981  0.990
 0.981  1.000  0.976
 0.990  0.976  1.000
```

المخطط الإنسيابي (Organigramme)



تمرين 5.19 مصفوفة المسافات بين الزبائن

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب **مصفوفة المسافات** `dist[5][5]` (مربع المسافة الإقليدية) بين 5 زبائن ممثلين بخاصيتين، مستغلًا **التناظر** `dist[a][b] = dist[b][a]` لحساب نصف الخانات فقط، ثم يجد أقرب زوج.

خطوات الحل

1) القطر أصفار.
2) لكل first < second: نحسب المسافة ونكتبها في الخانتين.
3) نحتفظ بأصغر مسافة.

البرنامج بلغة C

```c
#include <stdio.h>

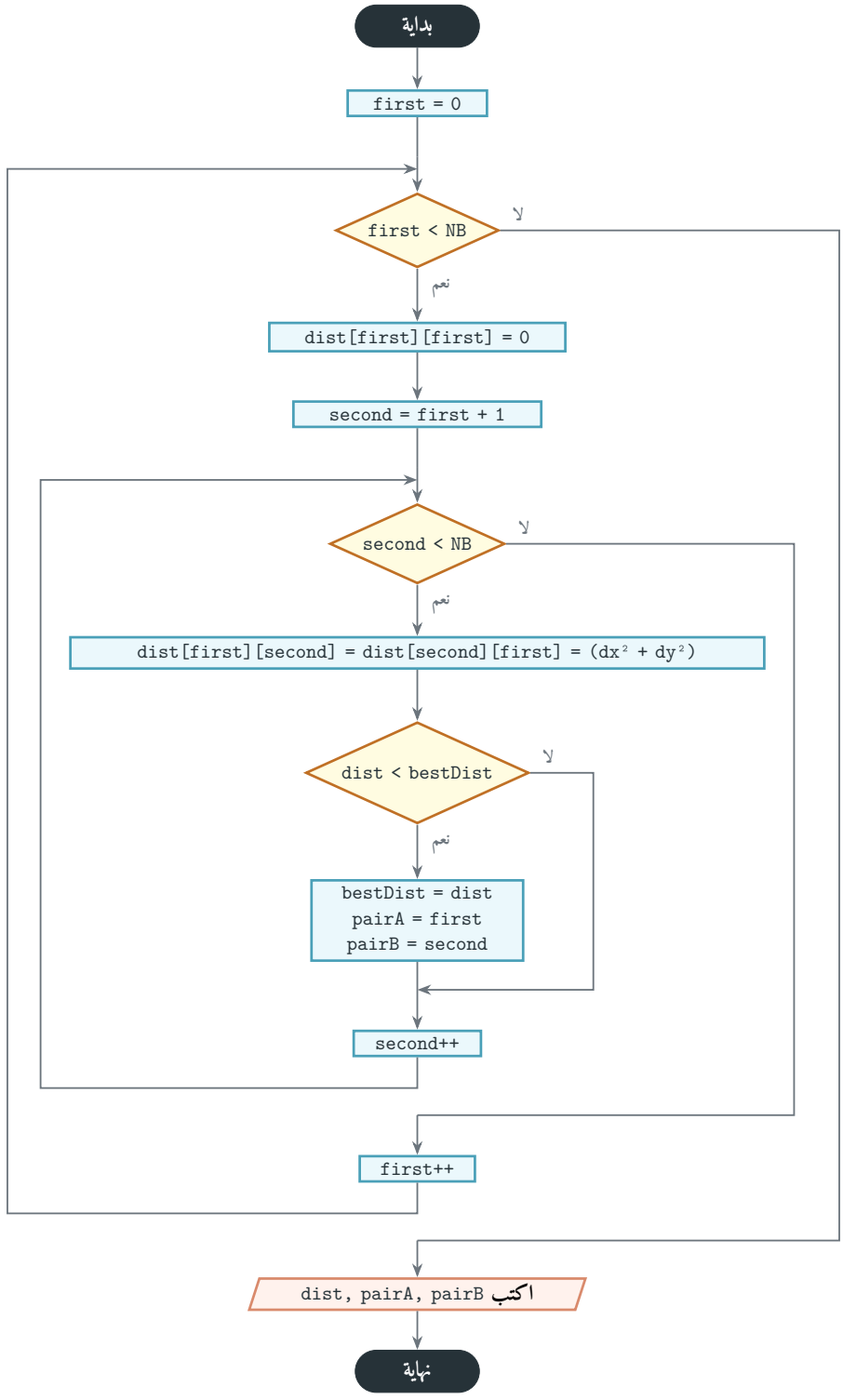
#define NB 5

int main(void) {
    double points[NB][2] = {{2, 1.5}, {8, 6}, {3, 2}, {9, 7.5}, {5, 3}};
    double dist[NB][NB], dx, dy, bestDist = -1;
    int first, second, pairA = 0, pairB = 0;
    for (first = 0; first < NB; first++) {
        dist[first][first] = 0;
        for (second = first + 1; second < NB; second++) {    // النصف العلوي فقط
            dx = points[first][0] - points[second][0];
            dy = points[first][1] - points[second][1];
            dist[first][second] = dist[second][first] = dx * dx + dy * dy;
            if (bestDist < 0 || dist[first][second] < bestDist) {
                bestDist = dist[first][second];
                pairA = first;
                pairB = second;
            }
        }
    }
    for (first = 0; first < NB; first++) {
        for (second = 0; second < NB; second++)
            printf("%7.2f", dist[first][second]);
        printf("\n");
    }
    printf("Closest customers: %d and %d\n", pairA, pairB);
    return 0;
}
```

نتيجة التنفيذ

```
   0.00  56.25   1.25  85.00  11.25
  56.25   0.00  41.00   3.25  18.00
   1.25  41.00   0.00  66.25   5.00
  85.00   3.25  66.25   0.00  36.25
  11.25  18.00   5.00  36.25   0.00
Closest customers: 0 and 2
```

المخطط الإنسيابي (Organigramme)



تمرين 5.20 مصفوفة الإلتباس والدقة

علم البيانات ★★ متوسط

نموذج يصنف التعليقات إلى 3 أصناف (0 سلبي، 1 محايد، 2 إيجابي). لدينا جدولا القيم الحقيقية والمتنبأ بها لـ 15 تعليقًا. اكتب برنامجًا يبني **مصفوفة الالتباس** `confusion[actual][predicted]` ويحسب **الدقة** (مجموع القطر / الكل).

خطوات الحل

1) نهيئ المصفوفة بأصفار.
2) لكل تعليق: `confusion[actual[index]][predicted[index]]++`.
3) الدقة = الأثر / عدد التعليقات.

البرنامج بلغة C

```c
#include <stdio.h>

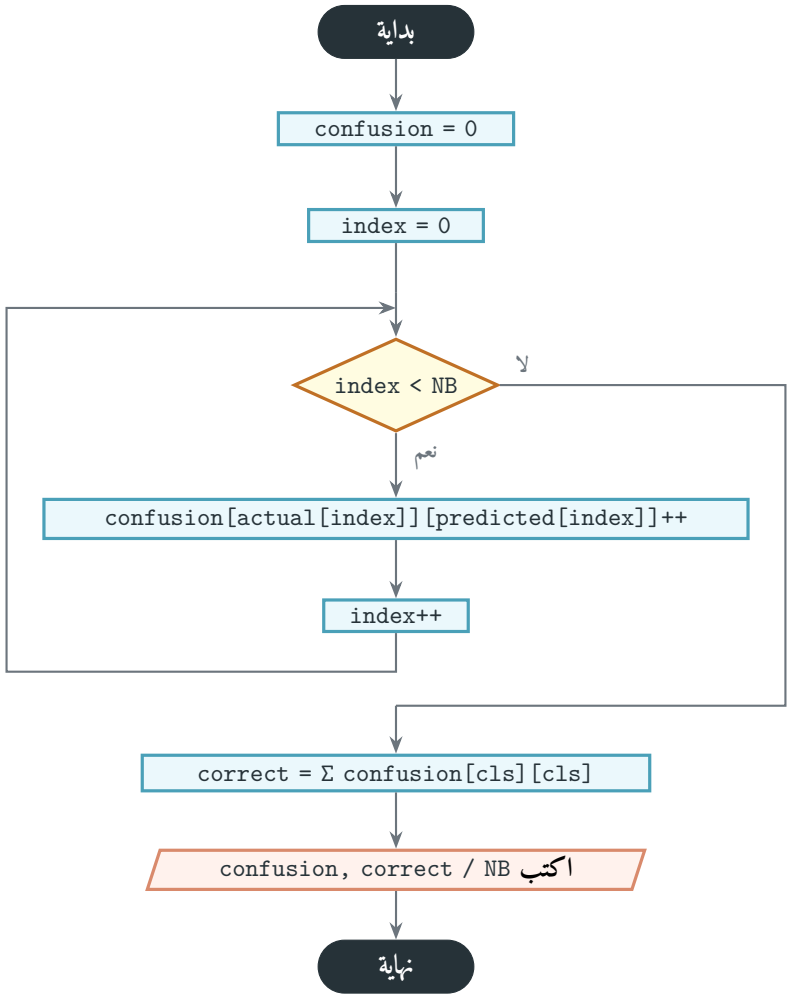
#define NB 15
#define NB_CLASSES 3

int main(void) {
    int actual[NB]    = {2, 0, 1, 2, 2, 0, 1, 1, 2, 0, 2, 1, 0, 2, 1};
    int predicted[NB] = {2, 0, 2, 2, 1, 0, 1, 0, 2, 0, 2, 1, 1, 2, 1};
    int confusion[NB_CLASSES][NB_CLASSES] = {{0}};
    int index, row, col, correct = 0;
    for (index = 0; index < NB; index++)
        confusion[actual[index]][predicted[index]]++;   // الفهرسان من الجدولين
    printf("actual\\pred  neg neu pos\n");
    for (row = 0; row < NB_CLASSES; row++) {
        printf("%11d", row);
        for (col = 0; col < NB_CLASSES; col++)
            printf(" %3d", confusion[row][col]);
        printf("\n");
        correct += confusion[row][row];   // القطر = التنبؤات الصحيحة
    }
    printf("Accuracy = %d/%d = %.2f\n", correct, NB, (double)correct / NB);
    return 0;
}
```

نتيجة التنفيذ

```
actual\pred  neg neu pos
          0   3   1   0
          1   1   3   1
          2   0   1   5
Accuracy = 11/15 = 0.73
```

المخطط الإنسيابي (Organigramme)



الفصل 6

السلاسل النصية

الفصل 7

الدوال

الفصل 8

العودية

الفصل 9

الهياكل

الفصل 10

المؤشرات والحجز الديناميكي

الفصل 11

الملفات

الفصل 12

القوائم المتصلة

الفصل 13

الأشجار الثنائية